

MobilityDB 1.4 Manual de usuario

Esteban Zimányi

COLABORADORES			
	TÍTULO : MobilityDB 1.4 Manual de usuario		
ACCIÓN	NOMBRE	FECHA	FIRMA
ESCRITO POR	Esteban Zimányi	23 de agosto de 2026	

HISTORIAL DE REVISIONES			
NÚMERO	FECHA	MODIFICACIONES	NOMBRE

Índice general

1. Introducción	1
1.1. Comité directivo del proyecto	1
1.2. Otros colaboradores del código	2
1.3. Patrocinadores	2
1.4. Licencias	2
1.5. Instalación a partir de las fuentes	2
1.5.1. Versión corta	2
1.5.2. Obtener las fuentes	3
1.5.3. Habilitación de la base de datos	4
1.5.4. Dependencias	4
1.5.5. Configuración	5
1.5.6. Construir e instalar	5
1.5.7. Pruebas	6
1.5.8. Documentación	6
1.6. Instalación a partir de binarios	7
1.6.1. Distribuciones de Linux basadas en Debian	7
1.6.2. Windows	7
1.7. Soporte	7
1.7.1. Reporte de problemas	7
1.7.2. Listas de correo	7
1.8. Migración de la versión 1.0 a la versión 1.1	8
2. Tipos de conjunto y de rango	9
2.1. Entrada y salida	10
2.2. Constructores	13
2.3. Conversión de tipos	14
2.4. Accesores	16
2.5. Transformaciones	19
2.6. Sistema de referencia espacial	22
2.7. Operaciones de conjuntos	23

2.8. Operaciones de cuadro delimitador	23
2.8.1. Operaciones topológicas	23
2.8.2. Operaciones de posición	25
2.8.3. Operaciones de división	26
2.9. Operaciones de distancia	27
2.10. Comparaciones	27
2.11. Agregaciones	28
2.12. Indexación	29
3. Tipos de cuadro delimitador	30
3.1. Entrada y salida	30
3.2. Constructores	32
3.3. Conversión de tipos	33
3.4. Accesores	34
3.5. Transformaciones	36
3.6. Sistema de referencia espacial	37
3.7. Operaciones de división	38
3.8. Operaciones de conjuntos	39
3.9. Operaciones de cuadro delimitador	40
3.9.1. Operaciones topológicas	40
3.9.2. Operaciones de posición	41
3.10. Comparaciones	42
3.11. Agregaciones	43
3.12. Indexación	43
4. Tipos temporales (Parte 1)	44
4.1. Introduction	44
4.2. Ejemplos de tipos temporales	46
4.3. Validez de los tipos temporales	48
4.4. Temporalización de operaciones	48
4.5. Notación	49
4.6. Entrada y salida	50
4.7. Constructores	54
4.8. Conversión de tipos	56
4.9. Accesores	57
4.10. Transformaciones	61

5. Tipos temporales (Parte 2)	64
5.1. Modificaciones	64
5.2. Restricciones	69
5.3. Operadores de cuadro delimitador	71
5.4. Comparaciones	72
5.4.1. Comparaciones tradicionales	72
5.4.2. Comparaciones alguna vez y siempre	72
5.4.3. Comparaciones temporales	74
5.5. Funciones de utilidad	74
6. Tipos alfanuméricos temporales	75
6.1. Notación	75
6.2. Conversión de tipos	75
6.3. Accessores	76
6.4. Transformaciones	77
6.5. Restrictions	77
6.6. Operaciones booleanas	79
6.7. Operaciones matemáticas	79
6.8. Operaciones de texto	82
7. Tipos temporales geométricos (Parte 1)	83
7.1. Tipos espaciotemporales	83
7.2. Funciones sobre Geometrías	83
7.3. Notación	86
7.4. Entrada y salida	86
7.5. Conversión de tipos	88
7.6. Accessores	90
7.7. Transformaciones	93
8. Tipos temporales geométricos (Parte 2)	96
8.1. Restricciones	96
8.2. Sistema de referencia espacial	97
8.3. Operaciones de cuadro delimitador	98
8.4. Operaciones de distancia	98
8.5. Relaciones espaciales	102
8.5.1. Relaciones alguna vez o siempre	103
8.5.2. Relaciones espaciotemporales	104

9. Tipos temporales: Operaciones de análisis	106
9.1. Simplificación	106
9.2. Reducción	107
9.3. Similaridad	110
9.4. Filtro de Kalman extendido	112
9.5. Operaciones de división	112
9.6. Mosaicos multidimensionales	116
9.6.1. Operaciones de intervalos	117
9.6.2. Operaciones de mosaicos	118
9.6.3. Operaciones de cuadro delimitador	120
9.6.4. Operaciones de división	122
10. Tipos temporales: Agregación e indexación	125
10.1. Agregación	125
10.2. Indexación	127
10.3. Estadísticas y selectividad	128
10.3.1. Colecta de estadísticas	128
10.3.2. Estimación de la selectividad	130
11. Puntos de red temporales	131
11.1. Tipos de red estáticos	131
11.1.1. Entrada y salida	133
11.1.2. Constructores	134
11.1.3. Conversión de tipos	134
11.1.4. Accesorios	135
11.1.5. Transformaciones	135
11.1.6. Operaciones espaciales	135
11.1.7. Comparaciones	136
11.2. Puntos de red temporales	136
11.3. Validez de los puntos de red temporal	137
11.4. Constructores	137
11.5. Conversión de tipos	138
11.6. Entrada y Salida MF-JSON	139
11.7. Accesorios	139
11.8. Transformaciones	140
11.9. Modificaciones	140
11.10 Restricciones	141
11.11 Operaciones de distancia	141
11.12 Operaciones espaciales	142

11.13Operaciones de cuadro delimitador	142
11.14Relaciones espaciales	143
11.15Comparaciones	144
11.16Agregacions	145
11.17Indexación	145
11.18Agregación	146
12. Búferes circulares temporales	147
12.1. Búferes circulares estáticos	148
12.1.1. Entrada y salida	148
12.1.2. Constructores	149
12.1.3. Conversión de tipos	149
12.1.4. Accesos	150
12.1.5. Transformaciones	150
12.1.6. Operaciones espaciales	150
12.1.7. Comparaciones	151
12.2. Búferes circulares temporales	151
12.3. Validez de los búferes circulares temporales	152
12.4. Entrada y salida	152
12.5. Constructores	154
12.6. Conversión de tipos	155
12.7. Accesos	156
12.8. Transformaciones	156
12.9. Modificaciones	157
12.10Operaciones espaciales	157
12.11Operaciones de distancia	159
12.12Restricciones	160
12.13Comparaciones	161
12.14Operaciones de cuadro delimitador	162
12.15Relaciones espaciales	162
12.16Agregaciones	163
12.17Simplificación	164
12.18Indexación	165
13. Poses temporales	166
13.1. Poses estáticas	166
13.1.1. Entrada y salida	167
13.1.2. Constructores	168
13.1.3. Conversión de tipos	168

13.1.4. Accesoros	169
13.1.5. Transformaciones	169
13.1.6. Sistema de referencia espacial	169
13.1.7. Operaciones de distancia	170
13.1.8. Comparaciones	171
13.2. Poses temporales	171
13.3. Validez de las poses temporales	172
13.4. Entrada y salida	172
13.5. Constructores	174
13.6. Conversión de tipos	175
13.7. Accesoros	176
13.8. Transformaciones	177
13.9. Modificaciones	177
13.10 Restricciones	178
13.11 Sistema de referencia espacial	179
13.12 Operaciones de cuadro delimitador	180
13.13 Operaciones de distancia	180
13.14 Relaciones espaciales	181
13.15 Comparaciones	182
13.16 Agregaciones	183
13.17 Indexación	183
13.18 Soporte de OGC GeoPose v1.0	184
13.18.1. Entrada y salida JSON	184
13.18.2. Renormalización de cuaterniones	184
13.18.3. Accesoros de ángulos de Euler	185
13.18.4. Registro de metadatos de marcos	186
13.18.5. Transformación rígida cuerpo↔mundo	187
13.18.6. E/S JSON temporal	188
14. Cadenas de poses	190
14.1. Cadenas de poses estáticas	190
14.1.1. Entrada y salida	190
14.1.2. Constructores	191
14.1.3. Conversión de tipos	192
14.1.4. Accesoros	192
14.1.5. Transformaciones	193
14.1.6. Sistema de referencia espacial	193
14.1.7. Comparaciones	194
14.2. Cadenas de poses temporales	194

14.2.1. Entrada y salida	195
14.2.2. Constructores	196
14.2.3. Conversiones	196
14.2.4. Funciones de acceso	196
14.2.5. Funciones de transformación	197
14.2.6. Sistema de referencia espacial	197
14.2.7. Cadena compuesta OGC GeoPose	198
14.2.8. Grafo compuesto OGC GeoPose	198
14.2.9. Funciones de restricción	199
14.2.10. Funciones y operadores de comparación	199
14.2.11. Operadores topológicos y de posición	199
15. Geometrías rígidas temporales	201
15.1. Entrada y salida	201
15.2. Constructores	203
15.3. Conversión de tipos	203
15.4. Accesores	204
15.5. Área recorrida	205
15.6. Funciones espaciales	206
15.7. Métricas de movimiento	206
15.8. Modificaciones	207
15.9. Restricciones espaciales	208
15.10. Operaciones de distancia	209
15.11. Similitud	210
15.12. Comparaciones	211
15.13. Operadores de caja delimitadora	211
15.14. Mosaicos	212
15.15. Cajas delimitadoras	212
15.16. Agregaciones	213
15.17. Indexación	213
16. Tipos JSON en MobilityDB	215
16.1. Operaciones sobre conjuntos JSONB	215
16.2. Valores JSONB temporales	221
16.3. Validez de los valores JSONB temporales	222
16.4. Entrada y salida	222
16.5. Constructores	223
16.6. Conversiones	224
16.7. Accesores	225

16.8. Transformaciones	225
16.9. Operaciones JSON temporales	226
16.10Restricciones	232
16.11Comparaciones	233
16.12Operaciones de caja delimitadora	234
16.13Agregaciones	234
16.14Indexación	235
17. Índices de Celdas H3 Temporales	236
17.1. Entrada y Salida	236
17.2. Conversiones	237
17.3. Cobertura de Geometrías Estáticas	237
17.4. Funciones de Acceso	238
17.5. Inspección de Índices	238
17.6. Jerarquía	239
17.7. Conversiones de Latitud/Longitud	240
17.8. Aristas Dirigidas	240
17.9. Vértices	241
17.10Recorrido de la Cuadrícula	242
17.11Métricas	242
17.12Comparaciones	243
17.12.1.Comparaciones Siempre y Alguna Vez	243
17.12.2.Comparaciones Temporales	243
17.13Operadores de Caja Envolvente	244
17.14Funciones que devuelven conjuntos	245
17.15Relaciones Espaciales	246
17.16Agregaciones	247
17.17Indexación	248
18. Índices de Celdas QUADBIN Temporales	249
18.1. Entrada y Salida	249
18.2. Conversiones	250
18.3. Celdas Estáticas	250
18.4. Accesores	251
18.5. Inspección del Índice	251
18.6. Jerarquía	252
18.7. Conversiones de Geometría	253
18.8. Teselas y Quadkeys	253
18.9. Métricas	254
18.10Comparaciones	254
18.11Agregaciones	254
18.12Indexación	255

19. Nubes de Puntos Temporales	256
19.1. Inicio rápido	256
19.1.1. Registrar un esquema	256
19.1.2. Construir valores base y temporales	257
19.1.3. Consultar con la superficie bbox	257
19.1.4. Índice para búsquedas impulsadas por bbox	258
19.1.5. Agregación	258
19.1.6. Acceso por punto	258
19.1.7. ¿Qué sigue?	258
19.2. Tipos estáticos <code>pcpoint</code> y <code>pcpatch</code>	258
19.2.1. Constructores ergonómicos	259
19.2.2. Funciones de acceso	259
19.3. Tipos de conjunto <code>pcpointset</code> y <code>pcpatchset</code>	260
19.3.1. Constructores	260
19.4. Caja Delimitadora <code>tpcbox</code>	260
19.4.1. Constructores	260
19.4.2. Conversiones	261
19.4.3. Accesos	261
19.4.4. Transformaciones	262
19.4.5. Operaciones de conjuntos	262
19.4.6. Operadores Topológicos	262
19.4.7. Operadores de Posición	263
19.4.8. Comparaciones	264
19.5. Tipo Temporal <code>tpcpoint</code>	264
19.5.1. Constructores	265
19.5.2. Funciones de Acceso	265
19.5.3. Proyecciones por Dimensión	265
19.5.4. Conversiones	266
19.5.5. Transformaciones	266
19.5.6. Comparaciones	267
19.5.7. Restricciones	268
19.5.8. Modificaciones	269
19.5.9. Fragmentación	270
19.5.10. Operadores de caja delimitadora	271
19.5.11. Ordenación B-tree, qué significa <code>ORDER BY tpcpoint</code>	272
19.5.12. Relaciones espaciales	272
19.6. Tipo Temporal <code>tpcpatch</code>	273
19.6.1. Constructores	273
19.6.2. Accesos	274

19.6.3. Transformaciones	275
19.6.4. Restricciones	275
19.6.5. Modificaciones	277
19.6.6. Fragmentación	278
19.6.7. Relaciones espaciales	279
19.6.8. Operadores de caja delimitadora	279
19.6.9. Comparaciones	279
19.6.10.Ordenación B-tree, qué significa ORDER BY tpcpatch	280
19.6.11.Operaciones por punto: qué está y qué no está disponible	281
19.7. Índices	281
19.8. Agregaciones	281
19.9. Representación binaria y portabilidad	283
19.10.Salida de texto	283
19.11.Salida MF-JSON	284
19.12.Integración con PDAL: readers.tpcpatch y writers.tpcpatch	284
19.12.1.readers.tpcpatch	284
19.12.2.writers.tpcpatch	285
19.12.3.Esquemas de compresión soportados	286
19.12.4.Limitaciones	286
19.13.Puente PCL: filtros invocables desde SQL y registro	286
19.14.Advertencias	287
20. Muestreo de Ráster	288
20.1. Operador de Muestreo	288
20.2. Accesores de Ráster	289
20.3. Muestreo Raquet / QUADBIN	289
20.4. Serialización de Raquet	292
20.5. Accesores y Comparación de Raquet	293
20.6. Operadores de Restricción y Predicado	294
20.7. Compilación e Instalación	295
20.8. Hoja de Ruta de Producción	296
21. Dialecto Portable de Funciones con Nombre	297
A. MobilityDB Reference	300
A.1. Tipos de MobilityDB	300
A.2. Set and Span Types	300
A.2.1. Entrada y salida	300
A.2.2. Constructores	301
A.2.3. Conversión de tipos	301

A.2.4. Accesosores	301
A.2.5. Transformaciones	302
A.2.6. Sistema de referencia espacial	302
A.2.7. Operaciones de conjuntos	302
A.2.8. Operaciones de cuadro delimitador	302
A.2.8.1. Operaciones topológicas	302
A.2.8.2. Operaciones de posición	302
A.2.8.3. Operaciones de división	303
A.2.9. Operaciones de distancia	303
A.2.10. Comparaciones	303
A.2.11. Agregaciones	303
A.3. Box Types	303
A.3.1. Entrada y salida	303
A.3.2. Constructores	303
A.3.3. Conversión de tipos	303
A.3.4. Accesosores	304
A.3.5. Transformaciones	304
A.3.6. Sistema de referencia espacial	304
A.3.7. Operaciones de división	304
A.3.8. Operaciones de conjuntos	304
A.3.9. Operaciones de cuadro delimitador	304
A.3.9.1. Operaciones topológicas	304
A.3.9.2. Operaciones de posición	305
A.3.10. Comparaciones	305
A.3.11. Agregaciones	305
A.4. Temporal Types	305
A.4.1. Entrada y salida	305
A.4.2. Constructores	305
A.4.3. Conversión de tipos	305
A.4.4. Accesosores	306
A.4.5. Transformaciones	306
A.4.6. Modificaciones	306
A.4.7. Restricciones	307
A.4.8. Comparaciones	307
A.4.8.1. Comparaciones tradicionales	307
A.4.8.2. Comparaciones alguna vez y siempre	307
A.4.8.3. Comparaciones temporales	307
A.4.9. Funciones de utilidad	307
A.5. Temporal Alphanumeric Types	307

A.5.1.	Conversión de tipos	307
A.5.2.	Accessores	307
A.5.3.	Transformaciones	308
A.5.4.	Restrictions	308
A.5.5.	Operaciones booleanas	308
A.5.6.	Operaciones matemáticas	308
A.5.7.	Operaciones de texto	308
A.6.	Temporal Geometry Types	309
A.6.1.	Funciones sobre Geometrías	309
A.6.2.	Entrada y salida	309
A.6.3.	Conversión de tipos	309
A.6.4.	Accessores	309
A.6.5.	Transformaciones	310
A.6.6.	Restricciones	310
A.6.7.	Sistema de referencia espacial	310
A.6.8.	Operaciones de cuadro delimitador	310
A.6.9.	Operaciones de distancia	311
A.6.10.	Relaciones espaciales	311
A.6.10.1.	Relaciones alguna vez o siempre	311
A.6.10.2.	Relaciones espaciotemporales	311
A.7.	Operations for Temporal Types: Analytics Operations	311
A.7.1.	Simplificación	311
A.7.2.	Reducción	312
A.7.3.	Similaridad	312
A.7.4.	Filtro de Kalman extendido	312
A.7.5.	Operaciones de división	312
A.7.6.	Mosaicos multidimensionales	312
A.7.6.1.	Operaciones de intervalos	312
A.7.6.2.	Operaciones de mosaicos	313
A.7.6.3.	Operaciones de cuadro delimitador	313
A.7.6.4.	Operaciones de división	313
A.7.7.	Agregación	313
A.8.	Temporal Poses	313
A.8.1.	Poses estáticas	313
A.8.1.1.	Entrada y salida	313
A.8.1.2.	Constructores	314
A.8.1.3.	Conversión de tipos	314
A.8.1.4.	Accesores	314
A.8.1.5.	Transformaciones	314

A.8.1.6.	Sistema de referencia espacial	314
A.8.1.7.	Operaciones de distancia	314
A.8.1.8.	Comparaciones	314
A.8.2.	Entrada y salida	315
A.8.3.	Constructores	315
A.8.4.	Conversión de tipos	315
A.8.5.	Accesores	315
A.8.6.	Transformaciones	316
A.8.7.	Modificaciones	316
A.8.8.	Restricciones	316
A.8.9.	Sistema de referencia espacial	316
A.8.10.	Operaciones de cuadro delimitador	316
A.8.11.	Operaciones de distancia	316
A.8.12.	Relaciones espaciales	316
A.8.13.	Comparaciones	316
A.8.14.	Agregaciones	317
A.8.15.	Soporte de OGC GeoPose v1.0	317
A.8.15.1.	Entrada y salida JSON	317
A.8.15.2.	Renormalización de cuaterniones	317
A.8.15.3.	Accesores de ángulos de Euler	317
A.8.15.4.	Registro de metadatos de marcos	317
A.8.15.5.	Transformación rígida cuerpo↔mundo	317
A.8.15.6.	E/S JSON temporal	317
A.9.	Pose Chains	318
A.9.1.	Cadenas de poses estáticas	318
A.9.1.1.	Entrada y salida	318
A.9.1.2.	Constructores	318
A.9.1.3.	Conversión de tipos	318
A.9.1.4.	Accesores	318
A.9.1.5.	Transformaciones	318
A.9.1.6.	Sistema de referencia espacial	318
A.9.1.7.	Comparaciones	318
A.9.2.	Cadenas de poses temporales	319
A.9.2.1.	Entrada y salida	319
A.9.2.2.	Constructores	319
A.9.2.3.	Conversiones	319
A.9.2.4.	Funciones de acceso	319
A.9.2.5.	Funciones de transformación	319
A.9.2.6.	Sistema de referencia espacial	319

A.9.2.7. Cadena compuesta OGC GeoPose	319
A.9.2.8. Grafo compuesto OGC GeoPose	319
A.9.2.9. Funciones de restricción	319
A.9.2.10. Funciones y operadores de comparación	319
A.9.2.11. Operadores topológicos y de posición	320
A.10. Temporal Network Points	320
A.10.1. Tipos de red estáticos	320
A.10.1.1. Entrada y salida	320
A.10.1.2. Constructores	320
A.10.1.3. Conversión de tipos	320
A.10.1.4. Accesores	320
A.10.1.5. Transformaciones	320
A.10.1.6. Operaciones espaciales	320
A.10.1.7. Comparaciones	321
A.10.2. Constructores	321
A.10.3. Conversión de tipos	321
A.10.4. Entrada y Salida MF-JSON	321
A.10.5. Accesores	321
A.10.6. Transformaciones	321
A.10.7. Modificaciones	321
A.10.8. Restricciones	321
A.10.9. Operaciones de distancia	322
A.10.10. Operaciones espaciales	322
A.10.11. Operaciones de cuadro delimitador	322
A.10.12. Relaciones espaciales	322
A.10.13. Comparaciones	322
A.10.14. Agregacions	322
A.10.15. Agregación	322
A.11. Temporal Circular Buffers	323
A.11.1. Búferes circulares estáticos	323
A.11.1.1. Entrada y salida	323
A.11.1.2. Constructores	323
A.11.1.3. Conversión de tipos	323
A.11.1.4. Accesores	323
A.11.1.5. Transformaciones	323
A.11.1.6. Operaciones espaciales	323
A.11.1.7. Comparaciones	323
A.11.2. Entrada y salida	324
A.11.3. Constructores	324

A.11.4. Conversión de tipos	324
A.11.5. Accesos	324
A.11.6. Transformaciones	324
A.11.7. Modificaciones	324
A.11.8. Operaciones espaciales	325
A.11.9. Operaciones de distancia	325
A.11.10 Restricciones	325
A.11.11 Comparaciones	325
A.11.12 Operaciones de cuadro delimitador	325
A.11.13 Relaciones espaciales	325
A.11.14 Agregaciones	326
A.11.15 Simplificación	326
A.12. Temporal Rigid Geometries	326
A.12.1. Entrada y salida	326
A.12.2. Constructores	326
A.12.3. Conversión de tipos	326
A.12.4. Accesos	327
A.12.5. Área recorrida	327
A.12.6. Funciones espaciales	327
A.12.7. Métricas de movimiento	327
A.12.8. Modificaciones	327
A.12.9. Restricciones espaciales	327
A.12.10 Operaciones de distancia	328
A.12.11 Similitud	328
A.12.12 Comparaciones	328
A.12.13 Operadores de caja delimitadora	328
A.12.14 Mosaicos	328
A.12.15 Cajas delimitadoras	328
A.12.16 Agregaciones	328
A.13. Temporal JSON	329
A.13.1. Operaciones sobre conjuntos JSONB	329
A.13.2. Entrada y salida	329
A.13.3. Constructores	329
A.13.4. Conversiones	330
A.13.5. Accesos	330
A.13.6. Transformaciones	330
A.13.7. Operaciones JSON temporales	330
A.13.8. Restricciones	330
A.13.9. Comparaciones	331

A.13.10Operaciones de caja delimitadora	331
A.13.11Agregaciones	331
A.14.Temporal H3 Cell Indices	331
A.14.1. Conversiones	331
A.14.2. Cobertura de Geometrías Estáticas	331
A.14.3. Funciones de Acceso	331
A.14.4. Inspección de Índices	332
A.14.5. Jerarquía	332
A.14.6. Conversiones de Latitud/Longitud	332
A.14.7. Aristas Dirigidas	332
A.14.8. Vértices	332
A.14.9. Recorrido de la Cuadrícula	333
A.14.10Métricas	333
A.14.11Comparaciones	333
A.14.11.1.Comparaciones Siempre y Alguna Vez	333
A.14.11.2.Comparaciones Temporales	333
A.14.12Operadores de Caja Envolvente	333
A.14.13Funciones que devuelven conjuntos	333
A.14.14Relaciones Espaciales	334
A.14.15Agregaciones	334
A.14.16Indexación	334
A.15.Temporal Quadbin Cell Indices	334
A.15.1. Conversiones	334
A.15.2. Celdas Estáticas	334
A.15.3. Accesores	335
A.15.4. Inspección del Índice	335
A.15.5. Jerarquía	335
A.15.6. Conversiones de Geometría	335
A.15.7. Teselas y Quadkeys	335
A.15.8. Métricas	336
A.15.9. Agregaciones	336
A.15.10Indexación	336
A.15.11Comparisons	336
A.16.Temporal Point Clouds	336
A.16.1. Tipos estáticos <code>pcpoint</code> y <code>pcpatch</code>	336
A.16.1.1. Constructores ergonómicos	336
A.16.1.2. Funciones de acceso	336
A.16.2. Tipos de conjunto <code>pcpointset</code> y <code>pcpatchset</code>	336
A.16.2.1. Constructores	336

A.16.3. Caja Delimitadora <code>tpcbox</code>	337
A.16.3.1. Constructores	337
A.16.3.2. Conversiones	337
A.16.3.3. Accesos	337
A.16.3.4. Transformaciones	337
A.16.3.5. Operaciones de conjuntos	337
A.16.3.6. Operadores Topológicos	337
A.16.3.7. Operadores de Posición	337
A.16.3.8. Comparaciones	338
A.16.4. Tipo Temporal <code>tpcpoint</code>	338
A.16.4.1. Constructores	338
A.16.4.2. Funciones de Acceso	338
A.16.4.3. Proyecciones por Dimensión	338
A.16.4.4. Conversiones	338
A.16.4.5. Transformaciones	338
A.16.4.6. Comparaciones	338
A.16.4.7. Restricciones	339
A.16.4.8. Modificaciones	339
A.16.4.9. Fragmentación	339
A.16.4.10. Operadores de caja delimitadora	339
A.16.4.11. Relaciones espaciales	339
A.16.5. Tipo Temporal <code>tpcpatch</code>	339
A.16.5.1. Constructores	339
A.16.5.2. Accesos	340
A.16.5.3. Transformaciones	340
A.16.5.4. Restricciones	340
A.16.5.5. Modificaciones	340
A.16.5.6. Fragmentación	340
A.16.5.7. Relaciones espaciales	341
A.16.5.8. Operadores de caja delimitadora	341
A.16.5.9. Comparaciones	341
A.16.6. Agregaciones	341
A.16.7. Salida de texto	341
A.16.8. Salida MF-JSON	342
A.16.9. Indexes	342
A.16.10. Binary Representation	342
A.16.11. PDAL Integration	342
A.16.12. PCL Bridge	342
A.17. Raster Sampling	342

A.17.1. Operador de Muestreo	342
A.17.2. Accesos de Ráster	342
A.17.3. Muestreo Raquet / QUADBIN	342
A.17.4. Serialización de Raquet	343
A.17.5. Accesos y Comparación de Raquet	343
A.17.6. Operadores de Restricción y Predicado	343

B. Generador de datos sintéticos	344
B.1. Generador para tipos PostgreSQL	344
B.2. Generador para tipos PostGIS	345
B.3. Generador para tipos de rango, de tiempo y de cuadro delimitador MobilityDB	346
B.4. Generador para tipos temporales MobilityDB	346
B.5. Generación de tablas con valores aleatorios	347
B.6. Generador para tipos de red temporales	352

Índice de figuras

- 3.1. Grilla multiresolución sobre datos de Bruselas obtenidos mediante el generador BerlinMOD. Cada celda contiene como máximo 10.000 (izquierda) y 1.000 (derecha) instantes durante todo el período de simulación (cuatro días en este caso). A la izquierda, podemos ver la alta densidad de tráfico en el anillo alrededor de Bruselas, mientras que a la derecha podemos ver otros ejes principales de la ciudad. 39
- 5.1. Operación de inserción para valores temporales. 64
- 5.2. Operaciones de actualización y supresión para valores temporales. 65
- 5.3. Operaciones de modificación para tablas temporales en SQL. 65
- 7.1. Jerarquía de tipos espaciotemporales en MobilityDB. 83
- 7.2. Ilustración del uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024. 84
- 7.3. Visualización de la velocidad de un objeto móvil usando una rampa de color en QGIS. 94
- 9.1. Diferencia entre la distancia espacial y la distancia sincronizada. 107
- 9.2. Muestreo de flotantes temporales con interpolación discreta, escalonada y lineal. 108
- 9.3. Cambio de precisión de números flotantes temporales con interpolación discreta, escalonada y lineal. 110
- 9.4. Mosaicos multidimensionales para números flotantes temporales. 117
- 12.1. Ilustración del uso del tipo `tcbuffer` para modelar el búfer circular alrededor de la franja de viento de las tormentas tropicales en 2024. 147

Índice de cuadros

1.1. Variables para la documentación del usuario y del desarrollador	6
A.1. Instancias actuales de los tipos de plantilla en MobilityDB	300

Resumen

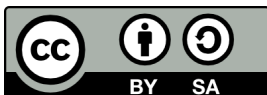
MobilityDB es una extensión del sistema de base de datos [PostgreSQL](#) y su extensión espacial [PostGIS](#). Permite almacenar en la base de datos objetos temporales y espacio-temporales, es decir, objetos cuyos valores de atributo y/o ubicación evolucionan en el tiempo. MobilityDB incluye funciones para el análisis y procesamiento de objetos temporales y espacio-temporales y proporciona soporte para índices GiST y SP-GiST. MobilityDB es de código abierto y su código está disponible en [Github](#). También está disponible un adaptador para el lenguaje de programación Python en [Github](#).



MobilityDB es desarrollado por el Departamento de Ingeniería Informática y de Decisiones de la Université libre de Bruxelles (ULB) bajo la dirección del Prof. Esteban Zimányi. ULB es un miembro asociado de OGC y miembro del Grupo de trabajo de estandarización de características móviles de OGC ([MF-SWG](#)).



El manual de MobilityDB tiene una licencia [Creative Commons Attribution-Share Alike 3.0 License 3](#). No dude en utilizar este material como desee, pero le pedimos que atribuya crédito al proyecto MobilityDB y, siempre que sea posible, un enlace a [MobilityDB](#).



Capítulo 1

Introducción

MobilityDB es una extensión de [PostgreSQL](#) y [PostGIS](#) que proporciona *tipos temporales*. Dichos tipos de datos representan la evolución en el tiempo de los valores de algún tipo de elemento, llamado tipo base del tipo temporal. Por ejemplo, se pueden usar enteros temporales para representar la evolución en el tiempo de la marcha utilizada por un automóvil en movimiento. En este caso, el tipo de datos es *entero temporal* y el tipo base es *entero*. Del mismo modo, se puede utilizar un número flotante temporal para representar la evolución en el tiempo de la velocidad de un automóvil. Como otro ejemplo, se puede usar un punto temporal para representar la evolución en el tiempo de la ubicación de un automóvil, como lo reportan los dispositivos GPS. Los tipos temporales son útiles porque representar valores que evolucionan en el tiempo es esencial en muchas aplicaciones, por ejemplo, en aplicaciones de movilidad. Además, los operadores de los tipos base (como los operadores aritméticos y la agregación para números enteros y flotantes, las relaciones topológicas y la distancia para las geometrías) se pueden generalizar intuitivamente cuando los valores evolucionan en el tiempo.

MobilityDB proporciona los siguientes tipos temporales: `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint` y `tgeogpoint`. Estos tipos temporales se basan, respectivamente, en los tipos de base `bool`, `integer`, `float` y `text` proporcionados por PostgreSQL, y en los tipos base de `geometry` y `geography` proporcionados por PostGIS, donde `tgeometry` y `tgeography` aceptan geometrías/geografías arbitrarias, mientras que `tgeompoint` y `tgeogpoint` solo aceptan puntos 2D o 3D.¹ Además, MobilityDB proporciona los tipos de plantilla *set*, *span* y *span set* para representar, respectivamente, un conjunto de valores, un rango de valores y un conjunto de rangos de valores de tipos de base o tipos de tiempo. Ejemplos de valores de tipos de conjunto son `intset`, `floatset` y `tstzset`, donde el último representa un conjunto de valores `timestampz`. Ejemplos de valores de tipos de rango son `intspan`, `floatspan` y `tstzspan`. Ejemplos de valores de tipos de conjuntos de rangos son `intspanset`, `floatspanset` y `tstzspanset`.

1.1. Comité directivo del proyecto

El comité directivo del proyecto MobilityDB (Project Steering Committee o PSC) coordina la dirección general, los ciclos de publicación, la documentación y los esfuerzos de divulgación para el proyecto MobilityDB. Además, el PSC proporciona soporte general al usuario, acepta y aprueba parches de la comunidad general de MobilityDB y vota sobre diversos problemas relacionados con MobilityDB, como el acceso de commit de los desarrolladores, nuevos miembros del PSC o cambios significativos en la interfaz de programación de aplicaciones (Application Programming Interface o API).

A continuación se detallan los miembros actuales en orden alfabético y sus principales responsabilidades:

- Mohamed Bakli: [MobilityDB-docker](#), versiones distribuidas y en la nube, integración con [Citius](#)
- Krishna Chaitanya Bommakanti: [MEOS \(Mobility Engine Open Source\)](#), [pyMEOS](#)
- Anita Graser: integración con [Moving Pandas](#) y el ecosistema de Python, integración con [QGIS](#)
- Darafei Praliaskouski: integración con [PostGIS](#)

¹ Aunque los puntos temporales 4D se pueden representar, la dimensión M actualmente no se tiene en cuenta.

- Mahmoud Sakr: cofundador del proyecto MobilityDB, [MobilityDB workshop](#), copresidente del OGC Moving Feature Standard Working Group ([MF-SWG](#))
- Vicky Vergara: integración con [pgRouting](#), enlace con [OSGeo](#)
- Esteban Zimányi (chair): cofundador del proyecto MobilityDB, coordinación general del proyecto, principal contribuidor del código de backend, [BerlinMOD generator](#)

1.2. Otros colaboradores del código

- Arthur Lesuisse
- Xinyiang Li
- Maxime Schoemans

1.3. Patrocinadores

Estas son organizaciones de financiación de investigación (en orden alfabético) que han contribuido con financiación monetaria al proyecto MobilityDB.

- [European Commission](#)
- [Fonds de la Recherche Scientifique \(FNRS\), Belgium](#)
- [Innoviris, Belgium](#)

Estas son entidades corporativas (en orden alfabético) que han contribuido con tiempo de desarrollador o financiación monetaria al proyecto MobilityDB.

- [Adonmo, India](#)
- [Georepublic, Germany](#)
- [Université libre de Bruxelles, Belgium](#)

1.4. Licencias

Las siguientes licencias se pueden encontrar en MobilityDB:

Recurso	Licencia
Código MobilityDB	Licencia PostgreSQL
Documentación MobilityDB	Licencia Creative Commons Attribution-Share Alike 3.0

1.5. Instalación a partir de las fuentes

1.5.1. Versión corta

Para compilar asumiendo que tiene todas las dependencias en su ruta de búsqueda

```
git clone https://github.com/MobilityDB/MobilityDB
mkdir MobilityDB/build
cd MobilityDB/build
cmake ..
make
sudo make install
```

Los comandos anteriores instalan la rama `master`. Si desea instalar otra rama, por ejemplo, `develop`, puede reemplazar el primer comando anterior de la siguiente manera:

```
git clone --branch develop https://github.com/MobilityDB/MobilityDB
```

También debe configurar lo siguiente en el archivo `postgresql.conf` según la versión de PostGIS que haya instalado (a continuación usamos PostGIS 3):

```
shared_preload_libraries = 'postgis-3'
max_locks_per_transaction = 128
```

Si no carga previamente la biblioteca PostGIS con la configuración anterior, no podrá cargar la biblioteca MobilityDB y obtendrá un mensaje de error como el siguiente:

```
ERROR: could not load library "/usr/local/pgsql/lib/libMobilityDB-1.1.so":
        undefined symbol: ST_Distance
```

Puede encontrar la ubicación del archivo `postgresql.conf` de la manera siguiente.

```
$ which postgres
/usr/local/pgsql/bin/postgres
$ ls /usr/local/pgsql/data/postgresql.conf
/usr/local/pgsql/data/postgresql.conf
```

Como puede verse, los binarios de PostgreSQL están en el subdirectorio `bin` mientras que el archivo `postgresql.conf` está en el subdirectorio `data`.

Una vez que MobilityDB está instalado, debe habilitarse en cada base de datos en la que desee usarlo. En el siguiente ejemplo, usamos una base de datos llamada `mobility`.

```
createdb mobility
psql mobility -c "CREATE EXTENSION PostGIS"
psql mobility -c "CREATE EXTENSION MobilityDB"
```

Las dos extensiones PostGIS y MobilityDB también se pueden crear en un solo comando.

```
psql mobility -c "CREATE EXTENSION MobilityDB cascade"
```

1.5.2. Obtener las fuentes

La última versión de MobilityDB se puede encontrar en <https://github.com/MobilityDB/MobilityDB/releases/latest>

wget

Para descargar esta versión:

```
wget -O mobilitydb-1.3.tar.gz https://github.com/MobilityDB/MobilityDB/archive/v1.3.tar.gz
```

Ir a la Sección [1.5.1](#) para las instrucciones de extracción y compilación.

git

Para descargar el repositorio

```
git clone https://github.com/MobilityDB/MobilityDB.git
cd MobilityDB
git checkout v1.3
```

Ir a la Sección [1.5.1](#) para las instrucciones de compilación (no hay tar ball).

1.5.3. Habilitación de la base de datos

MobilityDB es una extensión que depende de PostGIS. Habilitar PostGIS antes de habilitar MobilityDB en la base de datos se puede hacer de la siguiente manera

```
CREATE EXTENSION postgis;
CREATE EXTENSION mobilitydb;
```

Alternativamente, esto se puede hacer con un solo comando usando `CASCADE`, que instala la extensión PostGIS requerida antes de instalar la extensión MobilityDB

```
CREATE EXTENSION mobilitydb CASCADE;
```

1.5.4. Dependencias

Dependencias de compilación

Para poder compilar MobilityDB, asegúrese de que se cumplan las siguientes dependencias:

- Sistema de compilación multiplataforma CMake.
- Compilador C `gcc` o `clang`. Se pueden usar otros compiladores ANSI C, pero pueden causar problemas al compilar algunas dependencias.
- GNU Make (`gmake` o `make`) versión 3.1 o superior. Para muchos sistemas, GNU make es la versión predeterminada de make. Verifique la versión invocando `make -v`.
- PostgreSQL versión 16 a 19. Fuera de ese rango, la configuración se detiene con un error explícito. PostgreSQL está disponible en <http://www.postgresql.org>.
- PostGIS versión 3 o superior. PostGIS está disponible en <https://postgis.net/>.

Nótese que PostGIS tiene sus propias dependencias, como Proj, GEOS, LibXML2 o JSON-C y estas bibliotecas también se utilizan en MobilityDB. Consulte <http://trac.osgeo.org/postgis/wiki/UsersWikiPostgreSQLPostGIS> para obtener una matriz de compatibilidad de PostGIS con PostgreSQL, GEOS y Proj.

Dependencias opcionales

Para la documentación del usuario

- Los archivos DocBook DTD y XSL son necesarios para crear la documentación. Para Ubuntu, son proporcionados por los paquetes `docbook` y `docbook-xsl`.
- El validador XML `xmllint` es necesario para validar los archivos XML de la documentación. Para Ubuntu, lo proporciona el paquete `libxml2`.
- El procesador XSLT `xsltproc` es necesario para crear la documentación en formato HTML. Para Ubuntu, lo proporciona el paquete `libxslt`.
- El programa `dblatex` es necesario para crear la documentación en formato PDF. Para Ubuntu, lo proporciona el paquete `dblatex`.

- El programa `dbtoepub` es necesario para construir la documentación en formato EPUB. Para Ubuntu, lo proporciona el paquete `dbtoepub`.

Para la documentación de los desarrolladores

- El programa `doxygen` es necesario para construir la documentación. Para Ubuntu, lo proporciona el paquete `doxygen`.

Ejemplo: instalar dependencias en distribuciones basadas en Debian

Dependencias de base de datos

```
sudo apt-get install postgresql-16 postgresql-server-dev-16 postgresql-16-postgis
```

Dependencias de construcción

```
sudo apt-get install cmake gcc
```

Ejemplo: instalar dependencias en distribuciones basadas en RPM

En Fedora, Red Hat Enterprise Linux y sus derivados, los paquetes de la distribución incluyen una versión compatible de PostgreSQL y de PostGIS, por lo que un solo comando cubre tanto las dependencias de base de datos como las de construcción.

```
sudo dnf install gcc gcc-c++ cmake make postgresql-server-devel postgis \
    geos-devel proj-devel json-c-devel
```

Las familias opcionales que habilita `-DALL=ON` requieren dos paquetes adicionales, `gdal-devel` para los operadores de rásteres y `h3-devel` para el tipo de índice H3.

1.5.5. Configuración

MobilityDB usa el sistema `cmake` para realizar la configuración. El directorio de compilación debe ser diferente del directorio de origen.

Para crear el directorio de compilación

```
mkdir build
```

Para ver las variables que se pueden configurar

```
cd build
cmake -L ..
```

1.5.6. Construir e instalar

Nótese que la versión actual de MobilityDB solo se ha probado en sistemas Linux, MacOS y Windows. Puede funcionar en otros sistemas similares a Unix, pero no se ha probado. Buscamos voluntarios que nos ayuden a probar MobilityDB en múltiples plataformas.

Las siguientes instrucciones comienzan desde `path/to/MobilityDB` en un sistema Linux

```
mkdir build
cd build
cmake ..
make
sudo make install
```

Cuando cambia la configuración

```
rm -rf build
```

e inicie el proceso de construcción como se mencionó anteriormente.

1.5.7. Pruebas

MobilityDB utiliza `ctest`, el programa controlador de pruebas de CMake, para realizar pruebas. Este programa ejecutará las pruebas e informará los resultados.

Para ejecutar todas las pruebas

```
ctest
```

Para ejecutar un archivo de prueba dado

```
ctest -R '021_tbox'
```

Para ejecutar un conjunto de archivos de prueba determinados se pueden utilizar comodines

```
ctest -R '022_*'
```

1.5.8. Documentación

La documentación del usuario de MobilityDB se puede generar en formato HTML, PDF y EPUB. Además, la documentación está disponible en inglés y en otros idiomas (actualmente, solo en español). La documentación del usuario se puede generar en todos los formatos y en todos los idiomas, o se pueden especificar formatos y/o idiomas específicos. La documentación del desarrollador de MobilityDB solo se puede generar en formato HTML y en inglés.

Las variables utilizadas para generar la documentación del usuario y del desarrollador son las siguientes:

Variable	Valor por defecto	Comentario
DOC_ALL	BOOL=OFF	La documentación del usuario se genera en formatos HTML, PDF y EPUB.
DOC_HTML	BOOL=OFF	La documentación del usuario se genera en formato HTML.
DOC_PDF	BOOL=OFF	La documentación del usuario se genera en formato PDF.
DOC_EPUB	BOOL=OFF	La documentación del usuario se genera en formato EPUB.
LANG_ALL	BOOL=OFF	La documentación del usuario se genera en inglés y en todas las traducciones disponibles.
ES	BOOL=OFF	La documentación del usuario se genera en inglés y en español.
DOC_DEV	BOOL=OFF	La documentación del desarrollador en inglés se genera en formato HTML

Cuadro 1.1: Variables para la documentación del usuario y del desarrollador

Generar la documentación del usuario y del desarrollador en todos los formatos y en todos los idiomas.

```
cmake -D DOC_ALL=ON -D LANG_ALL=ON -D DOC_DEV=ON ..
make doc
make doc_dev
```

Generar la documentación del usuario en formato HTML y en todos los idiomas.

```
cmake -D DOC_HTML=ON -D LANG_ALL=ON ..
make doc
```

Generar la documentación del usuario en inglés en todos los formatos.

```
cmake -D DOC_ALL=ON ..
make doc
```

Generar la documentación del usuario en formato PDF en inglés y en español.

```
cmake -D DOC_PDF=ON -D ES=ON ..
make doc
```

1.6. Instalación a partir de binarios

1.6.1. Distribuciones de Linux basadas en Debian

Se está desarrollando soporte para sistemas Linux basados en Debian, como Ubuntu y Arch Linux.

1.6.2. Windows

Desde la versión 3.3.3 de PostGIS, MobilityDB se distribuye en el PostGIS Bundle para Windows, que está disponible en application stackbuilder y en el sitio web de OSGeo. Para más información refiérase a la [documentación PostGIS](#).

1.7. Soporte

El soporte de la comunidad de MobilityDB está disponible a través de la página github de MobilityDB, documentación, tutoriales, listas de correo y otros.

1.7.1. Reporte de problemas

Los errores son registrados y manejados en un [issue tracker](#). Por favor siga los siguientes pasos:

1. Busque las entradas para ver si su problema ya ha sido informado. Si es así, agregue cualquier contexto adicional que haya encontrado, o al menos indique que usted también está teniendo el problema. Esto nos ayudará a priorizar los problemas comunes.
2. Si su problema no se ha informado, cree un [nuevo asunto](#) para ello.
3. En su informe, incluya instrucciones explícitas para replicar su problema. Las mejores entradas incluyen consultas SQL exactas que se necesitan para reproducir un problema. Reporte también el sistema operativo y las versiones de MobilityDB, PostGIS y PostgreSQL.
4. Se recomienda utilizar el siguiente envoltorio en su problema para determinar el paso que está causando el problema.

```
SET client_min_messages TO debug;  
<your code>  
SET client_min_messages TO notice;
```

1.7.2. Listas de correo

Hay dos listas de correo para MobilityDB alojadas en el servidor de listas de correo OSGeo:

- Lista de correo de usuarios: <http://lists.osgeo.org/mailman/listinfo/mobilitydb-users>
- Lista de distribución de desarrolladores: <http://lists.osgeo.org/mailman/listinfo/mobilitydb-dev>

Para preguntas generales y temas sobre cómo usar MobilityDB, escriba a la lista de correo de usuarios.

1.8. Migración de la versión 1.0 a la versión 1.1

MobilityDB 1.1 es una revisión importante con respecto a la versión inicial 1.0. El cambio más significativo en la versión 1.1 fue extraer la funcionalidad central para la gestión de datos temporales y espaciotemporales de MobilityDB en la biblioteca C Mobility Engine Open Source (**MEOS**). De esta forma, la misma funcionalidad que proporciona MobilityDB en un entorno de base de datos está disponible en un programa C para ser utilizada, por ejemplo, en un entorno de streaming. La biblioteca MEOS para la gestión de la movilidad proporciona una funcionalidad similar a la biblioteca C/C++ Geometry Engine Open Source (**GEOS**) para geometría computacional. Además, están disponibles contenedores para la biblioteca MEOS en otros lenguajes de programación, en particular para Python con **PyMEOS**. Además, contenedores para C#, Java y Javascript, están en desarrollo.

Fueron necesarios varios cambios con respecto a la versión 1.0 de MobilityDB para habilitar lo anterior. Uno importante fue la definición de nuevos tipos de datos `span` y `spanset`, que brindan una funcionalidad similar a los tipos de datos de PostgreSQL `range` y `multirange` pero también se puede usar en varios lenguajes de programación independientemente de PostgreSQL. Estos son *tipos de plantilla*, lo que significa que son contenedores de otros tipos, de forma similar a como las listas y matrices contienen valores de otros tipos. Además, también se agregó un nuevo tipo de datos de plantilla `set`. Por lo tanto, los tipos `timestampset`, `period` y `periodset` en la versión 1.0 se reemplazan por los tipos `tstzset`, `tstzspan` y `tstzspanset` en la versión 1.1. El nombre de las funciones constructoras para estos tipos se modificó en consecuencia.

Finalmente, se simplificó la API de MEOS y MobilityDB para mejorar la usabilidad. Detallamos a continuación los cambios más importantes en la API.

- Las funciones `atTimestamp`, `atTimestampSet`, `atPeriod`, and `atPeriodSet` fueron renombradas a `atTime`.
- Las funciones `minusTimestamp`, `minusTimestampSet`, `minusPeriod` y `minusPeriodSet` fueron renombradas a `minusTime`.
- Las funciones `atValue`, `atValues`, `atRange` y `atRanges` fueron renombradas a `atValues`.
- Las funciones `minusValue`, `minusValues`, `minusRange` y `minusRanges` fueron renombradas a `minusValues`.
- Las funciones `contains`, `disjoint`, `dwithin`, `intersects` y `touches` fueron renombradas, respectivamente, a `eContains`, `eDisjoint`, `eDwithin`, `eIntersects` y `eTouches`.

Capítulo 2

Tipos de conjunto y de rango

MobilityDB proporciona los tipos de *conjunto*, *rango* y *conjunto de rangos* para representar conjuntos de valores de otro tipo, que se denomina *tipo base*. Los tipos de conjunto son similares a los tipos de matrices de PostgreSQL restringidos a una dimensión, pero imponen la restricción de que los conjuntos no tienen duplicados. Los tipos de rango y conjunto de rangos en MobilityDB corresponden a los tipos de rango y multirango en PostgreSQL pero tienen restricciones adicionales. En particular, los tipos de rango en MobilityDB tienen una longitud fija y no permiten rangos vacíos ni límites infinitos. Si bien los tipos de rango en MobilityDB proporcionan una funcionalidad similar a los tipos de rango en PostgreSQL, los tipos de rango en MobilityDB permiten aumentar el rendimiento. En particular, se elimina la sobrecarga del procesamiento de tipos de longitud variable y, además, se pueden utilizar la aritmética de punteros y la búsqueda binaria.

Los tipos de base que se utilizan para construir tipos de conjunto, de rango y de conjunto de rangos son los tipos `integer`, `bigint`, `float`, `text`, `date`, and `timestampz` (marca de tiempo con zona horaria) proporcionados por PostgreSQL, los tipos `geometry` y `geography` proporcionados por PostGIS, y los tipos `cbuffer` (búfer circular, ver Capítulo 12) y `npoint` (*network point* o punto de red, ver Capítulo 11) proporcionados por MobilityDB. MobilityDB proporciona los siguientes tipos de conjunto y de rango:

- `set`: `intset`, `bigintset`, `floatset`, `textset`, `dateset`, `tstzset`, `geomset`, `geogset`, `cbufferset`, `npointset`.
- `span`: `intspan`, `bigintspan`, `floatspan`, `datespan`, `tstzspan`.
- `spanset`: `intspanset`, `bigintspanset`, `floatspanset`, `datespanset`, `tstzspanset`.

A continuación presentamos las funciones y operadores para tipos de conjunto y de rango. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo de resultado puede depender del tipo de los argumentos. Para expresar esto en la firma de las funciones y los operadores, utilizamos la siguiente notación:

- `set` representa cualquier tipo de conjunto, como `intset` o `tstzset`.
- `span` representa cualquier tipo de rango, como `intspan` o `tstzspan`.
- `spanset` representa cualquier tipo de conjunto de rangos, como `intspanset` o `tstzspanset`.
- `spans` representa cualquier tipo de rango o conjunto de rangos, como `intspan` o `tstzspanset`.
- `base` representa cualquier tipo de base de un tipo de conjunto o de rango, como `integer` o `timestampz`.
- `number` representa cualquier tipo de base de un tipo de rango numérico, como `integer` o `float`,
- `numset` representa cualquier tipo de conjunto numérico, como `intset` o `floatset`.
- `numspans` representa cualquier tipo de rango o conjunto de rangos numérico, como `intspan` o `floatspanset`.
- `numbers` representa cualquier tipo de conjunto o de rango numérico, como `integer`, `intset`, `intspan` o `intspanset`,
- `dates` representa cualquier tipo de tiempo con granularidad `date`, a saber `date`, `dateset`, `datespan` o `datespanset`,

- `numset` representa cualquier tipo de conjunto numérico, como `intset` o `floatset`.
- `numspans` representa cualquier tipo de rango o conjunto de rangos numérico, como `intspan` o `floatspanset`.
- `numbers` representa cualquier tipo de conjunto o de rango numérico, como `integer`, `intset`, `intspan` o `intspanset`,
- `dates` representa cualquier tipo de tiempo con granularidad `date`, a saber `date`, `dateset`, `datespan` o `datespanset`,
- `times` representa cualquier tipo de tiempo con granularidad `timestampz`, a saber `timestampz`, `tstzset`, `tstzspan` o `tstzspanset`,
- Un conjunto de tipos como `{set, base}` representa cualquiera de los tipos enumerados,
- `type[]` representa una matriz de `type`.

Como ejemplo, la firma del operador contiene (`@>`) es como sigue:

```
{set, spans} @> {base, set, spans} → boolean
```

Nótese que la firma anterior es una versión abreviada de la firma más precisa a continuación

```
set @> {base, set} → boolean
spans @> {base, spans} → boolean
```

ya que los conjuntos y los rangos no se pueden mezclar en las operaciones y, por lo tanto, por ejemplo, no se puede preguntar si un rango contiene un conjunto. A continuación, por concisión, utilizamos el estilo abreviado de las firmas anteriores. Además, la parte de tiempo de las marcas de tiempo se omite en la mayoría de los ejemplos. Recuerde que en ese caso PostgreSQL asume el tiempo `00:00:00`.

A continuación, dado que los tipos de rango y conjunto de rangos tienen funciones y operadores similares, cuando hablamos de tipos rango nos referimos a los tipos de rango y conjuntos de rangos, a menos que nos refiramos explícitamente a los tipos de rango *unitarios* y a los tipos de *conjunto* de rangos para distinguirlos. Además, cuando nos referimos a tipos de tiempo, nos referimos a uno de los siguientes tipos: `timestampz`, `tstzset`, `tstzspan` o `tstzspanset`.

2.1. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (**WKT**) y Well-Known Binary (**WKB**) del Open Geospatial Consortium (**OGC**) a para todos sus tipos. De esta forma, las aplicaciones pueden intercambiar datos entre ellas utilizando un formato de intercambio estandarizado. El formato WKT es legible por humanos, mientras que el formato WKB es más compacto y más eficiente que el formato WKT. El formato WKB se puede generar como una cadena binaria o como una cadena de caracteres codificada en ASCII hexadecimal.

Los tipos de conjunto representan un conjunto *ordenado* de valores *diferentes*. Un conjunto debe contener al menos un elemento. Ejemplos de valores de tipos de conjunto son como sigue:

```
SELECT tstzset '{2001-01-01 08:00:00, 2001-01-03 09:30:00}';
-- Conjunto unitario
SELECT textset '{"highway"}';
-- Conjunto erróneo: elementos desordenados
SELECT floatset '{3.5, 1.2}';
-- Conjunto erróneo: elementos duplicados
SELECT geomset '{"Point(1 1)", "Point(1 1)}';
-- Conjunto de búferes circulares: cada elemento es un cbuffer en WKT
SELECT cbuffereset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)}';
```

Nótese que los elementos de los conjuntos `textset`, `geomset`, `geogset`, `cbuffereset` y `npointset` deben estar delimitados entre commillas dobles. Nótese también que las geometrías y las geografías utilizan el orden definido por PostGIS.

Un valor de un tipo de rango unitario tiene dos límites, el *límite inferior* y el *límite superior*, que son valores del *tipo de base* subyacente. Por ejemplo, un valor del tipo `tstzspan` tiene dos límites, que son valores de `timestampz`. Los límites

pueden ser inclusivos o exclusivos. Un límite inclusivo significa que el instante límite está incluido en el rango, mientras que un límite exclusivo significa que el instante límite no está incluido en el rango. En el formato textual de un valor de un rango, los límites inferiores inclusivos y exclusivos están representados, respectivamente, por “[” y “(”. Asimismo, los límites superiores inclusivos y exclusivos se representan, respectivamente, por “]” y “)”. En un valor de un rango, el límite inferior debe ser menor o igual que el límite superior. Un valor de rango con límites iguales e inclusivos se llama *rango instantáneo* y corresponde a un valor del tipo de base. Ejemplos de valores de rango son como sigue:

```
SELECT intspan '[1, 3)';
SELECT floatspan '[1.5, 3.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-03 09:30:00]';
-- Rangos instantáneos
SELECT intspan '[1, 1]';
SELECT floatspan '[1.5, 1.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
-- Rango erróneo: límites inválidos
SELECT tstzspan '[2001-01-01 08:10:00, 2001-01-01 08:00:00]';
-- Rango erróneo: rango vacío
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
```

Los valores de `intspan`, `bigintspan` y `datespan` son convertidos en *forma normal* para que los valores equivalentes tengan representaciones idénticas. En la representación canónica de estos tipos, el límite inferior es inclusivo y el límite superior es exclusivo, como se muestra en los siguientes ejemplos:

```
SELECT intspan '[1, 1]';
-- [1, 2)
SELECT bigintspan '(1, 3]';
-- [2, 4)
SELECT datespan '[2001-01-01, 2001-01-03]';
-- [2001-01-01, 2001-01-04)
```

Un valor de un tipo de conjunto de rangos representa un conjunto *ordenado* de valores de rango *disjuntos*. Un valor de conjunto de rangos debe contener al menos un elemento, en cuyo caso corresponde a un único valor de rango. Ejemplos de valores conjunto de rangos son los siguientes:

```
SELECT floatspanset '{{[8.1, 8.5],[9.2, 9.4]}}';
-- Singleton spanset
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00]}}';
-- Erroneous spanset: unordered elements
SELECT intspanset '{{[3,4],[1,2]}}';
-- Erroneous spanset: overlapping elements
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
[2001-01-01 08:05:00, 2001-01-01 08:15:00]}}';
```

Los valores de los tipos conjunto de rangos son convertidos en *forma normal* de modo que los valores equivalentes tengan representaciones idénticas. Para ello, los valores de rango consecutivos que son adyacentes se fusionan cuando es posible. Ejemplos de transformación a forma normal son los siguientes:

```
SELECT intspanset '{{[1,2],[3,4]}}';
-- {[1, 5)}
SELECT floatspanset '{{[1.5,2.5],[2.5,4.5]}}';
-- {[1.5, 4.5]}
SELECT tstzspanset '{{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
[2001-01-01 08:10:00, 2001-01-01 08:10:00], (2001-01-01 08:10:00, 2001-01-01 08:20:00]}}';
-- {[2001-01-01 08:00:00+00,2001-01-01 08:20:00+00]}
```

Damos a continuación las funciones de entrada y salida de tipos de conjunto y de rango en formato textual (Well-Known Text o WKT) y binario (Well-Known Binary o WKB). El formato de salida predeterminado de todos los tipos de conjuntos y de rango es el formato de texto conocido. La función `asText` que se da a continuación permite determinar la salida de valores de punto flotante.

- Devuelve la representación textual conocida (Well-Known Text o WKT)

`asText({floatset, floatspans}, maxdecdigits=15) → text`

El argumento `maxdecdigits` se puede utilizar para definir el número máximo de decimales para la salida de los valores de coma flotante (por defecto 15).

```
SELECT asText(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT asText(floatspanset ' {[1.55,2.55],[4,5]} ',0);
-- {[2, 3], [4, 5]}
```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)

`asBinary({set, spans}, endian text="") → bytea`

`asHexWKB({set, spans}, endian text="") → text`

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(dateset '{2001-01-01, 2001-01-03}');
-- \x010500010200000006e01000070010000
SELECT asBinary(intspan '[1, 3]');
-- \x0113000101000000030000000
SELECT asBinary(floatspanset ' {[1, 2], [4, 5]} ', 'XDR');
-- \x00000e00000002033ff000000000000040000000000000034010000000000004014000000000000
SELECT asHexWKB(dateset '{2001-01-01, 2001-01-03}');
-- 010500010200000006E01000070010000
SELECT asHexWKB(intspan '[1, 3]');
-- 0113000101000000030000000
SELECT asHexWKB(floatspanset ' {[1, 2], [4, 5]} ', 'XDR');
-- 00000E00000002033FF000000000000040000000000000034010000000000004014000000000000
```

- Devuelve la representación binaria extendida conocida (Extended Well-Known Binary o EWKB) o la representación hexadecimal binaria extendida conocida (HexEWKB) de un conjunto espacial

`asEWKB({geomset, geogset, cbuffer, npointset, poseset, posechainset, pcpointset, pcpatchset}, text="") → bytea`

`asHexEWKB({geomset, geogset, cbuffer, npointset, poseset, posechainset, pcpointset, pcpatchset}, text="") → text`

Para `geomset`, `geogset`, `cbuffer`, `npointset`, `poseset` y `posechainset` el SRID (cuando se conoce) se incluye una única vez en la cabecera del conjunto: `asEWKB` y `asHexEWKB` lo incluyen, mientras que `asBinary` y `asHexWKB` devuelven la representación binaria conocida (WKB) sin él, siguiendo la convención OGC y el `asBinary/asHexWKB` de los tipos base `cbuffer`, `npoint` y `pose`. Para `pcpointset` y `pcpatchset` el esquema, y su SRID, se resuelve fuera de banda a partir del catálogo `pointcloud_formats` en lugar de incluirse en el valor, por lo que las cuatro funciones siempre devuelven los mismos bytes para estos dos tipos.

```
SELECT asBinary(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b00010100000001010000000000000000f03f000000000000f03f
SELECT asHexWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B00010100000001010000000000000000F03F000000000000F03F
SELECT asEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b0041e6100000010000000101000020e61000000000000000f03f000000000000f03f
SELECT asHexEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B0041E6100000010000000101000020E61000000000000000F03F000000000000F03F
SELECT asEWKB(cbuffer 'SRID=4326;{"Cbuffer(Point(1 1),0.5)"}');
-- \x013a0041e6100000010000000000000000f03f000000000000f03f000000000000e03f
SELECT asHexEWKB(npoinset '{"NPoint(1,0.5)"}');
-- 01310041E61000000100000041E610000001000000000000000000000000E03F
```

- Entrar a partir de la representación binaria conocida (WKB)

```
settypeFromBinary(bytea) → set
spantypeFromBinary(bytea) → span
spansettypeFromBinary(bytea) → spanset
```

Hay una función por tipo de conjunto o de rango, el nombre de la función tiene como prefijo el nombre del tipo.

```
SELECT datesetFromBinary('\x010500010200000006e01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromBinary('\x011300010100000003000000');
-- [1, 3)
SELECT floatspansetFromBinary(
  '\x00000e00000002033ff000000000000040000000000000034010000000000004014000000000000');
-- {[1, 2], [4, 5]}
```

- Entrar a partir de la representación hexadecimal binaria conocida (HexWKB)

```
settypeFromHexWKB(text) → set
spantypeFromHexWKB(text) → span
spansettypeFromHexWKB(text) → spanset
```

Hay una función por tipo de conjunto o de rango, el nombre de la función tiene como prefijo el nombre del tipo.

```
SELECT datesetFromHexWKB('010500010200000006E01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromHexWKB('011300010100000003000000');
-- [1, 3)
SELECT floatspanFromHexWKB('01060001000000000000F83F0000000000000440');
-- [1.5, 2.5)
SELECT floatspansetFromHexWKB(
  '00000E00000002033FF000000000000040000000000000034010000000000004014000000000000');
-- {[1, 2], [4, 5]}
```

2.2. Constructores

La función constructora para los tipos de conjunto tiene un único argumento que es una matriz de valores del tipo base correspondiente. Los valores deben estar ordenados y no pueden tener nulos ni duplicados.

- Constructor para tipos de conjunto

```
set(base[]) → set
```

```
SELECT set(ARRAY['highway', 'primary', 'secondary']);
-- {"highway", "primary", "secondary"}
SELECT set(ARRAY[timestampz '2001-01-01 08:00:00', '2001-01-03 09:30:00']);
-- {2001-01-01 08:00:00+00, 2001-01-03 09:30:00+00}
```

Los tipos de rango unitarios tienen una función constructora que acepta cuatro argumentos, donde los dos últimos son opcionales. Los primeros dos argumentos especifican, respectivamente, el límite inferior y el superior, y los dos últimos argumentos son valores booleanos que indican, respectivamente, si los límites inferior y el superior son inclusivos o no. Se supone que los dos últimos argumentos son, respectivamente, verdadero y falso si no se especifican. Nótese que los rangos de enteros se transforman en *forma normal*, es decir, con límite inferior inclusivo y límite superior exclusivo.

- Constructor para tipos de rango

```
span(lower base, upper base, leftInc bool=true, rightInc bool=false) → span
```

```
SELECT span(20.5, 25);
-- [20.5, 25)
SELECT span(20, 25, false, true);
-- [21, 26)
SELECT span(timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00', false, true);
-- (2001-01-01 08:00:00, 2001-01-03 09:30:00]
```

La función constructora para los tipos de conjuntos de rangos tiene un solo argumento que es una matriz de rangos del mismo subtipo.

- Constructor for conjunto de rangos

`spanset(span[]) → spanset`

```
SELECT spanset(ARRAY[intspan '[10,12]', '[13,15]']);
-- {[10, 16)}
SELECT spanset(ARRAY[floatspan '[10.5,12.5]', '[13.5,15.5]']);
-- {[10.5, 12.5], [13.5, 15.5]}
SELECT spanset(ARRAY[tstzspan '[2001-01-01 08:00, 2001-01-01 08:10]',
                              '[2001-01-01 08:20, 2001-01-01 08:40]']);
-- {[2001-01-01 08:00, 2001-01-01 08:10], [2001-01-01 08:20, 2001-01-01 08:40]};
```

2.3. Conversión de tipos

Los valores de los tipos de conjunto y de rango se pueden convertir entre sí o convertirse a tipos de rango de PostgreSQL y desde ellos mediante la función `CAST` o mediante la notación `::`.

- Convertir un valor de base en un valor de conjunto, rango o conjunto de rangos

`base::{set, span, spanset}`

`set(base) → set`

`span(base) → span`

`spanset(base) → spanset`

```
SELECT CAST(timestamptz '2001-01-01 08:00:00' AS tstzset);
-- {2001-01-01 08:00:00}
SELECT timestamptz '2001-01-01 08:00:00'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:00:00]
SELECT spanset(timestamptz '2001-01-01 08:00:00');
-- {[2001-01-01 08:00:00, 2001-01-01 08:00:00]}
```

- Convertir un valor de conjunto en un valor de conjunto de rangos

`set::spanset`

`spanset(set) → spanset`

```
SELECT spanset(tstzset '{2001-01-01 08:00:00, 2001-01-01 08:15:00,
                          2001-01-01 08:25:00}');
/* {[2001-01-01 08:00:00, 2001-01-01 08:00:00],
    [2001-01-01 08:15:00, 2001-01-01 08:15:00],
    [2001-01-01 08:25:00, 2001-01-01 08:25:00]} */
```

- Convertir un valor de rango a un valor de conjunto de rangos

`span::spanset`

`spanset(span) → spanset`

```
SELECT floatspan '[1.5,2.5]':::floatspanset;
-- {[1.5, 2.5]}
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':::tstzspanset;
-- {[2001-01-01 08:00:00, 2001-01-01 08:30:00]}
```

- Convertir un conjunto de valores o un conjunto de rangos a un rango, ignorando las posibles brechas de tiempo

```
{set, spanset}::span
span({set, spanset}) → span
```

```
SELECT span(dateset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- [2001-01-01, 2001-01-06]
SELECT span(tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}');
-- [2001-01-01, 2001-01-04]
```

- Convertir un valor de rango en MobilityDB hacia y desde un valor de rango de PostgreSQL

```
span::range
range::span
range(span) → range
span(range) → span
```

Nótese que los valores de rango en PostgreSQL aceptan rangos vacíos y rangos con límites infinitos, que no están permitidos como valores de rango en MobilityDB

```
SELECT intspan '[10, 20]':::int4range;
-- [10,20]
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':::tstzrange;
-- ["2001-01-01 08:00:00","2001-01-01 08:30:00")
SELECT int4range '[10, 20]':::intspan;
-- [10,20]
SELECT int4range 'empty':::intspan;
-- ERROR: Range cannot be empty
SELECT int4range '[10,)':::intspan;
-- ERROR: Range bounds cannot be infinite
SELECT tstzrange '[2001-01-01 08:00:00, 2001-01-01 08:30:00]':::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:30:00]
```

- Convertir un valor de conjunto de rangos de MobilityDB hacia y desde un valor de multirango de PostgreSQL

```
spanset::multirange
multirange::spanset
multirange(spanset) → multirange
spanset(multirange) → spanset
```

```
SELECT intspanset '{{[1,2],[4,5]}}':::int4multirange;
-- {[1,3],[4,6]}
SELECT tstzspanset '{{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}}':::tstzmultirange;
-- {[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}
SELECT int4multirange '{{[1,2],[4,5]}}':::intspanset;
-- {[1, 3],[4, 6]}
SELECT tstzmultirange '{{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}}':::tstzspanset;
-- {[2001-01-01, 2001-01-02],[2001-01-04, 2001-01-05]}
```

2.4. Accesores

- Devuelve el tamaño de la memoria en bytes

`memSize({set, spanset}) → integer`

```
SELECT memSize(tstzset '{2001-01-01, 2001-01-02, 2001-01-03}');
-- 48
SELECT memSize(tstzspanset ' {[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04],
[2001-01-05, 2001-01-06]} ');
-- 112
```

- Devuelve el límite inferior o superior

`lower(spans) → base`

`upper(spans) → base`

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
-- 1
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT upper(intspanset '{[1,2],[4,5]}');
-- 6
SELECT lower(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
-- 1
```

```
SELECT upper(floatspan '[20.5, 25.3]');
-- 25.3
SELECT upper(tstzspan '[2001-01-01, 2001-01-05]');
-- 2001-01-05
```

- ¿Es el límite inferior or superior inclusivo?

`lowerInc(spans) → boolean`

`upperInc(spans) → boolean`

```
SELECT lowerInc(datespan '[2001-01-01, 2001-01-05]');
-- true
SELECT lowerInc(intspanset '{[1,2],[4,5]}');
-- true
```

```
SELECT lowerInc(tstzspan '[2001-01-01, 2001-01-05]');
-- true
SELECT upperInc(intspanset '{[1,2],[4,5]}');
-- false
SELECT upper(floatspan '[20.5, 25.3]');
-- true
SELECT upperInc(tstzspan '[2001-01-01, 2001-01-05]');
-- false
```

- Devuelve el ancho del rango como un número de punto flotante

`width(numspan) → float`

`width(numspanset, boundspan=false) → float`

Se puede establecer a verdadero un parámetro adicional para calcular el ancho del rango delimitador, ignorando así las posibles brechas de valores

```
SELECT width(floatspan '[1, 3]');
-- 2
SELECT width(intspanset '{{[1,3],[5,7]}}');
-- 4
SELECT width(intspanset '{{[1,3],[5,7]}}', true);
-- 6
```

- Devuelve el rango de tiempo

`duration({datespan,tstzspan}) → interval`

`duration({datespanset,tstzspanset},boundspan bool=false) → interval`

Se puede establecer a verdadero un parámetro adicional para calcular la duración en el rango delimitador, ignorando así las posibles brechas de tiempo

```
SELECT duration(datespan '[2001-01-01, 2001-01-03]');
-- 2 days
SELECT duration(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}');
-- 3 days
SELECT duration(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}', true);
-- 4 days
```

- Devuelve el número de valores o los valores

`numValues(set) → integer`

`getValues(set) → base[]`

```
SELECT numValues(intset '{1,3,5,7}');
-- 4
SELECT getValues(tstzset '{{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}}');
-- {"2001-01-01", "2001-01-03", "2001-01-05", "2001-01-07"}
```

- Devuelve el valor inicial, final o enésimo

`startValue(set) → base`

`endValue(set) → base`

`valueN(set,integer) → base`

```
SELECT startValue(intset '{1,3,5,7}');
-- 1
SELECT endValue(dateset '{{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}}');
-- 2001-01-07
SELECT valueN(floatset '{1,3,5,7}',2);
-- 3
```

- Devuelve el número de rangos o los rangos

`numSpans(spanset) → integer`

`spans(spanset) → span[]`

```
SELECT numSpans(intspanset '{{[1,3],[4,5],[6,7]}}');
-- 3
SELECT numSpans(datespanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05],
[2001-01-06, 2001-01-07]}}');
-- 3
SELECT spans(floatspanset '{{[1,3],[4,4],[6,7]}}');
-- {"[1,3)","[4,4)","[6,7)"}
SELECT spans(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06]}}');
-- {"[2001-01-01,2001-01-03)", "[2001-01-04,2001-01-04)", "[2001-01-05,2001-01-06)"}
```


- Devuelve el rango inicial, final o enésimo

`startSpan(spanset) → span`

`endSpan(spanset) → span`

`spanN(spanset, integer) → span`

```
SELECT startSpan(intspanset '{[1,3],[4,5],[6,7]}');
-- [1,3]
SELECT startSpan(datespanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05],
[2001-01-06, 2001-01-07]}');
-- [2001-01-01,2001-01-03]
```

```
SELECT endSpan(floatspanset '{[1,3],[4,4],[6,7]}');
-- [6,7]
SELECT endSpan(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06]}');
-- [2001-01-05,2001-01-06]
```

```
SELECT spanN(floatspanset '{[1,3],[4,4],[6,7]}',2);
-- [4,4]
SELECT spanN(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-04],
[2001-01-05, 2001-01-06]}', 2);
-- [2001-01-04,2001-01-04]
```

- Devuelve el (number of) different dates

`numDates(datespanset) → integer`

`dates(datespanset) → dataset`

```
SELECT numDates(datespanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}');
-- 4
SELECT dates(datespanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}');
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04}
```

- Devuelve el start, end, or n-th date

`startDate(datespanset) → date`

`endDate(datespanset) → date`

`dateN(datespanset, integer) → date`

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startDate(datespanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}');
-- 2001-01-01
SELECT endDate(datespanset '{[2001-01-01, 2001-01-03], (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT dateN(datespanset '{[2001-01-01, 2001-01-03], (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

- Devuelve el número o las marcas de tiempo diferentes

`numTimestamps(tstzspanset) → integer`

`timestamps(tstzspanset) → tstzset`

```
SELECT numTimestamps(tstzspanset '{[2001-01-01, 2001-01-03], (2001-01-03, 2001-01-05)}');
-- 3
SELECT timestamps(tstzspanset '{[2001-01-01, 2001-01-03], (2001-01-03, 2001-01-05)}');
-- {"2001-01-01 00:00:00", "2001-01-03 00:00:00", "2001-01-05 00:00:00"}
```

- Devuelve la marca de tiempo inicial, final, o enésima

`startTimestamp(tstzspanset) → timestamptz`

`endTimestamp(tstzspanset) → timestamptz`

`timestampN(tstzspanset, integer) → timestamptz`

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startTimestamp(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 2001-01-01
SELECT endTimestamp(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT timestampN(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

2.5. Transformaciones

- Expandir o reducir los límites de un rango con un valor o un intervalo de tiempo

`expand(numspan, base) → numspan`

`expand(tstzspan, interval) → tstzspan`

La función devuelve NULL si el valor o intervalo dado como segundo argumento es negativo y el rango resultante de desplazar los límites con el argumento está vacío.

```
SELECT expand(floatspan '[1, 3]', 1);
-- [0, 4]
SELECT expand(floatspan '[1, 3]', -1);
-- [2, 2]
SELECT expand(floatspan '[1, 3]', -1);
-- NULL
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2000-12-31, 2001-01-04]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-1 day');
-- [2001-01-02, 2001-01-02]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-2 day');
-- NULL
```

- Desplazar con un valor o rango de tiempo

`shift(numbers, base) → numbers`

`shift(dates, integer) → dates`

`shift(times, interval) → times`

```
SELECT shift(dateset '{2001-01-01, 2001-01-03, 2001-01-05}', 1);
-- {2001-01-02, 2001-01-04, 2001-01-06}
SELECT shift(intspan '[1, 4]', -1);
-- [0, 3]
SELECT shift(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2001-01-02, 2001-01-04]
SELECT shift(floatspanset '{{[1, 2], [3, 4]}}', -1);
-- {[0, 1], [2, 3]}
SELECT shift(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}',
interval '1 day');
-- {[2001-01-02, 2001-01-04], [2001-01-05, 2001-01-06]}
```

- Escalar con un valor o un rango de tiempo

`scale(numbers, base) → numbers`

`scale(dates, integer) → dates`

`scale(times,interval) → times`

Si el ancho o el lapso de tiempo del valor de entrada es cero (por ejemplo, para un conjunto de marcas de tiempo único), el resultado es el valor de entrada. El valor o rango de tiempo dado debe ser estrictamente mayor que cero.

```
SELECT scale(tstzset '{2001-01-01}', interval '1 day');
-- {2001-01-01}
SELECT scale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '2 days');
-- {2001-01-01, 2001-01-02, 2001-01-03}
SELECT scale(intspan '[1, 4]', 4);
-- [1, 6)
SELECT scale(datespan '[2001-01-01, 2001-01-04)', 4);
-- [2001-01-01, 2001-01-06)
SELECT scale(tstzspan '[2001-01-01, 2001-01-03]', '1 day');
-- [2001-01-01, 2001-01-02]
SELECT scale(floatspanset '{{[1, 2], [3, 4]}}', 6);
-- {[1, 3], [5, 7]}
SELECT scale(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}', '1 day');
-- {[2001-01-01 00:00:00, 2001-01-01 12:00:00],
  [2001-01-01 18:00:00, 2001-01-02 00:00:00]}
SELECT scale(tstzset '{2001-01-01}', '-1 day');
-- ERROR: The duration must be a positive interval: -1 days
```

■ Desplazar y escalar con los valores o rangos de tiempo

`shiftScale(numbers,base,base) → numbers`

`shiftScale(dates,integer,integer) → dates`

`shiftScale(times,interval,interval) → times`

Estas funciones combinan las funciones **shift** y **scale**.

```
SELECT shiftScale(tstzset '{2001-01-01}', '1 day', '1 day');
-- {2001-01-02}
SELECT shiftScale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '1 day', '2 days');
-- {2001-01-02, 2001-01-03, 2001-01-04}
SELECT shiftScale(intspan '[1, 4]', -1, 4);
-- [0, 5)
SELECT shiftScale(datespan '[2001-01-01, 2001-01-04)', -1, 4);
-- [2001-12-31, 2001-01-05)
SELECT shiftScale(tstzspan '[2001-01-01, 2001-01-03]', '1 day', '1 day');
-- [2001-01-02, 2001-01-03]
SELECT shiftScale(floatspanset '{{[1, 2], [3, 4]}}', -1, 6);
-- {[0, 2], [4, 6]}
SELECT shiftScale(tstzspanset '{{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}}',
  '1 day', '1 day');
/* {[2001-01-02 00:00:00, 2001-01-02 12:00:00],
  [2001-01-02 18:00:00, 2001-01-03 00:00:00]} */
```

■ Redondear al entero inferior o superior

`floor({floatset,floatspans}) → {floatset,floatspans}`

`ceil({floatset,floatspans}) → {floatset,floatspans}`

```
SELECT floor(floatset '{1.5,2.5}');
-- {1, 2}
SELECT ceil(floatspan '[1.5,2.5]');
-- [2, 3)
SELECT floor(floatspan '(1.5, 1.6)');
-- [1, 1]
SELECT ceil(floatspanset '{{[1.5, 2.5],[3.5,4.5]}}');
-- {[2, 3], [4, 5]}
```

■ Redondear a un número de decimales

`round({floatset,floatspans},integer=0) → {floatset,floatspans}`

```
SELECT round(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT round(floatspan '[1.123456789,2.123456789]', 3);
-- [1.123,2.123]
SELECT round(floatspan '[1.123456789, inf]', 3);
-- [1.123,Infinity)
SELECT round(floatspanset '{{1.123456789, 2.123456789],[3.123456789,4.123456789]}' , 3);
-- {[1.123, 2.123], [3.123, 4.123]}
```

■ Convertir a grados o radianes

`degrees({floatset,floatspans}, normalize=false) → {floatset,floatspans}`

`radians({floatset,floatspans}) → {floatset,floatspans}`

El parámetro adicional en la función `degrees` puede ser utilizado para normalizar los valores entre 0 y 360 grados.

```
SELECT round(degrees(floatset '{0, 0.5, 0.7, 1.0}', true), 3);
-- {0, 28.648, 40.107, 57.296}
SELECT round(radians(floatspanset '{{[0, 45], [90, 135]}}', 3);
-- {[0, 0.785], [1.571, 2.356]}
```

■ Transformar en minúsculas, mayúsculas o initcap

`lower(textset) → textset`

`upper(textset) → textset`

`initcap(textset) → textset`

```
SELECT lower(textset '{"AAA", "BBB", "CCC"}');
-- {"aaa", "bbb", "ccc"}
SELECT upper(textset '{"aaa", "bbb", "ccc"}');
-- {"AAA", "BBB", "CCC"}
SELECT initcap(textset '{"aaa", "bbb", "ccc"}');
-- {"Aaa", "Bbb", "Ccc"}
```

■ Concatenación de texto

`{text,textset} || {text,textset} → textset`

```
SELECT textset '{aaa, bbb}' || text 'XX';
-- {"aaaXX", "bbbXX"}
SELECT text 'XX' || textset '{aaa, bbb}';
-- {"XXaaa", "XXbbb"}
```

■ Establecer la precisión temporal del valor de tiempo al rango con respecto al origen

`tprecision(times,interval,origin timestamptz='2000-01-03') → times`

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000.

```
SELECT tprecision(timestamptz '2001-12-03', '30 days');
-- 2001-11-23
SELECT tprecision(timestamptz '2001-12-03', '30 days', '2001-12-01');
-- 2001-12-01
SELECT tprecision(tstzset '{2001-01-01 08:00, 2001-01-01 08:10, 2001-01-01 09:00,
  2001-01-01 09:10}', '1 hour');
-- {"2001-01-01 08:00:00+01", "2001-01-01 09:00:00+01"}
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-01 09:00]', '1 day');
-- [2001-12-01, 2001-12-02)
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-15 09:00]', '1 day');
-- [2001-12-01, 2001-12-16)
```

```
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
[2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02]}
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
[2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02]}
```

2.6. Sistema de referencia espacial

- Devuelve o especifica el identificador de referencia espacial

SRID(spatialset) → integer

setSRID(spatialset) → spatialset

```
SELECT SRID(geomset '{"Point(1 1)", "Point(2 2)}');
-- 0
SELECT SRID(geogset '{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 4326
SELECT SRID(geomset 'SRID=5676;{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 5676
SELECT asEWKT(setSRID(geomset '{Point(1 1), Point(2 2)}', 5676));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(setSRID(poseset '{"Pose(Point(1 1),1)", "Pose(Point(2 2),3)}', 5676));
-- SRID=5676;{"Pose(POINT(2 2),3)", "Pose(POINT(1 1),1)"}
SELECT SRID(cbuffer(geomset 'SRID=4326;{"Cbuffer(Point(1 1),0.5)",
"Cbuffer(Point(2 2),1.0)}');
-- 4326
SELECT asEWKT(setSRID(cbuffer(geomset '{"Cbuffer(Point(1 1),0.5)",
"Cbuffer(Point(2 2),1.0)}', 3812));
-- SRID=3812;{"Cbuffer(POINT(1 1),0.5)", "Cbuffer(POINT(2 2),1)"}

```

- Transformar a una referencia espacial diferente

transform(spatialset,to_srid integer) → spatialset

transformPipeline(spatialset,pipeline text,to_srid integer,is_forward bool=true) → spatialset

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el conjunto tiene un SRID desconocido (representado por 0). La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas en el siguiente formato:

urn:ogc:def:coordinateOperation:AUTHORITY::CODE

El SRID del conjunto de entrada se ignora y el SRID de la conjunto de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(geomset 'SRID=4326;{Point(2.340088 49.400250),
Point(6.575317 51.553167)}', 3812), 6);
-- SRID=3812;{"POINT(502773.429981 511805.120402)", "POINT(803028.908265 751590.742629)"}
WITH test(geomset, pipeline) AS (
  SELECT geogset 'SRID=4326;{"Point(4.3525 50.846667 100.0)",
    "Point(-0.1275 51.507222 100.0)}"',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(geomset, pipeline, 4326), pipeline,
    4326, false), 6)
FROM test;
-- SRID=4326;{"POINT Z (4.3525 50.846667 100)", "POINT Z (-0.1275 51.507222 100)"}

```

2.7. Operaciones de conjuntos

Los tipos de conjunto y de rango tienen operadores de conjuntos asociados, a saber, unión, diferencia e intersección, que se representan, respectivamente, por $+$, $-$ y $*$. Los operadores de conjunto para los tipos de rango y tiempo se dan a continuación.

■ Unión, diferencia o intersección de conjuntos o de rangos

set $\{+, -, *\}$ set $\rightarrow$ set

spans $\{+, -, *\}$ spans $\rightarrow$ spans

```
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' +
  dateset '{2001-01-03, 2001-01-06}';
-- {2001-01-01, 2001-01-03, 2001-01-05, 2001-01-06}
SELECT intspan '[1, 3]' + intspan '[3, 5]';
-- [1, 5]
SELECT floatspan '[1, 3]' + floatspan '[4, 5]';
-- {[1, 3), [4, 5)}
SELECT tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}' +
  tstzspan '[2001-01-03, 2001-01-04)';
-- {[2001-01-01, 2001-01-05)}
```

```
SELECT intset '{1, 3, 5}' - intset '{3, 6}';
-- {1, 5}
SELECT datespan '[2001-01-01, 2001-01-05)' - datespan '[2001-01-03, 2001-01-07)';
-- {[2001-01-01, 2001-01-03)}
SELECT floatspan '[1, 5]' - floatspan '[3, 4]';
-- {[1, 3), (4, 5]}
SELECT tstzspanset '{[2001-01-01, 2001-01-06], [2001-01-07, 2001-01-10]}' -
  tstzspanset '{[2001-01-02, 2001-01-03], [2001-01-04, 2001-01-05],
  [2001-01-08, 2001-01-09]}';
/* {[2001-01-01,2001-01-02), (2001-01-03,2001-01-04), (2001-01-05,2001-01-06],
  [2001-01-07,2001-01-08), (2001-01-09,2001-01-10)} */
```

```
SELECT tstzset '{2001-01-01, 2001-01-03}' * tstzset '{2001-01-03, 2001-01-05}';
-- {2001-01-03}
SELECT intspan '[1, 5]' * intspan '[3, 6]';
-- [3, 5]
SELECT floatspanset '{[1, 5],[6, 8]}' * floatspan '[1, 6]';
-- {[3, 5]}
SELECT tstzspan '[2001-01-01, 2001-01-05)' * tstzspan '[2001-01-03, 2001-01-07)';
-- [2001-01-03, 2001-01-05)
```

2.8. Operaciones de cuadro delimitador

2.8.1. Operaciones topológicas

A continuación se presentan los operadores topológicos disponibles para los tipos de conjunto y de rango.

Un valor, el rango que contiene únicamente ese valor y el conjunto de rangos que contiene únicamente ese rango denotan el mismo conjunto, al igual que un rango y el conjunto de rangos que contiene únicamente a él. Todas las operaciones dan la misma respuesta para todas estas escrituras.

Un conjunto de rangos es una unión de rangos disjuntos. Las operaciones $\&\&$, $\&>$, $\<\&$ y $\sim=$ responden sobre el valor mismo, por lo que un valor que cae en un hueco de un conjunto de rangos no está contenido en él. La operación $-|-$ y las operaciones de posición de la sección siguiente responden sobre la *caja delimitadora* del valor, el rango más pequeño que lo contiene: los huecos de un conjunto de rangos no son fronteras de él.

- ¿Se superponen los valores (tienen valores en común)?

{set, spans} && {set, spans} → boolean

```
SELECT intset '{1, 3}' && intset '{2, 3, 4}';
-- true
SELECT floatspan '[1, 3]' && floatspan '[3, 4]';
-- false
SELECT floatspanset '{{[1, 5), [6, 8)}}

```

- ¿Contiene el primer valor el segundo?

{set, spans} @> {base, set, span} → boolean

```
SELECT floatset '{1.5, 2.5}' @> 2.5;
-- true
SELECT tstzspan '[2001-01-01, 2001-05-01)' @> timestampz '2001-02-01';
-- true
SELECT floatspanset '{{[1, 2), (2, 3)}}

```

- ¿Está el primer valor contenido en el segundo?

{base, set, spans} <@ {set, spans} → boolean

```
SELECT timestampz '2001-01-10' <@ tstzspan '[2001-01-01, 2001-05-01)';
-- true
SELECT floatspan '[2, 5]' <@ floatspan '[1, 5)';
-- false
SELECT floatspanset '{{[1, 2), [3, 4)}}

```

- ¿Es el primer valor adyacente al segundo? Dos valores son adyacentes cuando se tocan sin solaparse, es decir, cuando comparten una frontera y no tienen ningún valor en común en ella. Para los tipos base continuos float y timestampz, un límite compartido es ese contacto. Para los tipos base discretos int, bigint y date, cuyos rangos se almacenan en forma normal, significa valores consecutivos.

spans -|- spans → boolean

```
SELECT intspan '[2, 6)' -|- intspan '[6, 7)';
-- true
SELECT intspan '[1, 5]' -|- intspan '[6, 10]';
-- true
SELECT intspan '[1, 5]' -|- intspan '[5, 10]';
-- false
SELECT floatspan '[2, 5)' -|- floatspan '(5, 6)';
-- true
SELECT floatspan '[1, 5]' -|- floatspan '[5, 9]';
-- true
SELECT floatspanset '{{[2, 3), [4, 5)}}

```

Un conjunto de rangos responde como su caja delimitadora, y las tres escrituras de un valor coinciden:

```
SELECT intspanset '{{[1, 3), [5, 8), [10, 12)}}

```

```
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)]}' -|- intspanset '{[3,3]}';
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)]}' -|- 12;
-- true
```

2.8.2. Operaciones de posición

Los operadores de posición disponibles para los tipos de conjunto y de rango se dan a continuación. Observe que los operadores para tipos de tiempo tienen un # adicional para distinguirlos de los operadores para tipos de números.

- ¿Está el primer valor estrictamente a la izquierda del segundo?

numbers << numbers → boolean

times <<# times → boolean

```
SELECT intspan '[15, 20)' << 20;
-- true
SELECT intspanset '{[15, 17], [18, 20)}' << 20;
-- true
SELECT floatspan '[15, 20)' << floatspan '(15, 20)';
-- false
SELECT dateset '{2001-01-01, 2001-01-02}' <<# dateset '{2001-01-03, 2001-01-05}';
-- true
```

- ¿Está el primer valor estrictamente a la derecha del segundo?

numbers >> numbers → boolean

times #>> times → boolean

```
SELECT intspan '[15, 20)' >> 10;
-- true
SELECT floatspan '[15, 20)' >> floatspan '[5, 10]';
-- true
SELECT floatspanset '{[15, 17], [18, 20)}' >> floatspan '[5, 10]';
-- true
SELECT tstzspan '[2001-01-04, 2001-01-05)' #>>
  tstzspanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}';
-- true
```

- ¿No está el primer valor a la derecha del segundo?

numbers &< numbers → boolean

times &<# times → boolean

```
SELECT intspan '[15, 20)' &< 18;
-- false
SELECT intspanset '{[15, 16], [17, 18)}' &< 18;
-- true
SELECT floatspan '[15, 20)' &< floatspan '[10, 20]';
-- true
SELECT dateset '{2001-01-02, 2001-01-05}' &<# dateset '{2001-01-01, 2001-01-04}';
-- false
```

- ¿No está el primer valor a la izquierda del segundo?

numbers &> numbers → boolean

times #&> times → boolean


```

SELECT intspan '[15, 20]' &> 30;
-- true
SELECT floatspan '[1, 6]' &> floatspan '(1, 3)';
-- false
SELECT floatspanset '{{[1, 2],[3, 4]}}' &> floatspan '(1, 3)';
-- false
SELECT timestamp '2001-01-01' #&> tstzspan '[2001-01-01, 2001-01-05]';
-- true

```

2.8.3. Operaciones de división

Al crear índices para tipos de conjuntos o conjunto de rangos, lo que se almacena en el índice no es el valor real, sino un cuadro delimitador que *representa* el valor. En este caso, el índice proporcionará una lista de valores candidatos que *pueden* satisfacer el predicado de la consulta, y se necesita un segundo paso para filtrar los valores candidatos calculando el predicado de la consulta sobre los valores reales.

Sin embargo, cuando los cuadros delimitadores tienen un gran espacio vacío no cubierto por los valores reales, el índice generará muchos valores candidatos que no satisfacen el predicado de la consulta, lo que reduce la eficiencia del índice. En estas situaciones, puede ser mejor representar un valor no con un cuadro delimitador *único*, sino con *múltiples* cuadros delimitadores. Esto aumenta considerablemente la eficiencia del índice, siempre que el índice sea capaz de gestionar múltiples cuadros delimitadores por valor. Las siguientes funciones se utilizan para generar múltiples rangos a partir de un único valor de conjunto o conjunto de rangos.

- Devuelve una matriz de N rangos obtenida fusionando los elementos de un conjunto o los rangos de un conjunto de rangos

```
splitNSpans(set, integer) → span[]
```

```
splitNSpans(spanset, integer) → span[]
```

El último argumento especifica el número de rangos de salida. Si el número de elementos o rangos de entrada es menor que el número especificado, la matriz resultante tendrá un rango por elemento o rango de entrada. De lo contrario, el número especificado de rangos de salida se obtendrá fusionando elementos o rangos de entrada consecutivos.

```

SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 11)"} */
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 3);
-- {"[1, 5)", "[5, 8)", "[8, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 3)", "[3, 5)", "[5, 7)", "[7, 9)", "[9, 10)", "[10, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
/* {"[1, 2)", "[2, 3)", "[3, 4)", "[4, 5)", "[5, 6)", "[6, 7)", "[7, 8)", "[8, 9)",
    "[9, 10)", "[10, 11)"} */

```

```

SELECT splitNSpans(intspanset '{{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10]}}');
-- {"[1, 2)", "[3, 4)", "[5, 6)", "[7, 8)", "[9, 10)"}
SELECT splitNSpans(floatspanset '{{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10]}}', 3);
-- {"[1, 4)", "[5, 8)", "[9, 10)"}
SELECT splitNSpans(datespanset '{{[2000-01-01, 2000-01-04), [2000-01-05, 2000-01-10]}}', 3);
-- {"[2000-01-01, 2000-01-04)", "[2000-01-05, 2000-01-10)"}

```

- Devuelve una matriz de rangos obtenida fusionando N elementos consecutivos de un conjunto o N rangos consecutivos de un conjunto de rangos

```
splitEachNSpans(set, integer) → span[]
```

```
splitEachNSpans(spanset, integer) → span[]
```

El último argumento especifica el número de elementos de entrada que se fusionan para producir un rango de salida. Si la cantidad de elementos de entrada es menor que el número especificado, la matriz resultante tendrá un rango de salida por elemento. De lo contrario, la cantidad especificada de elementos de entrada consecutivos se fusionará en un único rango de salida en la respuesta. Observe que, a diferencia de la función **splitNSpans**, el número de rangos en el resultado depende del número de elementos o de rangos de entrada.

```

SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 2)","[2, 3)","[3, 4)","[4, 5)","[5, 6)","[6, 7)","[7, 8)","[8, 9)","
   "[9, 10)","[10, 11)"} */
SELECT splitEachNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 6)","[7, 10)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 7)","[7, 11)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
-- {"[1, 11)"}

```

2.9. Operaciones de distancia

El operador de distancia `<->` para los tipos de rango o de tiempo considera el lapso o período delimitador y devuelve la distancia más pequeña entre los dos valores. En el caso de valores de tiempo, el operador devuelve la cantidad de días o la cantidad de segundos entre los dos valores de tiempo. El operador de distancia también se puede usar para búsquedas de vecinos más cercanos utilizando un índice GiST o SP-GiST (ver Sección 2.12).

- Devuelve la distancia mínima

numbers `<->` numbers $\rightarrow$ base

dates `<->` dates $\rightarrow$ integer

times `<->` times $\rightarrow$ float

```

SELECT 3 <-> intspan '[6, 8)';
-- 3
SELECT floatspan '[1, 3]' <-> floatspan '(5.5, 7]';
-- 2.5
SELECT floatspan '[1, 3]' <-> floatspanset '{(5.5, 7],[8, 9]}';
-- 2.5
SELECT tstzspan '[2001-01-02, 2001-01-06)' <-> timestamptz '2001-01-07';
-- 86400
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' <->
  dateset '{2001-01-02, 2001-01-04}';
-- 0

```

2.10. Comparaciones

Los operadores de comparación (`=`, `<`, etc.) requieren que los argumentos izquierdo y derecho sean del mismo tipo. Exceptuando la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real, pero permiten construir índices de árbol B en tipos de rango o de tiempo. Para los valores de rango, los operadores comparan primero el límite inferior y luego el límite superior. Para los valores de conjunto de marcas de tiempo y conjunto de períodos, los operadores comparan primero los períodos delimitadores y, si son iguales, comparan los primeros N instantes o períodos, donde N es el mínimo del número de instantes o períodos que componen ambos valores.

Los operadores de comparación disponibles para los tipos de conjunto y de rango se dan a continuación. Recuerde que los rangos de enteros siempre se representan por su forma canónica.

- Comparaciones tradicionales

set `{=, <>, <, >, <=, >=}` set $\rightarrow$ boolean

spans `{=, <>, <, >, <=, >=}` spans $\rightarrow$ boolean

```

SELECT intspan '[1,3]' = intspan '[1,4)';
-- true
SELECT floatspanset '{[1, 2), [2, 3)}' = floatspanset '{[1, 3)}';

```

```

-- true
SELECT tstzset '{2001-01-01, 2001-01-04}' <> tstzset '{2001-01-01, 2001-01-05}';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-04)' <> tstzspan '[2001-01-03, 2001-01-05)';
-- true
SELECT floatspan '[3, 4]' < floatspan '(3, 4]';
-- true
SELECT intspanset '{{[1,2],[3,4]}}' < intspanset '{{[3, 4]}}';
-- true
SELECT floatspan '[3, 4]' > floatspan '[3, 4)';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-04)' > tstzspan '[2001-01-02, 2001-01-05)';
-- true
SELECT floatspanset '{{[1, 4]}}' <= floatspanset '{{[1, 5), [6, 7]}}';
-- true
SELECT tstzspanset '{{[2001-01-01, 2001-01-04]}}' <=
  tstzspanset '{{[2001-01-01, 2001-01-05), [2001-01-06, 2001-01-07]}}';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-05)' >= tstzspan '[2001-01-03, 2001-01-04)';
-- true
SELECT intspanset '{{[1, 4]}}' >= intspanset '{{[1, 5), [6, 7]}}';
-- false

```

2.11. Agregaciones

Hay varias funciones agregadas definidas para los tipos de conjunto y de rango. Estas se describen a continuación.

- La función `extent` devuelve el rango o período delimitador que engloba un conjunto de valores de rango o de tiempo.
- La unión es una operación muy útil para los tipos de conjunto y de rango. Como hemos visto en la Sección 2.7, podemos calcular la unión de dos valores de conjunto o de rango usando el operador `+`. Sin embargo, también es muy útil tener una versión agregada del operador de unión para combinar un número arbitrario de valores. Las funciones `setUnion` y `spanUnion` se pueden utilizar para este propósito.
- La función `tCount` generaliza la función de agregación tradicional `count`. El conteo temporal se puede utilizar para calcular en cada momento el número de objetos disponibles (por ejemplo, el número de períodos). La función `tCount` devuelve un entero temporal (ver el Capítulo 10). La función tiene dos parámetros opcionales que especifican la granularidad (un `interval`) y el origen del tiempo (un `timestampz`). Cuando se dan estos parámetros, el conteo temporal se calcula en rangos de tiempo de la granularidad dada (ver Sección 9.6)

■ Rango o período delimitador

`extent({set, spans}) → span`

```

WITH spans(r) AS (
  SELECT floatspan '[1, 4)' UNION SELECT floatspan '(5, 8)' UNION
  SELECT floatspan '(7, 9)' )
SELECT extent(r) FROM spans;
-- [1,9)

WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT extent(ts) FROM times;
-- [2001-01-01, 2001-01-06]

WITH periods(ps) AS (
  SELECT tstzspanset '{{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}}' UNION
  SELECT tstzspanset '{{[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06]}}' UNION
  SELECT tstzspanset '{{[2001-01-02, 2001-01-06]}}' )

```

```
SELECT extent(ps) FROM periods;
-- [2001-01-01, 2001-01-06]
```

■ Unión agregada

setUnion({value,set}) → set

spanUnion(spans) → spanset

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT setUnion(ts) FROM times;
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04, 2001-01-05, 2001-01-06}

WITH periods(ps) AS (
  SELECT tstzspanset '[[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]]' UNION
  SELECT tstzspanset '[[2001-01-02, 2001-01-03], [2001-01-05, 2001-01-06]]' UNION
  SELECT tstzspanset '[[2001-01-07, 2001-01-08]]' )
SELECT spanUnion(ps) FROM periods;
-- {[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06], [2001-01-07, 2001-01-08]}
```

■ Conteo temporal

tCount(times) → {tintSeq,tintSeqSet}

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT tCount(ts) FROM times;
-- {2@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-04, 1@2001-01-05, 1@2001-01-06}

WITH periods(ps) AS (
  SELECT tstzspanset '[[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04]]' UNION
  SELECT tstzspanset '[[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06]]' UNION
  SELECT tstzspanset '[[2001-01-02, 2001-01-06))' )
SELECT tCount(ps) FROM periods;
-- {[2@2001-01-01, 3@2001-01-03, 1@2001-01-04, 2@2001-01-05, 2@2001-01-06]}
```

2.12. Indexación

Se pueden crear índices GiST y SP-GiST en columnas de tablas de los tipos de conjunto y de rango. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple. Un ejemplo de creación de un índice GiST en una columna During de tipo tstzspan en una tabla Reservation es como sigue:

```
CREATE TABLE Reservation (ReservationID integer PRIMARY KEY, RoomID integer,
  During tstzspan);
CREATE INDEX Reservation_During_Idx ON Reservation USING GIST(During);
```

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores: =, &&, <@, @>, -|- , <<, >>, &<, &>, <<#, #>>, &<#, #&> y <->.

Además, se pueden crear índices de árbol B para columnas de tabla de un tipo de rango o de tiempo. Para estos tipos de índices, básicamente la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos de rango o de tiempo con los correspondientes operadores <, <=, > y >=, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte del árbol B está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Finalmente, se pueden crear índices hash para columnas de tabla de un tipo de rango o de tiempo. Para estos tipos de índices, la única operación definida es la igualdad.

Capítulo 3

Tipos de cuadro delimitador

A continuación presentamos las funciones y operadores para tipos cuadro delimitador. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo del resultado puede depender del tipo de los argumentos. Para expresar esto, usamos la siguiente notación:

- `box` representa cualquier tipos cuadro delimitador, es decir `tbox` o `stbox`.

3.1. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (WKT) y Well-Known Binary (WKB) del Open Geospatial Consortium para todos los tipos temporales. Presentamos a continuación las funciones de entrada y salida para los tipos de cuadro delimitador.

Un `tbox` se compone de dimensiones de valor numérico y/o de tiempo. Para cada dimensión, se proporciona un rango, es decir, ya sea un `intspan` o un `floatspan` para la dimensión de valor y un `tstzspan` para la dimensión de tiempo. Ejemplos de entrada de valores `tbox` son los siguientes:

```
-- Dimensiones de valor y tiempo
SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-02])';
SELECT tbox 'TBOXFLOAT XT([1.5,2.5],[2001-01-01,2001-01-02])';
-- Sólo dimensión de valor
SELECT tbox 'TBOXINT X([1,3])';
SELECT tbox 'TBOXFLOAT X((1.5,2.5))';
-- Sólo dimensión de tiempo
SELECT tbox 'TBOX T((2001-01-01,2001-01-02))';
```

Un `stbox` se compone de dimensiones espacial y/o temporal, donde las coordenadas de la dimensión espacial pueden ser 2D o 3D. Para la dimensión de tiempo se da un `tstzspan`, y para la dimensión espacial se dan los valores mínimos y máximos de las coordenadas, donde estas últimas pueden ser cartesianas (planas) o geodésicas (esféricas). Se puede especificar el SRID de las coordenadas; si no es el caso, se asume un valor de 0 (desconocido) y 4326 (correspondiente a WGS84), respectivamente, para coordenadas planas y geodésicas. Los cuadros geodésicos siempre tienen una dimensión Z para tener en cuenta la curvatura de la esfera o esferoide subyacente. Ejemplos de entrada de valores `stbox` son los siguientes:

```
-- Sólo dimensión de valor con coordenadas X e Y
SELECT stbox 'STBOX X((1.0,2.0),(1.0,2.0))';
-- Sólo dimensión de valor con coordenadas X, Y y Z
SELECT stbox 'STBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Dimensiones de valor (con coordenadas X e Y) y de tiempo
SELECT stbox 'STBOX XT(((1.0,2.0),(1.0,2.0)), [2001-01-03,2001-01-03])';
-- Dimensiones de valor (con coordenadas X, Y y Z) y de tiempo
SELECT stbox 'STBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)), [2001-01-01,2001-01-03])';
-- Sólo dimensión de tiempo
```

```

SELECT stbox 'STBOX T([2001-01-03,2001-01-03])';
-- Sólo dimensión de valores con coordenadas geodéticas X, Y y Z
SELECT stbox 'GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Dimensiones de valor (con coordenadas geodéticas X, Y y Z) y de tiempo
SELECT stbox 'GEODSTBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)), [2001-01-04,2001-01-04])';
-- Sólo dimensión temporal para cuadro geodético
SELECT stbox 'GEODSTBOX T([2001-01-03,2001-01-03])';
-- Se indica el SRID
SELECT stbox 'SRID=5676;STBOX XT(((1.0,2.0),(1.0,2.0)), [2001-01-04,2001-01-04])';
SELECT stbox 'SRID=4326;GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';

```

Damos a continuación las funciones de entrada y salida de tipos de cuadro delimitador en formato textual (Well-Known Text o WKT) y binario (Well-Known Binary o WKB).

- Devuelve la representación textual conocida (Well-Known Text o WKT)

`asText(box,maxdecdigits=15) → text`

El argumento `maxdecdigits` se puede utilizar para definir el número máximo de decimales para la salida de los valores de coma flotante (por defecto 15).

```

SELECT asText(tbox 'TBOXFLOAT XT([1.123456789,2.123456789],[2001-01-01,2001-01-02])', 3);
-- TBOXFLOAT XT([1.123, 2.123],[2001-01-01, 2001-01-02])
SELECT asText(stbox 'STBOX Z((1.55,1.55,1.55),(2.55,2.55,2.55))', 0);
-- STBOX Z((2,2,2),(3,3,3))

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)

`asBinary(box,endian text="") → bytea`

`asHexWKB(box,endian text="") → text`

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```

SELECT asBinary(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- \x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])', 'XDR');
-- \x000300270100001cc1d3579c0000001cd5f12efc00000d013ff00000000000004000000000000000
SELECT asBinary(stbox 'STBOX X((1,1),(2,2))');
-- \x0101000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- 0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])', 'XDR');
-- 000300270100001cc1d3579c0000001cd5f12efc00000d013ff00000000000004000000000000000
SELECT asHexWKB(stbox 'STBOX X((1,1),(2,2))');
-- 0101000000000000f03f0000000000000040000000000000f03f0000000000000040

```

- Entrar a partir de representación binaria conocida (WKB) o de la representación hexadecimal binaria conocida (HexWKB)

`boxFromBinary(bytea) → box`

`boxFromHexWKB(text) → box`

En las funciones anteriores, `box` reemplaza cualquier tipo de cuadro delimitador, a saber, `tbox` o `stbox`.

```

SELECT tboxFromBinary(
  '\x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');
-- TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])
SELECT stboxFromBinary(
  '\x0101000000000000f03f0000000000000040000000000000f03f0000000000000040');
-- STBOX X((1,1),(2,2))
SELECT tboxFromHexWKB(
  '0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');

```

```
-- TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02]))
SELECT stboxFromHexWKB(
  '0101000000000000F03F00000000000004000000000000F03F000000000000040');
-- STBOX X((1,1),(2,2))
```

3.2. Constructores

El tipo `tbox` tienen varias funciones constructoras dependiendo de si se da la extensión de valor y/o de tiempo. La extensión de valor se puede especificar mediante un número o un intervalo, mientras que la extensión de tiempo se puede especificar mediante un tipo de tiempo.

■ Constructor para `tbox`

```
tbox({number,numspan}) → tbox
tbox({timestamptz,tstzspan}) → tbox
tbox({number,numspan},{timestamptz,tstzspan}) → tbox
```

```
-- Dimensiones de valores y de tiempo
SELECT tbox(1.0, timestamptz '2001-01-01');
SELECT tbox(floatspan '[1.0,2.0]', tstzspan '[2001-01-01,2001-01-02]');
-- Sólo dimensión de valores
SELECT tbox(floatspan '[1.0,2.0]');
-- Sólo dimensión de tiempo
SELECT tbox(tstzspan '[2001-01-01,2001-01-02]');
```

El tipo `stbox` tiene varias funciones constructoras dependiendo de si se da la extensión espacial y/o de tiempo. Las coordenadas de la extensión espacial pueden ser 2D o 3D y pueden ser cartesianas o geodésicas. La extensión espacial se puede especificar mediante los valores de coordenadas mínimo y máximo. El SRID se puede especificar en un último argumento opcional. Si no se proporciona, se asume un valor 0 (respectivamente 4326) por defecto para los cuadros planos (respectivamente geodésicos). La extensión espacial también se puede especificar mediante una geometría o una geografía. La extensión temporal se puede especificar mediante un tipo de tiempo.

■ Constructor para `stbox`

```
stboxX(float,float,float,float,srid=0) → stbox
stboxZ(float,float,float,float,float,float,srid=0) → stbox
stboxT({timestamptz,tstzspan}) → stbox
stboxXT(float,float,float,float,{timestamptz,tstzspan},srid=0) → stbox
stboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},srid=0) → stbox
geodstboxZ(float,float,float,float,float,float,srid=4326) → stbox
geodstboxT({timestamptz,tstzspan}) → stbox
geodstboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},srid=4326)
→ stbox
stbox(geo) → stbox
stbox(geo,{timestamptz,tstzspan}) → stbox
```

```
-- Sólo dimensión de valores con coordenadas X e Y
SELECT stboxX(1.0,2.0,1.0,2.0);
-- Sólo dimensión de valores con coordenadas X, Y y Z
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Sólo dimensión de valores con coordenadas X, Y y Z con SRID
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0,5676);
-- Sólo dimensión de tiempo
```

```

SELECT stboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Dimensiones de valor (con coordenadas X e Y) y de tiempo
SELECT stboxXT(1.0,2.0, 1.0,2.0, tstzspan '[2001-01-03,2001-01-03]');
-- Dimensiones de valor (con coordenadas X, Y y Z) y de tiempo
SELECT stboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-03]');
-- Sólo dimensión de valores con coordenadas geodéticas X, Y y Z
SELECT geodstboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Sólo dimensión de tiempo para cuadro geodético
SELECT geodstboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Dimensiones de valor (con coordenadas geodéticas X, Y y Z) y de tiempo
SELECT geodstboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-04]');
-- Geometry and time dimensión
SELECT stbox(geometry 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03, 2001-01-05]');
-- Geography and time dimensión
SELECT stbox(geography 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03, 2001-01-05]');

```

3.3. Conversión de tipos

■ Convertir un tbox a otro tipo

tbox:: {intspan, floatspan, tstzspan}

intspan(tbox) → tbox

floatspan(tbox) → tbox

tstzspan(tbox) → tbox

```

SELECT tbox 'TBOXINT XT([1,4],[2001-01-01,2001-01-02))'::intspan;
-- [1,4)
SELECT tbox 'TBOXFLOAT XT((1,2],[2001-01-01,2001-01-02))'::floatspan;
-- (1, 2]
SELECT tbox 'TBOXFLOAT XT((1,2],[2001-01-01,2001-01-02))'::tstzspan;
-- [2001-01-01, 2001-01-02)

```

■ Convertir otro tipo a un tbox

{numbers, times, tnumber}::tbox

tbox({numbers, times, tnumber}) → tbox

```

SELECT intset '{1,2}'::tbox;
-- TBOXINT X([1, 3))
SELECT intspan '[1,3]'::tbox;
-- TBOXINT X([1, 3))
SELECT floatspan '(1.0,2.0)'::tbox;
-- TBOXFLOAT X((1, 2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::tbox;
-- TBOX T((2001-01-01,2001-01-04))

```

■ Convertir un stbox a otro tipo

stbox:: {box2d, box2d, geo, tstzspan}

box2d(stbox) → box2d

box3d(stbox) → box2d

geometry(stbox) → geometry

geography(stbox) → geography

tstzspan(stbox) → tstzspan


```

SELECT stbox 'STBOX XT((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03] '::box2d;
-- BOX(1 2,3 4)
SELECT ST_AsEWKT(stbox 'SRID=4326;STBOX XT((1,1),(5,5)),[2001-01-01,2001-01-05] '::
  geometry);
-- SRID=4326;POLYGON((1 1,1 5,5 5,5 1,1 1))
SELECT ST_AsEWKT(stbox 'STBOX XT((1,1),(1,5)),[2001-01-01,2001-01-05] '::geometry);
-- LINESTRING(1 1,1 5)
SELECT ST_AsEWKT(stbox 'GEODSTBOX XT((1,1),(1,1)),[2001-01-01,2001-01-05] '::geography);
-- SRID=4326;POINT(1 1)
SELECT ST_AsEWKT(stbox 'STBOX ZT((1,1,1),(5,5,5)),[2001-01-01,2001-01-05] '::
  geometry);
/* POLYHEDRALSURFACE(((1 1 1,1 5 1,5 5 1,5 1 1,1 1 1)),
  ((1 1 5,5 1 5,5 5 1,5 5,1 1 5)),((1 1 1,1 1 5,1 5 5,1 5 1,1 1 1)),
  ((5 1 1,5 5 1,5 5 5,5 1 5,5 1 1)),((1 1 1,5 1 1,5 1 5,1 1 5,1 1 1)),
  ((1 5 1,1 5 5,5 5 5,5 5 1,1 5 1))) */
SELECT stbox 'STBOX XT((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03] '::tstzspan;
-- [2001-01-01, 2001-01-03]

```

■ Convertir otro tipo a un stbox

```

{box2d,box3d,geo,times,tgeo}::stbox
stbox({box2d,box3d,geo,times,tgeo}) → stbox

```

```

SELECT geometry 'Linestring(1 1 1,2 2 2) '::box3d::stbox;
-- STBOX Z((1,1,1),(2,2,2))
SELECT geography 'Linestring(1 1,2 2) '::stbox;
-- SRID=4326;GEODSTBOX X((1,1),(2,2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)} '::stbox;
-- STBOX T((2001-01-01,2001-01-04))

```

3.4. Accesorios

■ ¿Tiene dimensión X/Z/T?

```

hasX(box) → boolean
hasZ(stbox) → boolean
hasT(box) → boolean

```

```

SELECT hasX(tbox 'TBOX T([2001-01-01,2001-01-03])');
-- false
SELECT hasX(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- true
SELECT hasZ(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
SELECT hasT(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03])');
-- true
SELECT hasT(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false

```

■ ¿Es geodética?

```

isGeodetic(stbox) → boolean

```

```

SELECT isGeodetic(stbox 'GEODSTBOX Z((1.0,1.0,0.0),(3.0,3.0,1.0))');
-- true
SELECT isGeodetic(stbox 'STBOX XT((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-02])');
-- false

```

- Devuelve el valor mínimo de X/Y/Z/T

xMin(box) → float
 yMin(stbox) → float
 zMin(stbox) → float
 tMin(box) → timestampz

```
SELECT xMin(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- 1
SELECT yMin(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 2
SELECT zMin(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 3
SELECT tMin(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-01
```

- Devuelve el valor máximo de X/Y/Z/T

xMax(box) → float
 yMax(stbox) → float
 zMax(stbox) → float
 tMax(box) → timestampz

```
SELECT xMax(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 3
SELECT yMax(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 4
SELECT zMax(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 6
SELECT tMax(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-03
```

- Es el mínimo valor X/T inclusivo?

xMinInc(tbox) → bool
 tMinInc(box) → bool

```
SELECT xMinInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMinInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Es el máximo valor X/T inclusivo?

xMaxInc(tbox) → bool
 tMaxInc(box) → bool

```
SELECT xMaxInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMaxInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Área, volumen, perímetro

area(stbox, spheroid bool=true) → float
 volume(stbox) → float
 perimeter(stbox, spheroid bool=true) → float

Para cuadros geodésicos, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido. La función `volume` no acepta cuadros geodésicos.

```

SELECT area(stbox 'STBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03))');
-- 4
SELECT volume(stbox 'STBOX ZT(((1,1,1),(3,3,3)), [2001-01-01,2001-01-03))');
-- 8
SELECT perimeter(stbox 'STBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03))');
-- 8
SELECT area(stbox 'GEODSTBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03))');
-- 49209676328.36632
SELECT perimeter(stbox 'GEODSTBOX XT(((1,1),(3,3)), [2001-01-01,2001-01-03))', false);
-- 889221.9544681838

```

3.5. Transformaciones

- Desplazar y/o escalar el rango de valores de un cuadro delimitador con uno o dos valores

shiftValue(tbox, {integer, float}) → tbox

scaleValue(tbox, {integer, float}) → tbox

shiftScaleValue(tbox, {integer, float}, {integer, float}) → tbox

```

SELECT shiftValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 1.0);
-- TBOXFLOAT XT([2.5, 3.5], [2001-01-01, 2001-01-02])
SELECT scaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 2.0);
-- TBOXFLOAT XT([1.5, 3.5], [2001-01-01, 2001-01-02])
SELECT shiftScaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])', 2.0, 3.0);
-- TBOXFLOAT XT([3.5, 6.5], [2001-01-01, 2001-01-02])

```

- Desplazar y/o escalar el período de un cuadro delimitador con uno o dos intervalos. Si el ancho o el lapso de tiempo del valor de tiempo es cero (es decir, los límites inferior y superior son iguales), el resultado es el cuadro delimitador. El valor o intervalo dado debe ser estrictamente mayor que cero.

shiftTime(box, interval) → box

scaleTime(box, interval) → box

shiftScaleTime(box, interval, interval) → box

```

SELECT shiftTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '1 day');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-02, 2001-01-03])
SELECT shiftTime(stbox 'STBOX T([2001-01-01,2001-01-02])', interval '-1 day');
-- STBOX T([2001-12-31, 2001-01-01])
SELECT shiftTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 day');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-02, 2001-01-03])
SELECT scaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '2 days');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-01, 2001-01-03])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 hour');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01 00:00:00, 2001-01-01 01:00:00])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '-1 day');
-- ERROR: The interval must be positive: -1 days
SELECT shiftScaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5], [2001-01-01,2001-01-02])',
  interval '1 day', interval '3 days');
-- TBOXFLOAT XT([1.5, 2.5], [2001-01-02, 2001-01-05])
SELECT shiftScaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01,2001-01-02])',
  interval '1 hour', interval '3 hours');
-- STBOX ZT(((1,1,1),(2,2,2)), [2001-01-01 01:00:00, 2001-01-01 04:00:00])

```

- Devuelve la dimensión espacial del cuadro delimitador, eliminando la dimensión temporal, si existe

`getSpace(stbox) → stbox`

```
SELECT getSpace(stbox 'STBOX ZT((1,1,1), (2,2,2)), [2001-01-01,2001-01-03]');
-- STBOX Z((1,1,1), (2,2,2))
```

Las funciones dadas a continuación expanden los cuadros delimitadores en la dimensión de valor y de tiempo o establecen la precisión de la dimensión de valor. Estas funciones generan un error si la dimensión correspondiente no está presente.

- Extender la dimensión numérica, espacial o temporal del cuadro delimitador con un valor o un intervalo

`expandValue(tbox,{integer,float}) → tbox`

`expandSpace(stbox,float) → stbox`

`expandTime(box,interval) → box`

La función devuelve NULL si el valor o intervalo dado como segundo argumento es negativo y el rango resultante de desplazar los límites con el argumento está vacío.

```
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', 1.0);
-- TBOXFLOAT XT((0,3), [2001-01-01,2001-01-03])
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', -1.0);
-- NULL
SELECT expandValue(tbox 'TBOX T([2001-01-01,2001-01-03])', 1);
-- The box must have value dimension
```

```
SELECT expandSpace(stbox 'STBOX ZT((1,1,1), (2,2,2)), [2001-01-01,2001-01-03]', 1);
-- STBOX ZT((0,0,0), (3,3,3)), [2001-01-01, 2001-01-03])
SELECT expandSpace(stbox 'STBOX T([2001-01-01,2001-01-03])', 1);
-- The box must have space dimension
```

```
SELECT expandTime(tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-03])', interval '1 day');
-- TBOXFLOAT XT((1,2), [2000-12-31,2001-01-04])
SELECT expandTime(stbox 'STBOX ZT((1,1,1), (2,2,2)), [2001-01-01,2001-01-03]',
  interval '-1 day');
-- STBOX ZT((1,1,1), (2,2,2)), [2001-01-02,2001-01-02])
SELECT expandTime(tbox 'TBOX XT((1,2), [2001-01-01,2001-01-03])', interval '-2 days');
-- NULL
```

- Redondear el valor o las coordenadas del cuadro delimitador a un número de decimales

`round(box,integer=0) → box`

```
SELECT round(tbox 'TBOXFLOAT XT((1.12345,2.12345), [2001-01-01,2001-01-02])', 2);
-- TBOXFLOAT XT((1.12, 2.12), [2001-01-01,2001-01-02])
SELECT round(stbox 'STBOX XT((1.12345,1.12345), (2.12345,2.12345)),
  [2001-01-01,2001-01-02])', 2);
-- STBOX XT((1.12,1.12), (2.12,2.12)), [2001-01-01,2001-01-02])
SELECT round(tstzspan '[2000-01-01, 2001-01-02]::tbox');
-- The tbox must have X dimension
```

3.6. Sistema de referencia espacial

- Devuelve o especifica el identificador de referencia espacial

`SRID(stbox) → integer`

`setSRID(stbox) → stbox`

```

SELECT SRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)), [2001-01-01,2001-01-02]))';
-- 0
SELECT SRID(stbox 'SRID=5676;STBOX XT(((1.0,2.0),(4.0,5.0)), [2001-01-01,2001-01-02]))';
-- 5676
SELECT SRID(stbox 'GEODSTBOX T([2001-01-01,2001-01-02]))';
-- ERROR: The box must have space dimension
SELECT setSRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)),
[2001-01-01,2001-01-02]), 5676);
-- SRID=5676;STBOX ZT(((1,2,3),(4,5,6)), [2001-01-01,2001-01-02])

```

■ Transformar a una referencia espacial diferente

`transform(stbox,to_srid integer) → stbox`

`transformPipeline(stbox,pipeline text,to_srid integer,is_forward bool=true) → stbox`

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el cuadro de entrada tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del cuadro de entrada se ignora y el SRID del cuadro de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```

SELECT round(transform(stbox 'SRID=4326;STBOX XT((2.340088, 49.400250),
(6.575317, 51.553167)), [2001-01-01,2001-01-02]), 3812), 6);
/* SRID=3812;STBOX XT((502773.429981,511805.120402),(803028.908265,751590.742629)),
[2001-01-01, 2001-01-02]) */
WITH test(box, pipeline) AS (
  SELECT stbox 'SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(box, pipeline, 4326),
pipeline, 4326, false), 6)
FROM test;
-- SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))

```

3.7. Operaciones de división

■ Dividir el cuadro delimitador en cuadrantes u octantes {}

`quadSplit(stbox) → {stbox}`

Como lo indica el símbolo {}, esta función es una *función de retorno de conjuntos* (también conocida como *función de tabla*) ya que normalmente devuelve más de un valor.

```

SELECT quadSplit(stbox 'STBOX XT((0,0),(4,4)), [2001-01-01,2001-01-05]))';
/* {"STBOX XT((0,0),(2,2)), [2001-01-01, 2001-01-05])",
"STBOX XT((2,0),(4,2)), [2001-01-01, 2001-01-05])",
"STBOX XT((0,2),(2,4)), [2001-01-01, 2001-01-05])",
"STBOX XT((2,2),(4,4)), [2001-01-01, 2001-01-05])"} */
SELECT quadSplit(stbox 'STBOX Z((0,0,0),(4,4,4))');
/* {"STBOX Z((0,0,0),(2,2,2))", "STBOX Z((2,0,0),(4,2,2))", "STBOX Z((0,2,0),(2,4,2))",
"STBOX Z((2,2,0),(4,4,2))", "STBOX Z((0,0,2),(2,2,4))", "STBOX Z((2,0,2),(4,2,4))",
"STBOX Z((0,2,2),(2,4,4))", "STBOX Z((2,2,2),(4,4,4))"} */

```

Tenga en cuenta que la función anterior es una *función de retorno de conjuntos* (también conocida como *función de tabla*) ya que normalmente devuelve más de un valor. Por lo tanto, la función está marcada con el símbolo $\{\}$. Esta función se usa normalmente para cuadrículas de resolución múltiple, donde el espacio se divide en celdas de modo que las celdas tengan un número máximo de elementos. Figura 3.1 muestra un ejemplo del resultado de usar esta función usando trayectorias sintéticas en Bruselas.

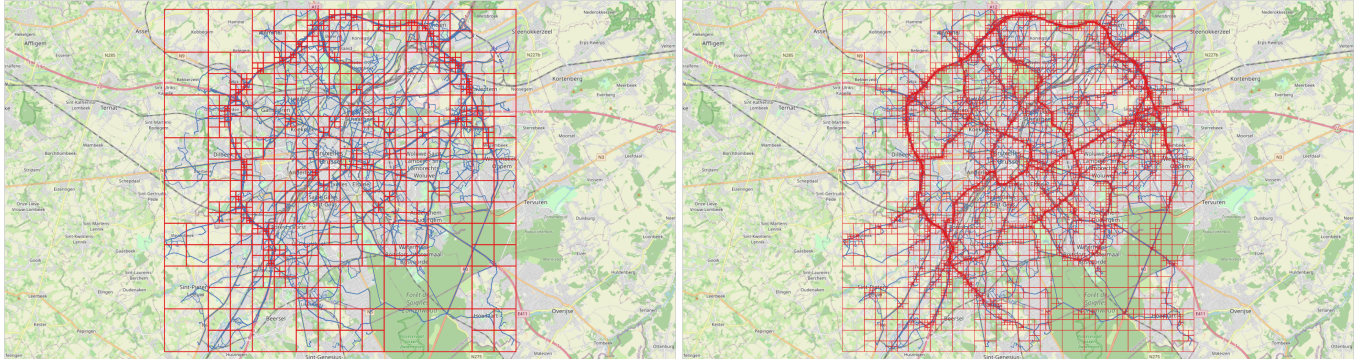


Figura 3.1: Grilla multiresolución sobre datos de Bruselas obtenidos mediante el generador **BerlinMOD**. Cada celda contiene como máximo 10.000 (izquierda) y 1.000 (derecha) instantes durante todo el período de simulación (cuatro días en este caso). A la izquierda, podemos ver la alta densidad de tráfico en el anillo alrededor de Bruselas, mientras que a la derecha podemos ver otros ejes principales de la ciudad.

3.8. Operaciones de conjuntos

Los operadores de conjuntos para los tipos de cuadro delimitador son la unión (+) y la intersección (*). En el caso de la unión, los operandos deben tener exactamente las mismas dimensiones, de lo contrario se genera un error. Además, si los operandos no se superponen en todas las dimensiones se genera un error, ya que esto daría como resultado una caja con valores disjuntos, que no se puede representar. El operador calcula la unión en todas las dimensiones que están presentes en ambos argumentos. En el caso de intersección, los operandos deben tener al menos una dimensión común, de lo contrario se genera un error. El operador calcula la intersección en todas las dimensiones que están presentes en ambos argumentos.

■ Unión e intersección de dos cuadros delimitadores

`box {+, *} box → box`

```
SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-03])' +
       tbox 'TBOXINT XT([2,4],[2001-01-02,2001-01-04])';
-- TBOXINT XT([1,4],[2001-01-01,2001-01-04])
SELECT stbox 'STBOX ZT((1,1,1),(2,2,2),[2001-01-01,2001-01-02])' +
          stbox 'STBOX XT((2,2),(3,3),[2001-01-01,2001-01-03])';
-- ERROR: The arguments must be of the same dimensionality
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-02])' +
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- ERROR: Result of box union would not be contiguous

SELECT tbox 'TBOXINT XT([1,3],[2001-01-01,2001-01-03])' *
       tbox 'TBOX T([2001-01-02,2001-01-04])';
-- TBOX T([2001-01-02,2001-01-03])
SELECT stbox 'STBOX ZT((1,1,1),(3,3,3),[2001-01-01,2001-01-02])' *
          stbox 'STBOX X((2,2),(4,4))';
-- STBOX X((2,2),(3,3))
```

3.9. Operaciones de cuadro delimitador

3.9.1. Operaciones topológicas

Hay cinco operadores topológicos: superposición (&&), contiene (@>), es contenido (<@), mismo (~=) y adyacente (-|-). Los operadores verifican la relación topológica entre los cuadros delimitadores teniendo en cuenta la dimensión de valor y/o de tiempo para todas las dimensiones que estén presentes en ambos argumentos.

- ¿Se superponen los cuadros delimitadores?

`box && box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' &&
       tbox 'TBOXFLOAT XT((2,4),[2001-01-02,2001-01-04])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' &&
       stbox 'STBOX XT([2001-01-02,2001-01-02])';
-- true
```

- ¿Contiene el primer cuadro delimitador el segundo?

`box @> box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' @>
       tbox 'TBOXFLOAT XT((2,3),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' @>
       stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- ¿Está el primer cuadro delimitador contenido en el segundo?

`box <@ box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <@
       tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' <@
       stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])';
-- true
```

- ¿Son los cuadros delimitadores iguales en sus dimensiones comunes?

`box ~= box → boolean`

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' ~=
       tbox 'TBOXFLOAT XT([2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03])' ~=
       stbox 'STBOX Z((1,1,1),(3,3,3))';
-- true
```

- ¿Son los cuadros delimitadores adyacentes?

`box -|- box → boolean`

Dos cuadros delimitadores son adyacentes si sus extensiones se encuentran en todas las dimensiones que los cuadros comparten y se tocan en un solo valor en al menos una de ellas. Una dimensión en la que los cuadros están separados los separa sea cual sea el comportamiento de las demás dimensiones.

```

SELECT tbox 'TBOXINT XT((1,2), [2001-01-01,2001-01-02])' -|-
tbox 'TBOXINT XT((2,3), [2001-01-02,2001-01-03])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2) [2001-01-01,2001-01-02])' -|-
tbox 'TBOX T([2001-01-02,2001-01-03])';
-- true
SELECT stbox 'STBOX XT(((1,1), (3,3)), [2001-01-01,2001-01-03])' -|-
stbox 'STBOX XT(((2,2), (4,4)), [2001-01-03,2001-01-04])';
-- true

```

3.9.2. Operaciones de posición

Los operadores de posición consideran la posición relativa de los cuadros delimitadores. Los operadores <<, >>, &< y &> consideran el valor X para el tipo tbox y las coordenadas X para el tipo stbox, los operadores <<|, |>>, &<| y |&> consideran las coordenadas Y para el tipo stbox, los operadores <</, />>, &</ y /&> consideran las coordenadas Z para el tipo stbox y los operadores <<#, #>>, #&< y #&> consideran la dimensión de tiempo para los tipos tbox y stbox. Los operadores generan un error si ambos cuadros delimitadores no tienen la dimensión requerida.

Los operadores para la dimensión numérica del tipo tbox se dan a continuación.

- ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente menores que los del segundo?

tbox {<<, <<#} tbox → boolean

stbox {<<, <<|, <</, <<#} stbox → boolean

```

SELECT tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-02])' <<
tbox 'TBOXFLOAT XT((3,4), [2001-01-03,2001-01-04])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-02])' <<
tbox 'TBOXFLOAT XT([2001-01-03,2001-01-04])';
-- ERROR: The box must have value dimension
SELECT stbox 'STBOX Z((1,1,1), (2,2,2))' << stbox 'STBOX Z((3,3,3), (4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1), (2,2,2))' <<| stbox 'STBOX Z((3,3,3), (4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1), (2,2,2))' <</ stbox 'STBOX Z((3,3,3), (4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-02])' <<#
tbox 'TBOXFLOAT XT((3,4), [2001-01-03,2001-01-04])';
-- true

```

- ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente mayores que los del segundo?

tbox {>>, #>>} tbox → boolean

stbox {>>, |>>, />>, #>>} stbox → boolean

```

SELECT tbox 'TBOXFLOAT XT((3,4), [2001-01-03,2001-01-04])' >>
tbox 'TBOXFLOAT XT((1,2), [2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((3,3,3), (4,4,4))' >> stbox 'STBOX Z((1,1,1), (2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3), (4,4,4))' |>> stbox 'STBOX Z((1,1,1), (2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3), (4,4,4))' />> stbox 'STBOX Z((1,1,1), (2,2,2))';
-- true
SELECT stbox 'STBOX XT((4,4), [2001-01-03,2001-01-04], ((3,3)))' #>>
stbox 'STBOX XT(((1,1), (2,2)), [2001-01-01,2001-01-02])';
-- true

```


- ¿No son los valores X/Y/Z/T del primer cuadro delimitador mayores que los del segundo?

```
tbox {&<, &<#} {tbox,stbox} → boolean
stbox &<, &<|, &</, &<#} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &< stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<#
       tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- ¿No son los valores X/Y/Z/T del primer cuadro delimitador menores que los del segundo?

```
tbox {&>, #&>} tbox → boolean
stbox {&>, |&>, /&>, #&>} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' &>
       tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' &> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- false
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' /&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' #&>
       stbox 'STBOX XT(((1,1),(4,4)),[2001-01-01,2001-01-04])';
-- true
```

3.10. Comparaciones

Los operadores de comparación tradicionales (=, <, etc.) se pueden aplicar a tipos de cuadro delimitador. Exceptuando la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten construir índices de árbol B en tipos de cuadro delimitador. Estos operadores comparan primero los valores de tiempo y, si son iguales, comparan los valores.

- Comparaciones tradicionales

```
box = box → boolean
box <> box → boolean
box < box → boolean
box > box → boolean
box <= box → boolean
box >= box → boolean
cmp(tbox,tbox) → int
cmp(stbox,stbox) → int
```

La función `cmp` devuelve -1, 0 o 1 según que el primer argumento sea, respectivamente, menor que, igual a o mayor que el segundo.

```

SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' =
  tbox 'TBOXINT XT([2,2],[2001-01-03,2001-01-05])';
-- false
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' <>
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <
  tbox 'TBOXINT XT([1,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-03,2001-01-04])' >
  tbox 'TBOXFLOAT XT([1,2],[2001-01-01,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <=
  tbox 'TBOXINT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' >=
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- false

```

3.11. Agregaciones

■ Extensión del cuadro delimitador

extent(box) → box

```

WITH boxes(b) AS (
  SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' UNION
  SELECT tbox 'TBOXFLOAT XT((5,7),[2001-01-05,2001-01-07])' UNION
  SELECT tbox 'TBOXFLOAT XT((6,8),[2001-01-06,2001-01-08])' )
SELECT extent(b) FROM boxes;
-- TBOXFLOAT XT((1,8),[2001-01-01,2001-01-08])
WITH boxes(b) AS (
  SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' UNION
  SELECT stbox 'STBOX Z((5,5,5),(7,7,7))' UNION
  SELECT stbox 'STBOX Z((6,6,6),(8,8,8))' )
SELECT extent(b) FROM boxes;
-- STBOX Z((1,1,1),(8,8,8))

```

3.12. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tablas de los tipos `tbox` y `stbox`. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple n-dimensional. Un ejemplo de creación de un índice GiST en una columna `Box` de tipo `stbox` en una tabla `Trips` es el siguiente:

```

CREATE TABLE Trips(TripID integer PRIMARY KEY, Trip tgeompoint, Box stbox);
CREATE INDEX Trips_Box_Idx ON Trips USING GIST(Box);

```

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores: `&&`, `<@`, `@>`, `~=`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<|`, `|>>`, `&<|`, `|&>`, `<</`, `/>>`, `&</`, `/&>`, `<<#`, `#>>`, `&<#` y `#&>`.

Además, se pueden crear índices de árbol B para columnas de tablas de un tipo cuadro delimitador. Para estos tipos de índices, básicamente la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos cuadro delimitador con los correspondientes operadores `<` y `>`, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte de árbol B está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Capítulo 4

Tipos temporales (Parte 1)

4.1. Introduction

MobilityDB proporciona varios tipos temporales basados en tipos PostgreSQL o PostGIS, a saber, `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint` y `tgeogpoint`, que se basan, respectivamente, en los tipos de base `bool`, `integer`, `float`, `text`, `geometry` y `geography`, donde `tgeometry` y `tgeography` aceptan geometrías/geografías arbitrarias, mientras que `tgeompoint` y `tgeogpoint` solo aceptan puntos 2D o 3D con dimensión Z. Además, MobilityDB proporciona otros tipos espaciales específicos, a saber, `cbuffer` (buffer circular), `npoint` (punto de red) y `pose`, y sus tipos espaciotemporales correspondientes, a saber, `tcbuffer`, `tnpoint`, `tpose` y `trgeometry` (geometría rígida temporal). En este capítulo y en el siguiente, presentamos las funciones y operadores disponibles para todos los tipos temporales, mientras que las funciones específicas para los tipos espaciotemporales se detallarán en capítulos posteriores.

La *interpolación* de un valor temporal establece cómo evoluciona el valor entre instantes sucesivos. La interpolación es *discreta* cuando se desconoce el valor entre dos instantes sucesivos. Pueden representar, por ejemplo, entradas/salidas cuando se utiliza un lector de tarjetas RFID para entrar o salir de un edificio. La interpolación es *escalonada* cuando el valor permanece constante entre dos instantes sucesivos. Por ejemplo, la marcha que utiliza un automóvil en movimiento puede representarse con un número entero temporal, lo que indica que su valor es constante entre dos instantes de tiempo. Por otro lado, la interpolación es *lineal* cuando el valor evoluciona linealmente entre dos instantes sucesivos. Por ejemplo, la velocidad de un automóvil puede representarse con un flotante temporal, lo que indica que los valores se conocen en los instantes de tiempo pero evolucionan continuamente entre ellos. De manera similar, la ubicación de un vehículo se puede representar mediante un punto temporal en el que la ubicación entre dos lecturas GPS consecutivas se obtiene mediante interpolación lineal. Los tipos temporales basados en tipos base discretos, es decir `tbool`, `tint` o `ttext` evolucionan necesariamente de manera escalonada. Por otro lado, los tipos temporales basados en tipos base continuos, es decir `tfloat`, `tgeompoint` o `tgeogpoint` pueden evolucionar de manera lineal o escalonada. Nótese que los tipos `tgeometry` y `tgeography` solo admiten interpolación discreta o por escalonada, ya que no es posible interpolar linealmente dos geometrías/geografías arbitrarias.

El *subtipo* de un valor temporal establece la extensión temporal en la que se registra la evolución de los valores. Los valores temporales vienen en tres subtipos, explicados a continuación.

Un valor temporal de subtipo *instante* (brevemente, un *valor de instante*) representa el valor en un instante de tiempo, por ejemplo

```
SELECT tfloat '17@2018-01-01 08:00:00';
```

Un valor temporal de subtipo *secuencia* (brevemente, un *valor de secuencia*) representa la evolución del valor durante una secuencia de instantes de tiempo, donde los valores entre estos instantes se interpolan usando una función discreta, escalonada, o lineal (ver arriba). Un ejemplo es el siguiente:

```
-- Interpolación discreta
SELECT tfloat '{17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00, 18@2018-01-01 08:10:00}';
-- Interpolación escalonada
SELECT tfloat 'Interp=Step;(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00,
  15@2018-01-01 08:10:00)';
-- Interpolación lineal
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00)';
```

Como puede verse, un valor de secuencia tiene un límite superior e inferior que pueden ser inclusivos (representados por '[' y ']') o exclusivos (representados por '(' y ')'). Por definición, ambos límites deben ser inclusivos cuando la interpolación es discreta o cuando la secuencia tiene un solo instante (que se llama *secuencia instantánea*), como el ejemplo siguiente.

```
SELECT tint '[10@2018-01-01 08:00:00]';
```

Los valores de secuencia deben ser *uniformes*, es decir, deben construirse a partir de valores de instante del mismo tipo de base. Los valores de secuencia con interpolación escalonada o lineal se denominan *secuencias continuas*.

El valor de una secuencia temporal se interpreta asumiendo que el período de tiempo definido por cada par de valores consecutivos $v_1@t_1$ y $v_2@t_2$ es inferior inclusivo y superior exclusivo, a menos que sean el primer o el último instantes de la secuencia y, en ese caso, se aplican los límites de toda la secuencia. Además, el valor que toma la secuencia temporal entre dos instantes consecutivos depende de si la interpolación es escalonada o lineal. Por ejemplo, la secuencia temporal anterior representa que el valor es 10 durante (2018-01-01 08:00:00, 2018-01-01 08:05:00), 20 durante [2018-01-01 08:05:00, 2018-01-01 08:10:00) y 15 en el instante final 2018-01-01 08:10:00. Por otro lado, la siguiente secuencia temporal

```
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00)';
```

representa que el valor evoluciona linealmente de 10 a 20 durante (2018-01-01 08:00:00, 2018-01-01 08:05:00) y evoluciona de 20 a 15 durante [2018-01-01 08:05:00, 2018-01-01 08:10:00).

Finalmente, un valor temporal de subtipo *conjunto de secuencias* (brevemente, un *valor de conjunto de secuencias*) representa la evolución del valor en un conjunto de secuencias, donde se desconocen los valores entre estas secuencias. Un ejemplo es el siguiente:

```
SELECT tfloat '{[17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00],
[18@2018-01-01 08:10:00, 18@2018-01-01 08:15:00]}';
```

Como se muestra en los ejemplos anteriores, los valores de conjunto de secuencias solo pueden ser de interpolación escalonada o lineal. Además, todas las secuencias que componen un valor de conjunto de secuencias deben ser del mismo tipo de base y de la misma interpolación.

Los valores de secuencia continuos se convierten en *forma normal* de modo que los valores equivalentes tengan representaciones idénticas. Para ello, los valores de instante consecutivos se fusionan cuando es posible. Para la interpolación escalonada, tres valores instantáneos consecutivos se pueden fusionar en dos si tienen el mismo valor. Para la interpolación lineal, tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Ejemplos de transformación en forma normal son los siguientes.

```
SELECT tint '[1@2001-01-01, 2@2001-01-03, 2@2001-01-04, 2@2001-01-05]';
-- [1@2001-01-01, 2@2001-01-03, 2@2001-01-05)
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
Point(1 1)@2001-01-01 08:10:00)');
-- [Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:10:00)
SELECT tfloat '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]';
-- [1@2001-01-01, 3@2001-01-05]
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(2 2)@2001-01-01 08:05:00,
Point(3 3)@2001-01-01 08:10:00)');
-- [Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]
```

De manera similar, los valores del conjunto de secuencias temporales se convierten en forma normal. Para ello, los valores de secuencia consecutivos se fusionan cuando es posible. Ejemplos de transformación en forma normal son los siguientes.

```
SELECT tint '{[1@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}';
-- '{[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)}'
SELECT tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-03, 3@2001-01-05)}';
-- '{[1@2001-01-01, 3@2001-01-05)}'
SELECT tfloat '{[1@2001-01-01, 3@2001-01-05), [3@2001-01-05)}';
-- '{[1@2001-01-01, 3@2001-01-05)}'
SELECT asText(tgeompoint '{[Point(0 0)@2001-01-01 08:00:00,
Point(1 1)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00),
[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)}');
-- '{[Point(0 0)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:10:00),
[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)}';
```

```

/* {[Point(0 0)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
   Point(1 1)@2001-01-01 08:15:00)} */
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,Point(2 2)@2001-01-01 08:05:00),
   [Point(2 2)@2001-01-01 08:05:00, Point(3 3)@2001-01-01 08:10:00]}');
-- {[Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]}
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,Point(3 3)@2001-01-01 08:10:00),
   [Point(3 3)@2001-01-01 08:10:00]}');
-- {[Point(1 1)@2001-01-01 08:00:00, Point(3 3)@2001-01-01 08:10:00]}

```

Los tipos temporales soportan *modificadores de tipo* (o *typmod* en terminología de PostgreSQL), que especifican información adicional para la definición de una columna. Por ejemplo, en la siguiente definición de tabla:

```
CREATE TABLE Department (DeptNo integer, DeptName varchar(25), NoEmps tint(Sequence));
```

el modificador de tipo para el tipo `varchar` es el valor 25, que indica la longitud máxima de los valores de la columna, mientras que el modificador de tipo para el tipo `tint` es la cadena de caracteres `Sequence`, que restringe el subtipo de los valores de la columna para que sean secuencias. En el caso de tipos alfanuméricos temporales (es decir, `tbool`, `tint`, `tfloat` y `ttext`), los valores posibles para el modificador de tipo son `Instant`, `Sequence` y `SequenceSet`. Si no se especifica ningún modificador de tipo para una columna, se permiten valores de cualquier subtipo.

Por otro lado, en el caso de tipos de puntos temporales (es decir, `tgeompoint` o `tgeogpoint`) el modificador de tipo se puede utilizar para especificar el subtipo, la dimensionalidad y/o el identificador de referencia espacial (SRID). Por ejemplo, en la siguiente definición de tabla:

```
CREATE TABLE Flight (FlightNo integer, Route tgeogpoint(Sequence, PointZ, 4326));
```

el modificador de tipo para el tipo `tgeogpoint` se compone de tres valores, el primero indica el subtipo como arriba, el segundo el tipo espacial de las geografías que componen el punto temporal y el último el SRID de las geografías que componen. Para los puntos temporales, los valores posibles para el primer argumento del modificador de tipo son los anteriores, los del segundo argumento son `Point` o `PointZ` y los del tercer argumento son SRID válidos. Los tres argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo, dimensionalidad y/o SRID.

Cada tipo temporal está asociado a otro tipo, conocido como su *cuadro delimitador*, que representan su extensión en la dimensión de valor y/o tiempo. El cuadro delimitador de los distintos tipos temporales es el siguiente:

- El tipo `tstzspan` para los tipos `tbool` y `ttext`, donde solo se considera la extensión temporal.
- El tipo `tbox` (temporal box) par los tipos `tint` y `tfloat`, donde la extensión del valor y de tiempo se define, respectivamente, por un rango numérico y un rango de tiempo.
- El tipo `stbox` (spatiotemporal box) para los tipos `tgeompoint` y `tgeogpoint`, donde la extensión espacial se define en las dimensiones X, Y y Z y la extensión de tiempo se define en un rango de tiempo.

Un amplio conjunto de funciones y operadores son disponibles para realizar varias operaciones en los tipos temporales. Estos se explican a continuación y en los próximos capítulos. Algunas de estas operaciones, en particular las relacionadas con índices, manipulan cuadros delimitadores por razones de eficiencia.

4.2. Ejemplos de tipos temporales

A continuación se dan ejemplos de uso de tipos alfanuméricos temporales.

```

CREATE TABLE Department (DeptNo integer, DeptName varchar(25), NoEmps tint);
INSERT INTO Department VALUES
  (10, 'Research', tint '[10@2001-01-01, 12@2001-04-01, 12@2001-08-01)'),
  (20, 'Human Resources', tint '[4@2001-02-01, 6@2001-06-01, 6@2001-10-01)');
CREATE TABLE Temperature (RoomNo integer, Temp tfloat);
INSERT INTO Temperature VALUES
  (1001, tfloat '{18.5@2001-01-01 08:00:00, 20.0@2001-01-01 08:10:00}'),

```

```

(2001, tfloat '{19.0@2001-01-01 08:00:00, 22.5@2001-01-01 08:10:00}');

-- Valor en una marca de tiempo
SELECT RoomNo, valueAtTimestamp(Temp, '2001-01-01 08:10:00')
FROM temperature;
-- 1001 | 20
-- 2001 | 22.5

-- Restricción a un valor
SELECT DeptNo, atValue(NoEmps, 10)
FROM Department;
-- 10 | [10@2001-01-01, 10@2001-04-01]
-- 20 |

-- Restricción a un período
SELECT DeptNo, atTime(NoEmps, tstzspan '[2001-01-01, 2001-04-01]')
FROM Department;
-- 10 | [10@2001-01-01, 12@2001-04-01]
-- 20 | [4@2001-02-01, 4@2001-04-01]

-- Comparación temporal
SELECT DeptNo, NoEmps #<= 10
FROM Department;
-- 10 | [t@2001-01-01, f@2001-04-01, f@2001-08-01]
-- 20 | [t@2001-02-01, t@2001-10-01]

-- Agregación temporal
SELECT tsum(NoEmps)
FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01,
    18@2001-06-01, 6@2001-08-01, 6@2001-10-01]} */

```

A continuación se dan ejemplos de uso de tipos de puntos temporales.

```

CREATE TABLE Trips(CarId integer, TripId integer, Trip tgeompoint);
INSERT INTO Trips VALUES
  (10, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:00:00, Point(2 0)@2001-01-01 08:10:00,
Point(2 1)@2001-01-01 08:15:00]}'),
  (20, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00,
Point(3 3)@2001-01-01 08:20:00]}');

-- Valor en una marca de tiempo
SELECT CarId, ST_AsText(valueAtTimestamp(Trip, timestamptz '2001-01-01 08:10:00'))
FROM Trips;
-- 10 | POINT(2 0)
-- 20 | POINT(1 1)

-- Restricción a un valor
SELECT CarId, asText(atValue(Trip, geometry 'Point(2 0)'))
FROM Trips;
-- 10 | {"[POINT(2 0)@2001-01-01 08:10:00+00]"}
-- 20 |

-- Restricción a un período
SELECT CarId, asText(atTime(Trip, tstzspan '[2001-01-01 08:05:00,2001-01-01 08:10:00]'))
FROM Trips;
-- 10 | {[POINT(1 0)@2001-01-01 08:05:00+00, POINT(2 0)@2001-01-01 08:10:00+00]}
-- 20 | {[POINT(0 0)@2001-01-01 08:05:00+00, POINT(1 1)@2001-01-01 08:10:00+00]}

-- Distancia temporal
SELECT T1.CarId, T2.CarId, T1.Trip <-> T2.Trip
FROM Trips T1, Trips T2

```

```
WHERE T1.CarId < T2.CarId;
/* 10 | 20 | {[1@2001-01-01 08:05:00+00, 1.4142135623731@2001-01-01 08:10:00+00,
1@2001-01-01 08:15:00+00)} */
```

4.3. Validez de los tipos temporales

Los valores de los tipos temporales deben satisfacer varias restricciones para que estén bien definidos. Estas restricciones se dan a continuación.

- Las restricciones en el tipo base y el tipo `timestamp_tz` deben satisfacerse.
- Un valor de secuencia debe estar compuesto por al menos un valor de instante.
- Un valor de secuencia instantánea o con interpolación discreta debe tener límites inferior y superior inclusivos.
- En un valor de secuencia, las marcas de tiempo de los instantes que la componen deben ser diferentes y estar ordenadas.
- En un valor de secuencia con interpolación escalonada, los dos últimos valores deben ser iguales si el límite superior es exclusivo.
- Un valor de conjunto de secuencias debe estar compuesto por al menos un valor de secuencia.
- En un valor de conjunto de secuencias, los valores de las secuencias componentes no deben superponerse y deben estar ordenados.

Se genera un error cuando no se satisface una de estas restricciones. Ejemplos de valores temporales incorrectos son los siguientes.

```
-- Valor del tipo de base incorrecto
SELECT tbool '1.5@2001-01-01 08:00:00';
-- Valor del tipo base no es un punto
SELECT tgeompoint 'Linestring(0 0,1 1)@2001-01-01 08:05:00';
-- Marca de tiempo incorrecta
SELECT tint '2@2001-02-31 08:00:00';
-- Secuencia vacía
SELECT tint '';
-- Límites incorrectos para la secuencia instantánea
SELECT tint '[1@2001-01-01 09:00:00)';
-- Marcas de tiempo duplicadas
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00]';
-- Marcas de tiempo desordenadas
SELECT tint '[1@2001-01-01 08:10:00, 2@2001-01-01 08:00:00]';
-- Valor final incorrecto para interpolación escalonada
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:10:00)';
-- Conjunto de secuencias vacío
SELECT tint '{}[]';
-- Marcas de tiempo duplicadas
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00}';
-- Períodos superpuestos
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-01 10:00:00),
[2@2001-01-01 09:00:00, 2@2001-01-01 11:00:00]}';
```

4.4. Temporalización de operaciones

Una forma común de generalizar las operaciones tradicionales a los tipos temporales es aplicar la operación en *cada instante*, lo que da un valor temporal como resultado. En ese caso, la operación sólo se define en la intersección de las extensiones temporales

de los operandos; si las extensiones temporales son disjuntas, el resultado es nulo. Por ejemplo, los operadores de comparación temporal, como #<, determinan si los valores tomados por sus operandos en cada instante satisfacen la condición y devuelven un booleano temporal. A continuación se dan ejemplos de las diversas generalizaciones de los operadores.

```
-- Comparación temporal
SELECT tfloat '[2@2001-01-01, 2@2001-01-03]' #< tfloat '[1@2001-01-01, 3@2001-01-03]';
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
SELECT tfloat '[1@2001-01-01, 3@2001-01-03]' #< tfloat '[3@2001-01-03, 1@2001-01-05]';
-- NULL

-- Adición temporal
SELECT tint '[1@2001-01-01, 1@2001-01-03]' + tint '[2@2001-01-02, 2@2001-01-05]';
-- [3@2001-01-02, 3@2001-01-03]

-- Intersección temporal
SELECT tintersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}

-- Distancia temporal
SELECT tgeompoint '[Point(0 0)@2001-01-01 08:00:00, Point(0 1)@2001-01-03 08:10:00]' <->
tgeompoint '[Point(0 0)@2001-01-02 08:05:00, Point(1 1)@2001-01-05 08:15:00]';
-- [0.5@2001-01-02 08:05:00+00, 0.745184033794557@2001-01-03 08:10:00+00]
```

Otro requisito común es determinar si los operandos satisfacen *alguna vez* o *siempre* una condición con respecto a una operación. Estos se pueden obtener aplicando los operadores de comparación alguna vez o siempre. Estos operadores se indican anteponiendo los operadores de comparación tradicionales con, respectivamente, ? (alguna vez) y % (siempre). A continuación se dan ejemplos de operadores de comparación alguna vez y siempre.

```
-- ¿Se cruzan los operandos alguna vez?
SELECT eIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))') ?= true;
-- true

-- ¿Se cruzan los operandos siempre?
SELECT aIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04]',
geometry 'Polygon((0 0,0 2,4 2,4 0,0 0))') %= true;
-- true

-- ¿Es el operando izquierdo alguna vez menor que el derecho?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03]' ?<
tfloat '[3@2001-01-01, 1@2001-01-03]') ?= true;
-- true

-- ¿Es el operando izquierdo siempre menor que el derecho?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03]' %<
tfloat '[2@2001-01-01, 4@2001-01-03]') %= true;
-- true
```

Por ejemplo, la función `eIntersects` determina si hay un instante en el que los dos argumentos se cruzan espacialmente.

A continuación describimos las funciones y operadores para tipos temporales. Para mayor concisión, en los ejemplos usamos principalmente secuencias compuestas por dos instantes.

4.5. Notación

A continuación presentamos las funciones y operadores para tipos temporales. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo del resultado puede depender del tipo de los argumentos. Para expresar esto, usamos la siguiente notación:

- `time` representa cualquier tipo de tiempo, es decir, `timestampz`, `tstzspan`, `tstzset` o `tstzspanset`,
- `ttype` representa cualquier tipo temporal,
- `ttypeInst`, `ttypeSeq` y `ttypeSeqSet` representan cualquier tipo temporal con subtipo instante, secuencia y conjunto de secuencias
- `tdisc` representa cualquier tipo temporal con tipo de base discreto, es decir, `tbool`, `tint`, or `ttext`,
- `tcont` cualquier tipo temporal con tipo de base continuo, es decir, `tfloat`, `tgeompoint`, or `tgeogpoint`,
- `ttypeDiscSeq` y `ttypeContSeq` representan cualquier tipo temporal con subtipo secuencia y, respectivamente, interpolación discreta y continua,
- `base` representa cualquier tipo de base de un tipo temporal, es decir, `boolean`, `integer`, `float`, `text`, `geometry` o `geography`,
- `values` representa cualquier conjunto de valores de un tipo base de un tipo temporal, por ejemplo, `integer`, `intset`, `intspan` y `intspanset` para el tipo base `integer`
- `type[]` representa una matriz de `type`.
- `<type>` en el nombre de una función representa las funciones obtenidas al remplazar `<type>` por un `type` específico. Por ejemplo, `tintDiscSeq` o `tfloatDiscSeq` son representadas por `ttypeDiscSeq`.

4.6. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (WKT), Moving Features JSON (MF-JSON) y Well-Known Binary (WKB) del Open Geospatial Consortium para todos los tipos temporales. Presentamos a continuación las funciones de entrada y salida para los tipos temporales. Empezamos describiendo formato WKT.

Un valor de instante es un par de la forma `v@t`, donde `v` es un valor del tipo de base y `t` es un valor de `timestampz`. Ejemplos de entrada de valores de instante son los siguientes:

```
SELECT tbool 'true@2001-01-01 08:00:00';
SELECT tint '1@2001-01-01 08:00:00';
SELECT tfloat '1.5@2001-01-01 08:00:00';
SELECT ttext 'AAA@2001-01-01 08:00:00';
SELECT tgeompoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeogpoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeometry 'Linestring(0 0,1 1)@2017-01-01 08:00:05';
SELECT tgeography 'Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:00:05';
```

Un valor de secuencia es un conjunto de valores `v1@t1, ..., vn@tn` delimitado por límites superior e inferior, que pueden ser inclusivo (representados por '[' y ']') o exclusivos (representados por '(' y ')'). Un valor de secuencia compuesto por una sola pareja `v@t` se denomina *secuencia instantánea*. Los valores de secuencia tienen una *función de interpolación* asociada que puede ser discreta, lineal o escalonada. Por definición, los límites inferior y superior de una secuencia instantánea o de un valor de secuencia con interpolación discreta son inclusivos. La extensión temporal de un valor de secuencia con interpolación discreta es un conjunto de marcas de tiempo. Ejemplos de valores de secuencia con interpolación discreta son los siguientes.

```
SELECT tbool '{true@2001-01-01 08:00:00, false@2001-01-03 08:00:00}';
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00}';
SELECT tint '{1@2001-01-01 08:00:00}'; -- Instantaneous sequence
SELECT tfloat '{1.0@2001-01-01 08:00:00, 2.0@2001-01-03 08:00:00}';
SELECT ttext '{AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00}';
SELECT tgeompoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeogpoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeometry '{Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00}';
SELECT tgeography '{Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00}';
```

La extensión temporal de un valor de secuencia con interpolación lineal o escalonada es un período definido por el primer y el último instante, así como por los límites inferior y superior. Ejemplos de valores de secuencia con interpolación lineal son los siguientes:

```
SELECT tbool '[true@2001-01-01 08:00:00, true@2001-01-03 08:00:00]';
SELECT tint '[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00]';
SELECT tfloat '[2.5@2001-01-01 08:00:00, 3@2001-01-03 08:00:00, 1@2001-01-04 08:00:00]';
SELECT tfloat '[1.5@2001-01-01 08:00:00]'; -- Instantaneous sequence
SELECT ttext '[BBB@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00]';
SELECT tgeompoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 0)@2017-01-01 08:05:00]';
SELECT tgeogpoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00,
    Point(0 0)@2017-01-01 08:10:00]';
SELECT tgeometry '[Point(0 0)@2017-01-01 08:00:00,
    Linestring(0 0,0 1)@2017-01-02 08:05:00]';
SELECT tgeography '[Point(0 0)@2017-01-01 08:00:00,
    Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00]';
```

Los valores de secuencia cuyo tipo base es continuo pueden especificar que la interpolación es escalonada con el prefijo `Interp=Step`. Si no se especifica, se supone que la interpolación es lineal por defecto. A continuación se dan ejemplos de valores de secuencia con interpolación escalonada:

```
SELECT tfloat 'Interp=Step;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:05:00, Point(1 1)@2017-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:05:00, Point(0 0)@2017-01-01 08:10:00]';
ERROR: Invalid end value for temporal sequence with step interpolation
SELECT tgeogpoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
    Point(1 1)@2017-01-01 08:10:00]';
```

Los dos últimos instantes de un valor de secuencia con interpolación discreta y límite superior exclusivo deben tener el mismo valor base, como se muestra en el segundo y tercer ejemplo anteriores.

Un *valor de conjunto de secuencias* es un conjunto $\{v_1, \dots, v_n\}$ donde cada v_i es un valor de secuencia. La interpolación de los valores conjunto de secuencias solo puede ser lineal o escalonada, no discreta. Todas las secuencias que componen un valor de conjunto de secuencias deben tener la misma interpolación. La extensión temporal de un valor de conjunto de secuencias es un conjunto de períodos. Ejemplos de valores de conjunto de secuencias con interpolación lineal son los siguientes:

```
SELECT tbool '{{[false@2001-01-01 08:00:00, false@2001-01-03 08:00:00),
    [true@2001-01-03 08:00:00], (false@2001-01-04 08:00:00, false@2001-01-06 08:00:00)}}';
SELECT tint '{{[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00),
    [2@2001-01-04 08:00:00, 3@2001-01-05 08:00:00, 3@2001-01-06 08:00:00)}}';
SELECT tfloat '{{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00, 2@2001-01-04 08:00:00,
    3@2001-01-06 08:00:00)}}';
SELECT ttext '{{[AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00, BBB@2001-01-04 08:00:00),
    [CCC@2001-01-05 08:00:00, CCC@2001-01-06 08:00:00)}}';
SELECT tgeompoint '{{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeogpoint '{{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeometry
    '{{[Point(0 0)@2017-01-01 08:00:00, Linestring(0 0,1 1)@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}}';
SELECT tgeography
    '{{[Point(0 0)@2017-01-01 08:00:00, Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:05:00),
    [Point(0 1)@2017-01-01 08:10:00, Linestring(0 0,1 1)@2017-01-01 08:15:00)}}';
```

Los valores de conjunto de secuencias cuyo tipo base es continuo pueden especificar que la interpolación es escalonada con el prefijo `Interp=Step`. Si no se especifica, se supone que la interpolación es lineal por defecto. A continuación se dan ejemplos de valores de conjunto de secuencias con interpolación escalonada:

```
SELECT tfloat 'Interp=Step;{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00,
  2@2001-01-04 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tgeompoint 'Interp=Step;{[Point(0 0)@2017-01-01 08:00:00,
  Point(0 1)@2017-01-01 08:05:00], [Point(0 1)@2017-01-01 08:10:00,
  Point(0 1)@2017-01-01 08:15:00]}';
SELECT tgeogpoint 'Interp=Step;{[Point(0 0)@2017-01-01 08:00:00,
  Point(0 1)@2017-01-01 08:05:00], [Point(0 1)@2017-01-01 08:10:00,
  Point(0 1)@2017-01-01 08:15:00]}';
```

Para geometrías temporales, es posible especificar el identificador de referencia espacial (SRID) utilizando la representación extendida de texto conocido (EWKT) de la siguiente manera:

```
SELECT tgeompoint 'SRID=5435;[Point(0 0)@2001-01-01,Point(0 1)@2001-01-02]';
SELECT tgeogpoint 'SRID=7844;[Point(0 0)@2001-01-01,LineString(1 0,0 1)@2001-01-02]';
```

Todas las geometrías componentes serán entonces del SRID dado. Además, cada geometría componente puede especificar su SRID con el formato EWKT como en el siguiente ejemplo

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=5435;Point(0 1)@2001-01-02]';
```

Se genera un error si las geometrías componentes no están todas en el mismo SRID o si el SRID de una geometría componente es diferente al del punto temporal.

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=4326;Point(0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (5435)
SELECT tgeogpoint 'SRID=7844;[SRID=4326;Point(0 0)@2001-01-01,
  SRID=4326;LineString(1 0,0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (7844)
```

Si no se especifica, el SRID predeterminado para los geometrías temporales es 0 (desconocido) y para los geografías temporales es 4326 (WGS 84). Los puntos temporales con interpolación escalonada también pueden especificar el SRID, como se muestra a continuación.

```
SELECT tgeompoint 'SRID=5435,Interp=Step;[Point(0 0)@2001-01-01,
  Point(0 1)@2001-01-02]';
SELECT tgeogpoint 'Interp=Step;[SRID=4326;Point(0 0)@2001-01-01,
  SRID=4326;Point(0 1)@2001-01-02]';
```

Damos a continuación las funciones de entrada y salida en formato textual (Well-Known Text o WKT), binario (Well-Known Binary o WKB) y Moving Features JSON (MF-JSON) para los tipos alfanuméricos temporales. Las funciones correspondientes para los puntos temporales se detallan en la Sección 7.4. El formato de salida predeterminado de todos los tipos alfanuméricos temporales es el formato de texto conocido. La función `asText` que se da a continuación permite determinar la salida de valores temporales de punto flotante.

- Devuelve la representación textual conocida (Well-Known Text o WKT)

`asText(tfloat,maxdecdigits=15) → text`

El argumento `maxdecdigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asText(tfloat '[10.55@2001-01-01, 25.55@2001-01-02]', 0);
-- [11@2001-01-01, 26@2001-01-02]
SELECT asText(tgeometry
  '[Point(1.55 1.55)@2001-01-01,LineString(1.55 1.55,3.55 3.55)@2001-01-02]', 0);
-- [Point(1 1)@2001-01-01, LineString(1 1,3 3)@2001-01-02]
```

- Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)

`asMFJSON(ttype,options=0,flags=0,maxdecdigits=15) → bytea`

El argumento `options` puede usarse para agregar un cuadro delimitador en la salida MFJSON:

- 0: significa que no hay opción (valor por defecto)
- 1: cuadro delimitador MFJSON

El argumento `flags` puede usarse para personalizar la salida JSON, por ejemplo, para producir una salida JSON fácil de leer (para lectores humanos). Consulte la documentación de la biblioteca `json-c` para conocer los valores posibles. Los valores típicos son los siguientes:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

El argumento `maxdecdigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asMFJSON(tbool 't@2001-01-01 18:00:00', 1);
/* {"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:00:00+01","lowerInc":true,"upperInc":true},
    "values":[true],"datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tint '{10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}', 1);
/* {"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:10:00+01"},"values":[10,25],"datetimes":["2001-01-01T18:00:00+01",
    "2001-01-01T18:10:00+01"],"lowerInc":true,"upperInc":true,
    "interpolation":"Discrete"} */
SELECT asMFJSON(tfloat '[10.5@2001-01-01 18:00:00+02, 25.5@2001-01-01 18:10:00+02]');
/* {"type":"MovingFloat","values":[10.5,25.5],"datetimes":["2001-01-01T17:00:00+01",
    "2001-01-01T17:10:00+01"],"lowerInc":true,"upperInc":true,"interpolation":"Linear"} */
SELECT asMFJSON(ttext '{[walking@2001-01-01 18:00:00+02,
    driving@2001-01-01 18:10:00+02]}');
/* {"type":"MovingText","sequences":[{"values":["walking","driving"],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lowerInc":true,"upperInc":true}],"interpolation":"Step"} */
SELECT asMFJSON(tgeometry '[Point(1 1)@2001-01-01 18:00:00+02,
    Linestring(1 1,2 2)@2001-01-01 18:10:00+02]');
/* {"type":"MovingGeometry","sequences":[{"values":[{"type":"Point",
    "coordinates":[1,1]},{ "type":"LineString","coordinates":[[1,1],[2,2]]}],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lower_inc":true,"upper_inc":true}],"interpolation":"Step"} */
```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)

`asBinary(ttype, endian text="") → bytea`

`asHexWKB(ttype, endian text="") → text`

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tbool 'true@2001-01-01');
-- \x011a000101009c57d3c11c0000
SELECT asBinary(tfloat '1.5@2001-01-01');
-- \x0121000100000000000000f83f009c57d3c11c0000
SELECT asHexWKB(tint '1@2001-01-01', 'XDR');
-- 000023010000000100001CC1D3579C00
SELECT asHexWKB(ttext 'AAA@2001-01-01');
-- 01290001040000000000000041414100009C57D3C11C0000
```

- Entrar a partir de la representación Moving Features JSON (MF-JSON)

`ttypeFromMFJSON(bytea) → ttype`

Hay una función por tipo temporal, el nombre de esta función tiene como prefijo el nombre del tipo, que son `tbool` o `tint` en los ejemplos a continuación

```

SELECT tboolFromMFJSON(text
  '{"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:00:00+01","lowerInc":true,"upperInc":true},
    "values":[true],"datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"}');
-- t@2001-01-01 18:00:00
SELECT tintFromMFJSON(text
  '{"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00+01",
    "end":"2001-01-01T18:10:00+01"},"values":[10,25],"datetimes":["2001-01-01T18:00:00+01",
    "2001-01-01T18:10:00+01"],"lowerInc":true,"upperInc":true,
    "interpolation":"Discrete"}');
-- {10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}
SELECT tfloatFromMFJSON(text
  '{"type":"MovingFloat","values":[10.5,25.5],"datetimes":["2001-01-01T17:00:00+01",
    "2001-01-01T17:10:00+01"],"lowerInc":true,"upperInc":true,
    "interpolation":"Linear"}');
-- [10.5@2001-01-01 18:00:00, 25.5@2001-01-01 18:10:00]
SELECT ttextFromMFJSON(text
  '{"type":"MovingText","sequences":[{"values":["walking","driving"],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lowerInc":true,"upperInc":true}], "interpolation":"Step"}');
-- [{"walking"@2001-01-01 18:00:00, "driving"@2001-01-01 18:10:00}]);
SELECT asText(tgeometryFromMFJSON(text
  '{"type":"MovingGeometry","sequences":[{"values":[{"type":"Point",
    "coordinates":[1,1]},{ "type":"LineString","coordinates":[[1,1],[2,2]]}],
    "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"],
    "lower_inc":true,"upper_inc":true}], "interpolation":"Step"}');
-- {[POINT(1 1)@2001-01-01 17:00:00, LINESTRING(1 1,2 2)@2001-01-01 17:10:00]}

```

- Entrar a partir de la representación binaria conocida (WKB) o de la representación hexadecimal binaria conocida (HexWKB)

ttypeFromBinary(bytea) → ttype

ttypeFromHexWKB(text) → ttype

Hay una función por tipo temporal, el nombre de la función tiene como prefijo el nombre del tipo, que son tbool o tint en los ejemplos a continuación.

```

SELECT tboolFromBinary('\x011a000101009c57d3c11c0000');
-- t@2001-01-01
SELECT tintFromBinary('\x000023010000000100001cc1d3579c00');
-- 1@2001-01-01
SELECT tfloatFromBinary('\x01210001000000000000f83f009c57d3c11c0000');
-- 1.5@2001-01-01
SELECT ttextFromBinary('\x01290001040000000000000041414100009c57d3c11c0000');
-- "AAA"@2001-01-01
SELECT tboolFromHexWKB('011A000101009C57D3C11C0000');
-- t@2001-01-01
SELECT tintFromHexWKB('000023010000000100001CC1D3579C00');
-- 1@2001-01-01
SELECT tfloatFromHexWKB('01210001000000000000F83F009C57D3C11C0000');
-- 1.5@2001-01-01
SELECT ttextFromHexWKB('01290001040000000000000041414100009C57D3C11C0000');
-- "AAA"@2001-01-01

```

4.7. Constructores

A continuación, damos las funciones de constructor para los distintos subtipos. El uso de la función de constructor suele ser más conveniente que escribir una constante literal.

■ Constructores para tipos temporales que tienen un valor constante

Estos constructores tienen dos argumentos, un tipo base y un tipo de tiempo, donde el último es un valor de `timestamp_tz`, `tstzset`, `tstzspan` o `tstzspanset` para construir, respectivamente, un valor de subtipo instante, una secuencia con interpolación discreta, una secuencia con interpolación lineal o escalonada o un conjunto de secuencias. Las funciones para valores de secuencia o de conjunto de secuencias con tipo de base continuo tienen además un tercer argumento opcional que indica si el valor temporal resultante tiene interpolación lineal o escalonada. Se asume por defecto una interpolación lineal si el argumento no se especifica.

```
ttype(base,timestamp_tz) → ttypeInst
ttype(base,tstzset) → ttypeDiscSeq
ttype(base,tstzspan,interp='linear') → ttypeContSeq
ttype(base,tstzspanset,interp='linear') → ttypeSeqSet
```

```
SELECT tbool(true, timestamp_tz '2001-01-01');
SELECT tint(1, timestamp_tz '2001-01-01');
SELECT tfloat(1.5, tstzset '{2001-01-01, 2001-01-02}');
SELECT ttext('AAA', tstzset '{2001-01-01, 2001-01-02}');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]', 'step');
SELECT tgeompoint('Point(0 0)', tstzspan '[2001-01-01, 2001-01-02]');
SELECT tgeography('SRID=7844;Point(0 0)', tstzspanset '{[2001-01-01, 2001-01-02],
[2001-01-03, 2001-01-04]}', 'step');
```

■ Constructores para tipos temporales de subtipo secuencia

Estos constructores tienen un primer argumento obligatorio, que es una matriz de valores de los valores instantáneos correspondientes y argumentos opcionales adicionales. El segundo argumento establece la interpolación. Si no se proporciona el argumento, es por defecto escalonada para los tipos de base discretos como los enteros y es lineal para tipos de base continuos como los flotantes. Se genera un error cuando se establece la interpolación lineal para valores temporales con tipos de base discretos. Para secuencias continuas, el tercero y el cuarto argumentos son valores booleanos que indican, respectivamente, si los límites izquierdo y derecho son inclusivos o exclusivos. Se supone que estos argumentos son verdaderos de forma predeterminada si no se especifican.

```
ttypeSeq(ttypeInst[],interp={'step','linear'},leftInc bool=true,
rightInc bool=true) → ttypeSeq
```

```
SELECT tboolSeq(ARRAY[tbool 'true@2001-01-01 08:00:00','false@2001-01-01 08:05:00'],
'discrete');
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00']);
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00'], 'linear');
-- ERROR: The temporal type cannot have linear interpolation
SELECT tfloatSeq(ARRAY[tfloat '1.0@2001-01-01 08:00:00', '2.0@2001-01-01 08:05:00'],
'step', false, true);
SELECT ttextSeq(ARRAY[ttext 'AAA@2001-01-01 08:00:00', 'BBB@2001-01-01 08:05:00']);
SELECT tgeompointSeq(ARRAY[tgeompoint 'Point(0 0)@2001-01-01 08:00:00',
'Point(0 1)@2001-01-01 08:05:00', 'Point(1 1)@2001-01-01 08:10:00'], 'discrete');
SELECT tgeographySeq(ARRAY[tgeography 'Point(1 1)@2001-01-01 08:00:00',
'Point(2 2)@2001-01-01 08:05:00']);
```

■ Constructores para tipos temporales de subtipo conjunto de secuencias

```
ttypeSeqSet(ttypeContSeq[]) → ttypeSeqSet
ttypeSeqSetGaps(ttypeInst[],maxt=NULL,maxdist=NULL,interp='linear') → ttypeSeqSet
```

Un primer conjunto de constructores tiene un solo argumento, que es una matriz de valores de los valores de *secuencia* correspondientes. La interpolación del valor temporal resultante depende de la interpolación de las secuencias que lo componen. Se genera un error si las secuencias que componen la matriz tienen una interpolación diferente.

```
SELECT tboolSeqSet(ARRAY[tbool '[false@2001-01-01 08:00:00, false@2001-01-01 08:05:00]',
'[true@2001-01-01 08:05:00]', '(false@2001-01-01 08:05:00, false@2001-01-01 08:10:00)']);
SELECT tintSeqSet(ARRAY[tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:05:00,
```

```

2@2001-01-01 08:10:00, 2@2001-01-01 08:15:00)');
SELECT tfloatSeqSet (ARRAY[tfloat '[1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00,
2.0@2001-01-01 08:10:00]', '[2.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']]);
SELECT tfloatSeqSet (ARRAY[tfloat 'Interp=Step;[1.0@2001-01-01 08:00:00,
2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
'Interp=Step;[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']]);
SELECT ttextSeqSet (ARRAY[ttext '[AAA@2001-01-01 08:00:00, AAA@2001-01-01 08:05:00]',
'[BBB@2001-01-01 08:10:00, BBB@2001-01-01 08:15:00)']]);
SELECT tgeompointSeqSet (ARRAY[tgeompoint '[Point(0 0)@2001-01-01 08:00:00,
Point(0 1)@2001-01-01 08:05:00, Point(0 1)@2001-01-01 08:10:00)',
'[Point(0 1)@2001-01-01 08:15:00, Point(0 0)@2001-01-01 08:20:00)']]);
SELECT tgeographySeqSet (ARRAY[tgeography
'[Point(0 0)@2001-01-01 08:00:00, Point(0 0)@2001-01-01 08:05:00)',
'[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)']]);
SELECT tfloatSeqSet (ARRAY[tfloat 'Interp=Step;[1.0@2001-01-01 08:00:00,
2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
'[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']]);
-- ERROR: The temporal values must have the same interpolation

```

Otro conjunto de constructores para valores de conjunto de secuencias tiene como primer argumento una matriz de valores de *instante* correspondientes, y dos argumentos que establecen un intervalo de tiempo máximo y una distancia máxima tal que se introduce un espacio entre secuencias del resultado siempre que dos instantes de entrada consecutivos tengan un intervalo de tiempo o una distancia mayor que estos valores. Para puntos temporales, la distancia se especifica en unidades del sistema de coordenadas. Estos dos argumentos de brechas son opcionales, pero al menos uno de ellos debe especificarse. Además, cuando el tipo base es continuo, un último argumento adicional establece si la interpolación es escalonada o lineal. Si no se especifica este argumento, se asume que es lineal por defecto.

Los parámetros de la función dependen del tipo temporal. Por ejemplo, el parámetro de interpolación no está permitido para tipos temporales con subtipos discretos como `tint`. De manera similar, el parámetro `maxdist` no está permitido para tipos escalares como `ttext` que no tienen una función de distancia estándar.

```

SELECT tintSeqSetGaps (ARRAY[tint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
'5@2001-01-05']);
-- {[1@2001-01-01, 3@2001-01-02, 4@2001-01-03, 5@2001-01-05]}
SELECT tintSeqSetGaps (ARRAY[tint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
'5@2001-01-05'], '1 day', 1);
-- {[1@2001-01-01], [3@2001-01-02, 4@2001-01-03], [5@2001-01-05]}
SELECT ttextSeqSetGaps (ARRAY[ttext 'AA@2001-01-01', 'BB@2001-01-02', 'AA@2001-01-03',
'CC@2001-01-05'], '1 day');
-- [{"AA"@2001-01-01, "BB"@2001-01-02, "AA"@2001-01-03}, [{"CC"@2001-01-05}]}
SELECT asText (tgeometrySeqSetGaps (ARRAY[tgeometry 'Point(1 1)@2001-01-01',
'Linestring(2 2,3 3)@2001-01-02', 'Point(4 2)@2001-01-03',
'Polygon((5 0,6 1,7 0,5 0))@2001-01-05'], '1 day', 1));
/* {[POINT(1 1)@2001-01-01], [LINESTRING(2 2,3 3)@2001-01-02],
[POINT(4 2)@2001-01-03], [POLYGON((5 0,6 1,7 0,5 0))@2001-01-05]} */
SELECT asText (tgeompointSeqSetGaps (ARRAY[tgeompoint 'Point(1 1)@2001-01-01',
'Point(2 2)@2001-01-02', 'Point(3 2)@2001-01-03', 'Point(3 2)@2001-01-05'],
'1 day', 1, 'step'));
/* Interp=Step;{[POINT(1 1)@2001-01-01], [POINT(2 2)@2001-01-02, POINT(3 2)@2001-01-03],
[POINT(3 2)@2001-01-05]} */

```

4.8. Conversión de tipos

Un valor temporal se puede convertir en un tipo compatible usando la notación `CAST(ttype1 AS ttype2)` o `ttype1::ttype2`.

- Convertir un valor temporal a un intervalo de marcas de tiempo

```
ttype::tstzspan
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
SELECT ttext '(A@2001-01-01, B@2001-01-03, C@2001-01-05]'::tstzspan;
-- (2001-01-01, 2001-01-05]
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
```

4.9. Accesores

- Devuelve el tamaño de la memoria en bytes

memSize(ttype) → integer

```
SELECT memSize(tint '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- 176
```

- Devuelve el subtipo temporal

tempSubtype(ttype) → {'Instant', 'Sequence', 'SequenceSet'}

```
SELECT tempSubtype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Sequence
```

- Devuelve el tipo base

tempBasetype(ttype) → text

```
SELECT tempBasetype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- int4
SELECT tempBasetype(tgeompoint 'Point(1 1)@2001-01-01');
-- geometry
```

- Devuelve la interpolación

interp(ttype) → {'None', 'Discrete', 'Step', 'Linear'}

```
SELECT interp(tbool 'true@2001-01-01');
-- None
SELECT interp(tfloat '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- Discrete
SELECT interp(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Linear
SELECT interp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tgeompoint 'Interp=Step;[Point(1 1)@2001-01-01,
    Point(2 2)@2001-01-02, Point(3 3)@2001-01-03]');
-- Step
SELECT interp(tgeometry '[Point(1 1)@2001-01-01,
    Linestring(1 1,2 2)@2001-01-02, Polygon((3 3,4 4,5 3,3 3))@2001-01-03]');
-- Step
```

- Devuelve el valor o la marca de tiempo de un instantes

getValue(ttypeInst) → base

getTimestamp(ttypeInst) → timestamptz


```
SELECT getValue(tint '1@2001-01-01');
-- 1
SELECT getTimestamp(tfloat '1@2001-01-01');
-- 2001-01-01
```

- Devuelve los valores o el tiempo en el que se define el valor temporal

getValues(talpha) → {bool[], spanset, textset}

getTime(ttype) → tstzspanset

```
SELECT getValues(tbool '[false@2001-01-01, true@2001-01-02, false@2001-01-03]');
-- {f,t}
SELECT getValues(tint '[1@2001-01-01, 3@2001-01-02, 1@2001-01-03]');
-- {[1, 2), [3, 4)}
SELECT getValues(tint '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [4@2001-01-04, 4@2001-01-05]}');
-- {[1, 3), [4, 5)}
SELECT getValues(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[1, 1], [2, 2]}
SELECT getValues(tfloat 'Interp=Step;{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 1], [2, 2], [3, 3]}
SELECT getValues(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- {[1, 2]}
SELECT getValues(tfloat '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 2], [3, 3]}
```

```
SELECT getTime(ttext 'walking@2001-01-01');
-- {[2001-01-01, 2001-01-01]}
SELECT getTime(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[2001-01-01, 2001-01-01], [2001-01-02, 2001-01-02], [2001-01-03, 2001-01-03]}
SELECT getTime(tint '[1@2001-01-01, 1@2001-01-15]');
-- {[2001-01-01, 2001-01-15]}
SELECT getTime(tfloat '{[1@2001-01-01, 1@2001-01-10], [12@2001-01-12, 12@2001-01-15]}');
-- {[2001-01-01, 2001-01-10], [2001-01-12, 2001-01-15]}
```

- Devuelve el período delimitador en el que se define el valor temporal

timeSpan(ttype) → tstzspan

```
SELECT timeSpan(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- [2001-01-01, 2001-01-03]
SELECT timeSpan(tint '[1@2001-01-01, 1@2001-01-15]');
-- [2001-01-01, 2001-01-15]
```

- Devuelve el valor inicial, final, o enésimo

startValue(ttype) → base

endValue(ttype) → base

valueN(ttype,int) → base

La función no tiene en cuenta si los límites son inclusivos o no.

```
SELECT startValue(tfloat '(1@2001-01-01, 2@2001-01-03)');
-- 1
SELECT endValue(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- 5
SELECT valueN(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}', 3);
-- 3
```

- Devuelve el valor en una marca de tiempo

`valueAtTimestamp(ttype,timestamp tz) → base`

```
SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 4@2001-01-04]', '2001-01-02');
-- 2
```

- Devuelve el intervalo de tiempo

`duration(ttype,bounds span=false) → interval`

Se puede poner en verdadero un parámetro adicional para calcular la duración del período limitador, ignorando así los posibles intervalos de tiempo

```
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}');
-- 00:00:00
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}', true);
-- 4 days
SELECT duration(tfloat '[1@2001-01-01, 2@2001-01-03, 2@2001-01-05]');
-- 4 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05]}');
-- 3 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05]}',
  true);
-- 4 days
```

- ¿Es el instante inicial/final inclusivo?

`lowerInc(ttype) → bool`

`upperInc(ttype) → bool`

```
SELECT lowerInc(tint '[1@2001-01-01, 2@2001-01-02]');
-- true
SELECT upperInc(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}');
-- false
```

- Devuelve el número o los instantes diferentes

`numInstants(ttype) → integer`

`instants(ttype) → ttypeInst[]`

```
SELECT numInstants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}');
-- 3
SELECT instants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}');
-- {"1@2001-01-01", "2@2001-01-02", "3@2001-01-03"}
```

- Devuelve el instante inicial, final o enésimo

`startInstant(ttype) → ttypeInst`

`endInstant(ttype) → ttypeInst`

`instantN(ttype,integer) → ttypeInst`

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT startInstant(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}');
-- 1@2001-01-01
SELECT endInstant(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}');
-- 3@2001-01-03
SELECT instantN(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03]}', 3);
-- 3@2001-01-03
```

- Devuelve el número o las marcas de tiempo diferentes

numTimestamps(ttype) → integer

timestamps(ttype) → timestamptz[]

```
SELECT numTimestamps(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 3
SELECT timestamps(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"2001-01-01", "2001-01-03", "2001-01-05"}
```

- Devuelve la marca de tiempo inicial, final o enésima

startTimestamp(ttype) → timestamptz

endTimestamp(ttype) → timestamptz

timestampN(ttype, integer) → timestamptz

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT startTimestamp(tfloat '[1@2001-01-01, 2@2001-01-03)');
-- 2001-01-01
SELECT endTimestamp(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2001-01-05
SELECT timestampN(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}', 3);
-- 2001-01-05
```

- Devuelve el número o las secuencias

numSequences({ttypeContSeq, ttypeSeqSet}) → integer

sequences({ttypeContSeq, ttypeSeqSet}) → ttypeContSeq[]

```
SELECT numSequences(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2
SELECT sequences(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"[1@2001-01-01, 2@2001-01-03)", "[3@2001-01-03, 5@2001-01-05)"}
```

- Devuelve la secuencia inicial, final, o enésima

startSequence({ttypeContSeq, ttypeSeqSet}) → ttypeContSeq

endSequence({ttypeContSeq, ttypeSeqSet}) → ttypeContSeq

sequenceN({ttypeContSeq, ttypeSeqSet}, integer) → ttypeContSeq

```
SELECT startSequence(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- [1@2001-01-01, 2@2001-01-03)
SELECT endSequence(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- [3@2001-01-03, 5@2001-01-05)
SELECT sequenceN(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}', 2);
-- [3@2001-01-03, 5@2001-01-05)
```

- Devuelve los segmentos

segments({ttypeContSeq, ttypeSeqSet}) → ttypeContSeq[]

```
SELECT segments(tint '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
  (3@2001-01-03, 5@2001-01-05)}');
/* {"[1@2001-01-01, 1@2001-01-02)", "[3@2001-01-02, 3@2001-01-03)", "[2@2001-01-03]",
  "(3@2001-01-03, 3@2001-01-05)", "[5@2001-01-05]" } */
```

```
SELECT segments(tfloat '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
(3@2001-01-03, 5@2001-01-05]}');
/* {[1@2001-01-01, 3@2001-01-02],[3@2001-01-02, 2@2001-01-03],
(3@2001-01-03, 5@2001-01-05)} */
```

4.10. Transformaciones

Un valor temporal se puede transformar en otro subtipo. Se genera un error si los subtipos son incompatibles.

■ Transformar un tipo temporal a otro subtipo

`ttypeInst(ttype) → ttypeInst`

`ttypeSeq(ttype) → ttypeSeq`

`ttypeSeqSet(ttype) → ttypeSeqSet`

```
SELECT tboolInst(tbool '{[true@2001-01-01]}');
-- t@2001-01-01
SELECT tboolInst(tbool '{[true@2001-01-01, true@2001-01-02]}');
-- ERROR: Cannot transform input value to a temporal instant
SELECT tintSeq(tint '1@2001-01-01');
-- [1@2001-01-01]
SELECT tfloatSeqSet(tfloat '2.5@2001-01-01');
-- {[2.5@2001-01-01]}
SELECT tfloatSeqSet(tfloat '{2.5@2001-01-01, 1.5@2001-01-02, 3.5@2001-01-03}');
-- {[2.5@2001-01-01], [1.5@2001-01-02], [3.5@2001-01-03]}
```

■ Transformar un valor temporal a otra interpolación

`setInterp(ttype, interp) → ttype`

```
SELECT setInterp(tbool 'true@2001-01-01', 'discrete');
-- {t@2001-01-01}
SELECT setInterp(tfloat '{[1@2001-01-01], [2@2001-01-02], [1@2001-01-03]}', 'discrete');
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]}
SELECT setInterp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02,
1@2001-01-03, 2@2001-01-04]', 'linear');
/* {[1@2001-01-01, 1@2001-01-02], [2@2001-01-02, 2@2001-01-03],
[1@2001-01-03, 1@2001-01-04], [2@2001-01-04]} */
SELECT asText(setInterp(tgeompoint 'Interp=Step;[Point(1 1)@2001-01-01,
Point(2 2)@2001-01-02], [Point(3 3)@2001-01-05, Point(4 4)@2001-01-06]', 'linear'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(2 2)@2001-01-02,
POINT(3 3)@2001-01-05, POINT(3 3)@2001-01-06], [POINT(4 4)@2001-01-06]} */
SELECT setInterp(tgeometry '[Point(1 1)@2001-01-01,
Linestring(1 1,2 2)@2001-01-02]', 'linear');
-- ERROR: The temporal type cannot have linear interpolation
```

■ Desplazar y/o escalar el intervalo de tiempo de un valor temporal a uno o dos intervalos

`shiftTime(ttype, interval) → ttype`

`scaleTime(ttype, interval) → ttype`

`shiftScaleTime(ttype, interval, interval) → ttype`

Cuando se escala, si el rango de valores o el intervalo de tiempo del valor temporal es cero (por ejemplo, para un instante temporal), el resultado es el valor temporal. Igualmente, el ancho o intervalo dado debe ser estrictamente mayor que cero.

```
SELECT shiftTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-02, 2@2001-01-04, 1@2001-01-06}
SELECT shiftTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-02, 2@2001-01-04]
```

```

SELECT asText(shiftTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-03],
[Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-04],
[POINT(2 2)@2001-01-05, POINT(1 1)@2001-01-06]} */
SELECT scaleTime(tint '1@2001-01-01', '1 day');
-- 1@2001-01-01
SELECT scaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-01, 2@2001-01-01 12:00:00+01, 1@2001-01-02}
SELECT scaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-01, 2@2001-01-02]
SELECT asText(scaleTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-01 00:00:00+01, POINT(2 2)@2001-01-01 06:00:00+01,
POINT(1 1)@2001-01-01 12:00:00+01], [POINT(2 2) @2001-01-01 18:00:00+01,
POINT(1 1)@2001-01-02 00:00:00+01]} */
SELECT scaleTime(tint '1@2001-01-01', '-1 day');
-- ERROR: The interval must be positive: -1 days
SELECT shiftScaleTime(tint '1@2001-01-01', '1 day', '1 day');
-- 1@2001-01-02
SELECT shiftScaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day', '1 day');
-- {1@2001-01-02, 2@2001-01-02 12:00:00+01, 1@2001-01-03}
SELECT shiftScaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day', '1 day');
-- [1@2001-01-02, 2@2001-01-03]
SELECT asText(shiftScaleTime(tgeometry '{[Point(1 1)@2001-01-01,
Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04,
Polygon((1 1,2 2,3 1,1 1))@2001-01-05]}', '1 day', '1 day'));
/* {[POINT(1 1)@2001-01-02 00:00:00, LINESTRING(1 1,2 2)@2001-01-02 06:00:00,
POINT(1 1)@2001-01-02 12:00:00], [POINT(2 2)@2001-01-02 18:00:00,
POLYGON((1 1,2 2,3 1,1 1))@2001-01-03 00:00:00]} */

```

- Devuelve un valor booleano temporal que indica si cada segmento de una secuencia (o conjunto de secuencias) temporal(es) continua tiene, respectivamente, al menos o como máximo una duración determinada.

`segmentMinDuration(temp, interval, bool strict=true) → tbool`

`segmentMaxDuration(temp, interval, bool strict=true) → tbool`

Si el argumento `strict` no se especifica, se asume el valor `true` por defecto y, por lo tanto, la función verifica si la duración del segmento es estrictamente menor o mayor que el intervalo. Si el argumento es `false`, la función verifica si la duración del segmento es menor/mayor o *igual* que el intervalo.

```

SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]', '1 day');
-- {[t@2001-01-01, f@2001-01-03, t@2001-01-04, t@2001-01-06]}
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]', '1 day', false);
-- {[t@2001-01-01, t@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]', '1 day');
-- {[f@2001-01-01, f@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]', '1 day', false);
-- {[f@2001-01-01, t@2001-01-03, f@2001-01-04, f@2001-01-06]}

```

- Transformar un valor temporal no lineal en un conjunto de filas, cada una es una pareja compuesta de un valor de base y un conjunto de períodos durante el cual el valor temporal tiene el valor de base {}

`unnest(ttype) → {(value,time)}`

```

SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '1@2001-01-01, 2@2001-01-02, 1@2001-01-03') AS un) t;
-- 1 | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-02]}

```

```
SELECT (un).value, (un).time
FROM (SELECT unnest(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- 1 | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-03]}
SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- ERROR: Operation not supported for temporal values with linear interpolation
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(tgeompoint 'Interp=Step;[Point(1 1)@2001-01-01,
    Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1) | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- POINT(2 2) | {[2001-01-02, 2001-01-03]}
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(tgeometry '[Point(1 1)@2001-01-01,
    Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1) | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- LINESTRING(1 1,2 2) | {[2001-01-02, 2001-01-03]}
```

Capítulo 5

Tipos temporales (Parte 2)

5.1. Modificaciones

A continuación, explicamos la semántica de las operaciones de modificación (es decir, `insert`, `update` y `delete`) para tipos temporales. Estas operaciones tienen una semántica similar a las operaciones correspondientes para tablas temporales de tiempo de aplicación (*application-time temporal tables*) introducidas en el estándar [SQL:2011](#). La principal diferencia es que SQL usa marcas de tiempo de tuplas (donde las marcas de tiempo se adjuntan a las tuplas), mientras que los valores temporales en MobilityDB usan marcas de tiempo de atributos (donde las marcas de tiempo se adjuntan a los valores de los atributos). Please refer to this [article](#) for a detailed discussion about tuple versus attribute timestamping in temporal databases.

La operación `insert` agrega a un valor temporal los instantes de otro sin modificar los instantes existentes, como se ilustra en la Figura 5.1.

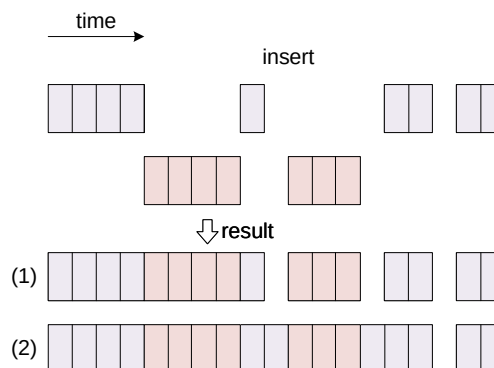


Figura 5.1: Operación de inserción para valores temporales.

Como se muestra en la figura, los valores temporales solo pueden intersectarse en su límite y, en ese caso, deben tener el mismo valor de base en sus marcas de tiempo comunes; de lo contrario, se genera un error. El resultado de la operación es la unión de los instantes para ambos valores temporales, como se muestra en el primer resultado de la figura. Esto es equivalente a una operación `merge` que se explica a continuación. Alternativamente, como se muestra en el segundo resultado de la figura, los fragmentos insertados que son disjuntos con el valor original se conectan al último instante anterior y al primer instante posterior al fragmento. Se utiliza un parámetro booleano `connect` para elegir entre los dos resultados, y el parámetro es verdadero por defecto. Nótese que esto solo se aplica a valores temporales continuos.

La operación `update` reemplaza los instantes en un primer valor temporal con los de un segundo valor como se ilustra en la Figura 5.2.

Como se muestra en la figura, el valor resultante contiene los instantes del segundo valor, independientemente de los instantes anteriores que tenía en el valor temporal original. Como en el caso de una operación `insert`, un parámetro booleano adicional determina si los nuevos fragmentos desconectados están conectados en el valor resultante, como se muestra en los dos posibles

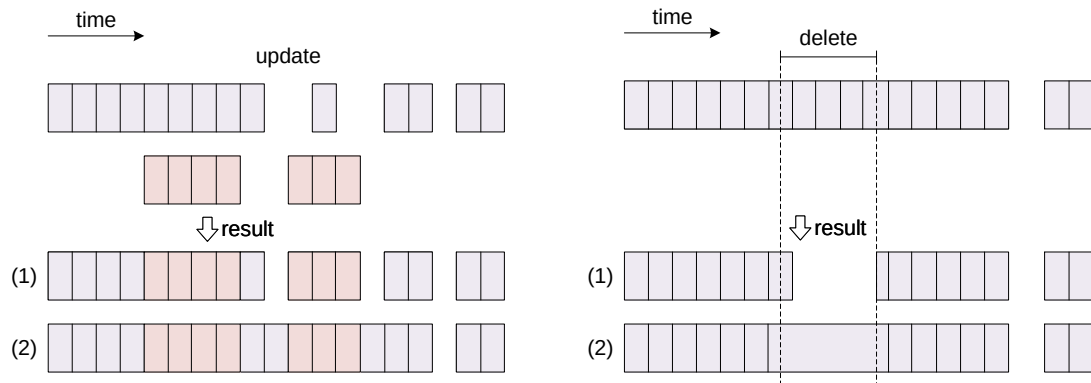


Figura 5.2: Operaciones de actualización y supresión para valores temporales.

resultados de la figura. Cuando los dos valores temporales son disjuntos o solo se intersectan en su límite, esto corresponde a una operación **insert** como se explicó anteriormente. En este caso, la operación **update** se comporta como una operación **upsert** en SQL.

La operación **deleteTime** elimina los instantes de un valor temporal que intersectan un valor de tiempo. Esta operación se puede utilizar en dos situaciones diferentes, ilustradas en la Figura 5.2.

1. En el primer caso, que se muestra como el resultado superior de la figura, el significado de la operación es introducir brechas de tiempo después de eliminar los instantes del valor temporal que intersectan el valor de tiempo. Esto es equivalente a las operaciones de restricción (Sección 5.2), que restringen un valor temporal al complemento del valor de tiempo.
2. El segundo caso, que se muestra como el resultado inferior de la figura, se usa para eliminar valores erróneos (por ejemplo, detectados como valores atípicos) sin introducir una brecha de tiempo, o para eliminar una brecha de tiempo. En este caso, los valores en el fragmento del valor temporal se eliminan y el último instante anterior y el primer instante posterior a una fragmento suprimido se conectan. Este comportamiento se especifica estableciendo un parámetro booleano adicional de la operación. Nótese que esto solo se aplica a valores temporales continuos.

La Figura 5.3 muestra las operaciones de modificación equivalentes para tablas temporales en el estándar SQL. Intuitivamente, estas figuras se obtienen girando 90 grados en el sentido de las agujas del reloj las figuras correspondientes para los valores temporales (Figura 5.1 y Figura 5.2). Esto se debe al hecho de que en SQL, las tuplas consecutivas ordenadas por tiempo generalmente se conectan a través de las funciones de ventana **LEAD** y **LAG**.

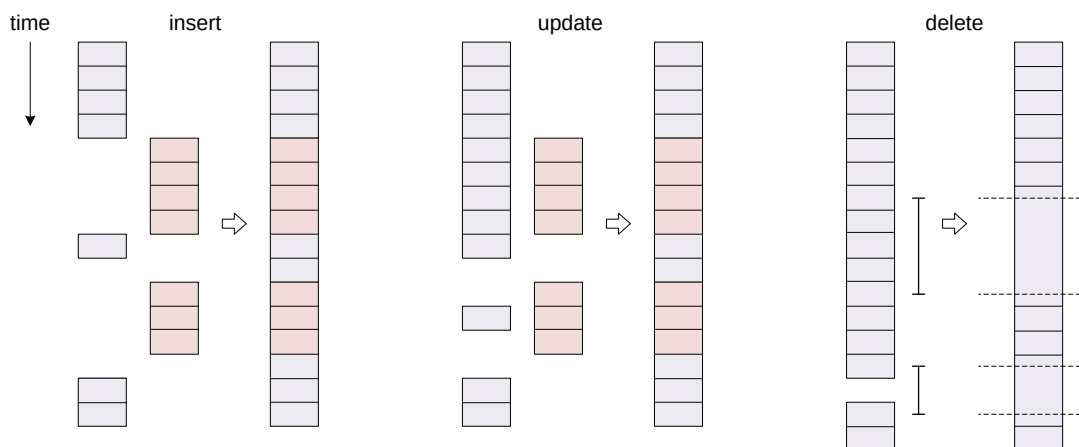


Figura 5.3: Operaciones de modificación para tablas temporales en SQL.

■ Insertar un valor temporal en otro

`insert(ttype,ttype,connect=true) → ttype`

```
SELECT insert(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{3@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 3@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT insert(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{5@2001-01-03, 7@2001-01-07}');
-- ERROR: The temporal values have different value at their overlapping instant 2001-01-03
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-05]
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]', false);
-- {[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 1@2001-01-05]}
SELECT asText(insert(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04],[Point(1 1 1)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-03'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (1 1 1)@2001-01-03,
  POINT Z (3 3 3)@2001-01-04], [POINT Z (1 1 1)@2001-01-05]} */
```

■ Actualizar un valor temporal con otro

`update(ttype,ttype,connect=true) → ttype`

```
SELECT update(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{5@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 5@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT update(tfloat '[1@2001-01-01, 1@2001-01-05]',
  tfloat '[1@2001-01-02, 3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 1@2001-01-02, 3@2001-01-03, 1@2001-01-04, 1@2001-01-05]}
SELECT asText(update(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(1 1 1)@2001-01-05], [Point(1 1 1)@2001-01-07]}',
  tgeompoint '[Point(2 2 2)@2001-01-02, Point(2 2 2)@2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (2 2 2)@2001-01-04,
  POINT Z (1 1 1)@2001-01-05], [POINT Z (1 1 1)@2001-01-07]} */
SELECT asText(update(
  tgeometry '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeometry '[Linestring(2 2,3 3)@2001-01-02, Linestring(3 3,2 2)@2001-01-04]'));
/* [POINT(1 1)@2001-01-01, LINESTRING(2 2,3 3)@2001-01-02,
  LINESTRING(3 3,2 2)@2001-01-04], (POINT(3 3)@2001-01-04, POINT(1 1)@2001-01-05) */
```

■ Eliminar de un valor temporal un valor de tiempo

`deleteTime(ttype,time,connect=true) → ttype`

```
SELECT deleteTime(tint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-02', false);
-- {[1@2001-01-01, 1@2001-01-02], (1@2001-01-02, 1@2001-01-03]}
SELECT deleteTime(tint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-02');
-- [1@2001-01-01, 1@2001-01-03]
SELECT deleteTime(tfloat '[1@2001-01-01, 4@2001-01-02, 2@2001-01-04, 5@2001-01-05]',
  tstzspan '[2001-01-02, 2001-01-04]');
-- [1@2001-01-01, 5@2001-01-05]
SELECT asText(deleteTime(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tstzspan '[2001-01-02, 2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-04,
  POINT Z (3 3 3)@2001-01-05]} */
```

■ Anexar un instante temporal a un valor temporal

`appendInstant(ttype,ttypeInst,interp=NULL) → ttype`

```
appendInstant(ttypeInst,interp=NULL,maxdist=NULL,maxt=NULL) → ttypeSeq
```

La primera versión de la función devuelve el resultado de agregar el segundo argumento al primero. Si cualquiera de las entradas es NULL, se devuelve NULL.

La segunda versión de la función anterior es una función de *agregación* que devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones SUM() y AVG() y, como la mayoría de los agregados, también ignora los valores NULL. Dos argumentos opcionales establecen una distancia máxima y un intervalo de tiempo máximo de modo que se introduce una brecha de tiempo cada vez que dos instantes consecutivos tienen una distancia o un intervalo de tiempo mayor que estos valores. Para puntos temporales, la distancia se especifica en unidades del sistema de coordenadas. Si uno de estos argumentos no se dan, no se tiene en cuenta para determinar las brechas.

```
SELECT appendInstant(tint '1@2001-01-01', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT appendInstant(tint '1@2001-01-01', tint '1@2001-01-02', 'discrete');
-- {1@2001-01-01, 1@2001-01-02}
SELECT appendInstant(tint '[1@2001-01-01]', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT asText(appendInstant(tgeompoint '{[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
tgeompoint 'Point(1 1 1)@2001-01-06'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
[POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendInstant(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
[Linestring(2 2,3 3)@2001-01-04, Point(3 3)@2001-01-05]}',
tgeometry 'Linestring(1 1,2 2)@2001-01-06'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(2 2,3 3)@2001-01-04,
POINT(3 3)@2001-01-05, LINESTRING(1 1,2 2)@2001-01-06]} */
```

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst ORDER BY inst) FROM temp;
-- [1@2001-01-01, 5@2001-01-05]
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst, 'discrete' ORDER BY inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
WITH temp(inst) AS (
  SELECT tgeogpoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeogpoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeogpoint 'Point(3 3)@2001-01-03' UNION
  SELECT tgeogpoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeogpoint 'Point(5 5)@2001-01-05' )
SELECT asText(appendInstant(inst ORDER BY inst)) FROM temp;
/* [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03,
POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05] */
```

Observe que en la primera consulta con tfloat, las observaciones intermedias fueron eliminadas por el proceso de normalización ya que eran redundantes debido a la interpolación lineal. Este no es el caso de la segunda consulta con interpolación discreta o la tercera consulta con tgeogpoint, donde se utilizan coordenadas geodésicas.

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '4@2001-01-04' UNION SELECT tfloat '5@2001-01-05' UNION
  SELECT tfloat '7@2001-01-07' )
SELECT appendInstant(inst, NULL, 0.0, '1 day' ORDER BY inst) FROM temp;
```

```
-- {1@2001-01-01, 2@2001-01-02}, [4@2001-01-04, 5@2001-01-05], [7@2001-01-07]}
WITH temp(inst) AS (
  SELECT tgeompoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeompoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeompoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeompoint 'Point(5 5)@2001-01-05' UNION
  SELECT tgeompoint 'Point(7 7)@2001-01-07' )
SELECT asText(appendInstant(inst, NULL, sqrt(2), '1 day' ORDER BY inst)) FROM temp;
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02],
    [POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05], [POINT(7 7)@2001-01-07]} */
```

■ Anexar una secuencia temporal a un valor temporal

```
appendSequence(ttype,ttypeSeq) → {ttypeSeq, ttypeSeqSet}
```

```
appendSequence(ttypeSeq) → {ttypeSeq,ttypeSeqSet}
```

La primera versión de la función devuelve el resultado de agregar el segundo argumento al primero. Si cualquiera de las entradas es NULL, se devuelve NULL.

La segunda versión de la función anterior es una función de *agregación* que devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones SUM() y AVG() y, como la mayoría de los agregados, también ignora los valores NULL.

```
SELECT appendSequence(tint '1@2001-01-01', tint '{2@2001-01-02, 3@2001-01-03}');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT appendSequence(tint '[1@2001-01-01, 2@2001-01-02]',
  tint '[2@2001-01-02, 3@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 3@2001-01-03]
SELECT asText(appendSequence(tgeompoint '{{Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02}, [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05}}',
  tgeompoint '[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]}');
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
    [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
    POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendSequence(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(3 3,2 2)@2001-01-04, Point(3 3)@2001-01-05}}',
  tgeometry '[Point(3 3)@2001-01-05, Linestring(3 3,1 1)@2001-01-06]}');
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(3 3,2 2)@2001-01-04,
    POINT(3 3)@2001-01-05, LINESTRING(3 3,1 1)@2001-01-06]} */
```

■ Fusionar los valores temporales

```
merge(ttype,ttype) → ttype
```

```
merge(ttype[]) → ttype
```

```
merge(ttype) → ttype
```

Los valores temporales solo pueden intersectar en su límite. En ese caso, para todos los tipos temporales excepto `tgeometry` y `tgeography`, sus valores de base en las marcas de tiempo comunes deben ser los mismos, de lo contrario se genera un error. Para los tipos `tgeometry` y `tgeography`, si el valor en sus marcas de tiempo comunes es diferente, el valor resultante después de la merge será la unión espacial de los valores. La razón de un comportamiento diferente para estos tipos es que son los únicos tipos temporales que no son tipos *escalares*, es decir, su valor en una marca de tiempo dada no es un único elemento del dominio del tipo base, sino más bien un subconjunto de su dominio.

La tercera versión de la función anterior es una función de *agregación* que devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones SUM() y AVG() y, como la mayoría de los agregados, también ignora los valores NULL.

```
SELECT merge(tint '1@2001-01-01', tint '1@2001-01-02');
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 1@2001-01-04]}
```

```

SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[1@2001-01-02, 2@2001-01-03]');
-- ERROR: The temporal values have different value at their common timestamp 2001-01-02
SELECT asText(merge(tgeompoint '{{Point(1 1 1)@2001-01-01,
    Point(2 2 2)@2001-01-02}, [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05}}',
    tgeompoint '{{Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06}}'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
    [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
    POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(merge(tgeometry 'Linestring(1 1,5 1)@2001-01-01',
    tgeometry '{Linestring(5 1,10 1)@2001-01-01, Point(3 3)@2001-01-02}'));
-- {LINESTRING(1 1,5 1,10 1)@2001-01-01, POINT(3 3)@2001-01-02}

```

```

SELECT merge(ARRAY[tint '1@2001-01-01', '1@2001-01-02']);
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{2@2001-01-02, 3@2001-01-03}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{3@2001-01-03, 4@2001-01-04}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04}
SELECT merge(ARRAY[tint '1@2001-01-01, 2@2001-01-02', '2@2001-01-02, 1@2001-01-03']);
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(ARRAY[tint '1@2001-01-01, 2@2001-01-02', '3@2001-01-03, 4@2001-01-04']);
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(merge(ARRAY[tgeompoint '{{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02},
    [Point(3 3)@2001-01-03, Point(4 4)@2001-01-04}}', '{[Point(4 4)@2001-01-04,
    Point(3 3)@2001-01-05], [Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]}']));
/* {[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02], [Point(3 3)@2001-01-03,
    Point(4 4)@2001-01-04, Point(3 3)@2001-01-05],
    [Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]} */
SELECT asText(merge(ARRAY[tgeompoint '{{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}}',
    '{[Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]
SELECT asText(merge(ARRAY[tgeometry '{{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}}',
    '{[Linestring(2 2,1 1)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- {[POINT(1 1)@2001-01-01, LINESTRING(2 2,1 1)@2001-01-02, POINT(1 1)@2001-01-03]}

```

```

WITH temp(inst) AS (
SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
SELECT tfloat '5@2001-01-05' )
SELECT merge(inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}

```

5.2. Restricciones

Hay dos conjuntos complementarios de funciones de restricción. El primer conjunto de funciones restringe el valor temporal con respecto a un valor o una extensión de tiempo. Dos ejemplos son `atValues` o `atTime`. El segundo conjunto de funciones restringe el valor temporal con respecto al *complement* de un valor o una extensión de tiempo. Dos ejemplos son `minusValues` o `minusTime`

- Restringir a (al complemento de) un valor o un conjunto de valores o, para un número temporal, un span o un conjunto de spans de valores

`atValue(ttype,value) → ttype`

`minusValue(ttype,value) → ttype`

`atValues(ttype,valueset) → ttype`

`minusValues(ttype,valueset) → ttype`

```

atSpan(tnumber,span) → tnumber
minusSpan(tnumber,span) → tnumber
atSpanset(tnumber,spanset) → tnumber
minusSpanset(tnumber,spanset) → tnumber

```

```

SELECT atValue(tint '[1@2001-01-01, 1@2001-01-15]', 1);
-- [1@2001-01-01, 1@2001-01-15]
SELECT atValues(tfloat '[1@2001-01-01, 4@2001-01-4]', floatset '{1, 3, 5}');
-- {[1@2001-01-01], [3@2001-01-03]}
SELECT atSpan(tfloat '[1@2001-01-01, 4@2001-01-4]', floatspan '[1,3]');
-- [1@2001-01-01, 3@2001-01-03]
SELECT atSpanset(tfloat '[1@2001-01-01, 5@2001-01-05]',
  floatspanset '{{[1,2], [3,4]}}');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(atValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03]',
  geometry 'Point(1 1 1)'));
-- {[POINT Z (1 1 1)@2001-01-02]}
SELECT asText(atValues(tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]',
  geomset '{"Point(0 0)", "Point(1 1)"}'));
-- {[POINT(0 0)@2001-01-01], [POINT(1 1)@2001-01-02]}
SELECT asText(atValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[LINESTRING(0 0,2 2)@2001-01-02, LINESTRING(0 0,2 2)@2001-01-03]}

```

```

SELECT minusValue(tint '[1@2001-01-01, 2@2001-01-02, 2@2001-01-03]', 1);
-- {[2@2001-01-02, 2@2001-01-03]}
SELECT minusValues(tfloat '[1@2001-01-01, 4@2001-01-4]', floatset '{2, 3}');
/* {[1@2001-01-01, 2@2001-01-02], (2@2001-01-02, 3@2001-01-03),
  (3@2001-01-03, 4@2001-01-04)} */
SELECT minusSpan(tfloat '[1@2001-01-01, 4@2001-01-4]', floatspan '[2,3]');
-- {[1@2001-01-01, 2@2001-01-02], (3@2001-01-03, 4@2001-01-04)}
SELECT minusSpanset(tfloat '[1@2001-01-01, 5@2001-01-05]',
  floatspanset '{{[1,2], [3,4]}}');
-- {(2@2001-01-02, 3@2001-01-03), (4@2001-01-04, 5@2001-01-05)}
SELECT asText(minusValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03]',
  geometry 'Point(1 1 1)'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
  (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03)} */
SELECT asText(minusValues(tgeompoint '[Point(0 0 0)@2001-01-01, Point(3 3 3)@2001-01-04]',
  geomset '{"Point(1 1 1)", "Point(2 2 2)"}'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
  (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03),
  (POINT Z (2 2 2)@2001-01-03, POINT Z (3 3 3)@2001-01-04)} */
SELECT asText(minusValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[POINT(0 0)@2001-01-01, POINT(0 0)@2001-01-02]}

```

■ Restringir a (al complemento de) un valor de tiempo

```

atTime(ttype,times) → ttype
minusTime(ttype,times) → ttype

```

```

SELECT atTime(tfloat '[1@2001-01-01, 5@2001-01-05]', timestampz '2001-01-02');
-- 2@2001-01-02
SELECT atTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzset '{2001-01-01, 2001-01-03}');
-- {1@2001-01-01, 1@2001-01-03}
SELECT atTime(tfloat '{[1@2001-01-01, 3@2001-01-03], [3@2001-01-04, 1@2001-01-06]}',
  tstzspan '[2001-01-02,2001-01-05]');
-- {[2@2001-01-02, 3@2001-01-03], [3@2001-01-04, 2@2001-01-05]}

```

```
SELECT atTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}');
-- {[1@2001-01-01, 1@2001-01-03), [1@2001-01-04, 1@2001-01-05)}
```

```
SELECT minusTime(tfloat '[1@2001-01-01, 5@2001-01-05]', timestampz '2001-01-02');
-- {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 5@2001-01-05)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzset '{2001-01-02, 2001-01-03}');
/* {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03),
  (1@2001-01-03, 1@2001-01-15)} */
SELECT minusTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02, 2001-01-05)');
-- {[1@2001-01-01, 2@2001-01-02), [2@2001-01-05, 1@2001-01-06)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15]',
  tstzspanset '{[2001-01-02, 2001-01-03), [2001-01-04, 2001-01-05)}');
/* {[1@2001-01-01, 1@2001-01-02), [1@2001-01-03, 1@2001-01-04),
  [1@2001-01-05, 1@2001-01-15)} */
```

- Mantener los instantes de un valor temporal que son anteriores o posteriores a una marca de tiempo, o iguales a ella

`beforeTimestamp(ttype,timestampz,strict bool=TRUE) → ttype`

`afterTimestamp(ttype,timestampz,strict bool=TRUE) → ttype`

```
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-02');
-- [1@2001-01-01, 1@2001-01-02)
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-01');
-- NULL
SELECT beforeTimestamp(tbigint '[1@2001-01-01, 1@2001-01-03]', timestampz '2001-01-01',
  false);
-- [1@2001-01-01]
SELECT afterTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03]', timestampz '2001-01-02',
  false);
-- [2@2001-01-02, 3@2001-01-03]
SELECT asText(afterTimestamp(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  timestampz '2001-01-03'));
/* {[POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05]} */
```

5.3. Operadores de cuadro delimitador

Estos operadores determinan si los cuadros delimitadores de sus argumentos satisfacen el predicado y dan como resultado un valor booleano. Como se indica en el Capítulo 4, el cuadro delimitador asociado a un tipo temporal depende del tipo base: es el tipo `tstzspan` para los tipos `tbool` y `ttext`, el tipo `tbox` para los tipos `tint` y `tfloat` y el tipo `stbox` para los tipos `tgeompoint` y `tgeogpoint`. Además, como se dijo en la Sección 3.3, muchos tipos PostgreSQL, PostGIS o MobilityDB se pueden convertir a los tipos `tbox` y `stbox`. Por ejemplo, los tipos numéricos y los rangos se pueden convertir al tipo `tbox`, los tipos `geometry` y `geography` se pueden convertir al tipo `stbox` y los tipos de tiempo y los tipos temporales se pueden convertir a los tipos `tbox` y `stbox`.

Un primer conjunto de operadores considera las relaciones topológicas entre los cuadros delimitadores. Hay cinco operadores topológicos: superposición (`&&`), contiene (`@>`), está contenido (`<@`), mismo (`~=`) y adyacente (`-|-`). Los argumentos de estos operadores pueden ser un tipo base, una cuadro delimitador o un tipo temporal y los operadores verifican la relación topológica teniendo en cuenta el valor y/o la dimensión temporal según el tipo de los argumentos.

Otro conjunto de operadores considera la posición relativa de los cuadros delimitadores. Los operadores `<<`, `>>`, `&<` y `&>` consideran la dimensión de valor para los tipos `tint` y `tfloat` y las coordenadas X para los tipos `tgeompoint` y `tgeogpoint`, los operadores `<<|`, `|>>`, `&<|` y `|&>` consideran las coordenadas Y para los tipos `tgeompoint` y `tgeogpoint`, los operadores `<<|`, `/>>`, `&<|` y `/&>` consideran las coordenadas Z para los tipos `tgeompoint` y `tgeogpoint` y los operadores `<<#`, `#>>`, `#&<` y `#&>` consideran la dimensión tiempo para todos los tipos temporales.

Finalmente, cabe destacar que los operadores de cuadro delimitador permiten mezclar geometrías 2D/3D pero en ese caso, el cálculo sólo se realiza en 2D.

Refiérase a la Sección 3.9 para los operadores de cuadro delimitador.

5.4. Comparaciones

5.4.1. Comparaciones tradicionales

Los operadores de comparación tradicionales (=, <, etc.) requieren que los operandos izquierdo y derecho sean del mismo tipo base. Excepto la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan sobre tipos temporales. Estos operadores comparan los períodos delimitadores (ver la Sección 2.10), después los cuadros delimitadores (ver la Sección 3.10) y si son iguales, entonces la comparación depende del subtipo. Para los valores de instante, primero comparan las marcas de tiempo y, si son iguales, comparan los valores. Para los valores de secuencia, comparan los primeros N instantes, donde N es el mínimo del número de instantes que componen ambos valores. Finalmente, para los valores de conjuntos de secuencias, comparan los primeros N valores de secuencia, donde N es el mínimo del número de secuencias que componen ambos valores.

Los operadores de igualdad y no igualdad consideran la representación equivalente para diferentes subtipos como se muestra a continuación.

```
SELECT tint '1@2001-01-01' = tint '{1@2001-01-01}';
-- true
SELECT tfloat '1.5@2001-01-01' = tfloat '[1.5@2001-01-01]';
-- true
SELECT ttext 'AAA@2001-01-01' = ttext '{[AAA@2001-01-01]}';
-- true
SELECT tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}' =
  tgeompoint ' {[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}';
-- true
SELECT tgeogpoint '{Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02}' =
  tgeogpoint ' {[Point(1 1 1)@2001-01-01], [Point(2 2 2)@2001-01-02]}';
-- true
```

■ Comparaciones tradicionales

ttype {=, <>, <, >, <=, >} ttype → boolean

```
SELECT tint '[1@2001-01-01, 1@2001-01-04]' = tint '[2@2001-01-03, 2@2001-01-05]';
-- false
SELECT tint '[1@2001-01-01, 1@2001-01-04]' <> tint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tint '[1@2001-01-01, 1@2001-01-04]' < tint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tint '[1@2001-01-01, 1@2001-01-04]' > tint '[2@2001-01-03, 2@2001-01-05]';
-- false
SELECT tint '[1@2001-01-01, 1@2001-01-04]' <= tint '[2@2001-01-03, 2@2001-01-05]';
-- true
SELECT tint '[1@2001-01-01, 1@2001-01-04]' >= tint '[2@2001-01-03, 2@2001-01-05]';
-- false
```

5.4.2. Comparaciones alguna vez y siempre

Una posible generalización de los operadores de comparación tradicionales (=, <>, <, <=, etc.) a tipos temporales consiste en determinar si la comparación es alguna vez o siempre verdadera. En este caso, el resultado es un valor booleano. MobilityDB proporciona operadores para probar si la comparación de un valor temporal y un valor del tipo base o dos valores temporales es alguna vez o siempre verdadera. Estos operadores se indican anteponiendo los operadores de comparación tradicionales con,

respectivamente, ? (alguna vez) y % (siempre). Algunos ejemplos son ?=, %<> o ?<=. La igualdad y la no igualdad alguna vez o siempre están disponibles para todos los tipos temporales, mientras que las desigualdades alguna vez o siempre sólo están disponibles para los tipos temporales cuyo tipo base tiene un orden total definido, es decir, tint, tfloat o ttext. Las comparaciones alguna vez y siempre son operadores inversos: por ejemplo, ?= es el inverso de %<> y ?> es el inverso de %<=.

■ Comparaciones alguna vez y siempre

{base,ttype} {?=, ?<>, ?<, ?>, ?<=, ?>=} {base,ttype} → boolean

{base,ttype} {%=, %<>, %<, %>, %<=, %>=} {base,ttype} → boolean

Los operadores no tienen en cuenta si los límites son inclusivos o no.

```
SELECT tint '[1@2001-01-01, 3@2001-01-04]' ?= 2;
-- false
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-04]' ?=
  geometry 'Linestring(1 1,0 0)';
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-04)' %= 1;
-- true
SELECT tgeometry '[Linestring(0 0,1 1)@2001-01-01, Linestring(1 1,0 0)@2001-01-04)' %=
  geometry 'Linestring(0 0,1 1)';
-- true
```

```
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' ?<> 2;
-- true
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' ?<>
  geometry 'Point(1 1)';
-- false
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' %<> 2;
-- false
SELECT tgeogpoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' %<>
  geography 'Point(2 2)';
-- true
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' ?< 2;
-- true
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' %< 2;
-- false
```

```
SELECT tint '[1@2001-01-03, 1@2001-01-05]' ?> 0;
-- true
SELECT tfloat '[1@2001-01-03, 1@2001-01-05)' %> 1;
-- false
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-05]' ?<= 2;
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' %<= 4;
-- true
```

```
SELECT ttext '{{AAA@2001-01-01, AAA@2001-01-03), [BBB@2001-01-04, BBB@2001-01-05}}'
  ?> 'AAA'::text;
-- true
SELECT ttext '{{AAA@2001-01-01, AAA@2001-01-03), [BBB@2001-01-04, BBB@2001-01-05}}'
  %> 'AAA'::text;
-- false
```


5.4.3. Comparaciones temporales

Otra posible generalización de los operadores de comparación tradicionales (=, <>, <, <=, etc.) a tipos temporales consiste en determinar si la comparación es verdadera o falsa en cada instante. En este caso, el resultado es un booleano temporal. Los operadores de comparación temporal se indican anteponiendo los operadores de comparación tradicionales con #. Algunos ejemplos son #= o #<=. La igualdad y no igualdad temporal están disponibles para todos los tipos temporales, mientras que las desigualdades temporales sólo están disponibles para los tipos temporales cuyo tipo base tiene un orden total definido, es decir, tint, tfloat o ttext.

■ Comparaciones temporales

{base,ttype} {#=, #<>, #<, #>, #<=, #>} {base,ttype} → boolean

```
SELECT tfloat '[1@2001-01-01, 2@2001-01-04]' #= 3;
-- {[f@2001-01-01, f@2001-01-04]}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #= tfloat '[4@2001-01-02, 1@2001-01-05]';
-- {[f@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04)}
SELECT tgeopoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]' #=
  geometry 'Point(1 1)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #=
  geometry 'Linestring(2 2,1 1)';
-- {[f@2001-01-01, t@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #<> 2;
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, 2@2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #<> tfloat '[2@2001-01-02, 2@2001-01-05]';
-- {[f@2001-01-02], (t@2001-01-02, t@2001-01-04)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #<>
  geometry 'Linestring(2 2,1 1)';
-- {[t@2001-01-01, f@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' #< 2;
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-04]}
SELECT 1 #> tint '[1@2001-01-03, 1@2001-01-05]';
-- [f@2001-01-03, f@2001-01-05]
SELECT tfloat '[1@2001-01-01, 1@2001-01-05]' #<= tfloat '{2@2001-01-03, 3@2001-01-04}';
-- {t@2001-01-03, t@2001-01-04}
SELECT 'AAA'::text #> ttext '{[AAA@2001-01-01, AAA@2001-01-03],
  [BBB@2001-01-04, BBB@2001-01-05]}';
-- {[f@2001-01-01, f@2001-01-03], [t@2001-01-04, t@2001-01-05]}
```

5.5. Funciones de utilidad

■ Versión de la extensión MobilityDB

mobilitydb_version() → text

```
SELECT mobilitydb_version();
-- MobilityDB 1.3.0
```

■ Versión de la extensión MobilityDB y de sus dependencias

mobilitydb_full_version() → text

```
/* MobilityDB 1.3.0, PostgreSQL 17.2, PostGIS 3.5.2, GEOS 3.12.0-CAPI-1.18.0, PROJ 9.2.0,
  JSON-C 0.13.1 */
```

Capítulo 6

Tipos alfanuméricos temporales

6.1. Notación

En Sección 4.5 presentamos la notación utilizada para definir la firma de las funciones y operadores para tipos temporales. A continuación, ampliamos estas notaciones para tipos alfanuméricos temporales.

- `torder` representa cualquier tipo temporal cuyo tipo de base tiene definido un orden total, es decir, `tint`, `tfloat` o `ttext`,
- `talpha` representa cualquier tipo temporal alfanumérico, por ejemplo, `tint` or `ttext`,
- `tnumber` representa cualquier tipo de número temporal, es decir, `tint` o `tfloat`,
- `number` represents any number base type, that is, integer or float,
- `numspan` represents any number span type, that is, either `intspan` or `floatspan`,

6.2. Conversión de tipos

Un valor temporal se puede convertir en un tipo compatible utilizando la notación `CAST (ttype1 AS ttype2)` o la notación `ttype1::ttype2`.

- Convertir un número temporal en un rango o un cuadro delimitador temporal

```
tnumber::{span,tbox}
```

```
valueSpan(tnumber) → numspan
```

```
timeSpan(tnumber) → tstzspan
```

```
tbox(tnumber) → tbox
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]':::intspan;
-- [1, 3]
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]':::floatspan;
-- (1, 3]
SELECT valueSpan(tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (1, 3]
SELECT timeSpan(tfloat 'Interp=Step;(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (2001-01-01, 2001-01-05]
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]':::tbox;
-- TBOXINT XT((1,2),[2001-01-01,2001-01-03])
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]':::tbox;
-- TBOXFLOAT XT((1,3),[2001-01-01,2001-01-05])
```

- Convertir entre un booleano temporal y un entero temporal

tbool::tint

tint(tbool) → tint

```
SELECT tbool '[true@2001-01-01, false@2001-01-03]':::tint;
-- [1@2001-01-01, 0@2001-01-03]
```

- Convertir entre enteros temporales, enteros grandes temporales y flotantes temporales

tnumber::tnumber

tnumber(tnumber) → tnumber

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]':::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03]
SELECT tbigint '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]':::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]
SELECT tfloat 'Interp=Step;[1.5@2001-01-01, 2.5@2001-01-03]':::tint;
-- [1@2001-01-01, 2@2001-01-03]
SELECT tint(tfloat '[1.5@2001-01-01, 2.5@2001-01-03]');
-- ERROR: Cannot cast temporal float with linear interpolation to temporal integer
```

6.3. Accessores

- Devuelve el intervalo de valores ignorando las brechas potenciales

valueSpan(tnumber) → numspan

```
SELECT valueSpan(tint '{[1@2001-01-01, 1@2001-01-03), [4@2001-01-03, 6@2001-01-05]}');
-- [1,7)
SELECT valueSpan(tfloat '{[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]}');
-- [1,3])
```

- Devuelve el conjunto de valores de un número temporal

valueSet(tnumber) → numset

```
SELECT valueSet(tint '[1@2001-01-01, 2@2001-01-03]');
-- {1, 2}
SELECT valueSet(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 4@2001-01-05]}');
-- {1, 2, 3, 4}
```

- Devuelve el valor mínimo, máximo, o promedio

minValue(torder) → base

maxValue(torder) → base

avgValue(tnumber) → base

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT minValue(tfloat '{[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]}');
-- 1
SELECT maxValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05]}');
-- 5
SELECT avgValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05]}');
-- 2.75
```

```
SELECT minValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- A
SELECT maxValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- E
```

- Devuelve el instante con el valor mínimo o máximo

`minInstant(torder) → base`

`maxInstant(torder) → base`

Las funciones no tienen en cuenta si los límites son inclusivos o no. Si varios instantes tienen el valor mínimo, se devuelve el primero.

```
SELECT minInstant(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1@2001-01-01
SELECT maxInstant(tfloat '{[1@2001-01-01, 2@2001-01-03], [3@2001-01-03, 5@2001-01-05]}');
-- 5@2001-01-05
```

6.4. Transformaciones

- Desplazar y/o escalar el rango de valores de un número temporal con uno o dos números

`shiftValue(tnumber,base) → tnumber`

`scaleValue(tnumber,width) → tnumber`

`shiftScaleValue(tnumber,base,base) → tnumber`

Cuando se escala, si el rango de valores del valor temporal es un valor único (por ejemplo, para un instante temporal), el resultado es el valor temporal. Además, el ancho data debe ser estrictamente superior que cero.

```
SELECT shiftValue(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', 1);
-- {2@2001-01-02, 3@2001-01-04, 2@2001-01-06}
SELECT shiftValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}', -1);
-- {[0@2001-01-01, 1@2001-01-02], [2@2001-01-03, 3@2001-01-04]}
SELECT scaleValue(tint '1@2001-01-01', 1);
-- 1@2001-01-01
SELECT scaleValue(tfloat '{[1@2001-01-01,2@2001-01-02], [3@2001-01-03,4@2001-01-04]}', 6);
-- {[1@2001-01-01, 3@2001-01-03], [5@2001-01-05, 7@2001-01-07]}
SELECT scaleValue(tint '1@2001-01-01', -1);
-- ERROR: The value must be strictly positive: -1
SELECT shiftScaleValue(tint '1@2001-01-01', 1, 1);
-- 2@2001-01-01
SELECT shiftScaleValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}',
-1, 6);
-- {[0@2001-01-01, 2@2001-01-02], [4@2001-01-03, 6@2001-01-04]}
```

- Extraer de un flotante temporal con interpolación lineal las subsecuencias donde los valores permanecen en un rango de un ancho dado durante al menos una duración dada

`stops(tfloat,maxDist=0.0,minDuration='0 minutes') → tfloatSeqSet`

Si `maxDist` no se especifica, el valor 0.0 se asume por defecto y por lo tanto, la función extrae los segmentos constantes del flotante temporal.

```
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]');
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]', 0.0, '2 days');
-- NULL
```

6.5. Restrictions

- Restringir al (complemento del) valor mínimo o máximo

`atMin(torder) → torder`

```
atMax(torder) → torder
minusMin(torder) → torder
minusMax(torder) → torder
```

La función devuelve un valor nulo si el valor mínimo o máximo sólo ocurre en límites exclusivos.

```
SELECT atMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {1@2001-01-01, 1@2001-01-05}
SELECT atMin(tint '{1@2001-01-01, 3@2001-01-03}');
-- {(1@2001-01-01, 1@2001-01-03)}
SELECT atMin(tfloat '{1@2001-01-01, 3@2001-01-03}');
-- NULL
SELECT atMin(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05)}');
-- {(AA@2001-01-01, AA@2001-01-03), [AA@2001-01-05]}
SELECT atMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {3@2001-01-05}
SELECT atMax(tfloat '{1@2001-01-01, 3@2001-01-03}');
-- NULL
SELECT atMax(tfloat '{(2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05]}');
-- {[2@2001-01-03, 2@2001-01-05]}
SELECT atMax(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05)}');
-- {"BB"@2001-01-03, "BB"@2001-01-05}
```

```
SELECT minusMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {2@2001-01-03}
SELECT minusMin(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tfloat '{1@2001-01-01, 3@2001-01-03}');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tint '{[1@2001-01-01, 1@2001-01-03], (1@2001-01-03, 1@2001-01-05)}');
-- NULL
SELECT minusMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {1@2001-01-01, 2@2001-01-03}
SELECT minusMax(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {[1@2001-01-01, 3@2001-01-03]}
SELECT minusMax(tfloat '{1@2001-01-01, 3@2001-01-03}');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '{[2@2001-01-01, 1@2001-01-03], [2@2001-01-03, 2@2001-01-05]}');
-- {(2@2001-01-01, 1@2001-01-03)}
SELECT minusMax(tfloat '{[1@2001-01-01, 3@2001-01-03], (3@2001-01-03, 1@2001-01-05)}');
-- {[1@2001-01-01, 3@2001-01-03], (3@2001-01-03, 1@2001-01-05)}
```

■ Restringir a (al complemento de) un tbox

```
atTbox(tnumber,tbox) → tnumber
minusTbox(tnumber,tbox) → tnumber
```

When the bounding box has both value and time dimensions, the functions restrict the temporal number with respect to the value *and* the time extents of the box.

```
SELECT atTbox(tfloat '[0@2001-01-01, 3@2001-01-04]',
  tbox 'TBOXFLOAT XT((0,2),[2001-01-02, 2001-01-04])');
-- {[1@2001-01-02, 2@2001-01-03]}
```

```
SELECT minusTbox(tfloat '[1@2001-01-01, 4@2001-01-04]',
  'TBOXFLOAT XT((1,4),[2001-01-03, 2001-01-04])');
-- {[1@2001-01-01, 3@2001-01-03]}
WITH temp(temp, box) AS (
  SELECT tfloat '[1@2001-01-01, 4@2001-01-04]',
    tbox 'TBOXFLOAT XT((1,2),[2001-01-03, 2001-01-04])' )
SELECT minusSpan(minusTime(temp, box::tstzspan), box::floatspan) FROM temp;
-- {[1@2001-01-01], [2@2001-01-02, 3@2001-01-03]}
```

6.6. Operaciones booleanas

■ Y temporal, o temporal

`{boolean,tbool} {& |} {boolean,tbool} → tbool`

```
SELECT tbool '[true@2001-01-03, true@2001-01-05]' &
  tbool '[false@2001-01-03, false@2001-01-05]';
-- [f@2001-01-03, f@2001-01-05]
SELECT tbool '[true@2001-01-03, true@2001-01-05]' &
  tbool '{[false@2001-01-03, false@2001-01-04), [true@2001-01-04, true@2001-01-05}}';
-- {[f@2001-01-03, t@2001-01-04, t@2001-01-05)}
```

```
SELECT tbool '[true@2001-01-03, true@2001-01-05]' |
  tbool '[false@2001-01-03, false@2001-01-05]';
-- [t@2001-01-03, t@2001-01-05]
```

■ No temporal

`~tbool → tbool`

```
SELECT ~tbool '[true@2001-01-03, true@2001-01-05]';
-- [f@2001-01-03, f@2001-01-05]
```

■ Devuelve el tiempo cuando un booleano temporal toma el valor verdadero

`whenTrue(tbool) → tstzspanset`

```
SELECT whenTrue(tfloat '[1@2001-01-01, 4@2001-01-04, 1@2001-01-07]' #> 2);
-- {(2001-01-02, 2001-01-06)}
SELECT whenTrue(tdwithin(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04,
  Point(1 1)@2001-01-07]', geometry 'Point(1 1)', sqrt(2)));
-- {[2001-01-01, 2001-01-02], [2001-01-06, 2001-01-07]}
```

6.7. Operaciones matemáticas

■ Adición, resta, multiplicación y división temporal

`{number,tnumber} {+, -, *, /} {number,tnumber} → tnumber`

La división temporal genera un error si el denominador es alguna vez igual a cero durante el intervalo de tiempo común de los argumentos.

```
SELECT tint '[2@2001-01-01, 2@2001-01-04]' + 1;
-- [3@2001-01-01, 3@2001-01-04]
SELECT tfloat '[2@2001-01-01, 2@2001-01-04]' + tfloat '[1@2001-01-01, 4@2001-01-04]';
-- [3@2001-01-01, 6@2001-01-04]
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' +
  tfloat '{[1@2001-01-01, 2@2001-01-02), [1@2001-01-02, 2@2001-01-04}}';
-- {[2@2001-01-01, 4@2001-01-04), [3@2001-01-02, 6@2001-01-04)}
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-04]' - tint '[2@2001-01-03, 2@2001-01-05]';
-- [-1@2001-01-03, -1@2001-01-04]
SELECT tfloat '[3@2001-01-01, 6@2001-01-04]' - tfloat '[2@2001-01-01, 2@2001-01-04]';
-- [1@2001-01-01, 4@2001-01-04]
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' * 2;
-- [2@2001-01-01, 8@2001-01-04]
SELECT tfloat '[1@2001-01-01, 4@2001-01-04]' * tfloat '[2@2001-01-01, 2@2001-01-04]';
-- [2@2001-01-01, 8@2001-01-04]
SELECT tfloat '[1@2001-01-01, 3@2001-01-03]' * '[3@2001-01-01, 1@2001-01-03]';
-- {[3@2001-01-01, 4@2001-01-02, 3@2001-01-03)}
```

```

SELECT 2 / tfloat '[1@2001-01-01, 3@2001-01-04]';
-- [2@2001-01-01, 0.666666666666667@2001-01-04]
SELECT tfloat '[1@2001-01-01, 5@2001-01-05]' / tfloat '[5@2001-01-01, 1@2001-01-05]';
-- {[0.2@2001-01-01, 1@2001-01-03, 2001-01-03, 5@2001-01-03, 2001-01-05]}
SELECT 2 / tfloat '[-1@2001-01-01, 1@2001-01-02]';
-- ERROR: Division by zero
SELECT tfloat '[-1@2001-01-04, 1@2001-01-05]' / tfloat '[-1@2001-01-01, 1@2001-01-05]';
-- [-2@2001-01-04, 1@2001-01-05]

```

- Devuelve el valor absoluto del número temporal

`abs(tnumber) → tnumber`

```

SELECT abs(tfloat '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03, 0@2001-01-04, 1@2001-01-05],
SELECT abs(tint '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 1@2001-01-05]

```

- Redondear al entero inferior o superior

`floor(tfloat) → tfloat`

`ceil(tfloat) → tfloat`

```

SELECT floor(tfloat '[0.5@2001-01-01, 1.5@2001-01-02]');
-- [0@2001-01-01, 1@2001-01-02]
SELECT ceil(tfloat '[0.5@2001-01-01, 0.6@2001-01-02, 0.7@2001-01-03]');
-- [1@2001-01-01, 1@2001-01-03]

```

- Redondear a un número de posiciones decimales

`round(tfloat, integer=0) → tfloat`

```

SELECT round(tfloat '[0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]', 2);
-- [0.79@2001-01-01, 2.36@2001-01-02]

```

- Convertir a grados o radianes

`degrees({float,tfloat}, normalize=false) → tfloat`

`radians(tfloat) → tfloat`

El parámetro adicional en la función `degrees` puede ser utilizado para normalizar los valores entre 0 y 360 grados.

```

SELECT degrees(pi() * 5);
-- 900
SELECT degrees(pi() * 5, true);
-- 180
SELECT round(degrees(tfloat '[0.7853981633974@2001-01-01, 2.3561944901923@2001-01-02]'));
-- [45@2001-01-01, 135@2001-01-02]
SELECT radians(tfloat '[45@2001-01-01, 135@2001-01-02]');
-- [0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]

```

- Devuelve la diferencia de valor entre instantes consecutivos del número temporal

`deltaValue(tnumber) → tnumber`

```

SELECT deltaValue(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, -1@2001-01-02, -1@2001-01-03]
SELECT deltaValue(tfloat ' {[1.5@2001-01-01, 2@2001-01-02, 1@2001-01-03],
[2@2001-01-04, 2@2001-01-05]} ');
/* Interp=Step; {[0.5@2001-01-01, -1@2001-01-02, -1@2001-01-03],
[0@2001-01-04, 0@2001-01-05]} */

```

- Devuelve la tendencia de un flotante temporal con interpolación lineal, que indica si su valor es creciente, constante o decreciente, representado, respectivamente, por 1, 0 y -1.

`trend(tfloat) → tint`

Nótese que la tendencia es NULL para secuencias instantáneas.

```
SELECT trend(tfloat '[1@2001-01-01, 2@2001-01-02, 4@2001-01-03, 4@2001-01-04, 3@2001-01-05, 2@2001-01-06]');
-- [1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]
SELECT trend(tfloat '[1@2001-01-01]');
-- NULL
```

- Devuelve la derivada sobre el tiempo del número flotante temporal en unidades por segundo

`derivative(tfloat) → tfloat`

El número flotante temporal debe tener interpolación lineal. Nótese que esta función corresponde a la función **speed** para los puntos temporales.

```
SELECT derivative(tfloat '{[0@2001-01-01, 10@2001-01-02, 5@2001-01-03],
[1@2001-01-04, 0@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[-10@2001-01-01, 5@2001-01-02, 5@2001-01-03],
[1@2001-01-04, 1@2001-01-05]} */
SELECT derivative(tfloat 'Interp=Step;[0@2001-01-01, 10@2001-01-02, 5@2001-01-03]');
-- ERROR: The temporal value must have linear interpolation
```

- Devuelve el área bajo la curva

`integral(tnumber) → float`

```
SELECT integral(tint '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1
SELECT integral(tfloat '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1.5
```

- Devuelve el promedio ponderado en el tiempo

`twAvg(tnumber) → float`

```
SELECT twAvg(tfloat '{[1@2001-01-01, 2@2001-01-03], [2@2001-01-04, 2@2001-01-06]}');
-- 1.75
```

- Devuelve el logaritmo natural y el logaritmo base 10 de un número flotante temporal

`ln(tfloat) → tfloat`

`log10(tfloat) → tfloat`

El número flotante temporal no puede ser cero o negativo

```
SELECT ln(tfloat '{[1@2001-01-01, 10@2001-01-02, 5@2001-01-03],
[1@2001-01-04, 1@2001-01-05]}');
/* {[0@2001-01-01, 2.302585092994046@2001-01-02, 1.6094379124341@2001-01-03],
[0@2001-01-04, 0@2001-01-05]} */
SELECT log10(tfloat 'Interp=Step;[-10@2001-01-01, 10@2001-01-02]');
-- ERROR: Cannot take logarithm of zero or a negative number
```

- Devuelve el exponencial (e elevado a la potencia dada) de un número flotante temporal

`exp(tfloat) → tfloat`

```
SELECT exp(tfloat '{[1@2001-01-01, 10@2001-01-02],
[1@2001-01-04, 1@2001-01-05]}');
/* {[2.718281828459045@2001-01-01, 22026.465794806718@2001-01-02],
[2.718281828459045@2001-01-04, 2.718281828459045@2001-01-05]} */
SELECT exp(tfloat '{-10@2001-01-01, 0@2001-01-02, 10@2001-01-03}');
-- {0.000045399929762@2001-01-01, 1@2001-01-02, 22026.465794806718@2001-01-03}
```


- Devuelve el seno, coseno o tangente de un número flotante temporal (argumento en radianes)

`sin(tfloat) → tfloat`

`cos(tfloat) → tfloat`

`tan(tfloat) → tfloat`

```
SELECT round(sin(tfloat '[0@2001-01-01, 6.283185307@2001-01-02]'), 6);
-- [0@2001-01-01, -0.000000@2001-01-02]
SELECT round(cos(tfloat '[0@2001-01-01, 3.141592654@2001-01-02]'), 6);
-- [1@2001-01-01, -1@2001-01-02]
SELECT round(tan(tfloat '{0@2001-01-01, 0.785398163@2001-01-02}'), 6);
-- {0@2001-01-01, 1@2001-01-02}
```

6.8. Operaciones de texto

- Concatenación de texto

`{text,ttext} || {text,ttext} → ttext`

```
SELECT ttext '[AA@2001-01-01, AA@2001-01-04]' || text 'B';
-- ["AAB"@2001-01-01, "AAB"@2001-01-04]
SELECT ttext '[AA@2001-01-01, AA@2001-01-04]' || ttext '[BB@2001-01-02, BB@2001-01-05]';
-- ["AABB"@2001-01-02, "AABB"@2001-01-04]
SELECT ttext '[A@2001-01-01, B@2001-01-03, C@2001-01-04]' ||
  ttext '{[D@2001-01-01, D@2001-01-02), [E@2001-01-02, E@2001-01-04]}';
-- [{"AD"@2001-01-01, "AE"@2001-01-02, "BE"@2001-01-03, "BE"@2001-01-04}]
```

- Transformar en minúsculas, mayúsculas o initcap

`lower(ttext) → ttext`

`upper(ttext) → ttext`

`initcap(ttext) → ttext`

```
SELECT upper(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["AA"@2001-01-01, "BB"@2001-01-02]
SELECT lower(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["aa"@2001-01-01, "bb"@2001-01-02]
SELECT initcap(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["aa"@2001-01-01, "bb"@2001-01-02]
```

Capítulo 7

Tipos temporales geométricos (Parte 1)

7.1. Tipos espaciotemporales

MobilityDB proporciona un conjunto de tipos espaciotemporales, a saber, `tgeometry` (geometría temporal), `tgeography` (geografía temporal), `tgeompoint` (punto geométrico temporal), `tgeogpoint` (punto geográfico temporal), `tcbuffer` (buffer circular temporal), `tnpoint` (punto de red temporal), `tpose` (pose temporal) y `trgeometry` (geometría rígida temporal). A nivel conceptual, estos tipos espaciotemporales se organizan en la jerarquía que se muestra en la Figura 7.1.

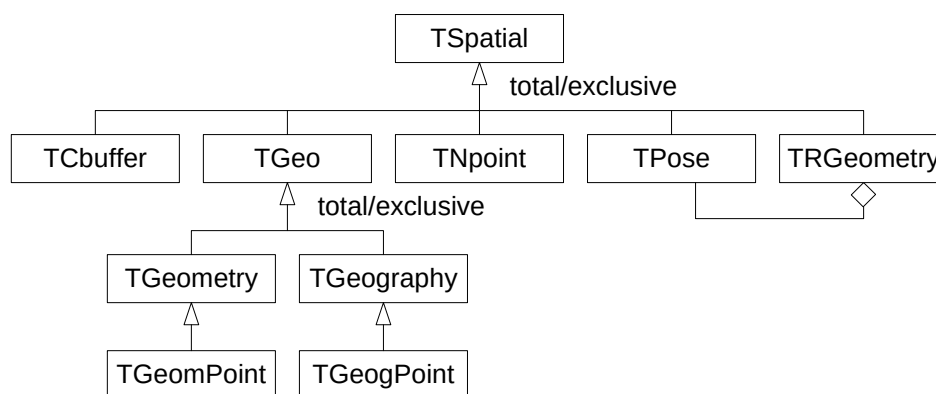


Figura 7.1: Jerarquía de tipos espaciotemporales en MobilityDB.

En este capítulo y el siguiente cubrimos el tipo `TGeo` y sus subtipos, es decir, los tipos espaciotemporales derivados de los tipos `geometry` y `geography` de PostGIS. En los capítulos siguientes, continuamos describiendo los tipos espaciotemporales restantes.

La Figura 7.2 ilustra el uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024. Los datos provienen de National Oceanic and Atmospheric Administration ([NOAA](#)). Como se ilustra en la figura, el tipo `tgeompoint` permite la interpolación *lineal*, mientras que el tipo `tgeometry` solo permite la interpolación *escalonada*.

7.2. Funciones sobre Geometrías

Las funciones de esta sección responden una pregunta sobre una geometría y no sobre un tipo temporal. Cada una indica cómo lee una geometría delimitada por arcos circulares, ya que una respuesta construida a partir de los arcos mismos y una construida a partir de una aproximación poligonal de ellos no son la misma respuesta.

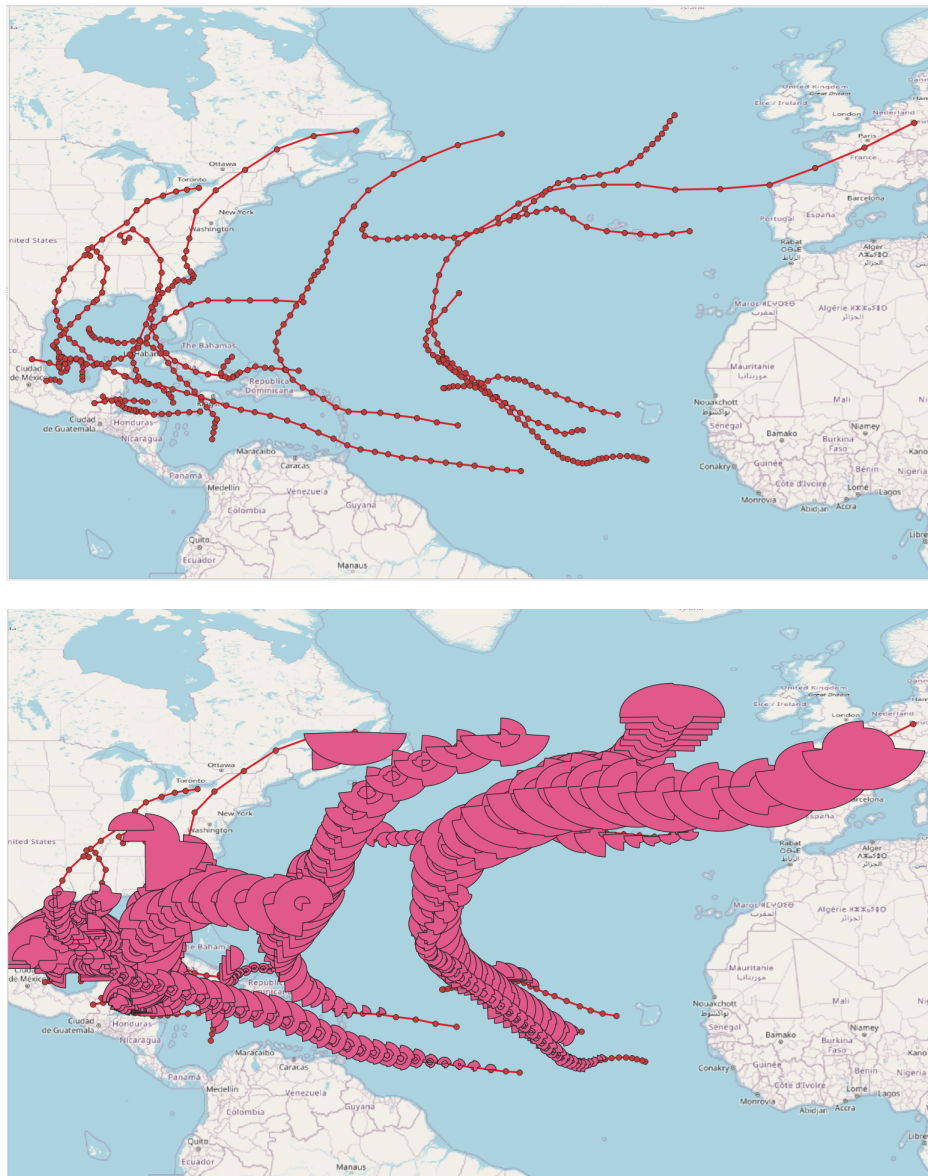


Figura 7.2: Ilustración del uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024.

- Devuelve el rectángulo de área mínima que encierra una geometría

`OrientedEnvelope(geometry) → geometry`

El rectángulo se orienta en el ángulo que lo hace más pequeño, que en general no es el ángulo de los ejes. Se devuelve como una línea cuando la geometría está contenida en una y como un punto cuando la geometría es uno. Una geometría que lleva un arco circular se responde sobre el arco mismo: el rectángulo se coloca contra el círculo en el que se encuentra el arco y no contra la cadena de segmentos que lo aproxima.

```
SELECT ST_AsText(round(OrientedEnvelope(geometry 'Polygon((0 0,3 4,-1 7,-4 3,0 0))'), 6));
-- POLYGON((0 0,3 4,-1 7,-4 3,0 0))
SELECT ST_AsText(round(OrientedEnvelope(geometry 'Linestring(0 0,10 0)'), 6));
-- LINESTRING(0 0,10 0)
SELECT ST_AsText(round(OrientedEnvelope(geometry 'Circularstring(0 0,0.5 0.1,1 0)'), 6));
-- POLYGON((1 0.1,0 0.1,0 0,1 0,1 0.1))
```

- Devuelve la geometría convexa más pequeña que encierra una geometría

`ConvexHull(geometry) → geometry`

El resultado es un polígono en el que cada esquina es un punto de la geometría, una línea cuando la geometría está contenida en una y un punto cuando la geometría es uno. Una geometría que lleva un arco circular se responde sobre el arco mismo, por lo que la envolvente está delimitada por el arco y se devuelve como una geometría curva y no como un polígono que la aproxima.

```
SELECT ST_AsText(round(ConvexHull(geometry 'Multipoint(0 0,10 0,10 10,0 10,5 5)'), 6));
-- POLYGON((0 0,0 10,10 10,10 0,0 0))
SELECT ST_AsText(round(ConvexHull(geometry 'Polygon((0 0,10 0,10 10,5 5,0 10,0 0))'), 6));
-- POLYGON((0 0,0 10,10 10,10 0,0 0))
SELECT ST_AsText(round(ConvexHull(geometry 'Circularstring(5 0,0 5,-5 0)'), 6));
-- CURVEPOLYGON(COMPOUNDCURVE((5 0,-5 0),CIRCULARSTRING(-5 0,0 5,5 0)))
```

- Devuelve verdadero si una geometría no tiene ningún punto en el que se cruce o se toque a sí misma

`isSimple(geometry) → boolean`

Un punto siempre es simple, un multipunto es simple cuando no repite ningún punto, una línea es simple cuando se encuentra consigo misma solo donde dos de sus segmentos se siguen y, cuando se cierra, en el punto donde se cierra, y una geometría de área es simple cuando cada uno de sus anillos lo es. Además, dos líneas de una multilínea pueden encontrarse en un punto que termina ambas. Una geometría que lleva un arco circular se encuentra consigo misma a lo largo de un arco y no en un punto, lo que la intersección de segmentos en la que esto se basa no lee, por lo que tal geometría se responde sobre la cadena de segmentos que aproxima sus arcos.

```
SELECT isSimple(geometry 'Linestring(0 0,10 10,10 0,0 10)');
-- false
SELECT isSimple(geometry 'Linestring(0 0,10 0,10 10,0 0)');
-- true
SELECT isSimple(geometry 'Multilinestring((0 0,10 0),(10 0,10 10))');
-- true
```

- Devuelve la geometría que contiene todos los puntos situados a una distancia dada de una geometría

`Buffer(geometry,float,options text='') → geometry`

La frontera del resultado está formada por los dos desplazamientos de la geometría a la distancia dada, por la unión que rellena el lado exterior de cada giro y por el extremo que cierra cada final de una línea abierta. Una distancia negativa contrae una geometría de área y devuelve una geometría vacía para una de cualquier otra dimensión.

La cadena `options` lleva los estilos como pares `clave=valor` separados por espacios: `endcap` es uno de `round`, `flat` o `square`, `join` uno de `round`, `mitre` o `bevel`, y `mitre_limit` la razón a partir de la cual una unión en inglete pasa a ser biselada.

```
SELECT ST_AsText(round(Buffer(geometry 'Point(0 0)', 1), 6));
-- CURVEPOLYGON(CIRCULARSTRING(-1 0,1 0,-1 0))
SELECT ST_AsText(round(Buffer(geometry 'Polygon((0 0,10 0,10 10,0 10,0 0))', -1, 'join= ←
mitre'), 6));
```

```
-- CURVEPOLYGON(COMPOUNDCURVE((1 1,9 1),(9 1,9 9),(9 9,1 9),(1 9,1 1)))
SELECT ST_AsText(round(Buffer(geometry 'Curvepolygon(Circularstring(0 0,2 2,4 0,2 -2,0 0)) ←
', 1), 6));
-- CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(-1 0,2 3,5 0),CIRCULARSTRING(5 0,2 -3,-1 0)))
```

7.3. Notación

Presentamos en la Sección 4.5 y en la Sección 6.1 la notación utilizada para definir la firma de las funciones y operadores para tipos temporales. A continuación, ampliamos estas notaciones para tipos espaciotemporales.

- `tspatial` representa un tipo espaciotemporal, por ejemplo, `tgeometry`, `tgeompoint`, `tpose`, o `tnpoint`,
- `tgeo` representa un tipo temporal geometría/geografía, es decir, `tgeometry`, `tgeography`, `tgeompoint`, o `tgeogpoint`,
- `tgeom` representa un tipo temporal geometría, es decir, `tgeometry` o `tgeompoint`,
- `tpoint` representa un tipo temporal punto, es decir, `tgeompoint` o `tgeogpoint`,
- `spatial` representa un tipo de base espacial, por ejemplo, `geometry`, `geography`, `pose`, o `npoint`,
- `geo` representa los tipos `geometry` o `geography`,
- `geompoint` representa el tipo `geometry` restringido a un punto.
- `point` representa los tipos `geometry` o `geography` restringidos a un punto.
- `lines` representa los tipos `geometry` o `geography` restringidos a una (multi)línea.

A continuación, se especifica con el símbolo  que la función admite geometrías en 3D y con el símbolo  que la función admite geografías.

7.4. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)  

```
asText({tspatial,tspatial[],spatial[]}) → {text,text[]}
```

```
asEWKT({tspatial,tspatial[],spatial[]}) → {text,text[]}
```

```
SELECT asText(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- [POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02]
SELECT asText(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
/* {"[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
  "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"} */
SELECT asText(ARRAY[geometry 'Point(0 0)', 'Point(1 1)']);
-- {"POINT(0 0)","POINT(1 1)"}
```

```
SELECT asEWKT(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- SRID=4326;[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02]
SELECT asEWKT(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
-- {"SRID=4326;[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
  "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"}
SELECT asEWKT(ARRAY[geometry 'SRID=5676;Point(0 0)', 'SRID=5676;Point(1 1)']);
-- {"SRID=5676;POINT(0 0)","SRID=5676;POINT(1 1)"}
```

- Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)  

`asMFJSON(tspatial, options=0, flags=0, maxdecdigits=15) → text`

El argumento `options` puede usarse para agregar BBOX y/o CRS en la salida MFJSON:

- 0: significa que no hay opción (valor por defecto)
- 1: MFJSON BBOX
- 2: MFJSON Short CRS (e.g., EPSG:4326)
- 4: MFJSON Long CRS (e.g., urn:ogc:def:crs:EPSG::4326)

El argumento `flags` puede usarse para personalizar la salida JSON, por ejemplo, para producir una salida JSON fácil de leer (para lectores humanos). Consulte la documentación de la biblioteca `json-c` para conocer los valores posibles. Los valores típicos son los siguientes:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

El argumento `maxdecdigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asMFJSON(tgeompoint 'Point(1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingPoint","coordinates":[[1,2]],"datetimes":["2019-01-01T17:00:00.15+01"],
   "interpolation":"None"} */
SELECT asMFJSON(tgeometry 'SRID=3812;LineString(1 1,1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingGeometry",
   "crs":{"type":"Name","properties":{"name":"EPSG:3812"}},
   "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]},
   "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"} */
SELECT asMFJSON(tgeompoint 'SRID=4326;
Point(50.813810 4.384260)@2019-01-01 18:00:00.15+02', 3, 0, 2);
/* {"type":"MovingPoint","crs":{"type":"name","properties":{"name":"EPSG:4326"}},
   "stBoundedBy":{"bbox":[50.81,4.38,50.81,4.38]},
   "period":{"begin":"2019-01-01 17:00:00.15+01","end":"2019-01-01 17:00:00.15+01"}},
   "coordinates":[[50.81,4.38]],"datetimes":["2019-01-01T17:00:00.15+01"],
   "interpolations":"None"} */
```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o la representación hexadecimal extendida binaria conocida (Extended Well-Known Binary o EWKB)  

`asBinary(tgeo, endian text="") → bytea`

`asEWKB(tgeo, endian text="") → bytea`

`asHexEWKB(tgeo, endian text="") → text`

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tgeompoint 'Point(1 2 3)@2001-01-01');
-- \x012e001101e9030000000000000000f03f000000000000040000000000000840009c57d3c11c0000
SELECT asEWKB(tgeogpoint 'SRID=7844;Point(1 2 3)@2001-01-01');
-- \x012f0071a41e0000000000000000f03f000000000000040000000000000840009c57d3c11c0000
SELECT asHexEWKB(tgeompoint 'SRID=3812;Point(1 2 3)@2001-01-01');
-- 012E0051E40E0000000000000000F03F000000000000040000000000000840009C57D3C11C0000
```

- Entrar a partir de la representación de texto conocido (Well-Known Text o WKT) o de la representación extendida de texto conocido (Extended Well-Known Text o EWKT)  

`tspatialFromText(text) → tspatial`

`tspatialFromEWKT(text) → tspatial`

En las funciones anteriores, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromText(text '[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]'));
-- [POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeogpointFromText(text
  '[Point(1 2)@2001-01-01, Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=4326;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```

```
SELECT asEWKT(tgeompointFromEWKT(text 'SRID=3812;[Point(1 2)@2001-01-01,
  Point(3 4)@2001-01-02]'));
-- SRID=3812;[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeogpointFromEWKT(text 'SRID=7844;[Point(1 2)@2001-01-01,
  Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=7844;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```

- Entrar a partir de la representación JSON de características móviles (Moving Features JSON o MF-JSON)  

`tspatialFromMFJSON(text) → tspatial`

En la función anterior, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromMFJSON(text '{"type":"MovingPoint","crs":{"type":"name",
  "properties":{"name":"EPSG:4326"}},'coordinates':[[50.81,4.38]],
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15+01
SELECT asEWKT(tgeogpointFromMFJSON(text '{"type":"MovingPoint","crs":{"type":"name",
  "properties":{"name":"EPSG:4326"}},'coordinates':[[50.81,4.38]],
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15+01
SELECT asEWKT(tgeogpointFromMFJSON(text '{"type":"MovingGeometry",
  "crs":{"type":"Name","properties":{"name":"EPSG:7844"}},
  "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]},
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=7844;LINESTRING(1 1,1 2)@2019-01-01 17:00:00.15+01
```

- Entrar a partir de su representación binaria conocida (WKB), de la representación extendida binaria conocida (EWKB), o de la representación hexadecimal extendida binaria conocida (HexEWKB)  

`tspatialFromBinary(bytea) → tspatial`

`tspatialFromEWKB(bytea) → tspatial`

`tspatialFromHexEWKB(text) → tspatial`

En las funciones anteriores, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromBinary(
  '\x012e0011000000000000f03f0000000000004000000000000840009c57d3c11c0000'));
-- POINT Z (1 2 3)@2001-01-01
SELECT asEWKT(tgeogpointFromEWKB(
  '\x012f0071a41e000000000000f03f0000000000004000000000000840009c57d3c11c0000'));
-- SRID=7844;POINT Z (1 1 1)@2001-01-01
SELECT asEWKT(tgeompointFromHexEWKB(
  '012E0051E40E000000000000f03F0000000000004000000000000840009C57D3C11C0000'));
-- SRID=3812;POINT(1 2 3)@2001-01-01
```

7.5. Conversión de tipos

- Convertir un valor espaciotemporal a un rango temporal o a un cuadro delimitador espaciotemporal

`tspatial::stbox`

`stbox(tspatial)`

```
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::stbox;
-- STBOX XT((1,1),(3,3)), [2001-01-01, 2001-01-03])
SELECT tgeography '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03]'::stbox;
-- SRID=4326;GEODSTBOX ZT((1,1,1),(3,3,3)), [2001-01-01, 2001-01-03])
```

■ Convertir entre una geometría temporal y una geografía temporal

```
tgeometry::tgeography
tgeography::tgeometry
tgeompoint::tgeogpoint
tgeogpoint::tgeompoint
```

```
SELECT asText((tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]')::tgeogpoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02]
SELECT asText((tgeography 'LineString(0 0,1 1)@2001-01-01')::tgeometry);
-- LINESTRING(0 0,1 1)@2001-01-01
```

Una forma común de almacenar puntos temporales en PostGIS es representarlos como geometrías de tipo `LINESTRING` y utilizar la dimensión M para codificar marcas de tiempo como segundos desde 1970-01-01 00:00:00. Estas geometrías aumentadas con tiempo, llamadas **trayectorias**, se pueden validar con la función `ST_IsValidTrajectory` para verificar que el valor M está creciendo de cada vértice al siguiente. Las trayectorias se pueden manipular con las funciones `ST_ClosestPointOfApproach`, `ST_DistanceCPA` y `ST_CPAWithin`. Los valores de puntos temporales se pueden convertir a/desde trayectorias de PostGIS.

■ Convertir entre un punto temporal y una trayectoria PostGIS

```
tpoint::geo
geo::tpoint
```

```
SELECT ST_AsText((tgeompoint 'Point(0 0)@2001-01-01')::geometry);
-- POINT M (0 0 978307200)
SELECT ST_AsText((tgeompoint '{Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
Point(1 1)@2001-01-03}'))::geometry);
-- MULTIPOINT M (0 0 978307200,1 1 978393600,1 1 978480000)
SELECT ST_AsText((tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02]')::geometry);
-- LINESTRING M (0 0 978307200,1 1 978393600)
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02],
[Point(1 1)@2001-01-03, Point(1 1)@2001-01-04],
[Point(1 1)@2001-01-05, Point(0 0)@2001-01-06]}'))::geometry);
/* MULTILINESTRING M ((0 0 978307200,1 1 978393600),(1 1 978480000,1 1 978566400),
(1 1 978652800,0 0 978739200)) */
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02],
[Point(1 1)@2001-01-03],
[Point(1 1)@2001-01-05, Point(0 0)@2001-01-06]}'))::geometry);
/* GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200)) */
```

```
SELECT asText(geometry 'LINESTRING M (0 0 978307200,0 1 978393600,
1 1 978480000)'::tgeompoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02, POINT(1 1)@2001-01-03]
SELECT asText(geometry 'GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200))'::tgeompoint);
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(1 1)@2001-01-03],
[POINT(1 1)@2001-01-05, POINT(0 0)@2001-01-06]} */
```


7.6. Accessores

- Obtener la trayectoria o el área atravesada  

`trajectory(tpoint, unary_union=false) → geo`

`traversedArea(tgeo, unary_union=false) → geo`

Esta función es equivalente a `getValues` para los valores temporales alfanuméricos. El último argumento indica si se aplica la función PostGIS `ST_UnaryUnion` para eliminar geometrías redundantes en el resultado. Tenga en cuenta que establecer este argumento como verdadero implica un alto consumo computacional.

```
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
Point(0 0)@2001-01-03]'));
-- LINESTRING(0 0,0 1,0 0)
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
Point(0 0)@2001-01-03]', true));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint '{[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02],
[Point(0 1)@2001-01-03, Point(0 0)@2001-01-04]}'));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint 'Interp=Step;{[Point(0 0)@2001-01-01,
Point(0 1)@2001-01-02], [Point(0 1)@2001-01-03, Point(1 1)@2001-01-04]}'));
-- MULTIPOINT((0 0),(0 1),(1 1))
```

```
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(1 1,2 2))
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]', true));
-- LINESTRING(1 1,2 2)
```

- Devuelve el centroide como un punto temporal 

`centroid(tgeo) → tpoint`

```
SELECT asText(centroid(tgeometry '[Point(1 1)@2000-01-01, Linestring(1 1,3 3)@2000-01-02,
Polygon((1 1,4 4,7 1,1 1))@2000-01-03]'));
-- Interp=Step;[POINT(1 1)@2000-01-01, POINT(2 2)@2000-01-02, POINT(4 2)@2000-01-03]
SELECT asText(centroid(tgeography '[MultiPoint(1 1,4 4,7 1)@2000-01-01,
Polygon((1 1,4 4,7 1,1 1))@2000-01-02]', 6));
-- Interp=Step;[POINT(4 2.001727)@2000-01-01, POINT(4 2.001727)@2000-01-02]
```

- Devuelve los valores de las coordenadas X/Y/Z como un número flotante temporal  

`getX(tpoint) → tfloat`

`getY(tpoint) → tfloat`

`getZ(tpoint) → tfloat`

```
SELECT getX(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
Point(5 6)@2001-01-03}');
-- {1@2001-01-01, 3@2001-01-02, 5@2001-01-03}
SELECT getX(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
Point(7 8 9)@2001-01-03]');
-- Interp=Step;[1@2001-01-01, 4@2001-01-02, 7@2001-01-03]
SELECT getY(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
Point(5 6)@2001-01-03}');
-- {2@2001-01-01, 4@2001-01-02, 6@2001-01-03}
SELECT getY(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
Point(7 8 9)@2001-01-03]');
-- Interp=Step;[2@2001-01-01, 5@2001-01-02, 8@2001-01-03]
```

```
SELECT getZ(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- The temporal point must have Z dimension
SELECT getZ(tgeogpoint 'Interp=Step;[Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02,
  Point(7 8 9)@2001-01-03]');
-- Interp=Step;[3@2001-01-01, 6@2001-01-02, 9@2001-01-03]
```

- Devuelve verdadero si el punto temporal no se auto-intersecta espacialmente 

isSimple(tpoint) → boolean

Nótese que un punto temporal de conjunto de secuencias es simple si cada una de las secuencias que lo componen es simple.

```
SELECT isSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]');
-- false
SELECT isSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]');
-- false
SELECT isSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- true
```

- Devuelve la longitud atravesada por el punto temporal  

length(tpoint) → float

```
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- 1.73205080756888
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(0 0 0)@2001-01-03]');
-- 3.46410161513775
SELECT length(tgeompoint 'Interp=Step;[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- 0
```

- Devuelve la longitud acumulada atravesada por el punto temporal  

cumulativeLength(tpoint) → tfloatSeq

```
SELECT round(cumulativeLength(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}'), 6);
-- {[0@2001-01-01, 1.414214@2001-01-02, 2.414214@2001-01-03],
  [2.414214@2001-01-04, 3.414214@2001-01-05]}
SELECT cumulativeLength(tgeogpoint 'Interp=Step;[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- Interp=Step;[0@2001-01-01, 0@2001-01-03]
```

- Devuelve la velocidad del punto temporal en unidades por segundo  

speed(tpoint) → tfloatSeqSet

El punto temporal debe tener interpolación lineal

```
SELECT speed(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}') * 3600 * 24;
/* Interp=Step;[1.4142135623731@2001-01-01, 1@2001-01-02, 1@2001-01-03],
  [1@2001-01-04, 1@2001-01-05]} */
SELECT speed(tgeogpoint 'Interp=Step;[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03]');
-- ERROR: The temporal value must have linear interpolation
```

- Devuelve el centroide ponderado en el tiempo

twCentroid(tgeompoint) → point

```
SELECT ST_AsText(twCentroid(tgeompoint '{[Point(0 0 0)@2001-01-01,
Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
-- POINT Z (0 0.6666666666666667 0.6666666666666667)
```

- Devuelve la dirección, es decir, el acimut entre las ubicaciones inicial y final

direction(tpoint) → float

El resultado se expresa en radianes. Es NULL si solo hay una ubicación o si las ubicaciones inicial y final son iguales.

```
SELECT round(degrees(direction(tgeompoint '[Point(0 0)@2001-01-01,
Point(-1 -1)@2001-01-02, Point(1 1)@2001-01-03]')),::numeric, 6);
-- 45.000000
SELECT direction(tgeompoint '{[Point(0 0 0)@2001-01-01,
Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- NULL
```

- Devuelve el acimut temporal

azimuth(tpoint) → tfloat

El resultado se expresa en radianes. El azimut es indefinido cuando dos localizaciones sucesivas son iguales y en este caso se añade una brecha de tiempo.

```
SELECT round(degrees(azimuth(tgeompoint '[Point(0 0 0)@2001-01-01,
Point(1 1 1)@2001-01-02, Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]')));
-- Interp=Step;{[45@2001-01-01, 45@2001-01-02], [225@2001-01-03, 225@2001-01-04]}
```

- Devuelve la diferencia angular temporal

angularDifference(tpoint) → tfloat

El resultado se expresa en grados.

```
SELECT round(angularDifference(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(1 1)@2001-01-03]'), 3);
-- {0@2001-01-01, 180@2001-01-02, 0@2001-01-03}
SELECT round(degrees(angularDifference(tgeompoint '{[Point(1 1)@2001-01-01,
Point(2 2)@2001-01-02], [Point(2 2)@2001-01-03, Point(1 1)@2001-01-04]}')), 3);
-- {0@2001-01-01, 0@2001-01-02, 0@2001-01-03, 0@2001-01-04}
```

- Devuelve el rumbo temporal

bearing({tpoint,point},{tpoint,point}) → tfloat

Nótese que esta función no acepta dos puntos geográficos temporales.

```
SELECT degrees(bearing(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]',
geometry 'Point(2 2)'));
-- [45@2001-01-01, 0@2001-01-02, 225@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]')), 3);
-- [63.435@2001-01-01, 0@2001-01-02, 296.565@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]',
tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]')), 3);
-- [243.435@2001-01-01, 116.565@2001-01-03]
```

7.7. Transformaciones

- Redondear los valores de las coordenadas a un número de decimales  

`round(tspatial,integer=0) → tgeo`

```
SELECT asText(round(tgeompoint '{Point(1.12345 1.12345 1.12345)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(1.12345 1.12345 1.12345)@2001-01-03}', 2));
/* {POINT Z (1.12 1.12 1.12)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
  POINT Z (1.12 1.12 1.12)@2001-01-03} */
SELECT asText(round(tgeography 'Linestring(1.12345 1.12345,2.12345 2.12345)@2001-01-01', ←
  2));
-- LINESTRING(1.12 1.12,2.12 2.12)@2001-01-01
```

- Devuelve una matriz de fragmentos del punto temporal que son simples 

`makeSimple(tpoint) → tgeompoint[]`

```
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02]",
  "[POINT(1 1)@2001-01-02, POINT(0 0)@2001-01-03]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]'));
/* {"[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 0 2)@2001-01-03,
  POINT Z (0 0 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 1)@2001-01-03, Point(1 0)@2001-01-04]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02, POINT(0 1)@2001-01-03]",
  "[POINT(0 1)@2001-01-03, POINT(1 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
/* {"{[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
  [POINT Z (1 1 1)@2001-01-03, POINT Z (0 0 0)@2001-01-04]}"} */
```

- Construir una geometría/geografía con medida M a partir de un punto temporal y un número flotante temporal  

`geoMeasure(tpoint,tfloat,segmentize=false) → geo`

El último argumento `segmentize` establece si el valor resultado ya sea es un `Linestring` Mo un `MultiLinestring` M donde cada componente es un segmento de dos puntos.

```
SELECT ST_AsText(geoMeasure(tgeompoint '{Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02}', '{5@2001-01-01, 5@2001-01-02}'));
-- MULTIPOINT ZM (1 1 1 5,2 2 2 5)
SELECT ST_AsText(geoMeasure(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04]}',
  '{[5@2001-01-01, 5@2001-01-02],[7@2001-01-03, 7@2001-01-04]}'));
-- GEOMETRYCOLLECTION M (POINT M (1 1 7),LINESTRING M (1 1 5,2 2 5))
SELECT ST_AsText(geoMeasure(tgeompoint '[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]',
  '[5@2001-01-01, 7@2001-01-02, 5@2001-01-03]', true));
-- MULTILINESTRING M ((1 1 5,2 2 5),(2 2 7,1 1 7))
```

Una visualización típica de los datos de movilidad es mostrar en un mapa la trayectoria del objeto móvil utilizando diferentes colores según la velocidad. La Figura 7.3 muestra el resultado de la consulta a continuación usando una rampa de color en QGIS.

```
WITH Temp(t) AS (
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-05,
    Point(2 0)@2001-01-08, Point(3 1)@2001-01-10, Point(4 0)@2001-01-11]' )
SELECT ST_AsText(geoMeasure(t, round(speed(t) * 3600 * 24, 2), true))
```

```
FROM Temp;
/* MULTILINESTRING M ((0 0 0.35,1 1 0.35),(1 1 0.47,2 0 0.47),(2 0 0.71,3 1 0.71),
(3 1 1.41,4 0 1.41)) */
```

La siguiente expresión se usa en QGIS para lograr esto. La función `scale_linear` transforma el valor `M` de cada segmento componente al rango `[0, 1]`. Este valor luego se pasa a la función `ramp_color`.

```
ramp_color('RdYlBu', scale_linear(
  m(start_point(geometry_n($geometry,@geometry_part_num)),
    0, 2, 0, 1) )
```



Figura 7.3: Visualización de la velocidad de un objeto móvil usando una rampa de color en QGIS.

- Devuelve la transformación afín 3D de una geometría temporal para traducirla, rotarla y/o escalarla en un solo paso 

```
affine(tgeo,float a,float b,float c,float d,float e,float f,float g,
float h,float i,float xoff,float yoff,float zoff) → tgeo
affine(tgeo,float a,float b,float d,float e,float xoff,float yoff) → tgeo
```

```
-- Rotate a 3D temporal point 180 degrees about the z axis
SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), 0, 0, 0, 1,
0, 0, 0))
FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
SELECT asEWKT(rotate(temp, pi()))
FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
-- Rotate a 3D temporal point 180 degrees in both the x and z axis
SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), -sin(pi()),
0, sin(pi()), cos(pi()), 0, 0, 0))
FROM (SELECT tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02]' AS temp) t;
-- [POINT(-1 -1)@2001-01-01, LINESTRING(-1 -1,-2 -2)@2001-01-02]
```

- Devuelve el punto temporal rotado en sentido antihorario sobre el punto de origen

```
rotate(tgeo,float radians) → tgeo
rotate(tgeo,float radians,float x0,float y0) → tgeo
rotate(tgeo,float radians,geometry pointOrigin) → tgeo
```

```
-- Rotate a temporal point 180 degrees
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()), 6);
-- [POINT(-5 -10)@2001-01-01, POINT(-5 -5)@2001-01-02, POINT(-10 -5)@2001-01-03]
-- Rotate 30 degrees counter-clockwise at x=5, y=10
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()/6, 5, 10), 6);
-- [POINT(5 10)@2001-01-01, POINT(7.5 5.67)@2001-01-02, POINT(11.83 8.17)@2001-01-03]
-- Rotate 60 degrees clockwise from centroid
SELECT asEWKT(rotate(temp, -pi()/3, ST_Centroid(traversedArea(temp))), 2)
FROM (SELECT tgeometry '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Linestring(5 5,10 5)@2001-01-03]' AS temp) AS t;
/* [POINT(10.58 9.67)@2001-01-01, POINT(6.25 7.17)@2001-01-02,
  LINESTRING(6.25 7.17,8.75 2.83)@2001-01-03] */
```

■ Devuelve un punto temporal escalado por factores dados

```
scale(tgeo,float Xfactor,float Yfactor,float Zfactor) → tgeo
scale(tgeo,float Xfactor,float Yfactor) → tgeo
scale(tgeo,geometry factor) → tgeo
scale(tgeo,geometry factor,geometry origin) → tgeo
```

```
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
  0.5, 0.75, 0.8));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
  0.5, 0.75));
-- [POINT Z (0.5 1.5 3)@2001-01-01, POINT Z (0.5 0.75 1)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
  geometry 'Point(0.5 0.75 0.8)'));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  geometry 'Point(2 2)', geometry 'Point(1 1)'));
-- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]
```

■ Transformar un punto geométrico temporal en el espacio de coordenadas de un Mapbox Vector Tile

```
asMVTGeom(tpoint,bounds,extent=4096,buffer=256,clip=true) → (geom,times)
```

El resultado es un par compuesto de un valor `geometry` y una matriz de valores de marca de tiempo asociados codificados como época de Unix. Los parámetros son los siguientes:

- `tpoint` es el punto temporal para transformar
- `bounds` es un `stbox` que define los límites geométricos del contenido del mosaico sin búfer
- `extent` es la extensión del mosaico en el espacio de coordenadas del mosaico
- `buffer` es la distancia del búfer en el espacio de coordenadas de mosaico
- `clip` es un booleano que determina si las geometrías resultantes y las marcas de tiempo deben recortarse o no

```
SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
  stbox 'STBOX X((40,40),(60,60))') AS mvt ) AS t;
-- LINESTRING(-256 4352,4352 -256) | {946714680,946734120}
SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
  stbox 'STBOX X((40,40),(60,60))', clip:=false) AS mvt ) AS t;
-- LINESTRING(-8192 12288,12288 -8192) | {946681200,946767600}
```

■ Extraer de un punto temporal con interpolación lineal las subsecuencias donde el punto permanece dentro de un área con un tamaño máximo especificado durante al menos la duración dada

```
stops(tpoint,maxDist=0.0,minDuration='0 minutes') → tpoint
```

El tamaño del área se calcula como la diagonal del rectángulo mínimo rotado de los puntos de la subsecuencia. Si no se especifica `maxDist`, se asume 0.0 y, por lo tanto, la función extrae los segmentos constantes del punto temporal dado. La distancia se calcula en las unidades del sistema de coordenadas. Tenga en cuenta que, aunque la función acepta geometrías 3D, el cálculo siempre se realiza en 2D.

```
SELECT asText(stops(tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02,
  Point(2 2)@2001-01-03, Point(2 2)@2001-01-04]'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(2 2)@2001-01-03,
  POINT(2 2)@2001-01-04]} */
SELECT asText(stops(tgeompoint '[Point(1 1 1)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 2 2)@2001-01-03, Point(2 2 2)@2001-01-04]', 1.75));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03,
  POINT Z (2 2 2)@2001-01-04]} */
```

Capítulo 8

Tipos temporales geométricos (Parte 2)

8.1. Restricciones

- Restringir a (al complemento de) una geometría 

`atGeometry(tgeom, geometry) → tgeom`

`minusGeometry(tgeom, geometry) → tgeom`

La geometría debe ser 2D y el cálculo con respecto a ella se realiza en 2D. El resultado conserva la dimensión Z del punto temporal, si existe.

```
SELECT asText(atGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {"[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]"}
SELECT asText(atGeometry(tgeompoint '[Point(0 0 0)@2001-01-01, Point(4 4 4)@2001-01-05]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {[POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03]}
SELECT asText(atGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (2 1 1)@2001-01-02, POINT Z (3 1 1)@2001-01-03, POINT Z (3 1 3)@2001-01-05]}
SELECT asText(atGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(1 1,5 1)@2001-01-01
```

```
SELECT asText(minusGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02), (POINT(2 2)@2001-01-03,
  POINT(3 3)@2001-01-04)} */
SELECT asText(minusGeometry(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(4 4 4)@2001-01-05]', geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
  (POINT Z (2 2 2)@2001-01-03, POINT Z (4 4 4)@2001-01-05)} */
SELECT asText(minusGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 1 1)@2001-01-02)}
SELECT asText(minusGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(5 1,10 1)@2001-01-01
```

- Restringir a (al complemento de) un stbox 

`atStbox(tgeom, stbox, borderInc bool=true) → tgeompoint`

`minusStbox(tgeom, stbox, borderInc bool=true) → tgeompoint`

El tercer argumento opcional se utiliza para mosaicos multidimensionales (ver Sección 9.6) para excluir el borde superior de los mosaicos cuando un valor temporal se divide en varios mosaicos, de modo que todos los fragmentos de la geometría temporal sean exclusivos.

```
SELECT asText(atStbox(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]',
  stbox 'STBOX XT(((0,0), (2,2)), [2001-01-02, 2001-01-04]))');
-- {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]}
SELECT asText(atStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2), (3,3,3))'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03, POINT Z (3 3 2)@2001-01-04,
  POINT Z (3 3 3)@2001-01-05]} */
SELECT asText(atStbox(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-03,
  Point(2 2)@2001-01-04, Linestring(3 3,4 4)@2001-01-09]', stbox 'STBOX X((2,2), (3,3))'));
-- {[LINESTRING(2 2,3 3)@2001-01-03, POINT(2 2)@2001-01-04, POINT(3 3)@2001-01-09]}
```

```
SELECT asText(minusStbox(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04]',
  stbox 'STBOX XT(((1,1), (2,2)), [2001-01-03,2001-01-04]))');
-- {[POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03]}
SELECT asText(minusStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2), (3,3,3))'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
  (POINT Z (3 3 3)@2001-01-05, POINT Z (3 3 7)@2001-01-09)} */
SELECT asText(minusStbox(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,3 3)@2001-01-03, Point(2 2)@2001-01-04,
  Linestring(1 1,4 4)@2001-01-09]', stbox 'STBOX X((2,2), (3,3))'));
/* {[POINT(1 1)@2001-01-01, LINESTRING(1 1,2 2)@2001-01-03,
  LINESTRING(1 1,2 2)@2001-01-04), [MULTILINESTRING((1 1,2 2), (3 3,4 4))@2001-01-09]} */
```

■ Restringir a (al complemento de) un rango de altitud

atElevation(tgeom,zspan) → tgeom

minusElevation(tgeom,zspan) → tgeom

```
SELECT astext(atElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
  [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
SELECT astext(minusElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
  [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
```

8.2. Sistema de referencia espacial

■ Devuelve o establece el identificador de referencia espacial

SRID(tgeo) → integer

setSRID(tgeo) → tgeo

```
SELECT SRID(tgeompoint 'Point(0 0)@2001-01-01');
-- 0
SELECT asEWKT(setSRID(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02]', 4326));
-- SRID=4326;[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02]
```

■ Transformar a una referencia espacial diferente

transform(tgeo,to_srid integer) → tgeo

transformPipeline(tgeo,pipeline text,to_srid integer,is_forward bool=true) → tgeo

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el punto temporal tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del punto temporal de entrada se ignora y el SRID del punto temporal de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(tgeompoint 'SRID=4326;Point(4.35 50.85)@2001-01-01', 3812));
-- SRID=3812;POINT(648679.018035303 671067.055638114)@2001-01-01
```

```
WITH test(tgeo, pipeline) AS (
  SELECT tgeopoint 'SRID=4326;{Point(4.3525 50.846667 100.0)@2001-01-01,
    Point(-0.1275 51.507222 100.0)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(tgeo, pipeline, 4326),
    pipeline, 4326, false), 6)
FROM test;
/* SRID=4326;{POINT Z (4.3525 50.846667 100)@2001-01-01,
  POINT Z (-0.1275 51.507222 100)@2001-01-02} */
```

8.3. Operaciones de cuadro delimitador

- Devuelve el cuadro delimitador espaciotemporal expandido en la dimensión espacial por un valor flotante  
`expandSpace({geo,tgeo},float) → stbox`

```
SELECT expandSpace(geography 'Linestring(0 0,1 1)', 2);
-- SRID=4326;GEODSTBOX X((-2,-2),(3,3))
SELECT expandSpace(tgeompoint 'Point(0 0)@2001-01-01', 2);
-- STBOX XT((-2,-2),(2,2),[2001-01-01,2001-01-01])
```

8.4. Operaciones de distancia

- Devuelve la distancia más pequeña que haya existido  
`{geo,tgeo,stbox} |=| {geo,tgeo,stbox} → float`

```
SELECT tgeompoint '[Point(0 0)@2001-01-02, Point(1 1)@2001-01-04, Point(0 0)@2001-01-06]'
  |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03, Point(0 0)@2001-01-05]'
  |=| tgeompoint '[Point(2 0)@2001-01-02, Point(1 1)@2001-01-04, Point(2 2)@2001-01-06]';
-- 0.5
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05]' |=| tgeompoint '[Point(2 0 0)@2001-01-02,
  Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]';
-- 0.5
SELECT tgeometry '(Point(1 1)@2001-01-01, Linestring(3 1,1 1)@2001-01-03)' |=|
  geometry 'Linestring(1 3,2 2,3 3)';
-- 1
```

El operador `|=|` se puede utilizar para realizar una búsqueda de vecino más cercano utilizando un índice GiST o SP-GIST (ver la Sección 10.2). Esta función corresponde a la función `ST_DistanceCPA` proporcionada por PostGIS, aunque este última requiere que ambos argumentos sean una trayectoria.

Cuando un argumento es una caja espaciotemporal que lleva un período, la distancia se mide desde la parte del valor temporal que está dentro de ese período, y el resultado es `NULL` cuando ninguna parte de él lo está. Una caja que no lleva período mide el valor temporal entero.

```
SELECT ST_DistanceCPA(
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]':geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]':geometry);
-- 0.5
```

- Devuelve el instante del primer punto temporal en el que los dos argumentos están a la distancia más cercana  
`nearestApproachInstant({geo,tgeo},{geo,tgeo}) → tgeo`

La función sólo devuelve el primer instante que encuentre si hay más de uno. El instante resultante puede tener un límite exclusivo.

```
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(2 1)@2001-01-02
SELECT asText(nearestApproachInstant(tgeompoint 'Interp=Step;(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-03]', tgeompoint '(Point(1 1)@2001-01-01, Point(4 1)@2001-01-03]'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,1 1 1)@2001-01-02
```

La función `nearestApproachInstant` generaliza the la función PostGIS `ST_ClosestPointOfApproach`. Primero, la última función requiere que ambos argumentos sean trayectorias. Segundo, la función `nearestApproachInstant` devuelve tanto el punto como la marca de tiempo del punto de aproximación más cercano, mientras que la función PostGIS sólo proporciona la marca de tiempo como se muestra a continuación.

```
SELECT to_timestamp(ST_ClosestPointOfApproach(
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
    Point(0 0 0)@2001-01-05]':geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
    Point(2 2 2)@2001-01-06]':geometry));
-- 2001-01-03 12:00:00+00
```

- Devuelve la distancia espacial mínima entre dos geometrías temporales sobre la unión de sus extensiones temporales
`minDistance({tgeo,geo},{tgeo,geo}) → float`
`minDistance(tgeo[],tgeo[]) → float`
`minDistance(tgeo,tgeo) → float (agregado)`

La forma escalar devuelve la distancia mínima alcanzada entre los dos argumentos sobre la intersección de sus extensiones temporales, equivalente al valor mínimo de la distancia temporal `<->`. La forma de arreglo generaliza esto a dos conjuntos de geometrías temporales y devuelve el mínimo sobre todos los pares. La forma de agregado, utilizada con `GROUP BY`, devuelve la distancia mínima alcanzada sobre todos los pares agregados en el grupo.

```
SELECT minDistance(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]',
  geometry 'Point(0 1)');
-- 0.707106781186548
SELECT minDistance(
```

```

    ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]',
      tgeompoint '[Point(5 5)@2001-01-01, Point(6 6)@2001-01-03]'],
    ARRAY[tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03]']]);
-- 0
SELECT g, minDistance(t1.Trip, t2.Trip)
FROM Trips t1, Trips t2, Groups g
WHERE t1.GroupId = g AND t2.GroupId = g AND t1.TripId < t2.TripId
GROUP BY g;

```

La forma de agregado se adapta bien a las consultas tipo BerlinMOD que calculan la distancia mínima de aproximación entre pares de trayectorias agrupados por licencia o tipo de vehículo. En un grupo de una sola fila, el agregado se reduce a la distancia mínima escalar entre el par.

- Devuelve los pares de índices que cumplen la relación entre dos arreglos de geometrías temporales, generalizando las relaciones espaciales escalares a una unión espacial conjunto-conjunto

```

eDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer)
aDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer)
eIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
aIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
eDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
aDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer)
eTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer)
aTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer)
tDwithinPairs(tgeo[],tgeo[],float) → setof(i integer, j integer, periods tstzspanset)
tIntersectsPairs(tgeo[],tgeo[]) → setof(i integer, j integer, periods tstzspanset)
tDisjointPairs(tgeo[],tgeo[]) → setof(i integer, j integer, periods tstzspanset)
tTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer, j integer, periods tstzspanset)

```

Cada función resuelve el producto cruzado podado de los dos arreglos de entrada en una sola llamada y devuelve los índices basados en uno *i* y *j* de los pares que cumplen la relación. Las funciones con prefijo *e* devuelven los pares para los que la relación se cumple alguna vez, las funciones con prefijo *a* los pares para los que se cumple siempre, y las funciones con prefijo *t* devuelven además los períodos durante los cuales se cumple. La caja espaciotemporal de cada entrada se usa como prefiltro de caja envolvente, de modo que la relación exacta se calcula solo sobre los pares candidatos que sobreviven, y los pares cuyas extensiones temporales no se solapan se excluyen, igual que en las relaciones escalares. Las funciones están definidas para puntos temporales y geometrías temporales tanto en el caso planar como en el geodésico, excepto la relación de toca, que está definida solo para geometrías planares. Los índices se relacionan con las identidades originales mediante un `array_agg(identidad ORDER BY ...)` paralelo.

```

SELECT i, j FROM eDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
    tgeompoint '[Point(50 50)@2001-01-01, Point(50 60)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- 1 | 1
SELECT i, j, periods FROM tDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- un par, con el período durante el cual está dentro de la distancia
SELECT i, j FROM aDisjointPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(5 0)@2001-01-01, Point(5 10)@2001-01-02]']]);
-- 1 | 1

```

- Devuelve la línea que conecta el punto de aproximación más cercano  

```
shortestLine({geo,tgeo},{geo,tgeo}) → geo
```

La función sólo devolverá la primera línea que encuentre si hay más de una.

```

SELECT ST_AsText(shortestLine(tgeompoint ' (Point(1 1)@2001-01-01,
    Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(2 1,2 2)
SELECT ST_AsText(shortestLine(tgeompoint 'Interp=Step;(Point(1 1)@2001-01-01,
    Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(1 1,2 2)
SELECT ST_AsText(shortestLine(tgeometry
    '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
    '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,2 0 0)

```

La función `shortestLine` se puede utilizar para obtener el resultado proporcionado por la función PostGIS `ST_CPAWithin` cuando ambos argumentos son trayectorias como se muestra a continuación.

```

SELECT ST_Length(shortestLine(
    tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
        Point(0 0 0)@2001-01-05]',
    tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
        Point(2 2 2)@2001-01-06]')) <= 0.5;
-- true
SELECT ST_CPAWithin(
    tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
        Point(0 0 0)@2001-01-05]':geometry,
    tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
        Point(2 2 2)@2001-01-06]':geometry, 0.5);
-- true

```

El operador de distancia temporal, denotado $\leftrightarrow$, calcula la distancia en cada instante de la intersección de las extensiones temporales de sus argumentos y da como resultado un número flotante temporal. Calcular la distancia temporal es útil en muchas aplicaciones de movilidad. Por ejemplo, un grupo en movimiento (también conocido como convoy o bandada) se define como un conjunto de objetos que se mueven cerca unos de otros durante un intervalo de tiempo prolongado. Esto requiere calcular la distancia temporal entre dos objetos en movimiento.

El operador de distancia temporal acepta una geometría/geografía restringida a un punto o un punto temporal como argumentos. Observe que los tipos temporales sólo consideran la interpolación lineal entre valores, mientras que la distancia es una raíz de una función cuadrática. Por lo tanto, el operador de distancia temporal proporciona una aproximación lineal del valor de distancia real para los puntos de secuencia temporal. En este caso, los argumentos se sincronizan en la dimensión de tiempo y para cada uno de los segmentos de línea que componen los argumentos, se calcula la distancia espacial entre el punto inicial, el punto final y el punto de aproximación más cercano, como se muestra en los ejemplos a continuación.

■ Devuelve la distancia temporal

$\{\text{geo}, \text{tgeo}\} \leftrightarrow \{\text{geo}, \text{tgeo}\} \rightarrow \text{tfloat}$

```

SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]' <->
    geometry 'Point(0 1)';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03]
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]' <->
    tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03]';
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03]
SELECT tgeompoint '[Point(0 1)@2001-01-01, Point(0 0)@2001-01-03]' <->
    tgeompoint '[Point(0 0)@2001-01-01, Point(1 0)@2001-01-03]';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03]
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-02]' <->
    tgeometry '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-02]';
-- Interp=Step;[1@2001-01-01, 1@2001-01-02]

```

8.5. Relaciones espaciales

Las relaciones topológicas como `ST_Intersects` y las relaciones de distancia como `ST_DWithin` pueden ser generalizadas a los puntos temporales. Los argumentos de estas funciones generalizadas son un punto temporal y un tipo base (es decir, un `geometry` o un `geography`) o dos puntos temporales. Además, ambos argumentos deben ser del mismo tipo base, es decir, estas funciones no permiten mezclar un punto de geometría temporal (o una geometría) y un punto de geografía temporal (o una geografía).

Hay tres versiones de las relaciones:

- Las relaciones *alguna vez* determinan si la relación topológica o de distancia se satisface alguna vez (ver Sección 5.4.2) y resultan en un `boolean`. Ejemplos son las funciones `eIntersects` y `eDwithin`.
- Las relaciones *siempre* determinan si la relación topológica o de distancia se satisface siempre (ver Sección 5.4.2) y resultan en un `boolean`. Ejemplos son las funciones `aIntersects` y `aDwithin`.
- Las relaciones *temporales* calculan la función topológica o de distancia en cada instante y dan como resultado un `tbool`. Ejemplos son las funciones `tIntersects` y `tDwithin`.

Por ejemplo, la siguiente consulta

```
SELECT eIntersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 2)@2001-01-01, Point(4 2)@2001-01-05]');
-- t
```

determina si el punto temporal se cruza alguna vez con la geometría. En este caso, la consulta es equivalente a la siguiente

```
SELECT ST_Intersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  geometry 'LineString(0 2,4 2)');
```

donde la segunda geometría se obtiene aplicando la función `trajectory` al punto temporal. Por otro lado, la consulta

```
SELECT tIntersects(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 2)@2001-01-01, Point(4 2)@2001-01-05]');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, f@2001-01-05)}
```

calcula en cada instante si el punto temporal se cruza con la geometría. Del mismo modo, la siguiente consulta

```
SELECT eDwithin(tgeompoint '[Point(3 1)@2001-01-01, Point(5 1)@2001-01-03]',
  tgeompoint '[Point(3 1)@2001-01-01, Point(1 1)@2001-01-03]', 2);
-- t
```

determina si la distancia entre los puntos temporales es alguna vez menor o igual a 2, mientras que la siguiente consulta

```
SELECT tDwithin(tgeompoint '[Point(3 1)@2001-01-01, Point(5 1)@2001-01-03]',
  tgeompoint '[Point(3 1)@2001-01-01, Point(1 1)@2001-01-03]', 2);
-- {[t@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
```

calcula en cada instante si la distancia entre los puntos temporales es menor o igual a 2.

Las relaciones alguna vez o siempre se utilizan normalmente en combinación con un índice espacio-temporal al calcular las relaciones temporales. Por ejemplo, la siguiente consulta

```
SELECT T.TripId, R.RegionId, tIntersects(T.Trip, R.Geom)
FROM Trips T, Regions R
WHERE eIntersects(T.Trip, R.Geom)
```

que verifica si un viaje `T` (que es un punto temporal) se cruza con una región `R` (que es una geometría) beneficiará de un índice espacio-temporal en la columna `T.Trip` dado que la función `intersects` realiza automáticamente la comparación del cuadro delimitador `T.Trip` & `R.Geom`. Esto se explica más adelante en este documento.

No todas las relaciones espaciales disponibles en PostGIS se han generalizado para geometrías temporales, solo las derivadas de las siguientes funciones: `ST_Contains`, `ST_Covers`, `ST_Disjoint`, `ST_Intersects`, `ST_Touches` y `ST_DWithin`. Estas funciones solo admiten geometrías 2D, y solo las funciones `ST_Covers`, `ST_Intersects` y `ST_DWithin` admiten geografías. Por lo tanto, esto mismo aplica a las funciones de MobilityDB derivadas de ellas, excepto que admiten 3D para puntos temporales, es decir, `tgeompoint` y `tgeogpoint`. Como se mencionó anteriormente, cada una de las funciones PostGIS mencionadas, como `ST_Contains`, tiene tres versiones generalizadas en MobilityDB: `eContains`, `aContains` y `tContains`. Además, no todas las combinaciones de parámetros son relevantes para las funciones generalizadas. Por ejemplo, `tContains(tpoint, geometría)` solo es relevante cuando la geometría es un solo punto, y `tContains(tpoint, tpoint)` es equivalente a `tintersects(tpoint, geometría)`.

Finalmente, cabe destacar que las relaciones temporales permiten mezclar geometrías 2D/3D pero en ese caso, el cálculo sólo se realiza en 2D.

8.5.1. Relaciones alguna vez o siempre

Presentamos a continuación las relaciones alguna vez o siempre. Estas relaciones incluyen automáticamente una comparación de cuadro delimitador que hace uso de cualquier índice espacial que esté disponible en los argumentos.

- Contiene alguna vez

`eContains(geometry, tgeom) → boolean`

`aContains(geometry, tgeom) → boolean`

Esta función devuelve verdadero si el punto temporal está alguna vez en el interior de la geometría. Recuerde que una geometría no contiene cosas en su borde y, por lo tanto, los polígonos y las líneas no contienen líneas y puntos que se encuentran en su borde. Consulte la documentación de la función `ST_Contains` en PostGIS.

```
SELECT eContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Ever or always covers

`eCovers({geometry,tgeom},{geometry,tgeom}) → boolean`

`aCovers({geometry,tgeom},{geometry,tgeom}) → boolean`

Please refer to the documentation of the `ST_Contains` and the `ST_Covers` function in PostGIS for detailed explanations about the difference between the two functions.

```
SELECT eCovers(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

■ Está disjuncto alguna vez

`eDisjoint({geometry,tgeom},{geometry,tgeom}) → boolean`

`aDisjoint({geo,tgeo},{geo,tgeo}) → boolean`

```
SELECT eDisjoint(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03]');
-- false
SELECT eDisjoint(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeometry '[Linestring(1 1 1,2 2 2)@2001-01-01, Point(2 2 2)@2001-01-03]');
-- true
```

■ Está alguna vez a distancia de

`eDwithin({geo,tgeo},{geo,tgeo},float) → boolean`

`aDwithin({geometry,tgeom},{geometry,tgeom},float) → boolean`

```
SELECT eDwithin(geometry 'Point(1 1 1)',
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 0)@2001-01-02]', 1);
-- true
SELECT eDwithin(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 2 2)@2001-01-01,Point(2 2 2)@2001-01-02]', 1);
-- false
```

■ Intersecta alguna vez

`eIntersects({geo,tgeo},{geo,tgeo}) → boolean`

`aIntersects({geometry,tgeom},{geometry,tgeom}) → boolean`

```
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03]');
-- false
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03]');
-- true
```

■ Toca alguna vez

`eTouches({geometry,tgeom},{geometry,tgeom}) → boolean`

`aTouches({geometry,tgeom},{geometry,tgeom}) → boolean`

```
SELECT eTouches(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-03]');
-- true
```

8.5.2. Relaciones espaciotemporales

■ Contiene temporal

`tContains(geometry,tgeom) → tbool`

```
SELECT tContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
```

```
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(1 4)@2001-01-01, Point(4 1)@2001-01-04]');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, f@2001-01-03, f@2001-01-04]}
```

■ Disjunto temporal

tDisjoint({geo,tgeo},{geo,tgeo}) → tbool

La función solo admite 3D o geografías para dos puntos temporales

```
SELECT tDisjoint(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]');
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-03], (t@2001-01-03, t@2001-01-04]}
SELECT tDisjoint(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05]');
-- {[t@2001-01-01, f@2001-01-03], (t@2001-01-03, t@2001-01-05]}
```

■ Está a distancia de temporal

tDwithin({geo,tgeo},{geo,tgeo},float) → tbool

```
SELECT tDwithin(geometry 'Point(1 1)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03]', sqrt(2));
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tDwithin(tgeompoint '[Point(1 0)@2001-01-01, Point(1 4)@2001-01-05]',
  tgeompoint 'Interp=Step;[Point(1 2)@2001-01-01, Point(1 3)@2001-01-05]', 1);
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, t@2001-01-05]}
```

■ Intersección temporal

tIntersects({geo,tgeo},{geo,tgeo}) → tbool

La función solo admite 3D o geografías para dos puntos temporales

```
SELECT tIntersects(geometry 'MultiPoint(1 1,2 2)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04]');
/* {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, t@2001-01-03),
  (f@2001-01-03, f@2001-01-04]} */
SELECT tIntersects(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05]');
-- {[f@2001-01-01, t@2001-01-03], (f@2001-01-03, f@2001-01-05)}
```

■ Toca temporal

tTouches({geometry,tgeom},{geometry,tgeom}) → tbool

```
SELECT tTouches(geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 0)@2001-01-04]');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}
```


Capítulo 9

Tipos temporales: Operaciones de análisis

9.1. Simplificación

- Simplificar un flotante o un punto temporal asegurándose de que los valores consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo  

```
minDistSimplify({tfloat,tpoint},mindist float) → {tfloat,tpoint}
```

```
minTimeDeltaSimplify({tfloat,tpoint},mint interval) → {tfloat,tpoint}
```

En el caso de puntos temporales la distancia se especifica en las unidades del sistema de coordenadas. Observe que la simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del punto temporal dado.

```
SELECT minDistSimplify(tfloat '[1@2001-01-01,2@2001-01-02,3@2001-01-04,4@2001-01-05]', 1);
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05]', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05]
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(4 4 4)@2001-01-05]', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (4 4 4)@2001-01-05]
```

```
SELECT minTimeDeltaSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-04,
4@2001-01-05]', '1 day');
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minTimeDeltaSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05]', '1 day'));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05]
```

- Simplificar un flotante o un punto temporal usando el **algoritmo de Douglas-Peucker** 

```
maxDistSimplify({tfloat,tgeompoint},maxdist float,syncdist=true) →
{tfloat,tgeompoint}
```

```
douglasPeuckerSimplify({tfloat,tgeompoint},maxdist float,syncdist=true) →
{tfloat,tgeompoint}
```

La diferencia entre las dos funciones es que `maxDistSimplify` usa una versión del algoritmo de un solo recorrido, mientras que `douglasPeuckerSimplify` usa el algoritmo recursivo estándar.

La función elimina los valores o los puntos cuya distancia es menor que la distancia pasada como segundo argumento. En el caso de puntos temporales la distancia se especifica en las unidades del sistema de coordenadas. El tercer argumento se aplica solo a puntos temporales y especifica si se utiliza la distancia espacial o la distancia sincronizada. Observe que la simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del punto temporal dado.

```
-- Only synchronous distance for temporal floats
SELECT maxDistSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 3@2001-01-04,
1@2001-01-05]', 1, false);
-- [1@2001-01-01, 1@2001-01-03, 3@2001-01-04, 1@2001-01-05]
-- Synchronous distance by default for temporal points
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-05, POINT(5 1)@2001-01-06]
-- Spatial distance
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2, false));
-- [POINT(1 1)@2001-01-01, POINT(5 1)@2001-01-06]

-- Spatial vs synchronized Euclidean distance
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, false));
-- [POINT(1 1)@2001-01-01, POINT(7 4)@2001-01-07]
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, true));
-- [POINT(1 1)@2001-01-01, POINT(6 1)@2001-01-06, POINT(7 4)@2001-01-07]
```

La diferencia entre la distancia espacial y la distancia sincronizada se ilustra en los dos últimos ejemplos anteriores y en la Figura 9.1. En el primer ejemplo, que usa la distancia espacial, se elimina el segundo instante ya que la distancia perpendicular entre POINT(2 2) y la línea definida por POINT(1 1) y POINT(7 4) es igual a 2.23. Por el contrario, en el segundo ejemplo se mantiene el segundo instante dado que la proyección de Point(6 2) en la marca de tiempo 2001-01-06 sobre el segmento de línea temporal da como resultado Point(6 3.5) y la distancia entre el punto original y su proyección es 2.5.

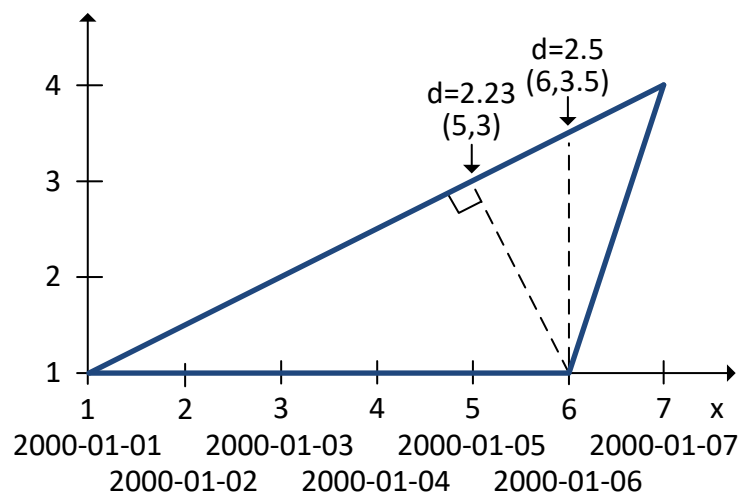


Figura 9.1: Diferencia entre la distancia espacial y la distancia sincronizada.

Un uso típico de la función `douglasPeuckerSimplify` es reducir el tamaño de un conjunto de datos, en particular con fines de visualización. Si la visualización es estática, se debe preferir la distancia espacial; si la visualización es dinámica o animada, se debe preferir la distancia sincronizada.

9.2. Reducción

- Muestrear un valor temporal con respecto a un intervalo

```
tsample(temporal,duration interval,torigin timestamptz='2000-01-03',
```

interp='discrete') → temporal

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000. El intervalo dado debe ser estrictamente mayor que cero.

```
SELECT tsample(tbool '[true@2001-01-01,false@2001-01-02]', '12 hours', '2001-01-01', 'step ←');
-- [t@2001-01-01, f@2001-01-02]
SELECT tsample(tint '{1@2001-01-01,5@2001-01-05}', '3 days', '2001-01-01');
-- {1@2001-01-01}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '1 day', '2001-01-01');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01');
-- {1@2001-01-01, 4@2001-01-04}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01', 'linear');
-- [1@2001-01-01, 4@2001-01-04]
```

```
SELECT asText(tsample(tgeopoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05]',
'2 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03, POINT(5 5)@2001-01-05}
SELECT asText(tsample(tgeopoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
[Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07}
SELECT asText(tsample(tgeopoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
[Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01', 'step'));
-- Interp=Step;[POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07]
```

La Figura 9.2 ilustra el muestreo para números flotantes temporales con varias interpolaciones. Como ilustra la figura, la operación de muestreo es más adecuada para valores temporales con interpolación continua.

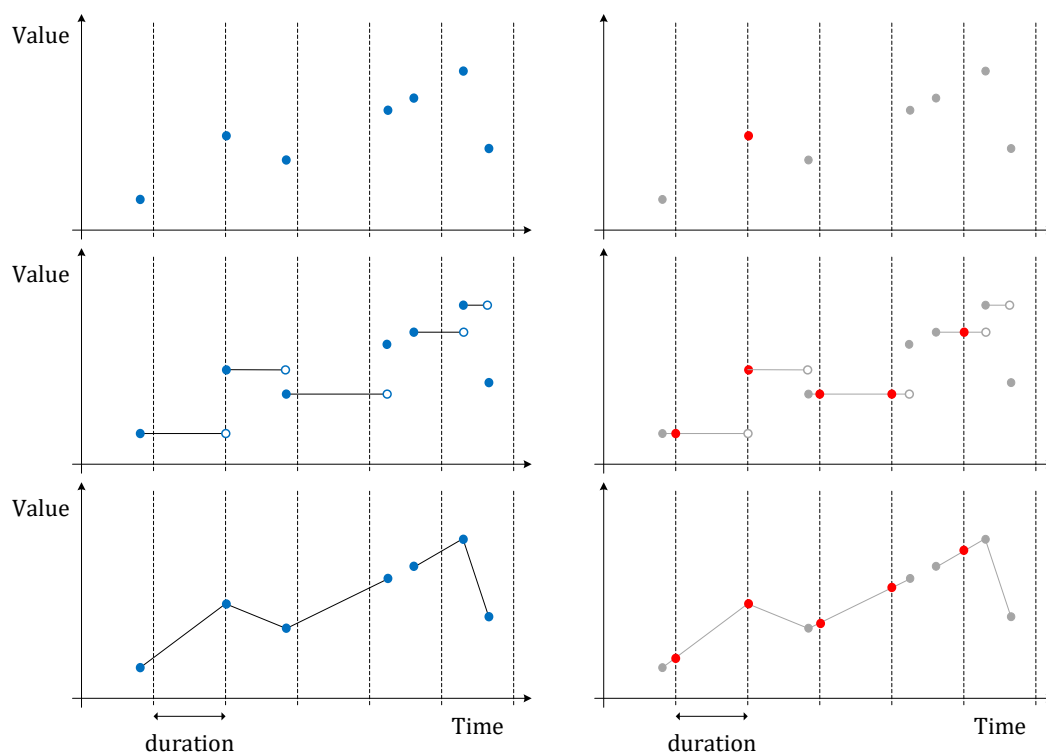


Figura 9.2: Muestreo de flotantes temporales con interpolación discreta, escalonada y lineal.

- Reducir la precisión temporal de un valor temporal con respecto a un intervalo calculando el promedio/centroide ponderado por el tiempo en cada intervalo de tiempo

```
tprecision({tnumber,tgeompoint,tcbuffer,tpose,tgeo,trgeometry},duration interval,torigin
timestampz='2000-01-03')
```

```
→ {tnumber,tgeompoint,tcbuffer,tpose,tgeo,trgeometry}
```

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000. El intervalo dado debe ser estrictamente mayor que cero.

```
SELECT tprecision(tint '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day',
'2001-01-01');
-- Interp=Step;[1@2001-01-01, 5@2001-01-05, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day',
'2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day',
'2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','2 days',
'2001-01-01');
-- [2@2001-01-01, 4@2001-01-03, 4@2001-01-05, 2@2001-01-07]
```

```
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '1 day', '2001-01-01'));
/* [POINT(1.5 1.5)@2001-01-01, POINT(4.5 4.5)@2001-01-04, POINT(4.5 4.5)@2001-01-05,
POINT(1.5 1.5)@2001-01-08] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '2 days', '2001-01-01'));
/* [POINT(2 2)@2001-01-01, POINT(4 4)@2001-01-03, POINT(4 4)@2001-01-05,
POINT(2 2)@2001-01-07] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
Point(1 1)@2001-01-09]', '4 days', '2001-01-01'));
-- [POINT(3 3)@2001-01-01, POINT(3 3)@2001-01-05]
```

Cambiar la precisión de un valor temporal es similar a cambiar su *granularidad temporal*, por ejemplo, de marcas de tiempo a horas o días, aunque la precisión se puede establecer en un intervalo arbitrario, como 2 horas y 15 minutos. La Figura 9.3 ilustra un cambio de precisión temporal para números flotantes temporales con varias interpolaciones.

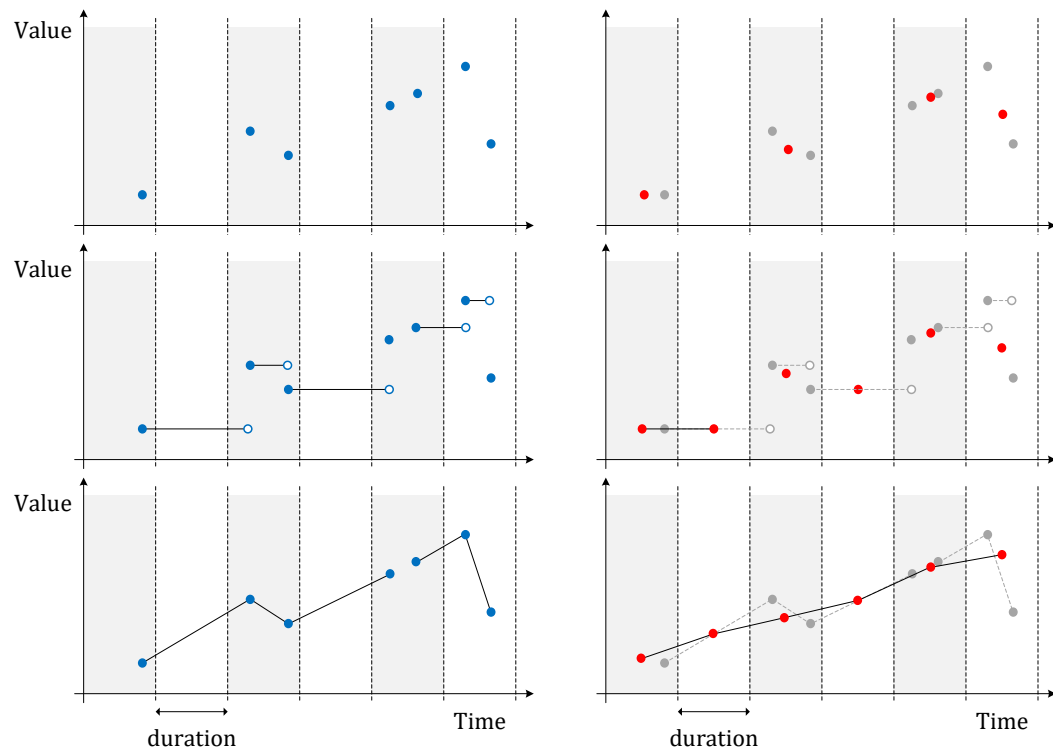


Figura 9.3: Cambio de precisión de números flotantes temporales con interpolación discreta, escalonada y lineal.

9.3. Similitud

- Devuelve la **distancia de Hausdorff** discreta, la **distancia de Fréchet** discreta, la distancia de distorsión de tiempo dinámica (**Dynamic Time Warp** o DTW), la distancia de Hausdorff promedio, o la distancia de **subsecuencia común más larga** (Longest Common SubSequence o LCSS) entre dos valores temporales 📦🌐

`hausdorffDistance({tnumber, tgeo}, {tnumber, tgeo}) → float`

`frechetDistance({tnumber, tgeo}, {tnumber, tgeo}) → float`

`dynTimeWarpDistance({tnumber, tgeo}, {tnumber, tgeo}) → float`

`averageHausdorffDistance({tnumber, tgeo}, {tnumber, tgeo}) → float`

`lcssDistance({tnumber, tgeo}, {tnumber, tgeo}, float) → float`

La función `lcssDistance` requiere un parámetro adicional que especifica el umbral de distancia por debajo del cual dos valores se consideran coincidentes.

Estas funciones tienen una complejidad de espacio lineal ya que solo dos filas de la matriz de distancia son asignadas en la memoria. Sin embargo, su complejidad de tiempo es cuadrática en el número de instantes de los valores temporales. Por lo tanto, las funciones requieren un tiempo considerable para valores temporales con gran número de instantes.

```
SELECT hausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(hausdorffDistance(tgeopoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeopoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
```

```
SELECT frechetDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(frechetDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
```

```
SELECT dynTimeWarpDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 2
SELECT round(dynTimeWarpDistance(tgeompoint '[Point(1 1)@2001-01-01,
  Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 3.380776
```

```
SELECT round(averageHausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]') ←
  ::numeric, 6);
-- 0.283333
SELECT round(averageHausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001 ←
  -01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 0.385218
```

```
SELECT lcassDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]', ←
  1.0);
-- 3
SELECT round(lcassDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]', 1.0)::numeric, 6);
-- 2.000000
```

- Devuelve las parejas de correspondencia entre dos valores temporales con respecto a la distancia de Fréchet discreta o la distance de distorsión de tiempo dinámica (Dynamic Time Warp o DTW)   { }

`frechetDistancePath({tnumber, tgeo}, {tnumber, tgeo}) → {(i, j)}`

`dynTimeWarpPath({tnumber, tgeo}, {tnumber, tgeo}) → {(i, j)}`

El resultado es un conjunto de pares (i, j) . Esta función requiere ubicar en memoria una matriz de distancias cuyo tamaño es cuadrático en el número de instantes de los valores temporales. Por tanto, la función fallará para valores temporales con gran número de instantes dependiendo de la memoria disponible.

```
SELECT frechetDistancePath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT frechetDistancePath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]');
```

```
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)

SELECT dynTimeWarpPath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT dynTimeWarpPath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

9.4. Filtro de Kalman extendido

- Devolver un flotante temporal o un punto temporal suavizado, obtenido al filtrar una trayectoria ruidosa con un **filtro de Kalman extendido** 

```
extendedKalmanFilter(tfloat, gate float, q float, variance float, to_drop boolean DEFAULT
TRUE) → tfloat
```

```
extendedKalmanFilter(tgeompoint, gate float, q float, variance float, to_drop boolean
DEFAULT TRUE) → tgeompoint
```

El filtro modela el movimiento subyacente de un objeto móvil y lo separa del ruido de medición y de los valores atípicos ocasionales. El parámetro `gate` controla el rigor de la detección de atípicos, `q` es la escala del ruido de proceso, `variance` es la varianza del ruido de medición y `to_drop` elige si los atípicos se eliminan (`true`) o se reemplazan por la trayectoria predicha (`false`).

Estas funciones están pensadas para trayectorias con al menos tres instantes y se aplican normalmente a valores temporales con interpolación lineal.

```
SELECT asEWKT(extendedKalmanFilter(tgeompoint
  '[Point(1 1)@2001-01-01, Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]',
  1e-5, 5e-10, 5e-10, true));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-03]
SELECT asEWKT(extendedKalmanFilter(tgeompoint
  '[Point(1 1)@2001-01-01, Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]',
  1e-5, 5e-10, 5e-10, false));
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]
```

9.5. Operaciones de división

Al crear índices para tipos temporales, lo que se almacena en el índice no es el valor real sino un cuadro delimitador que *representa* el valor. En este caso, el índice proporcionará una lista de valores candidatos que *pueden* satisfacer el predicado de la consulta, y se necesita un segundo paso para filtrar los valores candidatos calculando el predicado de la consulta sobre los valores reales.

Sin embargo, cuando los cuadros delimitadores tienen un gran espacio vacío no cubierto por los valores reales, el índice generará muchos valores candidatos que no satisfacen el predicado de la consulta, lo que reduce la eficiencia del índice. En estas situaciones, puede ser mejor representar un valor no con un cuadro delimitador *único*, sino con *múltiples* cuadros delimitadores. Esto aumenta considerablemente la eficiencia del índice, siempre que el índice sea capaz de gestionar múltiples cuadros delimitadores por valor. Las siguientes funciones se utilizan para generar múltiples cuadros delimitadores para un único valor temporal.

- Devuelve una matriz de N rangos de tiempo a partir de los instantes o segmentos de un valor temporal  

`splitNSpans(temp, integer) → tstzspan[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de rangos de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un rango por instante o segmento del valor temporal. En caso contrario, el número de rangos especificado se obtendrá fusionando instantes o segmentos consecutivos.

```
SELECT splitNSpans(ttext '{A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
A@2000-01-05}', 1);
-- {"[2000-01-01, 2000-01-05]"}
SELECT splitNSpans(tfloat '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
1@2000-01-05}', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-04, 2000-01-05]"}
SELECT splitNSpans(tgeompoint '[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(1 1)@2000-01-03, Point(2 2)@2000-01-04, Point(1 1)@2000-01-05]', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-03, 2000-01-05]"}
SELECT splitNSpans(tgeompoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
[Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05]}', 2);
-- {"[2000-01-01, 2000-01-04]", "[2000-01-05, 2000-01-05]"}
SELECT splitNSpans(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
2@2000-01-05}', 6);
/* {"[2000-01-01, 2000-01-01]", "[2000-01-02, 2000-01-02]", "[2000-01-03, 2000-01-03]",
"[2000-01-04, 2000-01-04]", "[2000-01-05, 2000-01-05]"} */
SELECT splitNSpans(ttext '{A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
A@2000-01-05}', 6);
/* {"[2000-01-01, 2000-01-02]", "[2000-01-02, 2000-01-03]",
"[2000-01-03, 2000-01-04]", "[2000-01-04, 2000-01-05]"} */
```

- Devuelve una matriz de rangos de tiempo obtenida fusionando N instantes o segmentos consecutivos de un valor temporal  

`splitEachNSpans(temp, integer) → tstzspan[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un rango de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un único rango por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada rango de salida. Observe que, a diferencia de la función `splitNSpans`, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNSpans(ttext '{A@2000-01-01, B@2000-01-02, A@2000-01-03, B@2000-01-04,
A@2000-01-05}', 1);
/* {"[2000-01-01, 2000-01-01]", "[2000-01-02, 2000-01-02]", "[2000-01-03, 2000-01-03]",
"[2000-01-04, 2000-01-04]", "[2000-01-05, 2000-01-05]"} */
SELECT splitEachNSpans(tfloat '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 2@2000-01-04,
1@2000-01-05}', 2);
-- {"[2000-01-01, 2000-01-03]", "[2000-01-03, 2000-01-05]"}
SELECT splitEachNSpans(tgeompoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
[Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05]}', 2);
-- {"[2000-01-01, 2000-01-02]", "[2000-01-03, 2000-01-04]", "[2000-01-05, 2000-01-05]"}
SELECT splitEachNSpans(tgeompoint '[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04, Point(1 1 1)@2000-01-05]', 6);
-- {"[2000-01-01, 2000-01-05]"}
SELECT splitEachNSpans(tgeompoint '{[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02],
```



```
[Point(1 1 1)@2000-01-03, Point(2 2 2)@2000-01-04], [Point(1 1 1)@2000-01-05}]", 6);
-- {"[2000-01-01, 2000-01-02]", "[2000-01-03, 2000-01-04]", "[2000-01-05, 2000-01-05]"}
```

- Devuelve una matriz de N cuadros temporales obtenida fusionando los instantes o segmentos de un número temporal

`splitNTboxes(tnumber, integer) → tbox[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de cuadros de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un cuadro por instante o segmento del número temporal. En caso contrario, el número de cuadros especificado se obtendrá fusionando instantes o segmentos consecutivos.

```
SELECT splitNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 1);
-- {"TBOXINT XT([1, 5], [2000-01-01, 2000-01-05])"}
SELECT splitNTboxes(tfloat '{[1@2000-01-01, 2@2000-01-02], [1@2000-01-03, 4@2000-01-04],
[1@2000-01-05]}', 2);
/* {"TBOXFLOAT XT([1, 4], [2000-01-01, 2000-01-04])",
    "TBOXFLOAT XT([1, 1], [2000-01-05, 2000-01-05])"} */
SELECT splitNTboxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3);
/* {"TBOXFLOAT XT([1, 2], [2000-01-01, 2000-01-03])",
    "TBOXFLOAT XT([1, 4], [2000-01-03, 2000-01-04])",
    "TBOXFLOAT XT([1, 4], [2000-01-04, 2000-01-05])"} */
SELECT splitNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 6);
/* {"TBOXINT XT([1, 2], [2000-01-01, 2000-01-01])",
    "TBOXINT XT([2, 3], [2000-01-02, 2000-01-02])",
    "TBOXINT XT([1, 2], [2000-01-03, 2000-01-03])",
    "TBOXINT XT([4, 5], [2000-01-04, 2000-01-04])",
    "TBOXINT XT([1, 2], [2000-01-05, 2000-01-05])"} */
SELECT splitNTboxes(tint '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 6);
/* {"TBOXINT XT([1, 3], [2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 3], [2000-01-02, 2000-01-03])",
    "TBOXINT XT([1, 5], [2000-01-03, 2000-01-04])",
    "TBOXINT XT([1, 5], [2000-01-04, 2000-01-05])"} */
```

- Devuelve una matriz de cuadros temporales obtenida fusionando N instantes o segmentos consecutivos de un número temporal

`splitEachNTboxes(tnumber, integer) → tbox[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un cuadro de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un único cuadro por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada cuadro de salida. Observe que, a diferencia de la función `splitNTboxes`, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNTboxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 1);
/* {"TBOXINT XT([1, 2], [2000-01-01, 2000-01-01])",
    "TBOXINT XT([2, 3], [2000-01-02, 2000-01-02])",
    "TBOXINT XT([1, 2], [2000-01-03, 2000-01-03])",
    "TBOXINT XT([4, 5], [2000-01-04, 2000-01-04])"} */
SELECT splitEachNTboxes(tint '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 1);
/* {"TBOXINT XT([1, 3], [2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 3], [2000-01-02, 2000-01-03])",
    "TBOXINT XT([1, 5], [2000-01-03, 2000-01-04])",
    "TBOXINT XT([1, 5], [2000-01-04, 2000-01-05])"} */
SELECT splitEachNTboxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3);
```

```

/* {"TBOXFLOAT XT([1, 4],[2000-01-01, 2000-01-04])",
   "TBOXFLOAT XT([1, 4],[2000-01-04, 2000-01-05])"} */
SELECT splitEachNtboxes(tfloat ' {[1@2000-01-01, 2@2000-01-02], [1@2000-01-03, 4@2000-01-04],
                               [1@2000-01-05]}', 6);
/* {"TBOXFLOAT XT([1, 2],[2000-01-01, 2000-01-02])",
   "TBOXFLOAT XT([1, 4],[2000-01-03, 2000-01-04])",
   "TBOXFLOAT XT([1, 1],[2000-01-05, 2000-01-05])"} */

```

- Devuelve ya sea una matriz de N cuadros espaciales obtenida fusionando los segmentos de una (multi)línea o una matriz de N cuadros espaciotemporales obtenida fusionando los instantes o segmentos de un punto temporal  

`splitNStboxes(lines, integer) → stbox[]`

`splitNStboxes(tpoint, integer) → stbox[]`

Para puntos temporales, la elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de cuadros de salida. Si el número de instantes o segmentos es menor que el número dado, la matriz resultante tendrá un cuadro por segmento de la (multi)línea o un cuadro por instante o segmento del punto temporal. En caso contrario, el número de cuadros especificado se obtendrá fusionando instantes o segmentos consecutivos.

```

SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1), (5,2))"}
SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1), (3,2))", "STBOX X((3,1), (5,2))"}
SELECT splitNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1), (2,2,1))",
   "SRID=4326;GEODSTBOX Z((2,1,1), (3,2,1))",
   "SRID=4326;GEODSTBOX Z((3,1,1), (4,2,1))",
   "SRID=4326;GEODSTBOX Z((4,1,1), (5,2,1))"} */
SELECT splitNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1), (4,2))", "STBOX X((5,1), (6,2))"}

```

```

SELECT splitNStboxes(tgeompoint ' {Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
   Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05}', 1);
-- {"STBOX XT(((1,1), (5,2)), [2000-01-01, 2000-01-05])"}
SELECT splitNStboxes(tgeompoint ' {Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
   Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05}';
/* {"STBOX XT(((1,1), (2,2)), [2000-01-01, 2000-01-02])",
   "STBOX XT(((2,1), (3,2)), [2000-01-02, 2000-01-03])",
   "STBOX XT(((3,1), (4,2)), [2000-01-03, 2000-01-04])",
   "STBOX XT(((4,1), (5,2)), [2000-01-04, 2000-01-05])"} */
SELECT splitNStboxes(tgeompoint ' {[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02],
   [Point(3 1)@2000-01-03, Point(4 2)@2000-01-04], [Point(5 1)@2000-01-05]}', 2);
/* {"STBOX XT(((1,1), (4,2)), [2000-01-01, 2000-01-04])",
   "STBOX XT(((5,1), (5,1)), [2000-01-05, 2000-01-05])"} */
SELECT splitNStboxes(tgeogpoint ' {Point(1 1 1)@2000-01-01, Point(2 2 1)@2000-01-02,
   Point(3 1 1)@2000-01-03, Point(4 2 1)@2000-01-04, Point(5 1 1)@2000-01-05}', 6);
/* {"SRID=4326;GEODSTBOX ZT(((1,1,1), (1,1,1)), [2000-01-01, 2000-01-01])",
   "SRID=4326;GEODSTBOX ZT(((2,2,1), (2,2,1)), [2000-01-02, 2000-01-02])",
   "SRID=4326;GEODSTBOX ZT(((3,1,1), (3,1,1)), [2000-01-03, 2000-01-03])",
   "SRID=4326;GEODSTBOX ZT(((4,2,1), (4,2,1)), [2000-01-04, 2000-01-04])",
   "SRID=4326;GEODSTBOX ZT(((5,1,1), (5,1,1)), [2000-01-05, 2000-01-05])"} */
SELECT splitNStboxes(tgeompoint ' {Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
   Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05}', 6);
/* {"STBOX XT(((1,1), (2,2)), [2000-01-01, 2000-01-02])",
   "STBOX XT(((2,1), (3,2)), [2000-01-02, 2000-01-03])",
   "STBOX XT(((3,1), (4,2)), [2000-01-03, 2000-01-04])",
   "STBOX XT(((4,1), (5,2)), [2000-01-04, 2000-01-05])"} */

```

- Devuelve ya sea una matriz de cuadros espaciales obtenida fusionando N segmentos consecutivos de una (multi)línea o una matriz de cuadros espaciotemporales obtenida fusionando N instantes o segmentos consecutivos de un punto temporal  

```
splitEachNStboxes(lines, integer) → stbox[]
splitEachNStboxes(tpoint, integer) → stbox[]
```

Para puntos temporales, la elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un cuadro de salida. Si el número de instantes o segmentos es menor que el número dado, la matriz resultante tendrá un único cuadro por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada cuadro de salida. Observe que, a diferencia de la función **splitNStboxes**, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1),(5,2))"}
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitEachNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1),(2,2,1))",
    "SRID=4326;GEODSTBOX Z((2,1,1),(3,2,1))",
    "SRID=4326;GEODSTBOX Z((3,1,1),(4,2,1))",
    "SRID=4326;GEODSTBOX Z((4,1,1),(5,2,1))"} */
SELECT splitEachNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitEachNStboxes(tgeompoint 'Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05', 1);
/* {"STBOX XT(((1,1),(1,1)), [2000-01-01, 2000-01-01])",
    "STBOX XT(((2,2),(2,2)), [2000-01-02, 2000-01-02])",
    "STBOX XT(((3,1),(3,1)), [2000-01-03, 2000-01-03])",
    "STBOX XT(((4,2),(4,2)), [2000-01-04, 2000-01-04])",
    "STBOX XT(((5,1),(5,1)), [2000-01-05, 2000-01-05])"} */
SELECT splitEachNStboxes(tgeompoint 'Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
    Point(3 1)@2000-01-03, Point(4 2)@2000-01-04, Point(5 1)@2000-01-05', 1);
/* {"STBOX XT(((1,1),(2,2)), [2000-01-01, 2000-01-02])",
    "STBOX XT(((2,1),(3,2)), [2000-01-02, 2000-01-03])",
    "STBOX XT(((3,1),(4,2)), [2000-01-03, 2000-01-04])",
    "STBOX XT(((4,1),(5,2)), [2000-01-04, 2000-01-05])"} */
SELECT splitEachNStboxes(tgeogpoint '{[Point(1 1)@2000-01-01, Point(2 2)@2000-01-02],
    [Point(3 1)@2000-01-03, Point(4 2)@2000-01-04], [Point(5 1)@2000-01-05]}', 2);
/* {"SRID=4326;GEODSTBOX XT(((1,1),(2,2)), [2000-01-01, 2000-01-02])",
    "SRID=4326;GEODSTBOX XT(((3,1),(4,2)), [2000-01-03, 2000-01-04])",
    "SRID=4326;GEODSTBOX XT(((5,1),(5,1)), [2000-01-05, 2000-01-05])"} */
```

9.6. Mosaicos multidimensionales

Los mosaicos multidimensionales son un mecanismo que se utiliza para dividir el dominio de valores temporales en intervalos o mosaicos de un número variable de dimensiones. En el caso de una sola dimensión, el dominio se puede dividir por valor o por tiempo utilizando intervalos del mismo ancho o la misma duración, respectivamente. Para los números temporales, el dominio se puede dividir en mosaicos bidimensionales del mismo ancho para la dimensión de valor y la misma duración para la dimensión de tiempo. Para los puntos temporales, el dominio se puede dividir en el espacio en mosaicos bidimensionales o tridimensionales, dependiendo del número de dimensiones de las coordenadas espaciales. Finalmente, para los puntos temporales, el dominio se puede dividir por espacio y por tiempo usando mosaicos tridimensionales o tetradimensionales. Además, los valores temporales también se pueden fragmentar de acuerdo con una malla multidimensional definida sobre el dominio subyacente.

Los mosaicos multidimensionales se pueden utilizar para diversos fines. Se pueden utilizar para calcular histogramas multidimensionales, donde los valores temporales se agregan de acuerdo con la partición subyacente del dominio. Por otro lado, los mosaicos multidimensionales también se pueden utilizar para fines de indexación, donde el cuadro delimitador de un valor temporal se puede fragmentar en múltiples cuadros para mejorar la eficiencia del índice. Finalmente, los mosaicos multidimensionales se pueden utilizar para fragmentar valores temporales de acuerdo con una malla multidimensional definida sobre el dominio subyacente. Esto permite la distribución de un conjunto de datos en un clúster de servidores, donde cada servidor contiene una partición

del conjunto de datos. La ventaja de este mecanismo de partición es que preserva la proximidad en valor/espacio y tiempo, a diferencia de los mecanismos de partición tradicionales basados en hash.

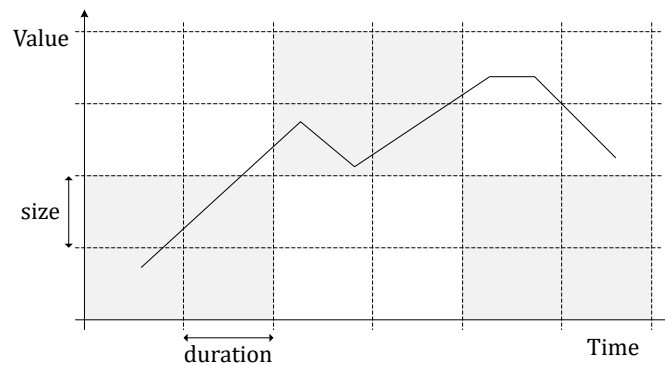


Figura 9.4: Mosaicos multidimensionales para números flotantes temporales.

La Figura 9.4 ilustra un mosaico multidimensional para números flotantes temporales. El dominio bidimensional se divide en mosaicos que tienen el mismo tamaño para la dimensión de valor y la misma duración para la dimensión de tiempo. Suponga que este esquema de mosaicos se usa para distribuir un conjunto de datos en un clúster de seis servidores, como sugiere el patrón gris en la figura. En este caso, los valores se fragmentan para que cada servidor reciba los datos de mosaicos contiguos. Esto implica en particular que cuatro nodos recibirán un fragmento del número flotante temporal que se muestra en la figura. Una ventaja de esta distribución de datos basada en mosaicos multidimensionales es que reduce los datos que deben intercambiarse entre nodos cuando se procesan consultas, un proceso que generalmente se denomina *reshuffling*.

Muchas de las funciones de esta sección son *funciones de retorno de conjuntos* (también conocidas como *funciones de tabla*) ya que normalmente devuelven más de un valor. En este caso, las funciones están marcadas con el símbolo `{}`.

9.6.1. Operaciones de intervalos

- Devuelve una matriz de intervalos que cubre el rango de valores o de tiempo con intervalos de la misma amplitud o duración

```
bins(numspans,width number,origin number=0) → span[]
```

```
bins(datespans,duration interval,origin date='2000-01-03') → span[]
```

```
bins(tstzspans,duration interval,origin timestamptz='2000-01-03') → span[]
```

Si el origen no se especifica, su valor se establece por defecto en 0 para rangos de valores y en lunes 3 de enero de 2000 para rangos de tiempo.

```
SELECT bins(floatspan '[-10, -1]', 2.5, -7);
-- {"[-12, -9.5)","[-9.5, -7)","[-7, -4.5)","[-4.5, -2)","[-2, 0.5)"}
SELECT bins(intspan '[15, 25]', 2);
-- {"[14, 16)","[16, 18)","[18, 20)","[20, 22)","[22, 24)","[24, 26)","[26, 28)"}
SELECT index, span
FROM unnest(bins(datespan '[2001-01-15, 2001-01-25]', '2 days'))
  WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-17)
-- 2 | [2001-01-17, 2001-01-19)
-- 3 | [2001-01-19, 2001-01-21)
-- ...
SELECT index, span
FROM unnest(bins(tstzspanset '{[2001-01-15, 2001-01-17],[2001-01-22, 2001-01-25]}',
  '2 days', '2001-01-02')) WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-16)
-- 2 | [2001-01-16, 2001-01-17)
-- 3 | [2001-01-22, 2001-01-24)
-- 4 | [2001-01-24, 2001-01-25]
```

- Devuelve el rango que contiene un número o una marca de tiempo

`getBin(number, size number, origin number=0) → span`

`getBin(date, duration interval, origin date='2000-01-03') → datespan`

`getBin(timestamptz, duration interval, origin timestamptz='2000-01-03') → tstzspan`

Si el origen no se especifica, su valor se establece por defecto en 0 para rangos de valores y en lunes 3 de enero de 2000 para rangos de tiempo.

```
SELECT getBin(2, 2);
-- [2, 4)
SELECT getBin(2, 2.5, 1.5);
-- [1.5, 4)
SELECT getBin('2001-01-04', interval '1 week');
-- [2001-01-01, 2001-01-08)
SELECT getBin('2001-01-04', interval '1 week', '2001-01-07');
-- [2000-12-31, 2001-01-07)
```

9.6.2. Operaciones de mosaicos

- Devuelve el conjunto de mosaicos que cubre un cuadro delimitador temporal con mosaicos del mismo tamaño y/o duración {}

`valueTiles(tbox, size float, vorigin float=0) → {(index, tile)}`

`timeTiles(tbox, duration interval, torigin timestamptz='2000-01-03') → {(index, tile)}`

`valueTimeTiles(tbox, size float, duration interval, vorigin float=0, torigin timestamptz='2000-01-03') → {(index, tile)}`

Si el origen de la dimensión de valores y/o de tiempo no se especifica, su valor se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente.

```
SELECT (gr).index, (gr).tile
FROM (SELECT valueTiles(tfloat '[15@2001-01-15, 25@2001-01-25]':tbox, 2.0) AS gr) t;
-- 1 | TBOXFLOAT X([14, 16))
-- 2 | TBOXFLOAT X([16, 18))
-- 3 | TBOXFLOAT X([18, 20))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT timeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]':tbox, '2 days') AS gr) t;
-- 1 | TBOX T([2001-01-15, 2001-01-17))
-- 2 | TBOX T([2001-01-17, 2001-01-19))
-- 3 | TBOX T([2001-01-19, 2001-01-21))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT valueTimeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]':tbox, 2.0, '2 days')
      AS gr) t;
-- 1 | TBOX XT([14,16), [2001-01-15,2001-01-17))
-- 2 | TBOX XT([16,18), [2001-01-15,2001-01-17))
-- 3 | TBOX XT([18,20), [2001-01-15,2001-01-17))
-- ...
SELECT valueTimeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]':tbox, 2.0, '2 days', 11.5);
-- (1,"TBOX XT([13.5,15.5), [2001-01-15,2001-01-17))")
-- (2,"TBOX XT([15.5,17.5), [2001-01-15,2001-01-17))")
-- (3,"TBOX XT([17.5,19.5), [2001-01-15,2001-01-17))")
-- ...
```

- Devuelve el conjunto de mosaicos que cubre un cuadro delimitador espaciotemporal con mosaicos del mismo tamaño y/o duración  {}

`spaceTiles(stbox, xsize float, [ysize float, zsize float,]`

```
sorigin geompoint='Point(0 0 0)',borderInc bool=true) → {(index,tile)}
timeTiles(stbox,duration interval,torigin timestamptz='2000-01-03',
borderInc bool=true) → {(index,tile)}
spaceTimeTiles(stbox,xsize float,[ysize float,zsize float,]duration interval,
sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
borderInc bool=true) → {(index,tile)}
```

Si el origen de las dimensiones espacial y/o de tiempo no se especifican, su valor se establece por defecto en 'Point (0 0 0)' y en el lunes 3 de enero de 2000, respectivamente. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan mosaicos adicionales que contienen el borde.

En el caso de una malla espacio-temporal, `ysize` y `zsize` son opcionales, se supone que el tamaño de las dimensiones faltantes es igual a `xsize`. El SRID de las coordenadas de los mosaicos está determinado por el del cuadro de entrada y el tamaño se da en las unidades del SRID. Si se especifica el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales al del cuadro delimitador, de lo contrario se genera un error.

```
SELECT spaceTiles(tgeompoint '[Point(3 3)@2001-01-15,
Point(15 15)@2001-01-25]':stbox, 2.0);
-- (1,"STBOX X((2,2),(4,4))")
-- (2,"STBOX X((4,2),(6,4))")
-- (3,"STBOX X((6,2),(8,4))")
-- ...
SELECT timeTiles(tgeompoint '[Point(3 3)@2001-01-15,
Point(15 15)@2001-01-25]':stbox, '2 days');
-- (1,"STBOX T([2001-01-15, 2001-01-17))")
-- (2,"STBOX T([2001-01-17, 2001-01-19))")
-- (3,"STBOX T([2001-01-19, 2001-01-21))")
-- ...
SELECT spaceTiles(tgeompoint 'SRID=3812;[Point(3 3)@2001-01-15,
Point(15 15)@2001-01-25]':stbox, 2.0, geometry 'Point(3 3)');
-- (1,"SRID=3812;STBOX X((3,3),(5,5))")
-- (2,"SRID=3812;STBOX X((5,3),(7,5))")
-- (3,"SRID=3812;STBOX X((7,3),(9,5))")
-- ...
SELECT spaceTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
Point(15 15 15)@2001-01-25]':stbox, 2.0, geometry 'Point(3 3 3)');
-- (1,"STBOX Z((3,3,3),(5,5,5))")
-- (2,"STBOX Z((5,3,3),(7,5,5))")
-- (3,"STBOX Z((7,3,3),(9,5,5))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3)@2001-01-15,
Point(15 15)@2001-01-25]':stbox, 2.0, interval '2 days');
-- (1,"STBOX XT(((2,2),(4,4)), [2001-01-15,2001-01-17))")
-- (2,"STBOX XT(((4,2),(6,4)), [2001-01-15,2001-01-17))")
-- (3,"STBOX XT(((6,2),(8,4)), [2001-01-15,2001-01-17))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
Point(15 15 15)@2001-01-25]':stbox, 2.0, interval '2 days',
'Point(3 3 3)', '2001-01-15');
-- (1,"STBOX ZT(((3,3,3),(5,5,5)), [2001-01-15,2001-01-17))")
-- (2,"STBOX ZT(((5,3,3),(7,5,5)), [2001-01-15,2001-01-17))")
-- (3,"STBOX ZT(((7,3,3),(9,5,5)), [2001-01-15,2001-01-17))")
-- ...
```

- Devuelve el mosaico temporal que cubre un valor y/o una marca de tiempo

```
getValueTile(value float,vsize float,vorigin float=0.0) → tbox
```

```
getTboxTimeTile(time timestamptz,duration interval,torigin timestamptz='2000-01-03')
torigin timestamptz='2000-01-03') → tbox
```

```
getValueTimeTile(value float,time timestampz,vsize float,duration interval,
  vorigin float=0.0,torigin timestampz='2000-01-03') → tbox
```

Si el origen de las dimensiones de valores y/o de tiempo no se especifica, su valor se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente.

```
SELECT getValueTile(15, 2);
-- TBOX ([14,16))
SELECT getTboxTimeTile('2001-01-15', interval '2 days');
-- TBOX ([2001-01-15,2001-01-17))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days');
-- TBOX ([14,16),[2001-01-15,2001-01-17))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days', 1, '2001-01-02');
-- TBOX XT([15,17),[2001-01-14,2001-01-16))
```

- Devuelve el mosaico espaciotemporal que cubre un punto y/o una marca de tiempo 

```
getSpaceTile(point geometry,xsize float,[ysize float,zsize float],
  sorigin geompoint='Point(0 0 0)') → stbox
getStboxTimeTile(time timestampz,duration interval,
  toorigin timestampz='2000-01-03') → stbox
getSpaceTimeTile(point geometry,time timestampz,xsize float,
  [ysize float,zsize float,]duration,interval,sorigin geompoint='Point(0 0 0)',
  toorigin timestampz='2000-01-03') → stbox
```

Si el origen de la dimensión espacial y/o de tiempo no se especifica, su valor se establece por defecto en 'Point(0 0 0)' y en el lunes 3 de enero de 2000, respectivamente.

En el caso de una malla espacio-temporal, ysize y zsize son opcionales, se supone que el tamaño de las dimensiones faltantes es igual a xsize. El SRID de las coordenadas de los mosaicos está determinado por el del cuadro de entrada y el tamaño se da en las unidades del SRID. Si se especifica el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales al del cuadro delimitador, de lo contrario se genera un error.

```
SELECT getSpaceTile(geometry 'Point(1 1 1)', 2.0);
-- STBOX Z((0,0,0),(2,2,2))
SELECT getStboxTimeTile(timestampz '2001-01-01', interval '2 days');
-- STBOX T([2001-01-01,2001-01-03))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days');
-- STBOX XT((0,0),(2,2),[2001-01-01,2001-01-03))
SELECT getSspaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days',
  'Point(1 1)', '2001-01-02');
-- STBOX XT((1,1),(3,3),[2000-12-31,2001-01-02))
```

9.6.3. Operaciones de cuadro delimitador

Estas operaciones fragmentan el cuadro delimitador de un valor temporal con respecto a un mosaico multidimensional. Ofrecen una alternativa a las operaciones de la Sección 9.5 para dividir los cuadros delimitadores especificando el tamaño máximo de los cuadros en las distintas dimensiones.

- Devuelve una matriz de rangos de valores o de tiempo obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores o de tiempo

```
valueBins(tnumber,vsize number,vorigin number=0) → numspan[]
timeBins(temp,duration interval,torigin timestampz='2000-01-03') → tstzspan[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Si no se especifica el origen de los valores, se establece por defecto en 0 para los rangos de valores y en el lunes 3 de enero de 2000 para los rangos de tiempo.

```
SELECT valueBins(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 3);
-- {"[1, 3)","[4, 5)"}
SELECT valueBins(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3, 1);
-- {"[1, 4)","[4, 4)"}

```

```
SELECT timeBins(ttext '{AAA@2000-01-01, BBB@2000-01-02, AAA@2000-01-03, CCC@2000-01-04,
AAA@2000-01-05}', '3 days');
-- {"[2000-01-01, 2000-01-02)","[2000-01-03, 2000-01-05)"}
SELECT timeBins(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', '3 days', '2000-01-01');
-- {"[2000-01-01, 2000-01-04)","[2000-01-04, 2000-01-05)"}

```

- Devuelve una matriz de cuadros temporales obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores y/o tiempo

`valueBoxes(tnumber, size number, vorigin number=0) → tbox[]`

`timeBoxes(tnumber, duration interval, torigin timestamptz='2000-01-03') → tbox[]`

`valueTimeBoxes(tnumber, size number, duration interval, vorigin number=0, torigin timestamptz='2000-01-03') → tbox[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Si el origen de la dimensión de valores y/o de tiempo no se especifica, se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente.

```
SELECT valueBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 3);
/* {"TBOXINT XT([1, 3),[2000-01-01, 2000-01-05])",
    "TBOXINT XT([4, 5),[2000-01-04, 2000-01-04])"} */
SELECT timeBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', '3 days');
/* {"TBOXINT XT([1, 3),[2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 5),[2000-01-03, 2000-01-05])"} */
SELECT valueTimeBoxes(tint '{1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05}', 3, '3 days');
/* {"TBOXINT XT([1, 3),[2000-01-01, 2000-01-02])",
    "TBOXINT XT([1, 2),[2000-01-03, 2000-01-05])",
    "TBOXINT XT([4, 5),[2000-01-04, 2000-01-04])"} */
SELECT valueTimeBoxes(tfloat '[1@2000-01-01, 2@2000-01-02, 1@2000-01-03, 4@2000-01-04,
1@2000-01-05]', 3, '3 days', 1, '2000-01-01');
/* {"TBOXFLOAT XT([1, 4),[2000-01-01, 2000-01-04])",
    "TBOXFLOAT XT([1, 4),[2000-01-04, 2000-01-05])",
    "TBOXFLOAT XT([4, 4),[2000-01-04, 2000-01-04])"} */

```

- Devuelve una matriz de cuadros espaciotemporales obtenidos a partir de los instantes o segmentos de un punto temporal con respecto a un mosaico espacial y/o temporal 

`spaceBoxes(tgeompoint, xsize float, [ysize float, zsize float,]`

`sorigin geompoint='Point(0 0 0)', borderInc bool=true) → stbox[]`

`timeBoxes(tgeompoint, duration interval, torigin timestamptz='2000-01-03',`

`borderInc bool=true) → stbox[]`

`spaceTimeBoxes(tgeompoint, xsize float, [ysize float, zsize float,] duration interval,`

`sorigin geompoint='Point(0 0 0)', torigin timestamptz='2000-01-03',`

`borderInc bool=true) → stbox[]`

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Los argumentos `ysize` y `zsize` son opcionales, se supone que el tamaño de las dimensiones faltantes es igual a `xsize`. El SRID de las coordenadas

del mosaico se determina por el punto temporal y los tamaños se dan en las unidades del SRID. Si se proporciona el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales a los del punto temporal, de lo contrario se genera un error. Si el origen de la dimensión espacial y/o el tiempo no se especifica, su valor se establece por defecto en 'Point(0 0 0)' y en el lunes 3 de enero de 2000, respectivamente. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan mosaicos adicionales que contienen el borde.

```
SELECT spaceBoxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(1 1)@2000-01-03, Point(4 4)@2000-01-04, Point(1 1)@2000-01-05}', 3);
/* {"STBOX XT((1,1),(2,2)), [2000-01-01, 2000-01-05]"},
   "STBOX XT((4,4),(4,4)), [2000-01-04, 2000-01-04]"} */
SELECT timeBoxes(tgeompoint '{Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
Point(1 1 1)@2000-01-03, Point(4 4 4)@2000-01-04, Point(1 1 1)@2000-01-05}',
interval '2 days', '2000-01-01');
/* {"STBOX ZT((1,1,1),(2,2,2)), [2000-01-01, 2000-01-02]"},
   "STBOX ZT((1,1,1),(4,4,4)), [2000-01-03, 2000-01-04]"},
   "STBOX ZT((1,1,1),(1,1,1)), [2000-01-05, 2000-01-05]"} */
SELECT spaceTimeBoxes(tgeompoint '{Point(1 1)@2000-01-01, Point(2 2)@2000-01-02,
Point(1 1)@2000-01-03, Point(4 4)@2000-01-04, Point(1 1)@2000-01-05}', 3, '3 days');
/* {"STBOX XT((1,1),(2,2)), [2000-01-01, 2000-01-02]"},
   "STBOX XT((1,1),(1,1)), [2000-01-03, 2000-01-05]"},
   "STBOX XT((4,4),(4,4)), [2000-01-04, 2000-01-04]"} */
SELECT spaceTimeBoxes(tgeompoint '[Point(1 1 1)@2000-01-01, Point(2 2 2)@2000-01-02,
Point(1 1 1)@2000-01-03, Point(4 4 4)@2000-01-04, Point(1 1 1)@2000-01-05]',
3, interval '3 days', 'Point(1 1 1)', '2000-01-01');
/* {"STBOX ZT((1,1,1),(4,4,4)), [2000-01-01, 2000-01-04]"},
   "STBOX ZT((1,1,1),(4,4,4)), [2000-01-04, 2000-01-05]"},
   "STBOX ZT((4,4,4),(4,4,4)), [2000-01-04, 2000-01-04]"} */
```

9.6.4. Operaciones de división

Estas funciones fragmentan un valor temporal con respecto a una secuencia de intervalos (ver la Sección 9.6.1) o un mosaico multidimensional (ver la Sección 9.6.2).

- Fragmentar un número temporal con respecto a intervalos de valores y/o de tiempo {}

`valueSplit(tnumber,width number,origin number=0) → {(number,tnumber)}`

`timeSplit(ttype,duration interval,origin timestampz='2000-01-03') → {(time,temp)}`

`valueTimeSplit(tnumber,width number,duration interval,vorigin number=0, torigin timestampz='2000-01-03') → {(number,time,tnumber)}`

El resultado es un conjunto de pares o un conjunto de triples. Si no se especifica el origen de la dimensión de valores o temporal, se establecen por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente.

```
SELECT (sp).number, (sp).tnumber
FROM (SELECT valueSplit(tint '[1@2001-01-01, 2@2001-01-02, 5@2001-01-05, 10@2001-01-10]',
2) AS sp) t;
-- 0 | {[1@2001-01-01, 1@2001-01-02]}
-- 2 | {[2@2001-01-02, 2@2001-01-05]}
-- 4 | {[5@2001-01-05, 5@2001-01-10]}
-- 10 | {[10@2001-01-10]}
SELECT valueSplit(tfloat '[1@2001-01-01, 10@2001-01-10]', 2.0, 1.0);
-- (1,"{[1@2001-01-01, 3@2001-01-03]}")
-- (3,"{[3@2001-01-03, 5@2001-01-05]}")
-- (5,"{[5@2001-01-05, 7@2001-01-07]}")
-- (7,"{[7@2001-01-07, 9@2001-01-09]}")
-- (9,"{[9@2001-01-09, 10@2001-01-10]}")
```

```

SELECT (ts).time, (ts).temp
FROM (SELECT timeSplit(tfloat '[1@2001-02-01, 10@2001-02-10]', '2 days') AS ts) t;
-- 2001-01-31 | [1@2001-02-01, 2@2001-02-02)
-- 2001-02-02 | [2@2001-02-02, 4@2001-02-04)
-- 2001-02-04 | [4@2001-02-04, 6@2001-02-06)
-- ...
SELECT (ts).time, asText((ts).temp) AS temp
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  '2 days', '2001-02-01') AS ts) AS t;
-- 2001-02-01 | [POINT(1 1)@2001-02-01, POINT(3 3)@2001-02-03)
-- 2001-02-03 | [POINT(3 3)@2001-02-03, POINT(5 5)@2001-02-05)
-- 2001-02-05 | [POINT(5 5)@2001-02-05, POINT(7 7)@2001-02-07)
-- ...

```

Observe que se puede fragmentar un valor temporal en intervalos de tiempo cíclicos (en lugar de lineales). Los siguientes dos ejemplos muestran cómo fragmentar un valor temporal por hora y por día de la semana.

```

SELECT (ts).time::time AS hour, merge((ts).temp) AS temp
FROM (SELECT timeSplit(tfloat '[1@2001-01-01, 10@2001-01-03]', '1 hour') AS ts) t
GROUP BY hour ORDER BY hour;
/* 00:00:00 | {[1@2001-01-01 00:00:00+01, 1.1875@2001-01-01 01:00:00+01),
  [5.5@2001-01-02 00:00:00+01, 5.6875@2001-01-02 01:00:00+01)} */
/* 01:00:00 | {[1.1875@2001-01-01 01:00:00+01, 1.375@2001-01-01 02:00:00+01),
  [5.6875@2001-01-02 01:00:00+01, 5.875@2001-01-02 02:00:00+01)} */
/* 02:00:00 | {[1.375@2001-01-01 02:00:00+01, 1.5625@2001-01-01 03:00:00+01),
  [5.875@2001-01-02 02:00:00+01, 6.0625@2001-01-02 03:00:00+01)} */
/* 03:00:00 | {[1.5625@2001-01-01 03:00:00+01, 1.75@2001-01-01 04:00:00+01),
  [6.0625@2001-01-02 03:00:00+01, 6.25@2001-01-02 04:00:00+01)} */
/* ... */
SELECT EXTRACT(DOW FROM (ts).time) AS dow_no, TO_CHAR((ts).time, 'Dy') AS dow,
  asText(round(merge((ts).temp), 2)) AS temp
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-01-01, Point(10 10)@2001-01-14]',
  '1 hour') AS ts) t
GROUP BY dow, dow_no ORDER BY dow_no;
/* 0 | Sun | {[POINT(1 1)@2001-01-01, POINT(1.69 1.69)@2001-01-02),
  [POINT(5.85 5.85)@2001-01-08, POINT(6.54 6.54)@2001-01-09)} */
/* 1 | Mon | {[POINT(1.69 1.69)@2001-01-02, POINT(2.38 2.38)@2001-01-03),
  [POINT(6.54 6.54)@2001-01-09, POINT(7.23 7.23)@2001-01-10)} */
/* 2 | Tue | {[POINT(2.38 2.38)@2001-01-03, POINT(3.08 3.08)@2001-01-04),
  [POINT(7.23 7.23)@2001-01-10, POINT(7.92 7.92)@2001-01-11)} */
/* ... */

```

```

SELECT (sp).number, (sp).time, (sp).tnumber
FROM (SELECT valueTimeSplit(tint '[1@2001-02-01, 2@2001-02-02, 5@2001-02-05,
  10@2001-02-10]', 5, '5 days') AS sp) t;
-- 0 | 2001-02-01 | {[1@2001-02-01, 2@2001-02-02, 2@2001-02-05)}
-- 5 | 2001-02-01 | {[5@2001-02-05, 5@2001-02-06)}
-- 5 | 2001-02-06 | {[5@2001-02-06, 5@2001-02-10)}
-- 10 | 2001-02-06 | {[10@2001-02-10)}
SELECT (sp).number, (sp).time, (sp).tnumber
FROM (SELECT valueTimeSplit(tfloat '[1@2001-02-01, 10@2001-02-10]', 5.0, '5 days', 1.0,
  '2001-02-01') AS sp) t;
-- 1 | 2001-01-01 | [1@2001-01-01, 6@2001-01-06)
-- 6 | 2001-01-06 | [6@2001-01-06, 10@2001-01-10)

```

■ Fragmentar un punto temporal con respecto a una malla espacial o espacio-temporal {}

```

spaceSplit(tgeompoint, xsize float, [ysize float, zsize float],
  origin geompoint='Point(0 0 0)', bitmatrix boolean=true, borderInc bool=true) →
  {(point, tpoint)}

```

```
timeSplit(tgeompoint,duration interval,torigin timestamptz='2000-01-03',
  bitmatrix boolean=true,borderInc boolean=true) → {(point,time,tpoint)}
spaceTimeSplit(tgeompoint,xsize float,[ysize float,zsize float,]
  duration interval,sorigin geompoint='Point(0 0 0)',
  toorigin timestamptz='2000-01-03',bitmatrix boolean=true,borderInc boolean=true) →
  {(point,time,tpoint)}
```

El resultado es un conjunto de pares o un conjunto de triples. Si el origen de la dimensión espacial y/o el tiempo no se especifica, su valor se establece por defecto en 'Point(0 0 0)' y en el lunes 3 de enero de 2000, respectivamente. Los argumentos `ysize` y `zsize` son opcionales, se supone que el tamaño de las dimensiones faltantes es igual a `xsize`. Si no se especifica el argumento `bitmatrix`, el cálculo utilizará una matriz de bits para acelerar el proceso. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión espacial y, por lo tanto, se generan mosaicos adicionales que contienen el borde.

```
SELECT ST_AsText((sp).point) AS point, asText((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1)@2001-03-01, Point(10 10)@2001-03-10]',
  2.0) AS sp) t;
-- POINT(0 0) | {[POINT(1 1)@2001-03-01, POINT(2 2)@2001-03-02]}
-- POINT(2 2) | {[POINT(2 2)@2001-03-02, POINT(4 4)@2001-03-04]}
-- POINT(4 4) | {[POINT(4 4)@2001-03-04, POINT(6 6)@2001-03-06]}
-- ...
SELECT ST_AsText((sp).point) AS point, asText((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1 1)@2001-03-01,
  Point(10 10 10)@2001-03-10]', 2.0, geometry 'Point(1 1 1)') AS sp) t;
-- POINT Z(1 1 1) | {[POINT Z(1 1 1)@2001-03-01, POINT Z(3 3 3)@2001-03-03]}
-- POINT Z(3 3 3) | {[POINT Z(3 3 3)@2001-03-03, POINT Z(5 5 5)@2001-03-05]}
-- POINT Z(5 5 5) | {[POINT Z(5 5 5)@2001-03-05, POINT Z(7 7 7)@2001-03-07]}
-- ...
```

```
SELECT (sp).time, asText((sp).temp) AS tpoint
FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  interval '2 days') AS sp) t;
-- 2001-01-31 | [POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02]
-- 2001-02-02 | [POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04]
-- 2001-02-04 | [POINT(4 4)@2001-02-04, POINT(6 6)@2001-02-06]
-- ...
```

```
SELECT ST_AsText((sp).point) AS point, (sp).time, asText((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  2.0, interval '2 days') AS sp) t;
-- POINT(0 0) | 2001-01-31 | {[POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-01-31 | {[POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-02-02 | {[POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04]}
-- ...
SELECT ST_AsText((sp).point) AS point, (sp).time, asText((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1 1)@2001-02-01,
  Point(10 10 10)@2001-02-10]', 2.0, interval '2 days', 'Point(1 1 1)',
  '2001-03-01') AS sp) t;
-- POINT Z(1 1 1) | 2001-02-01 | {[POINT Z(1 1 1)@2001-02-01, POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-01 | {[POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-03 | {[POINT Z(3 3 3)@2001-02-03, POINT Z(5 5 5)@2001-02-05]}
-- ...
```

Capítulo 10

Tipos temporales: Agregación e indexación

10.1. Agregación

Las funciones agregadas temporales generalizan las funciones agregadas tradicionales. Su semántica es que calculan el valor de la función en cada instante de la *unión* de las extensiones temporales de los valores a agregar. En contraste, recuerde que todas las otras funciones que manipulan tipos temporales calculan el valor de la función en cada instante de la *intersección* de las extensiones temporales de los argumentos.

Las funciones agregadas temporales son las siguientes:

- Para todos los tipos temporales, la función `tCount` generaliza la función tradicional `count`. El conteo temporal se puede utilizar para calcular en cada momento el número de objetos disponibles (por ejemplo, el número de coches en un área).
- Para todos los tipos temporales, la función `extent` devuelve un cuadro delimitador que engloba un conjunto de valores temporales. Dependiendo del tipo de base, el resultado de esta función puede ser un `tstzspan`, un `tbox` o un `stbox`.
- Para el tipo booleano temporal, las funciones `tAnd` y `tOr` generalizan las funciones tradicionales `and` y `or`.
- Para los tipos numéricos temporales hay dos tipos de funciones agregadas temporales. Las funciones `tmin`, `tMax`, `tSum` y `tAvg` generalizan las funciones tradicionales `min`, `max`, `sum` y `avg`. Además, las funciones `wMin`, `wMax`, `wCount`, `wSum` y `wAvg` son versiones de ventana (o acumulativas) de las funciones tradicionales que, dado un intervalo de tiempo `w`, calculan el valor de la función en un instante `t` considerando los valores durante el intervalo `[t-w, t]`. Todas las funciones agregadas de ventana están disponibles para enteros temporales, mientras que para flotantes temporales sólo son significativos el mínimo y el máximo de ventana.
- Para el tipo texto temporal, las funciones `tMin` y `tMax` generalizan las funciones tradicionales `min` y `max`.
- Finalmente, para puntos temporales, la función `tCentroid` generaliza la función `ST_Centroid` proporcionada por PostGIS. Por ejemplo, dado un conjunto de objetos que se mueven juntos (es decir, un convoy o una bandada), el centroide temporal producirá un punto temporal que representa en cada instante el centro geométrico (o el centro de masa) de todos los objetos en movimiento.

nota

Los nombres `tMin`, `tMax`, `merge`, `appendInstant` y `appendSequence` designan tanto una función escalar como una agregación. El nombre canónico de la forma de agregación lleva el sufijo `Agg` (`tMinAgg`, `tMaxAgg`, `mergeAgg`, `appendInstantAgg` y `appendSequenceAgg`), de modo que los nombres escalar y de agregación sean distintos. Los nombres de agregación sin sufijo siguen disponibles como alias de compatibilidad y están obsoletos; se eliminarán en una versión mayor futura. Las formas escalares (los accesores de caja `tMin/tMax` y el constructor binario `merge`) conservan sus nombres.

En los ejemplos que siguen, suponemos que las tablas `Department` y `Trip` contienen las dos tuplas introducidas en la Sección 4.2.

■ Conteo temporal

`tCount(ttype) → {tintSeq,tintSeqSet}`

```
SELECT tCount(NoEmps) FROM Department;
-- {[1@2001-01-01, 2@2001-02-01, 1@2001-08-01, 1@2001-10-01]}
```

■ Extensión del cuadro delimitador

`extent(temp) → {tstzspan,tbox,stbox}`

```
SELECT extent(noEmps) FROM Department;
-- TBOX XT((4,12),[2001-01-01,2001-10-01])
SELECT extent(Trip) FROM Trips;
-- STBOX XT(((0,0),(3,3),[2001-01-01 08:00:00+01, 2001-01-01 08:20:00+01]))
```

■ Y, o temporal

`tAnd(tbool) → tbool`

`tOr(tbool) → tbool`

```
SELECT tAnd(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-04-01, f@2001-10-01]}
SELECT tOr(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-08-01, f@2001-10-01]}
```

■ Mínimo, máximo, suma y promedio temporal

`tMin(ttype) → ttype`

`tMax(ttype) → ttype`

`tSum(tnumber) → {tnumberSeq,tnumberSeqSet}`

`tAvg(tnumber) → {tfloatSeq,tfloatSeqSet}`

```
SELECT tMin(NoEmps) FROM Department;
-- {[10@2001-01-01, 4@2001-02-01, 6@2001-06-01, 6@2001-10-01]}
SELECT tMax(NoEmps) FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-01, 6@2001-10-01]}
SELECT tSum(NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01, 18@2001-06-01, 6@2001-08-01,
6@2001-10-01]} */
SELECT tAvg(NoEmps) FROM Department;
/* {[10@2001-01-01, 10@2001-02-01), [7@2001-02-01, 7@2001-04-01),
[8@2001-04-01, 8@2001-06-01), [9@2001-06-01, 9@2001-08-01),
[6@2001-08-01, 6@2001-10-01) */
```

■ Conteo de ventana

`wCount(tnumber,interval) → {tintSeq,tintSeqSet}`

```
SELECT wCount(NoEmps, interval '2 days') FROM Department;
/* {[1@2001-01-01, 2@2001-02-01, 3@2001-04-01, 2@2001-04-03, 3@2001-06-01, 2@2001-06-03,
1@2001-08-03, 1@2001-10-03]} */
```

■ Mínimo, máximo, suma y promedio de ventana

`wMin(tnumber,interval) → {tnumberSeq,tnumberSeqSet}`

`wMax(tnumber,interval) → {tnumberSeq,tnumberSeqSet}`

`wSum(tint,interval) → {tintSeq,tintSeqSet}`

`wAvg(tint,interval) → {tfloatDiscSeq,tfloatSeqSet}`

```
SELECT wMin(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 4@2001-04-01, 6@2001-06-03, 6@2001-10-03]}
SELECT wMax(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-03, 6@2001-10-03]}
SELECT wSum(NoEmps, interval '2 days') FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 26@2001-04-01, 16@2001-04-03, 22@2001-06-01,
    18@2001-06-03, 6@2001-08-03, 6@2001-10-03]} */
SELECT round(wAvg(NoEmps, interval '2 days'), 3) FROM Department;
/* Interp=Step;{[10@2001-01-01, 7@2001-02-01, 8.667@2001-04-01, 8@2001-04-03,
    7.333@2001-06-01, 9@2001-06-03, 6@2001-08-03, 6@2001-10-03]} */
```

■ Centroide temporal

tCentroid(tgeompoint) → tgeompoint

```
SELECT tCentroid(Trip) FROM Trips;
/* {[POINT(0 0)@2001-01-01 08:00:00+00, POINT(1 0)@2001-01-01 08:05:00+00),
    [POINT(0.5 0)@2001-01-01 08:05:00+00, POINT(1.5 0.5)@2001-01-01 08:10:00+00,
    POINT(2 1.5)@2001-01-01 08:15:00+00),
    [POINT(2 2)@2001-01-01 08:15:00+00, POINT(3 3)@2001-01-01 08:20:00+00]} */
```

10.2. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de tipos temporales. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple n-dimensional. Ejemplos de creación de índices son los siguientes:

```
CREATE INDEX Department_NoEmps_Gist_Idx ON Department USING Gist(NoEmps);
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para los tipos temporales. Como se explica en el Capítulo 4, estos son

- el tipo tstzspan para los tipos tbool y ttext,
- el tipo tbox par los tipos tint y tfloat,
- el tipo stbox para los tipos tgeompoint y tgeogpoint.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores (consulte la Sección 5.3 para obtener más información):

- <<, &<, &>, >>, que sólo consideran la dimensión de valores en tipos alfanuméricos temporales,
- <<, &<, &>, >>, <<|, &<|, |&>, |>>, &</, <</, />> y /&>, que sólo consideran la dimensión espacial en tipos de puntos temporales,
- &<#, <<#, #>>, #&>, que sólo consideran la dimensión temporal para todos los tipos temporales,
- &&, @>, <@, ~= y |=|, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta. Estos operadores trabajan en cuadros delimitadores (es decir, tstzspan, tbox o stbox), no los valores completos.

Por ejemplo, dado el índice definido anteriormente en la tabla `Department` y una consulta que implica una condición con el operador `&&` (superposición), si el argumento derecho es un flotante temporal, entonces se consideran tanto el valor como las dimensiones de tiempo para filtrar las tuplas de la relación, mientras que si el argumento derecho es un valor flotante, un rango flotante o un tipo de tiempo, entonces el valor o la dimensión de tiempo se utilizará para filtrar las tuplas de la relación. Además, se puede construir un cuadro delimitador a partir de un valor/rango y/o una marca de tiempo/período, que se puede usar para filtrar las tuplas de la relación. Ejemplos de consultas que utilizan el índice en la tabla `Department` definida anteriormente se dan a continuación.

```
SELECT * FROM Department WHERE NoEmps && intspan '[1, 5)';
SELECT * FROM Department WHERE NoEmps && tstzspan '[2001-04-01, 2001-05-01)';
SELECT * FROM Department WHERE NoEmps &&
    tbox(intspan '[1, 5)', tstzspan '[2001-04-01, 2001-05-01)');
SELECT * FROM Department WHERE NoEmps &&
    tfloat '{[1@2001-01-01, 1@2001-02-01), [5@2001-04-01, 5@2001-05-01)}';
```

Del mismo modo, los ejemplos de consultas que utilizan el índice en la tabla `Trips` definida anteriormente se dan a continuación.

```
SELECT * FROM Trips WHERE Trip && geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))';
SELECT * FROM Trips WHERE Trip && timestampz '2001-01-01';
SELECT * FROM Trips WHERE Trip && tstzspan '[2001-01-01, 2001-01-05)';
SELECT * FROM Trips WHERE Trip &&
    stbox(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))', tstzspan '[2001-01-01, 2001-01-05)');
SELECT * FROM Trips WHERE Trip &&
    tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 1)@2001-01-05)}';
```

Finalmente, se pueden crear índices de árbol B para columnas de tabla de todos los tipos temporales. Para este tipo de índice, la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos temporales, con los correspondientes operadores `<`, `<=`, `>` y `>=`, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte de árbol B para tipos temporales está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Para acelerar algunas de las funciones de los tipos temporales, se puede agregar en la cláusula `WHERE` de las consultas una comparación de cuadro delimitador que hace uso de los índices disponibles. Por ejemplo, este sería típicamente el caso de las funciones que proyectan los tipos temporales a las dimensiones de valor/espacio y/o tiempo. Esto filtrará las tuplas con un índice como se muestra en la siguiente consulta.

```
SELECT atTime(T.Trip, tstzspan '[2001-01-01, 2001-01-02)')
FROM Trips T
-- Filtro de índice con cuadro delimitador
WHERE T.Trip && tstzspan '[2001-01-01, 2001-01-02)';
```

En el caso de los geometrías temporales, todas las relaciones espaciales con la semántica alguna vez o siempre (ver la Sección 8.5) incluyen automáticamente una comparación de cuadro delimitador que hará uso de cualquier índice que esté disponible en los puntos temporales. Por esta razón, la primera versión de las relaciones se usa típicamente para filtrar las tuplas con la ayuda de un índice al calcular las relaciones temporales como se muestra en la siguiente consulta.

```
SELECT tIntersects(T.Trip, R.Geom)
FROM Trips T, Regions R
-- Filtro de índice con cuadro delimitador
WHERE eIntersects(T.Trip, R.Geom);
```

10.3. Estadísticas y selectividad

10.3.1. Colecta de estadísticas

El planificador de PostgreSQL se basa en información estadística sobre el contenido de las tablas para generar el plan de ejecución más eficiente para las consultas. Estas estadísticas incluyen una lista de algunos de los valores más comunes en cada columna y un histograma que muestra la distribución aproximada de datos en cada columna. Para tablas grandes, se toma una muestra aleatoria del contenido de la tabla, en lugar de examinar cada fila. Esto permite analizar tablas grandes en poco tiempo. La información estadística es recopilada por el comando `ANALYZE` y es almacenada en la tabla de catálogo `pg_statistic`. Dado que diferentes tipos de estadísticas pueden ser apropiados para diferentes tipos de datos, la tabla sólo almacena estadísticas muy generales (como el número de valores nulos) en columnas dedicadas. Todo lo demás se almacena en cinco “slots”, que son pares de columnas de matriz que almacenan las estadísticas de una columna de un tipo arbitrario.

Las estadísticas recopiladas para tipos de tiempo y tipos temporales se basan en las recopiladas por PostgreSQL para tipos escalares y tipos de rango. Para tipos escalares, como `float`, se recopilan las siguientes estadísticas:

1. fracción de valores nulos,
2. ancho promedio, en bytes, de valores no nulos,
3. número de diferentes valores no nulos,
4. matriz de los valores más comunes y matriz de sus frecuencias,
5. histograma de valores, donde se excluyen los valores más comunes,
6. correlación entre el orden de filas físico y lógico.

Para los tipos de rango, como `tstzspan`, se recopilan tres histogramas adicionales:

7. histograma de longitud de rangos no vacíos,
8. histogramas de límites superior e inferior.

Para geometrías, además de (1)–(3), se recopilan las siguientes estadísticas:

9. número de dimensiones de los valores, cuadro delimitador N-dimensional, número de filas en la tabla, número de filas en la muestra, número de valores no nulos,
10. Histograma N-dimensional que divide el cuadro delimitador en varias celdas y mantiene la proporción de valores que se cruzan con cada celda.

Las estadísticas recopiladas para columnas de los tipos de tiempo y de rango `tstzset`, `tstzspan`, `tstzspanset`, `intspan` y `floatspan` replican las recopiladas por PostgreSQL para `tstzrange`. Esto es claro para los tipos de rango en MobilityDB, que son versiones más eficientes de los tipos de rango en PostgreSQL. Para los tipos `tstzset` y `tstzspanset`, un valor se convierte en su período delimitador y luego se recopilan las estadísticas del tipo `tstzspan`.

Las estadísticas recopiladas para columnas de los tipos temporales dependen del subtipo temporal y del tipo base. Además de las estadísticas (1)–(3) que se recopilan para todos los tipos temporales, las estadísticas se recopilan para la dimensiones de tiempo y de valor de forma independiente. Más precisamente, se recopilan las siguientes estadísticas para la dimensión de tiempo:

- Para columnas de subtipo instante, las estadísticas (4)–(6) se recopilan para las marcas de tiempo.
- Para columnas de otro subtipo, las estadísticas (7)–(8) se recopilan para los períodos delimitadores.

Las siguientes estadísticas se recopilan para la dimensión de valores:

- Para columnas de tipos temporales con interpolación escalonada (es decir, `tbool`, `ttext` o `tint`):
 - Para el subtipo instante, las estadísticas (4)–(6) se recopilan para los valores.
 - Para todas los demás subtipos, las estadísticas (7)–(8) se recopilan para los valores.
- Para columnas de tipos temporales flotantes (es decir, `tfloat`):
 - Para el subtipo instante, las estadísticas (4)–(6) se recopilan para los valores.
 - Para todas los demás subtipos, las estadísticas (7)–(8) se recopilan por los rangos delimitadores de valores.
- Para columnas de tipos de puntos temporales (es decir, `tgeompoint` y `tgeogpoint`) las estadísticas (9)–(10) se compilan para los puntos.

10.3.2. Estimación de la selectividad

Los operadores booleanos en PostgreSQL se pueden asociar con dos funciones de selectividad, que calculan la probabilidad de que un valor de un tipo dado coincida con un criterio dado. Estas funciones de selectividad se basan en las estadísticas recopiladas. Hay dos tipos de funciones de selectividad. Las funciones de selectividad de *restricción* intentan estimar el porcentaje de filas en una tabla que satisfacen una condición en la cláusula `WHERE` de la forma `column OP constant`. Por otro lado, las funciones de selectividad de *unión* intentan estimar el porcentaje de filas en una tabla que satisfacen una condición en la cláusula `WHERE` de la forma `table1.column1 OP table2.column2`.

MobilityDB define 23 clases de operadores booleanos (como `=`, `<`, `&&`, `<<`, etc.), cada uno de los cuales puede tener como argumentos izquierdo o derecho un tipo PostgreSQL (como `integer`, `timestampz`, etc.) o un tipo MobilityDB (como `tstzspan`, `tint`, etc.). Como consecuencia, existe un número muy elevado de operadores con diferentes argumentos a considerar para las funciones de selectividad. El enfoque adoptado fue agrupar estas combinaciones en clases correspondientes a las dimensiones de valor o de tiempo. Las clases corresponden al tipo de estadísticas recopiladas como se explica en la sección anterior.

MobilityDB estima la selectividad de restricción y de combinación para tipos de tiempo, de rango y temporales.

Capítulo 11

Puntos de red temporales

Los puntos temporales que hemos considerado hasta ahora representan el movimiento de objetos que pueden moverse libremente en el espacio ya que se supone que pueden cambiar su posición de un lugar a otro sin ninguna restricción de movimiento. Este es el caso de los animales y de los objetos voladores como aviones o drones. Sin embargo, en muchos casos, los objetos no se mueven libremente en el espacio sino dentro de redes integradas espacialmente, como rutas o ferrocarriles. En este caso, es necesario tener en cuenta de las redes integradas al describir los movimientos de estos objetos en movimiento. Los puntos de red temporales tienen en cuenta estos requisitos.

En comparación con los puntos temporales de espacio libre, los puntos basados en red tienen las siguientes ventajas:

- Los puntos de red temporales proporcionan restricciones que reflejan los movimientos reales de los objetos en movimiento.
- La información geométrica no se almacena con el punto móvil, sino de una vez por todas en las redes fijas. De esta forma, las representaciones e interpolaciones de la ubicación son más precisas.
- Los puntos de red temporales son más eficientes en términos de almacenamiento de datos, actualización de ubicación, formulación de consultas e indexación. Estos aspectos se tratan más adelante en este documento.

Los puntos de red temporales se basan en **pgRouting**, una extensión de PostgreSQL para desarrollar aplicaciones de enrutamiento de red y realizar análisis de gráficos. Por lo tanto, los puntos de red temporal asumen que la red subyacente está definida en una tabla llamada `ways`, que tiene al menos tres columnas: `gid` que contiene el identificador de ruta único, `length` que contiene la longitud de la ruta y `the_geom` que contiene la geometría de la ruta.

Hay dos tipos de red estáticos, `npoint` (abreviatura de *network point*) y `nsegment` (abreviatura de *network segment*), que representan, respectivamente, un punto y un segmento de una ruta. Un valor `npoint` se compone de un identificador de ruta y un número flotante en el rango `[0,1]` que determina una posición relativa de la ruta, donde 0 corresponde al comienzo de la ruta y 1 al final de la ruta. Un valor de `nsegment` se compone de un identificador de ruta y dos números flotantes en el rango `[0,1]` que determinan las posiciones relativas de inicio y finalización. Un valor de `nsegment` cuyas posiciones inicial y final son iguales corresponde a un valor de `npoint`.

El tipo `npoint` sirve como tipo base para definir el tipo punto de red temporal `tnpoint`. El tipo `tnpoint` tiene una funcionalidad similar al tipo de punto temporal `tgeompoint` con la excepción de que solo considera dos dimensiones. Por lo tanto, todas las funciones y operadores descritos anteriormente para el tipo `tgeompoint` también son aplicables para el tipo `tnpoint`. Además, hay funciones específicas definidas para el tipo `tnpoint`.

11.1. Tipos de red estáticos

Un valor `npoint` es un par de la forma `(rid, position)` donde `rid` es un valor `bigint` que representa un identificador de ruta y `position` es un valor `float` en el rango `[0,1]` que indica su posición relativa. Los valores 0 y 1 de `position` denotan, respectivamente, la posición inicial y final de la ruta. La distancia de la ruta entre un valor de `npoint` y la posición inicial de la ruta con el identificador `rid` se calcula multiplicando `position` por `length`, donde este último es la longitud de la ruta. Ejemplos de entrada de valores de puntos de red son los siguientes:

```
SELECT npoint 'Npoint(76, 0.3)';
SELECT npoint 'Npoint(64, 1.0)';
```

La función de constructor para puntos de red tiene un argumento para el identificador de ruta y un argumento para la posición relativa. Un ejemplo de un valor de punto de red definido con la función constructora es el siguiente:

```
SELECT npoint(76, 0.3);
```

Un valor `nsegment` es un triple de la forma `(rid, startPosition, endPosition)` donde `rid` es un valor `bigint` que representa un identificador de ruta y `startPosition` y `endPosition` son valores de `float` en el rango `[0,1]` tal que $startPosition \leq endPosition$. Semánticamente, un segmento de red representa un conjunto de puntos de red `(rid, position)` con $startPosition \leq position \leq endPosition$. Si $startPosition = 0$ y $endPosition = 1$, el segmento de red es equivalente a la ruta completa. Si $startPosition = endPosition$, el segmento de red representa un único punto de red. Ejemplos de entrada de valores de puntos de red son los siguientes:

```
SELECT nsegment 'Nsegment(76, 0.3, 0.5)';
SELECT nsegment 'Nsegment(64, 0.5, 0.5)';
SELECT nsegment 'Nsegment(64, 0.0, 1.0)';
SELECT nsegment 'Nsegment(64, 1.0, 0.0)';
-- convertido a nsegment 'Nsegment(64, 0.0, 1.0)';
```

Como se puede ver en el último ejemplo, los valores `startPosition` y `endPosition` se invertirán para asegurar que la condición $startPosition \leq endPosition$ siempre se satisface. La función de constructor para segmentos de red tiene un argumento para el identificador de ruta y dos argumentos opcionales para las posiciones inicial y final. Los ejemplos de valores de segmento de red definidos con la función constructora son los siguientes:

```
SELECT nsegment(76, 0.3, 0.3);
SELECT nsegment(76); -- se asume que las posiciones inicial y final son 0 y 1
SELECT nsegment(76, 0.5); -- se asume que la posición final es 1
```

Los valores del tipo `npoint` se pueden convertir al tipo `nsegment` usando un `CAST` explícito o usando la notación `::` como se muestra a continuación.

```
SELECT npoint(76, 0.33)::nsegment;
```

Los valores de los tipos de red estáticos deben satisfacer varias restricciones para que estén bien definidos. Estas restricciones se dan a continuación.

- El identificador de ruta `rid` debe encontrarse en la columna `gid` de la tabla `ways`.
- Los valores de `position`, `startPosition` y `endPosition` deben estar en el rango `[0,1]`. Se genera un error cuando no se cumple una de estas restricciones.

Ejemplos de valores de tipo de red estática incorrectos son los siguientes.

```
-- Valor rid incorrecto
SELECT npoint 'Npoint(87.5, 1.0)';
-- Valor de posición incorrecto
SELECT npoint 'Npoint(87, 2.0)';
-- Valor rid inexistente en la table ways
SELECT npoint 'Npoint(99999999, 1.0)';
```

Damos a continuación las funciones y operadores para los tipos de redes estáticas.

11.1.1. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
asText({npoint,npoint[]}) → {text,text[]}
```

```
asEWKT({npoint,npoint[]}) → {text,text[]}
```

```
SELECT asText(npoint 'SRID=5676;Npoint(1,0.5)');
-- NPoint(1,0.5)
SELECT asText(ARRAY[npoint 'Npoint(1,0.2)', 'Npoint(1,0.5)']);
-- {"NPoint(1,0.2)","NPoint(1,0.5)"}
SELECT asEWKT(npoint 'SRID=5676;Npoint(1,0.5)');
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(ARRAY[npoint 'SRID=5676;Npoint(1,0.2)', 'SRID=5676;Npoint(1,0.5)']);
-- {"SRID=5676;NPoint(1,0.2)","SRID=5676;NPoint(1,0.5)"}

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

```
asBinary(npoint,endian text="") → bytea
```

```
asEWKB(npoint,endian text="") → bytea
```

```
asHexWKB(npoint,endian text="") → text
```

```
asHexEWKB(npoint,endian text="") → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Un punto de red toma su SRID de la red de rutas subyacente: `asEWKB` y `asHexEWKB` lo incluyen, mientras que las formas simples `asBinary` y `asHexWKB` lo omiten.

```
SELECT asBinary(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x01010100000000000000000000000000e03f
SELECT asEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x01412c1600000100000000000000000000000000e03f
SELECT asHexWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 01010100000000000000000000000000E03F
SELECT asHexEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 01412C1600000100000000000000000000000000E03F

```

- Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
npointFromText(text) → npoint
```

```
npointFromEWKT(text) → npoint
```

```
SELECT asEWKT(npointFromText(text 'Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromEWKT(text 'SRID=5676;Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)

```

- Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

```
npointFromBinary(bytea) → npoint
```

```
npointFromEWKB(bytea) → npoint
```

```
npointFromHexEWKB(text) → npoint
```

```
SELECT asEWKT(npointFromEWKB(
  '\x01412c1600000100000000000000000000000000e03f'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromHexEWKB(
  '01412C1600000100000000000000000000000000E03F'));
-- SRID=5676;NPoint(1,0.5)
```

11.1.2. Constructores

- Constructores de puntos o segmentos de red

`npoint(bigint,double precision) → npoint`

`nsegment(bigint,double precision,double precision) → nsegment`

```
SELECT npoint(76, 0.3);
SELECT nsegment(76, 0.3, 0.5);
```

11.1.3. Conversión de tipos

Los valores de los tipos `npoint` y `nsegment` se pueden convertir al tipo `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación. De manera similar, los valores `geometry` del subtipo `point` o `linestring` (restringidos a dos puntos) se pueden convertir, respectivamente, a valores `npoint` y `nsegment`. Para ello, se debe encontrar la ruta que interseca los puntos dados, donde se supone una tolerancia de 0,00001 unidades (según el sistema de coordenadas), por lo que se considera que un punto y una ruta que están cerca se intersecan. Si no se encuentra dicha ruta, se devuelve un valor nulo.

- Convertir entre un punto o un segmento de red y una geometría

`{npoint,nsegment}::geometry`

`geometry::{npoint,nsegment}`

```
SELECT ST_AsText(npoint(76, 0.33)::geometry);
-- POINT(21.6338731332283 50.0545869554067)
SELECT ST_AsText(nsegment(76, 0.33, 0.66)::geometry);
-- LINESTRING(21.6338731332283 50.0545869554067,30.7475989651999 53.9185062927473)
SELECT ST_AsText(nsegment(76, 0.33, 0.33)::geometry);
-- POINT(21.6338731332283 50.0545869554067)
```

```
SELECT geometry 'SRID=5676;Point(279.269156511873 811.497076880187)::npoint;
-- NPoint(3,0.781413)
SELECT geometry 'SRID=5676;LINESTRING(406.729536784738 702.58583437902,
  383.570801314823 845.137059419277)::nsegment;
-- NSegment(3,0.6,0.9)
SELECT geometry 'SRID=5676;Point(279.3 811.5)::npoint;
-- NULL
SELECT geometry 'SRID=5676;LINESTRING(406.7 702.6,383.6 845.1)::nsegment;
-- NULL
```

- Construir una caja espaciotemporal a partir de un punto de red y, opcionalmente, una marca de tiempo o un período

`stbox(npoint) → stbox`

`stbox(npoint,{timestamptz,tstzspan}) → stbox`

```
SELECT stbox(npoint 'NPoint(1,0.3)');
-- SRID=5676;STBOX X((62.786633,80.143556),(62.786633,80.143556))
SELECT stbox(npoint 'NPoint(1,0.3)', timestamptz '2001-01-01');
```

```
-- SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)), [2001-01-01, ↔
2001-01-01])
SELECT stbox(npoint 'NPoint(1,0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)), [2001-01-01, ↔
2001-01-02])
```

11.1.4. Accesores

- Devuelve el identificador de ruta

route({npoint,nsegment}) → bigint

```
SELECT route(npoint 'Npoint(63, 0.3)');
-- 63
SELECT route(nsegment 'Nsegment(76, 0.3, 0.3)');
-- 76
```

- Devuelve la posición

getPosition(npoint) → float

```
SELECT getPosition(npoint 'Npoint(63, 0.3)');
-- 0.3
```

- Devuelve la posición inicial/final

startPosition(nsegment) → float

endPosition(nsegment) → float

```
SELECT startPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.3
SELECT endPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.5
```

11.1.5. Transformaciones

- Redondear la(s) posición(es) del punto de red or el segmento de red en el número de posiciones decimales

round({npoint,nsegment},integer=0) → {npoint,nsegment}

```
SELECT round(npoint(76, 0.123456789), 6);
-- NPoint(76,0.123457)
SELECT round(nsegment(76, 0.123456789, 0.223456789), 6);
-- NSegment(76,0.123457,0.223457)
```

11.1.6. Operaciones espaciales

- Devuelve el identificador de referencia espacial

SRID({npoint,nsegment}) → integer

```
SELECT SRID(npoint 'Npoint(76, 0.3)');
-- 5676
SELECT SRID(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 5676
```

Dos valores npoint pueden tener diferentes identificadores de ruta pero pueden representar el mismo punto espacial en la intersección de las dos rutas. La función equals se utiliza para verificar la igualdad espacial de los puntos de la red.

- Igualdad espacial para puntos de red

`equals(npoint, npoint)::Boolean`

```
WITH inter(geom) AS (
  SELECT st_intersection(t1.the_geom, t2.the_geom)
  FROM ways t1, ways t2 WHERE t1.gid = 1 AND t2.gid = 2),
fractions(f1, f2) AS (
  SELECT ST_LineLocatePoint(t1.the_geom, i.geom), ST_LineLocatePoint(t2.the_geom, i.geom)
  FROM ways t1, ways t2, inter i WHERE t1.gid = 1 AND t2.gid = 2)
SELECT equals(npoint(1, f1), npoint(2, f2)) FROM fractions;
-- true
```

11.1.7. Comparaciones

Los operadores de comparación (=, < y así sucesivamente) para tipos de red estáticos requieren que los argumentos izquierdo y derecho sean del mismo tipo. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en tipos de red estáticos.

- Comparaciones tradicionales

`npoint {=, <>, <, >, <=, >} npoint`
`nsegment {=, <>, <, >, <=, >} nsegment`

```
SELECT npoint 'Npoint(3, 0.5)' = npoint 'Npoint(3, 0.5)';
-- true
SELECT npoint 'Npoint(3, 0.5)' <> npoint 'Npoint(3, 0.6)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' < nsegment 'Nsegment(3, 0.5, 0.6)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' > nsegment 'Nsegment(2, 0.5, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.5)' <= npoint 'Npoint(2, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.6)' >= npoint 'Npoint(1, 0.5)';
-- true
```

11.2. Puntos de red temporales

El tipo de punto de red temporal `tnpoint` permite representar el movimiento de objetos en una red. Corresponde al tipo de punto temporal `tgeompoint` restringido a coordenadas bidimensionales. Como todos los demás tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de `tnpoint` en estos subtipos.

```
SELECT tnpoint 'Npoint(1, 0.5)@2001-01-01';
SELECT tnpoint '{Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03}';
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint '{[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03], [Npoint(2, 0.6)@2001-01-04, Npoint(2, 0.6)@2001-01-05]}';
```

El tipo de punto de red temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL). Los valores posibles para el modificador de tipo son `Instant`, `Sequence` y `SequenceSet`. Si no se especifica ningún modificador de tipo para una columna, se permiten valores de cualquier subtipo.

```
SELECT tnpoint(Sequence) '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
    Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint(Sequence) 'Npoint(1, 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Los valores de puntos de red temporales del subtipo de secuencia et interpolación linear o escalonada deben definirse en una única ruta. Por lo tanto, se necesita un valor de subtipo de conjunto de secuencias para representar el movimiento de un objeto que atraviesa varias rutas, incluso si no hay un espacio temporal. Por ejemplo, en el siguiente valor

```
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.5)@2001-01-03],
    [NPoint(2, 0.4)@2001-01-03, NPoint(2, 0.6)@2001-01-04]}';
```

el punto de red cambia su ruta en 2001-01-03.

Los valores de puntos de red temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
    NPoint(1, 0.6)@2001-01-03]';
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.6)@2001-01-03)
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.3)@2001-01-02,
    NPoint(1, 0.5)@2001-01-03], [NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04]}';
-- {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02, NPoint(1,0.7)@2001-01-04]}
```

11.3. Validez de los puntos de red temporal

Los valores de los puntos de red temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- No se permiten valores nulos
SELECT tnpoint 'NULL@2001-01-01 08:05:00';
SELECT tnpoint 'Point(0 0)@NULL';
-- El tipo base no es un punto de red
SELECT tnpoint 'Point(0 0)@2001-01-01 08:05:00';
-- Múltiples rutas en una secuencia
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01 09:00:00, Npoint(2, 0.2)@2001-01-01 09:05:00]';
```

Damos a continuación las funciones y operadores para los tipos de puntos de red. La mayoría de las funciones para tipos temporales descritas en los capítulos precedentes se pueden aplicar para tipos de puntos de red temporales. Por lo tanto, en las firmas de las funciones, la notación `base` también representa un `npoint` y las notaciones `ttype`, `tpoint` y `tgeompoint` también representan un `tnpoint`. Además, las funciones que tienen un argumento de tipo `geometry` aceptan además un argumento de tipo `npoint`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para puntos de red temporales.

11.4. Constructores

- Constructor para puntos de red temporales con valor constante

```
tnpoint(npoint,timestamp tz) → tnpointInst
```

```
tnpoint(npoint,tstzset) → tnpointDiscSeq
```

```
tnpoint(npoint,tstzspan,interp='linear') → tnpointContSeq
```

```
tnpoint(npoint,tstzspanset,interp='linear') → tnpointSeqSet
```



```

SELECT tnpoint('Npoint(1, 0.5)', timestampz '2001-01-01');
-- NPoint(1,0.5)@2001-01-01
SELECT tnpoint('Npoint(1, 0.3)', tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.3)@2001-01-03, NPoint(1,0.3)@2001-01-05}
SELECT tnpoint('Npoint(1, 0.5)', tstzspan '[2001-01-01, 2001-01-02]');
-- [NPoint(1,0.5)@2001-01-01, NPoint(1,0.5)@2001-01-02]
SELECT tnpoint('Npoint(1, 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}', 'step');
-- Interp=Step;{NPoint(1,0.2)@2001-01-01, NPoint(1,0.2)@2001-01-03}

```

■ Constructor para puntos de red temporal de subtipo secuencia

```

tnpointSeq(tnpointInst[], interp={'step', 'linear'}, leftInc bool=true,
rightInc bool=true) → tnpointSeq

```

```

SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.3)@2001-01-01',
'Npoint(1, 0.5)@2001-01-02', 'Npoint(1, 0.5)@2001-01-03']);
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.5)@2001-01-02, NPoint(1,0.5)@2001-01-03}
SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.2)@2001-01-01',
'Npoint(1, 0.4)@2001-01-02', 'Npoint(1, 0.5)@2001-01-03']);
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03]

```

■ Constructor para puntos de red temporal de subtipo conjunto de secuencias

```

tnpointSeqSet(tnpoint[]) → tnpointSeqSet
tnpointSeqSetGaps(tnpointInst[], maxt=NULL, maxdist=NULL, interp='linear') →
tnpointSeqSet

```

```

SELECT tnpointSeqSet(ARRAY[tnpoint '[Npoint(1,0.2)@2001-01-01, Npoint(1,0.4)@2001-01-02,
Npoint(1,0.5)@2001-01-03]', '[Npoint(2,0.6)@2001-01-04, Npoint(2,0.6)@2001-01-05]']]);
/* {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03],
[NPoint(2,0.6)@2001-01-04, NPoint(2,0.6)@2001-01-05]} */
SELECT tnpointSeqSetGaps(ARRAY[tnpoint 'Npoint(1,0.1)@2001-01-01',
'Npoint(1,0.3)@2001-01-03', 'Npoint(1,0.5)@2001-01-05'], '1 day');
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.3)@2001-01-03], [NPoint(1,0.5)@2001-01-05]}

```

11.5. Conversión de tipos

Un valor de punto de red temporal se puede convertir en y desde un punto de geometría temporal. Esto se puede hacer usando un CAST explícito o usando la notación `::`. Se devuelve un valor nulo si alguno de los valores de puntos de geometría que componen no se puede convertir en un valor `tnpoint`.

■ Convertir entre un punto de red temporal y un punto de geometría temporal

```

tnpoint::tgeompoint
tgeompoint::tnpoint

```

```

SELECT astext((tnpoint '[NPoint(1, 0.2)@2001-01-01,
NPoint(1, 0.3)@2001-01-02]')::tgeompoint);
/* [POINT(23.057077727326 28.7666335767956)@2001-01-01,
POINT(48.7117553116406 20.9256801894708)@2001-01-02) */
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
POINT(48.7117553116406 20.9256801894708)@2001-01-02]')::tnpoint
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02]
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
POINT(48.7117553116406 20.9)@2001-01-02]')::tnpoint
-- NULL

```

11.6. Entrada y Salida MF-JSON

Un punto de red temporal se escribe y se lee en Moving-Features JSON. Cada instante representa su punto de red como {"route":<route>,"position":<position>}, el id de ruta como un número JSON y la posición como un float en [0, 1], coincidiendo con la superficie de accesores SQL (route, getPosition).

- Genera un punto de red temporal como MF-JSON y lo lee de vuelta; el ciclo de ida y vuelta es sin pérdida. La salida toma los argumentos de opciones, banderas y dígitos decimales compartidos con los demás tipos temporales.

asMFJSON(tnpoint, integer, integer, integer) → text

tnpointFromMFJSON(text) → tnpoint

```
SELECT tnpointFromMFJSON(asMFJSON(tnpoint 'NPoint(1, 0.5)@2001-01-01'))
= tnpoint 'NPoint(1, 0.5)@2001-01-01';
-- true
```

11.7. Accesores

- Devuelve los valores

getValues(tnpoint) → npoint[]

```
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]}');
-- {"NPoint(1,0.3)","NPoint(1,0.5)"}
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]}');
-- {"NPoint(1,0.3)"}

```

- Devuelve los identificadores de ruta

routes(tnpoint) → bigintset

```
SELECT routes(tnpoint '{NPoint(3, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02}');
-- {1, 3}
```

- Devuelve el valor en una marca de tiempo

valueAtTimestamp(tnpoint, timestamptz) → npoint

```
SELECT valueAtTimestamp(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]',
'2001-01-02');
-- NPoint(1,0.4)
```

- Devuelve la longitud atravesada por el punto de red temporal

length(tnpoint) → float

```
SELECT length(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]');
-- 54.3757408468784
```

- Devuelve la longitud acumulada atravesada por el punto de red temporal

cumulativeLength(tnpoint) → tfloat

```
SELECT cumulativeLength(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02,
NPoint(1, 0.5)@2001-01-03], [NPoint(1, 0.6)@2001-01-04, NPoint(1, 0.7)@2001-01-05]}');
/* {[0@2001-01-01, 54.3757408468784@2001-01-02, 54.3757408468784@2001-01-03],
[54.3757408468784@2001-01-04, 81.5636112703177@2001-01-05]} */
```

- Devuelve la velocidad del punto de red temporal en unidades por segundo

speed({tnpointSeq, tnpointSeqSet}) → tfloatSeqSet

```
SELECT speed(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
NPoint(1, 0.6)@2001-01-03]') * 3600 * 24;
/* Interp=Step;[21.4016800272077@2001-01-01, 14.2677866848051@2001-01-02,
14.2677866848051@2001-01-03] */
```

11.8. Transformaciones

- Transformar un punto de red temporal en otro subtipo

```
tnpointInst(tnpoint) → tnpointInst
tnpointSeq(tnpoint,interp) → tnpointSeq
tnpointSeqSet(tnpoint) → tnpointSeqSet
```

```
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01', 'discrete');
-- {NPoint(1,0.5)@2001-01-01}
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- [NPoint(1,0.5)@2001-01-01]
SELECT tnpointSeqSet(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- {[NPoint(1,0.5)@2001-01-01]}
```

- Transformar un punto de red temporal en otra interpolación

```
setInterp(tnpoint, interp) → tnpoint
```

```
SELECT setInterp(tnpoint 'NPoint(1,0.2)@2001-01-01', 'linear');
-- [NPoint(1,0.2)@2001-01-01]
SELECT setInterp(tnpoint '{[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}',
'discrete');
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02}
```

- Redondear la fracción del punto de red temporal en el número de lugares decimales

```
round(tnpoint,integer=0) → tnpoint
```

```
SELECT round(tnpoint '{[NPoint(1,0.123456789)@2001-01-01, NPoint(1,0.5)@2001-01-02]}', 6);
-- {[NPoint(1,0.123457)@2001-01-01, NPoint(1,0.5)@2001-01-02]}
```

11.9. Modificaciones

- Agregar un instante temporal o una secuencia temporal

```
appendInstant(tnpoint,tnpointInst) → tnpoint
appendSequence(tnpoint,tnpointSeq) → tnpoint
```

```
SELECT asText(appendInstant(tnpoint 'Npoint(1, 0.1)@2001-01-01', 'Npoint(1, 0.2)@2001-01-02'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02]
SELECT asText(appendSequence(tnpoint '[Npoint(1, 0.1)@2001-01-01]', '[Npoint(1, 0.2)@2001-01-02]'));
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}
```

- Fusionar los puntos de red temporales

```
merge(tnpoint,tnpoint) → tnpoint
merge(tnpoint[]) → tnpoint
mergeAgg(tnpoint) → tnpoint
```

```

SELECT asText(merge(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  '[Npoint(1, 0.3)@2001-01-03, Npoint(1, 0.5)@2001-01-05]'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.2)@2001-01-02]' ↔
  ,
  '[Npoint(1, 0.2)@2001-01-02, Npoint(1, 0.3)@2001-01-03]']));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tnpoint 'Npoint(1, 0.1)@2001-01-01' UNION
  SELECT tnpoint 'Npoint(1, 0.2)@2001-01-02' UNION
  SELECT tnpoint 'Npoint(1, 0.3)@2001-01-03' )
SELECT asText(mergeAgg(inst)) FROM temp;
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02, NPoint(1,0.3)@2001-01-03}

```

11.10. Restricciones

- Restringir al (complemento de) un valor o un conjunto de valores

atValue(tnpoint,value) → tnpoint

minusValue(tnpoint,value) → tnpoint

atValues(tnpoint,valueSet) → tnpoint

minusValues(tnpoint,valueSet) → tnpoint

```

SELECT atValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
-- {[NPoint(2,0.5)@2001-01-02]}
SELECT minusValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
/* {[NPoint(2,0.3)@2001-01-01, NPoint(2,0.5)@2001-01-02],
  (NPoint(2,0.5)@2001-01-02, NPoint(2,0.7)@2001-01-03]} */

```

- Restringir al (complemento de) una geometría

atGeometry(tnpoint,geometry) → tnpoint

minusGeometry(tnpoint,geometry) → tnpoint

```

SELECT atGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(NPoint(2,0.342593)@2001-01-01 05:06:40.364673+01,
  NPoint(2,0.7)@2001-01-03 00:00:00+01)} */

```

11.11. Operaciones de distancia

- Devuelve la distancia más cercana

{geo,npoint,tnpoint,stbox} |=| {geo,npoint,tnpoint,stbox} → float

El operador |=| se puede utilizar para realizar búsquedas más cercanas utilizando un índice GiST o SP-GIST (ver Sección 10.2).

```

SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
  geometry 'SRID=5676;Linestring(50 50,55 55)';
-- 31.69220882252415
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|

```

```

npoint 'NPoint(1, 0.5)';
-- 19.49691305292373
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-02]';
-- 5.231180723735304

```

- Devuelve el instante del primer punto de red temporal en el que los dos argumentos están a la distancia más cercana
`nearestApproachInstant({geo,npoint,tnpoint},{geo,npoint,tnpoint}) → tnpoint`

```

SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
-- NPoint(2,0.349928)@2001-01-01 02:59:44.402905+01
SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)');
-- NPoint(2,0.592181)@2001-01-01 17:31:51.080405+01

```

- Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
`shortestLine({geo,npoint,tnpoint},{geo,npoint,tnpoint}) → geometry`
La función devuelve la primera línea que encuentre si hay más de una.

```

SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(50.7960725266492 48.8266286733015,50 50)
SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)'));
-- LINESTRING(77.0902838115125 66.6659083092593,90.8134936900394 46.4385792121146)

```

- Devuelve la distancia temporal

`tgeompoint <-> tnpoint → tfloat`

```

SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
npoint 'NPoint(1, 0.2)';
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
geometry 'Point(50 50)';
-- [25.0496666945044@2001-01-01, 26.4085688426232@2001-01-03]
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
tnpoint '[NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-04]';
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]

```

11.12. Operaciones espaciales

- Devuelve el centroide ponderado en el tiempo

`twCentroid(tnpoint) → geometry(Point)`

```

SELECT ST_AsText(twCentroid(tnpoint '{[NPoint(1, 0.3)@2001-01-01,
NPoint(1, 0.5)@2001-01-02, NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04]}'));
-- POINT(79.9787466444847 46.2385558051041)

```

11.13. Operaciones de cuadro delimitador

- Operadores de rutas, que comparan los identificadores de ruta que visita un punto de red temporal: contenido en, contiene, se superpone e igual

`{bigint,bigintset,npoint,tnpoint} {?@, @?, @@, @=} {bigint,bigintset,npoint,tnpoint}`
→ boolean

```

SELECT bigint '1' ?@ tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @? bigint '1';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @@
  bigintset '{3, 7}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @=
  bigintset '{1, 3}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @=
  bigintset '{1}';
-- false

```

■ Operadores topológicos

{tstzspan,stbox,tnpoint} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tnpoint} → boolean

```

SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' &&
  tstzspan '[2001-01-02,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @>
  stbox(npoint 'NPoint(1, 0.5)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' ~=
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.35)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03]';
-- true

```

■ Operadores de posición

{stbox,tnpoint} {<<, &<, >>, &>} {stbox,tnpoint} → boolean

{stbox,tnpoint} {<<|, &<|, |>>, |&>} {stbox,tnpoint} → boolean

{tstzspan,stbox,tnpoint} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tnpoint} → boolean

```

SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<
  stbox(npoint 'NPoint(1, 0.2)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<|
  stbox(npoint 'NPoint(1, 0.5)');
-- false
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' #&>
  tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.3)@2001-01-05]' #>>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]';
-- true

```

11.14. Relaciones espaciales

Las relaciones topológicas y de distancia descritas en Sección 8.5, como `eIntersects`, `adWithinNotContains`, se definen en el espacio geográfico, mientras que los puntos de la red temporal se definen en el espacio de la red. Cada una de estas relaciones también acepta directamente un argumento `tnpoint`: internamente, cada operando de punto de red se convierte en un punto geométrico temporal mediante la conversión estándar `tnpoint::tgeompoint` (el punto resuelto a lo largo de su ruta) y luego en `tgeometry`, y la relación `tgeometry` correspondiente calcula el resultado, exactamente como si la conversión se hubiera escrito manualmente.

Dado que el objetivo de la delegación, `tgeometry`, solo admite interpolación escalonada, un argumento `tnpoint` de secuencia con varios instantes debe usar interpolación escalonada (`Interp=Step; . . .`); una secuencia con la interpolación lineal predefinida produce un error en la conversión implícita, la misma restricción ya vigente al convertir un `tgeompoint`/`tgeogpoint` lineal en `tgeometry`/`tgeography`. Un único instante no tiene interpolación y no se ve afectado.

■ Relaciones alguna vez y siempre

```
SELECT eContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- false
SELECT eDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- true
SELECT eIntersects(
  tnpoint 'Npoint(1, 0.1)@2001-01-01',
  tnpoint 'Npoint(2, 0.0)@2001-01-01');
-- false
```

■ Relaciones espaciotemporales

```
SELECT tContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Interp=Step;[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- [f@2001-01-01, f@2001-01-03]
SELECT tDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint 'Interp=Step;[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- [t@2001-01-01, t@2001-01-03]
SELECT tDwithin(
  tnpoint 'Interp=Step;[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-03]',
  tnpoint 'Interp=Step;[Npoint(1, 0.5)@2001-01-01, Npoint(1, 0.3)@2001-01-03]', 1);
-- [f@2001-01-01, f@2001-01-03]
```

11.15. Comparaciones

■ Comparaciones tradicionales

`tnpoint {=, <>, <, >, <=, >} tnpoint → boolean`

```
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-02],
  [NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-03]}' =
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- true
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]}' <>
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- false
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.6)@2001-01-03]';
-- true
```

■ Comparaciones alguna vez y siempre

`{npoint,tnpoint} {?=?,%=?} {npoint,tnpoint} → boolean`

```
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-04]' ?= Npoint(1, 0.3);
-- true
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.2)@2001-01-04]' &= Npoint(1, 0.2);
-- true
```

■ Comparaciones temporales

`{npoint,tnpoint} {#=#, #<>} {npoint,tnpoint} → tbool`

```

SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03]' #=
  npoint 'NPoint(1, 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.8)@2001-01-03]' #<>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}

```

11.16. Agregacions

Las tres funciones agregadas para puntos de red temporales se ilustran a continuación.

■ Conteo temporal

`tCount(tnpoint) → {tintSeq,tintSeqSet}`

```

WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04]' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}

```

■ Conteo de ventana

`wCount(tnpoint) → {tintSeq,tintSeqSet}`

```

WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04]' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' )
SELECT wCount(Temp, '1 day')
FROM Temp
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
  1@2001-01-06]} */

```

■ Centroide temporal

`tCentroid(tnpoint) → tgeompoint`

```

WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03]' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' )
SELECT astext(tCentroid(Temp))
FROM Temp
/* {[POINT(72.451531682218 76.5231414472853)@2001-01-01,
  POINT(55.7001249027598 72.9552602410653)@2001-01-03]} */

```

11.17. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de puntos de redes temporales. A continuación, se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```


Los índices GiST y SP-GiST almacenan el cuadro delimitador para los puntos de la red temporal, que es un `stbox` y, por lo tanto, almacena las coordenadas absolutas del espacio subyacente.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en puntos de la red temporal,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en puntos de la red temporal,
- `&&`, `@>`, `<@`, `~=`, `-|-` y `|=|`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

11.18. Agregación

- Devuelve el cuadro delimitador espaciotemporal que cubre cada posición materializada en un conjunto de puntos de red. El agregado es polimórfico con los agregados `extent` de los demás tipos temporales espaciales, por lo que una sola consulta puede calcular un único cuadro a través de un conjunto heterogéneo de columnas `tgeompoint`, `tgeometry` y `tnpoint`. La extensión espacial de cada instante se resuelve a través del registro de `ways`, lo que domina el coste sobre una entrada grande.

`extent(tnpoint) → stbox`

```
SELECT extent(temp) FROM tbl_tnpoint;
```


12.1. Búferes circulares estáticos

Un valor `cbuffer` es un par de la forma `(point, radius)` donde `point` es un punto geométrico y `radius` es un valor `float` mayor o igual que cero que representa un radio alrededor del punto expresado utilizando las unidades del SRID del punto. Ejemplos de entrada de valores de búfer circular son los siguientes:

```
SELECT cbuffer 'Cbuffer(Point(1 1), 0.3)';
SELECT cbuffer 'Cbuffer(Point(1 1), 10.0)';
```

El tipo `cbuffer` corresponde al resultado de la función de PostGIS [ST_MinimumBoundingRadius](#).

Los valores del tipo de búfer circular deben satisfacer varias restricciones para que estén bien definidos. Ejemplos de valores incorrectos del tipo de búfer circular son los siguientes.

```
-- valor de punto incorrecto
SELECT cbuffer 'Cbuffer(Linestring(1 1,2 2), 1.0)';
-- punto 3D incorrecto
SELECT cbuffer 'Cbuffer(Point Z(1 1 1), 1.0)';
-- valor de radio incorrecto
SELECT cbuffer 'Cbuffer(Point(1 1), -2.0)';
```

A continuación damos las funciones y operadores para el tipo de búfer circular.

12.1.1. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
asText({cbuffer,cbuffer[]}) → {text,text[]}
```

```
asEWKT({cbuffer,cbuffer[]}) → {text,text[]}
```

```
SELECT asText(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- Cbuffer(POINT(1 1),0.5)
SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(1 1),0.2)', 'Cbuffer(Point(1 1),0.5)']);
-- {"Cbuffer(POINT(1 1),0.2)","Cbuffer(POINT(1 1),0.5)"}
SELECT asEWKT(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.2)', 'Cbuffer(SRID=5676;Point ↵
(1 1),0.5)']);
-- {"SRID=5676;Cbuffer(POINT(1 1),0.2)","SRID=5676;Cbuffer(POINT(1 1),0.5)"}

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

```
asBinary(cbuffer,endian text="") → bytea
```

```
asEWKB(cbuffer,endian text="") → bytea
```

```
asHexWKB(cbuffer,endian text="") → text
```

```
asHexEWKB(cbuffer,endian text="") → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Un búfer circular toma su SRID de su punto subyacente: `asEWKB` y `asHexEWKB` lo incluyen, mientras que las formas simples `asBinary` y `asHexWKB` lo omiten.

```
SELECT asBinary(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- \x0100000000000000f03f000000000000f03f000000000000e03f
SELECT asEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- \x01402c160000000000000000f03f000000000000f03f000000000000e03f
```

```
SELECT asHexWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- 0100000000000000F03F000000000000F03F000000000000E03F
SELECT asHexEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- 01402C160000000000000000F03F000000000000F03F000000000000E03F
```

- Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

`cbufferFromText(text) → cbuffer`

`cbufferFromEWKT(text) → cbuffer`

```
SELECT asText(cbufferFromText(text 'Cbuffer(Point(1 1),0.5)'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKT(text 'SRID=5676;Cbuffer(Point(1 1),0.5)'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

- Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

`cbufferFromBinary(bytea) → cbuffer`

`cbufferFromEWKB(bytea) → cbuffer`

`cbufferFromHexEWKB(text) → cbuffer`

```
SELECT asEWKT(cbufferFromBinary(
  '\x0100000000000000f03f000000000000f03f000000000000e03f'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKB(
  '\x01402c160000000000000000f03f000000000000f03f000000000000e03f'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromHexEWKB(
  '01402C160000000000000000F03F000000000000F03F000000000000E03F'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

12.1.2. Constructores

- Constructor para búferes circulares

`cbuffer(geompoint,float) → cbuffer`

```
SELECT asText(cbuffer(ST_Point(1,1), 0.3));
-- Cbuffer(POINT(1 1),0.3)
```

12.1.3. Conversión de tipos

Los valores del tipo `cbuffer` se pueden convertir al tipo `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación. De manera similar, una `geometry` se puede convertir a un valor `cbuffer` utilizando la función `ST_MinimumBoundingRadius` de PostGIS.

- Convertir entre un búfer circular y una geometría

`geometry::cbuffer`

`cbuffer::geometry`

```
SELECT ST_AsText(cbuffer(ST_Point(1,1), 1)::geometry);
-- CURVEPOLYGON(CIRCULARSTRING(0 1,2 1,0 1))
SELECT asText(geometry 'SRID=5676;CIRCULARSTRING(0 1,2 1,0 1)::cbuffer);
-- Cbuffer(POINT(1 1),1)
SELECT asText(geometry 'SRID=5676;LINESTRING(0 0,2 2)::cbuffer);
-- Cbuffer(POINT(1 1),1.414213562373095)
```

- Convertir un búfer circular y, opcionalmente, una marca de tiempo o un período, a un cuadro espaciotemporal

`stbox(cbuffer) → stbox`

`stbox(cbuffer, {timestampz, tstzspan}) → stbox`

```
SELECT stbox(cbuffer 'SRID=5676;Cbuffer(Point(1 1),0.3)');
-- SRID=5676;STBOX X((0.7,0.7),(1.3,1.3))
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', timestampz '2001-01-01');
-- STBOX XT(((0.7,0.7),(1.3,1.3)), [2001-01-01, 2001-01-01])
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((0.7,0.7),(1.3,1.3)), [2001-01-01, 2001-01-02])
```

12.1.4. Accesores

- Devuelve el punto

`point(cbuffer) → geompoint`

```
SELECT ST_AsText(point(cbuffer 'Cbuffer(Point(1 1), 0.3)'));
-- Point(1 1)
```

- Devuelve el radio

`radius(cbuffer) → float`

```
SELECT radius(cbuffer 'Cbuffer(Point(1 1), 0.3)');
-- 0.3
```

12.1.5. Transformaciones

- Redondear el punto y el radio del búfer circular en el número de lugares decimales

`round(cbuffer, integer=0) → cbuffer`

```
SELECT asText(round(cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Cbuffer(POINT(1.123457 1.123457),0.123457)
```

12.1.6. Operaciones espaciales

- Devuelve o establece el identificador de referencia espacial

`SRID(cbuffer) → integer`

`setSRID(cbuffer) → cbuffer`

```
SELECT SRID(cbuffer 'Cbuffer(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(cbuffer 'Cbuffer(Point(0 0),1)', 4326));
-- SRID=4326;Cbuffer(POINT(0 0),1)
```

- Transformar a una referencia espacial diferente

`transform(cbuffer, integer) → cbuffer`

`transformPipeline(cbuffer, pipeline text, to_srid integer, is_forward bool=true) → cbuffer`

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el búfer circular de entrada tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena:

```
urn:ogc:def:coordinateOperation:AUTHORITY::CODE
```

El SRID del búfer circular de entrada se ignora y el SRID del búfer circular de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(cbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)', 3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)
```

```
WITH test(cbuffer, pipeline) AS (
  SELECT cbuffer 'Cbuffer(SRID=4326;Point(4.3525 50.846667),1)',
         text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(cbuffer, pipeline, 4326), pipeline,
4326, false), 6)
FROM test;
-- SRID=4326;Cbuffer(POINT(4.3525 50.846667),1)
```

12.1.7. Comparaciones

Los operadores de comparación (`=`, `<` y así sucesivamente) están disponibles para búferes circulares. Comparan primero los puntos y luego el radio de los argumentos. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en búferes circulares.

■ Comparaciones tradicionales

```
cbuffer {=, <>, <, >, <=, >=} cbuffer
```

```
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' = cbuffer 'Cbuffer(Point(3 3), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' <> cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' < cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.6)' > cbuffer 'Cbuffer(Point(2 2), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)' <= cbuffer 'Cbuffer(Point(2 2), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.6)' >= cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
```

12.2. Búferes circulares temporales

El tipo de búfer circular temporal `tcbuffer` permite representar el movimiento de objetos junto con un radio circular alrededor de ellos. Como todos los tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de `tcbuffer` en estos subtipos.

```
SELECT tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01';
SELECT tcbuffer '{Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02,
  Cbuffer(Point(1 1), 0.5)@2001-01-03}';
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01, Cbuffer(Point(1 1), 0.4)@2001-01-02,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]';
SELECT tcbuffer '{[Cbuffer(Point(1 1), 1)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-02],
  [Cbuffer(Point(2 2), 2)@2001-01-04, Cbuffer(Point(2 2), 3)@2001-01-05]}';
```

El tipo de búfer circular temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo y/o el identificador de referencia espacial (SRID). Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet`. Los dos argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo y/o SRID.

```
SELECT asEWKT(tcbuffer(Sequence,5676) 'SRID=5676;[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-03]
SELECT tcbuffer(Sequence) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tcbuffer(5676) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal circular buffer SRID (0) does not match column SRID (5676)
```

Los valores de búfer circular temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT asText(tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(2 2), 0.4)@2001-01-02, Cbuffer(Point(3 3), 0.6)@2001-01-03]');
-- [Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(3 3),0.6)@2001-01-03]
SELECT asText(tcbuffer '{[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(2 2), 0.3)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03),
    [Cbuffer(Point(2 2), 0.5)@2001-01-03, Cbuffer(Point(2 2), 0.7)@2001-01-04)]}');
/* {[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.3)@2001-01-02,
    Cbuffer(Point(2 2),0.7)@2001-01-04)} */
```

12.3. Validez de los búferes circulares temporales

Los valores de búfer circular temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- No se permiten valores nulos
SELECT tcbuffer 'NULL@2001-01-01 08:05:00';
SELECT tcbuffer 'Point(0 0)@NULL';
-- El tipo base no es un búfer circular
SELECT tcbuffer 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para el búfer circular temporal. La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos precedentes se pueden aplicar para los tipos de búfer circular temporal. Por lo tanto, en las firmas de las funciones, la notación base representa un `cbuffer` y las notaciones `ttype`, `tpoint` y `tgeompoint` también representan un `tcbuffer`. Además, las funciones que tienen un argumento de tipo `geometry` aceptan además un argumento de tipo `cbuffer`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para búferes circulares temporales.

12.4. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
asText({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
```

```
asEWKT({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
```

```
SELECT asText(tcbuffer 'SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01,
    Cbuffer(Point(1 1),2)@2001-01-02]');
-- [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02]
```

```
SELECT asText (ARRAY[cbuffer 'Cbuffer(Point(0 0),1)', 'Cbuffer(Point(1 1),2)']);
-- {"Cbuffer(POINT(0 0),1)","Cbuffer(POINT(1 1),2)"}
SELECT asEWKT(tcbuffer 'SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)');
-- SRID=4326;[Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)
SELECT asEWKT (ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(0 0),1)',
  'Cbuffer(SRID=5676;Point(1 1),2)']);
-- {"Cbuffer(SRID=5676;POINT(0 0),1)","Cbuffer(SRID=5676;POINT(1 1),2)"}

```

- Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)

asMFJSON(tcbuffer) → text

Un búfer circular siempre es planar 2D. Cada valor es un objeto con un miembro `point`, un arreglo de coordenadas [x, y] y un miembro numérico `radius`, que reflejan los accesores `point` y `radius`.

```
SELECT asMFJSON(tcbuffer 'Cbuffer(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingCircularBuffer","bbox":[[0.5,1.5],[1.5,2.5]],
  "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-01T00:00:00+01",
  "lower_inc":true,"upper_inc":true},
  "values":[{"point":[1,2],"radius":0.5}],
  "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tcbuffer 'SRID=4326;[Cbuffer(Point(1 1),0.2)@2001-01-01,
  Cbuffer(Point(2 2),0.4)@2001-01-02)', 3);
/* {"type":"MovingCircularBuffer","crs":{"type":"Name",
  "properties":{"name":"EPSG:4326"}}, "bbox":[[0.8,0.8],[2.4,2.4]],
  "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-02T00:00:00+01",
  "lower_inc":true,"upper_inc":false},
  "values":[{"point":[1,1],"radius":0.2}, {"point":[2,2],"radius":0.4}],
  "datetimes":["2001-01-01T00:00:00+01","2001-01-02T00:00:00+01"],
  "lower_inc":true,"upper_inc":false,"interpolation":"Linear"} */

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

asBinary(tcbuffer, endian text="") → bytea

asEWKB(tcbuffer, endian text="") → bytea

asHexWKB(tcbuffer, endian text="") → text

asHexEWKB(tcbuffer, endian text="") → text

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asEWKB(tcbuffer 'SRID=7844;Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0041a41e00000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asHexWKB(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
-- 013B0001000000000000F03F000000000000400000000000F03F009C57D3C11C0000
SELECT asHexEWKB(tcbuffer 'SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01');
-- 013B0041E40E00000000000000F03F000000000000400000000000F03F009C57D3C11C0000

```

- Entrar a partir de la representación de texto conocido (Well-Known Text o WKT) o de la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

tcbufferFromText(text) → tcbuffer

tcbufferFromEWKT(text) → tcbuffer


```
SELECT asEWKT(tcbufferFromText(text '[Cbuffer(Point(1 2),1)@2001-01-01,
  Cbuffer(Point(3 4),2)@2001-01-02]'));
-- [Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tcbufferFromEWKT(text 'SRID=3812;[Cbuffer(Point(1 2),1)@2001-01-01,
  Cbuffer(Point(3 4),2)@2001-01-02]'));
-- SRID=3812;[Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]
```

- Entrar a partir de la representación JSON de características móviles (Moving Features JSON o MF-JSON)

tcbufferFromMFJSON(text) → tcbuffer

```
SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer
  'SRID=3812;Cbuffer(Point(1 2),0.5)@2001-01-01', 3)));
-- SRID=3812;Cbuffer(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer
  'SRID=5676;[Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]', 3)));
-- SRID=5676;[Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
```

- Entrar a partir de la representación binaria conocida (WKB), de la representación extendida binaria conocida (EWKB), o de la representación hexadecimal extendida binaria conocida (HexEWKB)

tcbufferFromBinary(bytea) → tcbuffer

tcbufferFromEWKB(bytea) → tcbuffer

tcbufferFromHexEWKB(text) → tcbuffer

```
SELECT asEWKT(tcbufferFromBinary(
  '\x013b0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000'));
-- Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromEWKB(
  '\x013b0041a41e00000000000000f03f000000000000400000000000f03f009c57d3c11c0000'));
-- SRID=7844;Cbuffer(Point(1 1),2)@2001-01-01
SELECT asEWKT(tcbufferFromHexEWKB(
  '013B0041E40E00000000000000f03f000000000000400000000000f03f009c57d3c11c0000'));
-- SRID=3812;Cbuffer(POINT(1 2),1)@2001-01-01
```

12.5. Constructores

- Constructor para búferes circulares temporales con valor constante

tcbuffer(cbuffer,timestampz) → tcbufferInst

tcbuffer(cbuffer,tstzset) → tcbufferDiscSeq

tcbuffer(cbuffer,tstzspan,interp='linear') → tcbufferContSeq

tcbuffer(cbuffer,tstzspanset,interp='linear') → tcbufferSeqSet

```
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', timestampz '2001-01-01'));
-- Cbuffer(Point(1 1),0.5)@2001-01-01
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.3)',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}'));
/* {Cbuffer(Point(1 1),0.3)@2001-01-01, Cbuffer(Point(1 1),0.3)@2001-01-03,
  Cbuffer(Point(1 1),0.3)@2001-01-05} */
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)',
  tstzspan '[2001-01-01, 2001-01-02]'));
-- [Cbuffer(Point(1 1),0.5)@2001-01-01, Cbuffer(Point(1 1),0.5)@2001-01-02]
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.2)',
  tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(1 1),0.2)@2001-01-03]}
```

- Constructor para búferes circulares temporales de subtipo secuencia

```
tcbufferSeq(tcbufferInst[], interp='linear', leftInc bool=true,
rightInc bool=true) → tcbufferSeq
```

```
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03']));
/* [Cbuffer(Point(1 1),0.3)@2001-01-01, Cbuffer(Point(2 2),0.5)@2001-01-02,
Cbuffer(Point(1 1),0.5)@2001-01-03] */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03',
'discrete']));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
Cbuffer(POINT(1 1),0.5)@2001-01-03} */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.2)@2001-01-01',
'Cbuffer(Point(1 1), 0.4)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03', 'step']));
/* Interp=Step;[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(1 1),0.4)@2001-01-02,
Cbuffer(Point(1 1),0.5)@2001-01-03] */
```

- Constructor para búferes circulares temporales de subtipo conjunto de secuencias

```
tcbufferSeqSet(tcbuffer[]) → tcbufferSeqSet
```

```
tcbufferSeqSetGaps(tcbufferInst[], maxt=NULL, maxdist=NULL, interp='linear') →
tcbufferSeqSet
```

```
SELECT asText(tcbufferSeqSet(ARRAY[tcbuffer
'[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02]',
'[Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.8)@2001-01-04]']));
/* {[Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02],
[Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.8)@2001-01-04]} */
SELECT asText(tcbufferSeqSetGaps(ARRAY[tcbuffer 'Cbuffer(Point(1 1),0.1)@2001-01-01',
'Cbuffer(Point(1 1),0.3)@2001-01-03', 'Cbuffer(Point(1 1),0.5)@2001-01-05', '1 day']));
/* {[Cbuffer(Point(1 1),0.1)@2001-01-01], [Cbuffer(Point(1 1),0.3)@2001-01-03],
[Cbuffer(Point(1 1),0.5)@2001-01-05]} */
```

- Construir un búfer circular temporal a partir de un punto de geometría temporal y un flotante temporal

```
tcbuffer(tgeompoint, tfloat) → tcbuffer
```

El marco de tiempo del resultado es la intersección de los marcos de tiempo del punto temporal y el flotante temporal. Si no se intersecan, se devuelve un valor nulo

```
SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
tfloat '[1@2001-01-01, 2@2001-01-02]'));
-- [Cbuffer(Point(1 1),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02]
SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
tfloat '[2@2001-01-03, 3@2001-01-04]'));
-- NULL
```

12.6. Conversión de tipos

Un valor de búfer circular temporal se puede convertir en y desde un punto de geometría temporal. Esto se puede hacer usando un CAST explícito o usando la notación `::`. Se devuelve un valor nulo si alguno de los valores de puntos de geometría que componen no se puede convertir en un valor `cbuffer`.

- Convertir un búfer circular temporal a un punto de geometría temporal

```
tcbuffer::tgeompoint
```

```
SELECT asText((tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-02]')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
```

- Convertir un búfer circular temporal a un flotante temporal

tcbuffer::tfloat

```
SELECT (tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-02]')::tfloat;
-- [0.2@2001-01-01, 0.3@2001-01-02)
```

12.7. Accesores

- Devuelve los valores

getValues(tcbuffer) → cbufferset

```
SELECT asText(getValues(tcbuffer '{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-02]}'));
-- {"Cbuffer(Point(1 1),0.3)","Cbuffer(Point(1 1),0.5)"}
SELECT asEWKT(getValues(tcbuffer 'SRID=5676;{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-02]}'));
-- SRID=5676;{"Cbuffer(Point(1 1),0.3)"}
```

- Devuelve los puntos o los radios

points(tcbuffer) → geomset

radius(tcbuffer) → floatset

```
SELECT asEWKT(points(tcbuffer 'SRID=5676;{Cbuffer(Point(3 3), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-02]}'));
-- SRID=5676;{Point(1 1), Point(3 3)}
SELECT radius(tcbuffer 'SRID=5676;{Cbuffer(Point(3 3), 0.3)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-02]}');
-- {0.3, 0.5}
```

- Devuelve el valor en una marca de tiempo

valueAtTimestamp(tcbuffer,timestamptz) → cbuffer

```
SELECT asText(valueAtTimestamp(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
    Cbuffer(Point(3 3), 0.5)@2001-01-03]', '2001-01-02'));
-- Cbuffer(Point(2 2),0.4)
```

12.8. Transformaciones

- Transformar un búfer circular temporal en otro subtipo

tcbufferInst(tcbuffer) → tcbufferInst

tcbufferSeq(tcbuffer) → tcbufferSeq

tcbufferSeqSet(tcbuffer) → tcbufferSeqSet

```
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Cbuffer(Point(1 1),0.5)@2001-01-01}
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- [Cbuffer(Point(1 1),0.5)@2001-01-01]
SELECT asText(tcbufferSeqSet(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- {[Cbuffer(Point(1 1),0.5)@2001-01-01]}
```

- Transformar un búfer circular temporal en otra interpolación

```
setInterp(tcbuffer, interp) → tcbuffer
```

```
SELECT asText(setInterp(tcbuffer 'Cbuffer(Point(1 1),0.2)@2001-01-01','linear'));
-- [Cbuffer(Point(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tcbuffer '{[Cbuffer(Point(1 1),0.1)@2001-01-01],
[Cbuffer(Point(1 1),0.2)@2001-01-02]}', 'discrete'));
-- {Cbuffer(Point(1 1),0.1)@2001-01-01, Cbuffer(Point(1 1),0.2)@2001-01-02}
```

- Redondear los puntos y los radios del búfer circular temporal en el número de lugares decimales

```
round(tcbuffer,integer) → tcbuffer
```

```
SELECT asText(round(tcbuffer '{[Cbuffer(Point(1 1.123456789),0.123456789)@2001-01-01,
Cbuffer(Point(1 1),0.5)@2001-01-02]}', 3));
-- {[Cbuffer(Point(1 1.123),0.123)@2001-01-01, Cbuffer(Point(1 1),0.5)@2001-01-02]}
```

12.9. Modificaciones

- Agregar un instante temporal o una secuencia temporal

```
appendInstant(tcbuffer,tcbufferInst) → tcbuffer
```

```
appendSequence(tcbuffer,tcbufferSeq) → tcbuffer
```

```
SELECT asText(appendInstant(tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01', 'Cbuffer(Point(2 2), 0.2)@2001-01-02'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02]
SELECT asText(appendSequence(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01]', '[Cbuffer(Point(2 2), 0.2)@2001-01-02]'));
-- {[Cbuffer(POINT(1 1),0.1)@2001-01-01], [Cbuffer(POINT(2 2),0.2)@2001-01-02]}
```

- Fusionar los búferes circulares temporales

```
merge(tcbuffer,tcbuffer) → tcbuffer
```

```
merge(tcbuffer[]) → tcbuffer
```

```
mergeAgg(tcbuffer) → tcbuffer
```

```
SELECT asText(merge(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03]',
'[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.2)@2001-01-02]',
'[Cbuffer(Point(1 1), 0.2)@2001-01-02, Cbuffer(Point(1 1), 0.3)@2001-01-03]']));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01' UNION
  SELECT tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02' )
SELECT asText(mergeAgg(inst)) FROM temp;
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02}
```

12.10. Operaciones espaciales

- Devuelve o establece el identificador de referencia espacial

```
SRID(tcbuffer) → integer
```

```
setSRID(tcbuffer) → tcbuffer
```

```
SELECT SRID(tcbuffer 'SRID=5676;[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02]', 5676));
-- SRID=5676;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
```

- Transformar a una referencia espacial diferente

`transform(tcbuffer, integer) → tcbuffer`

`transformPipeline(tcbuffer, pipeline text, to_srid integer, is_forward bool=true) → tcbuffer`

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el búfer circular temporal de entrada tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena: `urn:ogc:def:coordinateOperation:AUTHORITY::CODE`. El SRID del búfer circular temporal de entrada se ignora y el SRID del búfer circular temporal de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(tcbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)@2001-01-01',
  3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tcbuffer, pipeline) AS (
  SELECT tcbuffer 'SRID=4326;{Cbuffer(Point(4.3525 50.846667),1)@2001-01-01,
    Cbuffer(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
  SELECT asEWKT(transformPipeline(transformPipeline(tcbuffer, pipeline, 4326), pipeline,
    4326, false), 6)
FROM test;
/* SRID=4326;{Cbuffer(POINT(4.3525 50.846667),1)@2001-01-01,
  Cbuffer(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

- Devuelve el área atravesada por el búfer circular temporal

`traversedArea(tcbuffer, unary_union=false) → geometry`

```
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]'));
-- CURVEPOLYGON(CIRCULARSTRING(0.7 1,1.3 1,0.7 1))
```

- Devuelve el punto geométrico temporal dado por los centros de un búfer circular temporal

`centroid(tcbuffer) → tgeompoint`

```
SELECT asText(centroid(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03]
```

- Devuelve la envolvente convexa del área atravesada por el búfer circular temporal

`convexHull(tcbuffer) → geometry`

La envolvente convexa se calcula sobre el área atravesada linealizada: los arcos circulares que delimitan la región barrida se convierten primero en segmentos rectos y la envolvente se calcula luego con GEOS. Por lo tanto, el resultado es un `POLYGON` lineal que aproxima el contorno circular, no un `CURVEDPOLYGON` curvo. Utilice `traversedArea` para obtener la región barrida curva exacta.

```

SELECT ST_GeometryType(convexHull(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(4 0), 1)@2001-01-02]'));
-- ST_Polygon
SELECT ST_NPoints(convexHull(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(4 0), 1)@2001-01-02]'));
-- 131
SELECT round(ST_Area(convexHull(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(4 0), 1)@2001-01-02]'))::numeric, 3);
-- 11.140
-- The exact swept region has area pi + 8 = 11.1416; the stroked hull is
-- slightly smaller because the arcs are replaced by chords.

```

12.11. Operaciones de distancia

- Devuelve la distancia más cercana

{geo,cbuffer,tcbuffer,stbox} |=| {geo,cbuffer,tcbuffer,stbox} → float

El operador |=| se puede utilizar para realizar búsquedas del vecino más cercano utilizando un índice GiST o SP-GiST (ver Sección 10.2).

```

SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| geometry 'Linestring(50 50,55 55)';
-- 67.58225099390856
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- 0.6142135623730951

```

- TODO Devuelve el instante del primer búfer circular temporal en el que los dos argumentos están a la distancia más cercana
nearestApproachInstant({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → tcbuffer

```

SELECT nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
-- Cbuffer(Point(2 2),0.349928)@2001-01-01 02:59:44.402905+01
SELECT nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- Cbuffer(Point(2 2),0.592181)@2001-01-01 17:31:51.080405+01

```

- TODO Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
shortestLine({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → geometry

La función devuelve la primera línea que encuentre si hay más de una

```

SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(50 50,2.212132034355964 2.212132034355964)
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- LINESTRING(1.353553390593274 1.353553390593274,1.787867965644036 1.787867965644036)

```

- TODO Devuelve la distancia temporal

{geo,cbuffer,tcbuffer} <-> {geo,cbuffer,tcbuffer} → tfloat

```

SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)';

```

```
-- [25.0496666945044@2001-01-01, 26.4085688426232@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <->
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-04]'
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
```

- Devuelve la distancia espacial mínima, ignorando el tiempo

`minDistance(tcbuffer, {geo, cbuffer}) → float`

`minDistance(tcbuffer, tcbuffer) → float (agregado)`

El resultado es la distancia espacial entre las áreas barridas por los dos argumentos durante toda su vida útil, igual a `ST_Distance(t.traversedArea(...))`. La forma de dos argumentos de búfer circular temporal es un agregado, de modo que una combinación cruzada agrupada por una clave produce el mínimo sobre cada par del grupo.

```
SELECT minDistance(tcbuffer '[Cbuffer(Point(0 0), 0.5)@2001-01-01,
  Cbuffer(Point(10 0), 0.5)@2001-01-02]', geometry 'Point(5 5)');
-- 4.5
SELECT minDistance(tcbuffer '[Cbuffer(Point(0 0), 0.5)@2001-01-01,
  Cbuffer(Point(10 0), 0.5)@2001-01-02]', cbuffer 'Cbuffer(Point(5 5), 1)');
-- 3.5
SELECT g, minDistance(t1.Noise, t2.Noise)
FROM Vessel v1, VesselTrip t1, Vessel v2, VesselTrip t2
WHERE v1.MMSI = t1.MMSI AND v2.MMSI = t2.MMSI
GROUP BY t1.ShipTypeLabel, t2.ShipTypeLabel;
```

12.12. Restricciones

- Restringir al (complemento de) un valor o un conjunto de valores

`atValue(tcbuffer, value) → tcbuffer`

`minusValue(tcbuffer, value) → tcbuffer`

`atValues(tcbuffer, valueset) → tcbuffer`

`minusValues(tcbuffer, valueset) → tcbuffer`

```
SELECT asText(atValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
-- {[Cbuffer(Point(2 2), 0.5)@2001-01-02]}
SELECT asText(minusValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
/* {[Cbuffer(Point(2 2), 0.3)@2001-01-01, Cbuffer(Point(2 2), 0.5)@2001-01-02),
  (Cbuffer(Point(2 2), 0.5)@2001-01-02, Cbuffer(Point(2 2), 0.7)@2001-01-03]} */
```

- TODO Restringir al (complemento de) una geometría

`atGeometry(tcbuffer, geometry) → tcbuffer`

`minusGeometry(tcbuffer, geometry) → tcbuffer`

```
SELECT asText(atGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0, 0 2, 2 2, 2 0, 0 0))'));
-- {[Cbuffer(POINT(2 2), 0.3)@2001-01-01, Cbuffer(POINT(2 2), 0.7)@2001-01-03]}
SELECT asText(minusGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0, 0 2, 2 2, 2 0, 0 0))'));
/* {(Cbuffer(Point(2 2), 0.342593)@2001-01-01 05:06:40.364673+01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03 00:00:00+01)} */
```

- Restringir un búfer circular temporal a (el complemento de) una caja espaciotemporal

`atStbox(tcbuffer,stbox,borderInc bool=true) → tcbuffer`

`minusStbox(tcbuffer,stbox,borderInc bool=true) → tcbuffer`

```
SELECT asText(atStbox(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2000-01-01,
  Cbuffer(Point(3 3), 0.5)@2000-01-03]', stbox 'STBOX X((0,0), (2,2))'));
-- {[Cbuffer(POINT(1 1),0.5)@2000-01-01, Cbuffer(POINT(2 2),0.5)@2000-01-02]}
```

12.13. Comparaciones

- Comparaciones tradicionales

`tcbuffer {=, <>, <, >, <=, >} tcbuffer → boolean`

```
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02),
  [Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]]' =
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]]' <>
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.6)@2001-01-03]';
-- false
```

- Comparaciones alguna vez y siempre

`{tcbuffer,cbuffer} {?=?,%=?} {tcbuffer,cbuffer} → boolean`

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' ?= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' ?=
  tcbuffer '[Cbuffer(Point(1 1), 0.4)@2001-01-01, Cbuffer(Point(1 1), 0.2)@2001-01-03]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-04]' %= cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- true
```

- Comparaciones temporales

`tcbuffer {#=?, #<>} tcbuffer → tbool`

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03]' #= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.8)@2001-01-03]' #<>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.7)@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```


12.14. Operaciones de cuadro delimitador

■ Operadores topológicos

{tstzspan,stbox,tcbuffer} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tcbuffer} → boolean

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' && cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' @> stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)::geometry <@
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' ~= tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.35)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
```

■ Operadores de posición

{stbox,tcbuffer} {<<, &<, >>, &>} {stbox,tcbuffer} → boolean

{stbox,tcbuffer} {<<|, &<|, |>>, |&>} {stbox,tcbuffer} → boolean

{tstzspan,stbox,tcbuffer} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tcbuffer} → boolean

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' << cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<| stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' &> cbuffer 'Cbuffer(Point(1 1), 0.3)::geometry';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' >>#
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]';
-- true
```

12.15. Relaciones espaciales

■ TODO Relaciones alguna vez y siempre

eContains(geometry,tcbuffer) → boolean

aContains(geometry,tcbuffer) → boolean

eDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

aDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

eDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean

aDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean

eIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

aIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

eTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

aTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

```

SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- false
SELECT eDisjoint(cbuffer 'Cbuffer(Point(2 2), 0.0)',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- true
SELECT eIntersects(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]',
  tcbuffer '[Cbuffer(Point(2 2), 0.0)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-03)'];
-- false

```

■ TODO Relaciones espaciotemporales

tContains(geometry,tcbuffer) → boolean

tDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

tDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean

tIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

tTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean

```

SELECT tDisjoint(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)'];
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tDwithin(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]', tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]', 1);
/* {[t@2001-01-01 00:00:00+01, t@2001-01-01 22:35:55.379053+01],
  (f@2001-01-01 22:35:55.379053+01, t@2001-01-02 01:24:04.620946+01,
  t@2001-01-03 00:00:00+01)} */

```

12.16. Agregaciones

Las tres funciones agregadas para búferes circulares temporales se ilustran a continuación.

■ Conteo temporal

tCount(tcbuffer) → {tintSeq,tintSeqSet}

```

WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}

```

■ Conteo de ventana

wCount(tcbuffer) → {tintSeq,tintSeqSet}

```

WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,

```

```

    Cbuffer(Point(1 1), 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day')
FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
    1@2001-01-06]} */

```

- Agregar los centros de un conjunto de búferes circulares temporales en su centroide temporal, convirtiendo a tgeompoint
tCentroid(tcbuffer::tgeompoint) → tgeompoint

La agregación pertenece al tipo tgeompoint: la trayectoria del centro de un búfer circular es un punto temporal, de modo que la conversión alcanza la agregación en lugar de que el tipo búfer lleve su propia copia.

```

WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tcbuffer '[Cbuffer(Point(3 3), 0.2)@2001-01-01,
    Cbuffer(Point(3 3), 0.4)@2001-01-03]' )
SELECT asText(tCentroid(temp::tgeompoint)) FROM Temp;
-- {[POINT(2 2)@2001-01-01, POINT(2 2)@2001-01-03]}

```

12.17. Simplificación

- Devuelve un búfer circular temporal simplificado asegurándose de que los centros consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo

minDistSimplify(tcbuffer, mindist float) → tcbuffer

minTimeDeltaSimplify(tcbuffer, mint interval) → tcbuffer

La simplificación opera sobre la trayectoria de los centros del búfer circular temporal y conserva el radio en cada instante que sobrevive. La simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del valor dado.

```

SELECT asText(minDistSimplify(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(1 0), 1)@2001-01-02,
    Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-03]
-- The result keeps the interpolation of the value it simplifies.
SELECT interp(minDistSimplify(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(1 0), 1)@2001-01-02,
    Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- Linear
SELECT asText(minTimeDeltaSimplify(tcbuffer
  '[Cbuffer(Point(0 0), 1)@2001-01-01, Cbuffer(Point(1 0), 1)@2001-01-02,
    Cbuffer(Point(4 0), 1)@2001-01-04]', interval '2 days'));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-04]

```

- Devuelve un búfer circular temporal simplificado usando el [algoritmo de Douglas-Peucker](#)

maxDistSimplify(tcbuffer, maxdist float, syncdist=true) → tcbuffer

douglasPeuckerSimplify(tcbuffer, maxdist float, syncdist=true) → tcbuffer

La diferencia entre las dos funciones es que maxDistSimplify utiliza una versión de un solo paso del algoritmo mientras que douglasPeuckerSimplify utiliza el algoritmo recursivo estándar. Al igual que las funciones anteriores, la simplificación opera sobre la trayectoria de los centros y conserva el radio en cada instante que sobrevive.

```

SELECT asText(douglasPeuckerSimplify(tcbuffer '[Cbuffer(Point(1 1),0.5)@2000-01-01,
  Cbuffer(Point(2 2),0.5)@2000-01-02, Cbuffer(Point(3 1),0.5)@2000-01-03,
  Cbuffer(Point(4 4),0.5)@2000-01-04]', 2.0));
-- {Cbuffer(POINT(1 1),0.5)@2000-01-01, Cbuffer(POINT(4 4),0.5)@2000-01-04}

```

12.18. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de búferes circulares temporales. A continuación, se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist (Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para los búferes circulares temporales, que es un `stbox`, como todos los tipos espaciotemporales.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en búferes circulares temporales,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en búferes circulares temporales,
- `&&`, `@>`, `<@`, `~=`, `-|-` y `|=|`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

Capítulo 13

Poses temporales

El tipo `pose` se utiliza para representar la *ubicación* y la *orientación* de objetos geométricos dentro de sistemas de coordenadas anclados a la superficie de la Tierra o dentro de otros sistemas de coordenadas astronómicas. La ubicación se representa mediante un punto 2D o 3D. Para las poses 2D, la orientación se define mediante un ángulo de rotación en $(-\pi, \pi]$ expresado en radianes. Para las poses 3D, la orientación se define mediante cuatro valores flotantes W, X, Y, Z , que representan un **cuaternión** unitario $Q = \langle W, X, Y, Z \rangle$ donde $\|Q\| = \sqrt{W^2 + X^2 + Y^2 + Z^2} = 1$.

El **grupo de trabajo de estándares GeoPose** (SWG), que trabaja bajo los auspicios del **Open Geospatial Consortium**, ha definido un estándar para intercambiar información de poses entre distintos usuarios, dispositivos y plataformas. Se puede encontrar más información sobre el estándar en el **repositorio de GitHub** del SWG GeoPose.

El tipo `pose` sirve como tipo base para definir el tipo de pose temporal `tpose`. El tipo `tpose` tiene una funcionalidad similar al tipo de punto temporal `tgeompoint`. Por lo tanto, la mayoría de las funciones y operadores descritos anteriormente para el tipo `tgeompoint` también son aplicables para el tipo `tpose`. Además, hay funciones específicas definidas para el tipo `tpose`. En este capítulo cubrimos estas funciones.

El tipo `tpose` se utiliza para definir el tipo `trgeometry` (es decir, geometría rígida temporal) definido en el siguiente capítulo. La implementación de estos tipos en MobilityDB se ha estudiado en la siguiente tesis doctoral.

- Maxime Schoemans, *Managing Rigid Temporal Geometries in Moving Object Databases*, tesis doctoral, Université libre de Bruxelles, 2025.

13.1. Poses estáticas

Una pose 2D es un par de la forma `(point2D, radius)` donde `point2D` es un punto geométrico 2D y `radius` es un valor `float` que representa un ángulo de rotación en $(-\pi, \pi]$ expresado en radianes. Una pose 3D es una tupla de la forma `(point3D, W, X, Y, Z)` donde `point3D` es un punto geométrico 3D, y W, X, Y, Z son cuatro `floats` que representan un *cuaternión unitario*. Ejemplos de entrada de valores de pose son los siguientes:

```
SELECT pose 'Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

Se puede especificar un SRID para una pose ya sea al comienzo del literal de pose o antes del literal de punto, como se muestra a continuación.

```
SELECT pose 'SRID=3812;Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(SRID=5676;Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

Los valores del tipo `pose` deben satisfacer varias restricciones para que estén bien definidos. Ejemplos de valores incorrectos del tipo `pose` son los siguientes.

```
-- Punto vacío
select pose 'Pose(Point empty, 0.5)';
-- Valor de punto incorrecto
SELECT pose 'Pose(Linestring(1 1,2 2), 1.0)';
-- Valor de radio incorrecto
SELECT pose 'Pose(Point(1 1), -10.0)';
-- Punto 3D incorrecto
SELECT pose 'Pose(Point Z(1 1), 1.0)';
-- Orientación 3D incompleta
SELECT pose 'Pose(Point Z(1 1 1), 1.0)';
```

A continuación damos las funciones y operadores para el tipo pose.

13.1.1. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

$$\text{asText}(\{\text{pose}, \text{pose}[]\}) \rightarrow \{\text{text}, \text{text}[]\}$$

```
asEWKT ({pose,pose[]}) → {text,text[]}
```

```
SELECT asText(pose 'SRID=4326;Pose(Point(0 0),1)');
-- Pose(Point(0 0),1)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(Point(0 0),1)","Pose(Point(1 1),2)"}
SELECT asEWKT(pose 'SRID=4326;Pose(Point(0 0),1)');
-- SRID=4326;Pose(Point(0 0),1)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2)"}

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

```
asBinary(pose, endian text="") → bytearray
```

```
asEWKB(pose, endian text="") → bytearray
```

```
asHexWKB(pose, endian text="") → text
```

```
asHexEWKB(pose, endian text="") → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(pose 'Pose(Point(1 2),1)');
-- \x0101000000000000f03f00000000000004000000000000f03f
SELECT asEWKB(pose 'SRID=7844;Pose(Point(1 2),1)');
-- \x0141a41e00000000000000f03f00000000000004000000000000f03f
SELECT asHexWKB(pose 'Pose(Point(1 2),1)');
-- 0101000000000000F03F00000000000004000000000000F03F
SELECT asHexEWKB(pose 'SRID=3812;Pose(Point(1 2),1)');
-- 0141E40E0000000000000000F03F00000000000004000000000000F03F
```

- Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
poseFromText (text) → pose
```

$$\text{poseFromEWKT}(\text{text}) \rightarrow \text{pose}$$

```
SELECT asEWKT(poseFromText(text 'Pose(Point(1 2),1)');
-- Pose(Point(1 2),1)
SELECT asEWKT(poseFromEWKT(text 'SRID=3812;Pose(Point(1 2),1)');
-- SRID=3812;Pose(Point(1 2),1)
```

- Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

poseFromBinary(bytea) → pose

poseFromEWKB(bytea) → pose

poseFromHexEWKB(text) → pose

```
SELECT asEWKT(poseFromBinary(
  '\x0101000000000000f03f000000000000400000000000f03f');
-- Pose(Point(1 2),1)
SELECT asEWKT(poseFromEWKB(
  '\x0141a41e00000000000000f03f000000000000400000000000f03f');
-- SRID=7844;Pose(Point(1 2),1)
SELECT asEWKT(poseFromHexEWKB(
  '0141E40E00000000000000F03F000000000000400000000000F03F');
-- SRID=3812;Pose(Point(1 2),1)
```

13.1.2. Constructores

- Constructor para poses

pose(geompoint2D,float) → pose

pose(geompoint3D,float,float,float,float) → pose

pose(geompoint3D,yaw float,pitch float,roll float) → pose

La forma de tres ángulos toma la orientación en la otra codificación que prescribe el estándar OGC GeoPose v1.0, una terna yaw / pitch / roll en radianes bajo la convención Tait-Bryan intrínseca ZYX, y almacena el cuaternión que denota. Es la inversa de **ypr**.

```
SELECT asText(pose(ST_Point(1,1), radians(45)), 6);
-- Pose(Point(1 1),0.785398)
SELECT asEWKT(pose(ST_Point(1,1,3812), radians(45)), 6);
-- SRID=3812;Pose(Point(1 1),0.785398)
SELECT asText(pose(ST_PointZ(1,1,1), 1, 0, 0, 0));
-- Pose(Point Z (1 1 1),1,0,0,0)
SELECT asText(pose(ST_PointZ(1,1,1), radians(90), 0, 0), 6);
-- Pose(Point Z (1 1 1),0.707107,0,0,0.707107)
```

13.1.3. Conversión de tipos

Los valores del tipo `pose` se pueden convertir al tipo de punto `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación.

- Convertir una pose y, opcionalmente, una marca de tiempo o un período, en una caja espaciotemporal

pose::stbox

stbox(pose) → stbox

stbox(pose,{timestampz,tstzspan}) → stbox

```
SELECT stbox(pose 'SRID=5676;Pose(Point(1 1),0.3)');
-- SRID=5676;STBOX X((1,1),(1,1))
SELECT stbox(pose 'Pose(Point(1 1),0.3)', timestampz '2001-01-01');
-- STBOX XT((1,1),(1.3,1.3)), [2001-01-01, 2001-01-01])
SELECT stbox(pose 'Pose(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT((1,1),(1.3,1.3)), [2001-01-01, 2001-01-02])
```

- Convertir una pose en un punto geométrico

pose::geompoint

```
SELECT ST_AsText(pose(ST_Point(1, 1), 1)::geometry);
-- Point(1 1)
SELECT ST_AsEWKT(pose(ST_PointZ(1, 1, 1, 5676), 1, 0, 0, 0)::geometry);
-- SRID=5676;POINT(1 1 1)
```

13.1.4. Accesores

- Devuelve el punto

point(pose) → geompoint

```
SELECT ST_AsText(point(pose 'Pose(Point(1 1), 0.3)'));
-- Point(1 1)
```

- Devuelve el cuaternión de orientación de una pose

quaternion(pose) → quaternion

Las componentes se devuelven en el orden W, X, Y, Z, la convención de Hamilton en la que una pose 3D las almacena. Esta es una de las dos codificaciones de orientación que prescribe el estándar OGC GeoPose v1.0; la otra, yaw / pitch / roll, la proporciona **ypr**. Ambas están definidas para las dos dimensiones: la orientación de una pose 2D es un giro alrededor de la vertical local por su ángulo almacenado, de modo que coloca la mitad de ese ángulo en W y Z y deja X e Y en cero. Este es el cuaternión que un documento GeoPose Basic-Quaternion lleva para una pose plana.

```
SELECT quaternion(pose 'Pose(Point Z(1 1 1), 0, 0, 0, 1)');
-- (0,0,0,1)
SELECT quaternion(pose 'Pose(Point(1 1), 0)');
-- (1,0,0,0)
```

13.1.5. Transformaciones

- Redondear el punto y la orientación de la pose al número de posiciones decimales

round(pose, integer=0) → pose

```
SELECT asText(round(pose(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Pose(POINT(1.123457 1.123457),0.123457)
```

13.1.6. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial

SRID(pose) → integer

setSRID(pose) → pose


```
SELECT SRID(pose 'Pose(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(pose 'Pose(Point(0 0),1)', 4326));
-- SRID=4326;Pose(POINT(0 0),1)
```

■ Transformar a un identificador de referencia espacial

`transform(pose, integer) → pose`

`transformPipeline(pose, pipeline text, to_srid integer, is_forward bool=true) → pose`

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando la pose de entrada tiene un SRID desconocido (representado por 0).

En una pose 3D, la orientación se reexpresa en la base del marco de destino en el punto de la pose. La corrección está definida para el par canónico de OGC GeoPose, WGS-84 geográfico (EPSG:4326) ↔ WGS-84 ECEF (EPSG:4978), y toma la base estándar Este-Norte-Arriba en el punto geográfico como pivote de rotación. Para cualquier otro par de SRID, `transform` emite un NOTICE y deja pasar la orientación sin cambios. En una pose 2D, el ángulo es intrínseco a la proyección de origen y se deja pasar sin cambios.

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID de la pose de entrada se ignora y el SRID de la pose de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `to_srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(pose 'SRID=4326;Pose(Point(4.35 50.85),1)', 3812), 6);
-- SRID=3812;Pose(POINT(648679.018035 671067.055638),1)
```

-- Un viaje de ida y vuelta de una pose 3D por ECEF aterriza en la pose de entrada

```
SELECT asEWKT(round(
  transform(transform(pose 'SRID=4326;Pose(Point(8 47 0), 1, 0, 0, 0)', 4978),
    4326), 6));
-- SRID=4326;Pose(POINT Z (8 47 0),1,0,0,0)
```

-- En el cruce del ecuador y el meridiano (lat=lon=0), el cuaternión identidad
-- del cuerpo en la base ECEF es la rotación canónica Este-Norte-Arriba a ECEF

```
SELECT asEWKT(round(
  transform(pose 'SRID=4326;Pose(Point(0 0 0), 1, 0, 0, 0)', 4978), 6));
-- SRID=4978;Pose(POINT Z (6378137 0 0),0.5,0.5,0.5,0.5)
```

```
WITH test(pose, pipeline) AS (
  SELECT pose 'Pose(SRID=4326;Point(4.3525 50.846667),1)',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(pose, pipeline, 4326), pipeline,
  4326, false), 6)
FROM test;
-- SRID=4326;Pose(POINT(4.3525 50.846667),1)
```

13.1.7. Operaciones de distancia

■ Devuelve la distancia

`distance({geo, stbox, pose}, pose) → float`

`distance(pose, {geo, stbox, pose}) → float`

`{geo, stbox, pose} <-> pose → float`

Solo se tiene en cuenta el componente de punto de la pose, la orientación se ignora. La distancia se calcula en dos dimensiones, incluso cuando los argumentos son tridimensionales. El resultado es NULL cuando la geometría está vacía.

```

SELECT round(distance(geometry 'Point(1 0)', pose 'Pose(Point(4 0),0)'), 6);
-- 3
SELECT round(distance(stbox 'STBOX X((1,-1),(3,1))', pose 'Pose(Point(4 0),0)'), 6);
-- 1
SELECT round(pose 'Pose(Point(0 0),0)' <-> pose 'Pose(Point(3 4),0)', 6);
-- 5
SELECT round(pose 'Pose(Point(0 0 0), 1, 0, 0, 0)' <->
  pose 'Pose(Point(3 4 12), 1, 0, 0, 0)', 6);
-- 5
SELECT round(geometry 'Point empty' <-> pose 'Pose(Point(4 0),0)', 6);
-- NULL

```

13.1.8. Comparaciones

Los operadores de comparación (=, < y así sucesivamente) están disponibles para poses. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en poses.

■ Comparaciones tradicionales

pose {=, <>, <, >, <=, >} pose

```

SELECT pose 'Pose(Point(3 3), 0.5)' = pose 'Pose(Point(3 3), 0.5)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' <> pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' < pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.6)' > pose 'Pose(Point(2 2), 0.6)';
-- true
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)' <= pose 'Pose(Point Z(2 2 2), 0.5, 0.5, 0.5, 0.5)';
-- true
SELECT pose 'Pose(Point(1 1), 0.6)' >= pose 'Pose(Point(1 1), 0.5)';
-- true

```

■ ¿Son las poses aproximadamente iguales con respecto a un valor épsilon?

pose ~= pose → boolean

```

SELECT pose 'Pose(SRID=5676;Point(1 1), 0.3)' ~=
  pose 'Pose(SRID=5676;Point(1 1.0000001), 0.30000001)';
-- true

```

13.2. Poses temporales

El tipo de pose temporal tpose permite representar la evolución en el tiempo de la posición de los objetos y su orientación. Como todos los tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de tpose en estos subtipos.

```

SELECT tpose 'Pose(Point(1 1), 0.5)@2001-01-01';
SELECT tpose '{Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}';
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03]';
SELECT tpose '{[Pose(Point(1 1), 1)@2001-01-01, Pose(Point(2 2), 1)@2001-01-02],
  [Pose(Point(2 2), 2)@2001-01-04, Pose(Point(2 2), 3)@2001-01-05]}';

```

Al igual que para los tipos de punto temporal, el tipo de pose temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo, la dimensionalidad del punto y/o el identificador de referencia espacial (SRID). Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet` y los valores posibles para el tipo de geometría son `Point` o `PointZ`. Los tres argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo, dimensionalidad y/o SRID.

```
SELECT asEWKT(tpose(Sequence,5676) 'SRID=5676;[Pose(Point(1 1), 0.2)@2001-01-01,
    Pose(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-03]
SELECT tpose(Sequence) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tpose(PointZ) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- Column has Z dimension but the tpose value does not
SELECT tpose(5676) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal pose SRID (0) does not match column SRID (5676)
```

Los valores de pose temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT asText(tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
    Pose(Point(2 2), 0.4)@2001-01-02, Pose(Point(3 3), 0.6)@2001-01-03]');
-- [Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(3 3),0.6)@2001-01-03]
SELECT asText(tpose '{[Pose(Point(1 1), 0.2)@2001-01-01,
    Pose(Point(2 2), 0.3)@2001-01-02, Pose(Point(2 2), 0.5)@2001-01-03),
    [Pose(Point(2 2), 0.5)@2001-01-03, Pose(Point(2 2), 0.7)@2001-01-04]}');
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.3)@2001-01-02,
    Pose(Point(2 2),0.7)@2001-01-04)} */
```

13.3. Validez de las poses temporales

Los valores de pose temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- No se permiten valores nulos
SELECT tpose 'NULL@2001-01-01 08:05:00';
SELECT tpose 'Point(0 0)@NULL';
-- El tipo base no es una pose
SELECT tpose 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para las poses temporales. La mayoría de las funciones y operadores para tipos temporales y tipos espaciotemporales descritos en los capítulos precedentes se pueden aplicar para los tipos de pose temporal. Por lo tanto, en las firmas de las funciones, el tipo `pose` se puede utilizar siempre que se especifiquen los tipos `base` y `spatial`, y el tipo `tpose` se puede utilizar siempre que se especifiquen los tipos `ttype` y `tpatial`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para poses temporales y nos remitimos a los capítulos precedentes para explicaciones detalladas sobre ellos.

13.4. Entrada y salida

- Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

```
asText({tpose,tpose[]}) → {text,text[]}
asEWKT({tpose,tpose[]}) → {text,text[]}
```

```

SELECT asText(tpose 'SRID=4326;[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02]');
-- [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02]
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(Point(0 0),1)","Pose(Point(1 1),2)"}
SELECT asEWKT(tpose 'SRID=4326;[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02]');
-- SRID=4326;[Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02]
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)',
  'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2)"}

```

- Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)

asMFJSON(tpose) → text

```

SELECT asMFJSON(tpose 'Pose(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2],[1,2]],"period":{"begin":"2001-01-01T00:00:00+01",
  "end":"2001-01-01T00:00:00+01","lower_inc":true,"upper_inc":true},
  "values":[{"position":{"lat":2,"lon":1,"yaw":0.5}},
  "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */
SELECT asMFJSON(tpose 'Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2,3],[1,2,3]],
  "period":{"begin":"2001-01-01T00:00:00+01","end":"2001-01-01T00:00:00+01",
  "lower_inc":true,"upper_inc":true},
  "values":[{"position":{"lat":2,"lon":1,"h":3,"quaternion":{"x":0,"y":0,"z":0,"w":1}}},
  "datetimes":["2001-01-01T00:00:00+01"],"interpolation":"None"} */

```

- Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

asBinary(tpose, endian text='') → bytea

asEWKB(tpose, endian text='') → bytea

asHexWKB(tpose, endian text='') → text

asHexEWKB(tpose, endian text='') → text

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```

SELECT asBinary(tpose 'Pose(Point(1 2),1)@2001-01-01');
-- \x013800010100000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asEWKB(tpose 'SRID=7844;Pose(Point(1 2),1)@2001-01-01');
-- \ ←
   x01380041a41e000041a41e00000000000000f03f000000000000400000000000f03f009c57d3c11c0000 ←
SELECT asHexWKB(tpose 'Pose(Point(1 2),1)@2001-01-01');
-- 0138000101000000000000f03f000000000000400000000000f03f009c57d3c11c0000
SELECT asHexEWKB(tpose 'SRID=3812;Pose(Point(1 2),1)@2001-01-01');
-- 01380041 ←
   E40E000041E40E00000000000000f03f000000000000400000000000f03f009c57d3c11c0000

```

- Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

tposeFromText(text) → tpose

tposeFromEWKT(text) → tpose

```
SELECT asEWKT(tposeFromText(text '[Pose(Point(1 2),1)@2001-01-01,
Pose(Point(3 4),2)@2001-01-02]'));
-- [Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tposeFromEWKT(text 'SRID=3812;[Pose(Point(1 2),1)@2001-01-01,
Pose(Point(3 4),2)@2001-01-02]'));
-- SRID=3812;[Pose(Point(1 2),1)@2001-01-01, Pose(Point(3 4),2)@2001-01-02]
```

- Entrada desde la representación JSON de características móviles (Moving Features JSON o MF-JSON)

`tposeFromMFJSON(bytea) → tpose`

```
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose
'SRID=3812;Pose(Point(1 2),0.5)@2001-01-01', 3)));
-- SRID=3812;Pose(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose
'SRID=5676;Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3)));
-- SRID=5676;Pose(POINT Z(1 2 3),1,0,0,0)@2001-01-01
```

- Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

`tposeFromBinary(bytea) → tpose`

`tposeFromEWKB(bytea) → tpose`

`tposeFromHexEWKB(text) → tpose`

```
SELECT asEWKT(tposeFromBinary(
'\x013a0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000'));
-- Pose(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tposeFromEWKB(
'\x013a0001000000000000f03f000000000000400000000000f03f009c57d3c11c0000'));
-- SRID=7844;Pose(Point(1 1),2)@2001-01-01
SELECT asEWKT(tposeFromHexEWKB(
'013A0041E40E0000E40E000000000000F03F...400000000000F03F009C57D3C11C0000'));
-- SRID=3812;Pose(POINT(1 2),1)@2001-01-01
```

13.5. Constructores

- Constructor para poses temporales con valor constante

`tpose(pose,timestamp tz) → tposeInst`

`tpose(pose,tstzset) → tposeDiscSeq`

`tpose(pose,tstzspan,interp='linear') → tposeContSeq`

`tpose(pose,tstzspanset,interp='linear') → tposeSeqSet`

```
SELECT asText(tpose('Pose(Point(1 1), 0.5)', timestamp tz '2001-01-01'));
-- Pose(Point(1 1),0.5)@2001-01-01
SELECT asText(tpose('Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)',
tstzset '{2001-01-01, 2001-01-05}'));
/* {Pose(POINT Z(1 1 1),0.5,0.5,0.5,0.5)@2001-01-01,
Pose(POINT Z(1 1 1),0.5,0.5,0.5,0.5)@2001-01-05} */
SELECT asText(tpose('Pose(Point(1 1), 0.5)',
tstzspan '[2001-01-01, 2001-01-02]'));
-- [Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(1 1),0.5)@2001-01-02]
SELECT asText(tpose('Pose(Point(1 1), 0.2)',
tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(1 1),0.2)@2001-01-03]}
```

- Constructor para poses temporales de subtipo secuencia

```
tposeSeq(tposeInst[],interp={'step','linear'},leftInc bool=true,
rightInc bool=true) → tposeSeq
```

```
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.3)@2001-01-01',
'Pose(Point(2 2), 0.5)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* {Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02,
Pose(Point(1 1),0.5)@2001-01-03} */
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.2)@2001-01-01',
'Pose(Point(1 1), 0.4)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* [Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(1 1),0.4)@2001-01-02,
Pose(Point(1 1),0.5)@2001-01-03] */
```

- Constructor para poses temporales de subtipo conjunto de secuencias

```
tposeSeqSet(tpose[]) → tposeSeqSet
```

```
tposeSeqSetGaps(tposeInst[],maxt=NULL,maxdist=NULL,interp='linear') → tposeSeqSet
```

```
SELECT asText(tposeSeqSet(ARRAY[tpose
'[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02]',
'Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.8)@2001-01-04']));
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02],
[Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.8)@2001-01-04]} */
SELECT asText(tposeSeqSetGaps(ARRAY[tpose 'Pose(Point(1 1),0.1)@2001-01-01',
'Pose(Point(1 1),0.3)@2001-01-03', 'Pose(Point(1 1),0.5)@2001-01-05', '1 day']));
/* {[Pose(Point(1 1),0.1)@2001-01-01], [Pose(Point(1 1),0.3)@2001-01-03],
[Pose(Point(1 1),0.5)@2001-01-05]} */
```

- Construir una pose temporal 2D a partir de un punto geométrico temporal y un flotante temporal

```
tpose(tgeompoint, tfloat) → tpose
```

El marco temporal del resultado es la intersección de los marcos temporales del punto temporal y el flotante temporal. Si no se intersecan, se devuelve un valor nulo

```
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
tfloat '[1@2001-01-01, 2@2001-01-02]'));
-- [Pose(Point(1 1),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02]
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02]',
tfloat '[2@2001-01-03, 3@2001-01-04]'));
-- NULL
```

13.6. Conversión de tipos

Un valor de pose temporal se puede convertir hacia y desde un punto temporal, que es un punto geométrico temporal para una pose planar y un punto geográfico temporal para una pose geodésica. Esto se puede hacer utilizando un CAST explícito o utilizando la notación `::`. Convertir una pose temporal al tipo de punto temporal del otro marco de referencia genera un error. Se devuelve un valor nulo si alguno de los valores de punto geométrico que lo componen no se puede convertir en un valor `pose`.

- Convertir una pose temporal en un punto geométrico temporal

```
tpose::tgeompoint
```

```
SELECT asText((tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
Pose(Point(1 1), 0.3)@2001-01-02]')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02]
SELECT asEWKT((tpose '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
Pose(Point Z(2 2 2), 1, 0, 0, 0)@2001-01-02]')::tgeompoint);
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02]
```

- Convertir una pose temporal geodésica en un punto geográfico temporal

`tpose::tgeogpoint`

```
SELECT asText((tpose '[GeodPose(Point(1 1), 0.2)@2001-01-01,
  GeodPose(Point(1 1), 0.3)@2001-01-02]')::tgeogpoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02]
```

13.7. Accesos

- Devuelve los valores

`getValues(tpose) → poseset`

```
SELECT asText(getValues(tpose '{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]}'));
-- {"Pose(Point(1 1),0.3)","Pose(Point(1 1),0.5)"}
SELECT asEWKT(getValues(tpose 'SRID=5676;{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02]}'));
-- SRID=5676;{"Pose(Point(1 1),0.3)"}
```

- Devuelve los puntos

`points(tpose) → geomset`

```
SELECT asEWKT(points(tpose 'SRID=5676;{Pose(Point(3 3), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]}'));
-- SRID=5676;{Point(1 1), Point(3 3)}
```

- Devuelve la trayectoria

`trajectory(tpose) → geometry`

Una pose tiene una posición y una orientación, pero no una extensión, por lo que la geometría que cubre a lo largo del tiempo es la trayectoria de su posición, como para un punto temporal. Una pose que interpola se desplaza por la línea entre dos posiciones consecutivas; una que no interpola permanece en cada una de ellas y no cubre nada entre ellas.

```
SELECT ST_AsText(trajectory(tpose '[Pose(Point(1 1),0.5)@2000-01-01,
  Pose(Point(3 3),0.5)@2000-01-02]'));
-- LINESTRING(1 1,3 3)
SELECT ST_AsText(trajectory(tpose 'Interp=Step;[Pose(Point(1 1),0.5)@2000-01-01,
  Pose(Point(3 3),0.5)@2000-01-02]'));
-- MULTIPOINT((1 1),(3 3))
```

- Devuelve el valor en una marca de tiempo

`valueAtTimestamp(tpose,timestamp) → pose`

```
SELECT asText(valueAtTimestamp(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(3 3), 0.5)@2001-01-03]', '2001-01-02'));
-- Pose(Point(2 2),0.4)
SELECT asText(valueAtTimestamp(tpose
  '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(3 3 3), 1, 0, 0, 0)@2001-01-03]', timestamp '2001-01-02'), 6);
-- Pose(POINT Z (2 2 2),0.866025,0.288675,0.288675,0.288675)
```

- Devuelve la velocidad de una pose temporal como un flotante temporal: la magnitud de la velocidad de la componente de posición (distancia recorrida por unidad de tiempo). La orientación no es considerada; para la velocidad angular véase `angularSpeed`.

`speed(tpose) → tfloat`

```
SELECT asText(speed(tpose '[Pose(Point(0 0), 0)@2000-01-01,
    Pose(Point(1 1), 0.5)@2000-01-02]'));
-- Interp=Step;[1.6368e-5@2000-01-01, 1.6368e-5@2000-01-02]
-- (sqrt(2) units / 86400 seconds, units depend on the SRID's CRS)
```

- Devuelve la velocidad angular de una pose temporal como un flotante temporal con interpolación por pasos (radianes por unidad de tiempo). Para las poses 2D es el delta theta del arco más corto por segmento dividido por la duración del segmento; para las poses 3D es el ángulo de arco de SLERP $2 \cdot \arccos(|q_1 \cdot q_2|)$ dividido por la duración del segmento. SLERP tiene por construcción velocidad angular constante a lo largo de un segmento, por lo que el resultado es constante a trozos.

angularSpeed(tpose) → tfloat

```
-- 90-degree yaw rotation over one day -> pi/2 rad / 86400 s
SELECT asText(angularSpeed(tpose
    '[Pose(Point(0 0 0), 1, 0, 0, 0)@2000-01-01,
    Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)@2000-01-02]'));
-- Interp=Step;[1.8181e-5@2000-01-01, 1.8181e-5@2000-01-02]
```

13.8. Transformaciones

- Transformar a otro subtipo

tposeInst(tpose) → tposeInst

tposeSeq(tpose) → tposeSeq

tposeSeqSet(tpose) → tposeSeqSet

```
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Pose(Point(1 1),0.5)@2001-01-01}
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01'));
-- [Pose(Point(1 1),0.5)@2001-01-01]
SELECT asText(tposeSeqSet(tpose 'Pose(PointZ(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01'));
-- {[Pose(PointZ(1 1 1),0.5,0.5,0.5,0.5)@2001-01-01]}
```

- Transformar a otra interpolación

setInterp(tpose, interp) → tpose

```
SELECT asText(setInterp(tpose 'Pose(Point(1 1),0.2)@2001-01-01', 'linear'));
-- [Pose(Point(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tpose '{[Pose(Point Z(1 1 1),1,0,0,0)@2001-01-01,
    [Pose(Point Z(2 2 2),0,0,0,1)@2001-01-02]]', 'discrete'));
-- {[Pose(PointZ(1 1 1),1,0,0,0)@2001-01-01, Pose(PointZ(2 2 2),0,0,0,1)@2001-01-02]}
```

- Redondear los puntos y las orientaciones de la pose temporal al número de posiciones decimales

round(tpose, integer=0) → tpose

```
SELECT asText(round(tpose '{[Pose(Point(1 1.123456789),0.123456789)@2001-01-01,
    Pose(Point(1 1),0.5)@2001-01-02]'}', 3));
-- {[Pose(Point(1 1.123),0.123)@2001-01-01, Pose(Point(1 1),0.5)@2001-01-02]}
```

13.9. Modificaciones

- Agregar un instante temporal o una secuencia temporal

appendInstant(tpose, tposeInst) → tpose

appendSequence(tpose, tposeSeq) → tpose


```

SELECT asText(appendInstant(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'Pose(Point(2 2), 1) @2001-01-02'));
-- [Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(2 2),1)@2001-01-02]
SELECT asText(appendSequence(tpose '[Pose(Point(1 1), 0.5)@2001-01-01]', '[Pose(Point(2 2), 1)@2001-01-02]'));
-- {[Pose(Point(1 1),0.5)@2001-01-01], [Pose(Point(2 2),1)@2001-01-02]}

```

■ Fusionar las poses temporales

`merge(tpose,tpose) → tpose`

`merge(tpose[]) → tpose`

`merge(tpose) → tpose`

`mergeAgg(tpose) → tpose`

```

SELECT asText(merge(tpose
  '[Pose(Point(1 1 1),1,0,0,0)@2001-01-01, Pose(Point(2 2 2),0,1,0,0)@2001-01-02]',
  '[Pose(Point(2 2 2),0,1,0,0)@2001-01-02, Pose(Point(3 3 3),0,0,0,1)@2001-01-03]'));
/* [Pose(Point(1 1 1),1,0,0,0)@2001-01-01, Pose(Point(2 2 2),0,1,0,0)@2001-01-02,
   Pose(Point(3 3 3),0,0,0,1)@2001-01-03] */
SELECT asText(merge(ARRAY[tpose
  '{[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(2 2),0.2)@2001-01-02],
   [Pose(Point(3 3),0.3)@2001-01-03, Pose(Point(4 4),0.4)@2001-01-04]}',
  '[Pose(Point(4 4),0.4)@2001-01-04, Pose(Point(5 5),0.5)@2001-01-05]']));
/* {[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(2 2),0.2)@2001-01-02],
   [Pose(Point(3 3),0.3)@2001-01-03, Pose(Point(5 5),0.5)@2001-01-05]} */
WITH temp(inst) AS (
  SELECT tpose 'Pose(Point(1 1),0.1)@2001-01-01' UNION
  SELECT tpose 'Pose(Point(2 2),0.2)@2001-01-02' UNION
  SELECT tpose 'Pose(Point(3 3),0.3)@2001-01-03' )
SELECT asText(mergeAgg(inst)) FROM temp;
/* {[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(2 2),0.2)@2001-01-02,
   Pose(Point(3 3),0.3)@2001-01-03]} */

```

13.10. Restricciones

■ Restringir al (complemento de) un valor o un conjunto de valores

`atValue(tpose,value) → tpose`

`minusValue(tpose,value) → tpose`

`atValues(tpose,valueSet) → tpose`

`minusValues(tpose,valueSet) → tpose`

```

SELECT atValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
-- {[Pose(Point(2 2),0.5)@2001-01-02]}
SELECT minusValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
/* {[Pose(Point(2 2),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02],
   [Pose(Point(2 2),0.5)@2001-01-02, Pose(Point(2 2),0.7)@2001-01-03]} */

```

■ Restringir al (complemento de) una geometría

`atGeometry(tpose,geometry) → tpose`

`minusGeometry(tpose,geometry) → tpose`

```
SELECT atGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-03]',
'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
Pose(Point(2 2), 0.7)@2001-01-03]',
'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(Pose(Point(2 2),0.342593)@2001-01-01 05:06:40.364673+01,
Pose(Point(2 2),0.7)@2001-01-03)} */
```

- Restringir al (complemento de) un span de elevación

`atElevation(tpose, floatspan) → tpose`

`minusElevation(tpose, floatspan) → tpose`

La elevación de una pose es la elevación de su posición, por lo que la restricción conserva los tiempos en los que esa posición se encuentra dentro del span. La pose temporal debe tener dimensión Z.

```
SELECT asText(atElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(Point Z (0 0 1),1,0,0,0)@2001-01-02,
Pose(Point Z (0 0 2),1,0,0,0)@2001-01-03]} */
SELECT asText(minusElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(Point Z (0 0 0),1,0,0,0)@2001-01-01,
Pose(Point Z (0 0 1),1,0,0,0)@2001-01-02,
(Pose(Point Z (0 0 2),1,0,0,0)@2001-01-03,
Pose(Point Z (0 0 4),1,0,0,0)@2001-01-05)} */
```

13.11. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial

`SRID(tpose) → integer`

`setSRID(tpose) → tpose`

```
SELECT SRID(tpose 'SRID=5676;[Pose(Point(0 0),1)@2001-01-01,
Pose(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tpose '[Pose(Point(0 0),1)@2001-01-01,
Pose(Point(1 1),2)@2001-01-02]', 5676));
-- SRID=5676;[Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)
```

- Transformar a un identificador de referencia espacial

`transform(tpose, integer) → tpose`

`transformPipeline(tpose, pipeline text, to_srid integer, is_forward bool=true) → tpose`

La pose de cada instante se transforma igual que la pose estática correspondiente, con la corrección de orientación incluida, como se describe para **transform**. Lo mismo ocurre con un `poseset`. Una `trgeometry` asocia sus poses con una geometría de referencia que comparte su marco, y la transformación a otro SRID no está admitida para ese tipo.

```
SELECT asEWKT(transform(tpose 'SRID=4326;Pose(Point(4.35 50.85),1)@2001-01-01',
3812));
-- SRID=3812;Pose(Point(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tpose, pipeline) AS (
SELECT tpose 'SRID=4326;{Pose(Point(4.3525 50.846667),1)@2001-01-01,
Pose(Point(-0.1275 51.507222),2)@2001-01-02}',
text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
```

```
SELECT asEWKT(transformPipeline(transformPipeline(tpose, pipeline, 4326), pipeline,
4326, false), 6)
FROM test;
/* SRID=4326; {Pose(Point(4.3525 50.846667),1)@2001-01-01,
Pose(Point(-0.1275 51.507222),2)@2001-01-02} */
```

13.12. Operaciones de cuadro delimitador

■ Operadores topológicos

{tstzspan, stbox, tpose} {&&, <@, @>, ~=, -|-} {tstzspan, stbox, tpose} → boolean

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' && tstzspan '[2001-01-02,2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' @> stbox(pose 'Pose(Point(1 1), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-03]' ~= tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.35)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
```

■ Operadores de posición

{stbox, tpose} {<<, &<, >>, &>} {stbox, tpose} → boolean

{stbox, tpose} {<<|, &<|, |>>, |&>} {stbox, tpose} → boolean

{stbox, tpose} {<</, &</, />>, /&>} {stbox, tpose} → boolean

{tstzspan, stbox, tpose} {<<#, &<#, #>>, #&>} {tstzspan, stbox, tpose} → boolean

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' << stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' <<| stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1 1), 1,0,0,0)@2001-01-01,
Pose(Point(1 1 1), 1,0,0,0)@2001-01-02]' <</ stbox(pose 'Pose(Point(2 2 2), 1,0,0,0)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
Pose(Point(1 1), 0.5)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
Pose(Point(1 1), 0.5)@2001-01-05]' #>>
tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02]';
-- true
```

13.13. Operaciones de distancia

A continuación presentamos las funciones que calculan operaciones de distancia para poses temporales. Tenga en cuenta que para estas operaciones solo se considera el componente de punto, no la orientación.

■ Devuelve la distancia más pequeña alguna vez

{geo, pose, tpose, stbox} |=| {geo, pose, tpose, stbox} → float

```

SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01, Pose(Point(2 2), 0.7)@2001-01-02]' ||
  geometry 'Linestring(50 50,55 55)';
-- 67.88225099390856
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01, Pose(Point(2 2), 0.7)@2001-01-02]' ||
  pose 'Pose(Point(1 1), 0.5)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' || pose 'Pose(Point(2 2), 0.2)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' || geometry 'Linestring(2 2,2 1,3 1)';
-- 1

```

El operador `||` se puede utilizar para realizar búsquedas del vecino más cercano utilizando un índice GiST o SP-GiST (ver Sección 10.2).

- Devuelve el instante de la primera pose temporal en el que los dos argumentos están a la distancia más cercana

`nearestApproachInstant({geo,pose,tpose},{geo,pose,tpose}) → tpose`

```

SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Pose(POINT(2 2),0.3)@2001-01-01
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- Pose(POINT(2 2),0.3)@2001-01-01

```

- Devuelve la línea que conecta el punto de aproximación más cercano

`shortestLine({geo,pose,tpose},{geo,pose,tpose}) → geometry`

La función devuelve la primera línea que encuentre si hay más de una.

```

SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2 2,50 50)
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- LINESTRING(2 2,1 1)

```

- Devuelve la distancia temporal

`tpose <-> tpose → tfloat`

```

SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> pose 'Pose(Point(2 2), 0.2)', 6);
-- [1.414214@2001-01-01, 1.414214@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)', 6);
-- [69.296465@2001-01-01, 69.296465@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.5)@2001-01-03]' <->
  tpose '[Pose(Point(1 2), 0.3)@2001-01-02, Pose(Point(2 1), 0.5)@2001-01-04]', 6);
-- [0.707107@2001-01-02, 0.5@2001-01-02 12:00:00+01, 0.707107@2001-01-03]

```

13.14. Relaciones espaciales

Las relaciones topológicas y de distancia descritas en la Sección 8.5 tales como `eIntersects`, `aDwithin` o `tContains` solo consideran la ubicación de las geometrías, no su orientación. Para poder aplicar estas relaciones espaciales a las poses temporales, se deben transformar a puntos geométricos temporales. Esto se puede realizar fácilmente utilizando las funciones `geometry` y `tgeompoint` o utilizando un casting explícito `::` como se muestra en los ejemplos a continuación.

■ Relaciones alguna vez y siempre

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)']);
-- true
SELECT eDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)']);
-- true
SELECT eIntersects(
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03]::tgeompoint,
  tpose '[Pose(Point(2 2),0.0)@2001-01-01, Pose(Point(2 2),1)@2001-01-03]::tgeompoint);
-- false
```

■ Relaciones espaciotemporales

```
SELECT tContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)']);
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
  tgeompoint(tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)']);
-- [t@2001-01-01, t@2001-01-03]
SELECT tDwithin(
  tpose '[Pose(Point(1 1),0.3)@2001-01-01,Pose(Point(1 1),0.5)@2001-01-03]::tgeompoint,
  tpose '[Pose(Point(1 1),0.5)@2001-01-01,Pose(Point(3 3),0.3)@2001-01-03]::tgeompoint,1);
/* {[t@2001-01-01 00:00:00+01, t@2001-01-01 16:58:14.025894+01],
  (f@2001-01-01 16:58:14.025894+01, f@2001-01-03 00:00:00+01)} */
```

13.15. Comparaciones

■ Comparaciones tradicionales

`tpose {=, <>, <, >, <=, >=} tpose → boolean`

```
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02,
  [Pose(Point(1 1), 0.3)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]]' =
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]]' <>
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.6)@2001-01-03]';
-- true
```

■ Comparaciones alguna vez y siempre

`tpose {?=?, %=} tpose → boolean`

```
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-04]' ?= Pose(Point(1 1), 0.3);
-- true
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.2)@2001-01-04]' &= Pose(Point(1 1), 0.2);
-- true
```

■ Comparaciones temporales

`tpose {#=#, #<>} tpose → tbool`

```

SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
Pose(Point(1 1), 0.4)@2001-01-03)' #= pose 'Pose(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
Pose(Point(1 1), 0.8)@2001-01-03)' #<>
tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}

```

13.16. Agregaciones

Las tres funciones de agregación para poses temporales se ilustran a continuación.

■ Conteo temporal

`tCount(tpose) → {tintSeq, tintSeqSet}`

```

WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
Pose(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
Pose(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
Pose(Point(1 1), 0.5)@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}

```

■ Conteo de ventana

`wCount(tpose) → {tintSeq, tintSeqSet}`

```

WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
Pose(Point(1 1), 0.3)@2001-01-03]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
Pose(Point(1 1), 0.4)@2001-01-04]' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03,
Pose(Point(1 1), 0.5)@2001-01-05]' )
SELECT wCount(Temp, '1 day')
FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
1@2001-01-06]} */

```

13.17. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tablas de poses temporales. Un ejemplo de creación de índice es el siguiente:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para las poses temporales, que es un `stbox`, como todos los tipos espaciotemporales.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en las poses temporales,

- `<<#, &<#, #&>, #>>`, que solo consideran la dimensión temporal en las poses temporales,
- `&&, @>, <@, ~=-, -|-`, y `|=|`, que consideran tantas dimensiones como las que comparten la columna indexada y el argumento de la consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

13.18. Soporte de OGC GeoPose v1.0

El tipo `pose` implementa el modelo de datos que subyace al **estándar OGC GeoPose v1.0**: una `position` más una `orientation` en un marco de referencia conocido, donde la orientación es un cuaternión unitario en la convención de Hamilton o, de forma equivalente, una terna yaw / pitch / roll bajo la convención Tait-Bryan intrínseca ZYX. Esta sección agrupa la superficie SQL que expone esa interoperabilidad: E/S JSON para las clases de conformidad Basic y Advanced del estándar, una función auxiliar de renormalización explícita para quienes ejecutan composiciones largas, y los accesoros de ángulos de Euler que esperan los consumidores Basic-YPR.

13.18.1. Entrada y salida JSON

- Convertir hacia o desde la codificación JSON de OGC GeoPose v1.0 (clase de conformidad Basic-Quaternion, Basic-YPR o Advanced)

```
asGeoPose(pose, conformance int4=0, maxdecimaldigits int4=-1) → text
poseFromGeoPose(text) → pose
```

El argumento `conformance` selecciona la clase de salida: 0 = Basic-Quaternion (predeterminado, sin pérdida), 1 = Basic-YPR (yaw, pitch, roll en grados, Tait-Bryan intrínseco ZYX), 2 = Advanced. El argumento `maxdecimaldigits` es el número de dígitos significativos que se conservan en los números JSON; -1 utiliza el valor predeterminado sin pérdida de json-c. La función de entrada detecta automáticamente la clase de conformidad a partir de las claves JSON, donde el miembro `frameSpecification` implica Advanced, el miembro `quaternion` implica Basic-Quaternion y el miembro `angles` implica Basic-YPR.

Las clases Basic llevan la posición en un miembro `position` y dejan el marco implícito. La clase Advanced no tiene miembro de posición: nombra su marco exterior de forma explícita y la pose se sitúa en el origen de ese marco, de modo que las dos codificaciones de una misma pose se releen iguales. Todas las clases exigen un marco exterior geográfico, por lo que la pose debe ser geodésica; una pose planar se rechaza en el límite de la conversión, al igual que un SRID proyectado.

```
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 0, 6);
/* {"position":{"lat":47,"lon":8,"h":1500},"quaternion":{"x":0,"y":0,"z":0.707107,
    "w":0.707107}} */
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 1, 6);
-- {"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 2, 6);
/* {"frameSpecification":{"authority":"/geopose/1.0","id":"LTP-ENU","parameters":
    "longitude=8&latitude=47&height=1500&crs=EPSG:4979"},
    "quaternion":{"x":0,"y":0,"z":0.707107,"w":0.707107}} */
SELECT asEWKT(poseFromGeoPose(
    '{"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}'), 6);
-- SRID=4326;GeodPose(POINT Z (8 47 1500),0.707107,0,0,0.707107)
```

13.18.2. Renormalización de cuaterniones

- Devuelve una pose cuyo cuaternión de orientación se ha llevado a una norma unitaria

```
poseNormalize(pose) → pose
```

Una pose 2D se devuelve sin cambios, ya que su orientación es un único ángulo. A una pose 3D se le divide el cuaternión por su norma euclidiana, de modo que composiciones largas de SLERP (o cualquier otra vía que acumule deriva de punto flotante en $|q|$) pueden restaurarse al invariante $|q| = 1$ que requiere el código de SLERP y de descomposición de Euler.

```
SELECT asText (poseNormalize (pose 'Pose(Point(1 1 1), 0.5, 0.5, 0.5, 0.5)'));
-- Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)
```

- Devuelve la inversa de una pose

`poseInverse(pose) → pose`

La inversa invierte la relación entre marcos: donde una pose lleva un valor del marco que nombra al marco en el que ella misma está expresada, la inversa lo lleva de vuelta. Es $R_{BA} = R_{AB}^T$ y $t_{BA} = -R_{AB}^T t_{AB}$, de modo que componer una pose con su inversa da la identidad. Esto es lo que expresa una posición del mundo en el marco de un vehículo.

Una pose sobre un marco geográfico no tiene inversa: el marco que nombraría no es un marco del elipsoide.

```
SELECT asEWKT(round(poseInverse(pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(-5 10),-1.570796)
SELECT asEWKT(round(applyPose(poseInverse(pose(ST_Point(10,5), pi()/2)),
pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(0 0),0)
SELECT asEWKT(round(poseInverse(pose 'Pose(Point(1 0 0), 0, 0, 0, 1)'), 6));
-- Pose(POINT Z (1 0 0),0,0,0,-1)
```

13.18.3. Accesorios de ángulos de Euler

- Devuelve el ángulo yaw, pitch o roll de una pose, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose

`yaw(pose) → float`

`pitch(pose) → float`

`roll(pose) → float`

Para una pose 2D, `yaw` devuelve la rotación almacenada `theta`, por convención, el yaw del marco del cuerpo— y `pitch` y `roll` devuelven 0. Para una pose 3D, los tres valores provienen de la descomposición Tait-Bryan intrínseca ZYX del cuaternión de orientación. El término de pitch así se acota a $[-1, 1]$ para absorber la pequeña deriva numérica que pueden introducir las composiciones largas de cuaterniones.

Sobre una pose temporal, la interpolación del resultado sigue a la dimensión. Una pose 2D interpola linealmente su ángulo almacenado y ese ángulo es su yaw, de modo que un `tpose` 2D lineal se proyecta en un `tfloat` lineal. Una pose 3D interpola mediante SLERP, cuya descomposición Tait-Bryan no es lineal en el tiempo, de modo que un `tpose` 3D se proyecta en un `tfloat` escalonado: leerlo entre dos instantes devuelve el ángulo del instante de la izquierda en lugar de un ángulo que ninguna pose contiene.

```
SELECT yaw(pose 'Pose(Point(1 1), 0.5)');
-- 0.5
SELECT pitch(pose 'Pose(Point(1 1), 0.5)');
-- 0
SELECT roll(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 0
SELECT yaw(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 1.5707963267948966
```

- Devuelve la orientación de una pose como una terna yaw / pitch / roll, en radianes

`ypr(pose) → ypr`

Esta es la codificación Basic-YPR de la orientación en OGC GeoPose, la contraparte de **quaternion**, y devuelve los tres ángulos que **yaw**, **pitch** y **roll** responden de uno en uno. Al igual que **quaternion**, está definida para ambas dimensiones: una pose 2D gira por su ángulo almacenado y no cabecea ni alabea. Los ángulos están en radianes, mientras que la codificación JSON de GeoPose los escribe en grados.


```
SELECT ypr(pose 'Pose(Point(1 1), 0.5)');
-- (0.5,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 1, 0, 0, 0)');
-- (0,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)');
-- (1.5707963267948966,0,1.5707963267948966)
```

- Eleva los accesorios de ángulos de Euler a través de una pose temporal, devolviendo un flotante temporal (radianes) por cada instante bajo la convención Tait-Bryan intrínseca ZYX

```
yaw(tpose) → tfloat
pitch(tpose) → tfloat
roll(tpose) → tfloat
```

```
SELECT asText(yaw(tpose '[Pose(Point(0 0), 0.0)@2000-01-01,
Pose(Point(1 1), 0.5)@2000-01-02]'));
-- [0@2000-01-01, 0.5@2000-01-02]
```

13.18.4. Registro de metadatos de marcos

El estándar OGC GeoPose v1.0 distingue el *marco exterior* (la referencia global, por ejemplo, WGS-84 geográfico o ECEF) del *marco interior* (el marco del cuerpo cuya orientación es el cuaternión de la pose). Las clases de conformidad Basic exigen WGS-84 geográfico como marco exterior y un marco interior implícito de ejes de cuerpo dextrógiros; la clase Advanced nombra su marco exterior de forma explícita y sitúa la pose en el origen de ese marco.

En MobilityDB, el tipo `pose` codifica el marco exterior de forma implícita mediante su SRID y utiliza el marco interior convencional de ejes de cuerpo dextrógiros. La tabla `geopose_frames` registra esta correspondencia en un catálogo paralelo a la tabla `pointcloud_formats` de `pgPointCloud`. Las pilas de marcos de la clase Advanced no están admitidas.

```
SELECT frame_id, authority, code, name, is_geographic
FROM geopose_frames ORDER BY frame_id;
-- 1 | EPSG | 4326 | WGS-84 geographic (lat/lon/h) | t
-- 2 | EPSG | 4978 | WGS-84 ECEF (Earth-Centred Earth-Fixed) | f
-- 3 | OGC | LTP | Local Tangent Plane (East-North-Up) | f
-- 4 | OGC | BODY | Right-handed body axes (default inner frame) | f
```

Tres funciones auxiliares SQL proporcionan una interfaz de consulta estable que es independiente de la disposición de la tabla:

- Devuelve el SRID de un marco, o NULL si el marco es paramétrico (LTP, BODY)

```
geoPoseFrameSRID(int) → int
```

```
SELECT geoPoseFrameSRID(1), geoPoseFrameSRID(2);
-- 4326 | 4978
SELECT geoPoseFrameSRID(3);
-- NULL
```

- Devuelve el nombre legible del marco

```
geoPoseFrameName(int) → text
```

```
SELECT geoPoseFrameName(1);
-- WGS-84 geographic (lat/lon/h)
SELECT geoPoseFrameName(2), geoPoseFrameName(3);
-- WGS-84 ECEF (Earth-Centred Earth-Fixed) | Local Tangent Plane (East-North-Up)
```

- Devuelve true para los marcos lat/lon/h, false para los cartesianos o proyectados

```
geoPoseFrameIsGeographic(int) → boolean
```

```
SELECT geoPoseFrameIsGeographic(1), geoPoseFrameIsGeographic(2);
-- true | false
```

Los usuarios pueden registrar marcos personalizados insertándolos en `geopose_frames`; el catálogo está marcado como una tabla de configuración, por lo que `pg_dump` preserva las filas del usuario.

13.18.5. Transformación rígida cuerpo↔mundo

- Aplica la transformación de cuerpo rígido codificada por una pose (la correspondencia *cuerpo al mundo* de OGC GeoPose) a una geometría en el marco del cuerpo, produciendo la geometría correspondiente en el marco del mundo

La transformación es

$$\text{world} = R(q) \cdot \text{body} + p$$

donde (p, q) son la posición y la orientación de la pose. La forma estática toma una única pose y una geometría estática; la forma temporal eleva la transformación rígida por instante de una `tpose` a lo largo de sus instantes, produciendo una trayectoria `tgeompoint` en el marco del mundo de la geometría del cuerpo. La forma estática admite una geometría de cuerpo de cualquier tipo y lleva su forma al marco de la pose; la forma temporal devuelve una `tgeompoint`, por lo que su geometría de cuerpo debe ser un punto. La pose y la geometría del cuerpo deben tener la misma dimensionalidad y deben coincidir en su SRID, adoptando el desconocido el del otro. La interpolación lineal de la trayectoria resultante es la cuerda de cada segmento, que aproxima la verdadera trayectoria de cuerpo rígido (un arco circular bajo SLERP), la misma concesión que MobilityDB ya adopta para las trayectorias espaciales.

`applyPose(geometry, pose) → geometry`

`applyPose(geometry, tpose) → tgeompoint`

`applyPose(pose, pose) → pose`

```
-- A body sensor offset 1 unit along the body X axis, traced through a
-- tpose that ends 90 degrees yawed and translated to (10, 20).
SELECT asText(applyPose(ST_Point(1,0), tpose '[Pose(Point(0 0), 0)@2026-01-01,
Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'));
-- [POINT(1 0)@2026-01-01, POINT(10 21)@2026-01-02]
```

```
-- La forma de la geometría del cuerpo se lleva al marco de la pose: un casco
-- de 4 por 2 alrededor del origen del cuerpo, situado en (10, 20)
SELECT ST_AsText(applyPose(ST_MakeEnvelope(-2,-1,2,1),
pose 'Pose(Point(10 20), 0)'));
-- POLYGON((8 19,8 21,12 21,12 19,8 19))
```

Cualquiera de los dos marcos puede moverse. Aplicar una pose a una `tpose` lleva un cuerpo fijo a través de todo el movimiento de su padre, que es lo que pide un sensor montado sobre un vehículo en movimiento; aplicar una `tpose` a una pose lleva un cuerpo en movimiento a un marco que permanece quieto; y aplicar una `tpose` a una `tpose` compone dos marcos que ambos se mueven, sobre el tiempo que comparten. `poseInverse` se eleva de la misma manera, cambiando el punto de vista sobre un objeto en movimiento durante todo su movimiento.

`applyPose(pose, tpose) → tpose`

`applyPose(tpose, pose) → tpose`

`applyPose(tpose, tpose) → tpose`

`poseInverse(tpose) → tpose`

```
-- Un sensor una unidad delante de un cuerpo que termina girado un cuarto de
-- vuelta en (10, 20) termina una unidad al norte de él
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)',
tpose '[Pose(Point(0 0), 0)@2026-01-01,
Pose(Point(10 20), 1.5707963267948966)@2026-01-02]', 6)));
/* [Pose(POINT(1 0),0)@2026-01-01, Pose(POINT(10 21),1.570796)@2026-01-02] */
```

Una pose nombra un marco, por lo que ella misma es un valor en el marco del cuerpo: aplicar una pose a otra compone las dos relaciones entre marcos. Escribiendo P_{WV} para la pose de un vehículo en el mundo y P_{VS} para la pose de un sensor en el vehículo, la pose del sensor en el mundo es $P_{WS} = P_{WV} \circ P_{VS}$. Esta es la operación que una cadena de poses pliega sobre sus eslabones, de modo que una cadena de dos eslabones se compone en lo que esto devuelve.

```
-- Un sensor una unidad delante de un vehículo girado un cuarto de vuelta
-- queda una unidad al norte de él, y lleva la orientación del vehículo
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)',
    pose(ST_Point(0,0), pi()/2)), 6));
-- Pose(POINT(0 1),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)',
    pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(10 6),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0 0), 1, 0, 0, 0)',
    pose 'Pose(Point(0 0 5), 1, 0, 0, 0)', 6));
-- Pose(POINT Z (1 0 5),1,0,0,0)
```

13.18.6. E/S JSON temporal

- Convierte una pose temporal hacia o desde una envoltura JSON TemporalGeoPose. Cada carga útil por instante es un documento GeoPose de clase Basic estrictamente válido según OGC más un miembro `validTime`; la envoltura registra la clase de conformidad, la interpolación y (para secuencias y conjuntos de secuencias) la inclusión de los límites del período. Un consumidor GeoPose ajeno a MEOS puede iterar sobre `instants[]` (o cada `sequences[].instants[]`) y consumir cada elemento como un documento GeoPose estático.

`asGeoPose(tpose, conformance int4=0, maxdecimaldigits int4=-1) → text`

`tposeFromGeoPose(text) → tpose`

La forma de la envoltura es

```
{
  "type":          "TemporalGeoPose",
  "version":       "1.0",
  "conformance":   "Basic-Quaternion" | "Basic-YPR",
  "interpolation": "None" | "Discrete" | "Step" | "Linear",
  "instants":      [...]              // for TInstant + TSequence
  "lower_inc":     true|false,         // for TSequence only
  "upper_inc":     true|false,         // for TSequence only
  "sequences":     [{...}, ...]       // for TSequenceSet only
}
```

```
SELECT asGeoPose(tpose ' [Pose(Point(8 47), 0)@2026-01-01,
                          Pose(Point(9 48), 0.5)@2026-01-02]',
    1, 4);
-- {"type":"TemporalGeoPose","version":"1.0","conformance":"Basic-YPR",
--  "interpolation":"Linear","lower_inc":true,"upper_inc":true,
--  "instants":[
--    {"position":{"lat":47,"lon":8,"h":0},
--     "angles":{"yaw":0,"pitch":0,"roll":0},
--     "validTime":"2026-01-01"},
--    {"position":{"lat":48,"lon":9,"h":0},
--     "angles":{"yaw":28.65,"pitch":0,"roll":0},
--     "validTime":"2026-01-02"}]}
```

- Escribe una pose temporal como un Stream de OGC GeoPose

`asGeoPoseStream(tpose, maxdecimaldigits int4=-1) → text`

Un stream lleva los marcos de una Serie Irregular sin declarar cuántas poses hay ni cuándo terminan, ya que pueden llegar más. El estándar lo escribe como un header, que contiene el modelo de transición y el marco exterior, y un arreglo

`streamElements` con un elemento por pose. El marco exterior es el plano tangente local Este-Norte-Arriba en la posición de la primera pose, de modo que la cabecera y cada elemento hablan de un mismo punto de tangencia.

La biblioteca C también escribe los dos documentos por partes, mediante `tpose_as_geopose_stream_header` y `tpose_as_geopose_stream_element`, para un productor que emite poses a medida que llegan. Una consulta devuelve un valor que ya posee entero, que es lo que esta función escribe.

```
SELECT asGeoPoseStream(tpose 'Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01', 6);
/* {"header":{"transitionModel":{"authority":"/geopose/1.0","id":"none",
  "parameters":"interpolation=None"},"outerFrame":{"authority":"/geopose/1.0",
  "id":"LTP-ENU","parameters":"longitude=0&latitude=0&height=0&crs=EPSG:4979"}},
  "streamElements":[{"streamElement":{"frame":{"authority":"/geopose/1.0",
  "id":"RotateTranslate","parameters":"translation=[0, 0, 0]&rotation=[1, 0, 0, 0]"},
  "validTime":1767254400000}}]} */
```

Capítulo 14

Cadenas de poses

El tipo `posechain` representa un cuerpo cuyas partes se sitúan unas respecto de otras: una cámara sobre un cardán sobre un vehículo, o un brazo cuyo antebrazo se sitúa respecto del brazo. Una cadena de poses es una lista ordenada de al menos una `pose`. El primer eslabón se expresa en el marco exterior de la cadena, y cada eslabón posterior se expresa en el marco que define el eslabón anterior, de modo que un eslabón indica dónde se sitúa una parte respecto de la parte que la sostiene y no respecto del mundo.

El estándar **OGC GeoPose** denomina a esta estructura una cadena de transformaciones de marcos, donde cada transformación es un par de marcos de referencia en el que el marco exterior es el dominio y el marco interior es el rango. El estándar establece que un marco interior no puede ser un marco topocéntrico, por lo que una cadena lleva exactamente un marco que nombra un lugar sobre la tierra, en su exterior. De ello se derivan dos propiedades, ambas verificadas: toda la cadena tiene un solo SRID y todos los eslabones tienen la misma dimensión. Solo el eslabón exterior puede ser geodésico y solo el eslabón exterior puede llevar un SRID.

La composición de todos los eslabones da la pose del marco más interior en el marco exterior de la cadena, que es lo que devuelve la conversión al tipo `pose`. Esa única conversión otorga a una cadena toda la superficie definida para las poses, de modo que el tipo en sí permanece reducido.

Cuando el marco exterior es geográfico, la posición del eslabón exterior está en grados mientras que la traslación de un eslabón posterior está en metros a lo largo de los ejes locales Este-Norte-Arriba en su marco padre. La composición realiza esa suma en coordenadas geocéntricas y lee de vuelta la posición como geográfica, de modo que una cadena sobre un marco geográfico se compone sobre el elipsoide y no en el plano, y la orientación se vuelve a expresar respecto de la base Este-Norte-Arriba en la posición que alcanza la composición.

14.1. Cadenas de poses estáticas

14.1.1. Entrada y salida

- Devuelve la representación en texto conocido (WKT) o en texto conocido extendido (EWKT)

```
asText({posechain,posechain[]}) → {text,text[]}
```

```
asEWKT({posechain,posechain[]}) → {text,text[]}
```

El eslabón exterior se escribe como una pose que lleva el marco de la cadena, y cada eslabón posterior se escribe sin él.

```
SELECT asText(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),1))');
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),1))
SELECT asText(ARRAY[posechain 'PoseChain(Pose(Point(0 0),0))',
'PoseChain(Pose(Point(1 1),2))']);
-- {"PoseChain(Pose(POINT(0 0),0))","PoseChain(Pose(POINT(1 1),2))"}
SELECT asEWKT(posechain 'SRID=3812;PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),1))');
-- SRID=3812;PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),1))
```

- Devuelve la representación en binario conocido (WKB), en binario conocido extendido (EWKB), en binario conocido hexadecimal (HexWKB) o en binario conocido extendido hexadecimal (HexEWKB)

`asBinary(posechain, endian text='') → bytea`

`asEWKB(posechain, endian text='') → bytea`

`asHexWKB(posechain, endian text='') → text`

`asHexEWKB(posechain, endian text='') → text`

La cadena escribe sus indicadores y su único SRID una sola vez, después el número de eslabones y después los valores de cada eslabón en orden. El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(posechain 'PoseChain(Pose(Point(1 2),1))');
-- \x010101000000000000000000f03f00000000000004000000000000f03f
SELECT asHexWKB(posechain 'PoseChain(Pose(Point(1 2),1))');
-- 010101000000000000000000F03F00000000000004000000000000F03F
SELECT asHexEWKB(posechain 'SRID=3812;PoseChain(Pose(Point(1 2),1))');
-- 0141E40E0000010000000000000000F03F00000000000004000000000000F03F
```

- Entrada desde la representación en texto conocido (WKT) o en texto conocido extendido (EWKT)

`posechainFromText(text) → posechain`

`posechainFromEWKT(text) → posechain`

```
SELECT asEWKT(posechainFromText('PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),1))'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),1))
SELECT asEWKT(posechainFromEWKT('SRID=3812;PoseChain(Pose(Point(0 0),0))'));
-- SRID=3812;PoseChain(Pose(POINT(0 0),0))
```

- Entrada desde la representación en binario conocido (WKB), en binario conocido extendido (EWKB) o en binario conocido extendido hexadecimal (HexEWKB)

`posechainFromBinary(bytea) → posechain`

`posechainFromEWKB(bytea) → posechain`

`posechainFromHexEWKB(text) → posechain`

```
SELECT posechainFromBinary(asBinary(posechain 'PoseChain(Pose(Point(1 2),0.5))')) =
posechain 'PoseChain(Pose(Point(1 2),0.5))';
-- t
SELECT posechainFromHexEWKB(asHexEWKB(
posechain 'SRID=3812;PoseChain(Pose(Point(1 2),0.5))')) =
posechain 'SRID=3812;PoseChain(Pose(Point(1 2),0.5))';
-- t
```

14.1.2. Constructores

- Construye una cadena de poses a partir de un arreglo de poses ordenadas desde el marco más exterior hacia dentro

`posechain(pose[]) → posechain`

```
SELECT asEWKT(posechain(ARRAY[pose 'Pose(Point(0 0),0)', pose 'Pose(Point(1 0),0)']));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

- Devuelve una cadena de poses con una pose añadida como su eslabón más interior

`appendPose(posechain, pose) → posechain`

La pose se lee en el marco que define el último eslabón de la cadena, por lo que no lleva ni un SRID ni el marcador geodésico.

```
SELECT asEWKT(appendPose(posechain 'PoseChain(Pose(Point(0 0),0))',
pose 'Pose(Point(1 0),0)'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

14.1.3. Conversión de tipos

- Convierte una pose en una cadena de poses de un solo eslabón

`posechain(pose) → posechain`

```
SELECT asEWKT(posechain(pose 'SRID=3812;Pose(Point(1 2),0.5)'));
-- SRID=3812;PoseChain(Pose(POINT(1 2),0.5))
```

- Devuelve la pose de un marco que define la cadena, leída en el marco exterior de la cadena

`pose(posechain) → pose`

`pose(posechain, integer) → pose`

Con un argumento se compone toda la cadena, lo que da la pose de su marco más interior. Con dos se componen los primeros `n` eslabones, lo que da la pose del marco que define el eslabón `n`-ésimo, es decir, dónde se sitúa esa articulación de la cadena.

```
SELECT asEWKT(pose(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),Pose(Point(1 0),0))'));
-- Pose(POINT(2 0),0)
SELECT asEWKT(pose(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),Pose(Point(1 0),0))', 2));
-- Pose(POINT(1 0),0)
```

- Convierte una cadena de poses en el punto geométrico de su marco más interior

`point(posechain) → geometry`

```
SELECT ST_AsText(point(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- POINT(1 0)
```

- Convierte una cadena de poses en una caja espaciotemporal

`stbox(posechain) → stbox`

La caja cubre la posición compuesta de cada prefijo de la cadena y no solo la del conjunto: cada articulación es un lugar, y una ventana de consulta que alcanza el codo pero no la mano alcanza igualmente la cadena.

```
SELECT stbox(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),Pose(Point(1 0),0))');
-- STBOX X((0,0),(2,0))
```

14.1.4. Accesores

- Devuelve el número de eslabones de una cadena de poses

`numPoses(posechain) → integer`

```
SELECT numPoses(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),Pose(Point(2 0),0))');
-- 3
```

- Devuelve los eslabones de una cadena de poses

`startPose(posechain) → pose`

`endPose(posechain) → pose`

`poseN(posechain, integer) → pose`

`poses(posechain) → pose[]`

Un eslabón se devuelve tal como está almacenado, es decir, en el marco que define el eslabón anterior. Dónde se sitúa ese marco en el marco exterior de la cadena es lo que responde `pose(posechain, integer)`. La función `poseN` devuelve `NULL` si la cadena tiene menos eslabones que el número dado.

```

SELECT asEWKT(startPose(posechain
  'SRID=3812;PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- SRID=3812;Pose(Point(0 0),0)
SELECT asEWKT(endPose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- Pose(Point(1 0),0)
SELECT asEWKT(poseN(posechain
  'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))', 2));
-- Pose(Point(1 0),0)
SELECT asEWKT(poses(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- {"Pose(Point(0 0),0)","Pose(Point(1 0),0)"}

```

14.1.5. Transformaciones

- Redondea los valores de los eslabones de una cadena de poses a un número de decimales

`round(posechain, integer=0) → posechain`

```

SELECT asEWKT(round(posechain
  'PoseChain(Pose(Point(1.123456789 2.123456789),0.123456789))', 3));
-- PoseChain(Pose(Point(1.123 2.123),0.123))

```

14.1.6. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial del marco exterior de una cadena de poses

`SRID(posechain) → integer`

`setSRID(posechain, integer) → posechain`

```

SELECT SRID(posechain 'SRID=3812;PoseChain(Pose(Point(1 2),0))');
-- 3812
SELECT asEWKT(setSRID(posechain 'PoseChain(Pose(Point(1 2),0))', 3812));
-- SRID=3812;PoseChain(Pose(Point(1 2),0))

```

- Transforma a un identificador de referencia espacial

`transform(posechain, integer) → posechain`

`transformPipeline(posechain, pipeline text, to_srid integer, is_forward bool=true) → posechain`

Solo el eslabón exterior nombra un marco, por lo que solo se transforma el eslabón exterior: cada uno de los demás eslabones es una transformación rígida leída en los ejes de su marco padre, y un cambio del marco exterior deja esos ejes como estaban. Se genera un error cuando la cadena de entrada tiene un SRID desconocido (representado por 0).

Para una cadena tridimensional sobre un marco geográfico, la orientación del eslabón exterior se vuelve a expresar en la base del marco de destino en su punto, exactamente como ocurre con una pose. Para una cadena bidimensional el ángulo es intrínseco a la proyección de origen y se transmite sin cambios.

La función `transformPipeline` especifica la transformación con una tubería de transformación de coordenadas definida representada con el siguiente formato de cadena:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID de la cadena de entrada se ignora, y el SRID de la cadena de salida es el que indica el parámetro `to_srid`. Según indica el último parámetro, la tubería se ejecuta de forma predeterminada en sentido directo; si se establece el parámetro en falso, la tubería se ejecuta en sentido inverso.

```

SELECT asEWKT(round(transform(posechain
  'SRID=4326;PoseChain(Pose(Point(4.35 50.85),1),Pose(Point(10 0),0))', 3812), 6));
-- SRID=3812;PoseChain(Pose(Point(648679.018035 671067.055638),1),Pose(Point(10 0),0))
SELECT asEWKT(round(transformPipeline(posechain
  'SRID=4326;PoseChain(Pose(Point(4.35 50.85),1))',
  'urn:ogc:def:coordinateOperation:EPSG::16031', 4326), 6));
-- SRID=4326;PoseChain(Pose(Point(595032.294558 5634012.833372),1))

```


14.1.7. Comparaciones

- Operadores de comparación tradicionales

```
{posechain} {=,<>,<,>,<=,>} {posechain} → boolean
```

El orden compara la dimensión, después el SRID, después los eslabones en orden y por último el número de eslabones, de modo que una cadena precede a toda cadena que la extiende.

```
SELECT posechain 'PoseChain(Pose(Point(0 0),0))' =
posechain 'PoseChain(Pose(Point(0 0),0))';
-- t
SELECT posechain 'PoseChain(Pose(Point(0 0),0))' <
posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))';
-- t
```

- Devuelve verdadero si dos cadenas de poses son iguales hasta la tolerancia de la comparación de valores de punto flotante

```
posechain ~= posechain → boolean
```

```
SELECT posechain 'PoseChain(Pose(Point(0 0),0))' ~=
posechain 'PoseChain(Pose(Point(0 0),0))';
-- t
```

14.2. Cadenas de poses temporales

El tipo de cadena de poses temporal `tposechain` representa la evolución en el tiempo de una cadena de marcos anidados, es decir, de un objeto articulado cuyas articulaciones se mueven unas respecto de otras. Como todos los tipos temporales, tiene tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores `tposechain` en estos subtipos.

```
SELECT asEWKT(tposechain 'PoseChain(Pose(Point(1 1), 0.5), Pose(Point(2 0), 0))@2001-01-01' ←
);
-- PoseChain(Pose(Point(1 1),0.5),Pose(Point(2 0),0))@2001-01-01
SELECT asEWKT(tposechain '{PoseChain(Pose(Point(1 1), 0.3))@2001-01-01,
PoseChain(Pose(Point(1 1), 0.5))@2001-01-02}');
-- {PoseChain(Pose(Point(1 1),0.3))@2001-01-01, PoseChain(Pose(Point(1 1),0.5))@2001-01-02}
SELECT asEWKT(tposechain '[PoseChain(Pose(Point(1 1), 0.2))@2001-01-01,
PoseChain(Pose(Point(1 1), 0.5))@2001-01-03]');
-- [PoseChain(Pose(Point(1 1),0.2))@2001-01-01, PoseChain(Pose(Point(1 1),0.5))@2001-01-03]
SELECT asEWKT(tposechain '{[PoseChain(Pose(Point(1 1), 1))@2001-01-01,
PoseChain(Pose(Point(2 2), 1))@2001-01-02],
[PoseChain(Pose(Point(2 2), 2))@2001-01-04]}');
/* {[PoseChain(Pose(Point(1 1),1))@2001-01-01, PoseChain(Pose(Point(2 2),1))@2001-01-02],
[PoseChain(Pose(Point(2 2),2))@2001-01-04]} */
```

Todos los valores de una cadena de poses temporal tienen el mismo número de eslabones. Una cadena que gana una articulación es una estructura distinta y no un valor posterior de la misma, y ese número constante es también lo que hace que la interpolación siguiente esté bien definida. En caso contrario se genera un error.

```
SELECT tposechain '[PoseChain(Pose(Point(0 0), 0))@2001-01-01,
PoseChain(Pose(Point(0 0), 0), Pose(Point(1 0), 0))@2001-01-02]';
-- ERROR: Operation on pose chains of 1 and 2 links
```

Una cadena de poses temporal se interpola eslabón por eslabón, cada eslabón como lo hace una pose, es decir, linealmente en la posición y a lo largo del arco más corto en la rotación. En el ejemplo siguiente un brazo de dos eslabones gira un cuarto de vuelta en el hombro mientras el codo permanece donde está: a la mitad, el hombro ha girado un octavo de vuelta y el codo no ha cambiado.

```
SELECT asEWKT(round(valueAtTimestamp(tposechain
  '[PoseChain(Pose(Point(0 0), 0), Pose(Point(10 0), 0))@2001-01-01,
    PoseChain(Pose(Point(0 0), 1.5707963267948966), Pose(Point(10 0), 0))@2001-01-03]',
  timestampz '2001-01-02'), 6));
-- PoseChain(Pose(POINT(0 0),0.785398),Pose(POINT(10 0),0))
```

Como para los demás tipos espaciotemporales, el tipo de cadena de poses temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo, la dimensionalidad y/o el identificador de referencia espacial (SRID).

```
SELECT asEWKT(tposechain(Sequence,5676) 'SRID=5676;[PoseChain(Pose(Point(1 1), 0.2))@2001-01-01,
  PoseChain(Pose(Point(1 1), 0.5))@2001-01-03]');
/* SRID=5676;[PoseChain(Pose(POINT(1 1),0.2))@2001-01-01,
  PoseChain(Pose(POINT(1 1),0.5))@2001-01-03] */
SELECT tposechain(Sequence) 'PoseChain(Pose(Point(1 1), 0.2))@2001-01-01';
/* ERROR: Temporal subtype of the tposechain value (Instant) does not match
  column subtype (Sequence) */
```

A continuación damos las funciones y los operadores para las cadenas de poses temporales. La mayoría de las funciones y los operadores para los tipos temporales y espaciotemporales descritos en los capítulos anteriores pueden aplicarse a las cadenas de poses temporales. Por lo tanto, en las firmas de las funciones, el tipo `posechain` puede usarse siempre que se especifiquen los tipos `base` y `spatial`, y el tipo `tposechain` puede usarse siempre que se especifiquen los tipos `ttype` y `tspatial`. Para evitar redundancias, solo presentamos algunos ejemplos de estas funciones y operadores y remitimos a los capítulos anteriores para explicaciones detalladas sobre ellos.

Una cadena de poses no tiene función de distancia, ya que la distancia entre dos configuraciones articuladas no está definida por el estándar que sigue el tipo. Por ello el tipo de cadena de poses temporal no tiene ni las relaciones espaciales siempre y alguna vez ni las temporales, todas ellas definidas en términos de una distancia.

14.2.1. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o de texto conocido extendido (EWKT)

```
asText({tposechain,tposechain[]}) → {text,text[]}
asEWKT({tposechain,tposechain[]}) → {text,text[]}
```

```
SELECT asText(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01');
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
```

- Devuelve una cadena de poses temporal a partir de su representación textual o binaria

```
tposechainFromText(text) → tposechain
tposechainFromBinary(bytea) → tposechain
tposechainFromHexEWKB(text) → tposechain
```

```
SELECT asEWKT(tposechainFromText('SRID=5676;PoseChain(Pose(Point(1 1),0.5))@2001-01-01'));
-- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechainFromBinary(asBinary(tposechain
  'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechainFromHexEWKB(asHexEWKB(tposechain
  'SRID=5676;PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
-- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
```

- Devuelve la representación JSON de Moving Features

```
asMFJSON(tposechain) → text
```

Una cadena se escribe como el arreglo de sus eslabones, en el orden que indica en el marco de quién se lee cada uno de ellos.

```
SELECT asMFJSON(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01');
/* {"type":"MovingPoseChain","values":[{"position":{"lat":1,"lon":1},"yaw":0.5}],
   "datetimes":["2001-01-01T00:00:00-08"],"interpolation":"None"} */
```

14.2.2. Constructores

- Construye una cadena de poses temporal a partir de una cadena de poses y un valor de tiempo

`tposechain(posechain, timestamptz) → tposechain`

`tposechain(posechain, tstzset) → tposechain`

`tposechain(posechain, tstzspan, text DEFAULT 'linear') → tposechain`

`tposechain(posechain, tstzspanset, text DEFAULT 'linear') → tposechain`

```
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  timestamptz '2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzset '{2001-01-01, 2001-01-02}'));
-- {PoseChain(Pose(POINT(1 1),0.5))@2001-01-01, PoseChain(Pose(POINT(1 1),0.5))@2001-01-02}
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzspan '[2001-01-01, 2001-01-02]'));
-- [PoseChain(Pose(POINT(1 1),0.5))@2001-01-01, PoseChain(Pose(POINT(1 1),0.5))@2001-01-02]
```

14.2.3. Conversiones

- Convierte una cadena de poses temporal en una pose temporal

`tpose(tposechain) → tpose`

Cada instante contiene la pose a la que compone toda la cadena, que es donde se sitúa su marco más interno en el marco de la cadena. El resultado es una función escalonada: componer una cadena multiplica las rotaciones de sus eslabones, por lo que la pose de la cadena interpolada entre dos instantes no es la pose interpolada entre las dos poses compuestas, y un resultado lineal respondería una pose que la cadena nunca tiene. Una cadena de un solo eslabón se compone a sí misma, y allí la interpolación del argumento se conserva sin cambios.

```
SELECT asEWKT(round(tpose(tposechain
  'PoseChain(Pose(Point(0 0), 0), Pose(Point(10 0), 0))@2001-01-01', 6));
-- Pose(POINT(10 0),0)@2001-01-01
SELECT asEWKT(round((tposechain
  '[PoseChain(Pose(Point(0 0), 0), Pose(Point(10 0), 0))@2001-01-01,
  PoseChain(Pose(Point(0 0), 1.5707963267948966), Pose(Point(10 0), 0))@2001-01-03]':: tpose, 6));
/* Interp=Step; [Pose(POINT(10 0),0)@2001-01-01,
  Pose(POINT(0 10),1.570796)@2001-01-03] */
```

14.2.4. Funciones de acceso

- Devuelve el número de eslabones que tiene todo valor de una cadena de poses temporal

`numPoses(tposechain) → integer`

El número es el mismo en todo instante, por lo que un instante responde por todo el valor.

```
SELECT numPoses(tposechain
  'PoseChain(Pose(Point(0 0), 0), Pose(Point(10 0), 0))@2001-01-01');
-- 2
```

- Devuelve el valor de un instante temporal, el conjunto de valores o el lapso de tiempo

`getValue(tposechain) → posechain`

`getValues(tposechain) → posechainset`

`timeSpan(tposechain) → tstzspan`

```
SELECT asEWKT(getValue(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))
SELECT asEWKT(getValues(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}'));
-- {"PoseChain(Pose(POINT(1 1),0.5))", "PoseChain(Pose(POINT(2 2),0.5))"}
SELECT timeSpan(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
PoseChain(Pose(Point(2 2), 0.5))@2001-01-02]');
-- [2001-01-01, 2001-01-02]
```

- Devuelve la caja espaciotemporal de una cadena de poses temporal

`stbox(tposechain) → stbox`

La caja abarca la posición compuesta de cada prefijo de la cadena, es decir, de cada una de sus articulaciones, y no solo la posición que almacenan sus eslabones.

```
SELECT round(stbox(tposechain
'[PoseChain(Pose(Point(0 0), 0), Pose(Point(10 0), 0))@2001-01-01,
PoseChain(Pose(Point(1 5), 0), Pose(Point(10 0), 0))@2001-01-02]'), 6);
-- STBOX XT((0,0),(11,5)), [2001-01-01, 2001-01-02])
```

14.2.5. Funciones de transformación

- Redondea las coordenadas a un número de decimales, o desplaza el tiempo

`round(tposechain, integer DEFAULT 0) → tposechain`

`shiftTime(tposechain, interval) → tposechain`

```
SELECT asEWKT(round(tposechain
'PoseChain(Pose(Point(1.123456789 2.123456789), 0.5))@2001-01-01', 3));
-- PoseChain(Pose(POINT(1.123 2.123),0.5))@2001-01-01
SELECT asEWKT(shiftTime(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01',
interval '1 day'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-02
```

14.2.6. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial del marco exterior de una cadena de poses temporal

`SRID(tposechain) → integer`

`setSRID(tposechain, integer) → tposechain`

```
SELECT SRID(tposechain 'SRID=3812;{PoseChain(Pose(Point(1 2),0),
Pose(Point(3 0),0.5))@2001-01-01, PoseChain(Pose(Point(2 3),0),
Pose(Point(3 0),0.5))@2001-01-02}');
-- 3812
SELECT asEWKT(setSRID(tposechain '{PoseChain(Pose(Point(1 2),0))@2001-01-01,
PoseChain(Pose(Point(2 3),0))@2001-01-02}', 3812));
/* SRID=3812;{PoseChain(Pose(POINT(1 2),0))@2001-01-01,
PoseChain(Pose(POINT(2 3),0))@2001-01-02} */
```

- Transforma a un identificador de referencia espacial

```
transform(tposechain, integer) → tposechain
```

```
transformPipeline(tposechain, pipeline text, to_srid integer, is_forward bool=true) → tposechain
```

La cadena de cada instante se transforma como se transforma la cadena estática correspondiente, es decir, solo se transforma el eslabón exterior, tal como se describe para **transform**.

```
SELECT asEWKT(round(transform(tposechain 'SRID=4326;PoseChain(
  Pose(Point(4.35 50.85),1),Pose(Point(10 0),0))@2001-01-01', 3812), 6));
/* SRID=3812;PoseChain(Pose(Point(648679.018035 671067.055638),1),
  Pose(Point(10 0),0))@2001-01-01 */
SELECT asEWKT(round(transformPipeline(tposechain
  'SRID=4326;PoseChain(Pose(Point(4.35 50.85),1))@2001-01-01',
  'urn:ogc:def:coordinateOperation:EPSG::16031', 4326), 6));
-- SRID=4326;PoseChain(Pose(Point(595032.294558 5634012.833372),1))@2001-01-01
```

14.2.7. Cadena compuesta OGC GeoPose

- Escribe o lee un documento de cadena compuesta OGC GeoPose

```
asGeoPose(tposechain, maxdecimaldigits integer=-1) → text
```

```
tposechainFromGeoPose(text) → tposechain
```

Un documento de cadena es un marco exterior y una secuencia de transformaciones que alcanza un marco interior final, que es lo que contiene una cadena de poses. El documento nombra el marco LTP-ENU tangente en la posición del eslabón exterior y una transformación por eslabón: la primera lleva ese marco tangente al marco propio del eslabón exterior, y cada una posterior es el eslabón tal como se almacena, leído en los ejes de su padre.

El documento lleva un único tiempo de validez, por lo que se escribe a partir de un solo instante; **atTime** obtiene uno de un valor más largo. Su cadena de marcos contiene al menos dos marcos, por lo que una cadena de un solo eslabón no tiene documento de cadena.

La clase de cadena nombra sus dos marcos `/Extrinsic/LTP-ENU` e `/Intrinsic/Translate-Rotate`, mientras que las clases `Advanced`, `Series` y `Stream` nombran esos mismos dos marcos `LTP-ENU` y `RotateTranslate`. Un documento se escribe con el par que usa su propia clase, y cualquiera de los dos pares se acepta al leer.

```
SELECT asGeoPose(tposechain 'SRID=4326;PoseChain(
  GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00', 6);
/* {"validTime":1619588170083,"outerFrame":{"authority":"/geopose/1.0",
  "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
  height=11&crs=EPSG:4979"},"frameChain":[{"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
  rotation=[1, 0, -1.38778e-17, 0]"},"{"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
  rotation=[1, 0, 0, 0]"}]} */
```

14.2.8. Grafo compuesto OGC GeoPose

- Escribe un documento de grafo compuesto OGC GeoPose

```
asGeoPose(tposechain[], maxdecimaldigits integer=-1) → text
```

Un documento de grafo es un conjunto de marcos y las transformaciones entre ellos, que es lo que contienen varias cadenas de poses que comparten su marco más exterior. La lista de marcos nombra el marco LTP-ENU tangente en ese eslabón más exterior seguido de los eslabones de cada cadena, y la lista de transformaciones nombra el padre y el hijo de cada arista por su posición en esa lista. Las aristas no llevan transformación propia; esta reside en los marcos que nombran.

El documento lleva un único tiempo de validez, por lo que se escribe a partir de cadenas leídas en uno y el mismo instante; **atTime** las obtiene de valores más largos. Su lista de marcos contiene al menos dos marcos, que un solo eslabón de una sola cadena ya proporciona junto al marco tangente.

```

SELECT asGeoPose (ARRAY[
  tposechain 'SRID=4326;PoseChain(
    GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
    Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00',
  tposechain 'SRID=4326;PoseChain(
    GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
    Pose(Point Z(0 3 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00'], 6);
/* {"validTime":1619588170083,"frameList":[{"authority":"/geopose/1.0",
  "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
  height=11&crs=EPSG:4979"}, {"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
  rotation=[1, 0, -1.38778e-17, 0]"}, {"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
  rotation=[1, 0, 0, 0]"}, {"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
  rotation=[1, 0, -1.38778e-17, 0]"}, {"authority":"/geopose/1.0",
  "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 3, 0]&
  rotation=[1, 0, 0, 0]"}],"transformList":[{"link":[0,1]},{ "link":[1,2]},{
  "link":[0,3]},{ "link":[3,4]}}] */

```

14.2.9. Funciones de restricción

- Restringe una cadena de poses temporal a un tiempo o a un valor

atTime(tposechain, time) → tposechain

atValue(tposechain, posechain) → tposechain

```

SELECT asEWKT(atTime(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-03]', timestampz '2001-01-02'));
-- PoseChain(Pose(Point(1.5 1.5), 0.5))@2001-01-02
SELECT asEWKT(atValue(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))');
-- {PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}

```

14.2.10. Funciones y operadores de comparación

- Compara dos cadenas de poses temporales, o una cadena de poses temporal y una cadena de poses

tposechain operator tposechain → boolean

eEq(tposechain, posechain) → boolean

```

SELECT tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01' =
  tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01';
-- t
SELECT eEq(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))');
-- t

```

14.2.11. Operadores topológicos y de posición

- Compara la caja delimitadora de una cadena de poses temporal con otra caja delimitadora

tposechain op {stbox, tposechain} → boolean

```
SELECT tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,  
    PoseChain(Pose(Point(2 2), 0.5))@2001-01-02]' && stbox 'STBOX X((1,1), (2,2))';  
-- t  
SELECT tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,  
    PoseChain(Pose(Point(2 2), 0.5))@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';  
-- t
```

Capítulo 15

Geometrías rígidas temporales

Una geometría rígida temporal (`trgeometry`) es una geometría de referencia 2D estática, normalmente un polígono que describe la forma de un cuerpo en movimiento— emparejada con una pose temporal (`tpose`) que describe cómo esa geometría se traslada y rota a lo largo del tiempo. La geometría materializada en el instante `t` es la geometría de referencia rotada y trasladada de acuerdo con la pose interpolada en `t`.

Esta es la representación natural para objetos en movimiento cuya *forma* importa, no solo su posición. El ejemplo motivador es el tráfico marítimo AIS, donde el casco de cada barco (un pentágono derivado de los desplazamientos de antena A, B, C, D) sigue una trayectoria de pose definida por informes de latitud/longitud y rumbo. La geometría materializada en cualquier instante es el contorno real del barco en ese momento.

El tipo `trgeometry` se construye sobre Capítulo 13. La mayoría de los operadores de acceso, comparación, topológicos, de posición y de indexación tienen la misma forma que las funciones `tpose` correspondientes; este capítulo cubre la superficie específica de `trgeometry`: la materialización de pose a forma, el área recorrida, la distancia materializada y las restricciones espaciales sobre la trayectoria del centroide.

15.1. Entrada y salida

Un literal `trgeometry` empareja una geometría de referencia con una pose temporal. La geometría de referencia va primero, separada de la pose por un punto y coma; la pose sigue la sintaxis estándar de `tpose` (instante, secuencia o conjunto de secuencias).

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001-01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(10 0), 1.5)@2001-01-02]';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{[Pose(Point(0 0), 0.0)@2001-01-01, Pose( ←
  Point(5 0), 0.0)@2001-01-02], [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(2 0), 0.0) ←
  @2001-01-05]}';
```

La geometría de referencia puede ser cualquier polígono 2D o superficie poliédrica. La interpolación de la pose es lineal en la traslación y de camino angular más corto en la rotación. Los SRID se heredan de la geometría de referencia y de la pose; deben coincidir.

Se puede especificar un SRID para una `trgeometry` ya sea al comienzo del literal o antes de la geometría de referencia, como se muestra a continuación.

```
SELECT trgeometry 'SRID=25832;Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001 ←
  -01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));SRID=25832;Pose(Point(0 0), 0.5)@2001 ←
  -01-01';
```


La interpolación escalonada también se puede especificar de la misma manera que para otros tipos temporales:

```
SELECT trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
```

Se admiten los formatos de transmisión estándar.

- Devuelve la representación de texto conocido (Well-Known Text o WKT)

```
asText({trgeometry,trgeometry[]} [, maxdecimaldigits int=15]) → text
```

```
SELECT asText(trgeometry 'Polygon((1 1,2 2,3 1,1 1));Pose(Point(1.123456789 1.123456789), 0.5)@2000-01-01', 6);
-- POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1.123457 1.123457),0.5)@2000-01-01
```

- Devuelve la representación extendida de texto conocido (Extended Well-Known Text o EWKT), prefijada con el SRID

```
asEWKT({trgeometry,trgeometry[]} [, maxdecimaldigits int=15]) → text
```

```
SELECT asEWKT(trgeometry 'SRID=25832;Polygon((1 1,2 2,3 1,1 1));Pose(Point(1 1),0.5)@2000-01-01');
-- SRID=25832;POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1 1),0.5)@2000-01-01
```

- Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)

```
asMFJSON(trgeometry [, options int=0 [, flags int=0 [, maxdecimaldigits int=15]]) → text
```

```
SELECT asMFJSON(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0),0)@2001-01-01');
;
```

- Devuelve la representación extendida binaria conocida (Extended Well-Known Binary o EWKB)

```
asEWKB(trgeometry [, endian text]) → bytea
```

```
asHexWKB(trgeometry [, endian text]) → text
```

```
asHexEWKB(trgeometry [, endian text]) → text
```

Las funciones complementarias `asHexWKB` y `asHexEWKB` devuelven el texto codificado en hexadecimal de, respectivamente, la representación simple y la que incluye el SRID; ambas coinciden solo cuando el valor no tiene SRID, como en el ejemplo siguiente.

```
SELECT length(asEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(1 1), 0.5)@2001-01-01'));
-- 130
-- The bytes round-trip through the matching input function.
SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(trgeometryFromHexEWKB(asHexWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
```

- Cada formato de salida tiene un analizador complementario:

```
trgeometryFromText(text) → trgeometry
```

```
trgeometryFromEWKT(text) → trgeometry
```

```
trgeometryFromMFJSON(text) → trgeometry
```

```
trgeometryFromEWKB(bytea) → trgeometry
```

```
trgeometryFromHexEWKB(text) → trgeometry
```

```
SELECT asText(trgeometryFromEWKT('SRID=4326;Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0)); ←
  Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
```

15.2. Constructores

Una `trgeometry` se puede construir directamente a partir de sus tipos componentes: una geometría de referencia más una pose temporal:

- Construir una geometría rígida temporal a partir de una geometría de referencia y una pose temporal

`trgeometry(geometry, tpose) → trgeometry`

```
SELECT asText(trgeometry(
  geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
  tpose 'Pose(Point(0 0), 0.0)@2001-01-01');
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(0 0),0)@2001-01-01
```

Ambos argumentos deben compartir el mismo SRID. La geometría de referencia puede ser un polígono o una superficie poliédrica; la pose puede ser de cualquier subtipo temporal.

- Agregar un único instante a una `trgeometry` existente, ampliando la trayectoria de pose subyacente

`appendInstant(trgeometry, trgeometry [, maxdist float [, maxt interval]]) → trgeometry`

```
SELECT appendInstant(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-02');
```

- Agregar una secuencia completa a una `trgeometry` existente

`appendSequence(trgeometry, trgeometry) → trgeometry`

```
SELECT asText(appendSequence(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(20 0), 0.0)@2001-01-03, Pose(Point(30 0), 0.0)@2001-01-04]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-02],
--   [Pose(POINT(20 0),0)@2001-01-03, Pose(POINT(30 0),0)@2001-01-04]}
```

15.3. Conversión de tipos

La `trgeometry` se puede proyectar a sus tipos componentes o a una trayectoria de puntos puros.

- Devuelve la pose temporal subyacente

`tpose(trgeometry) → tpose`

```
SELECT asText(tpose(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.5)@2001-01-02]'));
-- [Pose(POINT(0 0),0)@..., Pose(POINT(10 0),0.5)@...]
```

- Devuelve la trayectoria del centroide (antena) como un punto temporal

`tgeompoint(trgeometry) → tgeompoint`

```
SELECT asText(tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0) ↔
@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- [POINT(0 0)@..., POINT(10 0)@...]
```

15.4. Accesos

Se aplican los accesores estándar de los tipos temporales: cada uno devuelve la geometría materializada en la marca de tiempo solicitada (la forma de referencia rotada y trasladada de acuerdo con la pose interpolada), o un valor de metadatos derivado de la trayectoria de pose.

- Devuelve la geometría materializada al inicio, al final o en una marca de tiempo elegida

`startValue(trgeometry) → geometry`

`endValue(trgeometry) → geometry`

`valueAtTimestamp(trgeometry, timestamptz) → geometry`

```
SELECT asText(startValue(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0) ↔
@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))

SELECT asText(valueAtTimestamp(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  timestamptz '2001-01-01 12:00:00'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0))
```

Tenga en cuenta que la rotación interpola linealmente a lo largo del camino angular más corto; una rotación de 90° entre dos instantes separados por un día devuelve un polígono rotado 45° en el punto medio.

- Devuelve el número de instantes, secuencias o marcas de tiempo distintos

`numInstants(trgeometry) → integer`

`numSequences(trgeometry) → integer`

`numTimestamps(trgeometry) → integer`

```
SELECT numInstants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
SELECT numSequences(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 1
SELECT numTimestamps(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
```

- Devuelve el arreglo de instantes, secuencias o segmentos entre instantes constituyentes

`instants(trgeometry) → trgeometry[]`

`sequences(trgeometry) → trgeometry[]`

`segments(trgeometry) → trgeometry[]`

```
SELECT array_length(
  instants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01,
Pose(Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}'),
  1);
-- 3
```

- Devuelve el conjunto de puntos del centroide (antena) distintos vistos a lo largo de la `trgeometry`

`points(trgeometry) → geomset`

```
SELECT asText(points(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0) ↔
    @2001-01-01, Pose(Point(5 5), 0.0}@2001-01-02, Pose(Point(0 0), 0.0}@2001-01-03}')));
-- {"POINT(0 0)", "POINT(5 5)"}
```

- Devuelve el ángulo yaw, pitch o roll como un flotante temporal, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose

`yaw(trgeometry) → tfloat`

`pitch(trgeometry) → tfloat`

`roll(trgeometry) → tfloat`

Una geometría rígida lleva su orientación en sus poses, de modo que cada una de estas funciones se proyecta sobre la pose temporal y se lo pregunta, exactamente como lo hacen los accesorios homónimos de `tpose`. Una geometría rígida 2D gira por el ángulo almacenado de cada pose y no cabecea ni alabea.

```
SELECT asText(yaw(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0}@2001-01-01, Pose(Point(0 0), 1.5}@2001-01-02]')));
-- [0@2001-01-01, 1.5@2001-01-02]
SELECT asText(pitch(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0}@2001-01-01, Pose(Point(0 0), 1.5}@2001-01-02]')));
-- [0@2001-01-01, 0@2001-01-02]
```

- Devuelve la geometría de referencia estática

`geom(trgeometry) → geometry`

```
SELECT ST_AsText(geom(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.5) ↔
    @2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
```

15.5. Área recorrida

La función `traversedArea` devuelve la región que el cuerpo en movimiento cubre sobre el dominio temporal de la `trgeometry`: las colocaciones que toma, o la unión de ellas cuando se le pide.

- Devuelve el área atravesada por la geometría rígida temporal

`traversedArea(trgeometry, unary_union=false) → geometry`

```
SELECT ST_AsText(traversedArea(
    trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0}@2001-01-01, Pose(Point(10 0), 0.0}@2001-01-02]')));
-- MULTIPOLYGON(((0 0,1 0,1 1,0 1,0 0)),((10 0,11 0,11 1,10 1,10 0)))
SELECT ST_AsText(traversedArea(
    trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0}@2001-01-01, Pose(Point(10 0), 0.0}@2001-01-02]', true));
-- POLYGON((0 0,0 1,11 1,11 0,0 0))
```

El argumento es falso salvo que se indique, y la función responde entonces las colocaciones mismas reunidas en una geometría, un multipolígono para un cuerpo que es un polígono, que es lo que quiere quien llama y hace su propio posprocesamiento. Con `unary_union = true` se disuelven en la región que cubren.

Advertencia de muestreo: la implementación muestrea en los instantes de entrada, de modo que los segmentos de traslación pura son exactos, pero la cinta barrida entre dos instantes donde ocurre una rotación se aproxima mediante la envolvente convexa de los dos polígonos extremos. Sobremuestrear la `tpose` antes de construir la `trgeometry` ajusta la aproximación.

15.6. Funciones espaciales

Estas funciones derivan un valor espacial del cuerpo en movimiento. `centroid` y `bodyPointTrajectory` devuelven puntos temporales que siguen una posición a lo largo del tiempo; `convexHull` devuelve una geometría estática.

- Devuelve el centroide del cuerpo en movimiento como un punto temporal

`centroid(trgeometry) → tgeompoint`

```
SELECT asText(centroid(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]')');
-- Interp=Step;[POINT(0.5 0.5)@2001-01-01, POINT(2.5 0.5)@2001-01-02]
```

- Devuelve la envolvente convexa del área recorrida por el cuerpo en movimiento

`convexHull(trgeometry) → geometry`

```
SELECT ST_GeometryType(convexHull(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]')');
-- ST_Polygon
```

- Devuelve la trayectoria en el marco del mundo de un punto del marco del cuerpo transportado por la geometría rígida en movimiento. El punto se expresa en el marco local de la forma de referencia y la pose variable en el tiempo se le aplica en cada instante.

`bodyPointTrajectory(trgeometry, geometry) → tgeompoint`

```
SELECT asText(bodyPointTrajectory(
  trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(↔
    Point(2 0),0)@2001-01-02]',
  geometry 'Point(0 0)');
-- Interp=Step;[POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-02]
```

15.7. Métricas de movimiento

Estas funciones informan sobre el movimiento de una geometría rígida temporal a lo largo de su trayectoria del centroide.

- Devuelve la longitud total recorrida por el centroide, o su longitud acumulada a lo largo del tiempo

`length(trgeometry) → float`

`cumulativeLength(trgeometry) → tfloat`

```
SELECT length(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01, ↔
  Pose(Point(3 4),0)@2001-01-02]');
-- 5
```

- Devuelve la velocidad del centroide como un flotante temporal

`speed(trgeometry) → tfloat`

```
SELECT speed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- Interp=Step;[0.000115740740741@2001-01-01, 0.000115740740741@2001-01-02]
```

- Devuelve la velocidad angular como un flotante temporal

`angularSpeed(trgeometry) → tfloat`

La forma de una geometría rígida nunca cambia, por lo que su velocidad angular es la de la pose que la lleva, en radianes por unidad de tiempo.

```
SELECT asText(angularSpeed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1)@2001-01-02]'));
-- Interp=Step;[0.000011574074074@2001-01-01, 0.000011574074074@2001-01-02]
```

- Devuelve el centroide ponderado en el tiempo de la trayectoria del centroide como una geometría

twCentroid(trgeometry) → geometry

```
SELECT ST_AsText(twCentroid(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0) ↵
@2001-01-01, Pose(Point(3 4),0)@2001-01-02]'));
-- POINT(1.5 2)
```

15.8. Modificaciones

Se aplican las funciones estándar de modificación de secuencias: round, setInterp, shiftTime, scaleTime, shiftScaleTime, deleteTimestamp, etc.

- Redondear todos los componentes numéricos a un número dado de posiciones decimales

round(trgeometry, integer) → trgeometry

```
SELECT asText(round(
trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(1.123456789 1.123456789), ↵
0.123456)@2001-01-01',
3));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1.123 1.123),0.123)@...
```

- Convertir entre interpolaciones lineal y escalonada

setInterp(trgeometry, text) → trgeometry

```
SELECT asText(setInterp(trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 'linear'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(0 0),0)@2001-01-02],
-- [Pose(POINT(10 0),0)@2001-01-02]}
```

- Desplazar, escalar, o desplazar y escalar el dominio temporal de una trgeometry

shiftTime(trgeometry, interval) → trgeometry

scaleTime(trgeometry, interval) → trgeometry

shiftScaleTime(trgeometry, interval, interval) → trgeometry

```
SELECT asText(shiftTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', interval '1 day' ↵
));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(scaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', interval '2 days ↵
'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(shiftScaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', interval '1 day' ↵
, interval '2 days'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-04]
```

15.9. Restricciones espaciales

Las funciones `atGeometry`, `minusGeometry`, `atStbox` y `minusStbox` restringen una `trgeometry` a (el complemento de) una geometría o una caja espaciotemporal. La semántica refleja la de `tpose`: una geometría rígida temporal está «en» una región en el instante `t` si y solo si su centroide (antena) en `t` se encuentra en esa región.

- Restringir una `trgeometry` a (el complemento de) una geometría

`atGeometry(trgeometry, geometry) → trgeometry`

`minusGeometry(trgeometry, geometry) → trgeometry`

```
SELECT temporalTime(atGeometry(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))');
-- {[2001-01-02, 2001-01-04]}

SELECT temporalTime(minusGeometry(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))');
-- {[2001-01-01, 2001-01-02], (2001-01-04, 2001-01-05]}
```

La restricción contra una geometría vacía devuelve NULL para `atGeometry` y la entrada original para `minusGeometry`.

- Restringir una `trgeometry` a (el complemento de) una caja espaciotemporal

`atStbox(trgeometry, stbox [, border_inc boolean = true]) → trgeometry`

`minusStbox(trgeometry, stbox [, border_inc boolean = true]) → trgeometry`

```
SELECT temporalTime(atStbox(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  stbox 'STBOX X((1, -1), (3, 1))');
-- {[2001-01-02, 2001-01-04]}

SELECT temporalTime(atStbox(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  stbox 'STBOX T([2001-01-02, 2001-01-04])');
-- {[2001-01-02, 2001-01-04]}
```

Si la `STBox` solo tiene un componente temporal, la llamada se reduce a `atTime` estándar; si solo tiene un componente espacial, se conserva el dominio temporal.

- Restringir una `trgeometry` al (complemento de) un span de elevación

`atElevation(trgeometry, floatspan) → trgeometry`

`minusElevation(trgeometry, floatspan) → trgeometry`

La elevación es la de la posición de la pose, no la del cuerpo: un cuerpo se extiende por encima y por debajo de su posición, y el span se evalúa únicamente sobre la posición. La geometría de referencia se conserva en el resultado. La geometría rígida temporal debe tener dimensión Z.

```
SELECT asText(atElevation(
  trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(0 0 4),1,0,0,0)@2001-01-05]',
  floatspan '[1, 2]');
/* POLYGON Z ((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
   {[Pose(POINT Z (0 0 1),1,0,0,0)@2001-01-02,
   Pose(POINT Z (0 0 2),1,0,0,0)@2001-01-03]} */
```

```
SELECT getTime(minusElevation(
  trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(0 0 4),1,0,0,0)@2001-01-05]',
  floatspan '[1, 2]'));
-- {[2001-01-01, 2001-01-02), (2001-01-03, 2001-01-05]}
```

15.10. Operaciones de distancia

La distancia entre una trgeometry y una geometría / pose / caja espaciotemporal / otra trgeometry se calcula sobre la geometría *materializada*: la forma de referencia rotada y trasladada de acuerdo con la pose. La distancia es exacta en cada marca de tiempo compartida; para entradas de secuencia, una bisección recursiva adaptativa emite marcas intermedias donde la trayectoria de distancia se desvía de una línea recta.

- Devuelve la distancia temporal entre dos geometrías rígidas temporales

```
{geometry,trgeometry} <-> {geometry,trgeometry} → tfloat
tDistance({geometry,trgeometry}, {geometry,trgeometry}) → tfloat
```

```
SELECT round(maxValue(tDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.0)@2001-01-01',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-01')), 6);
-- 4

SELECT asText(tDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(2 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- [1@2001-01-01 ..., 9@2001-01-02 ...]
```

El despachador cubre cada combinación de subtipos: TINSTANT × TINSTANT, TINSTANT × TSEQUENCE asimétrica, TSEQUENCE × TSEQUENCE, TSEQUENCE × TSEQUENCESET, TSEQUENCESET × TSEQUENCESET. El núcleo de submuestreo adaptativo subyace a cada combinación continua; las combinaciones de instantes se materializan una vez. Los segmentos de traslación pura convergen en profundidad 0 (dos marcas extremas); los segmentos con mucha rotación emiten hasta 32 marcas por intervalo entre instantes. La cota sobre la aproximación interpolada LINEAL entre marcas es $1e-3$ por construcción.

- Devuelve la distancia más pequeña entre dos geometrías rígidas temporales sobre su dominio temporal compartido

```
{geometry,trgeometry,stbox} |=| {geometry,trgeometry,stbox} → float
nearestApproachDistance({geometry,trgeometry,stbox}, {geometry,trgeometry,stbox}) → float
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'
|=|
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(50 0), 0.0)@2001-01-01, Pose(Point(50 0), 0.0)@2001-01-02]';
-- 39
```

El operador `|=|` se puede utilizar para búsquedas de vecino más cercano contra índices GiST o SP-GiST (véase Sección 15.17).

- Devuelve el instante en el que los dos argumentos están a su menor distancia mutua

```
nearestApproachInstant({geometry,trgeometry}, {geometry,trgeometry}) → trgeometry
```



```
SELECT asText(nearestApproachInstant(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  geometry 'Point(5 5)'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0));Pose(POINT(5 0),0)@2001-01-01 12:00:00+00
```

- Devuelve la línea que conecta los dos argumentos en su aproximación más cercana

shortestLine({geometry,trgeometry}, {geometry,trgeometry}) → geometry

```
SELECT ST_AsText(shortestLine(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.0)@2001-01-01',
  geometry 'Point(10 0)'));
-- LINESTRING(1 0,10 0)
```

15.11. Similitud

Las funciones de similitud comparan dos geometrías rígidas temporales a través de sus trayectorias del centroide.

- Devuelve la distancia de Hausdorff discreta, la distancia de Fréchet discreta, o la distancia de deformación dinámica del tiempo (Dynamic Time Warp) entre dos geometrías rígidas temporales

hausdorffDistance(trgeometry, trgeometry) → float

frechetDistance(trgeometry, trgeometry) → float

dynTimeWarpDistance(trgeometry, trgeometry) → float

```
SELECT round(frechetDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(2 0),0)@2000-01-03]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(0 2),0)@2000-01-03]'), 6);
-- 2.828427
SELECT round(hausdorffDistance(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(2 0),0)@2000-01-03]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2000-01-01, Pose(Point(0 2),0)@2000-01-03]'), 6);
-- 2
```

- Devuelve los pares de correspondencia entre dos geometrías rígidas temporales con respecto a la distancia de Fréchet discreta o a la distancia de deformación dinámica del tiempo (Dynamic Time Warp), como un conjunto de pares de índices de instante (i, j)

frechetDistancePath(trgeometry, trgeometry) → {(i, j)}

dynTimeWarpPath(trgeometry, trgeometry) → {(i, j)}

```
SELECT frechetDistancePath(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(20 0), 0.0)@2001-01-02]');
-- (0,0)
-- (1,1)
SELECT dynTimeWarpPath(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(20 0), 0.0)@2001-01-02]');
```

```
-- (0,0)
-- (1,1)
```

15.12. Comparaciones

Los operadores de igualdad y de siempre / alguna vez comparan dos geometrías rígidas temporales sobre la geometría materializada: tanto la forma de referencia como la trayectoria de pose deben coincidir en cada instante compartido.

- Probar la (des)igualdad exacta de dos geometrías rígidas temporales

```
trgeometry = trgeometry → boolean
```

```
trgeometry <> trgeometry → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' = trgeometry '↔
Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' <> trgeometry '↔
Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- f
```

- Probar si la geometría materializada es alguna vez (no) igual al operando en algún momento

```
{geometry,trgeometry} ?= {geometry,trgeometry} → boolean
```

```
{geometry,trgeometry} ?<> {geometry,trgeometry} → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(↔
Point(5 0), 0.0)@2001-01-02]';
?= geometry 'Polygon((5 0,6 0,6 1,5 1,5 0))';
-- t
```

- Probar si la geometría materializada es siempre (no) igual al operando

```
{geometry,trgeometry} %= {geometry,trgeometry} → boolean
```

```
{geometry,trgeometry} %<> {geometry,trgeometry} → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' %= trgeometry '↔
Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
```

15.13. Operadores de caja delimitadora

Los operadores topológicos y de posición comparan las cajas delimitadoras espaciotemporales de los operandos; ambos pueden ser una `trgeometry`, un `stbox`, una `geometry` o un `tstzspan`.

- Operadores topológicos estándar sobre cajas delimitadoras

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(↔
Point(10 0), 0.0)@2001-01-02]';
&&
stbox 'STBOX X((5, -1), (15, 1))';
-- t
```

■ Operadores de posición espacial y temporal

El conjunto completo <<, >>, &<, &>, <<|, &<|, |>>, |&>, <</, &</, />>, /&>, <<#, &<#, #>>, #&> acepta `trgeometry` en cualquiera de los lados. Los operadores <</, &</, />> y /&> comparan el eje Z y requieren que ambos argumentos tengan dimensión Z.

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' << stbox 'STBOX X((5,5),(6,6))';
-- true
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  <</ stbox 'STBOX Z((5,5,5),(6,6,6))';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  /&> stbox 'STBOX Z((5,5,5),(6,6,6))';
-- false
```

15.14. Mosaicos

Estas funciones devuelven las cajas espaciotemporales de una cuadrícula multidimensional que la geometría rígida temporal interseca, calculadas sobre su trayectoria del centroide.

■ Devuelve las cajas espaciotemporales de una geometría rígida temporal dividida con respecto a una cuadrícula espacial, temporal o espaciotemporal

`spaceBoxes(trgeometry, xsize float [, ysize float [, zsize float]], sorigin geometry = 'Point(0 0 0)', bitmatrix bool = true, borderInc bool = true) → stbox[]`

`timeBoxes(trgeometry, interval, torigin timestamp_tz = '2000-01-03', bitmatrix bool = true, borderInc bool = true) → stbox[]`

`spaceTimeBoxes(trgeometry, xsize float [, ysize float [, zsize float]], interval, sorigin geometry = 'Point(0 0 0)', torigin timestamp_tz = '2000-01-03', bitmatrix bool = true, borderInc bool = true) → stbox[]`

```
SELECT array_length(spaceBoxes(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0,0)@2001-01-02]',
  2.0), 1);
-- 6
```

15.15. Cajas delimitadoras

Estas funciones descomponen una geometría rígida temporal en las cajas delimitadoras o los lapsos de tiempo de sus segmentos o de un número solicitado de contenedores.

■ Devuelve el arreglo de cajas espaciotemporales o lapsos de tiempo de los segmentos de una geometría rígida temporal

`stboxes(trgeometry) → stbox[]`

`spans(trgeometry) → tstzspan[]`

```
SELECT stboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"STBOX XT((0,0),(11,1)), [2001-01-01, 2001-01-02]}"
```

```
SELECT spans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"[2001-01-01, 2001-01-02]"}
```

- Devuelve las cajas espaciotemporales o los lapsos de tiempo de una geometría rígida temporal dividida en un número dado de contenedores, o con un número dado de segmentos por contenedor

```
splitNSTboxes(trgeometry, integer) → stbox[]
splitEachNSTboxes(trgeometry, integer) → stbox[]
splitNSpans(trgeometry, integer) → tstzspan[]
splitEachNSpans(trgeometry, integer) → tstzspan[]
```

```
SELECT splitNSTboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"STBOX XT(((0,0), (11,1)), [2001-01-01, 2001-01-02])"}
SELECT splitNSpans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"[2001-01-01, 2001-01-02]"}
```

15.16. Agregaciones

- Número de geometrías rígidas temporales solapadas en cada instante, equivalente a `tCount` sobre la `tpose` subyacente

```
tCount(trgeometry) → tint
```

```
SELECT tCount(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {[1@2001-01-01, 1@2001-01-02]}
```

- Caja delimitadora agregada de una columna de geometrías rígidas temporales

```
extent(trgeometry) → stbox
```

```
SELECT extent(temp) FROM tbl_trgeometry;
```

- Combinar dos `trgeometries` con la misma geometría de referencia en un único conjunto de secuencias

```
merge(trgeometry, trgeometry) → trgeometry
```

```
mergeAgg(trgeometry) → trgeometry
```

```
SELECT asText(merge(
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(5 0), 0.0)@2001-01-05]'));
-- {"[2001-01-01, 2001-01-02] [2001-01-04, 2001-01-05]"}
```

15.17. Indexación

Hay tres tipos de índices disponibles sobre columnas `trgeometry`. Cada uno utiliza la caja delimitadora espaciotemporal de la geometría materializada y admite consultas topológicas, de posición y de distancia KNN.

- `GIST(temp)`, GiST de árbol R.
- `SPGIST(temp trgeometry_quadtree_ops)`, SP-GiST de árbol cuádruple.
- `SPGIST(temp trgeometry_kdtree_ops)`, SP-GiST de árbol kd.

```
CREATE INDEX tbl_trgeometry_gist_idx
  ON tbl_trgeometry
  USING GIST(temp);

-- topological lookup
SELECT count(*) FROM tbl_trgeometry
WHERE temp && stbox 'STBOX XT((0, 0), (10, 10)),[2001-01-01, 2001-01-02]';

-- KNN (ordered scan)
SELECT mmsi
FROM tbl_trgeometry
ORDER BY temp <-> trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(100 100), 0.0)@2001 ←
-01-01'
LIMIT 10;
```

La variante de árbol kd suele ganar en cargas de trabajo dominadas por consultas KNN con una cota de ordenación ajustada; la variante de árbol cuádruple es más uniforme. El GiST de árbol R es el valor predeterminado seguro para cargas de trabajo mixtas de lectura/escritura.

Capítulo 16

Tipos JSON en MobilityDB

PostgreSQL proporciona dos tipos para almacenar datos JSON: `json` y `jsonb`. El tipo `json` almacena una copia exacta del texto de entrada, que debe volver a analizarse cada vez que se procesa. Por otro lado, el tipo `jsonb` almacena los datos JSON en un formato binario descompuesto que hace que su entrada sea ligeramente más lenta debido a la sobrecarga de conversión añadida, pero significativamente más rápida de procesar, ya que no es necesario volver a analizarla. El tipo `jsonb` también admite indexación.

En MobilityDB, los tipos `textset` y `ttext` se utilizan para representar, respectivamente, conjuntos de valores JSON y valores JSON temporales. Además, el tipo `jsonb` sirve como tipo base para definir los tipos `jsonbset` y `tjsonb`. La mayoría de las funciones y operadores descritos en los capítulos anteriores para los tipos conjunto y temporal también son aplicables a los tipos JSON correspondientes. Además, hay funciones específicas definidas para estos tipos, que se derivan de las funciones correspondientes de los tipos `json` y `jsonb`.

En este capítulo, describimos los tipos JSON de MobilityDB y sus operaciones asociadas. Remitimos a la [documentación](#) de PostgreSQL para una explicación detallada de los tipos `json` y `jsonb` y su funcionalidad. Hemos buscado habilitar la misma sintaxis de las funciones originales de PostgreSQL para los tipos correspondientes de MobilityDB, como se ilustra en este [documento](#).

Considere por ejemplo el operador `->`, que extrae un campo de objeto JSON con una clave dada.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Aplicar el mismo operador a un conjunto JSONB y a un valor JSONB temporal produce los siguientes resultados

```
SELECT jsonbset '{"{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}]' -> text 'speed';
-- {"10", "20"}
SELECT tjsonb '["unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]' -> text 'speed';
-- {[10@2001-01-01, 20@2001-01-02]}
```

que se obtienen aplicando el operador de PostgreSQL `->` a cada elemento del conjunto JSONB y a cada instante del valor JSONB temporal.

16.1. Operaciones sobre conjuntos JSONB

Al manipular colecciones JSON de estructura variable, puede ocurrir que un elemento esté definido en algunos de los documentos pero no en todos. PostgreSQL proporciona el *modo lax* para este propósito, donde el argumento `null_value_treatment` determina el comportamiento en el caso en que una función devuelve un valor NULL. El argumento puede tomar uno de los siguientes valores: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, donde `'use_json_null'` es el valor por defecto. En PostgreSQL el modo lax se admite solo para operaciones de *actualización* (no de *consulta*) con la función `jsonb_set_lax`. En MobilityDB, hemos mantenido la misma semántica para la función correspondiente `jsonbsetSetLax`,

pero habilitamos un comportamiento similar para *todas* las operaciones JSON que pueden devolver un valor nulo, salvo que reemplazamos el valor 'return_target' por 'return_null', ya que el primero no tiene sentido para operaciones sobre conjuntos. Para operadores como $\rightarrow$, se utiliza el valor por defecto 'use_json_null' y no se puede cambiar, mientras que para las funciones correspondientes jsonbsetObjectField, el último argumento especifica el comportamiento en el caso en que la función devuelve NULL. Ilustramos este comportamiento para la función jsonbsetObjectField a continuación.

■ Extraer un campo de objeto JSON especificado por una clave

```
{textset, jsonbset} -> text -> {textset, jsonbset}
jsonbset ->> text -> textset
jsonbsetObjectField(jsonbset, text, null_handle text='use_json_null') -> jsonbset
jsonbsetObjectFieldText(jsonbset, text, null_handle text='use_json_null') -> textset
```

```
SELECT textset '{{{"unit": "km", "speed": 10},
{"unit": "km", "speed": 20}}' -> text 'speed';
-- [{"10", "20"}]
SELECT jsonbset '{{{"position": "Point(1 1)", "speed": 10},
{"position": "Point(2 2)", "speed": 20}}' ->> text 'position';
-- [{"Point(1 1)", "Point(2 2)"}]
SELECT textset '{{{"unit": "km", "speed": 10},
{"unit": "km"}, {"unit": "km", "speed": 10}}' -> text 'speed';
-- [{"10", null, 10}]
```

Mostramos a continuación el comportamiento de la función jsonbsetObjectField para los posibles valores del argumento null_handle. Todas las funciones JSON que pueden devolver un valor NULL se comportan de forma similar en MobilityDB.

```
SELECT jsonbsetObjectField(textset '{{{"unit": "km", "speed": 10},
{"unit": "km"}, {"unit": "km", "speed": 10}}', 'speed',
'raise_exception');
-- ERROR: The lifted operation returned NULL
SELECT jsonbsetObjectField(textset '{{{"unit": "km", "speed": 10},
{"unit": "km"}, {"unit": "km", "speed": 10}}', 'speed',
'use_json_null');
-- [{"10", null, 10}]
SELECT jsonbsetObjectField(textset '{{{"unit": "km", "speed": 10},
{"unit": "km"}, {"unit": "km", "speed": 10}}', 'speed',
'delete_key');
-- [{"10", 10}, [10]]
SELECT jsonbsetObjectField(textset '{{{"unit": "km", "speed": 10},
{"unit": "km"}, {"unit": "km", "speed": 10}}', 'speed',
'return_null');
-- NULL
```

■ Extraer un campo de objeto JSON especificado por una ruta

```
{textset, jsonbset} #> text[] -> {textset, jsonbset}
jsonbsetExtractPath(jsonbset, text[], null_handle text='use_json_null') -> jsonbset
jsonbsetExtractPathText(jsonbset, text[], null_handle text='use_json_null') -> textset
```

```
SELECT textset '{{"speed": {"unit": "km", "value": 10},
"speed": {"unit": "km", "value": 20}}' #> ARRAY[text 'speed', 'value'];
-- [{"10", "20"}]
SELECT jsonbset '{{"position": "Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]},
{"position": "Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}}' #>>
ARRAY[text 'PoIs', '1'];
-- [{"La Bourse", "Manneken Pis"]}
```

■ Extraer un elemento de un arreglo JSON

```
jsonbsetArrayElement(jsonbset, integer, null_handle text='use_json_null') → jsonbset
jsonbsetArrayElementText(jsonbset, integer, null_handle text='use_json_null') → jsonbset
```

```
SELECT jsonbsetArrayElement(textset '["Grand Place", "La Bourse"],
  ["Palais Royal", "Manneken Pis"]', 0);
-- ["Grand Place", "Palais Royal"]
SELECT jsonbsetArrayElement(jsonbset '["Grand Place", "La Bourse"],
  ["Palais Royal", "Manneken Pis"]', 1);
-- ["La Bourse", "Manneken Pis"]
```

■ Extraer un valor alfanumérico de un conjunto JSONB dado por una clave

```
intset(jsonbset, text, null_handle text='raise_exception') → tint
floatset(jsonbset, text, null_handle text='raise_exception') → tfloat
textset(jsonbset, text, null_handle text='raise_exception') → textset
```

Tenga en cuenta que el valor `use_json_null` no puede utilizarse para las funciones anteriores y por lo tanto se utiliza el valor por defecto `'raise_exception'`.

```
SELECT intset(jsonbset '{"{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}'}', 'speed');
-- {10, 20}
SELECT floatset(jsonbset '{"{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}', '{"speed": 25}'}', 'speed');
-- {10, 20, 25}
SELECT textset(jsonbset '{"{"road": "Bvd Gén. Jacques", "category": "primary"}',
  '{"road": "Bvd de la Cambre", "category": "residential"}'}', 'category');
-- {primary, residential}
```

■ Concatenación JSONB temporal

```
jsonbset || {jsonb, jsonbset} → jsonbset
```

```
SELECT jsonbset ' [{"speed":10}, {"speed":20}]' ||
  '{"unit":"km"}'::jsonb;
-- [{"unit": "km", "speed": 10}, {"unit": "km", "speed": 20}]
SELECT jsonbset ' [{"speed":10}, {"speed":20}]' ||
  jsonbset ' [{"Position":"Point(1 1)", {"Position":"Point(2 2)"}]';
/* [{"speed": 10, "Position": "Point(1 1)",
  {"speed": 20, "Position": "Point(2 2)"}] */
```

■ Eliminación JSONB temporal

```
jsonbset - {int, text, text[]} → jsonbset
```

```
jsonbset #- text[] → jsonbset
```

```
SELECT jsonbset ' [{"unit": "km", "speed": 10},
  {"unit": "km", "speed": 20}]' - 'unit';
-- [{"speed": 10}, {"speed": 20}]
SELECT jsonbset ' ["Grand Place", "La Bourse",
  ["Palais Royal", "Manneken Pis"]]' - 1;
-- ["Grand Place", ["Palais Royal"]]
SELECT jsonbset ' [{"location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"],
  "location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}]' #-
  ARRAY[text "\"PoIs\"", "1"];
-- ["La Bourse", "Manneken Pis"]
```

Como se muestra a continuación, el resultado de una operación de eliminación puede ser un registro vacío o un arreglo vacío. Para eliminar estos valores, puede utilizarse la función `minusValue`.


```

SELECT jsonbset '({{"unit": "km", "speed": 10, "light": true},
  {"unit": "km", "speed": 20}})' - ARRAY[text 'unit', 'speed'];
-- ({{"light": true}, {}})
SELECT jsonbset '(["Grand Place", "La Bourse",
  ["Palais Royal"]])' - 0;
-- [{"La Bourse"}, []]

SELECT minusValues(jsonbset '({{"unit": "km", "speed": 10, "light": true},
  {"unit": "km", "speed": 20}})' - ARRAY[text 'unit', 'speed'],
  jsonbset '{"[]", "{}"}');
-- ({{"light": true}, {"light": true}})
SELECT minusValues(jsonbset '(["Grand Place", "La Bourse",
  ["Palais Royal"]])' - 0, jsonbset '{"[]", "{}"}');
-- [{"La Bourse"}, ["La Bourse"]]

```

■ Existencia en un conjunto JSONB

```

jsonbset ? text → boolean[]
jsonbset ?| text[] → boolean[]
jsonbset ?& text[] → boolean[]

```

Como en PostgreSQL, los operadores `?|` y `?&` comprueban, respectivamente, si *alguna* o *todas* las cadenas del arreglo de texto existen como claves de nivel superior o elementos de arreglo.

El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```

SELECT jsonbset '{"{\\"speed\\": 10}", "{\\"speed\\": 20, \\"units\\": \\"km/h\\"}" }';
-- '{"{\\"speed\\": 20, \\"units\\": \\"km/h\\"}", "{\\"speed\\": 10}" }'
SELECT jsonbset '{"{\\"speed\\": 10}", "{\\"speed\\": 20, \\"units\\": \\"km/h\\"}" }' ? text ' ←
  units';
-- {t,f}
SELECT jsonbset '{"{\\"speed\\": 10}", "{\\"speed\\": 20, \\"units\\": \\"km/h\\"}" }'
  ?| ARRAY[text 'units', text 'lights'];
-- {t,f}
SELECT jsonbset '{"{\\"speed\\": 10}", "{\\"speed\\": 20, \\"units\\": \\"km/h\\"}" }'
  ?& ARRAY[text 'speed', text 'units'];
-- {t,f}

```

■ Asignación JSONB temporal

```

jsonbsetSet(jsonbset, path text[], jsonb, create boolean=true) → jsonbset
jsonbsetSetLax(jsonbset, path text[], jsonb, create boolean=true, handle_null text) → jsonbset

```

Devuelve el valor `jsonbset` con el elemento especificado por la ruta reemplazado por `jsonb`, o añadido si `create` es verdadero y el elemento no existe. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, y `create_if_missing` es verdadero, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

La función `jsonbsetSetLax` se comporta de forma idéntica a `jsonbsetSet` si el valor dado no es NULL; de lo contrario, se comporta según el valor de `handle_null`, que debe ser uno de `'raise_exception'`, `'use_json_null'`, `'delete_key'` o `'return_source'`. Como se indicó anteriormente, mantuvimos el comportamiento de PostgreSQL para esta función habilitando el valor `'return_source'`, mientras que para todas las demás operaciones de *consulta*, este valor se ha reemplazado por `'return_null'`.

```

SELECT jsonbsetSet(jsonbset '({"speed":10}, {"speed":20})',
  ARRAY['units'], '"km/h"::jsonb);
-- [{"speed": 10, "units": "km/h"}, {"speed": 20, "units": "km/h"}]
SELECT jsonbsetSet(jsonbset '({"speed":10}, {"speed":20})',
  ARRAY['units'], '"km/h"::jsonb, false);
-- [{"speed": 10}, {"speed": 20}]

```

```
SELECT jsonbsetSet(jsonbset ' [{"speed": 10, "units": "km/h"},
  {"speed": 20, "units": "km/h"}]', ARRAY['units'], 'mi/h'::jsonb);
-- [{"speed": 10, "units": "mi/h"}, {"speed": 20, "units": "mi/h"}]
SELECT jsonbsetSetLax(jsonbset ' [{"speed": 10, "units": "km/h"},
  {"speed": 20}]', ARRAY['units'], 'null'::jsonb, true, 'delete_key');
-- [{"speed": 10, "units": "km/h"}, {"speed": 20}]
```

■ Inserción JSONB temporal

`jsonbsetInsert(jsonbset, path text[], jsonb, after boolean=false) → jsonbset`

Devuelve el valor `jsonbset` con `jsonb` insertado. Si el elemento especificado por la ruta es un elemento de arreglo, el nuevo valor se insertará antes de ese elemento si `after` es falso, o después en caso contrario. Si el elemento especificado por la ruta es un campo de objeto, el nuevo valor se insertará solo si el objeto no contiene ya esa clave. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

```
SELECT jsonbsetInsert(jsonbset ' [{"speed":10}, {"speed":20}]',
  ARRAY['units'], 'km/h'::jsonb);
-- [{"speed": 10, "units": "km/h"}, {"speed": 20, "units": "km/h"}]
SELECT jsonbsetInsert(jsonbset ' [{"speed":10}, {"speed":20}]',
  ARRAY['units'], 'km/h'::jsonb, false);
-- [{"speed": 10}, {"speed": 20}]
SELECT jsonbsetInsert(jsonbset ' [{"speed": 10, "units": "km/h"},
  {"speed": 20, "units": "km/h"}]', ARRAY['units'], 'mi/h'::jsonb);
-- [{"speed": 10, "units": "mi/h"}, {"speed": 20, "units": "mi/h"}]
```

■ Devolver un conjunto JSONB sin nulos

`jsonbsetStripNulls(jsonbset, strip_in_arrays bool=false) → jsonbset`

El último argumento indica si los elementos de arreglo nulos también se eliminan. Los valores nulos simples nunca se eliminan.

```
SELECT jsonbsetStripNulls(textset ' [{"speed": 10, "lights": null},
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}]');
/* [{"speed": 10},
  {"PoIs": ["Grand Place", "La Bourse", null], "speed": 20}] */
SELECT jsonbsetStripNulls(jsonbset ' [{"speed": 10, "lights": null},
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}]', true);
/* [{"speed": 10},
  {"PoIs": ["Grand Place", "La Bourse"], "speed": 20}] */
SELECT jsonbsetStripNulls(jsonbset
  ' [{"road": "Bvd Gén. Jacques", "category": "primary"},
  {"road": "Bvd de la Cambre", "category": "primary"},
  {"road": "rue de l'Abbaye", "category": null}]');
/* [{"road": "Bvd Gén. Jacques", "category": "primary"},
  {"road": "Bvd de la Cambre", "category": "primary"},
  {"road": "rue de l'Abbaye"}] */
```

Como se muestra a continuación, la función no elimina los valores nulos simples. Para este propósito puede utilizarse la función `minusValue`.

```
SELECT jsonbsetStripNulls(textset ' [{"speed": 10}, null,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]} ]');
/* [{"speed":10}, null,
  {"speed":20,"PoIs":["Grand Place","La Bourse"]} */
SELECT minusValues(textset ' [{"speed": 10}, null,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]} ]', 'null');
/* [{"speed": 10}, {"speed": 10},
  [{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}]]
```

- ¿Devuelve la ruta JSON algún elemento para el conjunto JSONB especificado?

```
jsonbset @? jsonpath → boolean[]
jsonbsetPathExists(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathExistsTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

El operador @? suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```
SELECT jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}"';
-- {"{\\"speed\\": 20, \\"units\\": \\"km/h\\"}", "{\\"speed\\": 10}"}
SELECT jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}"' @? '$.units' ←
;
-- {t,f}
SELECT jsonbsetPathExists(jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}'
    \\"}"}',
    '$.speed ? (@ > $min)', '{"min": 15}');
-- {t,f}
```

- Devolver el resultado de una comprobación de predicado de ruta JSON para un conjunto JSONB

```
jsonbset @@ jsonpath → boolean[]
jsonbsetPathMatch(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathMatchTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

El operador @@ suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```
SELECT jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}"';
-- {"{\\"speed\\": 20, \\"units\\": \\"km/h\\"}", "{\\"speed\\": 10}"}
SELECT jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}"' @@ '$.speed ←
    > 15';
-- {t,f}
SELECT jsonbsetPathMatch(jsonbset '{"{\\"speed\\": 10}","{\\"speed\\": 20, \\"units\\": \\"km/h\\"}'
    \\"}"}',
    '$.speed > $min', '{"min": 15}');
-- {t,f}
```

- Devolver todos los elementos devueltos por una ruta JSON de un conjunto JSONB, como un arreglo JSON

```
jsonbsetPathQueryArray(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryArrayTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

La función anterior puede suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT jsonbsetPathQueryArray(jsonbset
    ' [{"speed":10}, {"speed": 20, "units": "km/h"}]',
    '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- [[{"speed": 10}], [{"speed": 20, "units": "km/h"}]]
-- TODO, it works without "_tz"
SELECT jsonbsetPathQueryArrayTz(jsonbset
    ' [{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]},
    {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}]',
```

```
'$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
  @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}, ["2000-01-07"]]
```

■ Devolver el primer elemento devuelto por una ruta JSON de un conjunto JSONB

```
jsonbsetPathQueryFirst(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryFirstTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

Las funciones anteriores pueden suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT jsonbsetPathQueryFirst(jsonbset
  '[{"speed":10}, {"speed": 20, "units": "km/h"}]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}], [{"speed": 20, "units": "km/h"}]]
-- TODO, it works without "_tz"
SELECT jsonbsetPathQueryArrayTz(jsonbset
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]},
    {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}, ["2000-01-07"]]
```

16.2. Valores JSONB temporales

El tipo temporal `tjsonb` permite representar la evolución en el tiempo de valores `jsonb`. Como todos los tipos temporales, viene en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores `tjsonb` en estos subtipos.

```
-- Instant
SELECT tjsonb '{"\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01';
-- Sequence with discrete interpolation
SELECT tjsonb '{"\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01,
  {\"vehicleId\": 1, \"location\": \"Point(2 2)\"}@2001-01-02}';
-- Sequence with step interpolation
SELECT tjsonb '{"\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01,
  {\"vehicleId\": 1, \"location\": \"Point(2 2)\"}@2001-01-02}';
-- Sequence set
SELECT tjsonb '{"[{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01,
  {\"vehicleId\": 1, \"location\": \"Point(2 2)\"}@2001-01-02},
  [{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}@2001-01-03,
  {\"vehicleId\": 1, \"location\": \"Point(3 3)\"}@2001-01-04}]}';
```

Como puede verse arriba, para el subtipo `Instant` necesitamos encerrar el valor JSONB entre comillas y escapar las comillas internas; de lo contrario, la llave de apertura del valor JSONB se interpretaría como el comienzo de un subtipo `Sequence` con interpolación discreta.

El tipo `tjsonb` acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo. Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet`. El argumento es opcional y, si no se especifica para una columna, se permiten valores de cualquier subtipo.

```
SELECT tjsonb(Instant) '{"\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01';
-- {\"location\": \"Point(1 1)\", \"vehicleId\": 1}@2001-01-01
SELECT tjsonb(Sequence) '{"\"vehicleId\": 1, \"location\": \"Point(1 1)\"}@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Los valores JSONB temporales de subtipo secuencia o conjunto de secuencias se convierten a una forma normal de modo que valores equivalentes tengan representaciones idénticas. Para ello, los valores de instantes consecutivos se fusionan cuando es posible. Tres valores de instantes consecutivos pueden fusionarse en dos si sus valores JSONB son iguales. Asimismo, dos secuencias consecutivas que son adyacentes y tienen el mismo valor JSONB de final e inicio pueden fusionarse en una sola. A continuación se dan ejemplos de transformación a una forma normal.

```
SELECT asText(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03] ');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(1 1)", "vehicleId": 1}@2001-01-03} */
SELECT asText(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03],
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-04] ');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-04} */
```

16.3. Validez de los valores JSONB temporales

Los valores JSONB temporales deben satisfacer las restricciones especificadas en Sección 4.3 para que estén bien definidos. Se genera un error siempre que no se satisfaga alguna de estas restricciones. A continuación se dan ejemplos de valores incorrectos.

```
-- Null values are not allowed
SELECT tjsonb 'NULL@2001-01-01 08:05:00';
SELECT tjsonb 'Point(0 0)@NULL';
-- Base type is not a JSONB
SELECT tjsonb 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para JSONB temporal. La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos anteriores pueden aplicarse a los tipos JSONB temporales. Por lo tanto, en las firmas de las funciones dadas anteriormente, la notación `base` representa un valor `tjsonb` y la notación `ttype` representa un valor `tjsonb`. Para evitar redundancia, solo presentamos a continuación algunos ejemplos de estas funciones y operadores para valores JSONB temporales.

16.4. Entrada y salida

- Devolver la representación en texto bien conocido (WKT)

`asText({tjsonb,tjsonb[]}) → {text,text[]}`

```
SELECT asText(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] ');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02} */
SELECT asText(ARRAY[tjsonb '{"\vehicleId\:1, \location\: \Point(1 1)\"}@2001-01-01',
'{"\vehicleId\: 1, \location\: \Point(2 2)\"}@2001-01-02'}]);
/* [{"\location\: \Point(1 1)\", \vehicleId\: 1}@2001-01-01",
"\location\: \Point(2 2)\", \vehicleId\: 1}@2001-01-02} */
```

Como puede verse arriba, para los arreglos de valores JSONB, los elementos deben encerrarse entre comillas y, por lo tanto, las comillas internas deben escaparse.

- Devolver la representación en binario bien conocido (WKB) o en binario bien conocido extendido hexadecimal (HexWKB)

`asBinary(tjsonb,endian text="") → bytea`

`asHexWKB(tjsonb, endian text="") → text`

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01'};
-- \x0141000138000000000000000200002008000080090000000a00000090000106c6f6361746966e7...
SELECT asHexWKB(tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01'};
-- 0141000138000000000000000200002008000080090000000a00000090000106C6F6361746966E7...
```

- Devolver la representación JSON de objetos en movimiento (MF-JSON)

`asMFJSON(text) → tjsonb`

```
SELECT asMFJSON(tjsonb '["Position": "Point(1 1)"@2001-01-01,
{"Position": "Point(2 2)"@2001-01-02}'];
/* {"type": "MovingJsonb", "values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
"datetimes": ["2001-01-01T00:00:00+01", "2001-01-02T00:00:00+01"],
"lower_inc": true, "upper_inc": true, "interpolation": "Step"} */
```

- Entrada a partir de la representación en texto bien conocido (WKT)

`tjsonbFromText(text) → tjsonb`

```
SELECT tjsonbFromText(text
'["vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"@2001-01-02}'];
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(2 2)", "vehicleId": 1}@2001-01-02} */
```

- Entrada a partir de la representación en binario bien conocido (WKB) o en binario bien conocido hexadecimal (HexWKB)

`tjsonbFromBinary(bytea) → tjsonb`

`tjsonbFromHexWKB(text) → tjsonb`

```
SELECT tjsonbFromBinary(asBinary(
tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01}'));
-- [{"location": "Point(1 2)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbFromHexWKB(asHexWKB(
tjsonb '{"vehicleId": 1, "location": "Point(1 2)"@2001-01-01}'));
-- [{"location": "Point(1 2)", "vehicleId": 1}@2001-01-01]
```

- Entrada a partir de la representación JSON de objetos en movimiento (MF-JSON)

`tjsonbFromMFJSON(text) → tjsonb`

```
SELECT tjsonbFromMFJSON('{"type": "MovingJsonb", "interpolation": "Step",
"values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
"datetimes": ["2001-01-01T10:00:00Z", "2001-01-01T12:00:00Z"]}');
/* [{"Position": "Point(1 1)"@2001-01-01 11:00:00+01,
{"Position": "Point(2 2)"@2001-01-01 13:00:00+01} */
```

16.5. Constructores

- Constructor de valores JSONB temporales que tienen un valor constante

`tjsonb(jsonb, timestamp tz) → tjsonbInst`

`tjsonb(jsonb, tstzset) → tjsonbDiscSeq`

`tjsonb(jsonb, tstzspan) → tjsonbContSeq`

`tjsonb(jsonb, tstzspanset) → tjsonbSeqSet`

```

SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}', timestampz '2001-01-01');
-- {"location": "Point(1 1)", "vehicleId": 1}@2001-01-01
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(1 1)", "vehicleId": 1}@2001-01-03,
    {"location": "Point(1 1)", "vehicleId": 1}@2001-01-05} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspan '[2001-01-01, 2001-01-02]');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspanset '{[2001-01-01, 2001-01-02],[2001-01-03, 2001-01-04]}');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02,
    [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-03,
     {"location": "Point(1 1)", "vehicleId": 1}@2001-01-04}] */

```

■ Constructor de valores JSONB temporales del subtipo secuencia

`tjsonbSeq(tjsonbInst[], leftInc bool=true, rightInc bool=true) → tjsonbSeq`

```

SELECT tjsonbSeq(ARRAY[tjsonb
  '{"\vehicleId\: 1, \location\: \Point(1 1)\"}@2001-01-01',
  '{"\vehicleId\: 1, \location\: \Point(2 2)\"}@2001-01-02',
  '{"\vehicleId\: 1, \location\: \Point(1 1)\"}@2001-01-03']);
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
    {"location": "Point(1 1)", "vehicleId": 1}@2001-01-03} */

```

■ Constructor de valores JSONB temporales del subtipo conjunto de secuencias

`tjsonbSeqSet(tjsonb[]) → tjsonbSeqSet`

`tjsonbSeqSetGaps(tjsonbInst[], maxt=NULL, maxdist=NULL) → tjsonbSeqSet`

```

SELECT tjsonbSeqSet(ARRAY[tjsonb
  '{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02',
  '{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04']);
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02,
    {"location": "Point(2 2)", "vehicleId": 1}@2001-01-03,
    {"location": "Point(3 3)", "vehicleId": 1}@2001-01-04} */
SELECT tjsonbSeqSetGaps(ARRAY[tjsonb
  '{"\vehicleId\: 1, \location\: \Point(1 1)\"}@2001-01-01',
  '{"\vehicleId\: 1, \location\: \Point(2 2)\"}@2001-01-03',
  '{"\vehicleId\: 1, \location\: \Point(3 3)\"}@2001-01-05', interval '1 day']);
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
    {"location": "Point(2 2)", "vehicleId": 1}@2001-01-03,
    {"location": "Point(3 3)", "vehicleId": 1}@2001-01-05} */

```

16.6. Conversiones

■ Convertir un JSONB temporal a un `tstzspan`

`tjsonb::tstzspan`

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] '::tstzspan;
-- [2001-01-01, 2001-01-02]
```

- Convertir entre un JSONB temporal y un texto

tjsonb::text

text::tjsonb

```
SELECT (text ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] '::jsonb;
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
 {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02] */
SELECT pg_typeof(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] '::text);
-- text
```

- Convertir entre un JSONB temporal y un texto temporal

tjsonb::ttext

ttext::tjsonb

```
SELECT (ttext ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] '::tjsonb;
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
 {"location": "Point(2 2)", "vehicleId": 1}@2001-01-02] */
SELECT pg_typeof(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02] '::ttext);
-- ttext
```

16.7. Accesos

- Devolver los valores

getValues(tjsonb) → jsonbset

```
SELECT getValues(tjsonb
 ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02] ');
-- [{"location": "Point(1 1)", "vehicleId": 1}]
```

- Devolver el valor en una marca de tiempo

valueAtTimestamp(tjsonb,timestamp tz) → jsonb

```
SELECT valueAtTimestamp(tjsonb
 ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
 {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]', '2001-01-02');
-- {"location": "Point(2 2)", "vehicleId": 1}
```

16.8. Transformaciones

- Transformar un JSONB temporal a otro subtipo

tjsonbInst(tjsonb) → tjsonbInst

tjsonbSeq(tjsonb,interp='step') → tjsonbSeq

tjsonbSeqSet(tjsonb) → tjsonbSeqSet


```
SELECT tjsonbSeq(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01'};
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbSeq(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01',
  'discrete');
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT tjsonbSeqSet(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01'};
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01}]
```

■ Transformar un valor JSONB temporal a otra interpolación

`setInterp(tjsonb,interp) → tjsonb`

```
SELECT setInterp(tjsonb '{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01', ' ←
  step');
-- [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01]
SELECT setInterp(tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"@2001-01-01},
  [{"vehicleId": 1, "location": "Point(1 1)"@2001-01-02}]', 'discrete');
/* [{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
  {"location": "Point(1 1)", "vehicleId": 1}@2001-01-02} */
```

16.9. Operaciones JSON temporales

En esta sección, damos las versiones temporales de las funciones para los tipos `json` y `jsonb`. Remitimos a la [documentación](#) de PostgreSQL para explicaciones detalladas de estas funciones. Hemos buscado mantener en la medida de lo posible la misma sintaxis para las versiones no temporales y temporales de estas funciones, como se ilustra en este [documento](#).

Como se indicó en Sección 4.4, una forma común de generalizar las operaciones tradicionales a los tipos temporales es aplicar la operación *en cada instante*, lo que produce un valor temporal como resultado. Considere por ejemplo el operador `->` que extrae un campo de objeto JSON con la clave dada.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Aplicar el mismo operador a un valor JSON o JSONB temporal produce el siguiente resultado

```
SELECT ttext ' [{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb ' [{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]
```

que se obtiene aplicando el operador de PostgreSQL `->` en cada instante del valor JSON o JSONB temporal.

Al manipular colecciones JSON de estructura variable, puede ocurrir que un elemento esté definido en algunos de los documentos pero no en todos. PostgreSQL proporciona el modo `lax` para este propósito, donde el argumento `null_value_treatment` determina el comportamiento en el caso en que una función devuelve un valor `NULL`. El argumento puede tomar uno de los siguientes valores: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, donde `'use_json_null'` es el valor por defecto. En PostgreSQL el modo `lax` se admite solo para operaciones de *actualización* (no de *consulta*) con la función `jsonb_set_lax`. En MobilityDB, hemos mantenido la misma semántica para la función correspondiente `tjsonbSetLax`, pero habilitamos un comportamiento similar para *todas* las operaciones JSON temporales que pueden devolver un valor nulo, salvo que reemplazamos el valor `'return_target'` por `'return_null'`, ya que el primero no tiene sentido para operaciones temporales. Para operadores como `->`, se utiliza el valor por defecto `'use_json_null'` y no se puede cambiar, mientras que para las funciones correspondientes `tjsonObjectField` y `tjsonbObjectField`, el último argumento especifica el comportamiento en el caso en que la función devuelve `NULL`. Ilustramos este comportamiento para la función `tjsonObjectField` a continuación.

- Extraer un campo de objeto JSON temporal especificado por una clave

```
{ttext,tjsonb} -> text -> {ttext,tjsonb}
tjsonb ->> text -> ttext
tjsonbObjectField(ttext,text,null_handle text='use_json_null') -> ttext
tjsonbObjectField(tjsonb,text,null_handle text='use_json_null') -> tjsonb
tjsonbObjectFieldText(tjsonb,text,null_handle text='use_json_null') -> ttext
```

```
SELECT ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02}]' -> text 'speed';
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb ' [{{"position": "Point(1 1)", "speed": 10}@2001-01-01,
  {"position": "Point(2 2)", "speed": 20}@2001-01-03}]' ->> text 'position';
-- [{"Point(1 1)"@2001-01-01, "Point(2 2)"@2001-01-03}]
SELECT ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]' -> text 'speed';
-- [{"10@2001-01-01, null@2001-01-02, 10@2001-01-03}]
```

Mostramos a continuación el comportamiento de la función `tjsonbObjectField` para los posibles valores del argumento `null_handle`. Todas las funciones JSON que pueden devolver un valor NULL se comportan de forma similar en MobilityDB.

```
SELECT tjsonbObjectField(ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'raise_exception');
-- ERROR: The lifted operation returned NULL
SELECT tjsonbObjectField(ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'use_json_null');
-- [{"10@2001-01-01, null@2001-01-02, 10@2001-01-03}]
SELECT tjsonbObjectField(ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'delete_key');
-- [{"10@2001-01-01, 10@2001-01-02}, [10@2001-01-03]]
SELECT tjsonbObjectField(ttext ' [{{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03}]', 'speed',
  'return_null');
-- NULL
```

- Extraer un campo de objeto JSON temporal especificado por una ruta

```
{ttext,tjsonb} #> text[] -> {ttext,tjsonb}
tjsonbExtractPath(ttext,text[],null_handle text='use_json_null') -> ttext
tjsonbExtractPath(tjsonb,text[],null_handle text='use_json_null') -> tjsonb
tjsonbExtractPathText(tjsonb,text[],null_handle text='use_json_null') -> ttext
```

```
SELECT ttext ' [{"speed": {"unit": "km", "value": 10}}@2001-01-01,
  {"speed": {"unit": "km", "value": 20}}@2001-01-02]' #> ARRAY[text 'speed', 'value'];
-- [{"10"@2001-01-01, "20"@2001-01-02}]
SELECT tjsonb ' [{"position": "Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]}@2001-01-01,
  {"position": "Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}@2001-01-03]' #>>
  ARRAY[text 'PoIs', '1'];
-- [{"La Bourse"@2001-01-01, "Manneken Pis"@2001-01-03}]
```

- Extraer un elemento de un arreglo JSON temporal

```
tjsonbArrayElement(tjsonb,integer,null_handle text='use_json_null') -> tjsonb
tjsonbArrayElement(tjsonb,integer,null_handle text='use_json_null') -> tjsonb
tjsonbArrayElementText(tjsonb,integer,null_handle text='use_json_null') -> tjsonb
```

```
SELECT tjsonArrayElement(ttext '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02]', 0);
-- ["Grand Place"@2001-01-01, "Palais Royal"@2001-01-02]
SELECT tjsonbArrayElement(tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02]', 1);
-- ["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-02]
```

■ Extraer un valor alfanumérico temporal de un valor JSONB temporal dado por una clave

```
tbool(tjsonb,text,null_handle text='raise_exception') → tbool
tint(tjsonb,text,null_handle text='raise_exception') → tint
tfloat(tjsonb,text,interp='linear',null_handle text='raise_exception') → tfloat
ttext(tjsonb,text, null_handle text='raise_exception') → ttext
```

Tenga en cuenta que el valor `use_json_null` no puede utilizarse para las funciones anteriores y por lo tanto se utiliza el valor por defecto `'raise_exception'`.

```
SELECT tbool(tjsonb ' [{"speed": 10, "lights": "on"}@2001-01-01,
{"speed": 20, "lights": "on"}@2001-01-02]', 'lights');
-- [true@2001-01-01, true@2001-01-02]
SELECT tint(tjsonb ' [{"speed": 10, "units": "km/h"}@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02]', 'speed');
-- [10@2001-01-01, 20@2001-01-02]
SELECT tfloat(tjsonb ' [{"speed": 10, "units": "km/h"}@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02],[{"speed": 20}@2001-01-03,
{"speed": 20}@2001-01-04}]', 'speed', 'step');
-- Interp=Step;[10@2001-01-01, 20@2001-01-02], [20@2001-01-03, 20@2001-01-04]]
SELECT ttext(tjsonb ' [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
{"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03]', 'category');
-- [primary@2001-01-01, residential@2001-01-03]
```

■ Concatenación JSONB temporal

```
tjsonb || {jsonb,tjsonb} → tjsonb
```

```
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}]' ||
' [{"unit": "km"}]::jsonb;
-- [{"unit": "km", "speed": 10}@2001-01-01, {"unit": "km", "speed": 20}@2001-01-02}]
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed":20}@2001-01-03}]' ||
tjsonb ' [{"Position": "Point(1 1)}@2001-01-02, {"Position": "Point(2 2)}@2001-01-04}]';
/* [{"speed": 10, "Position": "Point(1 1)}@2001-01-02,
{"speed": 20, "Position": "Point(2 2)}@2001-01-03}] */
```

■ Eliminación JSONB temporal

```
tjsonb - {int,text,text[]} → tjsonb
```

```
tjsonb #- text[] → tjsonb
```

```
SELECT tjsonb ' [{"unit": "km", "speed": 10}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}]' - 'unit';
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02}]
SELECT tjsonb ' [{"Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal", "Manneken Pis"]@2001-01-02}]' - 1;
-- [{"Grand Place"]@2001-01-01, ["Palais Royal"]@2001-01-02]
SELECT tjsonb ' [{"location": "Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]@2001-01-01,
"location": "Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]@2001-01-02}]' #-
ARRAY[text "\PoIs\","", "1"];
-- ["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-02]
```

Como se muestra a continuación, el resultado de una operación de eliminación puede ser un registro vacío o un arreglo vacío. Para eliminar estos valores, puede utilizarse la función `minusValue`.

```
SELECT tjsonb '{{{"unit": "km", "speed": 10, "light": true}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}}' - ARRAY[text 'unit', 'speed'];
-- {{{"light": true}@2001-01-01, {}@2001-01-02}}
SELECT tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal"]@2001-01-02]' - 0;
-- [{"La Bourse"}@2001-01-01, []@2001-01-02]
```

```
SELECT minusValues(tjsonb '{{{"unit": "km", "speed": 10, "light": true}@2001-01-01,
{"unit": "km", "speed": 20}@2001-01-02}}' - ARRAY[text 'unit', 'speed'],
jsonbset '{"[]", "{}"}');
-- {{{"light": true}@2001-01-01, {"light": true}@2001-01-02}}
SELECT minusValues(tjsonb '["Grand Place", "La Bourse"]@2001-01-01,
["Palais Royal"]@2001-01-02]' - 0, jsonbset '{"[]", "{}"}');
-- {{{"La Bourse"}@2001-01-01, ["La Bourse"]@2001-01-02}}
```

■ Existencia JSONB temporal

```
tjsonb ? jsonb → tbool
tjsonb ?| jsonb[] → tbool
tjsonb ?& jsonb[] → tbool
```

Como en PostgreSQL, los operadores `?|` y `?&` comprueban, respectivamente, si *alguna* o *todas* las cadenas del arreglo de texto existen como claves de nivel superior o elementos de arreglo.

```
SELECT tjsonb '"{"geom": "Point(1 1)"}"@2001-01-01' ? text 'geom';
-- t@2001-01-01
SELECT tjsonb '{{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
{"geom": "Point(1 1)"}@2001-01-03}}' ?| ARRAY[text 'geom'];
-- {t@2001-01-01, t@2001-01-02, t@2001-01-03}
SELECT tjsonb '{{{"speed": 10, "units": "km/h"}@2001-01-01,
{"speed": 20, "units": "km/h"}@2001-01-02}}' ?& ARRAY['speed', 'units'];
-- {[t@2001-01-01, t@2001-01-02]}
```

■ Contiene/contenido JSONB temporal

```
{tjsonb, jsonb} @> {tjsonb, jsonb} → tbool
{tjsonb, jsonb} <@ {tjsonb, jsonb} → tbool
```

```
SELECT tjsonb '"{"geom": "Point(1 1)"}"@2001-01-01' @> jsonb '{"geom": "Point(1 1)"}';
-- t@2001-01-01
SELECT tjsonb '{{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
{"geom": "Point(1 1)"}@2001-01-03}}' @> jsonb '{"geom": "Point(1 1)"}';
-- {t@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT jsonb '{"geom": "Point(1 1)"}' <@ tjsonb '{{{"geom": "Point(1 1)"}@2001-01-01,
{"geom": "Point(1 1)"}@2001-01-02, {"geom": "Point(1 1)"}@2001-01-03,
{"geom": "Point(2 2)"}@2001-01-04, {"geom": "Point(2 2)"}@2001-01-05}}';
-- {[t@2001-01-01, t@2001-01-03], [f@2001-01-04, f@2001-01-05]}
```

■ Asignación JSONB temporal

```
tjsonbSet(tjsonb, path text[], jsonb, create boolean=true) → tjsonb
tjsonbSetLax(tjsonb, path text[], jsonb, create boolean=true, handle_null text) → tjsonb
```

Devuelve el valor `tjsonb` con el elemento especificado por la ruta reemplazado por `jsonb`, o añadido si `create` es verdadero y el elemento no existe. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, y `create_if_missing` es verdadero, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

La función `tjsonbSetLax` se comporta de forma idéntica a `tjsonbSet` si el valor dado no es `NULL`; de lo contrario, se comporta según el valor de `handle_null`, que debe ser uno de `'raise_exception'`, `'use_json_null'`, `'delete_key'` o `'return_source'`. Como se indicó anteriormente, mantuvimos el comportamiento de PostgreSQL para esta función habilitando el valor `'return_source'`, mientras que para todas las demás operaciones de *consulta*, este valor se ha reemplazado por `'return_null'`.

```
SELECT tjsonbSet(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"'::jsonb);
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]
SELECT tjsonbSet(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02]
SELECT tjsonbSet(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02}', ARRAY['units'], '"mi/h"'::jsonb);
-- [{"speed": 10, "units": "mi/h"}@2001-01-01, {"speed": 20, "units": "mi/h"}@2001-01-02]
SELECT tjsonbSetLax(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20}@2001-01-02}', ARRAY['units'], 'null'::jsonb, true, 'delete_key');
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20}@2001-01-02]
```

■ Inserción JSONB temporal

`tjsonbInsert(tjsonb,path text[],jsonb,after boolean=false) → tjsonb`

Devuelve el valor `tjsonb` con `jsonb` insertado. Si el elemento especificado por la ruta es un elemento de arreglo, el nuevo valor se insertará antes de ese elemento si `after` es falso, o después en caso contrario. Si el elemento especificado por la ruta es un campo de objeto, el nuevo valor se insertará solo si el objeto no contiene ya esa clave. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

```
SELECT tjsonbInsert(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"'::jsonb);
-- [{"speed": 10, "units": "km/h"}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]
SELECT tjsonbInsert(tjsonb '{"speed":10}@2001-01-01, {"speed":20}@2001-01-02}',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- [{"speed": 10}@2001-01-01, {"speed": 20}@2001-01-02]
SELECT tjsonbInsert(tjsonb '{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02}', ARRAY['units'], '"mi/h"'::jsonb);
-- [{"speed": 10, "units": "mi/h"}@2001-01-01, {"speed": 20, "units": "mi/h"}@2001-01-02]
```

■ Devolver un valor JSON temporal sin nulos

`tjsonStripNulls(ttext,strip_in_arrays bool=false) → ttext`

`tjsonbStripNulls(tjsonb,strip_in_arrays bool=false) → tjsonb`

El último argumento indica si los elementos de arreglo nulos también se eliminan. Los valores nulos simples nunca se eliminan.

```
SELECT tjsonStripNulls(ttext '{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02}');
/* [{"speed": 10}@2001-01-01,
  {"PoIs": ["Grand Place", "La Bourse", null], "speed": 20}@2001-01-02} */
SELECT tjsonbStripNulls(tjsonb '{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02}', true);
/* [{"speed": 10}@2001-01-01,
  {"PoIs": ["Grand Place", "La Bourse"], "speed": 20}@2001-01-02} */
SELECT tjsonbStripNulls(tjsonb
  '{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l'Abbaye", "category": null}@2001-01-03}');
/* [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l'Abbaye"}@2001-01-03] */
```

Como se muestra a continuación, la función no elimina los valores nulos simples. Para este propósito puede utilizarse la función `minusValue`.

```
SELECT tjsonStripNulls(ttext ' [{"speed": 10}@2001-01-01, null@2001-01-02,
{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03}');
/* [{"speed":10}@2001-01-01, null@2001-01-02,
{"speed":20,"PoIs":["Grand Place","La Bourse"]}@2001-01-03} */
SELECT minusValues(ttext ' [{"speed": 10}@2001-01-01, null@2001-01-02,
{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03}', 'null');
```

■ ¿Devuelve la ruta JSON algún elemento para el valor JSONB temporal especificado?

`tjsonb @? jsonpath → tbool`

`tjsonbPathExists(tjsonb,vars jsonb='{}',silent boolean=false) → tbool`

`tjsonbPathExistsTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool`

El operador `@?` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]' @?
'$.units ? (@ == "km/h")';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathExists(tjsonb
' [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
{"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
'$.category ? (@ == "residential")');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
-- TODO, it works without ".datetime()"
SELECT tjsonbPathExistsTz(tjsonb
' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"}@2001-01-01,
{"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"}@2001-01-02}',
'$.lastInspection ? (@.datetime() > "2000-06-01".datetime())');
-- [t@2001-01-01, f@2001-01-02]
```

■ Devolver el resultado de una comprobación de predicado de ruta JSON para un valor JSONB temporal

`tjsonb @@ jsonpath → tbool`

`tjsonbPathMatch(tjsonb,vars jsonb='{}',silent boolean=false) → tbool`

`tjsonbPathMatchTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool`

El operador `@@` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonb ' [{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]' @@
'$.units == "km/h"';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathMatch(tjsonb
' [{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
{"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
'$.category == "residential"');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
-- TODO, it works without ".datetime()"
SELECT tjsonbPathMatchTz(tjsonb
' [{"speed":10, "cameraId": "25", "lastInspection": "2000-08-01 12:00:00"}@2001-01-01,
```

```

    {"speed":20, "cameraId": "35", "lastInspection": "2000-03-01 12:00:00"}@2001-01-02}',
    '$.lastInspection.datetime() > "2000-06-01".datetime()');
-- [t@2001-01-01, f@2001-01-02]

```

- Devolver todos los elementos devueltos por una ruta JSON de un valor JSONB temporal, como un arreglo JSON

```
tjsonbPathQueryArray(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

```
tjsonbPathQueryArrayTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

La función anterior puede suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```

SELECT tjsonbPathQueryArray(tjsonb
  '[{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}@2001-01-01, [{"speed": 20, "units": "km/h"}@2001-01-02]
-- TODO, it works without "_tz"
SELECT tjsonbPathQueryArrayTz(tjsonb
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]}@2001-01-01,
    {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}@2001-01-02]',
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
    @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}@2001-01-01, [{"2000-01-07"}@2001-01-02]

```

- Devolver el primer elemento devuelto por una ruta JSON de un valor JSONB temporal

```
tjsonbPathQueryFirst(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

```
tjsonbPathQueryFirstTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

Las funciones anteriores pueden suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```

SELECT tjsonbPathQueryFirst(tjsonb
  '[{"speed":10}@2001-01-01, {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- [{"speed": 10}@2001-01-01, [{"speed": 20, "units": "km/h"}@2001-01-02]
-- TODO, it works without "_tz"
SELECT tjsonbPathQueryArrayTz(tjsonb
  '[{"cameraId":25, "inspections":["2000-01-03", "2000-01-06", "2000-01-09"]}@2001-01-01,
    {"cameraId":35, "inspections":["2000-01-04", "2000-01-07", "2000-01-10"]}@2001-01-02]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2000-01-05", "max":"2000-01-09"}');
-- [{"2000-01-06", "2000-01-09"}@2001-01-01, [{"2000-01-07"}@2001-01-02]

```

16.10. Restricciones

- Restringir a (el complemento de) un valor o un conjunto de valores

```
atValue(tjsonb,value) → tjsonb
```

```
minusValue(tjsonb,value) → tjsonb
```

```
atValues(tjsonb,valueSet) → tjsonb
```

```
minusValues(tjsonb,valueSet) → tjsonb
```

```

SELECT atValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
-- [{"location": "Point(2 2)", "vehicleId": 1}@2001-01-03]

```



```
SELECT minusValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
/* {[{"location": "Point(1 1)", "vehicleId": 1}@2001-01-01,
{"location": "Point(1 1)", "vehicleId": 1}@2001-01-03]} */
```

16.11. Comparaciones

■ Comparaciones tradicionales

tjsonb {=, <>, <, >, <=, >} tjsonb → boolean

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' =
tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' <>
tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' <
tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
```

■ Comparaciones siempre y alguna vez

{jsonb,tjsonb} {?=, %=} {jsonb,tjsonb} → boolean

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' ?=
jsonb '{"vehicleId": 1, "location": "Point(1 1)"}';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' %=
jsonb '{"vehicleId": 1, "location": "Point(1 1)"}';
-- false
```

■ Comparaciones temporales

{jsonb,tjsonb} {#=, #<>} {jsonb,tjsonb} → tbool

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' #=
jsonb '{"vehicleId": 1, "location": "Point(1 1)"}';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' #<>
tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```


16.12. Operaciones de caja delimitadora

■ Operadores topológicos

`{tjsonb,tstzspan} {&&, <@#, #@>, ~=, -|-} {tjsonb,tstzspan} → boolean`

Los operadores temporales contiene y contenido se marcan con un # para distinguirlos de los operadores JSONB correspondientes.

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' &&
  tstzspan '[2001-01-01, 2001-03-01]';
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' #@>
  timestampz '[2001-01-02]::tstzspan;
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' -|-
  tjsonb ' [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

■ Operadores de posición

`{tjsonb,tstzspan} {<<#, &<#, #>>, #&>} {tjsonb,tstzspan} → boolean`

```
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' <<#
  tstzspan '[2001-01-01, 2001-03-01]';
-- false
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #>>
  timestampz '[2001-01-01]::tstzspan;
-- true
SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #&>
  tjsonb ' [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

16.13. Agregaciones

Las funciones de agregación para valores JSONB temporales se ilustran a continuación.

■ Conteo temporal

`tCount(tjsonb) → {tintSeq,tintSeqSet}`

```
WITH Temp(temp) AS (
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb ' [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-03], (1@2001-01-03, 1@2001-01-04]}
```

■ Conteo en ventana

`wCount(tjsonb) → {tintSeq,tintSeqSet}`

```

WITH Temp(temp) AS (
  SELECT tjsonb '["vehicleId": 1, "location": "Point(1 1)"@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"@2001-01-03]' UNION
  SELECT tjsonb '["vehicleId": 1, "location": "Point(1 1)"@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"@2001-01-04]' )
SELECT wCount(Temp, interval '2 days')
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-05], (1@2001-01-05, 1@2001-01-06]}

```

16.14. Indexación

Pueden crearse índices GiST y SP-GiST para columnas de tabla de valores JSONB temporales. A continuación se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan la caja delimitadora de los valores JSONB temporales, que es un `stbox`, como todos los tipos espaciotemporales.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran los siguientes operadores:

- `<<, &<, &>, >>, <<|, &<|, |&>, |>>`, que solo consideran la dimensión espacial en los valores JSONB temporales,
- `<<#, &<#, #&>, #>>`, que solo consideran la dimensión temporal en los valores JSONB temporales,
- `&&, @>, <@, ~=, -|- y |=|`, que consideran tantas dimensiones como las que comparten la columna indexada y el argumento de la consulta.

Estos operadores trabajan sobre cajas delimitadoras, no sobre los valores completos.

Capítulo 17

Índices de Celdas H3 Temporales

El tipo de índice de celda H3 temporal `th3index` representa el movimiento de los objetos como una secuencia de identificadores de celdas hexagonales H3. **H3 de Uber** es un sistema de cuadrícula global discreta que tesela la esfera en 122 celdas base en la resolución 0 y subdivide cada celda en 7 hijos en cada resolución más fina (hasta la resolución 15). Cada celda tiene un identificador entero de 64 bits que codifica la celda base, la resolución y la posición jerárquica.

El tipo `th3index` comparte su representación en disco con `tbigint`: ambos almacenan un entero temporal de 64 bits. El tipo SQL distinto existe únicamente para que el verificador de tipos rechace las funciones específicas de H3 cuando se aplican a trayectorias `tbigint` arbitrarias (y viceversa). Las conversiones desde y hacia `tbigint` son *explícitas* (AS ASSIGNMENT) y de coerción binaria solamente, sin coste en tiempo de ejecución pero requieren un `::` explícito en SQL.

Como con otros tipos temporales, `th3index` tiene cuatro subtipos: `Instant`, secuencia discreta, secuencia continua con interpolación de pasos (las celdas H3 son discretas, la interpolación lineal carece de sentido), y `SequenceSet`.

17.1. Entrada y Salida

Un valor `th3index` se introduce utilizando la sintaxis temporal estándar con el identificador de celda H3 de 64 bits subyacente. El analizador acepta la forma hexadecimal canónica (la que produce la CLI de H3 y `h3_cell_to_string` en el proyecto original), con un prefijo `0x` opcional y como máximo 16 dígitos significativos. El valor debe además denotar algo del sistema de indexación H3, es decir, una celda, una arista dirigida o un vértice, de modo que un literal hexadecimal bien formado como `12345`, que no es ninguno de los tres, se rechaza en lugar de almacenarse como un valor sin ubicación. El hexadecimal es idiomático en el ecosistema H3 y es lo que emite la función de salida, todos los ejemplos siguientes lo usan.

```
SELECT th3index '831c02fffffffff@2001-01-01';
SELECT th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}';
SELECT th3index '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]';
SELECT th3index '[[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02],
[880326b885fffff@2001-01-03, 880326b88dfffff@2001-01-04]]';
```

El tipo acepta modificadores de tipo (`typmod`) para restringir una columna a un subtipo específico: `Instant`, `Sequence`, o `SequenceSet`.

```
SELECT th3index(Instant) '831c02fffffffff@2001-01-01';
SELECT th3index(SequenceSet) '{[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]}';
-- ERROR: el tipo temporal (Instant) no coincide con el tipo de columna (Sequence)
SELECT th3index(Sequence) '831c02fffffffff@2001-01-01';
```

Las funciones `asText`, `asBinary` y `asHexWKB` serializan un valor `th3index`, y `th3indexFromBinary` y `th3indexFromHexWKB` lo reconstruyen. La forma hexadecimal WKB permite que una columna de índice H3 temporal se transfiera de ida y vuelta mediante un COPY basado en texto.

```
SELECT asHexWKB(th3index '831c02fffffffff@2001-01-01');
SELECT th3indexFromHexWKB(asHexWKB(th3index '831c02fffffffff@2001-01-01'));
-- 831c02fffffffff@2001-01-01
```

17.2. Conversiones

Un valor `th3index` comparte la misma representación en disco que `tbigint`. La conversión por coerción binaria entre ellos es *explícita*, requiere `::`, para que las funciones específicas de H3 no puedan ser invocadas silenciosamente sobre trayectorias `tbigint` arbitrarias.

- Convertir entre un `th3index` y un `tbigint` (explícita, sin coste)

`tbigint::th3index`

`th3index::tbigint`

```
SELECT (tbigint '590464338553208831@2001-01-01')::th3index;
-- 831c02fffffffff@2001-01-01
SELECT ((th3index '831c02fffffffff@2001-01-01')::tbigint)::th3index
= th3index '831c02fffffffff@2001-01-01';
-- true
```

- Convertir una celda H3 temporal en un punto temporal geodésico o planar de los centroides de las celdas. Las dos sobrecargas dan al usuario una elección entre la representación geodésica (`tgeogpoint`) y la planar SRID 4326 (`tgeompoint`).

`th3index::tgeogpoint`

`th3index::tgeompoint`

```
SELECT (th3index '831c02fffffffff@2001-01-01')::tgeogpoint;
SELECT (th3index '831c02fffffffff@2001-01-01')::tgeompoint;
```

17.3. Cobertura de Geometrías Estáticas

Estas funciones convierten una geometría ordinaria (no temporal) en la celda o el conjunto de celdas H3 que la cubren a una resolución dada, y comprueban si una trayectoria H3 temporal entra alguna vez en ese conjunto de celdas. La salida de conjunto de celdas junto con el predicado de intersección «alguna vez» forman el prefiltro espacial multiplataforma usado por las consultas portables de BerlinMOD.

- Devuelve la celda H3 que contiene una geometría de punto a la resolución dada. La geometría debe ser un `POINT`.

`geoToH3Cell(geometry, integer) → h3index`

```
SELECT geoToH3Cell(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- 871fa4418ffffff
```

- Devuelve el conjunto de celdas H3 que cubren una geometría a la resolución dada. Se admiten todos los tipos de geometría: un `POINT` produce una sola celda, una `LINESTRING` muestrea sus segmentos, un `POLYGON` rellena su interior, y los valores `MULTI*` / `GEOMETRYCOLLECTION` devuelven la unión de sus componentes.

`geoToH3IndexSet(geometry, integer) → h3indexset`

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- {"871fa4418ffffff"}
```

- Devuelve si una trayectoria H3 temporal toma alguna vez una de las celdas de un conjunto de celdas. Combinada con `geoToH3IndexSet` responde si un viaje pasa alguna vez por una región, sin materializar la geometría de la trayectoria, el prefiltro robusto y conservador para el predicado exacto `eIntersects(geometry, th3index)`.

`eEq(h3indexset, th3index) → boolean`

`h3indexset ?= th3index → boolean`

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7) ?=
th3index '871fa4418ffffff@2001-01-01';
-- true
```

17.4. Funciones de Acceso

- Devolver el primer o el último identificador de celda H3 de un valor temporal

startValue(th3index) → h3index

endValue(th3index) → h3index

```
SELECT startValue(th3index '831c02fffffffff@2001-01-01');
-- 831c02fffffffff
SELECT endValue(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}');
-- 831c00fffffffff
```

- Devolver el n-ésimo identificador de celda H3 distinto (indexado desde 1). Devuelve NULL si está fuera del rango.

valueN(th3index, integer) → h3index

```
SELECT valueN(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}', 1);
-- 831c02fffffffff
SELECT valueN(th3index '831c02fffffffff@2001-01-01', 99);
-- NULL
```

- Devolver el conjunto de identificadores de celda H3 distintos que toma la trayectoria

getValues(th3index) → h3indexset

```
SELECT getValues(th3index '831c02fffffffff@2001-01-01');
-- {831c02fffffffff}
```

- Devolver el identificador de celda H3 en una marca de tiempo dada, utilizando interpolación de pasos

valueAtTimestamp(th3index, timestamptz) → h3index

```
SELECT valueAtTimestamp(th3index
  '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-05]',
  '2001-01-03');
-- 831c02fffffffff
```

17.5. Inspección de Índices

- Devolver la resolución temporal (0–15) de cada celda en una trayectoria

th3GetResolution(th3index) → tint

```
SELECT th3GetResolution(th3index '831c02fffffffff@2001-01-01');
-- 3@2001-01-01
```

- Devolver el número de celda base temporal (0–121) de cada celda en una trayectoria

th3GetBaseCellNumber(th3index) → tint

```
SELECT th3GetBaseCellNumber(th3index '831c02fffffffff@2001-01-01');
```

- Devolver un booleano temporal que indica en cada instante si el valor es una celda H3 válida

isValidCell(th3index) → tbool

```
SELECT isValidCell(th3index '0@2001-01-01');
-- f@2001-01-01
```

- Devolver un booleano temporal que indica si cada celda tiene orientación de Clase III. La Clase III se alterna con la resolución: las resoluciones impares son de clase III, las pares son de clase II.

th3IsResClassIii(th3index) → tbool

```
SELECT th3IsResClassIii(th3index '871fa44a8ffffff@2001-01-01');
-- t@2001-01-01
```

- Devolver un booleano temporal que indica si cada celda es un pentágono (12 celdas por resolución son pentágonos; las restantes 110 en cada celda base descienden como hexágonos)

th3IsPentagon(th3index) → tbool

```
SELECT th3IsPentagon(th3index '831c00fffffffff@2001-01-01');
-- t@2001-01-01
SELECT th3IsPentagon(th3index '831c02fffffffff@2001-01-01');
-- f@2001-01-01
```

17.6. Jerarquía

- Devolver la celda padre temporal en la resolución dada. La forma sin argumento desciende a la siguiente resolución más gruesa.

th3CellToParent(th3index, integer) → th3index

th3CellToParent(th3index) → th3index

```
SELECT th3GetResolution(
  th3CellToParent(th3index '8a2a1072b59ffff@2001-01-01', 5));
-- 5@2001-01-01
```

- Devolver la celda hijo central temporal en la resolución dada. La forma sin argumento desciende a la siguiente resolución más fina.

th3CellToCenterChild(th3index, integer) → th3index

th3CellToCenterChild(th3index) → th3index

```
SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01', 8);
-- 881fa44a81ffffff@2001-01-01
-- Without a resolution the child is one level finer.
SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01');
-- 881fa44a81ffffff@2001-01-01
```

- Devolver la posición ordinal (basada en 0) de cada celda entre los hijos de su padre en la resolución dada

th3CellToChildPos(th3index, integer) → tbigint

```
SELECT th3CellToChildPos(th3index '871fa44a8ffffff@2001-01-01', 6);
-- 0@2001-01-01
```

- Devolver la celda hijo temporal en una posición ordinal dada del padre dado, en la resolución de hijo dada. Los dos operandos temporales se sincronizan sobre su eje de tiempo compartido.

th3ChildPosToCell(tbigint, th3index, integer) → th3index

```
SELECT th3ChildPosToCell(tbigint '0@2001-01-01', th3index '871fa44a8ffffff@2001-01-01', 8) ←
;
-- 881fa44a81ffffff@2001-01-01
```

17.7. Conversiones de Latitud/Longitud

- Devolver la celda H3 temporal en la resolución dada que contiene cada punto de un punto temporal geodésico o planar (el SRID debe ser 4326 para la sobrecarga de tgeompoint)

th3index(tgeompoint,integer) → th3index

th3index(tgeompoint,integer) → th3index

```
SELECT th3GetResolution(th3index(
  tgeompoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT th3GetResolution(th3index(
  tgeompoint 'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
```

- Devolver el centroide geodésico temporal de cada celda en una trayectoria. Una sobrecarga planar (SRID 4326) está disponible bajo el nombre th3CellToLatlngTgeompoint.

th3CellToLatlng(th3index) → tgeompoint

th3CellToLatlngTgeompoint(th3index) → tgeompoint

```
SELECT asText(th3CellToLatlng(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
SELECT asText(th3CellToLatlngTgeompoint(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
```

- Devolver el límite poligonal temporal de cada celda en una trayectoria, como una geografía temporal

th3CellToBoundary(th3index) → tgeography

```
SELECT ST_GeometryType(getValue(th3CellToBoundary(th3index '871fa44a8ffffff@2001-01-01'))) ←
  ::geometry);
-- ST_Polygon
-- A hexagon closes on its first vertex, so it has seven points.
SELECT ST_NPoints(getValue(th3CellToBoundary(th3index '871fa44a8ffffff@2001-01-01')):: ←
  geometry);
-- 7
```

17.8. Aristas Dirigidas

Las aristas dirigidas H3 son a su vez identificadores de 64 bits (codificados de manera diferente a los identificadores de celda, isValidDirectedEdge los distingue). El tipo th3index de MobilityDB transporta las aristas dirigidas por convención: el mismo tipo, interpretado a través de las funciones de acceso específicas de las aristas.

- Devolver un booleano temporal que indica si dos celdas temporales son vecinas en la cuadrícula en cada instante. Los dos operandos se sincronizan.

th3AreNeighborCells(th3index,th3index) → tbool

```
SELECT th3AreNeighborCells(
  th3index '880326b885ffffff@2001-01-01',
  th3index '880326b88dffffff@2001-01-01');
-- t@2001-01-01
```

- Devolver un identificador de arista dirigida temporal a partir de un par de celdas temporales origen/destino

th3CellsToDirectedEdge(th3index,th3index) → th3index

```
SELECT th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44a8ffffff@2001-01-01');
-- 1671fa44a8ffffff@2001-01-01
```

- Devolver un booleano temporal que indica en cada instante si el valor es una arista dirigida H3 válida

isValidDirectedEdge(th3index) → tbool

```
SELECT isValidDirectedEdge(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44a8ffffff@2001-01-01'));
-- t@2001-01-01
SELECT isValidDirectedEdge(th3index '871fa44a8ffffff@2001-01-01');
-- f@2001-01-01
```

- Devolver la celda origen o destino temporal de una arista dirigida temporal

th3GetDirectedEdgeOrigin(th3index) → th3index

th3GetDirectedEdgeDestination(th3index) → th3index

```
SELECT th3GetDirectedEdgeOrigin(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44a8ffffff@2001-01-01'));
-- 871fa44a8ffffff@2001-01-01
SELECT th3GetDirectedEdgeDestination(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44a8ffffff@2001-01-01'));
-- 871fa44a8ffffff@2001-01-01
```

- Devolver el límite poligonal temporal de cada arista dirigida en una trayectoria, como una geografía temporal

th3DirectedEdgeToBoundary(th3index) → tgeography

```
SELECT ST_NPoints(getValue(th3DirectedEdgeToBoundary(
    th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44a8ffffff@2001-01-01'))::geometry);
-- 3
```

17.9. Vértices

- Devolver el vértice H3 temporal en el número de vértice dado (0–5 para hexágonos, 0–4 para pentágonos)

th3CellToVertex(th3index, integer) → th3index

```
SELECT th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0);
-- 2071fa44a8ffffff@2001-01-01
```

- Devolver las coordenadas geodésicas temporales de cada vértice en una trayectoria

th3VertexToLatLng(th3index) → tgeogpoint

```
SELECT asText(th3VertexToLatLng(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0))
    , 6);
-- POINT(4.280027 50.807655)@2001-01-01
```

- Devolver un booleano temporal que indica en cada instante si el valor es un vértice H3 válido

isValidVertex(th3index) → tbool

```
SELECT isValidVertex(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0));
-- t@2001-01-01
```


17.10. Recorrido de la Cuadrícula

- Devolver la distancia temporal de saltos en la cuadrícula (número de pasos de una sola celda) entre dos celdas temporales. De forma equivalente, el operador `<->` devuelve la misma distancia. Los dos operandos se sincronizan sobre su eje de tiempo compartido.

```
th3GridDistance(th3index,th3index) → tbigint
```

```
th3index <-> th3index → tbigint
```

```
SELECT th3index '880326b885fffff@2001-01-01'
      <-> th3index '880326b88dfffff@2001-01-01';
-- 1@2001-01-01
```

- Convertir entre una celda temporal y un par de coordenadas locales (I, J) temporales relativas a una celda temporal de origen. La coordenada local se expone como un `tgeompoint` con el eje I en el slot x y J en y.

```
th3CellToLocalIj(th3index,th3index) → tgeompoint
```

```
th3LocalIjToCell(th3index,tgeompoint) → th3index
```

```
SELECT asText(th3CellToLocalIj(th3index '871fa44a8fffff@2001-01-01',
      th3index '871fa44aefffff@2001-01-01'));
-- POINT(497 320)@2001-01-01
-- The two are inverse of each other.
SELECT th3LocalIjToCell(th3index '871fa44a8fffff@2001-01-01',
      th3CellToCell(th3index '871fa44a8fffff@2001-01-01',
      th3index '871fa44aefffff@2001-01-01'));
-- 871fa44aefffff@2001-01-01
```

17.11. Métricas

Las tres funciones métricas aceptan un argumento opcional de unidad. Para áreas: `km2` (por defecto), `m2`, o `rads2`. Para longitudes: `km` (por defecto), `m`, o `rads`. Pasar una unidad de área a una función de longitud (o viceversa) provoca un error.

- Devolver el área temporal de cada celda en una trayectoria, en la unidad solicitada

```
th3CellArea(th3index,text='km2') → tfloat
```

```
SELECT th3CellArea(th3index '871fa44a8fffff@2001-01-01');
-- 4.441914270110932@2001-01-01
SELECT th3CellArea(th3index '871fa44a8fffff@2001-01-01', 'm2');
-- 4441914.270110932@2001-01-01
```

- Devolver la longitud temporal de cada arista dirigida en una trayectoria, en la unidad solicitada

```
th3EdgeLength(th3index,text='km') → tfloat
```

```
SELECT th3EdgeLength(th3CellsToDirectedEdge(th3index '871fa44a8fffff@2001-01-01',
      th3index '871fa44aefffff@2001-01-01'));
-- 1.272084901305088@2001-01-01
```

- Devolver la distancia temporal del gran círculo entre dos puntos geodésicos temporales (no entre celdas). Los dos operandos se sincronizan.

```
greatCircleDistance(tgeogpoint,tgeogpoint,text='km') → tfloat
```

```
SELECT greatCircleDistance(tgeogpoint 'Point(0 0)@2001-01-01',
      tgeogpoint 'Point(1 0)@2001-01-01');
-- 111.19505197522943@2001-01-01
```

17.12. Comparaciones

17.12.1. Comparaciones Siempre y Alguna Vez

El operando de prueba puede ser un identificador de celda H3 `h3index` simple o otra trayectoria `th3index`; ambos son soportados en cualquier posición de operando.

- Devolver `true` si una celda H3 temporal es alguna vez igual a la prueba en al menos un instante

```
eEq({h3index,th3index},th3index) → boolean
```

```
eEq(th3index,h3index) → boolean
```

```
{h3index,th3index} ?= th3index
```

```
th3index ?= h3index
```

```
SELECT th3index
      '[831c02fffffffff@2001-01-01, 880326b885fffff@2001-01-05]'
      ?= 880326b885fffff::h3index;
-- true
```

- Devolver `true` si una celda H3 temporal es siempre igual a la prueba a lo largo de todo su eje de tiempo

```
aEq({h3index,th3index},th3index) → boolean
```

```
aEq(th3index,h3index) → boolean
```

```
{h3index,th3index} %= th3index
```

```
th3index %= h3index
```

```
SELECT aEq(th3index '871fa44a8ffffffff@2001-01-01', h3index '871fa44a8ffffffff');
-- t
```

- Devolver `true` si una celda H3 temporal es alguna vez diferente de la prueba

```
eNe({h3index,th3index},th3index) → boolean
```

```
eNe(th3index,h3index) → boolean
```

```
SELECT eNe(th3index '871fa44a8ffffffff@2001-01-01', h3index '871fa44aeffffffff');
-- t
```

- Devolver `true` si una celda H3 temporal es siempre diferente de la prueba

```
aNe({h3index,th3index},th3index) → boolean
```

```
aNe(th3index,h3index) → boolean
```

```
SELECT aNe(th3index '871fa44a8ffffffff@2001-01-01', h3index '871fa44aeffffffff');
-- t
```

17.12.2. Comparaciones Temporales

- Devolver un booleano temporal que da la igualdad (o desigualdad) por instante entre una celda H3 temporal y una prueba

```
tEq({h3index,th3index},th3index) → tbool
```

```
tEq(th3index,h3index) → tbool
```

```
tNe({h3index,th3index},th3index) → tbool
```

```
tNe(th3index,h3index) → tbool
```

```
SELECT tEq(th3index '871fa44a8ffffffff@2001-01-01', h3index '871fa44a8ffffffff');
-- t@2001-01-01
SELECT tNe(th3index '871fa44a8ffffffff@2001-01-01', h3index '871fa44a8ffffffff');
-- f@2001-01-01
```

17.13. Operadores de Caja Envolvente

Los siguientes operadores comparan la *caja envolvente* espaciotemporal de las celdas H3 temporales, no la inclusión jerárquica de celdas. `th3index` es un tipo temporal espacial: su caja envolvente es un `stbox` que combina la extensión espacial geodésica de los límites de las celdas (su envolvente de longitud/latitud en SRID 4326) con el período temporal `tstzspan` de la trayectoria. Los operadores a continuación comparan esa caja a lo largo de los ejes espacial y/o temporal. Consulte la sección de indexación para las clases de operadores que hacen estos operadores indexables.

nota

El operador `@>` prueba la inclusión de la caja envolvente espaciotemporal (extensión espacial y tiempo juntos), *no* la inclusión jerárquica de H3. Para probar si una celda es el padre de otra en una resolución dada, use `th3CellToParent`.

- Solapamiento de caja envolvente, contiene, contenido por, igual y adyacente

```
th3index && {tstzspan,stbox,th3index} → boolean
th3index @> {tstzspan,stbox,th3index} → boolean
th3index <@ {tstzspan,stbox,th3index} → boolean
th3index ~= {tstzspan,stbox,th3index} → boolean
th3index -|- {tstzspan,stbox,th3index} → boolean
```

```
SELECT th3index '871fa44a8ffffff@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' ~= th3index '871fa44a8ffffff@2001-01-01';
-- t
```

- Posición relativa a lo largo de los ejes espaciales (estrictamente a la izquierda, solapa a la izquierda, solapa a la derecha, estrictamente a la derecha; estrictamente abajo, solapa abajo, solapa arriba, estrictamente arriba)

```
th3index << {stbox,th3index} → boolean
th3index <& {stbox,th3index} → boolean
th3index &> {stbox,th3index} → boolean
th3index >> {stbox,th3index} → boolean
th3index <<| {stbox,th3index} → boolean
th3index <&| {stbox,th3index} → boolean
th3index |&> {stbox,th3index} → boolean
th3index |>> {stbox,th3index} → boolean
```

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<
  th3index '871fa44aeffffff@2001-01-01';
-- f
```

- Posición relativa a lo largo del eje de tiempo (estrictamente antes, solapa antes, estrictamente después, solapa después)

```
th3index <<# {tstzspan,stbox,th3index} → boolean
th3index <&# {tstzspan,stbox,th3index} → boolean
th3index #>> {tstzspan,stbox,th3index} → boolean
th3index #&> {tstzspan,stbox,th3index} → boolean
```

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<#
  th3index '871fa44aeffffff@2001-01-05';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' #>>
  th3index '871fa44aeffffff@2001-01-05';
-- f
```

17.14. Funciones que devuelven conjuntos

Las siguientes funciones estáticas devuelven un `h3indexset` (o `intset` para `h3GetIcosahedronFaces`), el tipo compañero de colección finita de `h3index`. Operan sobre un `h3index` estático (o un `h3indexset`) y no están elevadas a `th3index`; el levantamiento temporal está diferido a la espera de una primitiva `tset<T>`.

- Todas las celdas dentro de `k` pasos de cuadrícula del origen (incluyendo el origen cuando `k=0`).

`h3GridDisk(h3index, integer) → h3indexset`

```
SELECT h3GridDisk(h3index '871fa44a8fffffff', 1);
-- {"871fa44a8fffffff", "871fa44a9fffffff", "871fa44aafffffff", "871fa44abfffffff",
-- "871fa44acfffffff", "871fa44adfffffff", "871fa44aefffffff"}
```

- Las celdas a *exactamente* `k` pasos de cuadrícula del origen. Falla cerca de pentágonos.

`h3GridRing(h3index, integer) → h3indexset`

```
SELECT h3GridRing(h3index '871fa44a8fffffff', 1);
-- {"871fa44a9fffffff", "871fa44aafffffff", "871fa44abfffffff", "871fa44acfffffff",
-- "871fa44adfffffff", "871fa44aefffffff"}
```

- El camino inclusivo de celdas entre dos celdas de la misma resolución.

`h3GridPathCells(h3index, h3index) → h3indexset`

```
SELECT h3GridPathCells(h3index '871fa44a8fffffff', h3index '871fa44b1fffffff');
-- {"871fa44a8fffffff", "871fa44a9fffffff", "871fa44aa8fffffff", "871fa44ab8fffffff",
-- "871fa44b1fffffff"}
```

- Todas las celdas hijas de una celda en la resolución objetivo.

`h3CellToChildren(h3index, integer) → h3indexset`

```
SELECT h3CellToChildren(h3index '831c02ffffffff', 4);
-- {"841c021ffffffff", "841c023ffffffff", "841c025ffffffff", "841c027ffffffff",
-- "841c029ffffffff", "841c02bffffffff", "841c02dffffffff"}
```

- Representación compactada de un conjunto de celdas, las celdas más finas se fusionan en su padre cuando el conjunto hexagonal completo de hermanas está presente.

`h3CompactCells(h3indexset) → h3indexset`

```
SELECT h3CompactCells(h3CellToChildren(h3index '831c02ffffffff', 4));
-- {"831c02ffffffff"}
```

- Representación descompactada a la resolución dada, la inversa de `h3CompactCells`.

`h3UncompactCells(h3indexset, integer) → h3indexset`

```
SELECT numValues(h3UncompactCells(
  h3CompactCells(h3CellToChildren(h3index '831c02ffffffff', 4)), 4));
-- 7
```

- Todas las aristas dirigidas salientes de una celda (6 para hexágonos, 5 para pentágonos).

`h3OriginToDirectedEdges(h3index) → h3indexset`

```
SELECT h3OriginToDirectedEdges(h3index '871fa44a8fffffff');
-- {"1171fa44a8fffffff", "1271fa44a8fffffff", "1371fa44a8fffffff",
-- "1471fa44a8fffffff", "1571fa44a8fffffff", "1671fa44a8fffffff"}
```

- Todos los vértices de una celda (6 para hexágonos, 5 para pentágonos).

`h3CellToVertexes(h3index) → h3indexset`

```
SELECT h3CellToVertexes(h3index '871fa44a8ffffff');
-- {"2071fa44a8ffffff", "2171fa44a8ffffff", "2271fa44a8ffffff",
-- "2371fa44a8ffffff", "2471fa44a8ffffff", "2571fa44a8ffffff"}
```

- Los índices de caras del icosaedro (0..19) intersectadas por una celda.

`h3GetIcosahedronFaces(h3index) → intset`

```
SELECT h3GetIcosahedronFaces(h3index '831c02ffffffff');
-- {6, 11}
```

17.15. Relaciones Espaciales

Las relaciones espaciales evalúan un argumento `th3index` a través de la geometría de frontera de celda en cada instante, la huella hexagonal de cada celda, devuelta por `th3CellToBoundary`— y luego aplican la relación correspondiente de los tipos geométricos temporales. Cada relación tiene tres formas cuantificadas: la forma *alguna vez* con prefijo `e` devuelve `true` cuando la relación se cumple en algún instante, la forma *siempre* con prefijo `a` cuando se cumple en todos los instantes, y la forma *temporal* con prefijo `t` devuelve un `tbool` con el resultado instante a instante. Todas las formas aceptan los tres órdenes de argumentos (`geometry, th3index`), (`th3index, geometry`) y (`th3index, th3index`).

- Si una huella contiene a la otra (solo interiores)

`eContains, aContains, tContains`

```
SELECT eContains(geometry 'SRID=4326;Polygon((-123 37,-123 38,-122 38,-122 37,-123 37))',
  th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]');
-- true
SELECT tContains(geometry 'SRID=4326;Polygon((-123 37,-123 38,-122 38,-122 37,-123 37))',
  th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]');
-- [t@2001-01-01, t@2001-01-02]
```

- Si una huella cubre a la otra (frontera incluida)

`eCovers, aCovers, tCovers`

```
SELECT eCovers(geometry 'SRID=4326;Polygon((-123 37,-123 38,-122 38,-122 37,-123 37))',
  th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]');
-- true
SELECT tCovers(geometry 'SRID=4326;Polygon((-123 37,-123 38,-122 38,-122 37,-123 37))',
  th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]');
-- [t@2001-01-01, t@2001-01-02]
```

- Si las huellas no comparten ningún punto

`eDisjoint, aDisjoint, tDisjoint`

```
SELECT eDisjoint(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[88283082b3ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]');
-- true
SELECT tDisjoint(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[88283082b3ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]');
-- [t@2001-01-01, t@2001-01-02]
```

- Si las huellas comparten al menos un punto

`eIntersects, aIntersects, tIntersects`

```
SELECT eIntersects(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[8828308281ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]');
-- true
SELECT tIntersects(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[8828308281ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]');
-- [t@2001-01-01, f@2001-01-02]
```

- Si las huellas comparten un punto de frontera pero ningún punto interior

eTouches, aTouches, tTouches

```
SELECT eTouches(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[8828308283ffffff@2001-01-01, 8828308285ffffff@2001-01-02]');
-- true
SELECT tTouches(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[8828308283ffffff@2001-01-01, 8828308285ffffff@2001-01-02]');
-- [t@2001-01-01, f@2001-01-02]
```

- Si las huellas se encuentran dentro de una distancia dada, pasada como tercer argumento

eDwithin, aDwithin, tDwithin (... , dist float)

```
SELECT eDwithin(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[88283082b3ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]', 5000);
-- true
SELECT tDwithin(th3index '[8828308281ffffff@2001-01-01, 8828308283ffffff@2001-01-02]',
  th3index '[88283082b3ffffff@2001-01-01, 88283082b1ffffff@2001-01-02]', 5000);
-- [t@2001-01-01, t@2001-01-02]
```

```
SELECT eContains(
  geometry 'SRID=4326;Polygon((4.20 50.70, 4.50 50.70, 4.50 50.90, 4.20 50.90, 4.20 50.70))' ←
  ,
  th3index '871fa44a8ffffff@2001-01-01');
-- true
```

17.16. Agregaciones

Las funciones de agregación para celdas H3 temporales se ilustran a continuación.

- Conteo temporal

tCount(th3index) → {tintSeq,tintSeqSet}

```
WITH Temp(temp) AS (
  SELECT th3index '[831c02fffffffff@2001-01-01, 831c02fffffffff@2001-01-03]' UNION
  SELECT th3index '[831c00fffffffff@2001-01-02, 831c00fffffffff@2001-01-04]' UNION
  SELECT th3index '[871fa44a8ffffff@2001-01-03, 871fa44a8ffffff@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}
```

- Conteo en ventana

wCount(th3index) → {tintSeq,tintSeqSet}

```
WITH Temp(temp) AS (
  SELECT th3index '[831c02fffffffff@2001-01-01, 831c02fffffffff@2001-01-03]' UNION
  SELECT th3index '[831c00fffffffff@2001-01-02, 831c00fffffffff@2001-01-04]' UNION
  SELECT th3index '[871fa44a8ffffff@2001-01-03, 871fa44a8ffffff@2001-01-05]' )
```

```
SELECT wCount(Temp, interval '1 day')
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05, 1@2001-01-06]}
```

17.17. Indexación

Las clases de operadores GiST y SP-GiST están definidas para que los operadores de caja envolvente de la sección anterior puedan ser utilizados con los índices correspondientes.

- Clase de operadores GiST para celdas H3 temporales. Indexa una caja envolvente espaciotemporal `stbox` por fila (la extensión geodésica de los límites de las celdas junto con el período temporal).

```
CREATE INDEX ON trips USING gist(cells);           -- usa th3_rtree_ops por defecto
CREATE INDEX ON trips USING gist(cells th3_rtree_ops);
```

- Clases de operadores SP-GiST para celdas H3 temporales, indexadas por la caja envolvente espaciotemporal `stbox`. `th3_quadtree_ops` es la predeterminada (partición por quadtree); `th3_kdtree_ops` utiliza partición por árbol k-d y debe ser nombrada explícitamente.

```
CREATE INDEX ON trips USING spgist(cells);         -- usa th3_quadtree_ops por defecto
CREATE INDEX ON trips USING spgist(cells th3_kdtree_ops);
```

Capítulo 18

Índices de Celdas QUADBIN Temporales

El tipo de índice de celda QUADBIN temporal `tquadbin` representa el movimiento de los objetos como una secuencia de identificadores de celdas cuadradas QUADBIN. **QUADBIN de CARTO** es un sistema de cuadrícula cuadrada jerárquica construido sobre el esquema de teselas *slippy-map* (*web-mercator*): una única celda mundial en la resolución 0 se subdivide en cuatro hijos iguales en cada resolución más fina (hasta la resolución 26). Cada celda tiene un identificador entero de 64 bits que codifica una etiqueta de cabecera, la resolución y las coordenadas de tesela de la celda.

El tipo `tquadbin` comparte su representación en disco con `tbigint`: ambos almacenan un entero temporal de 64 bits. El tipo SQL distinto existe únicamente para que el verificador de tipos rechace las funciones específicas de QUADBIN cuando se aplican a trayectorias `tbigint` arbitrarias (y viceversa). Las conversiones desde y hacia `tbigint` son *explícitas* (`AS ASSIGNMENT`) y de coerción binaria solamente, sin coste en tiempo de ejecución pero requieren un `::` explícito en SQL.

Como con otros tipos temporales, `tquadbin` tiene cuatro subtipos: `Instant`, secuencia discreta, secuencia continua con interpolación de pasos (las celdas QUADBIN son discretas, la interpolación lineal carece de sentido), y `SequenceSet`.

18.1. Entrada y Salida

Un valor `tquadbin` se introduce utilizando la sintaxis temporal estándar con el identificador de celda QUADBIN de 64 bits subyacente. El analizador acepta la forma hexadecimal canónica, con un prefijo `0x` opcional, en cualquiera de las dos cajas de letra, tal como la lee la implementación de referencia de CARTO. El hexadecimal es idiomático en el ecosistema QUADBIN y es lo que emite la función de salida, todos los ejemplos siguientes lo usan. Como referencia, el valor hexadecimal `480fffffffffffffff` empleado en los ejemplos es la celda mundial de resolución 0, igual al valor decimal `5192650370358181887`.

```
SELECT tquadbin '480fffffffffffffff@2001-01-01';
SELECT tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}';
SELECT tquadbin '[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02]';
SELECT tquadbin '([480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02],
[48a6227affffffffff@2001-01-03, 485623fffffffffffffff@2001-01-04])';
```

El tipo acepta modificadores de tipo (`typmod`) para restringir una columna a un subtipo específico: `Instant`, `Sequence`, o `SequenceSet`.

```
SELECT tquadbin(Instant) '480fffffffffffffff@2001-01-01';
SELECT tquadbin(SequenceSet) '([480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02])' ←
;
-- ERROR: el tipo temporal (Instant) no coincide con el tipo de columna (Sequence)
SELECT tquadbin(Sequence) '480fffffffffffffff@2001-01-01';
```

Las funciones `asText`, `asBinary` y `asHexWKB` serializan un valor `tquadbin`, y `tquadbinFromBinary` y `tquadbinFromHex` lo reconstruyen. La forma hexadecimal WKB permite que una columna de índice QUADBIN temporal se transfiera de ida y vuelta mediante un COPY basado en texto.


```
SELECT asHexWKB(tquadbin '480fffffffffffffff@2001-01-01');
SELECT tquadbinFromHexWKB(asHexWKB(tquadbin '480fffffffffffffff@2001-01-01'));
-- 480fffffffffffffff@2001-01-01
```

La representación *MF-JSON* está disponible mediante `asMFJSON(tquadbin)` y `tquadbinFromMFJSON(text)`. La carga útil de la celda se representa como el identificador int64 de la celda en el arreglo `values`, la misma representación que utiliza `tbigint`.

```
SELECT asMFJSON(tquadbin '480fffffffffffffff@2001-01-01');
SELECT tquadbinFromMFJSON(asMFJSON(tquadbin '480fffffffffffffff@2001-01-01'));
-- 480fffffffffffffff@2001-01-01
```

18.2. Conversiones

Un valor `tquadbin` comparte la misma representación en disco que `tbigint`. La conversión por coerción binaria entre ellos es *explícita*, requiere `::`, para que las funciones específicas de QUADBIN no puedan ser invocadas silenciosamente sobre trayectorias `tbigint` arbitrarias.

- Convertir entre un `tquadbin` y un `tbigint` (explícito, sin coste)

```
tbigint::tquadbin
tquadbin::tbint
```

```
SELECT (tbint '5192650370358181887@2001-01-01')::tquadbin;
-- 480fffffffffffffff@2001-01-01
SELECT ((tquadbin '480fffffffffffffff@2001-01-01')::tbint)::tquadbin
= tquadbin '480fffffffffffffff@2001-01-01';
-- true
```

- Convertir una celda QUADBIN temporal a un punto planar temporal (SRID 4326) de los centroides de las celdas. La conversión delega en `tquadbinCellToPoint`.

```
tquadbin::tgeompoint
```

```
SELECT (tquadbin '480fffffffffffffff@2001-01-01')::tgeompoint;
```

18.3. Celdas Estáticas

El tipo estático complementario `quadbin` contiene un único identificador de celda QUADBIN (no temporal); `quadbinset` contiene un conjunto finito de ellos. Comparten la misma representación textual hexadecimal / decimal que el tipo temporal. El tipo estático lleva los operadores estándar de igualdad / orden (una clase de operadores `btree` y una clase `hash`), y expone los predicados de validez siguientes.

- Devuelve si un valor es un identificador QUADBIN estructuralmente válido (etiqueta de cabecera y campo de resolución correctos)

```
isValidIndex(quadbin) → boolean
```

```
SELECT isValidIndex(quadbin '480fffffffffffffff');
-- t
-- The input function already rejects malformed text, so a value that is
-- structurally invalid can only arrive through the bigint conversion.
SELECT isValidIndex(0::bigint::quadbin);
-- f
```

- Devuelve si un valor es una celda QUADBIN válida (un índice válido cuyas coordenadas de tesela están dentro de los límites de su resolución)

`isValidCell(quadbin) → boolean`

```
SELECT isValidCell(quadbin '480fffffffffffffff');
-- true
SELECT isValidCell(quadbin '0');
-- false
```

18.4. Accesos

- Devuelve el primer o el último identificador de celda QUADBIN de un valor temporal

`startValue(tquadbin) → quadbin`

`endValue(tquadbin) → quadbin`

```
SELECT startValue(tquadbin '480fffffffffffffff@2001-01-01');
-- 480fffffffffffffff
SELECT endValue(tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}');
-- 48427fffffffffffffff
```

- Devuelve el n-ésimo identificador de celda QUADBIN distinto (índice base 1). Devuelve NULL si está fuera de rango.

`valueN(tquadbin, integer) → quadbin`

```
SELECT valueN(tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}', 1);
-- 480fffffffffffffff
SELECT valueN(tquadbin '{480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02}', 2);
-- 48427fffffffffffffff
SELECT valueN(tquadbin '480fffffffffffffff@2001-01-01', 99);
-- NULL
```

- Devuelve el conjunto de identificadores de celda QUADBIN distintos que toma la trayectoria

`getValues(tquadbin) → quadbinset`

```
SELECT getValues(tquadbin '480fffffffffffffff@2001-01-01');
-- {480fffffffffffffff}
SELECT getValues(tquadbin
  '[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-02,
  48a6227affffffff@2001-01-03]');
-- {480fffffffffffffff, 48427fffffffffffffff, 48a6227affffffff}
```

- Devuelve el identificador de celda QUADBIN en un instante dado, usando interpolación de pasos

`valueAtTimestamp(tquadbin, timestamptz) → quadbin`

```
SELECT valueAtTimestamp(tquadbin
  '[480fffffffffffffff@2001-01-01, 48427fffffffffffffff@2001-01-05]',
  '2001-01-03');
-- 480fffffffffffffff
```

18.5. Inspección del Índice

- Devuelve el nivel de resolución (zoom) (0–26) de una celda. La forma estática devuelve un `integer`; la forma temporal devuelve la resolución de cada celda de una trayectoria como un `tint`.

`quadbinGetResolution(quadbin) → integer`

`tquadbinGetResolution(tquadbin) → tint`

```
SELECT quadbinGetResolution(quadbin '480fffffffffffffff');
-- 0
SELECT tquadbinGetResolution(tquadbin '48427fffffffffffffff@2001-01-01');
-- 4@2001-01-01
```

- Devuelve un booleano temporal que indica en cada instante si el valor es una celda QUADBIN válida

`isValidCell(tquadbin) → tbool`

```
SELECT isValidCell(tquadbin '480fffffffffffffff@2001-01-01');
-- t@2001-01-01
```

18.6. Jerarquía

- Devuelve la celda padre en la resolución más gruesa dada. La forma estática toma un único quadbin; la forma temporal devuelve el padre de cada celda de una trayectoria.

`quadbinCellToParent(quadbin, integer) → quadbin`

`tquadbinCellToParent(tquadbin, integer) → tquadbin`

```
SELECT quadbinCellToParent(quadbin '48a6227affffffffff', 5);
-- 485623fffffffffffff
SELECT tquadbinGetResolution(
  tquadbinCellToParent(tquadbin '48a6227affffffffff@2001-01-01', 5));
-- 5@2001-01-01
```

- Devuelve las cuatro celdas hijas en la resolución más fina solicitada como un quadbinset. Es un auxiliar estático que devuelve un conjunto sobre un único quadbin; no tiene elevación temporal (los hijos son un conjunto por instante, diferido a la espera de una primitiva `tset<T>`).

`quadbinCellToChildren(quadbin, integer) → quadbinset`

```
SELECT quadbinCellToChildren(quadbin '480fffffffffffffff', 1);
-- {4813fffffffffffffff, 4817fffffffffffffff, 481bfffffffffffffff, 481fffffffffffffff}
```

- Devuelve la celda vecina en la misma resolución en la dirección dada (up / down / left / right)

`quadbinCellSibling(quadbin, text) → quadbin`

```
SELECT quadbinCellSibling(
  quadbinCellSibling(quadbin '48427fffffffffffffff', 'right'), 'left')
= quadbin '48427fffffffffffffff';
-- true
```

- Devuelve todas las celdas dentro de k pasos de cuadrícula del origen (inclusive del origen en $k=0$), como un quadbinset. En la cuadrícula cuadrada el disco en el paso k es el bloque cuadrado de $(2k+1) \times (2k+1)$ celdas centrado en el origen.

`quadbinGridDisk(quadbin, integer) → quadbinset`

```
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffffff', 0)); -- 1
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffffff', 1)); -- 9
SELECT numValues(quadbinGridDisk(quadbin '48a6227affffffffff', 2)); -- 25
```

18.7. Conversiones de Geometría

- Devuelve la celda QUADBIN en la resolución dada que contiene una geometría de punto. La geometría debe ser un POINT; las longitudes se interpretan en SRID 4326.

`geoToQuadbinCell(geometry, integer) → quadbin`

```
SELECT geoToQuadbinCell(geometry 'SRID=4326;POINT(4.35 50.85)', 10)
= quadbin '48a6227affffffff';
-- true
```

- Devuelve el centroide de una celda como un punto SRID 4326. La forma estática toma un único quadbin; la forma temporal devuelve el centroide de cada celda de una trayectoria como un tgeompoint.

`quadbinCellToPoint(quadbin) → geometry`

`tquadbinCellToPoint(tquadbin) → tgeompoint`

```
SELECT ST_AsText(quadbinCellToPoint(quadbin '48427fffffffffffff'));
-- POINT(-101.25 48.92249926375824)
SELECT asText(tquadbinCellToPoint(tquadbin
'{480fffffffffffff@2001-01-01, 48427fffffffffffff@2001-01-02}'));
-- {POINT(0 0)@2001-01-01,
-- POINT(-101.25 48.92249926375824)@2001-01-02}
```

- Devuelve el contorno poligonal cuadrado de una celda como una geometría SRID 4326. La forma estática toma un único quadbin; la forma temporal devuelve el contorno de cada celda de una trayectoria como un tgeometry.

`quadbinCellToBoundary(quadbin) → geometry`

`tquadbinCellToBoundary(tquadbin) → tgeometry`

```
SELECT ST_AsText(quadbinCellToBoundary(quadbin '480fffffffffffff'));
-- POLYGON((-180 -85.0511287798066,180 -85.0511287798066,
--          180 85.0511287798066,-180 85.0511287798066,
--          -180 -85.0511287798066))
```

18.8. Teselas y Quadkeys

QUADBIN está construido sobre el esquema de teselas slippy-map (web-mercator), por lo que una celda se corresponde directamente con un triple de coordenadas de tesela (x , y , z) y con la cadena quadkey en base 4 usada por los servidores de teselas. Estas conversiones son el puente específico de QUADBIN con el ecosistema de teselado.

- Construye una celda a partir de coordenadas de tesela slippy-map: columna x , fila y , y zoom z

`quadbinTileToCell(integer, integer, integer) → quadbin`

```
SELECT quadbinTileToCell(0, 0, 0) = quadbin '480fffffffffffff';
-- true
SELECT quadbinTileToCell(3, 5, 4) = quadbin '48427fffffffffffff';
-- true
SELECT quadbinTileToCell(524, 343, 10) = quadbin '48a6227affffffff';
-- true
```

- Descompone una celda en su triple de coordenadas de tesela, devuelto como un arreglo de enteros $\{x, y, z\}$, la inversa de `quadbinTileToCell`

`quadbinCellToTile(quadbin) → integer[]`

```
SELECT quadbinCellToTile(quadbin '480fffffffffffffff');
-- {0,0,0}
SELECT quadbinCellToTile(quadbin '48a6227a');
-- {524,343,10}
```

- Emite la cadena quadkey en base 4 de una celda (un dígito por nivel de zoom, primero el más significativo; la celda mundial de resolución 0 se corresponde con la cadena vacía). La forma estática toma un único quadbin; la forma temporal devuelve el quadkey de cada celda de una trayectoria como un ttext.

quadbinCellToQuadkey(quadbin) → text

tquadbinCellToQuadkey(tquadbin) → ttext

```
SELECT quadbinCellToQuadkey(quadbin '480fffffffffffffff');
-- (cadena vacía)
SELECT quadbinCellToQuadkey(quadbin '48427fffffffffffffff');
-- 0213
SELECT quadbinCellToQuadkey(quadbin '48a6227a');
-- 1202021322
```

18.9. Métricas

- Devuelve el área de una celda sobre la esfera, en metros cuadrados. La forma estática toma un único quadbin; la forma temporal devuelve el área de cada celda de una trayectoria como un tfloat. El área de la celda se reduce hacia los polos, ya que los cuadrados web-mercator cubren progresivamente menos terreno a latitudes más altas.

quadbinCellArea(quadbin) → float8

tquadbinCellArea(tquadbin) → tfloat

```
SELECT round(quadbinCellArea(quadbin '480fffffffffffffff')::numeric, 0);
-- 508164135963886
SELECT round(quadbinCellArea(quadbin '48427fffffffffffffff')::numeric, 0);
-- 2726563416104
```

18.10. Comparaciones

18.11. Agregaciones

Las funciones de agregación para celdas quadbin temporales se ilustran a continuación.

- Conteo temporal

tCount(tquadbin) → {tintSeq,tintSeqSet}

```
WITH Temp(temp) AS (
  SELECT tquadbin '[480fffffffffffffff@2001-01-01, 480fffffffffffffff@2001-01-03]' UNION
  SELECT tquadbin '[48427fffffffffffffff@2001-01-02, 48427fffffffffffffff@2001-01-04]' UNION
  SELECT tquadbin '[48a6227a@2001-01-03, 48a6227a@2001-01-05]' )
SELECT tCount(Temp)
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05]}
```

- Conteo en ventana

wCount(tquadbin) → {tintSeq,tintSeqSet}

```

WITH Temp(temp) AS (
  SELECT tquadbin '[480fffffffffffffff@2001-01-01, 480fffffffffffffff@2001-01-03)' UNION
  SELECT tquadbin '[48427fffffffffffffff@2001-01-02, 48427fffffffffffffff@2001-01-04)' UNION
  SELECT tquadbin '[48a6227afffffffffff@2001-01-03, 48a6227afffffffffff@2001-01-05)' )
SELECT wCount(Temp, interval '1 day')
FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05, 1@2001-01-06)}

```

18.12. Indexación

Se definen clases de operadores de árbol B y hash para que los operadores de comparación de la sección anterior puedan utilizarse con los índices correspondientes.

- Clase de operadores de árbol B por defecto para las celdas QUADBIN temporales, ordenando por el valor temporal subyacente.

```
CREATE INDEX ON trips USING btree(cells);      -- usa tquadbin_btree_ops por defecto
```

- Clase de operadores hash por defecto para las celdas QUADBIN temporales, usada por las uniones hash y la agregación hash.

```
CREATE INDEX ON trips USING hash(cells);      -- usa tquadbin_hash_ops por defecto
```

Capítulo 19

Nubes de Puntos Temporales

La extensión **pgPointCloud** almacena datos de nubes de puntos LIDAR en PostgreSQL mediante dos tipos estáticos: `pcpoint`, un único punto multidimensional cuyas dimensiones se describen mediante un *esquema* almacenado en la tabla de catálogo `pointcloud_formats`; y `pcpatch`, un lote comprimido de valores `pcpoint` que comparten el mismo `pcid` (identificador de esquema). Cada esquema fija las dimensiones presentes (X, Y, Z, Intensity, ...), su tipo numérico, escala y desplazamiento.

MobilityDB eleva estos tipos al mundo temporal mediante `tpcpoint` (un sensor LIDAR/GPS en movimiento) y `tpcpatch` (una serie temporal de grupos de puntos comprimidos). También añade los tipos de conjunto `pcpointset` / `pcpatchset` y el tipo de caja delimitadora espaciotemporal `tpcbox`. Una referencia completa de los tipos estáticos `pcpoint` y `pcpatch` se ofrece en la [documentación de pgPointCloud](#); este capítulo cubre las adiciones de MobilityDB sobre `pgPointCloud`.

Para usar los tipos descritos aquí, MobilityDB debe compilarse con la opción de CMake `POINTCLOUD=ON`. En tal compilación, el archivo `mobilitydb.control` generado declara `requires = 'postgis, pointcloud'`, de modo que un único CASCADE crea la pila completa:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE: installing required extension "postgis"
-- NOTICE: installing required extension "pointcloud"
```

19.1. Inicio rápido

Esta sección recorre un ejemplo mínimo de extremo a extremo: registrar un esquema, construir valores, elevar al tipo temporal, consultar con la superficie `bbox` e indexar. El objetivo es que cada sección posterior de este capítulo esté anclada a un uso concreto. Siga los pasos en orden sobre una base de datos nueva.

19.1.1. Registrar un esquema

Cada valor `pcpoint` y `pcpatch` lleva un `pcid` que se resuelve a través de `pointcloud_formats` en un esquema XML. El esquema declara las dimensiones (X, Y, Z, ...), su tipo numérico en disco y la escala por dimensión. Un esquema 3D mínimo con todas las dimensiones almacenadas como `int32` con escala unitaria:

```
INSERT INTO pointcloud_formats (pcid, srid, schema) VALUES (1, 4326, '
<?xml version="1.0" encoding="UTF-8"?>
<pc:PointCloudSchema xmlns:pc="http://pointcloud.org/schemas/PC/1.1">
  <pc:dimension><pc:position>1</pc:position><pc:size>4</pc:size>
    <pc:name>X</pc:name><pc:interpretation>int32_t</pc:interpretation>
    <pc:scale>1</pc:scale></pc:dimension>
  <pc:dimension><pc:position>2</pc:position><pc:size>4</pc:size>
    <pc:name>Y</pc:name><pc:interpretation>int32_t</pc:interpretation>
    <pc:scale>1</pc:scale></pc:dimension>
```

```
<pc:dimension><pc:position>3</pc:position><pc:size>4</pc:size>
  <pc:name>Z</pc:name><pc:interpretation>int32_t</pc:interpretation>
  <pc:scale>1</pc:scale></pc:dimension>
</pc:PointCloudSchema>');
```

Los esquemas son globales a la base de datos; normalmente se registran una sola vez durante la carga inicial de datos. El SRID es por esquema y es heredado por cada valor con ese `pcid`.

19.1.2. Construir valores base y temporales

MobilityDB incluye constructores ergonómicos sobre los `PC_MakePoint` / `PC_Patch` de `pgPointCloud`:

```
-- a single point at (1, 2, 3) under pcid=1
SELECT pcpoint(1, 1.0, 2.0, 3.0);

-- a patch holding two points
SELECT pcpatch(1, pcpoint(1, 1.0, 2.0, 3.0),
               pcpoint(1, 4.0, 5.0, 6.0));

-- a single-instant tpcpoint at (10, 20, 30) at 2024-01-01
SELECT tpcpoint(pcpoint(1, 10, 20, 30), '2024-01-01'::timestampz);

-- a 3-instant moving sensor track (step interpolation, the default for
-- pointcloud temporals because dimensions like Intensity don't interpolate
-- linearly the way coordinates do)
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 0, 0, 0), '2024-01-01'::timestampz),
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-02'::timestampz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-03'::timestampz)]);
```

Para sesiones largas, declare la columna con un *typmod* para que el `pcid` de cada fila se imponga en el momento del `INSERT`:

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
INSERT INTO scans VALUES (1,
  tpcpointSeq(ARRAY[
    tpcpoint(pcpoint(1, 0, 0, 0), '2024-01-01'::timestampz),
    tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-02'::timestampz)]),
  tpcpatch(pcpatch(1, pcpoint(1,1,1,1), pcpoint(1,2,2,2)),
    '2024-01-01'::timestampz));
```

Insertar un valor cuyo `pcid` no coincide con el *typmod* de la columna genera `ERROR: Pcid of tpcpoint value (N) does not match column typmod pcid (M)`; mezclar `pcids` en una columna sin restricciones también se rechaza en el momento de la agregación con `extent`.

19.1.3. Consultar con la superficie bbox

Cada `tpcpoint` y `tpcpatch` lleva una `tpcbox` (espejo de `stbox` + `pcid`) calculada en el momento de la construcción. La superficie de operadores a nivel `bbox`, topológica (`&&`, `@>`, `<@`, `~=`, `-|-`), direccional en `X/Y/Z/tiempo` y distancia `KNN` `|=|`, funciona entre cualquier par de (`tpcbox`, `tpcpoint`, `tpcpatch`):

```
-- rows whose track ever entered a 3D-temporal box
SELECT id FROM scans
WHERE traj && tpcbox_zt(0, 0, 0, 1, 1, 1,
  tstzspan '[2024-01-01, 2024-01-02]', 1, 4326);

-- 5 nearest tracks to a query box (KNN, GiST-orderable)
SELECT id, traj |=| my_box AS dist FROM scans
ORDER BY traj |=| my_box LIMIT 5;
```

El filtrado a nivel de parche (descartar instantes completos cuyo parche no se superpone con una caja) se proporciona mediante `atTpcbox` / `minusTpcbox`. Véase [Operaciones por punto](#) para el ámbito de granularidad.

19.1.4. Índice para búsquedas impulsadas por bbox

Las clases de operadores R-tree GiST y quadtree / kd-tree SP-GiST aceleran todos los predicados bbox. Elija GiST para cargas de trabajo de propósito general (y para ordenación KNN); SP-GiST kd-tree es competitivo en datos de solo puntos con muchas lecturas:

```
CREATE INDEX scans_traj_gist ON scans USING gist(traj);
CREATE INDEX scans_full_spgist ON scans USING spgist(full_scan);
```

Las clases de operadores SP-GiST almacenan un `stbox` internamente y recuperan el filtro `pcid` en el momento de la verificación, por lo que una columna con `pcids` mixtos devuelve un conjunto de candidatos ligeramente mayor para verificar, no supone ningún problema cuando cada tabla contiene valores de un único esquema, que es lo que impone la receta `typmod` en **Construir valores base y temporales**.

19.1.5. Agregación

```
-- spatiotemporal extent of every track (one tpcbox)
SELECT extent(traj) FROM scans;

-- count of active tracks at each timestamp (tint)
SELECT tCount(traj) FROM scans;

-- merge per-source instants into a single tpcpoint
SELECT id, merge(traj) FROM scans GROUP BY id;
```

`extent` rechaza `pcids` mixtos (las dimensiones bbox serían ininterpretables entre esquemas); `merge` y `tCount` siguen las mismas reglas que sus equivalentes sin nube de puntos.

19.1.6. Acceso por punto

La función de retorno de conjuntos **points** expande el parche de cada instante en `pcpoints` individuales, lo que resulta útil para uniones con predicados por punto que la capa bbox no puede expresar:

```
-- explode every patch in the column into one row per (instant timestamp, point)
SELECT id, t, point FROM scans, points(full_scan);

-- total points across every instant of every track
SELECT id, numPoints(full_scan) FROM scans;
```

El *filtrado* por punto, construir un nuevo parche a partir de un subconjunto de los puntos del original, se proporciona mediante **atTpcboxFine** / `minusTpcboxFine` (contra una `tpcbox`) y **atGeometry** / `minusGeometry` (contra una geometría 2D); véase **Operaciones por punto** para la matriz de granularidad.

19.1.7. ¿Qué sigue?

El resto del capítulo profundiza en cada una de las superficies abordadas aquí: **funciones de acceso de `pcpoint` / `pcpatch`, `pcpointset` / `pcpatchset`, la caja delimitadora `tpcbox`, `tpcpoint`, `tpcpatch`, índices y agregaciones**.

19.2. Tipos estáticos `pcpoint` y `pcpatch`

Los tipos estáticos provienen de `pgPointCloud`. MobilityDB expone las funciones de acceso de coordenadas con conocimiento del esquema necesarias para proyectar los valores de `pgPointCloud` en el resto del ecosistema de tipos temporales.

19.2.1. Constructores ergonómicos

La función `pcpoint_in` de `pgPointCloud` solo acepta hex-WKB, por lo que el constructor canónico es `PC_MakePoint (pcid, ARRAY[...]::float[])`. MobilityDB incluye dos sobrecargas SQL ligeras sobre él para literales de prueba más cortos y consultas ad hoc; la validación subyacente de dimensiones del esquema sigue ocurriendo dentro de `PC_MakePoint`.

- Construye un `pcpoint` 2D o 3D directamente a partir de coordenadas x/y/z)

```
pcpoint(pcid integer, x float, y float) → pcpoint
pcpoint(pcid integer, x float, y float, z float) → pcpoint
```

```
SELECT pcpoint(1, 10.0, 20.0, 30.0);
-- equivalent to PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]::float[])
```

- Construye un `pcpatch` a partir de una lista variádica de `pcpoints` que comparten el mismo `pcid`

```
pcpatch(pcid integer, VARIADIC points pcpoint[]) → pcpatch
```

```
SELECT pcpatch(1, pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0));
-- equivalent to PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]::float[]),
--                          PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0]::float[])])
```

19.2.2. Funciones de acceso

- Devuelve el identificador de esquema de un `pcpoint` o `pcpatch`

```
pcid(pcpoint) → integer
pcid(pcpatch) → integer
```

```
SELECT pcid(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- 1
```

- Devuelve una coordenada de un `pcpoint`, NULL si la dimensión está ausente del esquema

```
getX(pcpoint) → float
getY(pcpoint) → float
getZ(pcpoint) → float
```

```
WITH pt AS (SELECT PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]) p)
SELECT getX(p), getY(p), getZ(p) FROM pt;
-- 10 | 20 | 30
```

- Devuelve cualquier dimensión con nombre de un `pcpoint`, NULL si el nombre es desconocido para el esquema

```
getDim(pcpoint, text) → float
```

```
SELECT getDim(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), 'x');
-- 10
```

- Devuelve el identificador de referencia espacial (heredado del esquema)

```
SRID(pcpoint) → integer
SRID(pcpatch) → integer
```

```
SELECT SRID(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- 0
SELECT SRID(PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), PC_MakePoint(1, ARRAY[
  11.0, 21.0, 31.0])])));
-- 0
```

19.3. Tipos de conjunto `pcpointset` y `pcpatchset`

Un `pcpointset` es un conjunto ordenado de valores `pcpoint` distintos; un `pcpatchset` es el tipo de conjunto análogo para `pcpatch`. Ambos siguen la semántica general de conjuntos de MobilityDB (véase Capítulo 2): los valores se deduplican en la construcción, la representación binaria es canónica, y todos los operadores genéricos a nivel de conjunto (`=`, `<>`, `<`, `<=`, `>`, `>=`, `@>`, `<@`, `&&`, `+`, `-`, `*`) y las funciones de acceso (`numValues`, `memSize`, `asText`, `asBinary`, `asEWKB`, `asHexWKB`, `asHexEWKB`, ...) están disponibles (véase [?listitem]). El esquema `pcid`, y su SRID, se resuelve fuera de banda a partir del catálogo `pointcloud_formats` en lugar de incluirse en el valor, por lo que `asBinary/asEWKB` y `asHexWKB/asHexEWKB` devuelven los mismos bytes para ambos tipos de conjunto.

Todos los valores de un conjunto deben compartir el mismo `pcid`. Mezclar esquemas en un único conjunto genera un error en el momento de la construcción.

19.3.1. Constructores

- Construye un conjunto a partir de un array de valores, todos los cuales deben compartir el mismo `pcid`

```
set (pcpoint[]) → pcpointset
```

```
set (pcpatch[]) → pcpatchset
```

```
SELECT numValues(set(ARRAY[PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
  PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), -- duplicate, deduped
  PC_MakePoint(1, ARRAY[11.0, 21.0, 31.0])]));
-- 2
```

19.4. Caja Delimitadora `tpcbox`

El tipo `tpcbox` es la caja delimitadora espaciotemporal para los tipos de nube de puntos temporal. Refleja la estructura de `stbox` (véase Capítulo 3), mismos ejes X/Y/Z/T, mismos bits de indicador para las dimensiones presentes, y añade un único campo: el `pcid`. Dos valores `tpcbox` con distintos valores de `pcid` describen esquemas distintos, y una operación entre ellos genera un error en lugar de comparar su extensión geométrica o temporal.

Propagar el `pcid` a través de la aritmética de cajas delimitadoras garantiza que la poda por caja delimitadora nunca descarte una fila cuyo esquema difiera del esquema de la consulta, y nunca devuelva un resultado entre esquemas incompatibles.

19.4.1. Constructores

- Construir un `tpcbox` a partir de límites X/Y/Z/T explícitos y un `pcid`

```
tpcbox (xmin, ymin, xmax, ymax, pcid, srid=0) → tpcbox
```

```
tpcbox_z (xmin, ymin, zmin, xmax, ymax, zmax, pcid, srid=0) → tpcbox
```

```
tpcbox_t (period, pcid) → tpcbox
```

```
tpcbox_xt (xmin, ymin, xmax, ymax, period, pcid, srid=0) → tpcbox
```

```
tpcbox_zt (xmin, ymin, zmin, xmax, ymax, zmax, period, pcid, srid=0) → tpcbox
```

```
SELECT tpcbox(0, 0, 10, 10, 1, 0);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcbox_zt(0, 0, 0, 10, 10, 10, tstzspan '2024-01-01, 2024-01-02', 1, 0);
-- TPCBOX(ZT(((0,0,0),(10,10,10)), [2024-01-01, 2024-01-02]), 1)
```

19.4.2. Conversiones

- Convertir un pcpoint a un tpcbox degenerado (de extensión cero); el SRID y el pcid provienen del esquema

tpcbox(pcpoint) → tpcbox

```
SELECT tpcbox(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```

- Convertir un pcpatch a un tpcbox usando los límites PCBOUNDS 2D integrados en el parche; la segunda forma acepta un SRID explícito en lugar de buscarlo en el esquema

tpcbox(pcpatch) → tpcbox

tpcbox(pcpatch, integer) → tpcbox

```
SELECT tpcbox(PC_Patch(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0])));
```

19.4.3. Accesores

- Devuelve si un tpcbox tiene establecidas las dimensiones X/Y, Z o T

hasX(tpcbox) → boolean

hasZ(tpcbox) → boolean

hasT(tpcbox) → boolean

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 1, 0) AS b)
SELECT hasX(b), hasZ(b) FROM b;
-- t | f
```

- Devuelve un límite de coordenada, NULL si la dimensión correspondiente está ausente

xmin(tpcbox) → float

xmax(tpcbox) → float

ymin(tpcbox) → float

ymax(tpcbox) → float

zmin(tpcbox) → float

zmax(tpcbox) → float

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 1, 0) AS b)
SELECT xmin(b), xmax(b) FROM b;
-- 0 | 10
```

- Devuelve un límite temporal, NULL si la caja no tiene dimensión T

tmin(tpcbox) → timestampz

tmax(tpcbox) → timestampz

```
SELECT tmin(tpcbox_t(tstzspan '[2024-01-01, 2024-01-02]', 1));
-- 2024-01-01
```

- Devuelve el identificador de esquema y el SRID

pcid(tpcbox) → integer

SRID(tpcbox) → integer

```
WITH b AS (SELECT tpcbox(0, 0, 10, 10, 7, 4326) AS b)
SELECT pcid(b), SRID(b) FROM b;
-- 7 | 4326
```

19.4.4. Transformaciones

- Devuelve un `tpcbox` con las coordenadas redondeadas al número de dígitos decimales indicado

`round(tpcbox, integer=0) → tpcbox`

```
SELECT round(tpcbox(0.123456, 1.234567, 2.345678, 3.456789, 1, 0), 2);
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
```

- Devuelve un `tpcbox` con el `SRID` sobrescrito. No reproyecta las coordenadas, para ello, proyecte primero el `tpcpoint` subyacente y tome la caja delimitadora del resultado

`setSRID(tpcbox, integer) → tpcbox`

```
SELECT SRID(setSRID(tpcbox(0, 0, 10, 10, 1, 0), 4326));
-- 4326
```

19.4.5. Operaciones de conjuntos

- Devuelve la unión de dos `tpcboxes`; ambas deben compartir el mismo `pcid`

`tpcbox_union(tpcbox, tpcbox) → tpcbox`

`tpcbox + tpcbox → tpcbox`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) + tpcbox(3, 3, 10, 10, 1, 0);
-- TPCBOX(X((0,0),(10,10)), 1)
```

- Devuelve la intersección de dos `tpcboxes`, `NULL` si son disjuntas o el `pcid` no coincide

`tpcbox_intersection(tpcbox, tpcbox) → tpcbox`

`tpcbox * tpcbox → tpcbox`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) * tpcbox(3, 3, 10, 10, 1, 0);
-- TPCBOX(X((3,3),(5,5)), 1)
```

Nota: no se proporciona un `minus` genérico entre cajas, siguiendo la convención de `stbox` y `tbox`: la diferencia de dos cajas no es en general una caja.

19.4.6. Operadores Topológicos

Todos los operadores topológicos toman dos valores `tpcbox` con `pcid` coincidente. Una discrepancia de `pcid` siempre devuelve `false` (no un error) para que los escaneos de índice se comporten de forma intuitiva.

- Devuelve `true` si el primer `tpcbox` contiene al segundo

`tpcbox @> tpcbox → boolean`

```
SELECT tpcbox(0, 0, 10, 10, 1, 0) @> tpcbox(2, 2, 8, 8, 1, 0);
-- t
```

- Devuelve `true` si el primer `tpcbox` está contenido en el segundo

`tpcbox <@ tpcbox → boolean`

```
SELECT tpcbox(2, 2, 8, 8, 1, 0) <@ tpcbox(0, 0, 10, 10, 1, 0);
-- t
```

- Devuelve `true` si dos `tpcboxes` se solapan

`tpcbox && tpcbox → boolean`

```
SELECT tpcbox(0, 0, 5, 5, 1, 0) && tpcbox(3, 3, 10, 10, 1, 0); -- t
SELECT tpcbox(0, 0, 5, 5, 1, 0) && tpcbox(0, 0, 5, 5, 2, 0);
-- ERROR: Operation on TPCBox values with different schemas: 1 vs 2
```

- Devuelve true si dos tpcboxes son iguales en las dimensiones que comparten

`tpcbox ~= tpcbox → boolean`

```
SELECT tpcbox(0, 0, 2, 2) ~= tpcbox(0, 0, 2, 2);
-- t
```

- Devuelve true si dos tpcboxes son adyacentes (comparten un límite)

`tpcbox -|- tpcbox → boolean`

```
SELECT tpcbox(0, 0, 2, 2) -|- tpcbox(5, 5, 7, 7);
-- f
```

19.4.7. Operadores de Posición

Dieciséis predicados de posición alineados por eje, paralelos a los de `stbox`. Cada uno comprueba la relación estricta o de solapamiento a lo largo de un eje. Todos requieren `pcid` coincidente; la discrepancia devuelve false. Los predicados del eje Z requieren que ambas cajas tengan dimensión Z; los del eje T requieren que ambas tengan dimensión T.

- Comprueba el orden en el eje X de dos tpcboxes

`tpcbox << tpcbox → boolean` (estrictamente a la izquierda)

`tpcbox <& tpcbox → boolean` (no se extiende a la derecha)

`tpcbox >> tpcbox → boolean` (estrictamente a la derecha)

`tpcbox &> tpcbox → boolean` (no se extiende a la izquierda)

```
SELECT tpcbox(0, 0, 2, 2) << tpcbox(5, 5, 7, 7);
-- t
```

- Comprueba el orden en el eje Y de dos tpcboxes

`tpcbox <<| tpcbox → boolean` (estrictamente por debajo)

`tpcbox <&| tpcbox → boolean` (no se extiende hacia arriba)

`tpcbox |>> tpcbox → boolean` (estrictamente por encima)

`tpcbox |&> tpcbox → boolean` (no se extiende hacia abajo)

```
SELECT tpcbox(0, 0, 2, 2) <<| tpcbox(5, 5, 7, 7);
-- t
```

- Comprueba el orden en el eje Z de dos tpcboxes (devuelve false si alguna carece de Z)

`tpcbox <</ tpcbox → boolean` (estrictamente al frente)

`tpcbox <&/ tpcbox → boolean` (no se extiende hacia atrás)

`tpcbox />> tpcbox → boolean` (estrictamente detrás)

`tpcbox /&> tpcbox → boolean` (no se extiende hacia el frente)

```
-- Neither box carries a z dimension, so the front/back test is false.
SELECT tpcbox(0, 0, 2, 2) <</ tpcbox(5, 5, 7, 7);
-- f
```

- Comprueba el orden en el eje temporal de dos tpcboxes (devuelve false si alguna carece de T)

```
tpcbox <<# tpcbox → boolean (estrictamente antes)
tpcbox &<# tpcbox → boolean (no se extiende después)
tpcbox #>> tpcbox → boolean (estrictamente después)
tpcbox #&> tpcbox → boolean (no se extiende antes)
```

```
-- Neither box carries a time span, so the before/after test is false.
SELECT tpcbox(0, 0, 2, 2) <<# tpcbox(5, 5, 7, 7);
-- f
```

19.4.8. Comparaciones

- Compara dos tpcboxes; orden total sobre la codificación canónica (pcid, srid, flags, período y a continuación límites espaciales en orden XYZ-min/max)

```
tpcbox = tpcbox → boolean
tpcbox <> tpcbox → boolean
tpcbox < tpcbox → boolean
tpcbox <= tpcbox → boolean
tpcbox > tpcbox → boolean
tpcbox >= tpcbox → boolean
```

```
SELECT tpcbox(0, 0, 2, 2) = tpcbox(0, 0, 2, 2);
-- t
SELECT tpcbox(0, 0, 2, 2) <> tpcbox(5, 5, 7, 7);
-- t
SELECT tpcbox(0, 0, 2, 2) < tpcbox(5, 5, 7, 7);
-- t
```

19.5. Tipo Temporal tpcpoint

Un tpcpoint es el levantamiento de pcpoin a través de la maquinaria temporal de MobilityDB. Admite los mismos tres subtipos que los demás tipos temporales, instante, secuencia (con interpolación discreta o de pasos) y conjunto de secuencias. Todos los instantes de un único valor tpcpoint deben compartir el mismo pcid; el constructor impone esta restricción.

La caja delimitadora de un tpcpoint es un tpcbox, calculada en el momento de la construcción a partir de las dimensiones X/Y/Z de los valores pcpoin subyacentes junto con las marcas temporales de los instantes.

Una columna o dominio puede fijar el esquema añadiendo el pcid como un typmod de entero único, exactamente igual que el geometry(Point, SRID) de PostGIS. La mezcla de esquemas en dicha columna es rechazada en el momento de INSERT o de la conversión. El mismo constructor se aplica a tpcpatch.

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
-- accepts pcid 1
INSERT INTO scans VALUES (1,
    tpcpoint(pcpoin(1, 1.0, 2.0, 3.0), '2024-01-01'::timestampz),
    tpcpatch(pcpoin(1, 1, 1, 1), pcpoin(1, 2, 2, 2)),
    '2024-01-01'::timestampz);
-- rejects any other pcid:
-- ERROR: Pcid of tpcpoint value (2) does not match column typmod pcid (1)
```

Omitir el typmod (tpcpoint sin pcid entre paréntesis) deja la columna sin restricción de esquema. La comprobación en tiempo de ejecución del agregado extent sigue rechazando entradas con pcid mixtos a mitad de consulta incluso en columnas sin restricciones.

19.5.1. Constructores

- Construir un pcpoin temporal de subtipo instante

tpcpoin (pcpoin,timestampz) → tpcpoin

```
SELECT tpcpoin(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]), '2024-01-01 12:00+00');
```

- Construir un pcpoin temporal de subtipo secuencia a partir de un array de instantes. La interpolación es una de 'discrete', 'step'; 'step' es el valor predeterminado para tpcpoin, ya que los pcpoins llevan dimensiones heterogéneas que no se interpolan de manera lineal

tpcpoinSeq(tpcpoin[]) → tpcpoin

tpcpoinSeq(tpcpoin[],text) → tpcpoin

```
SELECT tpcpoinSeq(ARRAY[tpcpoin(PC_MakePoint(1, ARRAY[1.0, 2.0, 3.0]), '2024-01-01'::timestampz),
    tpcpoin(PC_MakePoint(1, ARRAY[4.0, 5.0, 6.0]), '2024-01-02'::timestampz)], 'step');
```

- Construir un pcpoin temporal de subtipo conjunto de secuencias a partir de un array de secuencias

tpcpoinSeqSet(tpcpoin[]) → tpcpoin

```
SELECT numSequences(tpcpoinSeqSet(ARRAY[tpcpoinSeq(ARRAY[
    tpcpoin(pcpoin(1, 1, 1, 1), '2001-01-01'::timestampz),
    tpcpoin(pcpoin(1, 2, 2, 2), '2001-01-02'::timestampz)]))));
-- 1
SELECT numInstants(tpcpoinSeqSet(ARRAY[tpcpoinSeq(ARRAY[
    tpcpoin(pcpoin(1, 1, 1, 1), '2001-01-01'::timestampz),
    tpcpoin(pcpoin(1, 2, 2, 2), '2001-01-02'::timestampz)]))));
-- 2
```

19.5.2. Funciones de Acceso

Se aplican todas las funciones de acceso temporales genéricas (véase Capítulo 4): numInstants, startInstant, endInstant, instantN, instants, startValue, endValue, getValues, numTimestamps, startTimestamp, endTimestamp, getTime, duration, memSize, interp. Adicionalmente:

- Devolver el id de esquema de pgPointCloud compartido por todos los instantes

pcid(tpcpoin) → integer

```
SELECT pcid(tpcpoin(pcpoin(1, 1, 1, 1), '2001-01-01'::timestampz));
-- 1
```

- Devolver el SRID heredado del esquema

SRID(tpcpoin) → integer

```
-- The SRID is that of the schema registered for the pcid.
SELECT SRID(tpcpoin(pcpoin(1, 1, 1, 1), '2001-01-01'::timestampz));
-- 0
```

19.5.3. Proyecciones por Dimensión

- Proyectar un tpcpoin sobre una dimensión espacial, produciendo un tfloat

getX(tpcpoin) → tfloat

getY(tpcpoin) → tfloat

getZ(tpcpoin) → tfloat


```
SELECT startValue(getX(tpcpoint(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
'2024-01-01'::timestampz)));
-- 10
```

- Proyectar un tpcpoint sobre cualquier dimensión de esquema con nombre, produciendo un tfloat

getDim(tpcpoint,text) → tfloat

```
SELECT getDim(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz)]), 'X');
-- Interp=Step;[1@2001-01-01, 2@2001-01-02]
```

19.5.4. Conversiones

- Convertir un tpcpoint a un punto de geometría temporal, proyectando todas las dimensiones no espaciales mientras se preservan las marcas temporales. El resultado tiene el mismo SRID que el esquema; su dimensionalidad (2D frente a 3D) sigue la presencia de Z en el esquema. No se proporciona una conversión inversa: reconstruir un pcpnt requiere un esquema destino que el valor temporal solo no puede suministrar

tgeompoint::tpcpoint no está definida

tpcpoint::tgeompoint

```
SELECT ST_AsText(startValue(tpcpoint(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0]),
'2024-01-01'::timestampz)::tgeompoint));
-- POINT Z (10 20 30)
```

19.5.5. Transformaciones

- Transformar un tpcpoint en otro subtipo

tpcpointInst(tpcpoint) → tpcpoint

tpcpointSeq(tpcpoint,interp='step') → tpcpoint

tpcpointSeqSet(tpcpoint) → tpcpoint

```
SELECT numInstants(tpcpointInst(tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz))) ←
;
-- 1
SELECT numInstants(tpcpointSeq(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz)])));
-- 2
```

- Transformar un tpcpoint a otra interpolación

setInterp(tpcpoint,interp) → tpcpoint

```
-- A tpcpoint carries step interpolation.
SELECT interp(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz)]));
-- Step
SELECT interp(setInterp(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz)], 'discrete'), 'step'));
-- Step
```

19.5.6. Comparaciones

- Comparar dos tpcpoints lexicográficamente sobre sus codificaciones canónicas

```
tpcpoint = tpcpoint → boolean
tpcpoint <> tpcpoint → boolean
tpcpoint < tpcpoint → boolean
tpcpoint <= tpcpoint → boolean
tpcpoint > tpcpoint → boolean
tpcpoint >= tpcpoint → boolean
```

```
SELECT tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz) = tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz);
-- t
SELECT tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz) <> tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz);
-- t
SELECT tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz) < tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz);
-- t
```

- Comprobar si el valor en algún instante (alguna vez) o en todos los instantes (siempre) es igual a, o distinto de, un pcpnt u otro tpcpoint

```
eEq(pcpnt, tpcpoint) → boolean
eEq(tpcpoint, pcpnt) → boolean
eEq(tpcpoint, tpcpoint) → boolean
aEq, eNe y aNe reciben los mismos tres pares de argumentos
pcpnt ?= tpcpoint → boolean
tpcpoint ?= pcpnt → boolean
tpcpoint ?= tpcpoint → boolean
```

Los operadores `%=`, `?<>` y `%<>` reciben los mismos tres pares de operandos

```
SELECT eEq(pcpnt(1, 1, 1, 1), tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]));
-- t
SELECT aEq(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), pcpnt(1, 1, 1, 1));
-- f
```

- Devolver la igualdad o desigualdad temporal de un tpcpoint con un pcpnt o con otro tpcpoint, como un tbool

```
tEq(pcpnt, tpcpoint) → tbool
tEq(tpcpoint, pcpnt) → tbool
tEq(tpcpoint, tpcpoint) → tbool
tNe recibe los mismos tres pares de argumentos
pcpnt #= tpcpoint → tbool
tpcpoint #= pcpnt → tbool
tpcpoint #= tpcpoint → tbool
```

El operador `#<>` recibe los mismos tres pares de operandos

```
SELECT tEq(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), pcpnt(1, 1, 1, 1));
-- [t@2001-01-01, f@2001-01-02, f@2001-01-03]
```

19.5.7. Restricciones

- Restringir un tpcpoint a (al complemento de) un valor pcpnt o un conjunto de valores pcpnt

```
atValue(tpcpoint,pcpnt) → tpcpoint
minusValue(tpcpoint,pcpnt) → tpcpoint
atValues(tpcpoint,pcpntset) → tpcpoint
minusValues(tpcpoint,pcpntset) → tpcpoint
```

```
-- With step interpolation the matching value is held on
-- [2001-01-01, 2001-01-02); its closing bound is stored as a second instant.
SELECT numInstants(atValue(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), pcpnt(1, 1, 1, 1)));
-- 2
SELECT numInstants(minusValue(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), pcpnt(1, 1, 1, 1)));
-- 2
```

- Restringir un tpcpoint a un selector temporal (marca de tiempo, periodo, conjunto o spanset), o eliminarlo

```
atTime(tpcpoint,timestampz) → tpcpoint
atTime(tpcpoint,tstzspan) → tpcpoint
atTime(tpcpoint,tstzset) → tpcpoint
atTime(tpcpoint,tstzspanset) → tpcpoint
minusTime(tpcpoint,timestampz) → tpcpoint
minusTime(tpcpoint,tstzspan) → tpcpoint
minusTime(tpcpoint,tstzset) → tpcpoint
minusTime(tpcpoint,tstzspanset) → tpcpoint
```

```
SELECT numInstants(atTime(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), timestampz '2001-01-02'));
-- 1
SELECT numInstants(atTime(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), tstzspan '[2001-01-01, ↵
  2001-01-02]'));
-- 2
```

- Restringir un tpcpoint al extent espacial / temporal de un tpcbox, o eliminarlo. Devuelve NULL cuando el pcid del tpcbox no coincide con el del tpcpoint. Internamente proyecta a un tgeompoint a través de la caché de esquemas, restringe la proyección mediante el stbox equivalente y devuelve el span temporal del resultado al tpcpoint original para preservar los valores pcpnt completos

```
atTpcbox(tpcpoint, tpcbox, border_inc bool=TRUE) → tpcpoint
minusTpcbox(tpcpoint, tpcbox, border_inc bool=TRUE) → tpcpoint
```

```
SELECT numInstants(atTpcbox(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)]),
  tpcbox_zt(0, 0, 0, 2, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
-- The box does not intersect the tpcpoint, so nothing is removed.
SELECT numInstants(minusTpcbox(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)]),
  tpcbox_zt(10, 10, 10, 12, 12, 12, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 3
```

- Restringir un tpcpoint a los instantes anteriores o posteriores a una marca de tiempo. El indicador strict excluye el instante en la marca de tiempo misma

```
beforeTimestamp(tpcpoint, timestamp, strict bool=TRUE) → tpcpoint
afterTimestamp(tpcpoint, timestamp, strict bool=TRUE) → tpcpoint
```

```
SELECT timeSpan(beforeTimestamp(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)]), timestamp '2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)]), timestamp '2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

19.5.8. Modificaciones

- Insertar un tpcpoint en otro, o actualizar otro con él

```
insert(tpcpoint, tpcpoint, connect bool=TRUE) → tpcpoint
update(tpcpoint, tpcpoint, connect bool=TRUE) → tpcpoint
```

```
SELECT numInstants(insert(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp)]), tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)])));
-- 3
```

- Eliminar un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpoint

```
deleteTime(tpcpoint, timestamp, connect bool=TRUE) → tpcpoint
deleteTime(tpcpoint, tstzset, connect bool=TRUE) → tpcpoint
deleteTime(tpcpoint, tstzspan, connect bool=TRUE) → tpcpoint
deleteTime(tpcpoint, tstzspanset, connect bool=TRUE) → tpcpoint
```

```
-- Deleting the middle instant splits the sequence into
-- {[2001-01-01, 2001-01-02), (2001-01-02, 2001-01-03]}; each half stores
-- its bound at the deleted timestamp, giving four instants.
SELECT numInstants(deleteTime(tpcpointSeq(ARRAY[
```

```
tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)), timestampz '2001-01-02'));
-- 4
```

- Añadir un instante o una secuencia a un tpcpoint

```
appendInstant(tpcpoint, tpcpoint) → tpcpoint
appendSequence(tpcpoint, tpcpoint) → tpcpoint
```

```
SELECT numInstants(appendInstant(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz)]),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)));
-- 3
```

19.5.9. Fragmentación

- Devolver los valores distintos de un tpcpoint con el conjunto de spans durante el cual toma cada uno de ellos

```
unnest(tpcpoint) → {(value,time)}
```

```
SELECT (un).time
FROM (SELECT unnest(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2024-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2024-01-02'::timestampz),
  tpcpoint(pcpnt(1, 1, 1, 1), '2024-01-03'::timestampz)])) AS un) t;
-- {[2024-01-01, 2024-01-02], [2024-01-03, 2024-01-03]}
-- {[2024-01-02, 2024-01-03]}
```

- Fragmentar un tpcpoint en fragmentos, uno por cada intervalo de tiempo

```
timeSplit(tpcpoint, bin_width interval, origin timestampz='2000-01-03') → {(time, tpcpoint)}
```

```
SELECT (ts).time, numInstants((ts).temp)
FROM (SELECT timeSplit(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2024-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2024-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2024-01-03'::timestampz)]),
  interval '1 day', '2024-01-01'::timestampz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpoint

```
spans(tpcpoint) → tstzspan[]
```

```
SELECT spans(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2024-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2024-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2024-01-03'::timestampz)]));
-- {"[2024-01-01, 2024-01-02]", "[2024-01-02, 2024-01-03]"}
```

- Devuelve los lapsos de tiempo de un tpcpoint dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

```
splitNSpans(tpcpoint, integer) → tstzspan[]
splitEachNSpans(tpcpoint, integer) → tstzspan[]
```

```
SELECT splitNSpans(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2024-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2024-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2024-01-03'::timestamp)]), 2);
-- {"[2024-01-01, 2024-01-02]", "[2024-01-02, 2024-01-03]"}
```

19.5.10. Operadores de caja delimitadora

Los operadores de caja delimitadora evalúan contra el `tpcbox` del valor; véase Sección 19.4.6 y Sección 19.4.7 para la geometría por operador. Cada operador a continuación está conectado contra `tpcbox`, `tstzspan` y el propio `tpcpoint`.

- Predicados topológicos: `&&` solapa, `@>` contiene, `<@` contenido en, `~=` mismo, `-|-` adyacente.

```
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (2,2,2)), [2024-01-01, 2024-01-03]), 1)' &&
  tpcbox 'TPCBOX(ZT((1,1,1), (3,3,3)), [2024-01-02, 2024-01-04]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (4,4,4)), [2024-01-01, 2024-01-04]), 1)' @>
  tpcbox 'TPCBOX(ZT((1,1,1), (2,2,2)), [2024-01-02, 2024-01-03]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (2,2,2)), [2024-01-01, 2024-01-03]), 1)' ~=
  tpcbox 'TPCBOX(ZT((0,0,0), (2,2,2)), [2024-01-01, 2024-01-03]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (2,2,2)), [2024-01-01, 2024-01-02]), 1)' -|-
  tpcbox 'TPCBOX(ZT((0,0,0), (2,2,2)), [2024-01-02, 2024-01-03]), 1)';
-- true
```

- Predicados direccionales estrictos sobre el eje X (`<<`, `>>`), eje Y (`<<|`, `|>>`), eje Z (`<</`, `/>>`) y eje temporal (`<<#`, `#>>`), más sus variantes "solapa o X": `&<`, `&>`, `&<|`, `|&>`, `&</`, `/&>`, `&<#`, `#&>`.

```
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (1,1,1)), [2024-01-01, 2024-01-02]), 1)' <<
  tpcbox 'TPCBOX(ZT((5,5,5), (6,6,6)), [2024-01-03, 2024-01-04]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (1,1,1)), [2024-01-01, 2024-01-02]), 1)' <<|
  tpcbox 'TPCBOX(ZT((5,5,5), (6,6,6)), [2024-01-03, 2024-01-04]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (1,1,1)), [2024-01-01, 2024-01-02]), 1)' <</
  tpcbox 'TPCBOX(ZT((5,5,5), (6,6,6)), [2024-01-03, 2024-01-04]), 1)';
-- true
SELECT tpcbox 'TPCBOX(ZT((0,0,0), (1,1,1)), [2024-01-01, 2024-01-02]), 1)' <<#
  tpcbox 'TPCBOX(ZT((5,5,5), (6,6,6)), [2024-01-03, 2024-01-04]), 1)';
-- true
```

- Distancia de aproximación más cercana, ordenable KNN mediante la clase de operadores GiST: `SELECT ... ORDER BY temp |=| query LIMIT N` se acelera con índice. El desajuste de `pcid` produce infinito.

`nearestApproachDistance(tpcpoint, tpcbox) → float`

`nearestApproachDistance(tpcpoint, tpcpoint) → float`

```
SELECT round(nearestApproachDistance(
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 10, 10, 10), '2001-01-01'::timestamp))::numeric, 4);
-- 15.5885
-- A tpcbox without a time span measures the spatial approach only.
SELECT round(nearestApproachDistance(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestamp),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestamp),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestamp)]),
  tpcbox_z(10, 10, 10, 12, 12, 12, 1))::numeric, 4);
-- 12.1244
```

```
SELECT round((tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz) |=|
tpcpoint(pcpnt(1, 10, 10, 10), '2001-01-01'::timestampz))::numeric, 4);
-- 15.5885
```

19.5.11. Ordenación B-tree, qué significa ORDER BY tpcpoint

La clase de operadores `tpcpoint_btree_ops` produce un orden total sobre los valores `tpcpoint`, pero el orden es byte a byte sobre la serialización `pcpnt` subyacente, no geométrico. Se aplican las mismas advertencias que al **B-tree de tpcpatch**:

- **pcid es el discriminador primario.** Los puntos con `pcid` menor se ordenan antes que los puntos con `pcid` mayor, independientemente de las coordenadas. La mezcla de `pcids` en una única columna agrupa las filas por esquema primero al ordenar.
- **Dentro de un único pcid, el orden sigue la disposición de dimensiones en disco del esquema**, típicamente X, Y, Z codificados como `int32` (tras aplicar la escala por dimensión del esquema). Dos puntos cuya distancia euclidiana es pequeña pueden ordenarse muy separados si sus dimensiones X difieren, ya que X se compara primero.
- **La igualdad es igualdad exacta de bytes.** Dos `pcpoints` con los mismos valores de dimensión en el mismo esquema son iguales; el mismo conjunto de coordenadas codificado en un esquema con escala u orden de dimensiones diferente no lo es.

Para una ordenación con significado espacial, proyecte a un `tgeompoint` mediante la conversión con conocimiento de esquema y use el KNN `<->` de PostGIS sobre la geometría resultante, o use el operador KNN GiST `!=` directamente sobre el `tpcpoint`. Para una ordenación con significado temporal, proyecte a `startTimestamp` o `timespan` y ordene sobre ese valor.

19.5.12. Relaciones espaciales

Se evalúan proyectando el `tpcpoint` a un `tgeompoint` a través de la caché de esquemas y delegando en la relación `tgeompoint` correspondiente. Cada función está conectada en tres órdenes de argumentos: `(geometry, tpcpoint)`, `(tpcpoint, geometry)` y `(tpcpoint, tpcpoint)`. `tpcpatch` queda intencionalmente fuera del alcance, la intersección a nivel de parche requeriría la descompresión punto a punto y un diseño diferente.

- **Alguna vez / siempre intersecta.**

```
eIntersects(tpcpoint, geometry) → bool, aIntersects(tpcpoint, geometry) → bool
eIntersects(tpcpoint, tpcpoint) → bool, aIntersects(tpcpoint, tpcpoint) → bool
```

```
SELECT eIntersects(tpcpointSeq(ARRAY[
tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aIntersects(tpcpointSeq(ARRAY[
tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (1 1 1)');
-- f
SELECT eIntersects(tpcpointSeq(ARRAY[
tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), tpcpointSeq(ARRAY[
tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]));
-- t
```

- Alguna vez / siempre disjunto.

`eDisjoint(tpcpoint, geometry) → bool`, `aDisjoint(tpcpoint, geometry) → bool`
`eDisjoint(tpcpoint, tpcpoint) → bool`, `aDisjoint(tpcpoint, tpcpoint) → bool`

```
SELECT eDisjoint(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aDisjoint(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (1 1 1)');
-- f
```

- Alguna vez / siempre dentro de la distancia.

`eDwithin(tpcpoint, geometry, float) → bool`, `aDwithin(tpcpoint, geometry, float) → bool`
`eDwithin(tpcpoint, tpcpoint, float) → bool`, `aDwithin(tpcpoint, tpcpoint, float) → bool`

```
SELECT eDwithin(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (5 5 5)', ←
  4);
-- t
SELECT aDwithin(tpcpointSeq(ARRAY[
  tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz),
  tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz),
  tpcpoint(pcpnt(1, 3, 3, 3), '2001-01-03'::timestampz)]), geometry 'POINT Z (5 5 5)', ←
  4);
-- f
```

19.6. Tipo Temporal `tpcpatch`

Un `tpcpatch` es la elevación temporal de `pcpatch`. Es el espejo de `tpcpoint` en todos los aspectos, mismos subtipos, mismas funciones de acceso, mismas conversiones y operadores, excepto que cada instante contiene un lote comprimido completo de puntos en lugar de un único punto. La caja delimitadora es de nuevo un `tpcbox`, calculada en $O(1)$ por instante a partir del `PCBOUNDS` embebido en el parche.

19.6.1. Constructores

- Construir un `pcpatch` temporal de subtipo instante

`tpcpatch(pcpnt, timestampz) → tpcpatch`

```
SELECT tempSubtype(tpcpatch(pcpnt(1, pcpnt(1, 1, 1, 1), pcpnt(1, 2, 2, 2)), ' ←
  2001-01-01'::timestampz));
-- Instant
```

- Construir un `pcpatch` temporal de subtipo secuencia

`tpcpatchSeq(tpcpatch[]) → tpcpatch`

`tpcpatchSeq(tpcpatch[], text) → tpcpatch`


```
SELECT numInstants(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)])), interp(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)]));
-- 2 | Step
```

- Construir un pcpatch temporal de subtipo conjunto de secuencias

`tpcpatchSeqSet(tpcpatch[]) → tpcpatch`

```
SELECT numSequences(tpcpatchSeqSet(ARRAY[tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)]))));
-- 1
```

19.6.2. Accesores

Se aplican todos los accesores temporales genéricos, además del `pcid` y el `SRID` al estilo de `tpcpoint`. Adicionalmente:

- Devolver el número de puntos del parche contenido en el primer o último instante

`startNumPoints(tpcpatch) → integer`

`endNumPoints(tpcpatch) → integer`

```
SELECT startNumPoints(tpcpatch(PC_Patch(PC_MakePoint(1, ARRAY[10.0, 20.0, 30.0])),
  '2024-02-01'::timestampz));
-- 1
```

- Devolver el número total de puntos de todos los parches de todos los instantes. Se lee directamente de la cabecera de cada parche, sin descompresión.

`numPoints(tpcpatch) → bigint`

```
SELECT numPoints(tpcpatchSeq(ARRAY[
  tpcpatch(PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]),
    PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0])]),
    '2024-01-01'::timestampz),
  tpcpatch(PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[3.0, 3.0, 3.0])]),
    '2024-01-02'::timestampz)));
-- 3
```

- Función devuelve-conjunto: emite una fila por cada (marca de tiempo de instante, punto) recorriendo el parche de cada instante y descomponiéndolo mediante la API C en memoria de `pgPointCloud`. Útil para combinar una columna `tpcpatch` con predicados por punto que los operadores de nivel de caja delimitadora no pueden expresar. El coste es $O(\text{total de puntos})$, una descompresión por instante más una emisión de fila por punto.

`points(tpcpatch) → setof (t timestampz, point pcpoint)`

```
SELECT t, point FROM points(tpcpatch(
  PC_Patch(ARRAY[PC_MakePoint(1, ARRAY[1.0, 1.0, 1.0]),
    PC_MakePoint(1, ARRAY[2.0, 2.0, 2.0])]),
  '2024-01-01'::timestampz));
```

```
-- ('2024-01-01', '01:000000000000000000000000F03F00000000000000F03F00000000000000F03F'::pcpoint) ←
-- ('2024-01-01', '01:0000000000000000000000004000000000000000400000000000000040'::pcpoint) ←
-- )
```

Véase Sección 19.6.11 para conocer los límites de estos accesorios y qué está y qué no está disponible a nivel de granularidad por punto.

19.6.3. Transformaciones

- Transformar un `tpcpatch` en otro subtipo

```
tpcpatchInst(tpcpatch) → tpcpatch
tpcpatchSeq(tpcpatch,interp='step') → tpcpatch
tpcpatchSeqSet(tpcpatch) → tpcpatch
```

```
SELECT tempSubtype(tpcpatchSeq(tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2, ←
2)), '2001-01-01'::timestampz)));
-- Sequence
```

- Transformar un `tpcpatch` a otra interpolación

```
setInterp(tpcpatch,interp) → tpcpatch
```

```
SELECT interp(setInterp(tpcpatchSeq(ARRAY[
tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)], 'discrete'), 'step'));
-- Step
```

19.6.4. Restricciones

- Restringir un `tpcpatch` a (al complemento de) un valor `pcpatch` o un conjunto de valores `pcpatch`

```
atValue(tpcpatch,pcpatch) → tpcpatch
minusValue(tpcpatch,pcpatch) → tpcpatch
atValues(tpcpatch,pcpatchset) → tpcpatch
minusValues(tpcpatch,pcpatchset) → tpcpatch
```

```
-- The matching value is held up to the next instant; the closing bound
-- is stored as a second instant.
SELECT numInstants(atValue(tpcpatchSeq(ARRAY[
tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)],
pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 2
SELECT numInstants(minusValue(tpcpatchSeq(ARRAY[
tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)],
pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 1
```

- Restringir un tpcpatch a los instantes cuyo parche supera una prueba de superposición aproximada de PCBOUNDS frente al tpcbox, o eliminarlos. La granularidad es a nivel de parche: cada instante superviviente conserva su payload pcpatch textualmente, sin descompresión por punto. Como el PCBOUNDS de pgPointCloud es 2D, la dimensión Z de la caja se ignora a esta granularidad. Devuelve NULL cuando el pcid del tpcbox no coincide.

atTpcbox(tpcpatch,tpcbox,border_inc bool=TRUE) → tpcpatch

minusTpcbox(tpcpatch,tpcbox,border_inc bool=TRUE) → tpcpatch

```
SELECT numInstants(atTpcbox(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  tpcbox_xt(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
-- The dropped patch stays present on the open interval before its instant,
-- so the complement keeps two instants.
SELECT numInstants(minusTpcbox(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  tpcbox_xt(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
```

- Variante por punto de la anterior: cada instante superviviente contiene un pcpatch recién construido que incluye solo los puntos dentro (o fuera, en el caso de minus) del tpcbox en 2D, y en 3D cuando la caja tiene dimensión Z. Los instantes cuyo parche filtra hasta cero puntos se descartan. Más lenta que la variante aproximada porque cada parche se descomprime y reconstruye; úsela cuando se necesite fidelidad a nivel de punto.

atTpcboxFine(tpcpatch,tpcbox,border_inc bool=TRUE) → tpcpatch

minusTpcboxFine(tpcpatch,tpcbox,border_inc bool=TRUE) → tpcpatch

```
SELECT numInstants(atTpcboxFine(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  tpcbox_xt(0, 0, 1, 1, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
```

- Restringir un tpcpatch a los puntos cuya proyección XY intersecta (o no intersecta, en el caso de minus) una geometría 2D. Z se ignora. El llamador debe garantizar la compatibilidad de SRID entre el esquema del parche y la geometría.

atGeometry(tpcpatch,geometry) → tpcpatch

minusGeometry(tpcpatch,geometry) → tpcpatch

```
-- The polygon boundary touches the point (3 3) of the patch at
-- 2001-01-02, so that patch survives both the at and the minus side.
SELECT numInstants(atGeometry(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))');
-- 2
SELECT numInstants(minusGeometry(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))');
```

```
-- 1
```

- Restringir un `tpcpatch` a los instantes anteriores o posteriores a una marca de tiempo. El indicador `strict` excluye el instante en la marca de tiempo misma

```
beforeTimestamp(tpcpatch,timestamptz,strict bool=TRUE) → tpcpatch
```

```
afterTimestamp(tpcpatch,timestamptz,strict bool=TRUE) → tpcpatch
```

```
SELECT timeSpan(beforeTimestamp(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]), timestamptz ' ←
  2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]), timestamptz ' ←
  2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

19.6.5. Modificaciones

- Insertar un `tpcpatch` en otro, o actualizar otro con él

```
insert(tpcpatch,tpcpatch,connect bool=TRUE) → tpcpatch
```

```
update(tpcpatch,tpcpatch,connect bool=TRUE) → tpcpatch
```

```
SELECT numInstants(insert(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz ←
  ))), tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)])));
-- 3
```

- Eliminar un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un `tpcpatch`

```
deleteTime(tpcpatch,timestamptz,connect bool=TRUE) → tpcpatch
```

```
deleteTime(tpcpatch,tstzset,connect bool=TRUE) → tpcpatch
```

```
deleteTime(tpcpatch,tstzspan,connect bool=TRUE) → tpcpatch
```

```
deleteTime(tpcpatch,tstzspanset,connect bool=TRUE) → tpcpatch
```

```
-- Deleting the middle instant splits the sequence into
-- {[2001-01-01, 2001-01-02), (2001-01-02, 2001-01-03]}; each half stores
-- its bound at the deleted timestamp, giving four instants.
SELECT numInstants(deleteTime(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz ←
  ),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]), timestamptz ' ←
  2001-01-02'));
-- 4
```

- Añadir un instante o una secuencia a un `tpcpatch`

```
appendInstant(tpcpatch, tpcpatch) → tpcpatch
appendSequence(tpcpatch, tpcpatch) → tpcpatch
```

```
SELECT numInstants(appendInstant(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5)), '2001-01-03'::timestampz)));
-- 3
```

19.6.6. Fragmentación

- Devolver los valores distintos de un `tpcpatch` con el conjunto de spans durante el cual toma cada uno de ellos

```
unnest(tpcpatch) → {(value,time)}
```

```
SELECT (un).time
FROM (SELECT unnest(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-03'::timestampz ←
))) AS un) t;
-- {[2024-01-02, 2024-01-03]}
-- {[2024-01-01, 2024-01-02], [2024-01-03, 2024-01-03]}
```

- Fragmentar un `tpcpatch` en fragmentos, uno por cada intervalo de tiempo

```
timeSplit(tpcpatch, bin_width interval, origin timestampz='2000-01-03') → {(time,tpcpatch)}
```

```
SELECT (ts).time, numInstants((ts).temp)
FROM (SELECT timeSplit(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-03'::timestampz ←
)),
  interval '1 day', '2024-01-01'::timestampz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Devuelve el arreglo de lapsos de tiempo de los segmentos de un `tpcpatch`

```
spans(tpcpatch) → tstzspan[]
```

```
SELECT spans(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-03'::timestampz ←
)))
-- {"[2024-01-01, 2024-01-02]", "[2024-01-02, 2024-01-03]"}
```

- Devuelve los lapsos de tiempo de un `tpcpatch` dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

`splitNSpans(tpcpatch, integer) → tstzspan[]`

`splitEachNSpans(tpcpatch, integer) → tstzspan[]`

```
SELECT splitNSpans(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-03'::timestampz ←
)], 2);
-- {"[2024-01-01, 2024-01-02]", "[2024-01-02, 2024-01-03]"}
```

19.6.7. Relaciones espaciales

- Devolver verdadero si y solo si al menos un punto de alguno de los instantes del `tpcpatch` intersecta la geometría (solo XY).

`eIntersects(tpcpatch, geometry) → boolean`

```
SELECT eIntersects(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)], geometry 'POINT(1 1)');
-- t
```

19.6.8. Operadores de caja delimitadora

La misma superficie de operadores de caja que en Sección 19.5.10 se aplica a `tpcpatch`. El predicado se evalúa sobre el `tpcbox` del valor, calculado en $O(1)$ por instante a partir del `PCBOUNDS` embebido en el parche.

- Predicados topológicos: `&&`, `@>`, `<@`, `~=`, `-|`. Véase Sección 19.4.6.
- Predicados direccionales estrictos sobre los ejes X / Y / Z / tiempo (`<<`, `>>`, `<<|`, `|>>`, `<</`, `/>>`, `<<#`, `#>>`) y sus variantes «solapa-o-X» (`&<`, `&>`, `&<|`, `|&>`, `&</`, `/&>`, `&<#`, `#&>`). Véase Sección 19.4.7.
- La distancia de aproximación más cercana (`|=|`) es ordenable KNN a través de la clase de operador GiST.

`nearestApproachDistance(tpcpatch, tpcbox) → float`

`nearestApproachDistance(tpcpatch, tpcpatch) → float`

19.6.9. Comparaciones

- Comprobar si el valor en algún instante (alguna vez) o en todos los instantes (siempre) es igual a, o distinto de, un `pcpatch` u otro `tpcpatch`

`eEq(pcpatch, tpcpatch) → boolean`

`eEq(tpcpatch, pcpatch) → boolean`

`eEq(tpcpatch, tpcpatch) → boolean`

`aEq`, `eNe` y `aNe` reciben los mismos tres pares de argumentos

`pcpatch ?= tpcpatch → boolean`

`tpcpatch ?= pcpatch → boolean`

`tpcpatch ?= tpcpatch → boolean`

Los operadores `%=`, `?<>` y `y<>` reciben los mismos tres pares de operandos

```

SELECT eEq(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)]));
-- t
SELECT aEq(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)], pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)));
-- f

```

- Devolver la igualdad o desigualdad temporal de un tpcpatch con un pcpatch o con otro tpcpatch, como un tbool

```
tEq(pcpatch, tpcpatch) → tbool
```

```
tEq(tpcpatch, pcpatch) → tbool
```

```
tEq(tpcpatch, tpcpatch) → tbool
```

tNe recibe los mismos tres pares de argumentos

```
pcpatch #= tpcpatch → tbool
```

```
tpcpatch #= pcpatch → tbool
```

```
tpcpatch #= tpcpatch → tbool
```

El operador #<> recibe los mismos tres pares de operandos

```

SELECT tEq(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestampz ←
),
  tpcpatch(pcpatch(1, pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestampz ←
)], pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)));
-- [t@2001-01-01, f@2001-01-02]

```

19.6.10. Ordenación B-tree, qué significa ORDER BY tpcpatch

La clase de operador tpcpatch_btree_ops produce un orden total sobre los valores tpcpatch, pero ese orden no es espacial. El comparador sobre el pcpatch subyacente es byte a byte (memcmp sobre los bytes varlena significativos); el comparador al nivel temporal es la combinación lexicográfica de las marcas de tiempo por instante y el orden de bytes de pcpatch por instante. Las consecuencias observables son:

- **pcid es el discriminador primario.** Un parche con un pcid menor se ordena antes que uno con un pcid mayor, independientemente de las coordenadas de los puntos, su cantidad o la compresión. Esto hace que ORDER BY sobre una columna de pcid mixto agrupe las filas primero por esquema, lo que a veces resulta útil como paso de agrupación gruesa «por esquema de origen».
- **Dentro de un mismo pcid, el orden sigue la disposición de bytes en disco:** esquema de compression, luego npoints, luego el PCBOUNDS 2D (xmin, xmax, ymin, ymax) y finalmente el payload data comprimido. Este orden está bien definido y es estable para una versión dada de pgPointCloud, pero *no* es un orden geométrico o espacial, dos parches cuyos cuadros delimitadores se superponen en el espacio 3D pueden ordenarse de forma arbitraria entre sí si difieren en su esquema de compresión o en el número de puntos.
- **La igualdad es igualdad exacta de bytes** sobre los bytes significativos (se excluye el relleno de ceros al final que reserva el varlena de pgPointCloud). Dos parches contruidos a partir del mismo conjunto de puntos en el mismo orden, con la misma compresión, son iguales; reordenar los puntos o cambiar la compresión los hace desiguales.

Para un orden con sentido espacial, use el operador KNN de GiST |=| (véase [distancia de aproximación más cercana en tpcpatch](#)); para un orden con sentido temporal, proyecte a startTimestamp o timespan y ordene sobre ese valor.

19.6.11. Operaciones por punto: qué está y qué no está disponible

Las operaciones sobre valores `tpcpatch` se realizan a dos granularidades: *nivel de parche* y *por punto*. La distinción importa porque los parches de `pgPointCloud` almacenan sus puntos en un payload comprimido que la capa de caja delimitadora no puede inspeccionar.

- **Nivel de parche, soportado.** El parche de cada instante se trata como opaco y se conserva o descarta como un todo, usando únicamente la cabecera `PCBOUNDS` de 4 valores `double` (`xmin / xmax / ymin / ymax`) y la marca de tiempo del instante. Esta es la granularidad de `atTpcbox / minusTpcbox` y de los operadores de caja delimitadora en Sección 19.6.8, así como del agregado `tpcbox` basado en caja. No se produce descompresión del payload; las consultas son $O(\text{número de instantes})$.

Como `PCBOUNDS` es 2D, la dimensión Z de cualquier argumento `tpcbox` se ignora a esta granularidad, incluso cuando el esquema subyacente tiene dimensión Z.

- **Inspección por punto, soportada.** La función devuelve-conjunto `points` emite una fila por cada (marca de tiempo de instante, punto) descomponiendo el parche de cada instante mediante la API C en memoria de `pgPointCloud`. El coste es $O(\text{total de puntos})$, una descompresión por instante más una emisión de fila por punto.
- **Filtrado / construcción por punto, soportado.** `atTpcboxFine / minusTpcboxFine`, `atGeometry / minusGeometry` y `eIntersects(tpcpatch, geometry)` recorren en C cada punto de cada instante, aplicando su predicado sobre las coordenadas reales del punto en lugar de limitarse al cuadro delimitador del parche. Los instantes supervivientes contienen un parche recién construido que incluye solo los puntos que superaron el predicado; los instantes cuyos parches filtran hasta cero puntos se descartan.

El flujo de trabajo recomendado cuando ambas granularidades están disponibles es: (a) podar al nivel de parche con `atTpcbox` o un escaneo de índice SP-GiST/GiST para descartar los instantes completos cuyo `PCBOUNDS` no se superpone con el área de interés, y (b) refinar sobre los supervivientes con `atTpcboxFine / atGeometry` cuando se necesite fidelidad a nivel de punto.

19.7. Índices

Las clases de operadores B-tree (`tpcpoint_btree_ops`, `tpcpatch_btree_ops`, `tpcbox_btree_ops`) y hash (`tpcpoint_hash_ops`, `tpcpatch_hash_ops`) admiten predicados de igualdad y ordenación. Las clases de operadores GiST R-tree y SP-GiST quadtree / kd-tree aceleran los predicados basados en la caja delimitadora (`&&`, `@>`, `<@`, `~=`, `-|-`) sobre los tres tipos `tpcbox`, `tpcpoint` y `tpcpatch`:

```
-- GiST (R-tree)
CREATE INDEX ON my_table USING gist(my_tpcpoint_column);
CREATE INDEX ON my_table USING gist(my_tpcpatch_column);

-- SP-GiST quadtree (default) or kd-tree
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column);
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column tpcpoint_kdtree_ops);
```

Las clases de operadores SP-GiST almacenan internamente un `stbox` (el filtro `pcid` se recupera mediante la revisión del operador), por lo que una consulta contra un índice sobre una columna cuyos valores mezclan múltiples `pcid` devolverá un conjunto ligeramente mayor de filas candidatas para revisión, algo irrelevante cuando cada tabla contiene valores de un único esquema, que es el caso común.

19.8. Agregaciones

- Agregado de caja delimitadora sobre una columna de valores `tpcpoint`, `tpcpatch` o `tpcbox`. Devuelve el `tpcbox` más pequeño que contiene todas las entradas. Agregar a través de `pcid` distintos genera un error, la caja delimitadora sería ininterpretable ya que las dimensiones son relativas al `pcid`.

`extent(tpcpoint) → tpcbox`


```
extent(tpcpatch) → tpcbox
```

```
extent(tpcbox) → tpcbox
```

```
SELECT extent(v) FROM (VALUES
  (tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz)),
  (tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz))) t(v);
-- TPCBOX(ZT((1,1,1),(2,2,2)), [2001-01-01, 2001-01-02]), 1)
```

- Conteo temporal y conteo por ventana de filas superpuestas en cada marca de tiempo. El resultado es un tint. Todas las filas de entrada deben compartir el mismo subtipo temporal (instante / secuencia / conjunto de secuencias).

```
tCount(tpcpoint) → tint, tCount(tpcpatch) → tint
```

```
wCount(tpcpoint, interval) → tint, wCount(tpcpatch, interval) → tint
```

```
SELECT tCount(v) FROM (VALUES
  (tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz)),
  (tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz))) t(v);
-- {1@2001-01-01, 1@2001-01-02}
SELECT wCount(v, interval '1 day') FROM (VALUES
  (tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz)),
  (tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz))) t(v);
-- {[1@2001-01-01, 2@2001-01-02], (1@2001-01-02, 1@2001-01-03]}
```

- Conteo de puntos de pcpatch por instante, sumado a través de las filas. Distinto de tCount(tpcpatch), que cuenta el número de parches (siempre 1 por instante). tnpoints se lee como «cuántos puntos había en la nube en el tiempo t», la primitiva natural para el control de calidad de la ingesta y los paneles de densidad a lo largo del tiempo.

```
tnpoints(tpcpatch) → tint
```

```
SELECT tnpoints(v) FROM (VALUES
  (tpcpatch(pcpnt(1, pcpnt(1, 1, 1, 1), pcpnt(1, 2, 2, 2)), '2001-01-01':: ↵
    timestampz)),
  (tpcpatch(pcpnt(1, pcpnt(1, 3, 3, 3), pcpnt(1, 4, 4, 4)), '2001-01-02':: ↵
    timestampz))) t(v);
-- {2@2001-01-01, 2@2001-01-02}
```

- Densidad de puntos por instante (numPoints / área-bbox-xy) sumada a través de las filas. Cada pcpatch lleva el PCBOUNDS en línea (xmin / xmax / ymin / ymax) por lo que el término de área de la caja delimitadora se lee directamente, sin recálculo. Un parche de 1 punto o colineal produce +Infinity para ese instante; filtre con isfinite() en consultas posteriores si es necesario.

```
tdensity(tpcpatch) → tfloat
```

```
-- Each patch holds 2 points over a 1x1 xy-bbox.
SELECT tdensity(v) FROM (VALUES
  (tpcpatch(pcpnt(1, pcpnt(1, 1, 1, 1), pcpnt(1, 2, 2, 2)), '2001-01-01':: ↵
    timestampz)),
  (tpcpatch(pcpnt(1, pcpnt(1, 3, 3, 3), pcpnt(1, 4, 4, 4)), '2001-01-02':: ↵
    timestampz))) t(v);
-- {2@2001-01-01, 2@2001-01-02}
```

- Combina múltiples valores temporales en uno solo con todos los instantes de entrada fusionados en orden temporal. Todas las entradas deben compartir el mismo pcid y subtipo.

```
merge(tpcpoint) → tpcpoint, merge(tpcpatch) → tpcpatch
```

```
mergeAgg(tpcpoint) → tpcpoint, mergeAgg(tpcpatch) → tpcpatch
```

```
SELECT numInstants(mergeAgg(v)) FROM (VALUES
  (tpcpoint(pcpnt(1, 1, 1, 1), '2001-01-01'::timestampz)),
  (tpcpoint(pcpnt(1, 2, 2, 2), '2001-01-02'::timestampz))) t(v);
-- 2
```

- Agregados de construcción en flujo: `appendInstant` ensambla instantes en una secuencia; `appendSequence` ensambla secuencias en un conjunto de secuencias. Útil para cargadores de trayectorias que consumen filas en orden temporal.

```
appendInstant(tpcpoint[,interp,maxt]) → tpcpoint,appendInstant(tpcpatch) → tpcpatch
appendInstantAgg(tpcpoint[,interp,maxt]) → tpcpoint,appendInstantAgg(tpcpatch) → tpcpatch
appendSequence(tpcpoint) → tpcpoint,appendSequence(tpcpatch) → tpcpatch
appendSequenceAgg(tpcpoint) → tpcpoint,appendSequenceAgg(tpcpatch) → tpcpatch
```

```
SELECT numInstants(appendInstant(v ORDER BY startTimestamp(v))) FROM (VALUES
  (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestampz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestampz))) t(v);
-- 2
```

19.9. Representación binaria y portabilidad

Tanto `tpcpoint` como `tpcpatch` admiten la E/S binaria estándar de MobilityDB a través de `asBinary/tpcpointFromBinary/tpcpatchFromBinary`, y a través de `COPY ... (FORMAT BINARY)` de PostgreSQL. La codificación incrusta el `pcid` de cada valor junto a la carga de dimensiones, por lo que el blob binario es inequívoco sobre bajo qué esquema se escribieron los valores.

Siempre que el backend de codificación tenga el XML de esquema de un `pcid` dado registrado en su caché de esquemas a nivel de MEOS (que la extensión de PG rellena de forma perezosa en el primer uso, escaneando `pointcloud_formats`), el codificador incrusta además el XML de esquema en el blob WKB. El bit `MEOS_WKB_PCSCHEMAFLAG` (0x80) en el byte de banderas temporales lo señala. Un receptor que aún no conoce el `pcid`, por ejemplo, un clúster diferente tras un `pg_dump --format=binary`, analiza y registra automáticamente el esquema incrustado, por lo que la importación tiene éxito sin un paso de precarga de esquema fuera de banda.

Si el backend de codificación solo tiene un `PCSCHEMA` analizado (sin XML de origen) para un `pcid`, el bit de esquema se deja sin establecer y el blob es portable solo entre backends que comparten el mismo catálogo `pointcloud_formats`. Esto coincide con el comportamiento de la codificación anterior sin esquema y es la ruta de reserva cuando la caché se rellenó a mano desde un programa MEOS independiente.

El receptor también necesita que `mobilitydb_init` se haya ejecutado, automático en cualquier backend que haya cargado la extensión `mobilitydb`, ya que el módulo de información de funciones instala allí el hook de análisis de XML.

19.10. Salida de texto

- Devolver la representación textual de un `tpcpoint`, o de un arreglo de ellos, y convertir un `tpcpoint` a texto

```
asText(tpcpoint) → text
asText(tpcpoint[]) → text[]
tpcpoint::text → text
```

```
-- The pcpoint payload renders as hex-encoded WKB.
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestampz));
-- 5C00000001000000640000006400000064000000000000@2001-01-01
```

- Devolver la representación textual de un `tpcpatch`, o de un arreglo de ellos, y convertir un `tpcpatch` a texto

```
asText(tpcpatch) → text
asText(tpcpatch[]) → text[]
tpcpatch::text → text
```

```
-- The pcpatch payload renders as hex-encoded WKB.
SELECT left(asText(tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '↔
  2001-01-01'::timestampz)), 24);
-- EC0100000100000000000000
```

A diferencia de los demás tipos temporales espaciales, aquí `asText` no recibe un argumento `maxdecimaldigits`: un `pcpoint` o un `pcpatch` se imprime como el hex-WKB de su forma serializada, que no tiene decimales que redondear. Lo mismo ocurre con `th3index`.

La salida de texto es el subconjunto seguro de la superficie textual: no existe `tpcpointFromText` / `tpcpatchFromText`, porque `pcpoint_in` / `pcpatch_in` de `pgPointCloud` solo aceptan hex-WKB y el esquema no se puede reconstruir a partir de la forma textual. Para la ida y vuelta use `asBinary` / `asHexWKB` con sus funciones de análisis correspondientes.

19.11. Salida MF-JSON

- Para `tpcpoint`, el tipo temporal se emite como `{"type": "MovingPCPoint", ... "coordinates": [[X,Y,Z], ...], "datetimes": [...]}`. `X/Y/[Z]` se extraen del `pcpoint` de cada instante mediante la caché de esquemas; si el esquema carece de `X/Y` el array de coordenadas está vacío para ese instante.

```
asMFJSON(tpcpoint, options int=0, flags int=0, maxdecimaldigits int=15) → text
```

```
SELECT asMFJSON(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestampz));
-- {"type": "MovingPCPoint", "coordinates": [[1,1,1]],
--   "datetimes": ["2001-01-01T00:00:00+00"], "interpolation": "None"}
```

- Para `tpcpatch`, la etiqueta de tipo es `"MovingPCPatch"` y cada instante emite un pequeño objeto `{"pcid": N, "npoints": N, "bounds": [xmin,xmax,ymin,ymax]}`. La carga de puntos comprimida no está intencionadamente en el JSON, use `asBinary` / `asHexWKB` para el ida y vuelta.

```
asMFJSON(tpcpatch, options int=0, flags int=0, maxdecimaldigits int=15) → text
```

```
SELECT asMFJSON(tpcpatch(pcpatch(1, pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01 ↵
::timestampz));
-- {"type": "MovingPCPatch", "values": [{"pcid": 1, "npoints": 2, "bounds": [1,2,1,2]}],
--   "datetimes": ["2001-01-01T00:00:00+00"], "interpolation": "None"}
```

Cuando `options` se establece en 1 la salida también incluye una clave `"bbox"`, para `tpcpoint` / `tpcpatch` esto es un objeto JSON `TPCBox` que añade un campo `"pcid"` junto al array `"bbox"` con forma de `stbox` estándar.

MF-JSON es solo de salida para `tpcpoint` / `tpcpatch`. La forma JSON no lleva el esquema (`pcid` + disposición de dimensiones) para `tpcpoint`, y es un resumen por instante que descarta la carga por punto para `tpcpatch`, por lo que un analizador inverso no puede reconstruir el valor original. Para un ida y vuelta sin pérdidas use `asBinary` / `tpcpointFromBinary` / `tpcpatchFromBinary`, o `asHexWKB` / `tpcpointFromHexWKB` / `tpcpatchFromHexWKB`.

19.12. Integración con PDAL: `readers.tpcpatch` y `writers.tpcpatch`

MobilityDB incluye complementos PDAL nativos bajo `contrib/pdal/` que leen y escriben valores `tpcpatch` directamente sin pasar por `readers.pgpointcloud + tpcpatchSeq()`. Compílelos con:

```
cd contrib/pdal
cmake -B build && cmake --build build -j
sudo cmake --install build
```

PDAL descubre los complementos mediante `PDAL_DRIVER_PATH` o, tras la instalación, en su ruta de búsqueda por defecto. Verifique con `pdal --drivers | grep tpcpatch`.

19.12.1. `readers.tpcpatch`

Lee cualquier consulta SQL que devuelva filas de (`timestamp`, `pcpatch`, `pcid`) y emite un `PointView` de PDAL con todas las dimensiones del esquema más una dimensión `time_t` que lleva microsegundos desde la época. La decodificación se delega a `pc_patch_from_wkb` de `pgPointCloud`, por lo que se manejan los tres tipos de compresión (`PC_NONE`, `PC_DIMENSIONAL`, `PC_LAZPERF`).

Para descomprimir un `tpcpatch` almacenado en filas por instante que el lector pueda consumir:

```
{
  "type": "readers.tpcpatch",
  "connection": "host=/tmp dbname=mydb",
  "query": "SELECT (EXTRACT(EPOCH FROM timestampN(traj, n))*1000000)::bigint AS t,
              valueN(traj, n)                AS pcp,
              PC_PCId(valueN(traj, n))        AS pcid
            FROM trajectories,
              generate_series(1, numInstants(traj)) n
            WHERE id = 42
            ORDER BY n"
}
```

19.12.2. writers.tpcpatch

Recibe un `PointView` con una dimensión de tiempo configurable, agrupa los puntos por marca de tiempo en valores `pcpatch` por instante, luego despacha un `INSERT`, `UPDATE`, fusión-append o `upsert` contra una columna `tpcpatch` usando los constructores `tpcpatch(pcpatch, timestamptz) + tpcpatchSeq(pcpatch[])` de MobilityDB. La codificación se delega a `pc_patch_to_wkb` de `pgPointCloud` con `pc_patch_compress` opcional. La escala / desplazamiento del esquema se aplica simétricamente, los valores fluyen como sus unidades semánticas (metros, grados, ...) a través de PDAL incluso cuando el esquema los almacena como `int32` escalado.

Opciones:

- `connection` (posicional), cadena de conexión libpq.
- `table` (posicional), nombre de la tabla de destino.
- `column`, nombre de la columna `tpcpatch` (por defecto `traj`).
- `pcid` (requerido), entrada en `pointcloud_formats` cuyo esquema deben coincidir las dimensiones de entrada.
- `id_column`, `id_value`, identifican la fila de destino para cualquier modo distinto de `insert`.
- `mode`, `insert` (por defecto, nueva fila por ejecución de la tubería), `update` (sobrescribe la fila identificada por `id_column = id_value`), `append` (concatena mediante `merge()` sobre la fila existente), o `upsert` (inserta si está ausente, añade si está presente). `append` / `update` generan un error claro «`matched no row`» si no se encuentra `id_value`, por lo que no pueden ocurrir escrituras silenciosas de cero filas.
- `time_dim`, nombre de la dimensión de tiempo de origen (por defecto `time_t`; establézcalo en `GpsTime` para consumir marcas de tiempo LAS directamente sin una etapa `filters.assign`).
- `time_format`, interpretación de `time_dim`: `unix_microseconds` (por defecto), `unix_seconds`, o `gps_adjusted` (Tiempo GPS Estándar Ajustado LAS-1.4, el escritor aplica el desplazamiento de época GPS-Unix y el offset de segundo intercalar de 2024).
- `compression`, `none` (por defecto), `dimensional`, o `laz`. MobilityDB normaliza el almacenamiento en disco de `tpcpatch` a `PC_NONE`, por lo que la opción solo afecta al tamaño de transporte libpq, la reducción típica es ~3x para `dimensional` en datos con dimensiones redundantes.
- `flush_threshold` (por defecto 1024), número de parches por instante completados almacenados en memoria antes de emitir un `INSERT/append`. Se respeta para `mode=append/upsert`; `mode=insert/update` difieren a un único vaciado final ya que cada ejecución de la tubería produce una única fila.

El escritor emite líneas de progreso periódicas de nivel `Info` mediante el registrador de PDAL y un resumen final en `done()`: total de puntos / parches escritos, columna de destino, `pcid`, compresión y modo. Visible con `pdal pipeline -v 5`.

19.12.3. Esquemas de compresión soportados

El lector y el escritor admiten los tres tipos de compresión de `pgPointCloud`: `PC_NONE`, `PC_DIMENSIONAL` y `PC_LAZPERF`. La regresión `PC_LAZPERF` en `contrib/pdal/examples/test_lazperf.sql` requiere que la instalación de `pgPointCloud` en ejecución esté compilada con `--with-lazperf=DIR`; el fixture SQL aborta con un mensaje claro en caso contrario. Los complementos se mantienen bajo `contrib/pdal/` en lugar de en el árbol de compilación principal de MobilityDB para que puedan compilarse y actualizarse de forma independiente.

19.12.4. Limitaciones

- *Endianness de WKB.* `readers.tpcpatch` acepta solo `pcpatch WKB little-endian (NDR)`. Un parche big-endian (primer byte `0x00`) se rechaza con un error claro en lugar de decodificarse mal silenciosamente. `pgPointCloud` emite NDR en toda plataforma común; el soporte XDR sería aditivo.
- *Compresión del `tpcpatch` almacenado.* La opción `compression` de `writers.tpcpatch` controla solo la forma de transporte libpq. El `tpcpatch` de MobilityDB normaliza su almacenamiento de `pcpatch` en disco a `PC_NONE`.
- *Escritura en flujo.* `writers.tpcpatch` implementa la interfaz `Streamable::processOne` de PDAL, por lo que una tubería `readers.las → writers.tpcpatch` se ejecuta en modo de flujo de PDAL automáticamente y nunca mantiene toda la nube de puntos en memoria. La cola de parches completados está limitada por la opción del escritor `flush_threshold` (por defecto 1024); en `mode=append` / `mode=upsert` cada vaciado activado por umbral emite un lote SQL incremental mediante `merge()`, mientras que `mode=insert` / `mode=update` difieren a un único vaciado final en `done()` (semántica de fila única). Esta es la ruta que hace tratable la ingesta de LAS de varios GB.

19.13. Puente PCL: filtros invocables desde SQL y registro

Una extensión complementaria opcional `mobilitydb_pcl` vive bajo `contrib/pcl/`. Enlaza el backend de PostgreSQL en ejecución con la **Point Cloud Library** y expone un pequeño conjunto de operaciones por `pcpatch` como funciones SQL. El despacho de tipo de punto consciente del esquema elige `pcl::PointXYZ`, `pcl::PointXYZI`, o `pcl::PointXYZRGB` en el momento de la llamada según la disposición de dimensiones del `pcid` de entrada.

- `pcpatch_to_pcd(pcpatch) → bytea` y `pcpatch_from_pcd(bytea, pcid) → pcpatch`, ida y vuelta a través del formato de cable PCD de PCL. El ida y vuelta preserva todos los valores de dimensión de forma bit-exacta.
- `pcpatch_voxel_grid(pcpatch, leaf double precision) → pcpatch`, submuestreo `VoxelGrid`, leaf en las unidades espaciales del esquema. Preserva `Intensity / RGB`.
- `pcpatch_sor(pcpatch, k integer DEFAULT 50, stddev_mul double precision DEFAULT 1.0) → pcpatch`, Eliminación de valores atípicos estadística (Statistical Outlier Removal). Descarta los puntos cuya distancia media a sus `k` vecinos más cercanos cae fuera de $(media \pm stddev_mul \times stddev)$. Preserva `Intensity / RGB`.
- `pcpatch_icp(source pcpatch, target pcpatch, max_iter int DEFAULT 50, max_corr double DEFAULT 1.0) → double[]` y `pcpatch_gicp(...)` con la misma firma, registro rígido `IterativeClosestPoint` y `GeneralizedICP`. Ambos devuelven un array de 8 dobles `[tx, ty, tz, qw, qx, qy, qz, fitness]` componible en una pose de MobilityDB mediante `pose_make_3d`. El coste de Mahalanobis de GICP sobre covarianzas locales por punto es notablemente más robusto que ICP en superficies no planas / con vegetación (escaneos de corredores viales, fachadas patrimoniales fotogramétricas).
- `pcpatch_normals(pcpatch, k int DEFAULT 10) → double[]`, normal de superficie + curvatura por punto mediante `NormalEstimation<PointXYZ, Normal>` de PCL sobre los `k` vecinos más cercanos. Devuelve un array plano de longitud $4 \times npoints$ dispuesto como `[nx_0, ny_0, nz_0, curv_0, nx_1, ny_1, nz_1, curv_1, ...]`. Casos de uso: detección de cambios de superficie orientada entre estudios repetidos, exportación Potree `--normal`, siembra de mallas fotogramétricas. Para parches muy grandes (>50 k puntos) considere submuestrear primero con `pcpatch_voxel`.

El puente está condicionado a `CREATE EXTENSION mobilitydb_pcl` en bases de datos que ya han cargado `pointcloud` + `mobilitydb`. Las instrucciones de compilación y una suite de regresión determinista están en `contrib/pcl/README.md`.

19.14. Advertencias

- La extensión `pointcloud` debe existir en la base de datos antes de que las definiciones SQL de `mobilitydb` hagan referencia a `pcpoint` / `pcpatch`. En una compilación `POINTCLOUD=ON` el `mobilitydb.control` generado declara esta dependencia en su cláusula `requires =`, por lo que `CREATE EXTENSION mobilitydb CASCADE` resuelve `pointcloud` automáticamente. La creación manual en el orden incorrecto sin `CASCADE` sigue generando un error de tipo ausente.
- La mezcla de esquemas se rechaza en el momento de la construcción: no se puede construir una secuencia `tpcpoint` cuyos instantes tengan valores `pcid` diferentes, y la misma restricción se aplica a `pcpointset` y `pcpatchset`. El esquema es lo que da significado a las dimensiones, y mezclar esquemas silenciosamente produciría valores cuyas coordenadas son ininterpretables.
- El flag de CMake `POINTCLOUD=ON` es obligatorio. Las compilaciones sin él producen un `libMobilityDB` que no declara ninguno de los tipos de este capítulo; los archivos SQL para ellos no se generan.

Capítulo 20

Muestreo de Ráster

La extensión **ráster de PostGIS** almacena datos de cobertura en cuadrícula (elevación, temperatura, imágenes satelitales, ...) en el tipo `raster`. Cada ráster tiene una o más bandas; una banda contiene un arreglo 2D de valores de píxel, una extensión espacial, un tamaño de píxel y un valor centinela opcional para datos nulos (`nodata`).

El operador de muestreo de ráster de MobilityDB evalúa una banda de ráster en los instantes de una trayectoria de punto móvil y devuelve un `tfloat`. Los instantes cuya posición cae fuera de la extensión del ráster o sobre un píxel `nodata` se descartan silenciosamente. Los operadores de muestreo son de valor doble sea cual sea el tipo de píxel de la banda, de modo que un tipo de píxel pertenece a la superficie de muestreo cuando todo valor que puede contener es exactamente representable en un número de doble precisión. Eso es lo que permite muestrear una banda de cualquier tipo en un único `tfloat`, y que una columna con teselas de varios tipos de píxel se lea con una sola consulta. Un tipo de píxel también se acepta con el nombre que le da el ráster de PostGIS, de modo que el tipo de banda que informa una llamada a `ST_BandPixelType` (8BUI, 16BSI, 32BF, ...) se pasa a los constructores de teselas tal cual. El ráster de PostGIS no lleva flotantes de 16 bits ni enteros de 64 bits, así que `float16`, `int64` y `uint64` toman la grafía que da su gramática, 16BF, 64BSI y 64BUI. Sus tipos 1BB, 2BUI y 4BUI acotan un píxel a menos de un byte mientras que PostGIS almacena cada uno de ellos a un byte por píxel, de modo que tal banda ya son los bytes de una banda de 8 bits sin signo y los constructores la leen como `uint8`; la cota es lo único que añade el nombre, y una tesela no la lleva. El nombre que informa una tesela es el que la especificación RaQuet da al tipo. Los tipos `uint64` e `int64` contienen valores fuera de ese dominio: una tesela de cualquiera de ellos se almacena y se vuelve a leer exactamente, mientras que muestrear un píxel cuyo valor ningún número de doble precisión nombra genera un error en lugar de responder con un valor vecino. Esto permite consultas como *¿qué perfil de elevación siguió cada vehículo?* o *¿qué viajes entraron en una banda de umbral de temperatura?*

Para usar los operadores descritos aquí, MobilityDB debe ser compilado con `-DRASTER=ON`. En tal compilación, el archivo `mobilitydb.control` generado declara `requires = 'postgis, postgis_raster'`, por lo que un único CASCADE crea la pila completa:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE: installing required extension "postgis"
-- NOTICE: installing required extension "postgis_raster"
```

20.1. Operador de Muestreo

- Muestrear una banda de ráster en cada instante de una trayectoria

```
rasterValue(tgeompoint, raster, band integer DEFAULT 1) → tfloat
```

Evalúa el píxel de la `band` en cada instante de `traj` usando muestreo bilineal-vecino más próximo (`ST_Value` con `resample=false`). Devuelve un `tfloat` de conjunto de instantes cuyos valores se redondean a cuatro decimales. Devuelve `NULL` cuando ningún instante se interseca con el ráster.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1x1 textdegree{} pixels)
WITH rast AS (
  SELECT ST_SetValues(
```

```

    ST_AddBand(
      ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
      '32BF'::text, 0.0::float8, NULL::float8),
    1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
          [40.0::float4, 50.0::float4, 60.0::float4],
          [70.0::float4, 80.0::float4, 90.0::float4]]) AS r
  )
SELECT rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2000-01-01,
  POINT(2.5 2.5)@2000-01-02, POINT(0.5 0.5)@2000-01-03]', r)::text
FROM rast;

```

Aplicado a un conjunto de datos de trayectorias con un ráster de elevación del terreno:

```

-- Elevation profile per trip (requires a loaded elevation raster)
SELECT tripid, rasterValue(trip, elev) AS elevation
FROM trips, elevation_raster;

-- Average elevation per trip
SELECT tripid, avgValue(rasterValue(trip, elev)) AS mean_elev
FROM trips, elevation_raster;

-- Trips that crossed terrain above 200 m
SELECT tripid
FROM trips, elevation_raster
WHERE atSpan(rasterValue(trip, elev), floatspan '[200, 9999]') IS NOT NULL;

```

20.2. Accesores de Ráster

- Devolver el número de bandas de un ráster

`numBands(raster) → integer`

El número de banda que aceptan `rasterValue` y los operadores construidos sobre él va de 1 hasta este valor.

```

WITH rast1 AS (
  SELECT ST_AddBand(
    ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
    '32BF'::text, 0.0::float8, NULL::float8
  ) AS r
), rast2 AS (
  SELECT ST_AddBand(r, '32BF'::text, 0.0::float8, NULL::float8) AS r
  FROM rast1
)
SELECT numBands((SELECT r FROM rast1)) AS num_bands_one,
       numBands((SELECT r FROM rast2)) AS num_bands_two;
-- 1 | 2

```

20.3. Muestreo Raquet / QUADBIN

Raquet es un formato ráster nativo en la nube que almacena teselas ráster como filas en un archivo Apache Parquet. Cada fila contiene los bytes de píxel de una tesela Web-Mercator junto con su identificador de celda **QUADBIN**, un entero de 64 bits que codifica el nivel de zoom y las coordenadas x/y intercaladas mediante Morton. No se requieren metadatos espaciales adicionales: el rectángulo delimitador y el mapeo de píxel a coordenadas quedan completamente determinados por el valor QUADBIN.

El patrón de consulta típico une una trayectoria con una tabla Raquet en dos pasos:


```
-- Step 1: identify which tiles the trajectory overlaps
SELECT DISTINCT q
FROM unnest(quadbins(traj, 8)) AS q;

-- Step 2: sample each matching tile
SELECT rasterTileValueQuadbin(traj,
    band_data, width, height, quadbin, 'float32', nodata, true)
FROM elevation_raquet
WHERE quadbin = ANY(quadbins(traj, 8));
```

- Devolver las celdas QUADBIN distintas en un nivel de zoom cubiertas por una trayectoria

`quadbins(tgeompoint, integer) → bigint[]`

Devuelve el conjunto de celdas QUADBIN únicas (deduplicadas) cuyas envolventes de tesela Web-Mercator contienen al menos un instante de `traj`. El resultado es adecuado como clave de unión en la cláusula `WHERE` contra una tabla `Raquet`. El parámetro `zoom` debe estar en `[0, 15]`.

```
-- Three instants spanning three tiles at zoom 3
SELECT quadbins(tgeompoint 'SRID=4326;{Point(-60 45)@2024-01-01,
    Point(0 45)@2024-01-02, Point(60 45)@2024-01-03}', 3);
-- {5202501994543054848, 5203346419473186816, 5203416788217364480}

-- Count of Raquet tiles a ship trajectory overlaps at zoom 8
SELECT array_length(quadbins(trip, 8), 1) AS tile_count
FROM trips
ORDER BY tile_count DESC;
```

- Muestrear un chip ráster de `Raquet` a lo largo de una trayectoria usando una celda QUADBIN para la georreferenciación

`rasterTileValueQuadbin(tgeompoint, bytea, integer, integer, bigint, text, float8, boolean) → tfloat`

Evalúa un arreglo de píxeles de banda única en orden de fila mayor en cada instante de `traj` (SRID 4326). La georreferenciación de la tesela (rectángulo delimitador, mapeo de píxel a coordenadas) se deriva únicamente de `quadbin` mediante la transformada de tesela deslizante Web-Mercator. El argumento `pixtype` debe ser uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`. Cuando `has_nodata` es verdadero, los instantes cuyo píxel equivale a `nodata` se descartan silenciosamente. Devuelve `NULL` cuando ningún instante sobrevive.

```
-- Sample elevation tiles from a Raquet Parquet table
-- for all ship trajectories
SELECT t.tripid, rasterTileValueQuadbin(t.trip,
    r.band_data, r.width, r.height, r.quadbin, 'float32', r.nodata, true)
FROM trips t
JOIN elevation_raquet r ON r.quadbin = ANY(quadbins(t.trip, 8));

-- Average sea-surface temperature per trajectory
SELECT t.tripid, avgValue(rasterTileValueQuadbin(t.trip,
    r.band_data, 256, 256, r.quadbin, 'int16', -9999, true)) AS mean_sst
FROM trips t
JOIN sst_raquet r ON r.quadbin = ANY(quadbins(t.trip, 6));
```

- Construir una tesela ráster `Raquet` a partir de sus bytes de píxeles, dimensiones, celda QUADBIN y tipo de píxel

`raquet(bytea, integer, integer, bigint, text, float8) → raquet`

Empaqueta un arreglo de píxeles `pixels` de banda única en orden de fila mayor de `width × height` píxeles de tipo `pixtype` (uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`), georreferenciado por la celda `quadbin`, en un único valor `raquet` autodescriptivo. El argumento `nodata` es opcional; cuando se omite (`NULL`) la tesela no lleva valor centinela de `nodata`. Se genera un error cuando el arreglo `pixels` es menor que `width × height × el tamaño del píxel`. Los píxeles de más de un byte son little-endian, que es el orden de bytes que les da la especificación `Raquet` sea cual sea el orden de bytes del servidor, de modo que una tesela conserva sus

valores cuando se lee en otra máquina. Un `raquet` es un valor de longitud variable, libre de GDAL, con una representación portátil en texto Hex-WKB y binaria, distinto de y coexistente con el tipo raster de PostgreSQL usado por `rasterValue`.

```
-- Empaquetar las columnas sueltas de una tabla Raquet en valores raquet
SELECT raquet(band_data, width, height, quadbin, 'float32', nodata) AS tile
FROM elevation_raquet;

-- Una tesela UINT8 de 2 x 2 sin nodata
SELECT raquet('\x01020304':bytea, 2, 2, 5193776270265024512, 'uint8');
```

- Decodificar un archivo ráster en memoria en una tesela ráster Raquet mediante GDAL

```
raquetRead(bytea,bigint) → raquet
```

Lee los bytes de `rasterfile`, un ráster en cualquier formato compatible con GDAL (GeoTIFF, PNG, ...), a través del sistema de archivos virtual `/vsimem/` de GDAL, y empaqueta su primera banda, en orden de fila mayor, en un único valor `raquet` autodescriptivo georreferenciado por la celda `quadbin`. El tipo de datos de la banda debe ser uno de `Byte`, `Int16`, `Int32`, `Float32` o `Float64`; el valor de `nodata` de la banda, cuando está definido, se traslada a la tesela. Como el archivo se suministra como bytes, no se requiere acceso a archivos del lado del servidor. GDAL realiza la decodificación, por lo que el controlador de formato del ráster debe estar permitido por la configuración `postgis.gdal_enabled_drivers` de PostgreSQL (que deshabilita todos los controladores de forma predeterminada); habilítelo con, por ejemplo, `SET postgis.gdal_enabled_drivers = 'ENABLE_ALL'`. Esta es la contraparte de ingesta del constructor `raquet`, que construye una tesela a partir de bytes de píxeles ya decodificados.

Cuando se omite `quadbin` o es `NULL`, el identificador de la tesela se deriva de la georreferenciación del ráster: el ráster debe llevar una referencia espacial EPSG:3857 (Web-Mercator) y una geotransformación alineada con los ejes que describa una única tesela QUADBIN. Un ráster sin referencia espacial, uno que no esté en EPSG:3857, o una geotransformación rotada o no alineada con la cuadrícula de teselas provoca un error en lugar de ser forzado.

```
-- Decodificar un GeoTIFF almacenado en una columna bytea en una tesela raquet
SELECT raquetRead(file_bytes, quadbin) AS tile
FROM elevation_files;
-- Derivar la celda QUADBIN de la georreferenciación de un GeoTIFF EPSG:3857
SELECT raquetRead(file_bytes) AS tile
FROM elevation_files;
```

- Muestrear una tesela ráster `raquet` a lo largo de una trayectoria

```
rasterTileValue(tgeompoint,raquet) → tfloat
```

Evalúa la tesela en cada instante de `traj` (SRID 4326), devolviendo los valores de píxel muestreados como un `tfloat`. Es el equivalente tipado de `rasterTileValueQuadbin`: las dimensiones, la celda QUADBIN, el tipo de píxel y el tratamiento de `nodata` se toman del valor `raquet` en lugar de argumentos separados. Devuelve `NULL` cuando ningún instante cae dentro de la tesela o sobrevive al filtrado de `nodata`.

```
-- Muestrear teselas raquet preempaquetadas a lo largo de trayectorias de barcos
SELECT t.tripid, rasterTileValue(t.trip, r.tile)
FROM trips t
JOIN elevation_tiles r ON r.quadbin = ANY(quadbins(t.trip, 8));
```

- Muestrear un arreglo de teselas ráster `raquet` a lo largo de una trayectoria

```
rasterTileValue(tgeompoint,raquet[]) → tfloat
```

Evalúa cada tesela de `rast` en cada instante de `traj` (SRID 4326) y devuelve los valores de píxel muestreados como un único `tfloat`. Una trayectoria que sale de una tesela queda cubierta por varias, y cada una aporta los instantes que caen dentro de ella. Las teselas de un mismo nivel de zoom particionan el plano, de modo que aportan instantes disjuntos. Las teselas de niveles de zoom distintos se solapan, y cuando dos de ellas muestrean el mismo instante se conserva el valor de la tesela de mayor zoom, que es la que tiene la resolución más fina. Devuelve `NULL` cuando ningún instante cae dentro de una tesela o sobrevive al filtrado de `nodata`.

```
-- Muestrear una trayectoria en todas las teselas que cruza, en un solo valor
SELECT t.tripid, rasterTileValue(t.trip, array_agg(r.tile))
FROM trips t
```

```
JOIN elevation_tiles r ON r.quadbin = ANY(quadbins(t.trip, 8))
GROUP BY t.tripid, t.trip;
```

20.4. Serialización de Raquet

Las funciones `asBinary` y `asHexWKB` serializan un valor `raquet`, y `raquetFromBinary` y `raquetFromHexWKB` lo reconstruyen. Una tesela lleva sus píxeles y su georreferenciación QUADBIN en un único valor, de modo que hace un viaje de ida y vuelta a través de una cadena de bytes portable sin intervención de ninguna extensión espacial, y la forma hexadecimal permite que una columna de teselas haga ese viaje mediante un COPY basado en texto. Ambas funciones de salida aceptan un argumento opcional de ordenación de bytes, NDR para little-endian o XDR para big-endian, cuyo valor predeterminado es la ordenación del servidor.

- Devolver la representación Well-Known Binary (WKB) de una tesela Raquet

`asBinary(raquet, endianencoding text DEFAULT '') → bytea`

```
SELECT length(asBinary(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512,
'uint8')));
-- 31
```

- Devolver la representación Well-Known Binary codificada en hexadecimal ASCII (HexWKB) de una tesela Raquet

`asHexWKB(raquet, endianencoding text DEFAULT '') → text`

```
SELECT length(asHexWKB(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512,
'uint8')));
-- 62
```

- Devolver los bytes de píxeles de un mosaico Raquet

`pixels(raquet) → bytea`

Los bytes están en orden de fila mayor y empaquetados, `width × height` píxeles del tamaño de `pixtype` cada uno, en little-endian cuando un píxel ocupa más de un byte, que es la disposición que aceptan los constructores. Por lo tanto, un mosaico se reconstruye a través de sus propios accesores.

```
SELECT pixels(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8'));
-- \x01020304
```

- Devolver una tesela Raquet a partir de su representación Well-Known Binary (WKB)

`raquetFromBinary(bytea) → raquet`

```
SELECT raquetFromBinary(asBinary(raquet('\x01020304'::bytea, 2, 2,
5193776270265024512, 'uint8')))
= raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8');
-- true
```

- Devolver una tesela Raquet a partir de su representación Well-Known Binary codificada en hexadecimal ASCII (HexWKB)

`raquetFromHexWKB(text) → raquet`

```
SELECT raquetFromHexWKB(asHexWKB(raquet('\x01020304'::bytea, 2, 2,
5193776270265024512, 'uint8')))
= raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8');
-- true
```

20.5. Accesorios y Comparación de Raquet

Un valor `raquet` es autodescritivo: los accesorios siguientes leen la georreferenciación y la disposición que transporta, de modo que una tesela empaquetada con el constructor `raquet` o decodificada con `raquetRead` puede inspeccionarse sin conservar las columnas sueltas a partir de las cuales se construyó.

Las teselas también admiten el conjunto completo de operadores de comparación. El orden es total y se define primero por la celda QUADBIN, luego por el tipo de píxel, el ancho, el alto y finalmente los bytes de píxeles; está pensado para indexación y deduplicación, no para una interpretación espacial. Una clase de operadores btree por defecto hace que `ORDER BY`, `DISTINCT` y las uniones por mezcla funcionen sobre columnas `raquet`, y una clase de operadores hash por defecto habilita las uniones y agregaciones hash.

- Devolver la celda QUADBIN de una tesela Raquet

`quadbin(raquet) → bigint`

```
SELECT quadbin(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8'));
-- 5193776270265024512
```

- Devolver el ancho en píxeles de una tesela Raquet

`width(raquet) → integer`

```
SELECT width(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8'));
-- 2
```

- Devolver el alto en píxeles de una tesela Raquet

`height(raquet) → integer`

```
SELECT height(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8'));
-- 2
```

- Devolver el valor centinela nodata de una tesela Raquet

`nodata(raquet) → float8`

```
SELECT nodata(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8', 255));
-- 255
-- The sentinel defaults to 0 when the constructor omits it.
SELECT nodata(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8'));
-- 0
```

- Devolver el nombre del tipo de dato de píxel de una tesela Raquet

`pixtype(raquet) → text`

El nombre devuelto es el que aceptan los constructores, es decir, uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`. Los constructores leen el nombre sin distinguir mayúsculas de minúsculas, de modo que el nombre que el ráster de PostGIS da a un tipo de píxel pasa a ellos tal cual; el nombre devuelto aquí es la gráfica en minúsculas que la especificación RaQuet da al campo `type` de una tesela, por lo que se compara igual a ese campo tal cual.

```
-- Leer la disposición de una tesela empaquetada
SELECT quadbin(tile), width(tile), height(tile), pixtype(tile), nodata(tile)
FROM (SELECT raquet('\x01020304'::bytea, 2, 2, 5193776270265024512, 'uint8') AS tile) t;
-- 5193776270265024512 | 2 | 2 | uint8 | 0
```

- Convertir una tesela Raquet en una caja espaciotemporal

`stbox(raquet) → stbox`

Devuelve la huella de la tesela: la envolvente de longitud y latitud de su celda QUADBIN, en SRID 4326 y sin dimensión temporal. La huella se deriva únicamente de la celda, por lo que teselas que difieren en píxeles, dimensiones o tipo de píxel la comparten. La conversión también está disponible como conversión de tipo.

```
-- The footprint of a tile covering the north-eastern quadrant at zoom 1
SELECT round(stbox(raquet('\x01020304'::bytea, 2, 2, 5193776270265024512,
    'uint8')), 6);
-- SRID=4326;STBOX X((0,0),(180,85.051129))

-- Keep the tiles whose footprint overlaps a region of interest
SELECT * FROM elevation_tiles
WHERE tile::stbox && stbox 'SRID=4326;STBOX X((0,0),(30,30))';
```

La huella admite los operadores, agregados y clases de operadores de stbox, por lo que una tabla de teselas se consulta por solapamiento espacial en cualquier nivel de zoom en lugar de por una prueba de igualdad sobre la celda QUADBIN en uno fijo. Indexar la conversión dota a la tabla de teselas de un índice espacial, y la extensión de un conjunto de teselas es la extensión de sus huellas.

```
-- A spatial index over the tile footprints
CREATE INDEX elevation_tiles_gist ON elevation_tiles USING gist (stbox(tile));
CREATE INDEX elevation_tiles_spgist ON elevation_tiles USING spgist (stbox(tile));

-- The extent covered by a set of tiles
SELECT extent(stbox(tile)) FROM elevation_tiles;

-- Tiles a trajectory passes through, at whatever zoom levels the table holds
SELECT t.tripid, r.tile
FROM trips t, elevation_tiles r
WHERE stbox(r.tile) && stbox(t.trip);
```

■ Comparar dos teselas Raquet

```
raquet = raquet → boolean
raquet <> raquet → boolean
raquet < raquet → boolean
raquet <= raquet → boolean
raquet >= raquet → boolean
raquet > raquet → boolean
```

Las funciones que respaldan estos operadores son eq, ne, lt, le, ge y gt, junto con cmp(raquet, raquet) → integer, que devuelve -1, 0 o 1.

```
-- Dos teselas con la misma celda, disposición y píxeles son iguales
SELECT raquet('\x0102'::bytea, 2, 1, 5193776270265024512, 'uint8') =
    raquet('\x0102'::bytea, 2, 1, 5193776270265024512, 'uint8');
-- true

-- Deduplicar teselas, lo que permiten las clases de operadores btree y hash
SELECT DISTINCT tile FROM elevation_tiles;
```

20.6. Operadores de Restricción y Predicado

Los siguientes operadores definidos en SQL componen rasterValue con las funciones estándar de restricción temporal y predicado de MobilityDB. Aceptan un argumento opcional band (por defecto 1) e invocan rasterValue exactamente una vez por invocación.

■ Devolver los instantes de una trayectoria donde el valor ráster muestreado cae dentro de un rango de flotante

```
atRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → tgeompoint
```

Evalúa rasterValue en cada instante de traj y devuelve la sub-trayectoria restringida a los tiempos en que el valor del píxel está dentro de vspan. Devuelve NULL cuando ningún instante satisface la condición.

```
WITH rast AS (
  SELECT ST_SetValues(
    ST_AddBand(
      ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
      '32BF'::text, 0.0::float8, NULL::float8),
    1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
          [40.0::float4, 50.0::float4, 60.0::float4],
          [70.0::float4, 80.0::float4, 90.0::float4]]) AS r
)
SELECT asText(atRasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2000-01-01,
POINT(1.5 1.5)@2000-01-02, POINT(0.5 0.5)@2000-01-03]', r, floatspan '[40, 90]'))
FROM rast;
```

- Devolver los instantes de una trayectoria donde el valor ráster muestreado cae fuera de un rango de flotante

`minusRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → tgeompoint`

El complemento de `atRasterValue`: devuelve la sub-trayectoria restringida a los tiempos en que el valor del píxel está fuera de `vspan`. Devuelve `NULL` cuando todos los instantes satisfacen la condición.

```
-- Keep only the instant whose raster value (10) is below 40
SELECT asText(minusRasterValue(traj, elev_raster, floatspan '[40, 9999]'))
FROM trips, elevation_raster;
```

- Devolver verdadero si la trayectoria alguna vez muestrea un valor de píxel ráster dentro de un rango de flotante

`eRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → boolean`

Devuelve `true` cuando al menos un instante dentro de la extensión de `traj` muestrea un valor de píxel dentro de `vspan`, `false` en caso contrario. Devuelve `NULL` únicamente cuando la entrada es `NULL`.

```
-- Trips that ever crossed terrain above 200 m
SELECT tripid
FROM trips, elevation_raster
WHERE eRasterValue(trip, elev_raster, floatspan '[200, 9999]');
```

- Devolver verdadero si cada instante de la trayectoria dentro de la extensión del ráster muestrea un valor de píxel dentro de un rango de flotante

`aRasterValue(tgeompoint, raster, floatspan, band integer DEFAULT 1) → boolean`

Devuelve `true` cuando cada instante dentro de la extensión de `traj` muestrea un valor de píxel dentro de `vspan`, `false` cuando al menos uno no lo hace. Devuelve `NULL` únicamente cuando la entrada es `NULL`.

```
-- Trips that stayed entirely below 100 m elevation
SELECT tripid
FROM trips, elevation_raster
WHERE aRasterValue(elev_raster, trip, floatspan '[0, 100]');
```

20.7. Compilación e Instalación

El ráster de PostGIS forma parte del paquete estándar `postgresql-N-postgis-3` en Debian/Ubuntu; no se requiere ningún paquete adicional. Pase `-DRASTER=ON` a CMake:

```
cmake -DRASTER=ON ..
make -j$(nproc)
sudo make install
```

Para incluir el ráster en la compilación de cobertura de CI junto con otras familias opcionales, añada `-DRASTER=ON` a la invocación de `cmake` en `.github/workflows/pgversion.yml`.

20.8. Hoja de Ruta de Producción

Las dos vías descritas en este capítulo sirven propósitos distintos y ambas se mantienen. La vía `raquet` es un tipo de valor MEOS autocontenido: transporta sus píxeles y su georreferenciación QUADBIN en un único valor y no necesita PostgreSQL para evaluarse, por lo que está disponible en todos los enlaces de lenguaje derivados de MEOS. La vía `raster` de PostGIS ofrece el procesamiento ráster mucho más rico de la extensión `postgis_raster`, álgebra de mapas, estadísticas, remuestreo, recorte, conversión a polígonos, únicamente dentro de PostgreSQL.

Ninguna vía reemplaza a la otra. El trabajo sobre la vía `raquet` amplía lo que una tesela nativa de la nube puede hacer a lo largo de una trayectoria; no pretende reimplementar el procesamiento ráster.

Las extensiones siguientes requieren un modelo de datos ráster general, es decir, sistemas de referencia espacial arbitrarios, georreferenciación afín arbitraria y varias bandas por tesela. Un valor `raquet` es una tesela de una sola banda cuya extensión y rejilla de píxeles se derivan de su celda QUADBIN, por lo que quedan fuera de ese modelo y siguen disponibles a través de la vía `raster` de PostGIS:

- *Teselas multibanda*, un `tfloat` por banda para imágenes multispectrales como RGB o escenas satelitales multicanal.
- *Sistemas de referencia espacial arbitrarios*, teselas fuera de la rejilla Web-Mercator y reproyección entre sistemas de referencia.
- *Procesamiento ráster*, álgebra de mapas, estadísticas de resumen, remuestreo y conversión a polígonos, junto con el recorte de una tesela a una región, como la extensión que un conjunto de trayectorias visita durante un intervalo de tiempo.

Capítulo 21

Dialecto Portable de Funciones con Nombre

Todo operador de MobilityDB puede invocarse también como una función con nombre. Una consulta escrita únicamente con funciones con nombre, sin símbolos de operador, es portable: el mismo SQL se ejecuta sin cambios en todo el ecosistema MobilityDB, los motores de base de datos MobilityDB (PostgreSQL), MobilityDuck (DuckDB) y MobilitySpark (Apache Spark), y los motores de *streaming*, varios de los cuales no pueden registrar símbolos de operador de PostgreSQL. Cada alias con nombre reutiliza la propia implementación del operador, por lo que devuelve exactamente el mismo resultado que el operador.

Este capítulo es la referencia canónica, a nivel de todo el ecosistema, de esa correspondencia. Un motor que no acepte un operador dado puede consultarlo aquí y usar la forma de función en su lugar. Por ejemplo, DuckDB no acepta ningún operador que contenga # (reserva # para las referencias posicionales de columna), de modo que en MobilityDuck las comparaciones temporales (#=, ...) se escriben `tEq, ...` y las pruebas de posición temporal (<<#, #>>, &<#, #&>) se escriben `before, after, overbefore, overafter`, mientras que los operadores que sí acepta pueden usarse directamente.

Las funciones de relación espacial (`eIntersects`, `aIntersects`, `eTouches`, `eDwithin`, ...) y las funciones de restricción (`atTime`, `atGeometry`, `atValue`, ...) son siempre funciones con nombre (no tienen operador). El comparador de ordenación de árbol B es asimismo la función con nombre `cmp(a, b)`, que devuelve -1, 0 o 1 y no tiene operador. Cada operador se corresponde con un nombre simple de la manera siguiente; para los operadores aritméticos y de teoría de conjuntos el nombre de la función lleva el prefijo de la familia de tipos del operando (`set`, `span` o `spanset`, y `t` para los valores temporales).

Categoría	Operador	Función portable
Topología	<code>&&</code>	<code>overlaps(a, b)</code>
	<code>@></code>	<code>contains(a, b)</code>
	<code><@</code>	<code>contained(a, b)</code>
	<code>- -</code>	<code>adjacent(a, b)</code>
Posición temporal	<code><<#</code>	<code>before(a, b)</code>
	<code>#>></code>	<code>after(a, b)</code>
	<code>&<#</code>	<code>overbefore(a, b)</code>
	<code>#&></code>	<code>overafter(a, b)</code>
Espacio, eje X	<code><<</code>	<code>left(a, b)</code>
	<code>>></code>	<code>right(a, b)</code>
	<code>&<</code>	<code>overleft(a, b)</code>
	<code>&></code>	<code>overright(a, b)</code>
Espacio, eje Y	<code><< </code>	<code>below(a, b)</code>
	<code> >></code>	<code>above(a, b)</code>
	<code>&< </code>	<code>overbelow(a, b)</code>
	<code> &></code>	<code>overabove(a, b)</code>
Espacio, eje Z	<code><</</code>	<code>front(a, b)</code>
	<code>/>></code>	<code>back(a, b)</code>
	<code>&</</code>	<code>overfront(a, b)</code>
	<code>/&></code>	<code>overback(a, b)</code>
Comparación temporal	<code>#=</code>	<code>tEq(a, b)</code>

Categoría	Operador	Función portable
	#<>	tNe(a, b)
	#<	tLt(a, b)
	#<=	tLe(a, b)
	#>	tGt(a, b)
	#>=	tGe(a, b)
Distancia	<->	tDistance(a, b)
	=	nearestApproachDistance(a, b)
Igual	~=	same(a, b)
Comparación tradicional	=	eq(a, b)
	<>	ne(a, b)
	<	lt(a, b)
	<=	le(a, b)
	>	gt(a, b)
	>=	ge(a, b)
Comparación alguna vez	?=	eEq(a, b)
	?<>	eNe(a, b)
	?<	eLt(a, b)
	?<=	eLe(a, b)
	?>	eGt(a, b)
	?>=	eGe(a, b)
Comparación siempre	%=	aEq(a, b)
	%<>	aNe(a, b)
	%<	aLt(a, b)
	%<=	aLe(a, b)
	%>	aGt(a, b)
	%>=	aGe(a, b)
Booleano (booleano temporales)	&	tAnd(a, b)
		tOr(a, b)
	~	tNot(a)
Aritmética (números temporales)	+	tAdd(a, b)
	-	tSub(a, b)
	*	tMul(a, b)
	/	tDiv(a, b)
Unión (set / span / span set)	+	setUnion / spanUnion / spansetUnion (a, b)
Intersección (set / span / span set)	*	setIntersection / spanIntersection / spansetIntersection (a, b)
Diferencia (set / span / span set)	-	setMinus / spanMinus / spansetMinus (a, b)
Concatenación (set / temporal)		setConcat / tConcat (a, b)

Por ejemplo, la superposición de cajas envolventes `t1.trip && t2.trip` se escribe de forma portable como `overlaps(t1.trip, t2.trip)`, y la prueba temporal «antes» `p1.period <<# p2.period` como `before(p1.period, p2.period)`.

Las funciones de agregación siguen el mismo principio y se enumeran en su propia tabla a continuación, de modo que un motor pueda localizar la maquinaria que necesita sin rastrear el texto. Una agregación SQL se compone de hasta tres funciones de la maquinaria de PostgreSQL, cada una expuesta bajo un nombre canónico formado a partir del nombre de la agregación y su rol: una función de transición <agregación>Transition (siempre presente), una función de combinación <agregación>Combine usada para la agregación en paralelo, y una función final <agregación>Final. Los roles de combinación y final son opcionales: una agregación cuyo estado de transición ya es su resultado no tiene función final (por ejemplo `extent` y `minDistance`), y una agregación que no admite la agregación parcial no tiene función de combinación (por ejemplo las agregaciones de `anexo`). Los roles se mantienen distintos y con un nombre uniforme para que cada *binding* pueda reconstruir la agregación a partir del catálogo. La siguiente tabla enumera cada agregación y sus funciones de maquinaria; un guion (—) indica un rol que la agregación no define.

Categoría	Transición	Combinación	Final
Conteo	tCountTransition	tCountCombine	tCountFinal
	wCountTransition	wCountCombine	wCountFinal

Categoría	Transición	Combinación	Final
Suma	tSumTransition	tSumCombine	tSumFinal
	wSumTransition	wSumCombine	wSumFinal
Promedio	tAvgTransition	tAvgCombine	tAvgFinal
	wAvgTransition	wAvgCombine	wAvgFinal
Mínimo / máximo	tMinTransition	tMinCombine	tMinFinal
	tMaxTransition	tMaxCombine	tMaxFinal
	wMinTransition	wMinCombine	wMinFinal
	wMaxTransition	wMaxCombine	wMaxFinal
Booleano	tAndTransition	tAndCombine	tAndFinal
	tOrTransition	tOrCombine	tOrFinal
Centroide	tCentroidTransition	tCentroidCombine	tCentroidFinal
Nube de puntos	tnpointsTransition	tnpointsCombine	tnpointsFinal
	tdensityTransition	tdensityCombine	tdensityFinal
Caja delimitadora	extentTransition	extentCombine	—
Distancia	minDistanceTransition	minDistanceCombine	—
Unión	setUnionTransition	setUnionCombine	setUnionFinal
	spanUnionTransition	spanUnionCombine	spanUnionFinal
	spansetUnionTransition	spansetUnionCombine	spansetUnionFinal
Fusión	mergeTransition	mergeCombine	mergeFinal
Anexado	appendInstantTransition	—	appendInstantFinal
	appendSequenceTransition	—	appendSequenceFinal

Apéndice A

MobilityDB Reference

A.1. Tipos de MobilityDB

MobilityDB define cuatro *tipos de plantilla* (o *template types*) que actúan como constructores de tipos sobre *tipos de base*. Estos tipos de plantilla son `set`, `span`, `spanset` y `temporal`. Tabla A.1 muestra los tipos definidos sobre estos tipos de plantilla. Además, MobilityDB define dos tipos de cuadros delimitadores, a saber, `tbox` y `stbox`.

Tipos de base	Tipos de plantilla			
	set	span	spanset	temporal
bool				tbool
text	textset			ttext
jsonb	jsonbset			tjsonb
integer	intset	intspan	intspanset	tint
bigint	bigintset	bigintspan	bigintspanset	tbigint
float	floatset	floatspan	floatspanset	tfloat
date	dateset	datespan	datespanset	
timestampz	tstzset	tstzspan	tstzspanset	
geometry	geomset			tgeompoint, tgeometry
geography	geogset			tgeogpoint, tgeography
pose	poseset			tpose, trgeometry
posechain	posechainset			tposechain
npoint	npointset			tnpoint
cbuffer	cbufferet			tcbuffer
h3index	h3indexset			th3index
quadbin	quadbinset			tquadbin
pcpoint	pcpointset			tpcpoint
pcpatch	pcpatchset			tpcpatch

Cuadro A.1: Instancias actuales de los tipos de plantilla en MobilityDB

A.2. Set and Span Types

A.2.1. Entrada y salida

- **asText**: Devuelve la representación textual conocida (Well-Known Text o WKT)
- **asBinary**, **asHexWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)

- **asEWKB, asHexEWKB**: Devuelve la representación binaria extendida conocida (Extended Well-Known Binary o EWKB) o la representación hexadecimal binaria extendida conocida (HexEWKB) de un conjunto espacial
- **settypeFromBinary, spantypeFromBinary, spansettypeFromBinary**: Entrar a partir de la representación binaria conocida (WKB)
- **settypeFromHexWKB, spantypeFromHexWKB, spansettypeFromHexWKB**: Entrar a partir de la representación hexadecimal binaria conocida (HexWKB)

A.2.2. Constructores

- **set**: Constructor para tipos de conjunto
- **span**: Constructor para tipos de rango
- **spanset**: Constructor for conjunto de rangos

A.2.3. Conversión de tipos

- **::, set, span, spanset**: Convertir un valor de base en un valor de conjunto, rango o conjunto de rangos
- **::, spanset**: Convertir un valor de conjunto en un valor de conjunto de rangos
- **::, spanset**: Convertir un valor de rango a un valor de conjunto de rangos
- **span**: Convertir un conjunto de valores o un conjunto de rangos a un rango, ignorando las posibles brechas de tiempo
- **::, span, range**: Convertir un valor de de rango en MobilityDB hacia y desde un valor de rango de PostgreSQL
- **::, spanset, multirange**: Convertir un valor de conjunto de rangos de MobilityDB hacia y desde un valor de multirango de PostgreSQL

A.2.4. Accesores

- **memSize**: Devuelve el tamaño de la memoria en bytes
 - **lower, upper**: Devuelve el límite inferior o superior
 - **lowerInc, upperInc**: ¿Es el límite inferior or superior inclusivo?
 - **width**: Devuelve el ancho del rango como un número de punto flotante
 - **duration**: Devuelve el rango de tiempo
 - **numValues, getValues**: Devuelve el número de valores o los valores
 - **startValue, endValue, valueN**: Devuelve el valor inicial, final o enésimo
 - **numSpans, spans**: Devuelve el número de rangos o los rangos
 - **startSpan, endSpan, spanN**: Devuelve el rango inicial, final o enésimo
 - **numDates, dates**: Devuelve el (number of) different dates
 - **startDate, endDate, dateN**: Devuelve el start, end, or n-th date
 - **numTimestamps, timestamps**: Devuelve el número o las marcas de tiempo diferentes
 - **startTimestamp, endTimestamp, timestampN**: Devuelve la marca de tiempo inicial, final, o enésima
-

A.2.5. Transformaciones

- **expand**: Expandir o reducir los límites de un rango con un valor o un intervalo de tiempo
- **shift**: Desplazar con un valor o rango de tiempo
- **scale**: Escalar con un valor o un rango de tiempo
- **shiftScale**: Desplazar y escalar con los valores o rangos de tiempo
- **floor, ceil**: Redondear al entero inferior o superior
- **round**: Redondear a un número de decimales
- **degrees, radians**: Convertir a grados o radianes
- **lower, upper, initcap**: Transformar en minúsculas, mayúsculas o initcap
- **||**: Concatenación de texto
- **tprecision**: Establecer la precisión temporal del valor de tiempo al rango con respecto al origen

A.2.6. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o especifica el identificador de referencia espacial
- **transform, transformPipeline**: Transformar a una referencia espacial diferente

A.2.7. Operaciones de conjuntos

- **+, -, ***: Unión, diferencia o intersección de conjuntos o de rangos

A.2.8. Operaciones de cuadro delimitador

A.2.8.1. Operaciones topológicas

- **&&**: ¿Se superponen los valores (tienen valores en común)?
- **@>**: ¿Contiene el primer valor el segundo?
- **<@**: ¿Está el primer valor contenido en el segundo?
- **-|**: ¿Es el primer valor adyacente al segundo? Dos valores son adyacentes cuando se tocan sin solaparse, es decir, cuando comparten una frontera y no tienen ningún valor en común en ella. Para los tipos base continuos `float` y `timestamp_tz`, un límite compartido es ese contacto. Para los tipos base discretos `int`, `bigint` y `date`, cuyos rangos se almacenan en forma normal, significa valores consecutivos.

A.2.8.2. Operaciones de posición

- **<<, <<#**: ¿Está el primer valor estrictamente a la izquierda del segundo?
 - **>>, #>>**: ¿Está el primer valor estrictamente a la derecha del segundo?
 - **&<, &<#**: ¿No está el primer valor a la derecha del segundo?
 - **&>, #&>**: ¿No está el primer valor a la izquierda del segundo?
-

A.2.8.3. Operaciones de división

- **splitNSpans**: Devuelve una matriz de N rangos obtenida fusionando los elementos de un conjunto o los rangos de un conjunto de rangos
- **splitEachNSpans**: Devuelve una matriz de rangos obtenida fusionando N elementos consecutivos de un conjunto o N rangos consecutivos de un conjunto de rangos

A.2.9. Operaciones de distancia

- **<->**: Devuelve la distancia mínima

A.2.10. Comparaciones

- **=, <, <=, >, >=, cmp**: Comparaciones tradicionales

A.2.11. Agregaciones

- **extent**: Rango o período delimitador
- **setUnion, spanUnion, spansetUnion**: Unión agregada
- **tCount**: Conteo temporal

A.3. Box Types

A.3.1. Entrada y salida

- **asText**: Devuelve la representación textual conocida (Well-Known Text o WKT)
- **asBinary, asHexWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)
- **boxFromBinary, boxFromHexWKB**: Entrar a partir de representación binaria conocida (WKB) o de la representación hexadecimal binaria conocida (HexWKB)

A.3.2. Constructores

- **tbox**: Constructor para tbox
- **stboxX, stboxZ, stboxT, stboxXT, stboxZT, geodstboxZ, geodstboxT, geodstboxZT, stbox**: Constructor para stbox

A.3.3. Conversión de tipos

- **::, intspan, floatspan, tstzspan**: Convertir un tbox a otro tipo
 - **::, tbox**: Convertir otro tipo a un tbox
 - **::, box2d, box3d, geometry, geography, tstzspan**: Convertir un stbox a otro tipo
 - **::, stbox**: Convertir otro tipo a un stbox
-

A.3.4. Accesores

- **hasX, hasZ, hasT**: ¿Tiene dimensión X/Z/T?
- **isGeodetic**: ¿Es geodética?
- **xMin, yMin, zMin, tMin**: Devuelve el valor mínimo de X/Y/Z/T
- **xMax, yMax, zMax, tMax**: Devuelve el valor máximo de X/Y/Z/T
- **xMinInc, tMinInc**: Es el mínimo valor X/T inclusivo?
- **xMaxInc, tMaxInc**: Es el máximo valor X/T inclusivo?
- **area, perimeter, volume**: Área, volumen, perímetro

A.3.5. Transformaciones

- **shiftValue, scaleValue, shiftScaleValue**: Desplazar y/o escalar el rango de valores de un cuadro delimitador con uno o dos valores
- **shiftTime, scaleTime, shiftScaleTime**: Desplazar y/o escalar el período de un cuadro delimitador con uno o dos intervalos. Si el ancho o el lapso de tiempo del valor de tiempo es cero (es decir, los límites inferior y superior son iguales), el resultado es el cuadro delimitador. El valor o intervalo dado debe ser estrictamente mayor que cero.
- **getSpace**: Devuelve la dimensión espacial del cuadro delimitador, eliminando la dimensión temporal, si existe
- **expandValue, expandSpace, expandTime**: Extender la dimensión numérica, espacial o temporal del cuadro delimitador con un valor o un intervalo
- **round**: Redondear el valor o las coordenadas del cuadro delimitador a un número de decimales

A.3.6. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o especifica el identificador de referencia espacial
- **transform, transformPipeline**: Transformar a una referencia espacial diferente

A.3.7. Operaciones de división

- **quadSplit**: Dividir el cuadro delimitador en cuadrantes u octantes {}

A.3.8. Operaciones de conjuntos

- **+, ***: Unión e intersección de dos cuadros delimitadores

A.3.9. Operaciones de cuadro delimitador

A.3.9.1. Operaciones topológicas

- **&&**: ¿Se superponen los cuadros delimitadores?
 - **@>**: ¿Contiene el primer cuadro delimitador el segundo?
 - **<@**: ¿Está el primer cuadro delimitador contenido en el segundo?
 - **~=**: ¿Son los cuadros delimitadores iguales en sus dimensiones comunes?
 - **-|**: ¿Son los cuadros delimitadores adyacentes?
-

A.3.9.2. Operaciones de posición

- `<<`, `<<|`, `<</`, `<<#`: ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente menores que los del segundo?
- `>>`, `>>|`, `>>/`, `>>#`: ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente mayores que los del segundo?
- `&<`, `&<|`, `&</`, `&<#`: ¿No son los valores X/Y/Z/T del primer cuadro delimitador mayores que los del segundo?
- `&>`, `&>|`, `&>/`, `&>#`: ¿No son los valores X/Y/Z/T del primer cuadro delimitador menores que los del segundo?

A.3.10. Comparaciones

- `=`, `<`, `>`, `<=`, `>=`, `cmp`: Comparaciones tradicionales

A.3.11. Agregaciones

- `extent`: Extensión del cuadro delimitador

A.4. Temporal Types

A.4.1. Entrada y salida

- `asText`: Devuelve la representación textual conocida (Well-Known Text o WKT)
- `asMFJSON`: Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- `asBinary`, `asHexWKB`: Devuelve la representación binaria conocida (Well-Known Binary o WKB) o la representación hexadecimal binaria conocida (HexWKB)
- `ttypeFromMFJSON`: Entrar a partir de la representación Moving Features JSON (MF-JSON)
- `ttypeFromBinary`, `ttypeFromHexWKB`: Entrar a partir de la representación binaria conocida (WKB) o de la representación hexadecimal binaria conocida (HexWKB)

A.4.2. Constructores

- `ttype`: Constructores para tipos temporales que tienen un valor constante
- `ttypeSeq`: Constructores para tipos temporales de subtipo secuencia
- `ttypeSeqSet`, `ttypeSeqSetGaps`: `ttypeSeqSet(ttypeContSeq[]) → ttypeSeqSet`

A.4.3. Conversión de tipos

- `::`: Convertir un valor temporal a un intervalo de marcas de tiempo

A.4.4. Accesores

- **memSize**: Devuelve el tamaño de la memoria en bytes
- **tempSubtype**: Devuelve el subtipo temporal
- **tempBasetype**: Devuelve el tipo base
- **interp**: Devuelve la interpolación
- **getValue, getTimestamp**: Devuelve el valor o la marca de tiempo de un instantes
- **getValues, getTime**: Devuelve los valores o el tiempo en el que se define el valor temporal
- **timeSpan**: Devuelve el período delimitador en el que se define el valor temporal
- **startValue, endValue, valueN**: Devuelve el valor inicial, final, o *enésimo*
- **valueAtTimestamp**: Devuelve el valor en una marca de tiempo
- **duration**: Devuelve el intervalo de tiempo
- **lowerInc, upperInc**: ¿Es el instante inicial/final inclusivo?
- **numInstants, instants**: Devuelve el número o los instantes diferentes
- **startInstant, endInstant, instantN**: Devuelve el instante inicial, final o *enésimo*
- **timestamps, numTimestamps**: Devuelve el número o las marcas de tiempo diferentes
- **startTimestamp, endTimestamp, timestampN**: Devuelve la marca de tiempo inicial, final o *enésima*
- **numSequences, sequences**: Devuelve el número o las secuencias
- **startSequence, endSequence, sequenceN**: Devuelve la secuencia inicial, final, o *enésima*
- **segments**: Devuelve los segmentos

A.4.5. Transformaciones

- **ttypeInst, ttypeSeq, ttypeSeqSet**: Transformar un tipo temporal a otro subtipo
- **setInterp**: Transformar un valor temporal a otra interpolación
- **shiftTime, scaleTime, shiftScaleTime, segmentMinDuration, segmentMaxDuration**: Devuelve un valor booleano temporal que indica si cada segmento de una secuencia (o conjunto de secuencias) temporal(es) continua tiene, respectivamente, al menos o como máximo una duración determinada.
- **segmentMinDuration, segmentMaxDuration**: Devuelve un valor booleano temporal que indica si cada segmento de una secuencia (o conjunto de secuencias) temporal(es) continua tiene, respectivamente, al menos o como máximo una duración determinada.
- **unnest**: Transformar un valor temporal no lineal en un conjunto de filas, cada una es una pareja compuesta de un valor de base y un conjunto de períodos durante el cual el valor temporal tiene el valor de base { }

A.4.6. Modificaciones

- **insert**: Insertar un valor temporal en otro
 - **update**: Actualizar un valor temporal con otro
 - **deleteTime**: Eliminar de un valor temporal un valor de tiempo
 - **appendInstant**: Anexar un instante temporal a un valor temporal
 - **appendSequence**: Anexar una secuencia temporal a un valor temporal
 - **merge**: Fusionar los valores temporales
-

A.4.7. Restricciones

- **atValue, minusValue, atValues, minusValues, atSpan, minusSpan, atSpanset, minusSpanset**: Restringir a (al complemento de) un valor o un conjunto de valores o, para un número temporal, un span o un conjunto de spans de valores
- **atTime, minusTime**: Restringir a (al complemento de) un valor de tiempo
- **beforeTimestamp, afterTimestamp**: Mantener los instantes de un valor temporal que son anteriores o posteriores a una marca de tiempo, o iguales a ella

A.4.8. Comparaciones

A.4.8.1. Comparaciones tradicionales

- **=, <>, <, >, <=, >=, cmp**: Comparaciones tradicionales

A.4.8.2. Comparaciones alguna vez y siempre

- **?=, %=, ?<>, %<>, ?<, %<, ?>, %>, ?<=, %<=, ?>=, %>=**: Comparaciones alguna vez y siempre

A.4.8.3. Comparaciones temporales

- **#=, #<>, #<, #>, #<=, #>=**: Comparaciones temporales

A.4.9. Funciones de utilidad

- **mobilitydb_version**: Versión de la extensión MobilityDB
- **mobilitydb_full_version**: Versión de la extensión MobilityDB y de sus dependencias

A.5. Temporal Alphanumeric Types

A.5.1. Conversión de tipos

- **::, valueSpan(tnumber), timeSpan(tnumber), tbox(tnumber), valueSpan, timeSpan, tbox**: Convertir un número temporal en un rango o un cuadro delimitador temporal
- **::, tint(tbool), tint**: Convertir entre un booleano temporal y un entero temporal
- **::, tnumber(tnumber)**: Convertir entre enteros temporales, enteros grandes temporales y flotantes temporales

A.5.2. Accessores

- **valueSpan**: Devuelve el intervalo de valores ignorando las brechas potenciales
 - **valueSet**: Devuelve el conjunto de valores de un número temporal
 - **minValue, maxValue, avgValue**: Devuelve el valor mínimo, máximo, o promedio
 - **minInstant, maxInstant**: Devuelve el instante con el valor mínimo o máximo
-

A.5.3. Transformaciones

- **shiftValue**, **scaleValue**, **shiftScaleValue**: Desplazar y/o escalear el rango de valores de un número temporal con uno o dos números
- **stops**: Extraer de un flotante temporal con interpolación lineal las subsecuencias donde los valores permanecen en un rango de un ancho dado durante al menos una duración dada

A.5.4. Restrictions

- **atMin**, **atMax**, **minusMin**, **minusMax**: Restringir al (complemento del) valor mínimo o máximo
- **atTbox**, **minusTbox**: Restringir a (al complemento de) un `tbox`

A.5.5. Operaciones booleanas

- **&**, **!**: Y temporal, o temporal
- **~**: No temporal
- **whenTrue**: Devuelve el tiempo cuando un booleano temporal toma el valor verdadero

A.5.6. Operaciones matemáticas

- **+**, **-**, *****, **/**: Adición, resta, multiplicación y división temporal
- **abs**: Devuelve el valor absoluto del número temporal
- **floor**, **ceil**: Redondear al entero inferior o superior
- **round**: Redondear a un número de posiciones decimales
- **degrees**, **radians**: Convertir a grados o radianes
- **deltaValue**: Devuelve la diferencia de valor entre instantes consecutivos del número temporal
- **trend**: Devuelve la tendencia de un flotante temporal con interpolación lineal, que indica si su valor es creciente, constante o decreciente, representado, respectivamente, por 1, 0 y -1.
- **derivative**: Devuelve la derivada sobre el tiempo del número flotante temporal en unidades por segundo
- **integral**: Devuelve el área bajo la curva
- **twAvg**: Devuelve el promedio ponderado en el tiempo
- **ln**, **log10**: Devuelve el logaritmo natural y el logaritmo base 10 de un número flotante temporal
- **exp**: Devuelve el exponencial (e elevado a la potencia dada) de un número flotante temporal
- **sin**, **cos**, **tan**: Devuelve el seno, coseno o tangente de un número flotante temporal (argumento en radianes)

A.5.7. Operaciones de texto

- **||**: Concatenación de texto
 - **lower**, **upper**, **initcap**: Transformar en minúsculas, mayúsculas o initcap
-

A.6. Temporal Geometry Types

A.6.1. Funciones sobre Geometrías

- **OrientedEnvelope**: Devuelve el rectángulo de área mínima que encierra una geometría
- **ConvexHull**: Devuelve la geometría convexa más pequeña que encierra una geometría
- **isSimple**: Devuelve verdadero si una geometría no tiene ningún punto en el que se cruce o se toque a sí misma
- **Buffer**: Devuelve la geometría que contiene todos los puntos situados a una distancia dada de una geometría

A.6.2. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)  
- **asMFJSON**: Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)  
- **asBinary, asEWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o la representación hexadecimal extendida binaria conocida (Extended Well-Known Binary o EWKB)  
- **tspatialFromText, tspatialFromEWKT**: Entrar a partir de la representación de texto conocido (Well-Known Text o WKT) o de la representación extendida de texto conocido (Extended Well-Known Text o EWKT)  
- **tspatialFromMFJSON**: Entrar a partir de la representación JSON de características móviles (Moving Features JSON o MF-JSON)  
- **tspatialFromBinary, tspatialFromEWKB, tspatialFromHexEWKB**: Entrar a partir de su representación binaria conocida (WKB), de la representación extendida binaria conocida (EWKB), o de la representación hexadecimal extendida binaria conocida (HexEWKB)  

A.6.3. Conversión de tipos

- **::, stbox(tspatial)**: Convertir un valor espaciotemporal a un rango temporal o a un cuadro delimitador espaciotemporal
- **:::** Convertir entre una geometría temporal y una geografía temporal
- **:::** Convertir entre un punto temporal y una trayectoria PostGIS

A.6.4. Accessores

- **trajectory, traversedArea**: Obtener la trayectoria o el área atravesada  
- **centroid**: Devuelve el centroide como un punto temporal 
- **getX, getY, getZ**: Devuelve los valores de las coordenadas X/Y/Z como un número flotante temporal  
- **isSimple**: Devuelve verdadero si el punto temporal no se auto-intersecta espacialmente 
- **length**: Devuelve la longitud atravesada por el punto temporal  
- **cumulativeLength**: Devuelve la longitud acumulada atravesada por el punto temporal  
- **speed**: Devuelve la velocidad del punto temporal en unidades por segundo  

- **twCentroid**: Devuelve el centroide ponderado en el tiempo 
- **direction**: Devuelve la dirección, es decir, el acimut entre las ubicaciones inicial y final  
- **azimuth**: Devuelve el acimut temporal  
- **angularDifference**: Devuelve la diferencia angular temporal 
- **bearing**: Devuelve el rumbo temporal  

A.6.5. Transformaciones

- **round**: Redondear los valores de las coordenadas a un número de decimales  
- **makeSimple**: Devuelve una matriz de fragmentos del punto temporal que son simples 
- **geoMeasure**: Construir una geometría/geografía con medida M a partir de un punto temporal y un número flotante temporal  
- **affine**: Devuelve la transformación afín 3D de una geometría temporal para traducirla, rotarla y/o escalarla en un solo paso 
- **rotate**: Devuelve el punto temporal rotado en sentido antihorario sobre el punto de origen
- **scale**: Devuelve un punto temporal escalado por factores dados 
- **asMVTGeom**: Transformar un punto geométrico temporal en el espacio de coordenadas de un Mapbox Vector Tile 
- **stops**: Extraer de un punto temporal con interpolación lineal las subsecuencias donde el punto permanece dentro de un área con un tamaño máximo especificado durante al menos la duración dada  

A.6.6. Restricciones

- **atGeometry, minusGeometry**: Restringir a (al complemento de) una geometría 
- **atStbox, minusStbox**: Restringir a (al complemento de) un `stbox` 
- **atElevation, minusElevation**: Restringir a (al complemento de) un rango de altitud 

A.6.7. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial  
- **transform, transformPipeline**: Transformar a una referencia espacial diferente  

A.6.8. Operaciones de cuadro delimitador

- **expandSpace**: Devuelve el cuadro delimitador espaciotemporal expandido en la dimensión espacial por un valor flotante  

A.6.9. Operaciones de distancia

- **|=|**: Devuelve la distancia más pequeña que haya existido  
- **nearestApproachInstant**: Devuelve el instante del primer punto temporal en el que los dos argumentos están a la distancia más cercana  
- **minDistance**: Devuelve la distancia espacial mínima entre dos geometrías temporales sobre la unión de sus extensiones temporales
- **eDwithinPairs, aDwithinPairs, tDwithinPairs, eIntersectsPairs, aIntersectsPairs, tIntersectsPairs, eDisjointPairs, aDisjointPairs, tDisjointPairs, eTouchesPairs, aTouchesPairs, tTouchesPairs**: Devuelve los pares de índices que cumplen la relación entre dos arreglos de geometrías temporales, generalizando las relaciones espaciales escalares a una unión espacial conjunto-conjunto
- **shortestLine**: Devuelve la línea que conecta el punto de aproximación más cercano  
- **<->**: Devuelve la distancia temporal  

A.6.10. Relaciones espaciales

A.6.10.1. Relaciones alguna vez o siempre

- **eContains, aContains**: Contiene alguna vez
- **eCovers, aCovers**: Ever or always covers
- **eDisjoint, aDisjoint**: Está disjunto alguna vez  
- **eDwithin, aDwithin**: Está alguna vez a distancia de  
- **eIntersects, aIntersects**: Intersecta alguna vez  
- **eTouches, aTouches**: Toca alguna vez

A.6.10.2. Relaciones espaciotemporales

- **tContains**: Contiene temporal
- **tDisjoint**: Disjunto temporal  
- **tDwithin**: Está a distancia de temporal 
- **tIntersects**: Intersección temporal  
- **tTouches**: Toca temporal

A.7. Operations for Temporal Types: Analytics Operations

A.7.1. Simplificación

- **minDistSimplify, minTimeDeltaSimplify**: Simplificar un flotante o un punto temporal asegurándose de que los valores consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo  
- **maxDistSimplify, douglasPeuckerSimplify**: Simplificar un flotante o un punto temporal usando el **algoritmo de Douglas-Peucker** 

A.7.2. Reducción

- **tsample**: Muestrear un valor temporal con respecto a un intervalo
- **tprecision**: Reducir la precisión temporal de un valor temporal con respecto a un intervalo calculando el promedio/centroide ponderado por el tiempo en cada intervalo de tiempo

A.7.3. Similitud

- **hausdorffDistance**, **frechetDistance**, **dynTimeWarpDistance**, **averageHausdorffDistance**, **lcssDistance**: Devuelve la **distancia de Hausdorff** discreta, la **distancia de Fréchet** discreta, la distancia de distorsión de tiempo dinámica (**Dynamic Time Warp** o DTW), la distancia de Hausdorff promedio, o la distancia de **subsecuencia común más larga** (Longest Common SubSequence o LCSS) entre dos valores temporales  
- **frechetDistancePath**, **dynTimeWarpPath**: Devuelve las parejas de correspondencia entre dos valores temporales con respecto a la distancia de Fréchet discreta o la distancia de distorsión de tiempo dinámica (Dynamic Time Warp o DTW)   

A.7.4. Filtro de Kalman extendido

- **extendedKalmanFilter**: Devolver un flotante temporal o un punto temporal suavizado, obtenido al filtrar una trayectoria ruidosa con un **filtro de Kalman extendido** 

A.7.5. Operaciones de división

- **splitNSpans**: Devuelve una matriz de N rangos de tiempo a partir de los instantes o segmentos de un valor temporal  
- **splitEachNSpans**: Devuelve una matriz de rangos de tiempo obtenida fusionando N instantes o segmentos consecutivos de un valor temporal  
- **splitNTboxes**: Devuelve una matriz de N cuadros temporales obtenida fusionando los instantes o segmentos de un número temporal
- **splitEachNTboxes**: Devuelve una matriz de cuadros temporales obtenida fusionando N instantes o segmentos consecutivos de un número temporal
- **splitNSTboxes**: Devuelve ya sea una matriz de N cuadros espaciales obtenida fusionando los segmentos de una (multi)línea o una matriz de N cuadros espaciotemporales obtenida fusionando los instantes o segmentos de un punto temporal  
- **splitEachNSTboxes**: Devuelve ya sea una matriz de cuadros espaciales obtenida fusionando N segmentos consecutivos de una (multi)línea o una matriz de cuadros espaciotemporales obtenida fusionando N instantes o segmentos consecutivos de un punto temporal  

A.7.6. Mosaicos multidimensionales

A.7.6.1. Operaciones de intervalos

- **bins**: Devuelve una matriz de intervalos que cubre el rango de valores o de tiempo con intervalos de la misma amplitud o duración
- **getBin**: Devuelve el rango que contiene un número o una marca de tiempo

A.7.6.2. Operaciones de mosaicos

- **valueTiles, timeTiles, valueTimeTiles**: Devuelve el conjunto de mosaicos que cubre un cuadro delimitador temporal con mosaicos del mismo tamaño y/o duración {}
- **spaceTiles, timeTiles, spaceTimeTiles**: Devuelve el conjunto de mosaicos que cubre un cuadro delimitador espaciotemporal con mosaicos del mismo tamaño y/o duración  {}
- **getValueTile, getTboxTimeTile, getValueTimeTile**: Devuelve el mosaico temporal que cubre un valor y/o una marca de tiempo
- **getSpaceTile, getStboxTimeTile, getSpaceTimeTile**: Devuelve el mosaico espaciotemporal que cubre un punto y/o una marca de tiempo 

A.7.6.3. Operaciones de cuadro delimitador

- **valueBins, timeBins**: Devuelve una matriz de rangos de valores o de tiempo obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores o de tiempo
- **valueBoxes, timeBoxes, valueTimeBoxes**: Devuelve una matriz de cuadros temporales obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores y/o tiempo
- **spaceBoxes, timeBoxes, spaceTimeBoxes**: Devuelve una matriz de cuadros espaciotemporales obtenidos a partir de los instantes o segmentos de un punto temporal con respecto a un mosaico espacial y/o temporal 

A.7.6.4. Operaciones de división

- **valueSplit, timeSplit, valueTimeSplit**: Fragmentar un número temporal con respecto a intervalos de valores y/o de tiempo {}
- **spaceSplit, spaceTimeSplit, timeSplit**: Fragmentar un punto temporal con respecto a una malla espacial o espacio-temporal {}

A.7.7. Agregación

- **tCount**: Conteo temporal
- **extent**: Extensión del cuadro delimitador
- **tAnd, tOr**: Y, o temporal
- **tMin, tMax, tSum, tAvg**: Mínimo, máximo, suma y promedio temporal
- **wCount**: Conteo de ventana
- **wMin, wMax, wSum, wAvg**: Mínimo, máximo, suma y promedio de ventana
- **tCentroid**: Centroide temporal

A.8. Temporal Poses

A.8.1. Poses estáticas

A.8.1.1. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)

- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)
- **poseFromText, poseFromEWKT**: Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **poseFromBinary, poseFromEWKB, poseFromHexEWKB**: Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

A.8.1.2. Constructores

- **pose**: Constructor para poses

A.8.1.3. Conversión de tipos

- **::, stbox**: Convertir una pose y, opcionalmente, una marca de tiempo o un período, en una caja espaciotemporal
- **:::** Convertir una pose en un punto geométrico

A.8.1.4. Accesos

- **point**: Devuelve el punto
- **quaternion**: Devuelve el cuaternión de orientación de una pose

A.8.1.5. Transformaciones

- **round**: Redondear el punto y la orientación de la pose al número de posiciones decimales

A.8.1.6. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial
- **transform, transformPipeline**: Transformar a un identificador de referencia espacial

A.8.1.7. Operaciones de distancia

- **<->**: Devuelve la distancia

A.8.1.8. Comparaciones

- **=, <>, <, >, <=, >=**: Comparaciones tradicionales
 - **~=**: ¿Son las poses aproximadamente iguales con respecto a un valor épsilon?
-

A.8.2. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **asMFJSON**: Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)
- **tposeFromText, tposeFromEWKT**: Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **tposeFromMFJSON**: Entrada desde la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- **tposeFromBinary, tposeFromEWKB, tposeFromHexEWKB**: Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

A.8.3. Constructores

- **tpose**: Constructor para poses temporales con valor constante
- **tposeSeq**: Constructor para poses temporales de subtipo secuencia
- **tposeSeqSet, tposeSeqSetGaps**: Constructor para poses temporales de subtipo conjunto de secuencias
- **tpose**: Construir una pose temporal 2D a partir de un punto geométrico temporal y un flotante temporal

A.8.4. Conversión de tipos

- **:::** Convertir una pose temporal en un punto geométrico temporal
- **:::** Convertir una pose temporal geodésica en un punto geográfico temporal

A.8.5. Accesores

- **getValues**: Devuelve los valores
- **points**: Devuelve los puntos
- **trajectory**: Devuelve la trayectoria
- **valueAtTimestamp**: Devuelve el valor en una marca de tiempo
- **speed**: Devuelve la velocidad de una pose temporal como un flotante temporal: la magnitud de la velocidad de la componente de posición (distancia recorrida por unidad de tiempo). La orientación no es considerada; para la velocidad angular véase `angularSpeed`.
- **angularSpeed**: Devuelve la velocidad angular de una pose temporal como un flotante temporal con interpolación por pasos (radianes por unidad de tiempo). Para las poses 2D es el delta theta del arco más corto por segmento dividido por la duración del segmento; para las poses 3D es el ángulo de arco de SLERP $2 \cdot \arccos(|q1 \cdot q2|)$ dividido por la duración del segmento. SLERP tiene por construcción velocidad angular constante a lo largo de un segmento, por lo que el resultado es constante a trozos.

A.8.6. Transformaciones

- **tposeInst**, **tposeSeq**, **tposeSeqSet**: Transformar a otro subtipo
- **setInterp**: Transformar a otra interpolación
- **round**: Redondear los puntos y las orientaciones de la pose temporal al número de posiciones decimales

A.8.7. Modificaciones

- **appendInstant**, **appendSequence**: Agregar un instante temporal o una secuencia temporal
- **merge**, **mergeAgg**: Fusionar las poses temporales

A.8.8. Restricciones

- **atValue**, **minusValue**, **atValues**, **minusValues**: Restringir al (complemento de) un valor o un conjunto de valores
- **atGeometry**, **minusGeometry**: Restringir al (complemento de) una geometría
- **atElevation**, **minusElevation**: Restringir al (complemento de) un span de elevación

A.8.9. Sistema de referencia espacial

- **SRID**, **setSRID**: Devuelve o establece el identificador de referencia espacial
- **transform**, **transformPipeline**: Transformar a un identificador de referencia espacial

A.8.10. Operaciones de cuadro delimitador

- **&&**, **<@**, **@>**, **~=**, **-|**: Operadores topológicos
- **::**: Operadores de posición

A.8.11. Operaciones de distancia

- **|=**: Devuelve la distancia más pequeña alguna vez
- **nearestApproachInstant**: Devuelve el instante de la primera pose temporal en el que los dos argumentos están a la distancia más cercana
- **shortestLine**: Devuelve la línea que conecta el punto de aproximación más cercano
- **<->**: Devuelve la distancia temporal

A.8.12. Relaciones espaciales

- **eContains**, **eDisjoint**, **eIntersects**: Relaciones alguna vez y siempre
- **tContains**, **tDisjoint**, **tDwithin**: Relaciones espaciotemporales

A.8.13. Comparaciones

- **::**: Comparaciones tradicionales
 - **?=**, **%=**: Comparaciones alguna vez y siempre
 - **::**: Comparaciones temporales
-

A.8.14. Agregaciones

- **tCount**: Conteo temporal
- **wCount**: Conteo de ventana

A.8.15. Soporte de OGC GeoPose v1.0

A.8.15.1. Entrada y salida JSON

- **asGeoPose**, **poseFromGeoPose**: Convertir hacia o desde la codificación JSON de OGC GeoPose v1.0 (clase de conformidad Basic-Quaternion, Basic-YPR o Advanced)

A.8.15.2. Renormalización de cuaterniones

- **poseNormalize**: Devuelve una pose cuyo cuaternión de orientación se ha llevado a una norma unitaria
- **poseInverse**: Devuelve la inversa de una pose

A.8.15.3. Accesores de ángulos de Euler

- **yaw**, **pitch**, **roll**: Devuelve el ángulo yaw, pitch o roll de una pose, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose
- **ypr**: Devuelve la orientación de una pose como una terna yaw / pitch / roll, en radianes
- **yaw**, **pitch**, **roll**: Eleva los accesores de ángulos de Euler a través de una pose temporal, devolviendo un flotante temporal (radianes) por cada instante bajo la convención Tait-Bryan intrínseca ZYX

A.8.15.4. Registro de metadatos de marcos

- **geoPoseFrameSRID**: Devuelve el SRID de un marco, o NULL si el marco es paramétrico (LTP, BODY)
- **geoPoseFrameName**: Devuelve el nombre legible del marco
- **geoPoseFrameIsGeographic**: Devuelve true para los marcos lat/lon/h, false para los cartesianos o proyectados

A.8.15.5. Transformación rígida cuerpo↔mundo

- **applyPose**: Aplica la transformación de cuerpo rígido codificada por una pose (la correspondencia *cuerpo al mundo* de OGC GeoPose) a una geometría en el marco del cuerpo, produciendo la geometría correspondiente en el marco del mundo

A.8.15.6. E/S JSON temporal

- **asGeoPose**, **tposeFromGeoPose**: Convierte una pose temporal hacia o desde una envoltura JSON TemporalGeoPose. Cada carga útil por instante es un documento GeoPose de clase Basic estrictamente válido según OGC más un miembro `validTime`; la envoltura registra la clase de conformidad, la interpolación y (para secuencias y conjuntos de secuencias) la inclusión de los límites del período. Un consumidor GeoPose ajeno a MEOS puede iterar sobre `instances[]` (o cada `sequences[].instances[]`) y consumir cada elemento como un documento GeoPose estático.
 - **asGeoPoseStream**: Escribe una pose temporal como un Stream de OGC GeoPose
-

A.9. Pose Chains

A.9.1. Cadenas de poses estáticas

A.9.1.1. Entrada y salida

- **asText, asEWKT**: Devuelve la representación en texto conocido (WKT) o en texto conocido extendido (EWKT)
- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación en binario conocido (WKB), en binario conocido extendido (EWKB), en binario conocido hexadecimal (HexWKB) o en binario conocido extendido hexadecimal (HexEWKB)
- **posechainFromText, posechainFromEWKT**: Entrada desde la representación en texto conocido (WKT) o en texto conocido extendido (EWKT)
- **posechainFromBinary, posechainFromEWKB, posechainFromHexEWKB**: Entrada desde la representación en binario conocido (WKB), en binario conocido extendido (EWKB) o en binario conocido extendido hexadecimal (HexEWKB)

A.9.1.2. Constructores

- **posechain**: Construye una cadena de poses a partir de un arreglo de poses ordenadas desde el marco más exterior hacia dentro
- **appendPose**: Devuelve una cadena de poses con una pose añadida como su eslabón más interior

A.9.1.3. Conversión de tipos

- **posechain**: Convierte una pose en una cadena de poses de un solo eslabón
- **pose**: Devuelve la pose de un marco que define la cadena, leída en el marco exterior de la cadena
- **point**: Convierte una cadena de poses en el punto geométrico de su marco más interior
- **stbox**: Convierte una cadena de poses en una caja espaciotemporal

A.9.1.4. Accesores

- **numPoses**: Devuelve el número de eslabones de una cadena de poses
- **startPose, endPose, poseN, poses**: Devuelve los eslabones de una cadena de poses

A.9.1.5. Transformaciones

- **round**: Redondea los valores de los eslabones de una cadena de poses a un número de decimales

A.9.1.6. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial del marco exterior de una cadena de poses
- **transform, transformPipeline**: Transforma a un identificador de referencia espacial

A.9.1.7. Comparaciones

- **=, <>, <, >, <=, >=**: Operadores de comparación tradicionales
 - **~=**: Devuelve verdadero si dos cadenas de poses son iguales hasta la tolerancia de la comparación de valores de punto flotante
-

A.9.2. Cadenas de poses temporales

A.9.2.1. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (WKT) o de texto conocido extendido (EWKT)
- **tposechainFromText, tposechainFromBinary, tposechainFromHexEWKB**: Devuelve una cadena de poses temporal a partir de su representación textual o binaria
- **asMFJSON**: Devuelve la representación JSON de Moving Features

A.9.2.2. Constructores

- **tposechain**: Construye una cadena de poses temporal a partir de una cadena de poses y un valor de tiempo

A.9.2.3. Conversiones

- **tpose**: Convierte una cadena de poses temporal en una pose temporal

A.9.2.4. Funciones de acceso

- **numPoses**: Devuelve el número de eslabones que tiene todo valor de una cadena de poses temporal
- **getValue, getValues, timeSpan**: Devuelve el valor de un instante temporal, el conjunto de valores o el lapso de tiempo
- **stbox**: Devuelve la caja espaciotemporal de una cadena de poses temporal

A.9.2.5. Funciones de transformación

- **round, shiftTime**: Redondea las coordenadas a un número de decimales, o desplaza el tiempo

A.9.2.6. Sistema de referencia espacial

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial del marco exterior de una cadena de poses temporal
- **transform, transformPipeline**: Transforma a un identificador de referencia espacial

A.9.2.7. Cadena compuesta OGC GeoPose

- **asGeoPose, tposechainFromGeoPose**: Escribe o lee un documento de cadena compuesta OGC GeoPose

A.9.2.8. Grafo compuesto OGC GeoPose

- **asGeoPose**: Escribe un documento de grafo compuesto OGC GeoPose

A.9.2.9. Funciones de restricción

- **atTime, atValue**: Restringe una cadena de poses temporal a un tiempo o a un valor

A.9.2.10. Funciones y operadores de comparación

- **eEq**: Compara dos cadenas de poses temporales, o una cadena de poses temporal y una cadena de poses

A.9.2.11. Operadores topológicos y de posición

- **&&, <<#**: Compara la caja delimitadora de una cadena de poses temporal con otra caja delimitadora

A.10. Temporal Network Points

A.10.1. Tipos de red estáticos

A.10.1.1. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)
- **npointFromText, npointFromEWKT**: Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **npointFromBinary, npointFromEWKB, npointFromHexEWKB**: Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

A.10.1.2. Constructores

- **npoint, nsegment**: Constructores de puntos o segmentos de red

A.10.1.3. Conversión de tipos

- **::**: Convertir entre un punto o un segmento de red y una geometría
- **stbox**: Construir una caja espaciotemporal a partir de un punto de red y, opcionalmente, una marca de tiempo o un período

A.10.1.4. Accesos

- **route**: Devuelve el identificador de ruta
- **getPosition**: Devuelve la posición
- **startPosition, endPosition**: Devuelve la posición inicial/final

A.10.1.5. Transformaciones

- **round**: Redondear la(s) posición(es) del punto de red o el segmento de red en el número de posiciones decimales

A.10.1.6. Operaciones espaciales

- **SRID**: Devuelve el identificador de referencia espacial
 - **::**: Igualdad espacial para puntos de red
-

A.10.1.7. Comparaciones

- `=, <>, <, >, <=, >=`: Comparaciones tradicionales

A.10.2. Constructores

- `tnpoint, tnpointSeq, tnpointSeqSet`: Constructor para puntos de red temporales con valor constante
- `tnpointSeq`: Constructor para puntos de red temporal de subtipo secuencia
- `tnpointSeqSet, tnpointSeqSetGaps`: Constructor para puntos de red temporal de subtipo conjunto de secuencias

A.10.3. Conversión de tipos

- `::`: Convertir entre un punto de red temporal y un punto de geometría temporal

A.10.4. Entrada y Salida MF-JSON

- `asMFJSON, tnpointFromMFJSON`: Genera un punto de red temporal como MF-JSON y lo lee de vuelta; el ciclo de ida y vuelta es sin pérdida. La salida toma los argumentos de opciones, banderas y dígitos decimales compartidos con los demás tipos temporales.

A.10.5. Accessores

- `getValues`: Devuelve los valores
- `routes`: Devuelve los identificadores de ruta
- `valueAtTimestamp`: Devuelve el valor en una marca de tiempo
- `length`: Devuelve la longitud atravesada por el punto de red temporal
- `cumulativeLength`: Devuelve la longitud acumulada atravesada por el punto de red temporal
- `speed`: Devuelve la velocidad del punto de red temporal en unidades por segundo

A.10.6. Transformaciones

- `tnpointInst, tnpointSeq, tnpointSeqSet`: Transformar un punto de red temporal en otro subtipo
- `setInterp`: Transformar un punto de red temporal en otra interpolación
- `round`: Redondear la fracción del punto de red temporal en el número de lugares decimales

A.10.7. Modificaciones

- `appendInstant, appendSequence`: Agregar un instante temporal o una secuencia temporal
- `merge, mergeAgg`: Fusionar los puntos de red temporales

A.10.8. Restricciones

- `atValue, minusValue, atValues, minusValues`: Restringir al (complemento de) un valor o un conjunto de valores
 - `atGeometry, minusGeometry`: Restringir al (complemento de) una geometría
-

A.10.9. Operaciones de distancia

- **|=|**: Devuelve la distancia más cercana
- **nearestApproachInstant**: Devuelve el instante del primer punto de red temporal en el que los dos argumentos están a la distancia más cercana
- **shortestLine**: Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
- **<->**: Devuelve la distancia temporal

A.10.10. Operaciones espaciales

- **twCentroid**: Devuelve el centroide ponderado en el tiempo

A.10.11. Operaciones de cuadro delimitador

- **?@, @?, @@, @=**: Operadores de rutas, que comparan los identificadores de ruta que visita un punto de red temporal: contenido en, contiene, se superpone e igual
- **&&, <@, @>, ~=, -|**: Operadores topológicos
- **::**: Operadores de posición

A.10.12. Relaciones espaciales

- **eContains, eDisjoint, eIntersects**: Relaciones alguna vez y siempre
- **tContains, tDisjoint, tDwithin**: Relaciones espaciotemporales

A.10.13. Comparaciones

- **::**: Comparaciones tradicionales
- **?=, %=**: Comparaciones alguna vez y siempre
- **::**: Comparaciones temporales

A.10.14. Agregaciones

- **tCount**: Conteo temporal
- **wCount**: Conteo de ventana
- **tCentroid**: Centroide temporal

A.10.15. Agregación

- **extent**: Devuelve el cuadro delimitador espaciotemporal que cubre cada posición materializada en un conjunto de puntos de red. El agregado es polimórfico con los agregados **extent** de los demás tipos temporales espaciales, por lo que una sola consulta puede calcular un único cuadro a través de un conjunto heterogéneo de columnas **tgeompoint**, **tgeometry** y **tnpoint**. La extensión espacial de cada instante se resuelve a través del registro de ways, lo que domina el coste sobre una entrada grande.

A.11. Temporal Circular Buffers

A.11.1. Búferes circulares estáticos

A.11.1.1. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)
- **cbufferFromText, cbufferFromEWKT**: Entrada desde la representación de texto conocido (Well-Known Text o WKT) o desde la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **cbufferFromBinary, cbufferFromEWKB, cbufferFromHexEWKB**: Entrada desde la representación binaria conocida (Well-Known Binary o WKB), desde la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), o desde la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)

A.11.1.2. Constructores

- **cbuffer**: Constructor para búferes circulares

A.11.1.3. Conversión de tipos

- **:::** Convertir entre un búfer circular y una geometría
- **stbox**: Convertir un búfer circular y, opcionalmente, una marca de tiempo o un período, a un cuadro espaciotemporal

A.11.1.4. Accesos

- **point**: Devuelve el punto
- **radius**: Devuelve el radio

A.11.1.5. Transformaciones

- **round**: Redondear el punto y el radio del búfer circular en el número de lugares decimales

A.11.1.6. Operaciones espaciales

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial
- **transform, transformPipeline**: Transformar a una referencia espacial diferente

A.11.1.7. Comparaciones

- **=, <>, <, >, <=, >=**: Comparaciones tradicionales
-

A.11.2. Entrada y salida

- **asText, asEWKT**: Devuelve la representación de texto conocido (Well-Known Text o WKT) o la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **asMFJSON**: Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- **asBinary, asEWKB, asHexWKB, asHexEWKB**: Devuelve la representación binaria conocida (Well-Known Binary o WKB), la representación extendida binaria conocida (Extended Well-Known Binary o EWKB), la representación hexadecimal binaria conocida (Hexadecimal Well-Known Binary o HexWKB), o la representación hexadecimal extendida binaria conocida (Hexadecimal Extended Well-Known Binary o HexEWKB)
- **tcbufferFromText, tcbufferFromEWKT**: Entrar a partir de la representación de texto conocido (Well-Known Text o WKT) o de la representación extendida de texto conocido (Extended Well-Known Text o EWKT)
- **tcbufferFromMFJSON**: Entrar a partir de la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- **tcbufferFromBinary, tcbufferFromEWKB, tcbufferFromHexEWKB**: Entrar a partir de la representación binaria conocida (WKB), de la representación extendida binaria conocida (EWKB), o de la representación hexadecimal extendida binaria conocida (HexEWKB)

A.11.3. Constructores

- **tcbuffer**: Constructor para búferes circulares temporales con valor constante
- **tcbufferSeq**: Constructor para búferes circulares temporales de subtipo secuencia
- **tcbufferSeqSet, tcbufferSeqSetGaps**: Constructor para búferes circulares temporales de subtipo conjunto de secuencias
- **tcbuffer**: Construir un búfer circular temporal a partir de un punto de geometría temporal y un flotante temporal

A.11.4. Conversión de tipos

- **:::** Convertir un búfer circular temporal a un punto de geometría temporal
- **:::** Convertir un búfer circular temporal a un flotante temporal

A.11.5. Accesores

- **getValues**: Devuelve los valores
- **points, radius**: Devuelve los puntos o los radios
- **valueAtTimestamp**: Devuelve el valor en una marca de tiempo

A.11.6. Transformaciones

- **tcbufferInst, tcbufferSeq, tcbufferSeqSet**: Transformar un búfer circular temporal en otro subtipo
- **setInterp**: Transformar un búfer circular temporal en otra interpolación
- **round**: Redondear los puntos y los radios del búfer circular temporal en el número de lugares decimales

A.11.7. Modificaciones

- **appendInstant, appendSequence**: Agregar un instante temporal o una secuencia temporal
 - **merge, mergeAgg**: Fusionar los búferes circulares temporales
-

A.11.8. Operaciones espaciales

- **SRID, setSRID**: Devuelve o establece el identificador de referencia espacial
- **transform, transformPipeline**: Transformar a una referencia espacial diferente
- **traversedArea**: Devuelve el área atravesada por el búfer circular temporal
- **centroid**: Devuelve el punto geométrico temporal dado por los centros de un búfer circular temporal
- **convexHull**: Devuelve la envolvente convexa del área atravesada por el búfer circular temporal

A.11.9. Operaciones de distancia

- **|=**: Devuelve la distancia más cercana
- **nearestApproachInstant**: TODO Devuelve el instante del primer búfer circular temporal en el que los dos argumentos están a la distancia más cercana
- **shortestLine**: TODO Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
- **<->**: TODO Devuelve la distancia temporal
- **minDistance**: Devuelve la distancia espacial mínima, ignorando el tiempo

A.11.10. Restricciones

- **atValue, minusValue, atValues, minusValues**: Restringir al (complemento de) un valor o un conjunto de valores
- **atGeometry, minusGeometry**: TODO Restringir al (complemento de) una geometría
- **atStbox, minusStbox**: Restringir un búfer circular temporal a (el complemento de) una caja espaciotemporal

A.11.11. Comparaciones

- **=, <>, <, >, <=, >=**: Comparaciones tradicionales
- **?=, %=**: Comparaciones alguna vez y siempre
- **::**: Comparaciones temporales

A.11.12. Operaciones de cuadro delimitador

- **&&, <@, @>, ~=, -|**: Operadores topológicos
- **::**: Operadores de posición

A.11.13. Relaciones espaciales

- **eContains, aContains, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches, eCovers, aCovers**: TODO Relaciones alguna vez y siempre
 - **tContains, tDisjoint, tDwithin, tIntersects, tTouches**: TODO Relaciones espaciotemporales
-

A.11.14. Agregaciones

- **tCount**: Conteo temporal
- **wCount**: Conteo de ventana
- **tCentroid**: Agregar los centros de un conjunto de búferes circulares temporales en su centroide temporal, convirtiendo a `tgeompoint`

A.11.15. Simplificación

- **minDistSimplify**, **minTimeDeltaSimplify**: Devuelve un búfer circular temporal simplificado asegurándose de que los centros consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo
- **maxDistSimplify**, **douglasPeuckerSimplify**: Devuelve un búfer circular temporal simplificado usando el [algoritmo de Douglas-Peucker](#)

A.12. Temporal Rigid Geometries

A.12.1. Entrada y salida

- **asText**: Devuelve la representación de texto conocido (Well-Known Text o WKT)
- **asEWKT**: Devuelve la representación extendida de texto conocido (Extended Well-Known Text o EWKT), prefijada con el SRID
- **asMFJSON**: Devuelve la representación JSON de características móviles (Moving Features JSON o MF-JSON)
- **asEWKB**, **asHexWKB**, **asHexEWKB**: Devuelve la representación extendida binaria conocida (Extended Well-Known Binary o EWKB)
- **trgeometryFromText**, **trgeometryFromMFJSON**, **trgeometryFromEWKB**, **trgeometryFromEWKT**, **trgeometryFromHexEWKB**: Cada formato de salida tiene un analizador complementario:

A.12.2. Constructores

- **trgeometry**: Construir una geometría rígida temporal a partir de una geometría de referencia y una pose temporal
- **appendInstant**: Agregar un único instante a una `trgeometry` existente, ampliando la trayectoria de pose subyacente
- **appendSequence**: Agregar una secuencia completa a una `trgeometry` existente

A.12.3. Conversión de tipos

- **tpose**: Devuelve la pose temporal subyacente
 - **tgeompoint**: Devuelve la trayectoria del centroide (antena) como un punto temporal
-

A.12.4. Accesores

- **startValue, endValue, valueAtTimestamp**: Devuelve la geometría materializada al inicio, al final o en una marca de tiempo elegida
- **numInstants, numSequences, numTimestamps**: Devuelve el número de instantes, secuencias o marcas de tiempo distintos
- **instants, sequences, segments**: Devuelve el arreglo de instantes, secuencias o segmentos entre instantes constituyentes
- **points**: Devuelve el conjunto de puntos del centroide (antena) distintos vistos a lo largo de la trgeometry
- **yaw, pitch, roll**: Devuelve el ángulo yaw, pitch o roll como un flotante temporal, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose
- **geom**: Devuelve la geometría de referencia estática

A.12.5. Área recorrida

- **traversedArea**: Devuelve el área atravesada por la geometría rígida temporal

A.12.6. Funciones espaciales

- **centroid**: Devuelve el centroide del cuerpo en movimiento como un punto temporal
- **convexHull**: Devuelve la envolvente convexa del área recorrida por el cuerpo en movimiento
- **bodyPointTrajectory**: Devuelve la trayectoria en el marco del mundo de un punto del marco del cuerpo transportado por la geometría rígida en movimiento. El punto se expresa en el marco local de la forma de referencia y la pose variable en el tiempo se le aplica en cada instante.

A.12.7. Métricas de movimiento

- **length, cumulativeLength**: Devuelve la longitud total recorrida por el centroide, o su longitud acumulada a lo largo del tiempo
- **speed**: Devuelve la velocidad del centroide como un flotante temporal
- **angularSpeed**: Devuelve la velocidad angular como un flotante temporal
- **twCentroid**: Devuelve el centroide ponderado en el tiempo de la trayectoria del centroide como una geometría

A.12.8. Modificaciones

- **round**: Redondear todos los componentes numéricos a un número dado de posiciones decimales
- **setInterp**: Convertir entre interpolaciones lineal y escalonada
- **shiftTime, scaleTime, shiftScaleTime**: Desplazar, escalar, o desplazar y escalar el dominio temporal de una trgeometry

A.12.9. Restricciones espaciales

- **atGeometry, minusGeometry**: Restringir una trgeometry a (el complemento de) una geometría
 - **atStbox, minusStbox**: Restringir una trgeometry a (el complemento de) una caja espaciotemporal
 - **atElevation, minusElevation**: Restringir una trgeometry al (complemento de) un span de elevación
-

A.12.10. Operaciones de distancia

- **<->, tDistance**: Devuelve la distancia temporal entre dos geometrías rígidas temporales
- **!=, nearestApproachDistance**: Devuelve la distancia más pequeña entre dos geometrías rígidas temporales sobre su dominio temporal compartido
- **nearestApproachInstant**: Devuelve el instante en el que los dos argumentos están a su menor distancia mutua
- **shortestLine**: Devuelve la línea que conecta los dos argumentos en su aproximación más cercana

A.12.11. Similitud

- **hausdorffDistance, frechetDistance, dynTimeWarpDistance**: Devuelve la distancia de Hausdorff discreta, la distancia de Fréchet discreta, o la distancia de deformación dinámica del tiempo (Dynamic Time Warp) entre dos geometrías rígidas temporales
- **frechetDistancePath, dynTimeWarpPath**: Devuelve los pares de correspondencia entre dos geometrías rígidas temporales con respecto a la distancia de Fréchet discreta o a la distancia de deformación dinámica del tiempo (Dynamic Time Warp), como un conjunto de pares de índices de instante (i, j)

A.12.12. Comparaciones

- **=, <>**: Probar la (des)igualdad exacta de dos geometrías rígidas temporales
- **?=, ?<>**: Probar si la geometría materializada es alguna vez (no) igual al operando en algún momento
- **%=, %<>**: Probar si la geometría materializada es siempre (no) igual al operando

A.12.13. Operadores de caja delimitadora

- **&&, @>, <@, ~=, -|**: Operadores topológicos estándar sobre cajas delimitadoras
- **<<, >>, &<, &>, <</, />>, <<#, #>>**: Operadores de posición espacial y temporal

A.12.14. Mosaicos

- **spaceBoxes, timeBoxes, spaceTimeBoxes**: Devuelve las cajas espaciotemporales de una geometría rígida temporal dividida con respecto a una cuadrícula espacial, temporal o espaciotemporal

A.12.15. Cajas delimitadoras

- **stboxes, spans**: Devuelve el arreglo de cajas espaciotemporales o lapsos de tiempo de los segmentos de una geometría rígida temporal
- **splitNSTboxes, splitEachNSTboxes, splitNSpans, splitEachNSpans**: Devuelve las cajas espaciotemporales o los lapsos de tiempo de una geometría rígida temporal dividida en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.12.16. Agregaciones

- **tCount**: Número de geometrías rígidas temporales solapadas en cada instante, equivalente a `tCount` sobre la `tpose` subyacente
- **extent**: Caja delimitadora agregada de una columna de geometrías rígidas temporales
- **merge, mergeAgg**: Combinar dos `trgeometries` con la misma geometría de referencia en un único conjunto de secuencias

A.13. Temporal JSON

A.13.1. Operaciones sobre conjuntos JSONB

- `->`, `->>`, `jsonbsetObjectField`, `jsonbsetObjectFieldText`: Extraer un campo de objeto JSON especificado por una clave
- `->`, `->>`, `jsonbsetExtractPath`, `jsonbsetExtractPathText`: Extraer un campo de objeto JSON especificado por una ruta
- `jsonbsetArrayElement`, `jsonbsetArrayElementText`: Extraer un elemento de un arreglo JSON
- `intset`, `floatset`, `textset`: Extraer un valor alfanumérico de un conjunto JSONB dado por una clave
- `||`: Concatenación JSONB temporal
- `-`: Eliminación JSONB temporal
- `?`: Existencia en un conjunto JSONB
- `jsonbsetSet`, `jsonbsetSetLax`: Asignación JSONB temporal
- `jsonbsetInsert`: Inserción JSONB temporal
- `jsonbsetStripNulls`: Devolver un conjunto JSONB sin nulos
- `@?`, `jsonbsetPathExists`, `jsonbsetPathExistsTz`: ¿Devuelve la ruta JSON algún elemento para el conjunto JSONB especificado?
- `@@`, `jsonbsetPathMatch`, `jsonbsetPathMatchTz`: Devolver el resultado de una comprobación de predicado de ruta JSON para un conjunto JSONB
- `jsonbsetPathQueryArray`, `jsonbsetPathQueryArrayTz`: Devolver todos los elementos devueltos por una ruta JSON de un conjunto JSONB, como un arreglo JSON
- `jsonbsetPathQueryFirst`, `jsonbsetPathQueryFirstTz`: Devolver el primer elemento devuelto por una ruta JSON de un conjunto JSONB

A.13.2. Entrada y salida

- `asText`: Devolver la representación en texto bien conocido (WKT)
- `asBinary`, `asHexWKB`: Devolver la representación en binario bien conocido (WKB) o en binario bien conocido extendido hexadecimal (HexWKB)
- `asMFJSON`: Devolver la representación JSON de objetos en movimiento (MF-JSON)
- `tjsonbFromText`: Entrada a partir de la representación en texto bien conocido (WKT)
- `tjsonbFromBinary`, `tjsonbFromHexWKB`: Entrada a partir de la representación en binario bien conocido (WKB) o en binario bien conocido hexadecimal (HexWKB)
- `tjsonbFromMFJSON`: Entrada a partir de la representación JSON de objetos en movimiento (MF-JSON)

A.13.3. Constructores

- `tjsonb`: Constructor de valores JSONB temporales que tienen un valor constante
 - `tjsonbSeq`: Constructor de valores JSONB temporales del subtipo secuencia
 - `tjsonbSeqSet`, `tjsonbSeqSetGaps`: Constructor de valores JSONB temporales del subtipo conjunto de secuencias
-

A.13.4. Conversiones

- `::`: Convertir un JSONB temporal a un `tstzspan`
- `::`: Convertir entre un JSONB temporal y un texto
- `::`: Convertir entre un JSONB temporal y un texto temporal

A.13.5. Accesores

- `getValues`: Devolver los valores
- `valueAtTimestamp`: Devolver el valor en una marca de tiempo

A.13.6. Transformaciones

- `tjsonbInst`, `tjsonbSeq`, `tjsonbSeqSet`: Transformar un JSONB temporal a otro subtipo
- `setInterp`: Transformar un valor JSONB temporal a otra interpolación

A.13.7. Operaciones JSON temporales

- `->`, `->>`, `tjsonbObjectField`, `tjsonbObjectField`, `tjsonbObjectFieldText`: Extraer un campo de objeto JSON temporal especificado por una clave
- `->`, `->>`, `tjsonbExtractPath`, `tjsonbExtractPath`, `tjsonbExtractPathText`: Extraer un campo de objeto JSON temporal especificado por una ruta
- `tjsonbArrayElement`, `tjsonbArrayElement`, `tjsonbArrayElementText`: Extraer un elemento de un arreglo JSON temporal
- `tbool`, `tint`, `tfloat`, `ttext`: Extraer un valor alfanumérico temporal de un valor JSONB temporal dado por una clave
- `||`: Concatenación JSONB temporal
- `-`: Eliminación JSONB temporal
- `?`: Existencia JSONB temporal
- `@>`, `<@`: Contiene/contenido JSONB temporal
- `tjsonbSet`, `tjsonbSetLax`: Asignación JSONB temporal
- `tjsonbInsert`: Inserción JSONB temporal
- `tjsonbStripNulls`, `tjsonbStripNulls`: Devolver un valor JSON temporal sin nulos
- `@?`, `tjsonbPathExists`, `tjsonbPathExistsTz`: ¿Devuelve la ruta JSON algún elemento para el valor JSONB temporal especificado?
- `@@`, `tjsonbPathMatch`, `tjsonbPathMatchTz`: Devolver el resultado de una comprobación de predicado de ruta JSON para un valor JSONB temporal
- `tjsonbPathQueryArray`, `tjsonbPathQueryArrayTz`: Devolver todos los elementos devueltos por una ruta JSON de un valor JSONB temporal, como un arreglo JSON
- `tjsonbPathQueryFirst`, `tjsonbPathQueryFirstTz`: Devolver el primer elemento devuelto por una ruta JSON de un valor JSONB temporal

A.13.8. Restricciones

- `atValue`, `minusValue`, `atValues`, `minusValues`: Restringir a (el complemento de) un valor o un conjunto de valores

A.13.9. Comparaciones

- `=, <>, <, >, <=, >=`: Comparaciones tradicionales
- `?=, %=`: Comparaciones siempre y alguna vez
- `:::` Comparaciones temporales

A.13.10. Operaciones de caja delimitadora

- `&&, <@, @>, ~=, -|`: Operadores topológicos
- `:::` Operadores de posición

A.13.11. Agregaciones

- `tCount`: Conteo temporal
- `wCount`: Conteo en ventana

A.14. Temporal H3 Cell Indices

A.14.1. Conversiones

- `:::` Convertir entre un `th3index` y un `tbigint` (explícita, sin coste)
- `:::` Convertir una celda H3 temporal en un punto temporal geodésico o planar de los centroides de las celdas. Las dos sobrecargas dan al usuario una elección entre la representación geodésica (`tgeogpoint`) y la planar SRID 4326 (`tgeompoint`).

A.14.2. Cobertura de Geometrías Estáticas

- `geoToH3Cell`: Devuelve la celda H3 que contiene una geometría de punto a la resolución dada. La geometría debe ser un `POINT`.
- `geoToH3IndexSet`: Devuelve el conjunto de celdas H3 que cubren una geometría a la resolución dada. Se admiten todos los tipos de geometría: un `POINT` produce una sola celda, una `LINESTRING` muestrea sus segmentos, un `POLYGON` rellena su interior, y los valores `MULTI* / GEOMETRYCOLLECTION` devuelven la unión de sus componentes.
- `eEq`: Devuelve si una trayectoria H3 temporal toma alguna vez una de las celdas de un conjunto de celdas. Combinada con `geoToH3IndexSet` responde si un viaje pasa alguna vez por una región, sin materializar la geometría de la trayectoria, el prefiltro robusto y conservador para el predicado exacto `eIntersects(geometry, th3index)`.

A.14.3. Funciones de Acceso

- `startValue, endValue`: Devolver el primer o el último identificador de celda H3 de un valor temporal
 - `valueN`: Devolver el n-ésimo identificador de celda H3 distinto (indexado desde 1). Devuelve `NULL` si está fuera del rango.
 - `getValues`: Devolver el conjunto de identificadores de celda H3 distintos que toma la trayectoria
 - `valueAtTimestamp`: Devolver el identificador de celda H3 en una marca de tiempo dada, utilizando interpolación de pasos
-

A.14.4. Inspección de Índices

- **th3GetResolution**: Devolver la resolución temporal (0–15) de cada celda en una trayectoria
- **th3GetBaseCellNumber**: Devolver el número de celda base temporal (0–121) de cada celda en una trayectoria
- **isValidCell**: Devolver un booleano temporal que indica en cada instante si el valor es una celda H3 válida
- **th3IsResClassIii**: Devolver un booleano temporal que indica si cada celda tiene orientación de Clase III. La Clase III se alterna con la resolución: las resoluciones impares son de clase III, las pares son de clase II.
- **th3IsPentagon**: Devolver un booleano temporal que indica si cada celda es un pentágono (12 celdas por resolución son pentágonos; las restantes 110 en cada celda base descienden como hexágonos)

A.14.5. Jerarquía

- **th3CellToParent**: Devolver la celda padre temporal en la resolución dada. La forma sin argumento desciende a la siguiente resolución más gruesa.
- **th3CellToCenterChild**: Devolver la celda hijo central temporal en la resolución dada. La forma sin argumento desciende a la siguiente resolución más fina.
- **th3CellToChildPos**: Devolver la posición ordinal (basada en 0) de cada celda entre los hijos de su padre en la resolución dada
- **th3ChildPosToCell**: Devolver la celda hijo temporal en una posición ordinal dada del padre dado, en la resolución de hijo dada. Los dos operandos temporales se sincronizan sobre su eje de tiempo compartido.

A.14.6. Conversiones de Latitud/Longitud

- **th3index**: Devolver la celda H3 temporal en la resolución dada que contiene cada punto de un punto temporal geodésico o planar (el SRID debe ser 4326 para la sobrecarga de `tgeompoint`)
- **th3CellToLatlng**, **th3CellToLatlngTgeompoint**: Devolver el centroide geodésico temporal de cada celda en una trayectoria. Una sobrecarga planar (SRID 4326) está disponible bajo el nombre `th3CellToLatlngTgeompoint`.
- **th3CellToBoundary**: Devolver el límite poligonal temporal de cada celda en una trayectoria, como una geografía temporal

A.14.7. Aristas Dirigidas

- **th3AreNeighborCells**: Devolver un booleano temporal que indica si dos celdas temporales son vecinas en la cuadrícula en cada instante. Los dos operandos se sincronizan.
- **th3CellsToDirectedEdge**: Devolver un identificador de arista dirigida temporal a partir de un par de celdas temporales origen/destino
- **isValidDirectedEdge**: Devolver un booleano temporal que indica en cada instante si el valor es una arista dirigida H3 válida
- **th3GetDirectedEdgeOrigin**, **th3GetDirectedEdgeDestination**: Devolver la celda origen o destino temporal de una arista dirigida temporal
- **th3DirectedEdgeToBoundary**: Devolver el límite poligonal temporal de cada arista dirigida en una trayectoria, como una geografía temporal

A.14.8. Vértices

- **th3CellToVertex**: Devolver el vértice H3 temporal en el número de vértice dado (0–5 para hexágonos, 0–4 para pentágonos)
 - **th3VertexToLatlng**: Devolver las coordenadas geodésicas temporales de cada vértice en una trayectoria
 - **isValidVertex**: Devolver un booleano temporal que indica en cada instante si el valor es un vértice H3 válido
-

A.14.9. Recorrido de la Cuadrícula

- **th3GridDistance, <->**: Devolver la distancia temporal de saltos en la cuadrícula (número de pasos de una sola celda) entre dos celdas temporales. De forma equivalente, el operador `<->` devuelve la misma distancia. Los dos operandos se sincronizan sobre su eje de tiempo compartido.
- **th3CellToLocallj, th3LocalljToCell**: Convertir entre una celda temporal y un par de coordenadas locales (I, J) temporales relativas a una celda temporal de origen. La coordenada local se expone como un `tgeompoint` con el eje I en el slot x y J en y.

A.14.10. Métricas

- **th3CellArea**: Devolver el área temporal de cada celda en una trayectoria, en la unidad solicitada
- **th3EdgeLength**: Devolver la longitud temporal de cada arista dirigida en una trayectoria, en la unidad solicitada
- **greatCircleDistance**: Devolver la distancia temporal del gran círculo entre dos puntos geodésicos temporales (no entre celdas). Los dos operandos se sincronizan.

A.14.11. Comparaciones

A.14.11.1. Comparaciones Siempre y Alguna Vez

- **eEq, ?=**: Devolver `true` si una celda H3 temporal es alguna vez igual a la prueba en al menos un instante
- **aEq, %=**: Devolver `true` si una celda H3 temporal es siempre igual a la prueba a lo largo de todo su eje de tiempo
- **eNe, ?<>**: Devolver `true` si una celda H3 temporal es alguna vez diferente de la prueba
- **aNe, %<>**: Devolver `true` si una celda H3 temporal es siempre diferente de la prueba

A.14.11.2. Comparaciones Temporales

- **tEq, #=, tNe, #<>**: Devolver un booleano temporal que da la igualdad (o desigualdad) por instante entre una celda H3 temporal y una prueba

A.14.12. Operadores de Caja Envolvente

- **&&, @>, <@, ~=, -|**: Solapamiento de caja envolvente, contiene, contenido por, igual y adyacente
- **<<, &<, &>, >>, <<|, &<|, |&>, |>>**: Posición relativa a lo largo de los ejes espaciales (estrictamente a la izquierda, solapa a la izquierda, solapa a la derecha, estrictamente a la derecha; estrictamente abajo, solapa abajo, solapa arriba, estrictamente arriba)
- **<<#, &<#, #>>, #&>**: Posición relativa a lo largo del eje de tiempo (estrictamente antes, solapa antes, estrictamente después, solapa después)

A.14.13. Funciones que devuelven conjuntos

- **h3GridDisk**: Todas las celdas dentro de k pasos de cuadrícula del origen (incluyendo el origen cuando k=0).
- **h3GridRing**: Las celdas a *exactamente* k pasos de cuadrícula del origen. Falla cerca de pentágonos.
- **h3GridPathCells**: El camino inclusivo de celdas entre dos celdas de la misma resolución.
- **h3CellToChildren**: Todas las celdas hijas de una celda en la resolución objetivo.

- **h3CompactCells**: Representación compactada de un conjunto de celdas, las celdas más finas se fusionan en su padre cuando el conjunto hexagonal completo de hermanas está presente.
- **h3UncompactCells**: Representación descompactada a la resolución dada, la inversa de **h3CompactCells**.
- **h3OriginToDirectedEdges**: Todas las aristas dirigidas salientes de una celda (6 para hexágonos, 5 para pentágonos).
- **h3CellToVertexes**: Todos los vértices de una celda (6 para hexágonos, 5 para pentágonos).
- **h3GetIcosahedronFaces**: Los índices de caras del icosaedro (0..19) intersectadas por una celda.

A.14.14. Relaciones Espaciales

- **eContains, aContains, tContains**: Si una huella contiene a la otra (solo interiores)
- **eCovers, aCovers, tCovers**: Si una huella cubre a la otra (frontera incluida)
- **eDisjoint, aDisjoint, tDisjoint**: Si las huellas no comparten ningún punto
- **eIntersects, aIntersects, tIntersects**: Si las huellas comparten al menos un punto
- **eTouches, aTouches, tTouches**: Si las huellas comparten un punto de frontera pero ningún punto interior
- **eDwithin, aDwithin, tDwithin**: Si las huellas se encuentran dentro de una distancia dada, pasada como tercer argumento

A.14.15. Agregaciones

- **tCount**: Conteo temporal
- **wCount**: Conteo en ventana

A.14.16. Indexación

- **th3_rtree_ops**: Clase de operadores GiST para celdas H3 temporales. Indexa una caja envolvente espaciotemporal `stbox` por fila (la extensión geodésica de los límites de las celdas junto con el período temporal).
- **th3_quadtree_ops, th3_kdtree_ops**: Clases de operadores SP-GiST para celdas H3 temporales, indexadas por la caja envolvente espaciotemporal `stbox`. `th3_quadtree_ops` es la predeterminada (partición por quadtree); `th3_kdtree_ops` utiliza partición por árbol k-d y debe ser nombrada explícitamente.

A.15. Temporal Quadbin Cell Indices

A.15.1. Conversiones

- **:::** Convertir entre un `tquadbin` y un `tbigint` (explícito, sin coste)
- **:::** Convertir una celda QUADBIN temporal a un punto planar temporal (SRID 4326) de los centroides de las celdas. La conversión delega en `tquadbinCellToPoint`.

A.15.2. Celdas Estáticas

- **isValidIndex**: Devuelve si un valor es un identificador QUADBIN estructuralmente válido (etiqueta de cabecera y campo de resolución correctos)
 - **isValidCell**: Devuelve si un valor es una celda QUADBIN válida (un índice válido cuyas coordenadas de tesela están dentro de los límites de su resolución)
-

A.15.3. Accesores

- **startValue, endValue**: Devuelve el primer o el último identificador de celda QUADBIN de un valor temporal
- **valueN**: Devuelve el n-ésimo identificador de celda QUADBIN distinto (índice base 1). Devuelve `NULL` si está fuera de rango.
- **getValues**: Devuelve el conjunto de identificadores de celda QUADBIN distintos que toma la trayectoria
- **valueAtTimestamp**: Devuelve el identificador de celda QUADBIN en un instante dado, usando interpolación de pasos

A.15.4. Inspección del Índice

- **quadbinGetResolution, tquadbinGetResolution**: Devuelve el nivel de resolución (zoom) (0–26) de una celda. La forma estática devuelve un `integer`; la forma temporal devuelve la resolución de cada celda de una trayectoria como un `tint`.
- **isValidCell**: Devuelve un booleano temporal que indica en cada instante si el valor es una celda QUADBIN válida

A.15.5. Jerarquía

- **quadbinCellToParent, tquadbinCellToParent**: Devuelve la celda padre en la resolución más gruesa dada. La forma estática toma un único `quadbin`; la forma temporal devuelve el padre de cada celda de una trayectoria.
- **quadbinCellToChildren**: Devuelve las cuatro celdas hijas en la resolución más fina solicitada como un `quadbinset`. Es un auxiliar estático que devuelve un conjunto sobre un único `quadbin`; no tiene elevación temporal (los hijos son un conjunto por instante, diferido a la espera de una primitiva `tset<T>`).
- **quadbinCellSibling**: Devuelve la celda vecina en la misma resolución en la dirección dada (`up / down / left / right`)
- **quadbinGridDisk**: Devuelve todas las celdas dentro de `k` pasos de cuadrícula del origen (inclusive del origen en `k=0`), como un `quadbinset`. En la cuadrícula cuadrada el disco en el paso `k` es el bloque cuadrado de $(2k+1) \times (2k+1)$ celdas centrado en el origen.

A.15.6. Conversiones de Geometría

- **geoToQuadbinCell**: Devuelve la celda QUADBIN en la resolución dada que contiene una geometría de punto. La geometría debe ser un `POINT`; las longitudes se interpretan en SRID 4326.
- **quadbinCellToPoint, tquadbinCellToPoint**: Devuelve el centroide de una celda como un punto SRID 4326. La forma estática toma un único `quadbin`; la forma temporal devuelve el centroide de cada celda de una trayectoria como un `tgeompoint`.
- **quadbinCellToBoundary, tquadbinCellToBoundary**: Devuelve el contorno poligonal cuadrado de una celda como una geometría SRID 4326. La forma estática toma un único `quadbin`; la forma temporal devuelve el contorno de cada celda de una trayectoria como un `tgeometry`.

A.15.7. Teselas y Quadkeys

- **quadbinTileToCell**: Construye una celda a partir de coordenadas de tesela `slippy-map`: columna `x`, fila `y`, y zoom `z`
- **quadbinCellToTile**: Descompone una celda en su triple de coordenadas de tesela, devuelto como un arreglo de enteros `{x, y, z}`, la inversa de `quadbinTileToCell`
- **quadbinCellToQuadkey, tquadbinCellToQuadkey**: Emite la cadena `quadkey` en base 4 de una celda (un dígito por nivel de zoom, primero el más significativo; la celda mundial de resolución 0 se corresponde con la cadena vacía). La forma estática toma un único `quadbin`; la forma temporal devuelve el `quadkey` de cada celda de una trayectoria como un `ttext`.

A.15.8. Métricas

- **quadbinCellArea**, **tquadbinCellArea**: Devuelve el área de una celda sobre la esfera, en metros cuadrados. La forma estática toma un único `quadbin`; la forma temporal devuelve el área de cada celda de una trayectoria como un `tfloat`. El área de la celda se reduce hacia los polos, ya que los cuadrados web-mercator cubren progresivamente menos terreno a latitudes más altas.

A.15.9. Agregaciones

- **tCount**: Conteo temporal
- **wCount**: Conteo en ventana

A.15.10. Indexación

- **tquadbin_btree_ops**: Clase de operadores de árbol B por defecto para las celdas QUADBIN temporales, ordenando por el valor temporal subyacente.
- **tquadbin_hash_ops**: Clase de operadores hash por defecto para las celdas QUADBIN temporales, usada por las uniones hash y la agregación hash.

A.15.11. Comparisons

- **=, <, <=, >, >=**: Traditional comparisons

A.16. Temporal Point Clouds

A.16.1. Tipos estáticos `pcpoint` y `pcpatch`

A.16.1.1. Constructores ergonómicos

- **pcpoint**: Construye un `pcpoint` 2D o 3D directamente a partir de coordenadas `x/y(/z)`
- **pcpatch**: Construye un `pcpatch` a partir de una lista variádica de `pcpoints` que comparten el mismo `pcid`

A.16.1.2. Funciones de acceso

- **pcid**: Devuelve el identificador de esquema de un `pcpoint` o `pcpatch`
- **getX**, **getY**, **getZ**: Devuelve una coordenada de un `pcpoint`, NULL si la dimensión está ausente del esquema
- **getDim**: Devuelve cualquier dimensión con nombre de un `pcpoint`, NULL si el nombre es desconocido para el esquema
- **SRID**: Devuelve el identificador de referencia espacial (heredado del esquema)

A.16.2. Tipos de conjunto `pcpointset` y `pcpatchset`

A.16.2.1. Constructores

- **set**: Construye un conjunto a partir de un array de valores, todos los cuales deben compartir el mismo `pcid`
-

A.16.3. Caja Delimitadora `tpcbox`

A.16.3.1. Constructores

- `tpcbox`, `tpcbox_z`, `tpcbox_t`, `tpcbox_xt`, `tpcbox_zt`: Construir un `tpcbox` a partir de límites X/Y/Z/T explícitos y un `pcid`

A.16.3.2. Conversiones

- `tpcbox`: Convertir un `pcpoint` a un `tpcbox` degenerado (de extensión cero); el `SRID` y el `pcid` provienen del esquema
- `tpcbox`: Convertir un `pcpatch` a un `tpcbox` usando los límites PCBOUNDS 2D integrados en el parche; la segunda forma acepta un `SRID` explícito en lugar de buscarlo en el esquema

A.16.3.3. Accesores

- `hasX`, `hasZ`, `hasT`: Devuelve si un `tpcbox` tiene establecidas las dimensiones X/Y, Z o T
- `xmin`, `xmax`, `ymin`, `ymax`, `zmin`, `zmax`: Devuelve un límite de coordenada, NULL si la dimensión correspondiente está ausente
- `tmin`, `tmax`: Devuelve un límite temporal, NULL si la caja no tiene dimensión T
- `pcid`, `SRID`: Devuelve el identificador de esquema y el `SRID`

A.16.3.4. Transformaciones

- `round`: Devuelve un `tpcbox` con las coordenadas redondeadas al número de dígitos decimales indicado
- `setSRID`: Devuelve un `tpcbox` con el `SRID` sobrescrito. No reproyecta las coordenadas, para ello, proyecte primero el `tpcpoint` subyacente y tome la caja delimitadora del resultado

A.16.3.5. Operaciones de conjuntos

- `tpcbox_union`: Devuelve la unión de dos `tpcboxes`; ambas deben compartir el mismo `pcid`
- `tpcbox_intersection`: Devuelve la intersección de dos `tpcboxes`, NULL si son disjuntas o el `pcid` no coincide

A.16.3.6. Operadores Topológicos

- `@>`: Devuelve true si el primer `tpcbox` contiene al segundo
- `<@`: Devuelve true si el primer `tpcbox` está contenido en el segundo
- `&&`: Devuelve true si dos `tpcboxes` se solapan
- `~=`: Devuelve true si dos `tpcboxes` son iguales en las dimensiones que comparten
- `-|-`: Devuelve true si dos `tpcboxes` son adyacentes (comparten un límite)

A.16.3.7. Operadores de Posición

- `<<`, `&<`, `>>`, `&>`: Comprueba el orden en el eje X de dos `tpcboxes`
- `<<|`, `&<|`, `|>>`, `|&>`: Comprueba el orden en el eje Y de dos `tpcboxes`
- `<<|`, `&<|`, `|>>`, `|&>`: Comprueba el orden en el eje Z de dos `tpcboxes` (devuelve false si alguna carece de Z)
- `<<#`, `&<#`, `#>>`, `#&>`: Comprueba el orden en el eje temporal de dos `tpcboxes` (devuelve false si alguna carece de T)

A.16.3.8. Comparaciones

- `=, <>, <, <=, >=, >`: Compara dos `tpcboxes`; orden total sobre la codificación canónica (`pcid`, `srid`, `flags`, período y a continuación límites espaciales en orden XYZ-min/max)

A.16.4. Tipo Temporal `tpcpoint`

A.16.4.1. Constructores

- `tpcpoint`: Construir un `pcpoint` temporal de subtipo instante
- `tpcpointSeq`: Construir un `pcpoint` temporal de subtipo secuencia a partir de un array de instantes. La interpolación es una de `'discrete'`, `'step'`; `'step'` es el valor predeterminado para `tpcpoint`, ya que los `pcpoints` llevan dimensiones heterogéneas que no se interpolan de manera lineal
- `tpcpointSeqSet`: Construir un `pcpoint` temporal de subtipo conjunto de secuencias a partir de un array de secuencias

A.16.4.2. Funciones de Acceso

- `pcid`: Devolver el id de esquema de `pgPointCloud` compartido por todos los instantes
- `SRID`: Devolver el SRID heredado del esquema

A.16.4.3. Proyecciones por Dimensión

- `getX`, `getY`, `getZ`: Proyectar un `tpcpoint` sobre una dimensión espacial, produciendo un `tfloat`
- `getDim`: Proyectar un `tpcpoint` sobre cualquier dimensión de esquema con nombre, produciendo un `tfloat`

A.16.4.4. Conversiones

- `:::` Convertir un `tpcpoint` a un punto de geometría temporal, proyectando todas las dimensiones no espaciales mientras se preservan las marcas temporales. El resultado tiene el mismo SRID que el esquema; su dimensionalidad (2D frente a 3D) sigue la presencia de Z en el esquema. No se proporciona una conversión inversa: reconstruir un `pcpoint` requiere un esquema destino que el valor temporal solo no puede suministrar

A.16.4.5. Transformaciones

- `tpcpointInst`, `tpcpointSeq`, `tpcpointSeqSet`: Transformar un `tpcpoint` en otro subtipo
- `setInterp`: Transformar un `tpcpoint` a otra interpolación

A.16.4.6. Comparaciones

- `=, <>, <, <=, >=, >`: Comparar dos `tpcpoints` lexicográficamente sobre sus codificaciones canónicas
 - `eEq`, `aEq`, `eNe`, `aNe`, `?=`, `%=`: Comprobar si el valor en algún instante (alguna vez) o en todos los instantes (siempre) es igual a, o distinto de, un `pcpoint` u otro `tpcpoint`
 - `tEq`, `tNe`, `#=`: Devolver la igualdad o desigualdad temporal de un `tpcpoint` con un `pcpoint` o con otro `tpcpoint`, como un `tbool`
-

A.16.4.7. Restricciones

- **atValue, minusValue, atValues, minusValues**: Restringir un tpcpoint a (al complemento de) un valor pcpoint o un conjunto de valores pcpoint
- **atTime, minusTime**: Restringir un tpcpoint a un selector temporal (marca de tiempo, periodo, conjunto o spanset), o eliminarlo
- **atTpcbox, minusTpcbox**: Restringir un tpcpoint al extent espacial / temporal de un tpcbox, o eliminarlo. Devuelve NULL cuando el pcid del tpcbox no coincide con el del tpcpoint. Internamente proyecta a un tgeompoint a través de la caché de esquemas, restringe la proyección mediante el stbox equivalente y devuelve el span temporal del resultado al tpcpoint original para preservar los valores pcpoint completos
- **beforeTimestamp, afterTimestamp**: Restringir un tpcpoint a los instantes anteriores o posteriores a una marca de tiempo. El indicador strict excluye el instante en la marca de tiempo misma

A.16.4.8. Modificaciones

- **insert, update**: Insertar un tpcpoint en otro, o actualizar otro con él
- **deleteTime**: Eliminar un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpoint
- **appendInstant, appendSequence**: Añadir un instante o una secuencia a un tpcpoint

A.16.4.9. Fragmentación

- **unnest**: Devolver los valores distintos de un tpcpoint con el conjunto de spans durante el cual toma cada uno de ellos
- **timeSplit**: Fragmentar un tpcpoint en fragmentos, uno por cada intervalo de tiempo
- **spans**: Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpoint
- **splitNSpans, splitEachNSpans**: Devuelve los lapsos de tiempo de un tpcpoint dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.16.4.10. Operadores de caja delimitadora

- **&&, @>, <@, ~=, -|**: Predicados topológicos: && solapa, @> contiene, <@ contenido en, ~= mismo, -| - adyacente.
- **<<, >>, <<|, |>>, <</, />>, <<#, #>>**: Predicados direccionales estrictos sobre el eje X (<<, >>), eje Y (<<|, |>>), eje Z (<</, />>) y eje temporal (<<#, #>>), más sus variantes "solapa o X": &<, &>, &<|, |&>, &</, /&>, &<#, #&>.
- **|=, nearestApproachDistance**: Distancia de aproximación más cercana, ordenable KNN mediante la clase de operadores GiST: `SELECT ... ORDER BY temp |=| query LIMIT N` se acelera con índice. El desajuste de pcid produce infinito.

A.16.4.11. Relaciones espaciales

- **eIntersects, aIntersects**: Alguna vez / siempre intersecta.
- **eDisjoint, aDisjoint**: Alguna vez / siempre disjunto.
- **eDwithin, aDwithin**: Alguna vez / siempre dentro de la distancia.

A.16.5. Tipo Temporal tpcpatch

A.16.5.1. Constructores

- **tpcpatch**: Construir un pcpatch temporal de subtipo instante
- **tpcpatchSeq**: Construir un pcpatch temporal de subtipo secuencia
- **tpcpatchSeqSet**: Construir un pcpatch temporal de subtipo conjunto de secuencias

A.16.5.2. Accesores

- **startNumPoints, endNumPoints**: Devolver el número de puntos del parche contenido en el primer o último instante
- **numPoints**: Devolver el número total de puntos de todos los parches de todos los instantes. Se lee directamente de la cabecera de cada parche, sin descompresión.
- **points**: Función devuelve-conjunto: emite una fila por cada (marca de tiempo de instante, punto) recorriendo el parche de cada instante y descomponiéndolo mediante la API C en memoria de pgPointCloud. Útil para combinar una columna tpcpatch con predicados por punto que los operadores de nivel de caja delimitadora no pueden expresar. El coste es $O(\text{total de puntos})$, una descompresión por instante más una emisión de fila por punto.

A.16.5.3. Transformaciones

- **tpcpatchInst, tpcpatchSeq, tpcpatchSeqSet**: Transformar un tpcpatch en otro subtipo
- **setInterp**: Transformar un tpcpatch a otra interpolación

A.16.5.4. Restricciones

- **atValue, minusValue, atValues, minusValues**: Restringir un tpcpatch a (al complemento de) un valor pcpatch o un conjunto de valores pcpatch
- **atTpcbox, minusTpcbox**: Restringir un tpcpatch a los instantes cuyo parche supera una prueba de superposición aproximada de PCBOUNDS frente al tpcbox, o eliminarlos. La granularidad es a nivel de parche: cada instante superviviente conserva su payload pcpatch textualmente, sin descompresión por punto. Como el PCBOUNDS de pgPointCloud es 2D, la dimensión Z de la caja se ignora a esta granularidad. Devuelve NULL cuando el pcid del tpcbox no coincide.
- **atTpcboxFine, minusTpcboxFine**: Variante por punto de la anterior: cada instante superviviente contiene un pcpatch recién construido que incluye solo los puntos dentro (o fuera, en el caso de minus) del tpcbox en 2D, y en 3D cuando la caja tiene dimensión Z. Los instantes cuyo parche filtra hasta cero puntos se descartan. Más lenta que la variante aproximada porque cada parche se descomprime y reconstruye; úsela cuando se necesite fidelidad a nivel de punto.
- **atGeometry, minusGeometry**: Restringir un tpcpatch a los puntos cuya proyección XY intersecta (o no intersecta, en el caso de minus) una geometría 2D. Z se ignora. El llamador debe garantizar la compatibilidad de SRID entre el esquema del parche y la geometría.
- **beforeTimestamp, afterTimestamp**: Restringir un tpcpatch a los instantes anteriores o posteriores a una marca de tiempo. El indicador strict excluye el instante en la marca de tiempo misma

A.16.5.5. Modificaciones

- **insert, update**: Insertar un tpcpatch en otro, o actualizar otro con él
- **deleteTime**: Eliminar un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpatch
- **appendInstant, appendSequence**: Añadir un instante o una secuencia a un tpcpatch

A.16.5.6. Fragmentación

- **unnest**: Devolver los valores distintos de un tpcpatch con el conjunto de spans durante el cual toma cada uno de ellos
- **timeSplit**: Fragmentar un tpcpatch en fragmentos, uno por cada intervalo de tiempo
- **spans**: Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpatch
- **splitNSpans, splitEachNSpans**: Devuelve los lapsos de tiempo de un tpcpatch dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.16.5.7. Relaciones espaciales

- **eIntersects**: Devolver verdadero si y solo si al menos un punto de alguno de los instantes del tpcpatch intersecta la geometría (solo XY).

A.16.5.8. Operadores de caja delimitadora

- **&&, @>, <@, ~=, -|-**: Predicados topológicos: **&&**, **@>**, **<@**, **~=**, **-|-**. Véase Sección 19.4.6.
- **<<, >>, <<|, |>>, <</, />>, <<#, #>>**: Predicados direccionales estrictos sobre los ejes X / Y / Z / tiempo (**<<**, **>>**, **<<|**, **|>>**, **<</**, **/>>**, **<<#**, **#>>**) y sus variantes «solapa-o-X» (**&<**, **&>**, **&<|**, **|&>**, **&</**, **/&>**, **&<#**, **#&>**). Véase Sección 19.4.7.
- **|=|, nearestApproachDistance**: La distancia de aproximación más cercana (**|=|**) es ordenable KNN a través de la clase de operador GiST.

A.16.5.9. Comparaciones

- **eEq, aEq, eNe, aNe, ?=, %=**: Comprobar si el valor en algún instante (alguna vez) o en todos los instantes (siempre) es igual a, o distinto de, un pcpatch u otro tpcpatch
- **tEq, tNe, #=**: Devolver la igualdad o desigualdad temporal de un tpcpatch con un pcpatch o con otro tpcpatch, como un tbool

A.16.6. Agregaciones

- **extent**: Agregado de caja delimitadora sobre una columna de valores tpcpoint, tpcpatch o tpcbox. Devuelve el tpcbox más pequeño que contiene todas las entradas. Agregar a través de pcid distintos genera un error, la caja delimitadora sería ininterpretable ya que las dimensiones son relativas al pcid.
- **tCount, wCount**: Conteo temporal y conteo por ventana de filas superpuestas en cada marca de tiempo. El resultado es un tint. Todas las filas de entrada deben compartir el mismo subtipo temporal (instante / secuencia / conjunto de secuencias).
- **tnpoints**: Conteo de puntos de pcpatch por instante, sumado a través de las filas. Distinto de **tCount(tpcpatch)**, que cuenta el número de parches (siempre 1 por instante). **tnpoints** se lee como «cuántos puntos había en la nube en el tiempo t», la primitiva natural para el control de calidad de la ingesta y los paneles de densidad a lo largo del tiempo.
- **tdensity**: Densidad de puntos por instante (**numPoints / área-bbox-xy**) sumada a través de las filas. Cada pcpatch lleva el PCBOUNDS en línea (**xmin / xmax / ymin / ymax**) por lo que el término de área de la caja delimitadora se lee directamente, sin recálculo. Un parche de 1 punto o colineal produce **+Infinity** para ese instante; filtre con **isfinite()** en consultas posteriores si es necesario.
- **merge, mergeAgg**: Combina múltiples valores temporales en uno solo con todos los instantes de entrada fusionados en orden temporal. Todas las entradas deben compartir el mismo pcid y subtipo.
- **appendInstant, appendSequence, appendInstantAgg, appendSequenceAgg**: Agregados de construcción en flujo: **appendInstant** ensambla instantes en una secuencia; **appendSequence** ensambla secuencias en un conjunto de secuencias. Útil para cargadores de trayectorias que consumen filas en orden temporal.

A.16.7. Salida de texto

- **asText**: Devolver la representación textual de un tpcpoint, o de un arreglo de ellos, y convertir un tpcpoint a texto
- **asText**: Devolver la representación textual de un tpcpatch, o de un arreglo de ellos, y convertir un tpcpatch a texto

A.16.8. Salida MF-JSON

- **asMFJSON**: Para `tpcpoint`, el tipo temporal se emite como `{"type": "MovingPCPoint", ... "coordinates": [[X, Y, Z], ...], "datetimes": [...]}`. X/Y/[Z] se extraen del `pcpoint` de cada instante mediante la caché de esquemas; si el esquema carece de X/Y el array de coordenadas está vacío para ese instante.
- **asMFJSON**: Para `tpcpatch`, la etiqueta de tipo es "MovingPCPatch" y cada instante emite un pequeño objeto `{"pcid": N, "npoints": N, "bounds": [xmin, xmax, ymin, ymax]}`. La carga de puntos comprimida no está intencionadamente en el JSON, use `asBinary` / `asHexWKB` para el ida y vuelta.

A.16.9. Indexes

- **GiST, SP-GiST, B-tree, and hash opclasses for tpcpoint, tpcpatch, and tpcbox**

A.16.10. Binary Representation

- **asBinary, tpcpointFromBinary, tpcpatchFromBinary**: Binary I/O with embedded schema for portability

A.16.11. PDAL Integration

- **readers.tpcpatch**: PDAL reader that reads `tpcpatch` values from a SQL query
- **writers.tpcpatch**: PDAL writer that writes `tpcpatch` values to a PostgreSQL table

A.16.12. PCL Bridge

- **pcpatch_to_pcd, pcpatch_from_pcd, pcpatch_voxel_grid, pcpatch_sor, pcpatch_icp, pcpatch_normals**: PCL-backed SQL functions for point cloud processing

A.17. Raster Sampling

A.17.1. Operador de Muestreo

- **rasterValue**: Muestrear una banda de ráster en cada instante de una trayectoria

A.17.2. Accesores de Ráster

- **numBands**: Devolver el número de bandas de un ráster

A.17.3. Muestreo Raquet / QUADBIN

- **quadbins**: Devolver las celdas QUADBIN distintas en un nivel de zoom cubiertas por una trayectoria
 - **rasterTileValueQuadbin**: Muestrear un chip ráster de Raquet a lo largo de una trayectoria usando una celda QUADBIN para la georreferenciación
 - **raquet**: Construir una tesela ráster Raquet a partir de sus bytes de píxeles, dimensiones, celda QUADBIN y tipo de píxel
 - **raquetRead**: Decodificar un archivo ráster en memoria en una tesela ráster Raquet mediante GDAL
 - **rasterTileValue**: Muestrear una tesela ráster `raquet` a lo largo de una trayectoria
 - **rasterTileValue**: Muestrear un arreglo de teselas ráster `raquet` a lo largo de una trayectoria
-

A.17.4. Serialización de Raquet

- **asBinary**: Devolver la representación Well-Known Binary (WKB) de una tesela Raquet
- **asHexWKB**: Devolver la representación Well-Known Binary codificada en hexadecimal ASCII (HexWKB) de una tesela Raquet
- **pixels**: Devolver los bytes de píxeles de un mosaico Raquet
- **raquetFromBinary**: Devolver una tesela Raquet a partir de su representación Well-Known Binary (WKB)
- **raquetFromHexWKB**: Devolver una tesela Raquet a partir de su representación Well-Known Binary codificada en hexadecimal ASCII (HexWKB)

A.17.5. Accesorios y Comparación de Raquet

- **quadbin**: Devolver la celda QUADBIN de una tesela Raquet
- **width**: Devolver el ancho en píxeles de una tesela Raquet
- **height**: Devolver el alto en píxeles de una tesela Raquet
- **nodata**: Devolver el valor centinela nodata de una tesela Raquet
- **pixtype**: Devolver el nombre del tipo de dato de píxel de una tesela Raquet
- **stbox**: Convertir una tesela Raquet en una caja espaciotemporal
- **=, <>, <, <=, >=, >, cmp**: Comparar dos teselas Raquet

A.17.6. Operadores de Restricción y Predicado

- **atRasterValue**: Devolver los instantes de una trayectoria donde el valor ráster muestreado cae dentro de un rango de flotante
 - **minusRasterValue**: Devolver los instantes de una trayectoria donde el valor ráster muestreado cae fuera de un rango de flotante
 - **eRasterValue**: Devolver verdadero si la trayectoria alguna vez muestrea un valor de píxel ráster dentro de un rango de flotante
 - **aRasterValue**: Devolver verdadero si cada instante de la trayectoria dentro de la extensión del ráster muestrea un valor de píxel dentro de un rango de flotante
-

Apéndice B

Generador de datos sintéticos

En muchas circunstancias, es necesario tener un conjunto de datos de prueba para evaluar enfoques de implementación alternativos o realizar evaluaciones comparativas. A menudo se requiere que tal conjunto de datos tenga requisitos particulares en tamaño o en las características intrínsecas de sus datos. Incluso si un conjunto de datos del mundo real pudiera estar disponible, puede que no sea ideal para tales experimentos por múltiples razones. Por lo tanto, un generador de datos sintéticos que pueda personalizarse para producir datos de acuerdo con los requisitos dados suele ser la mejor solución. Obviamente, los experimentos con datos sintéticos deben complementarse con experimentos con datos del mundo real para tener una comprensión profunda del problema en cuestión.

MobilityDB proporciona un generador simple de datos sintéticos que se puede utilizar para tales fines. En particular, se utilizó este generador de datos para generar la base de datos utilizada para las pruebas de regresión en MobilityDB. El generador de datos está programado en PL/pgSQL para que se pueda personalizar fácilmente. Se encuentra en el directorio `datagen` en el repositorio. En este apéndice, presentamos brevemente la funcionalidad básica del generador. Primero enumeramos las funciones que generan valores aleatorios para varios tipos de datos de PostgreSQL, PostGIS y MobilityDB y luego damos ejemplos de cómo se usan estas funciones para generar tablas de dichos valores. Los parámetros de las funciones no están especificados, consulte los archivos fuente donde se pueden encontrar explicaciones detalladas sobre los distintos parámetros.

B.1. Generador para tipos PostgreSQL

- `random_bool`: Generar un booleano aleatorio
 - `random_int`: Generar un entero aleatorio
 - `random_int_array`: Generar una matriz de enteros aleatorios
 - `random_int4range`: Generar un rango aleatorio de enteros
 - `random_float`: Generar un número flotante aleatorio
 - `random_float_array`: Generar una matriz de números flotantes aleatorios
 - `random_text`: Generar un texto aleatorio
 - `random_timestamptz`: Generar una marca de tiempo con zona horaria aleatoria
 - `random_timestamptz_array`: Generar una matriz de marcas de tiempo con zona horaria aleatorias
 - `random_minutes`: Generar un intervalo de minutos aleatorio
 - `random_tstzrange`: Generar rango aleatorio de marcas de tiempo con zona horaria
 - `random_tstzrange_array`: Generar una matriz de rangos aleatorios de marcas de tiempo con zona horaria
-

B.2. Generador para tipos PostGIS

- `random_geom_point`: Generar un punto geométrico 2D aleatorio
 - `random_geom_point3D`: Generar un punto geométrico 3D aleatorio
 - `random_geog_point`: Generar un punto geográfico 2D aleatorio
 - `random_geog_point3D`: Generar un punto geográfico 3D aleatorio
 - `random_geom_point_array`: Generar una matriz de puntos geométricos 2D aleatorios
 - `random_geom_point3D_array`: Generar una matriz de puntos geométricos 3D aleatorios
 - `random_geog_point_array`: Generar una matriz puntos geográficos 2D aleatorios
 - `random_geog_point3D_array`: Generar una matriz de puntos geográficos 3D aleatorios
 - `random_geom_linestring`: Generar una cadena lineal geométrica 2D aleatoria
 - `random_geom_linestring3D`: Generar una cadena lineal geométrica 3D aleatoria
 - `random_geog_linestring`: Generar una cadena lineal geográfica 2D aleatoria
 - `random_geog_linestring3D`: Generar una cadena lineal geográfica 3D aleatoria
 - `random_geom_polygon`: Generar un polígono geométrico 2D sin agujeros aleatorio
 - `random_geom_polygon3D`: Generar un polígono geométrico 3D sin agujeros aleatorio
 - `random_geog_polygon`: Generar un polígono geográfico 2D sin agujeros aleatorio
 - `random_geog_polygon3D`: Generar un polígono geográfico 3D sin agujeros aleatorio
 - `random_geom_multipoint`: Generar un multipunto geométrico 2D aleatorio
 - `random_geom_multipoint3D`: Generar un multipunto geométrico 3D aleatorio
 - `random_geog_multipoint`: Generar un multipunto geográfico 2D aleatorio
 - `random_geog_multipoint3D`: Generar un multipunto geográfico 3D aleatorio
 - `random_geom_multilinestring`: Generar una multicadena lineal geométrica 2D aleatoria
 - `random_geom_multilinestring3D`: Generar una multicadena lineal geométrica 3D aleatoria
 - `random_geog_multilinestring`: Generar una multicadena lineal geográfica 2D aleatoria
 - `random_geog_multilinestring3D`: Generar una multicadena lineal geográfica 3D aleatoria
 - `random_geom_multipolygon`: Generar un multipolígono geométrico 2D sin agujeros aleatorio
 - `random_geom_multipolygon3D`: Generar un multipolígono geométrico 3D sin agujeros aleatorio
 - `random_geog_multipolygon`: Generar un multipolígono geográfico 2D sin agujeros aleatorio
 - `random_geog_multipolygon3D`: Generar un multipolígono geográfico 3D sin agujeros aleatorio
-

B.3. Generador para tipos de rango, de tiempo y de cuadro delimitador MobilityDB

- `random_intspan`: Generar un rango aleatorio de enteros
- `random_floatspan`: Generar un rango aleatorio de números flotantes
- `random_tstzspan`: Generar un `tstzspan` aleatorio
- `random_tstzspan_array`: Generar una matriz de valores `tstzspan` aleatorios
- `random_tstzset`: Generar un `tstzset` aleatorio
- `random_tstzspanset`: Generar un `tstzspanset` aleatorio
- `random_tbox`: Generar un `tbox` aleatorio
- `random_stbox`: Generar un `stbox` 2D aleatorio
- `random_stbox3D`: Generar un `stbox` 3D aleatorio
- `random_geodstbox`: Generar un `stbox` geodético 2D aleatorio
- `random_geodstbox3D`: Generar un `stbox` geodético 3D aleatorio

B.4. Generador para tipos temporales MobilityDB

- `random_tbool_inst`: Generar un booleano temporal aleatorio de subtipo instante
- `random_tint_inst`: Generar un entero temporal aleatorio de subtipo instante
- `random_tfloat_inst`: Generar un flotante temporal aleatorio de subtipo instante
- `random_ttext_inst`: Generar un texto temporal aleatorio de subtipo instante
- `random_tgeompoint_inst`: Generar un punto geométrico temporal 2D aleatorio de subtipo instante
- `random_tgeompoint3D_inst`: Generar un punto geométrico temporal 3D aleatorio de subtipo instante
- `random_tgeogpoint_inst`: Generar un punto geográfico temporal 2D aleatorio de subtipo instante
- `random_tgeogpoint3D_inst`: Generar un punto geográfico temporal 3D aleatorio de subtipo instante
- `random_tbool_discseq`: Generar un booleano temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tint_discseq`: Generar un entero temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tfloat_discseq`: Generar un flotante temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_ttext_discseq`: Generar un texto temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeompoint_discseq`: Generar un punto temporal geométrico 2D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeompoint3D_discseq`: Generar un punto geométrico temporal 3D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeogpoint_discseq`: Generar un punto geográfico temporal 2D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeogpoint3D_discseq`: Generar un punto geográfico temporal 3D aleatorio de subtipo secuencia con interpolación discreta
- `random_tbool_seq`: Generar un booleano temporal aleatorio de subtipo secuencia

- `random_tint_seq`: Generar un entero temporal aleatorio de subtipo secuencia
- `random_tfloat_seq`: Generar un flotante temporal aleatorio de subtipo secuencia
- `random_ttext_seq`: Generar un texto temporal aleatorio de subtipo secuencia
- `random_tgeompoint_seq`: Generar un punto geométrico temporal 2D aleatorio de subtipo secuencia
- `random_tgeompoint3D_seq`: Generar un punto geométrico temporal 3D aleatorio de subtipo secuencia
- `random_tgeogpoint_seq`: Generar un punto geográfico temporal 2D aleatorio de subtipo secuencia
- `random_tgeogpoint3D_seq`: Generar un punto geográfico temporal 3D aleatorio de subtipo secuencia
- `random_tbool_seqset`: Generar un booleano temporal aleatorio de subtipo conjunto de secuencias
- `random_tint_seqset`: Generar un entero temporal aleatorio de subtipo conjunto de secuencias
- `random_tfloat_seqset`: Generar un flotante temporal aleatorio de subtipo conjunto de secuencias
- `random_ttext_seqset`: Generar un texto temporal aleatorio de subtipo conjunto de secuencias
- `random_tgeompoint_seqset`: Generar un punto geométrico temporal 2D aleatorio de subtipo conjunto de secuencias
- `random_tgeompoint3D_seqset`: Generar un punto geométrico temporal 3D aleatorio de subtipo conjunto de secuencias
- `random_tgeogpoint_seqset`: Generar un punto geográfico temporal 2D aleatorio de subtipo conjunto de secuencias
- `random_tgeogpoint3D_seqset`: Generar un punto geográfico temporal 3D aleatorio de subtipo conjunto de secuencias

B.5. Generación de tablas con valores aleatorios

Los archivos `create_test_tables_temporal.sql` y `create_test_tables_tpoint.sql` dan ejemplos de utilización de las funciones que generan valores aleatorios listadas arriba. Por ejemplo, el primer archivo define la función siguiente.

```
CREATE OR REPLACE FUNCTION create_test_tables_temporal(size integer DEFAULT 100)
RETURNS text AS $$
DECLARE
    perc integer;
BEGIN
    perc := size * 0.01;
    IF perc < 1 THEN perc := 1; END IF;

    -- ... Table generation ...

RETURN 'The End';
END;
$$ LANGUAGE 'plpgsql';
```

La función tiene un parámetro `size` que define el número de filas en las tablas. Si no se proporciona, crea por defecto tablas de 100 filas. La función define una variable `perc` que calcula el 1 % del tamaño de las tablas. Este parámetro se utiliza, por ejemplo, para generar tablas con un 1 % de valores nulos. A continuación ilustramos algunos de los comandos que generan tablas.

La creación de una tabla `tbl_float` que contiene valores aleatorios `float` en el rango [0,100] con 1 % de valores nulos se da a continuación.

```
CREATE TABLE tbl_float AS
/* Add perc NULL valores */
SELECT k, NULL AS f
FROM generate_series(1, perc) AS k UNION
SELECT k, random_float(0, 100)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tbox` que contiene valores aleatorios `tbox` donde los límites de los valores están en el rango [0,100] y los límites de las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] se da a continuación.

```
CREATE TABLE tbl_tbox AS
/* Add perc NULL valores */
SELECT k, NULL AS b
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbox(0, 100, '2001-01-01', '2001-12-31', 10, 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_floatspan` que contiene valores aleatorios `floatspan` donde los límites de los valores están en el rango [0,100] y la máxima diferencia entre los límites inferiores y superiores es 10 se da a continuación.

```
CREATE TABLE tbl_floatspan AS
/* Add perc NULL valores */
SELECT k, NULL AS f
FROM generate_series(1, perc) AS k UNION
SELECT k, random_floatspan(0, 100, 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tstzset` que contiene valores aleatorios `tstzset` que tienen entre 5 y 10 marcas de tiempo donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y el máximo intervalo entre marcas de tiempo consecutivas es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tstzset AS
/* Add perc NULL valores */
SELECT k, NULL AS ts
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzset('2001-01-01', '2001-12-31', 10, 5, 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tstzspan` que contiene valores aleatorios `tstzspan` donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y la máxima diferencia entre los límites inferiores y superiores es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tstzspan AS
/* Add perc NULL valores */
SELECT k, NULL AS p
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzspan('2001-01-01', '2001-12-31', 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_geom_point` que contiene valores aleatorios `geometry` 2D point valores, donde las coordenadas x e y están en el rango [0, 100] y en SRID 3812 se da a continuación.

```
CREATE TABLE tbl_geom_point AS
SELECT 1 AS k, geometry 'SRID=3812;point empty' AS g UNION
SELECT k, random_geom_point(0, 100, 0, 100, 3812)
FROM generate_series(2, size) k;
```

Observe que la tabla contiene un valor de punto vacío. Si no se proporciona el SRID, se establece de forma predeterminada en 0.

La creación de una tabla `tbl_geog_point3D` que contiene valores aleatorios `geography` 3D point valores, donde las coordenadas x, y, y z están, respectivamente, en los rangos [-10, 32], [35, 72] y [0, 1000] y en SRID 7844 se da a continuación.

```
CREATE TABLE tbl_geog_point3D AS
SELECT 1 AS k, geography 'SRID=7844;pointZ empty' AS g UNION
SELECT k, random_geog_point3D(-10, 32, 35, 72, 0, 1000, 7844)
FROM generate_series(2, size) k;
```

Nótese que los valores de latitud y longitud se eligen para cubrir aproximadamente la Europa continental. Si no se proporciona el SRID, se establece de forma predeterminada en 4326.

La creación de una tabla `tbl_geom_linestring` que contiene valores aleatorios `geometry` 2D `linestring` valores que tienen entre 5 y 10 vértices, donde las coordenadas `x` e `y` están en el rango `[0, 100]` y en SRID 3812 y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k, random_geom_linestring(0, 100, 0, 100, 10, 5, 10, 3812)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_linestring` que contiene valores aleatorios `geometry` 2D `linestring` valores que tienen entre 5 y 10 vértices, donde las coordenadas `x` e `y` están en el rango `[0, 100]` y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k, random_geom_linestring(0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_polygon3D` que contiene valores aleatorios `geometry` 3D `polygon` valores sin agujeros, que tienen entre 5 y 10 vértices, donde las coordenadas `x`, `y`, y `z` están en el rango `[0, 100]` y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_polygon3D AS
SELECT 1 AS k, geometry 'polygon Z empty' AS g UNION
SELECT k, random_geom_polygon3D(0, 100, 0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_multipoint` que contiene valores aleatorios `geometry` 2D `multipunto` valores que tienen entre 5 y 10 points, donde las coordenadas `x` e `y` están en el rango `[0, 100]` y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_multipoint AS
SELECT 1 AS k, geometry 'multipunto empty' AS g UNION
SELECT k, random_geom_multipoint(0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geog_multilinestring` que contiene valores aleatorios `geography` 2D `multilinestring` valores que tienen entre 5 y 10 `linestrings`, cada una teniendo entre 5 y 10 vértices, donde las coordenadas `x` e `y` estan, respectivamente, en el rangos `[-10, 32]` y `[35, 72]` y la máxima diferencia entre valores de coordenadas consecutivos es 10 se da a continuación.

```
CREATE TABLE tbl_geog_multilinestring AS
SELECT 1 AS k, geography 'multilinestring empty' AS g UNION
SELECT k, random_geog_multilinestring(-10, 32, 35, 72, 10, 5, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geometry3D` que contiene valores aleatorios `geometry` 3D de varios tipos se da a continuación. Esta función requiere que las tablas para los diversos tipos de geometría se hayan creado previamente.

```
CREATE TABLE tbl_geometry3D (
  k serial PRIMARY KEY,
  g geometry);
INSERT INTO tbl_geometry3D(g)
(SELECT g FROM tbl_geom_point3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_linestring3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_polygon3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipoint3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multilinestring3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipolygon3D ORDER BY k LIMIT (size * 0.2));
```

La creación de una tabla `tbl_tbool_inst` que contiene valores aleatorios `tbool` valores de subtipo instante donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] se da a continuación.

```
CREATE TABLE tbl_tbool_inst AS
/* Add perc NULL valores */
SELECT k, NULL AS inst
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbool_inst('2001-01-01', '2001-12-31')
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT inst FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows con the same timestamp */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT tboolInst(random_bool(), getTimestamp(inst))
FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
```

Como se puede ver arriba, la tabla tiene un porcentaje de valores nulos, de duplicados y de filas con la misma marca de tiempo.

La creación de una tabla `tbl_tint_discseq` que contiene valores aleatorios `tint` valores de subtipo secuencia con interpolación discreta que tienen entre 5 y 10 marcas de tiempo donde the integer valores están en el rango [0, 100], las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre dos valores consecutivos es 10 y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tint_discseq AS
/* Add perc NULL valores */
SELECT k, NULL AS ti
FROM generate_series(1, perc) AS k UNION
SELECT k, random_tint_discseq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS ti
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows con the same timestamp */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti + random_int(1, 2) FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc rows that meet */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc rows that overlap */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute', (endTimestamp(ti)-startTimestamp(ti))/2))
FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

Como se puede ver arriba, la tabla tiene un porcentaje de valores nulos, de duplicados, de filas con la misma marca de tiempo, de filas que se encuentran y de filas que se superponen.

La creación de una tabla `tbl_tfloat_seq` que contiene valores aleatorios `tfloat` valores de subtipo secuencia que tienen entre 5 y 10 marcas de tiempo donde los valores `float` están en el rango [0, 100], las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre dos valores consecutivos es 10 y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tfloat_seq AS
/* Add perc NULL valores */
SELECT k, NULL AS seq
```

```

FROM generate_series(1, perc) AS k UNION
SELECT k, random_tfloat_contseq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS seq
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq + random_int(1, 2) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, timeSpan(seq)) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, date_trunc('minute', timeSpan(seq)/2))
FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);

```

La creación de una tabla `tbl_ttext_seqset` que contiene valores aleatorios `ttext` de subtipo conjunto de secuencias que tienen entre 5 y 10 secuencias, cada una teniendo entre 5 y 10 marcas de tiempo, donde los valores de texto tienen máximo 10 caracteres, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```

CREATE TABLE tbl_ttext_seqset AS
/* Add perc NULL valores */
SELECT k, NULL AS ts
FROM generate_series(1, perc) AS k UNION
SELECT k, random_ttext_seqset('2001-01-01', '2001-12-31', 10, 10, 5, 10, 5, 10) AS ts
FROM generate_series(perc+1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples con the same timestamp */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts || text 'A' FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);

```

La creación de una tabla `tbl_tgeompoint_discseq` que contiene valores aleatorios `tgeompoint` 2D valores de subtipo secuencia con interpolación discreta que tienen entre 5 y 10 instantes, donde the x e y coordenadas están en el rango [0, 100] y en SRID 3812, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre coordenadas successive máximo 10 unidades en el SRID subyacente y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```

CREATE TABLE tbl_tgeompoint_discseq AS
SELECT k, random_tgeompoint_discseq(0, 100, 0, 100, '2001-01-01', '2001-12-31',
10, 10, 5, 10, 3812) AS ti
FROM generate_series(1, size) k;
/* Add perc duplicates */

```

```

UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT ti FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples con the same timestamp */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT round(ti,6) FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, endTimeStamp(ti)-startTimeStamp(ti))
FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute', (endTimeStamp(ti)-startTimeStamp(ti))/2))
FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);

```

Finalmente, la creación de una tabla `tbl_tgeompoint3D_seqset` que contiene valores aleatorios `tgeompoint 3D` valores de subtipo conjunto de secuencias que tienen entre 5 y 10 secuencias, cada una teniendo entre 5 y 10 marcas de tiempo, donde las coordenadas x, y, y z están en el rango [0, 100] y en SRID 3812, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre coordenadas sucesivas máximo 10 unidades en el SRID subyacente y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```

DROP TABLE IF EXISTS tbl_tgeompoint3D_seqset;
CREATE TABLE tbl_tgeompoint3D_seqset AS
SELECT k, random_tgeompoint3D_seqset(0, 100, 0, 100, 0, 100, '2001-01-01', '2001-12-31',
10, 10, 5, 10, 5, 10, 3812) AS ts
FROM generate_series(1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT ts FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples con the same timestamp */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT round(ts,3) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+ ←
perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);

```

B.6. Generador para tipos de red temporales

- `random_fraction`: Generar una fracción aleatoria en el rango [0,1]
- `random_npoint`: Genera un punto de red aleatorio
- `random_nsegment`: Genera un segmento de red aleatorio
- `random_tnpoint_inst`: Generar un punto de red temporal aleatorio de subtipo instant
- `random_tnpoint_discseq`: Generar un punto de red temporal aleatorio de subtipo secuencia e interpolación discreta

- `random_tnpoint_seq`: Generar un punto de red temporal aleatorio de subtipo secuencia e interpolación linear o escalonada
- `random_tnpoint_seqset`: Generar un punto de red temporal aleatorio de subtipo conjunto de secuencias

El archivo `/datagen/npoin/create_test_tables_tnpoint.sql` da ejemplos de utilización de las funciones que generan valores aleatorios listadas arriba.